UNISYS | Securing Your Tomorrow™

Application Development Solutions

COBOL Compiler
Programming Reference Manual

Volume 2 Compiler and System Interface

12R4

July 2018 7831 0455–021

NO WARRANTIES OF ANY NATURE ARE EXTENDED BY THIS DOCUMENT. Any product or related information described herein is only furnished pursuant and subject to the terms and conditions of a duly executed agreement to purchase or lease equipment or to license software. The only warranties made by Unisys, if any, with respect to the products described in this document are set forth in such agreement. Unisys cannot accept any financial or other responsibility that may be the result of your use of the information in this document or software material, including direct, special, or consequential damages.

You should be very careful to ensure that the use of this information and/or software material complies with the laws, rules, and regulations of the jurisdictions with respect to which it is used.

The information contained herein is subject to change without notice. Revisions may be issued to advise of such changes and/or additions.

Notice to U.S. Government End Users: This software and any accompanying documentation are commercial items which have been developed entirely at private expense. They are delivered and licensed as commercial computer software and commercial computer software documentation within the meaning of the applicable acquisition regulations. Use, reproduction, or disclosure by the Government is subject to the terms of Unisys' standard commercial license for the products, and where applicable, the restricted/limited rights provisions of the contract data rights clauses.

Unisys and other Unisys product and service names mentioned herein, as well as their respective logos, are trademarks or registered trademarks of Unisys Corporation. All other trademarks referenced herein are the property of their respective owners.

Contents

Section 1. Introduction

1.1. About This Manual 1–1
1.1.1. Purpose 1–1
1.1.2. Scope 1–1
1.1.3. Audience 1–2
1.1.4. Prerequisites 1–2
1.1.5. How to Use This Manual 1–2
1.1.6. Conventions 1–2
1.2. Documentation Updates 1–3
1.3. UCS COBOL Overview 1–3
1.4. The Universal Compiling System 1–4
1.5. Developing UCS COBOL Programs 1–4
1.5.1. Compiling a UCS COBOL Program 1–6
1.5.2. Executing and Linking a UCS COBOL Program 1–6
1.5.3. Debugging a UCS COBOL Program 1–7
1.6. Creating Shared Subsystems (Common Banks) 1–8
1.7. Organization of UCS COBOL Documentation 1–8
1.8. New Features and Enhancements 1–10

Section 2. Compiling Your Program

2.1. Calling the UCS COBOL Compiler (@UCOB) 2–1
2.2. Specifying Compiler Call Options 2–5
2.2.1. Commonly Used Sets of Options 2–5
2.2.2. Specifying Letter Options 2–8
2.2.3. Specifying Keyword Options 2–10
2.2.4. Compiler Call Keyword Options for Source Control 2–13
2.2.5. Compiler Call Keyword Options for Compilation Control 2–15
2.2.6. Compiler Call Keyword Options for Listing Control 2–18
2.2.7. Compiler Call Keyword Options for Program Control 2–23
2.2.8. Compiler Call Keyword Options for Diagnostic Message Control 2–50
2.3. Composition of a Compiler Listing 2–53
2.3.1. Sign-On Line 2–54
2.3.2. Options Listing 2–56
2.3.3. Diagnostic Messages 2–56
2.3.4. Source Listing 2–57
2.3.5. Tracecopy Listing 2–60

2.3.6. Allocation Listing 2–61
2.3.7. Cross-Reference Listing 2–65
2.3.8. Object Code Listing 2–68
2.3.9. Linker Information Listing 2–71
2.3.10. Bank Sizes Line 2–75
2.3.11. Sign-Off Line 2–76
2.4. Separate Compilations for a Single Source Element (Back-to-Back COBOL Programs) 2–77
2.4.1. Example of Separate Compilations 2–78
2.4.2. Execution of the Object Modules Produced 2–80
2.5. Using the Run-Condition Word to Detect Compilation Errors 2–81

Section 3. Linking, Executing, and Debugging Your Program

3.1. Linking Your Program 3–1
3.1.1. Dynamic Linking 3–3
3.1.2. Static Linking 3–4
3.1.3. Creating Fixed-Gate Subsystems 3–5
3.2. Executing Your Program 3–6
3.3. Debugging with PADS 3–7
3.3.1. PADS Features 3–8
3.3.2. Compiler Option Considerations 3–10
3.3.3. Excluding PADS 3–12
3.3.4. Activating PADS 3–12

Section 4. Changing I/O System Limits and I/O Optimization

4.1. System I/O Overview 4–1
4.2. Regenerating the URTS$TABLES Object Module 4–3
4.3. Using the Alternate URTS$TABLES Object Module 4–6
4.4. UCS Runtime I/O 4–8

Section 5. Using System Interfaces

5.1. Preparing Your UCS COBOL Library 5–1
5.2. Assigning a UCS COBOL File 5–2
5.2.1. Assigning Nonsymbiont Files 5–2
5.2.2. Assigning Symbiont Files 5–3
5.2.3. Assigning Alternate Symbiont Files 5–4
5.3. Using the Define File Processor (DFP) 5–4
5.3.1. Blocking Factors for Non-MSAM Files 5–5
5.3.2. Blocking Factors for MSAM Files 5–5
5.3.3. Record Formats 5–7
5.3.4. Prep Factors 5–8
5.3.5. Label Records 5–8
5.3.6. EBCDIC Translate 5–9
5.3.7. OPEN Parameter 5–9
5.3.8. CLOSE Parameter 5–9
5.3.9. SHARE Parameter 5–10

5.3.10. Concatenation of Data Files 5–10
5.4. Accessing the Processor Common Input/Output System (PCIOS) 5–12
5.5. Accessing the Shared File System (SFS) 5–12
5.6. Retrieving Shared File System (SFS) Status Field 3 (SFSSTATUS$) 5–13
5.7. Passing Procedure Name Arguments to Non-COBOL Programs 5–14
5.8. Changing the Sort-Merge Main Storage Size 5–16
5.9. System Constraints for DENSITY Clause 5–17
5.10. Interpreting Operator Console Messages 5–17
5.10.1. ACCEPT . . . FROM CONSOLE 5–17
5.10.2. Blank Tape 5–17
5.10.3. DISPLAY . . . UPON CONSOLE 5–17
5.10.4. Unlabeled Tape 5–18
5.10.5. VALUE OF CARTRIDGE-ID 5–18
5.10.6. VALUE OF FORMS-REQUEST 5–18

Section 6. Accessing a DMS Database

6.1. The UCS COBOL Interface 6–1
6.2. Sample Program 6–4
6.3. Database Key Usage 6–5

Section 7. Accessing a Relational Database

7.1. Using SQL Commands in a UCS COBOL Program 7–3
7.2. Declaring Variables for SQL Commands 7–5
7.2.1. Embedded SQL Data Variables 7–5
7.2.2. Embedded SQL Error Variables 7–6
7.2.3. Interpreted SQL Variables 7–8
7.2.4. Binary Large Objects – BLOBs 7–9
7.3. Data Types for Passing Values between RDMS and UCS COBOL 7–24
7.3.1. Equivalent Data Types 7–24
7.3.2. Passing Values to RDMS from Program Variables 7–25
7.3.3. Passing Values from RDMS to Program Variables 7–27
7.4. Coding Embedded SQL Commands 7–28
7.4.1. Marking Embedded SQL Commands for the Compiler 7–28
7.4.2. Example of Using Embedded SQL Commands 7–29
7.5. Coding Interpreted SQL 7–33
7.5.1. Calling the RDMS Interpreter Interface (ENTER MASM) 7–33
7.5.2. Example of Using the Interpreter Interface 7–34
7.6. Allocating and Initializing the RDMS Work Area (RSA$INIT) 7–38
7.6.1. Determining the Work Area Size 7–38
7.6.2. Calling RSA$INIT 7–39

7.7. Compiling Programs That Access RDMS 7–39
7.8. Linking Programs That Access RDMS 7–40
7.8.1. Statically Linking Applications 7–41
7.8.2. Dynamically Linking Applications 7–42
7.9. Compile Time Explain Reports 7–43

Section 8. Accessing the Unisys Repository Manager (UREP)

8.1. Specifying an Application Group 8–2
8.2. Generating Code for Your Source Program (INVOKE) 8–3
8.2.1. Inserting SELECT Statement Information with INVOKE 8–3
8.2.2. Inserting File Descriptions with INVOKE 8–5
8.2.3. Inserting Data Descriptions with INVOKE 8–7
8.2.4. Inserting Source Code with INVOKE 8–29
8.3. Updating UREP (UREP-XREF) 8–30

Section 9. Accessing Program-Callable FURPUR (PCFP)

Section 10. Using Executive Requests

10.1. Dynamic Control Statement Requests (ERACSF, FACSF, FACSF2, IACSF, IACSF2) 10–4
10.2. Setting and Retrieving the Condition Word (FSETC, FCOND) 10–6
10.3. Retrieving Date and Time in ASCII (ADATE) 10–8
10.4. Retrieving Time and Date (UCSTDATE$) 10–9
10.5. Activity Termination (FEXIT, FERR, FABORT, FEABT) 10–10
10.6. Input/Output ERs (FIO, FIOW, FSYMB, FUNLCK, FWANY, FWST) 10–11
10.6.1. Building a Symbiont I/O Packet (UCSSYMBPK$) 10–13
10.6.2. Building an I/O Packet (UCSMAKEIOPK$) 10–15
10.6.3. Retrieving a Buffer Address (LOC$) 10–21
10.6.4. I/O Executive Request Examples 10–22
10.7. Program File Maintenance Executive Requests 10–24
10.7.1. Building a Skeleton Packet for Program File Maintenance ERs (UCSMAKEPFPK$) 10–24
10.7.2. Deleting, Inserting, or Searching for an Element (UCSPFD$, UCSPFI$, UCSPFS$) 10–25
10.7.3. Updating and Retrieving Write Limits (UCSPFUWL$, UCSPFWL$) 10–26
10.8. Retrieving @XQT or Processor Call Statement Options (UCSOPT$) 10–28
10.9. Retrieving Program Control Table (UCSPCT$) 10–28
10.10. Reading and Writing Tape Label Blocks (UCSTLBL$) 10–29
10.11. Tape File Initialization (UCSTINTL$) 10–30
10.12. Tape Swapping (UCSTSWAP$) 10–31
10.13. Control Statement Listing (UCSCLIST$) 10–32
10.14. Retrieving Facility Assignment (UCSFITEM$) 10–33
10.15. Miscellaneous Information Retrieval (UCSINFO$) 10–34

10.16. General Executive Requests (UCSGENERAL$) 10–43
10.17. Milliseconds Past Midnight (UCSTIME$) 10–49
10.18. Retrieving Time (UCSDWTIME$) 10–50
10.19. Converting Time in UCSDWTIME$ Format (UCSCNVTIM$) 10–51
10.20. Converting Time in UCSDWTIME$ Format to a Date (UCSCNVTIM$CC) 10–52
10.21. Initial Register Contents (UCSINITREG$) 10–53
10.22. Converting between Fieldata and ASCII Characters (UCSASCFD$, UCSFDASC$) 10–56
10.23. Converting Extended Mode Addresses to Basic Mode Addresses (UCSEMTOBM$) 10–58
10.24. Activating Auxiliary Prologue and Epilogue (UCS$ACTAUX and UCS$DEACTAUX) 10–61
10.25. Subprograms for Performing System Level I/O 10–62
10.25.1. BULKOPEN$ 10–63
10.25.2. BULKCLOSE$ Subprogram 10–68
10.25.3. BULKREAD$ Subprogram 10–69
10.25.4. BULKWRITE$ Subprogram 10–72
10.25.5. BULKGETFILESIZE$ Subprogram 10–73
10.25.6. BULKGETSECTOR$ Subprogram 10–74
10.25.7. BULKSETSECTOR$ Subprogram 10–75
10.25.8. Example of Bulk I/O 10–76
10.26. Subprograms for Dynamic Memory Allocation 10–78
10.26.1. UCOBMALLOC$ 10–81
10.26.2. UCOBCALLOC$ 10–81
10.26.3. UCOBFREE$ 10–82
10.26.4. UCOBREALLOC$ 10–82
10.26.5. UCOBMALLOC$2 10–83
10.26.6. UCOBCALLOC$2 10–84
10.26.7. UCOBFREE$2 10–84
10.26.8. UCOBREALLOC$2 10–85
10.27. Interface to Zip Utilities 10–86
10.27.1. ZIP$NBRENTRIES 10–86
10.27.2. ZIP$NAMEENTRIES 10–87
10.27.3. ZIP$READENTRY 10–89
10.27.4. ZIP$WRITEENTRY 10–90
10.27.5. ZIP$CLOSEENTRY 10–92
10.27.6. ZIP$PRINTEXCEPTION 10–92
10.27.7. ZIP$SETCOMPRESSION 10–93
10.27.8. Example Program 10–94
10.28. Retrieving Coordinated Universal Time (UTC) 10–97
10.28.1. UTC$TIME 10–98
10.28.2. TIMEB2DWTIM$ 10–99
10.28.3. Example Program 10–99

Section 11. Transaction Processing in UCS

11.1. Using DPS 2200 Functions to Code Transactions 11–3
11.2. Using MCB Functions to Code Transactions 11–4

11.3. Using Primitives That Do Not Handle Messages to Code Transactions 11–5
11.4. Retrieving the TIP RESTART Status (UCOB_RESTARTCHECK and UCS_RESTARTCHECK) 11–6
11.5. Considerations 11–8

Section 12. Contingency Handling

12.1. User Contingency Handler Registration 12–2
12.1.1. Contingency Registration Packet Interface 12–2
12.1.2. Contingency Registration Version or Function Interface 12–5
12.1.3. Contingency Packet Processing 12–7
12.2. User Contingency Handler Deregistration 12–11
12.3. User Termination Handler Registration 12–12
12.4. User Termination Handler Deregistration 12–15
12.5. UCS Runtime System Contingency Handling 12–16
12.5.1. Home Subsystem Operation 12–16
12.5.2. Shared Subsystem Restrictions 12–16

Section 13. Processing Tapes for Exchanging Information

13.1. Environment Division—Input-Output Section 13–2
13.1.1. The FILE-CONTROL Paragraph 13–2
13.1.2. The File Control Entry 13–2
13.2. Data Division—File Section 13–4
13.2.1. The RECORDING MODE Clause 13–4
13.2.2. Record Description Restrictions 13–4

Section 14. Processing IBM Tapes

14.1. Function 14–1
14.2. Language Concepts 14–1
14.2.1. Organization 14–1
14.2.2. Access Mode 14–2
14.2.3. File Position Indicator 14–2
14.2.4. The At-End Condition 14–2
14.2.5. Allocation Listing 14–2
14.3. Environment Division—Input-Output Section 14–2
14.3.1. The FILE-CONTROL Paragraph 14–2
14.3.2. The File Control Entry 14–3
14.3.3. The I-O-CONTROL Paragraph 14–4
14.4. Data Division—File Section 14–6
14.4.1. The File Description—Complete Entry Skeleton 14–6
14.4.2. The BLOCK CONTAINS Clause 14–7
14.4.3. The DATA RECORDS Clause 14–7
14.4.4. The LABEL RECORDS Clause 14–8
14.4.5. The RECORD Clause 14–8
14.4.6. The RECORDING MODE Clause 14–11

14.4.7. The VALUE OF Clause....14–12
14.4.8. The Record Description—Complete Entry Skeleton 14–13
14.4.9. Record Selector Fields 14–19
14.5. Procedure Division 14–20
14.5.1. The CLOSE Statement 14–20
14.5.2. The OPEN Statement 14–23
14.5.3. The READ Statement 14–25
14.5.4. The WRITE Statement 14–26

Section 15. Interlanguage Calls

15.1. UCS COBOL Calling UCS FORTRAN 15–3
15.2. UCS COBOL Calling UCS C 15–7

Section 16. Internationalization

16.1. I18NLIB Components 16–1
16.2. Internationalization of Programs 16–2

Section 17. CIFS COBOL Program Interface

17.1. CIFS Copy Procedures 17–1
17.2. COBOL APIs 17–2
17.3. Example Program 17–5

Section 18. XML Generator

18.1. Creating an XML File 18–1
18.1.1. Create Record Name File 18–1
18.1.2. Pre-compilation Setup 18–2
18.1.3. Compile the COBOL Program 18–3
18.2. Example Output 18–3
18.3. XML File names 18–6
18.4. Back-to-Back COBOL Programs 18–6
18.5. Nested COBOL Programs 18–6

Appendix A. Compilation-Time Diagnostic Messages

A.1. Diagnostic Message Types and Format A–1
A.2. COBOL FIPS Flagging A–4
A.3. UCS COBOL Front-End Compiler Messages (UCOB) A–7
A.4. UCS Language Support System Messages (LSS) A–91

Appendix B. Execution Time Diagnostic Messages

B.1. UCS Runtime System Messages (URTS) B–2
B.2. UCS Runtime System Messages for 19002 and 35001 .... B–57
B.3. HVTIP2/FLSS Error Messages B–64

B.4. URTS AFCB-Detected Mixedmode Errors B–67
B.5. Mixedmodeskl Error List B–69
B.6. Sort/Merge Messages B–74

Appendix C. Compiler Limits

C.1. Compiler Limits C–1
C.2. Additional Limits C–5
C.2.1. Size of an Array or Common Block C–5
C.2.2. Program Addresses C–5
C.2.3. Expression Evaluation C–6
C.2.4. Nested Procedure Calls C–6
C.2.5. Number of External Names in an Object Module C–8

Appendix D. Character Code

Appendix E. Label Processing

E.1. The Label Records Clause E–1
E.2. Label Formats E–1
E.2.1. TAPE and INTERCHANGE Files E–1
E.2.2. INDEXED DISC Files E–5
E.2.3. RELATIVE and SEQUENTIAL DISC Files E–5
E.3. The VALUE OF Clause E–6
E.3.1. FILE-ACCESS E–6
E.3.2. FILE-ID E–6
E.3.3. PURGE-DATE E–6
E.4. Procedure Division I-O Statements E–7
E.4.1. The CLOSE Statement E–7
E.4.2. The OPEN Statement E–9
E.4.3. The READ Statement E–10
E.4.4. The WRITE Statement E–10

Appendix F. UCS COBOL and ASCII COBOL Differences

F.1. Conversion Tools F–2
F.2. Implementation Differences F–4
F.2.1. COMPUTATIONAL Fields Data Representation F–4
F.2.2. COMPUTATIONAL-3 Deleted F–4
F.2.3. SYNCHRONIZED Field Alignment F–4
F.2.4. INDEX, COMP-1, COMP-2 Alignment F–5
F.2.5. Data Allocation in IBM Records F–5
F.2.6. IBM Data Allocation for Programs F–5
F.2.7. Intermediate Operands F–5
F.2.8. Floating-Point Computations F–6
F.2.9. HIGH-VALUES F–6
F.2.10. Undefined Results F–6
F.2.11. Calling Sequence F–7
F.2.12. Register Conventions F–7

F.2.13. ASCII COBOL Level 3R1 Calling Sequence F–7
F.2.14. Segmentation Map Symbolic F–8
F.2.15. CALL Identifier F–8
F.2.16. Extra Options Deleted F–8
F.2.17. Referencing Record Areas before OPEN F–9
F.2.18. Index-Name Formats F–9
F.2.19. INDEX Data Items F–9
F.2.20. SEARCH ALL F–9
F.2.21. Overpunched Signed Numeric Fields F–10
F.2.22. Class Names F–10
F.2.23. Subscript Checking F–11
F.2.24. Numeric PROGRAM-ID Name F–11
F.2.25. Paragraph Followed by a GO TO Statement F–11
F.2.26. FILE STATUS with APPLY EXDEF and APPLY EXREF Clause F–12
F.2.27. Duplicate Data Names from Schema F–12
F.2.28. Report Format for Nonprint Files F–12
F.2.29. Communications Module Not Implemented F–13
F.2.30. Data Allocation F–13
F.2.31. Record Key/Alternate Key Alignment F–13
F.2.32. IBM Labeled Tapes F–14
F.2.33. Blocking Size F–15
F.3. Discontinued Extensions F–15
F.3.1. IOS Files F–15
F.3.2. Compatible Files F–15
F.3.3. 770-PRINTER Deleted F–16
F.3.4. ALPHABET SECTION Deleted F–16
F.3.5. Asynchronous Processing Module Deleted F–16
F.3.6. APPLY INLINE Deleted F–16
F.3.7. POINT LOCATION Clause Deleted F–16
F.3.8. REMARKS Deleted F–17
F.3.9. MONITOR Deleted F–17
F.3.10. Special Registers F–17
F.3.11. ACCEPT and DISPLAY F–18
F.3.12. EXAMINE, EXHIBIT, TRANSFORM Deleted F–18
F.3.13. TRAILING Option of INSPECT Deleted F–18
F.3.14. NOTE Statement Deleted F–18
F.3.15. ON Integer Deleted F–19
F.3.16. COBOL 68 Format COPY Statements F–19
F.3.17. Old ENTER Statements Deleted F–19
F.3.18. Mnemonic Name in CODE Clause F–19
F.3.19. COBOL 68 CODE Clause Deleted F–19
F.3.20. EQUALS, EXCEEDS, UNEQUAL F–20
F.3.21. Fieldata F–20
F.4. Substantive Changes from the American National Standard (ANS) F–20
F.4.1. Length of ALL Literal F–20
F.4.2. Alphabet-Name Clause F–21
F.4.3. Collating Sequence F–21
F.4.4. CURRENCY SIGN Clause F–21
F.4.5. RELATIVE KEY Phrase F–22
F.4.6. MULTIPLE FILE TAPE Clause F–22

F.4.7. LINAGE Clause F–22
F.4.8. FOOTING Phrase F–23
F.4.9. OCCURS Clause F–23
F.4.10. PICTURE Symbol P F–23
F.4.11. PROCEDURE DIVISION Header F–24
F.4.12. Exponentiation F–24
F.4.13. Order of Execution for a Conditional Expression F–24
F.4.14. Class Condition F–25
F.4.15. CANCEL Statement F–25
F.4.16. CLOSE Statement F–25
F.4.17. COPY Statement in Comments F–26
F.4.18. COPY Statement on a Debugging Line F–26
F.4.19. DISPLAY Statement F–26
F.4.20. DIVIDE Statement F–26
F.4.21. EXIT PROGRAM Statement Used Implicitly F–27
F.4.22. EXIT PROGRAM Statement Used with the PERFORM Statement F–27
F.4.23. INSPECT Statement F–27
F.4.24. MERGE Statement F–27
F.4.25. PERFORM Statement — Initialization of Variables F–28
F.4.26. PERFORM Statement — Setting and Augmenting Variables F–28
F.4.27. PERFORM Statement — Evaluation of Subscripts F–28
F.4.28. READ Statement F–29
F.4.29. RETURN Statement F–29
F.4.30. STOP RUN Statement/Program Termination — File Closure F–29
F.4.31. STOP RUN Statement — Message Queue Management F–30
F.4.32. STRING Statement F–30
F.4.33. UNSTRING Statement F–30
F.4.34. WRITE Statement F–30
F.4.35. File Position Indicator — Record Pointer Replacement F–31
F.4.36. File Position Indicator — Effect on READ/WRITE Statements F–31
F.4.37. File Position Indicator — Effect of Rewrite Statement F–31
F.4.38. Reserved Words F–32
F.4.39. I-O Statuses F–34
F.5. Obsolete Language Elements F–34
F.5.1. Character Substitution F–34
F.5.2. ALL Literal and Numeric or Numeric Edited Items F–34
F.5.3. Identification Division Paragraphs F–35
F.5.4. MEMORY SIZE Clause F–35
F.5.5. RERUN Clause F–36
F.5.6. MULTIPLE FILE TAPE Clause F–36
F.5.7. LABEL RECORDS Clause F–36
F.5.8. VALUE OF Clause F–37

F.5.9. DATA RECORDS Clause F–37
F.5.10. ALTER Statement F–37
F.5.11. The Optional Use of a Procedure Name in a GO TO Statement F–38
F.5.12. REVERSED Phrase of the OPEN Statement F–38
F.5.13. STOP Literal Statement F–38
F.5.14. Segmentation Module F–39
F.5.15. Debug Module F–39

Appendix G. Compatibility, Restrictions, and Operational Considerations

G.1. Compatibility G–1
G.1.1. Language Support System (LSS) Level G–1
G.1.2. Compatibility with ASCII COBOL G–2
G.1.3. Compatibility with Previous Versions of UCOB G–2
G.2. Operational Considerations G–2

Index 1

Figures

1–1. Extended Mode Software Products 1–5
1–2. Developing Programs in Extended Mode 1–5

3–1. Program Development under the Linking System 3–3

6–1. Calling DMS from UCS COBOL 6–3

7–1. Accessing a Relational Database from UCS COBOL 7–2

10–1. Using Executive Requests from UCS COBOL 10–1

Tables

2–1. Typical Compilations and the Options to Perform Them 2–5
2–2. Compiler Call Letter Options 2–9
2–3. List of Keyword Options 2–10
2–4. Source Control Compiler Keyword Options 2–13
2–5. Compilation Control Compiler Keyword Options 2–15
2–6. Summary of Listing Control Compiler Keyword Options 2–18
2–7. Program Control Compiler Keyword Options 2–23
2–8. Diagnostic Message Control Compiler Keyword Options 2–50
2–9. Composition of a Compiler Listing 2–53

5–1. Rollback/Deadlock Error Processing 5–13

7–1. Equivalent Data Types for UCS COBOL and RDMS 7–24

10–1. Executive Requests 10–2
10–2. Registers Retrieved 10–54
10–3. Permitted Values for the Type Argument on the BULKOPEN$ Subprogram 10–64
10–4. Permitted Values for the type-name and type-code on the BULKOPEN$ Subprogram 10–66
10–5. Status Values Set by the BULKOPEN$ Subprogram 10–67
10–6. Status Values Set by the BULKCLOSE$ Subprogram 10–69
10–7. Status Values Set by the BULKREAD$ Subprogram 10–71
10–8. Status Values Set by the BULKWRITE$ Subprogram 10–73
10–9. Status Values Set by the BULKGETFILESIZE$ Subprogram 10–74
10–10. Status Values Set by the BULKGETSECTOR Subprogram 10–75
10–11. Status Values Set by the BULKSETSECTOR$ Subprogram 10–76
10–12. Status Values Returned in return-status 10–85

11–1. Transaction Processing Interfaces from UCS Languages 11–1

14–1. CLOSE Statement File Categories and Formats 14–21
14–2. Permissible Statements 14–24

15–1. COBOL vs. FORTRAN Data Types 15–3
15–2. COBOL vs. C Data Types 15–7

A–1. Diagnostic Message Classes and Subclasses A–2
A–2. Modules, Levels, and Subsets A–5

C–1. Compiler Limits of UCS COBOL C–1

D–1. ASCII Character Codes D–1

E–1. Volume Header Label (VOL1) Format E–2
E–2. First File Header Label (HDR1) Format E–2
E–3. Second File Header Label (HDR2) Format E–4

Examples

2–1. Options Listing (NARROW) 2–56
2–2. Source Listing (WIDE) 2–59
2–3. Source Listing (NARROW) 2–60
2–4. Tracecopy Listing 2–60
2–5. Allocation Listing (NARROW) 2–64
2–6. Cross-Reference Listing (NARROW) 2–68
2–7. Object Code Listing (NARROW) 2–71
2–8. Linker Information Listing 2–74
2–9. Separate Compilations 2–80

6–1. Sample UCS COBOL Program Accessing DMS 6–4

7–1. UCS COBOL Program Using Embedded SQL 7–33
7–2. UCS COBOL Program Using the Interpreter Interface 7–38

8–1. Invoking a SELECT Statement 8–4
8–2. Invoking a File Description. 8–6
8–3. Invoking Data Descriptions 8–8
8–4. Invoking Table and View Descriptions 8–9
8–5. Inserting Screen Definitions with the INVOKE Statement 8–17
8–6. Inserting Screen Definitions with Specific Initial Values 8–18
8–6. Inserting Screen Definitions with Specific Initial Values (contd.) 8–18
8–7. Inserting Screen Definitions with Repeating Image 8–24
8–7. Inserting Screen Definitions with Repeating Image (contd.) 8–25
8–8. Invoking a SOURCE Statement 8–30

15–1. UCS COBOL Main Program to Call UCS FORTRAN Subprogram 15–6
15–2. UCS FORTRAN Subprogram Called from UCS COBOL Main Program 15–6
15–3. Output of UCS COBOL Program Calling UCS FORTRAN Subprogram 15–7
15–4. UCS COBOL Main Program to Call UCS C Subprogram 15–10
15–5. UCS C Subprogram Called from UCS COBOL Main Program 15–11
15–6. Output of UCS COBOL Program Calling UCS C Subprogram 15–11

Section 1
Introduction

1.1. About This Manual

1.1.1. Purpose

This manual provides detailed information about how to convert your COBOL source program into an executable object module. This manual is the companion to the *OS 2200 Universal Compiling System (UCS) COBOL Programming Reference Manual Volume 1, COBOL Statements*. Use Volume 1 as a reference source for topics relating to the language. Use Volume 2 for procedural information about using the features and capabilities of UCS COBOL. Both volumes give examples to emphasize and clarify the accompanying text instructions.

This manual describes how to compile, link, execute, and debug your UCS COBOL source program. In addition, it shows the necessary steps required to make your program interact with other system software, such as relational databases and Executive requests.

1.1.2. Scope

This manual presents Unisys specific information to help you convert your UCS COBOL program source code into a fully executable program. This volume does not provide information about writing your initial UCS COBOL source program. The companion volume, Volume 1, presents this information.

Each section in this manual discusses a particular task or group of related tasks. One section is devoted entirely to providing the steps required to compile your UCS COBOL source program. This section also discusses compiler listings and provides examples of various listing capabilities.

Later sections give an overview of how to link, execute, and debug your compiled program using the Programmers Advanced Debugging System (PADS). Other sections provide enough information so that your program can interact with other system software, for example:

- Relational databases
- Executive requests
- DMS 2200 databases

Appendixes A and B contain numerical listings of compilation-time and execution-time diagnostic messages. Where necessary, brief explanations provide supplementary information.

1.1.3. Audience

This manual is intended for systems and applications programmers and analysts who are already familiar with the UCS COBOL programming language and need specific information about how to use UCS COBOL.

1.1.4. Prerequisites

To use this manual effectively, you should have knowledge of the UCS COBOL programming language as well as familiarity with basic OS 2200 job control language concepts.

1.1.5. How to Use This Manual

Use this manual as a tutorial document by following the directions provided in the applicable sections. The tasks described are organized in a logical sequence for easy referencing. Many sections present the information using a step-by-step format that is easy to use as a learning tool. You can read the manual through to become familiar with the processes described or you can refer to selected sections or pages to learn about one topic in detail.

The following points apply throughout this manual:

- The term “COBOL” refers to the COBOL language generally or to standard COBOL. “UCS COBOL” refers to this implementation of COBOL.
- Text with yellow color highlighting indicates where technical information has been added or modified for this release of the programming reference manual. This material is different from the previous level of UCS COBOL.

1.1.6. Conventions

This manual uses the following conventions to illustrate COBOL syntax:

- UPPERCASE represents COBOL keywords that are required in the syntax. Keywords must be spelled exactly as shown.
- [UPPERCASE in brackets] represents keywords that you can optionally use to make your program more readable. Keywords must be spelled exactly as shown.

Change bars in the margins indicate where technical information has been added or modified for this release of the reference manual.

1.2. Documentation Updates

This document contains all the information that was available at the time of publication. Changes identified after release of this document are included in problem list entry (PLE) 19206173. To obtain a copy of the PLE, contact your Unisys representative or access the current PLE from the Unisys Product Support website:

http://www.support.unisys.com/all/ple/19206173

Note: *If you are not logged into the Product Support site, you will be asked to do so.*

1.3. UCS COBOL Overview

Common Business Oriented Language (COBOL) is a high-level programming language that is procedure-oriented. The language is designed for organizational data processing applications that typically use large files and extensive input and output to produce large formatted reports. The language commands are derived from English grammar, consisting of language components such as

- Clauses
- Sentences
- Paragraphs
- Word order

Universal Compiling System (UCS) COBOL is the implementation of COBOL for your system. UCS COBOL includes all of the required features of the following standards:

- American National Standard COBOL, X3.23-1985
- Federal Information Processing Standards Publication 21-2 (FIPS PUB 21-2)

The UCS COBOL implementation contains some extensions that are additions or enhancements to American National Standard COBOL. In addition, this implementation includes compatibility features for migrating ASCII COBOL programs to UCS COBOL.

The UCS COBOL compiler uses the American Standard Code for Information Interchange (ASCII) and International Standards Organization (ISO) character sets, and, optionally, the kanji character set, in processing source text and execution-time character data. The UCS COBOL compiler is fully reentrant and produces optimized reentrant code usable in multibanked applications, shared subsystems, or transaction programs.

1.4. The Universal Compiling System

UCS COBOL is a component of the Universal Compiling System (UCS). UCS consists of

- A set of independent front-end language processors (for languages such as C and COBOL)
- The Language Support System (LSS), which is a code generator used in common by all front-end language processors
- The UCS Runtime System, which sets up the run-time environment for a program

For more information about UCS, refer to the *UCS Conceptual Overview*.

1.5. Developing UCS COBOL Programs

UCS COBOL belongs to a suite of software products specifically designed to execute on the extended mode architecture that is featured on Unisys systems. This hardware and software combination enhances your program development environment with the following benefits:

- Latest programming language standards
- Easier methods of constructing, executing, debugging and maintaining programs
- Better control over sharing data between different parts of the system
- Better protection of programs
- Standard calling sequence across programming languages and the operating system
- Dynamic replacement of processors
- Automatic support of multiple user data banks
- Use of a single command interface for compiling and interactive debugging
- Dramatically enlarged data access through extended mode hardware addressing

Such software includes the following:

- Universal Compiling System
- Linking System
- Service Library
- Programmers Advanced Debugging System (PADS)
- Software subsystems (shared banks)
- Executive system support

Figure 1–1 illustrates the extended mode software products hierarchy.

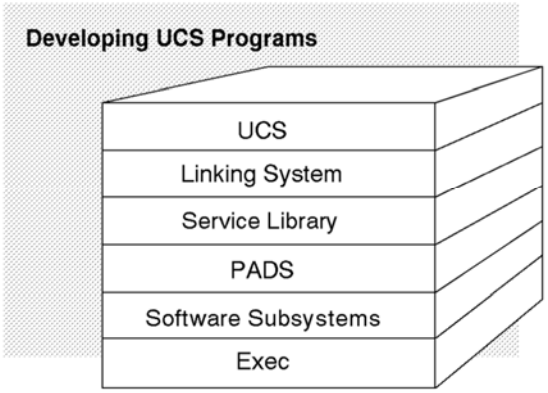


Figure 1–1. Extended Mode Software Products

You use this suite of products to compile, link, execute, and debug programs. Figure 1–2 illustrates the different software components involved in compiling, linking, executing, and debugging a program.

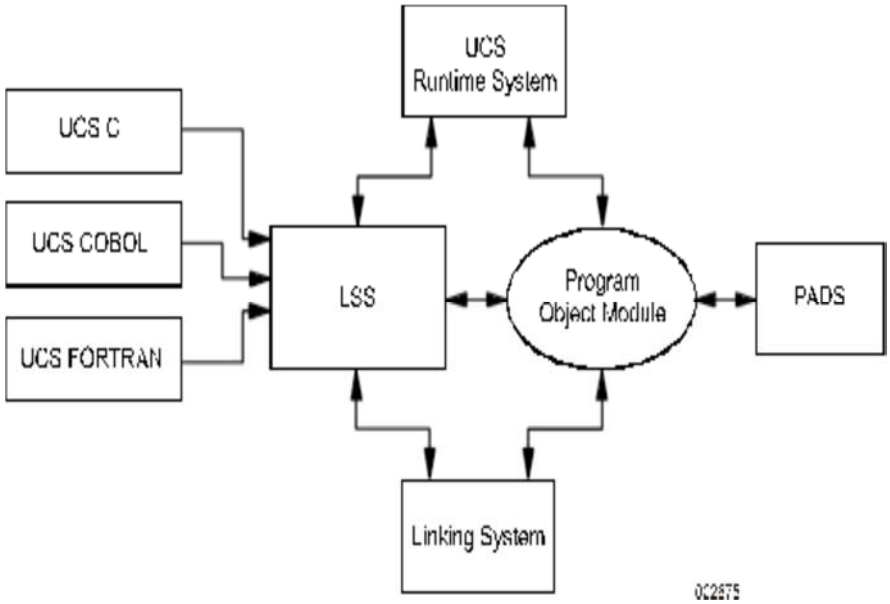


Figure 1–2. Developing Programs in Extended Mode

The following subsections give an overview of how to develop UCS COBOL programs.

1.5.1. Compiling a UCS COBOL Program

Program compilation occurs in the following steps:

1. The user calls the UCS COBOL compiler with the compiler call (@UCOB).
2. The front-end processor takes the text of the source program and translates it into an intermediate form that is independent of the language being used.
3. LSS translates the intermediate, language-independent information into machine-dependent code, producing an object module element that is ready to be executed on your extended mode system.

See Section 2 for specific information about compiling your program.

1.5.2. Executing and Linking a UCS COBOL Program

Immediately after compilation, you can execute a UCS COBOL object module element with an @XQT statement. For example,

```
@XQT qual*file.om
```

The execution statement causes the Linking System to load the object module into main storage. If required, the Linking System links the object module to any other object modules that are referenced (called) by the executed object module. Such references can be to any of the following:

- External routines
- External data items
- Global constants
- Common blocks

Once loaded into main storage, the object module is executed by the Exec.

You can choose when linking takes place. The Linking System can dynamically link a called object module to its caller after the caller has been loaded and execution has begun. Alternatively, you can link object modules before loading into main storage; this causes efficient execution because linking does not interrupt execution.

The UCS Runtime System aids program execution by setting up the environment in which your program executes. The UCS Runtime System also provides interfaces to other UCS programs needed at run time. The UCS Runtime System includes both

- Routines that are common to all of the UCS languages
- Routines that are unique to one language

UCS Runtime System routines include the following:

- Environment control routines that
  - Manage main storage
  - Handle interrupts, contingencies, and interlanguage calls
- Data manipulation routines that handle
  - Intrinsic functions
  - Conversions
  - Comparisons too complicated to do with generated code
- Debugging routines that interface with the PADS
- System interface modules that handle machine-dependent features
- Processor interface modules for
  - Communications
  - Data Management System (DMS 2200)
  - Relational Data Management System (RDMS 2200)
  - Sort/Merge
  - Transaction programming environments (TIP and HVTIP)
- Interfaces for the Processor Common Input/Output System (PCIOS), allowing all UCS languages to read or write from the same physical file
- Run-time message routines that produce diagnostic messages when run-time errors occur (see Appendix B)

See Section 3 for specific information about linking and executing your program.

1.5.3. Debugging a UCS COBOL Program

A symbolic debugging tool called the Programmers Advanced Debugging System (PADS) is available to help you debug your UCS COBOL programs. PADS is a high-level interactive debugging tool that is common to all of the UCS languages. You can turn PADS on or off at the start of program execution without having to recompile the program each time. It has a simple interactive command language that lets you stop programs at desired places, examine or modify the values of program variables, and resume program execution. PADS is automatically called if your program fails; there is no overhead due to PADS if your program does not fail.

See Section 3 for specific information about debugging your program with PADS.

1.6. Creating Shared Subsystems (Common Banks)

Extended mode shared subsystems are the functional equivalent of basic mode common banks. A shared subsystem is a group of one or more object modules that multiple programs can access at the same time. Each program accessing a shared bank accesses the same copy of the bank.

Shared banks help limit the amount of main storage required by a system because multiple users can all use the same main storage space represented by the bank. A shared subsystem is partitioned off from the rest of the system by a combination of hardware and software mechanisms so that access to the shared subsystem can be controlled.

Shared subsystems begin as ordinary object modules (compiled code). For information about how to create shared subsystems from ordinary object modules, refer to the *Linking System Subsystems Programming Guide*.

1.7. Organization of UCS COBOL Documentation

This manual is Volume 2 of a two-volume set of reference manuals. It contains information about the following:

- Compiling, linking, executing, and debugging a program
- Interfacing with other system software

Volume 1 is titled *OS 2200 Universal Compiling Systems (UCS) COBOL Programming Reference Manual Volume 1, COBOL Statements* (7831 0448). It contains information about COBOL language statements

The overall structure of UCS documentation is designed in a logical order of introductory and technical information. This logical order is as follows:

1. The *UCS Conceptual Overview* (7831 0802) introduces the reader to UCS. It describes UCS software components and discusses how they are used to develop user applications in a high-level programming language. It is intended for data processing managers and potential UCS users.
2. The *UCS Evaluation Overview* (7833 0792) provides a more detailed description of UCS software. It describes specific programming statements and commands available and gives examples of their use. It is intended for managers and technical specialists who are evaluating whether to acquire or increase use of UCS.
3. The *Application Development Programming Guide* (7831 4077) is intended for programmers. It also introduces the reader to UCS. It is intended to give enough detailed programming instructions and guidelines concerning UCS so that a programmer can in many cases use it to create, compile, link, and execute programs, without necessarily having to reference other UCS documentation. Programmers should read this document first.

4. A series of documents describes the separate tasks of language programming, linking, shared (common) bank creation, and debugging in detail:

   - Language programming

     The language documentation is described in two volumes. Volume 1, which describes source language statements, is logically read first. Volume 2, which describes how to interface with other system software, is logically read next. (Normally, users will use these documents for reference only and will not read them in their entirety).

   - Linking

     The *Linking System Programming Reference Manual* (7831 0505) describes this topic.

   - Shared bank creation

     The *Linking System Subsystems Programming Guide* (7830 7451) describes this topic.

   - Debugging

     The *Programmers Advanced Debugging System (PADS) Programming Guide* (7831 0562) describes this topic.

1.8. New Features and Enhancements

The following table lists the new features and enhancements for recent levels of UCS COBOL.

UCOB Level	Feature Name	Description	Reference
12R3	XML	This feature gives the compiler the ability to create an XML document that describes the record layout of a specified 01 record description.	Volume 2 Section 18
12R2	COMP-4	Allows elementary items to be described as either COMPUTATIONAL-4 or COMP-4. Internally are treated as USAGE BINARY.	Volume 1, subsection 3.5.16
12R2	COMP-3	When the COMP-BIN or COMPAT keyword options are used, data-items can be declared as USAGE COMPUTATIONAL-3 or USAGE COMP-3	Volume 2, subsection 2.2.7
12R2	POINTER	A data-item can be defined with USAGE IS POINTER.	Volume 1, subsection 3.5.18
12R2	SET	Added format 5 to handle setting pointer data types.	Volume 1, subsection 3.6.24
12R2	Unreserved keywords NULL and NULLS	NULL and NULLS are treated as two words of binary zeroes for the VALUE clause, sending field of a SET statement, and conditional expressions.	Volume 1, subsection 2.4.8
12R2	Unreserved keyword ADDRESS	Used to obtain a two-word pointer address of a data-item within a SET statement.	Volume 1, subsection 3.6.23
12R2	GOBACK	Treated as an EXIT PROGRAM in a sub program or a STOP RUN in a main program.	Volume 1, subsection 3.6.14
12R2	COPY	Allows for nested COPY statements.	Volume 1, subsection 9.2
12R2	RDMS host variable qualification	Alternate qualification for host variables in embedded SQL	Volume 2, subsection 7.3.2
12R1D	MIN-ALS	Can reduce the amount of ALS storage required when the INITIAL clause is used on the PROGRAM-ID statement	Volume 2, subsection 2.2.7
12R1	ZIP$SETCOMPRESSION	New zip function ZIP$SETCOMPRESSION for setting the compression level.	Volume 2, subsection 10.26.7

UCOB Level	Feature Name	Description	Reference
12R1	New time functions	New time functions for retrieving the Coordinated Universal Time (UTC) also referred to as Greenwich Mean Time (GMT).	Volume 2, subsection 10.27
12R1G	ZIP function rewrite	The functions provided for accessing zip utilities are now written in UCS C.	Volume 2, subsection 10.26
11R1B	TRIM-CRLF	New compiler keyword option TRIM-CRLF will automatically strip a CR and/or LF from the end of a source line when a program is compiled.	Volume 2, subsection 2.2.5
11R1A	New ZIP functions	New functions and include file for accessing the zip utilities provided by the Virtual Machine for the Java Platform on the ClearPath OS 2200.	Volume 2, subsection 10.26.
11R1	MIN-DBANK	Reduces the size of $0 static storage (at the expense of additional executable code) by allocating some packets on the ALS instead of in $0.	Volume 2, subsection 2.2.7
11R1	URTS$TABLES	New compiler keyword option determines how much space to reserve in the URTS$TABLES heap bank for data items that must be in the same bank. Useful in eliminating run-time error 37001.	Volume 2, subsection 2.2.7

Section 2
Compiling Your Program

This section describes the compiler call you use to compile your UCS COBOL program. It also describes options that control compilation and compiler listings.

2.1. Calling the UCS COBOL Compiler (@UCOB)

To compile your program, call the UCS COBOL compiler by using the @UCOB statement. The @UCOB statement

- Specifies the location of your program source statements
- Specifies the location (name) of the compiled object module to be created
- Specifies compiler call options (see 2.2), which control the compilation

Examples

```
@UCOB     qual*file.source,qual*file.om
@UCOB,I  ,qual*file.om
@UCOB     qual*file.source,qual*file.om,,,WARNING,DOUBLESPACE
@UCOB     qual*file.source,qual*file.om,,qual*file.keywords
```

General Format

```
@UCOB[,letter-options]  [source-program][,[object-program]
     [,,[keyword-option-element][,keyword-option]...]]
```

where:

[ ]

Brackets mean the enclosed syntax unit is optional. Brackets are notational symbols only; do not actually specify them.

letter-options

Letter options control the compilation and are described in Table 2–2. In case of conflict, keyword options take precedence over letter options; letter options take precedence over keyword options listed in *keyword-option-element*.

source-program

Source-program is the name of the program file symbolic source element containing the program to be compiled. If you omit the source program name, the following conditions must be met:

- You must use the I option.
- The source program must immediately follow the compiler call in the runstream.
- The @EOF statement (or another system statement) must follow the source program runstream to signal termination of the source program.

If you use the I option and specify *source-program*, the source program from the runstream is saved in *source-program*.

If *source-program* does not have a file name, the system uses the file TPF$.

object-program

Object-program is the name of the program file object module element in which the compiled program will be stored. If you omit the object program name

- The compiled object module receives the file and element name of *source-program*.
- If you also omit *source-program*, the compiled object module receives the file and element name TPF$.NAME$.

,,

The two commas preceding *keyword-option-element* denote an unused field. If you specify anything beyond the *object-program* field, you must specify these commas.

keyword-option-element

keyword-option-element is the name of the symbolic element containing keyword options used to control this compilation. Compiler keyword options are described in 2.2.4 through 2.2.8.

A keyword option element lets you store commonly used sets of keyword options so that you do not have to specify the same keywords over and over in different compilations.

This element contains one or more lines containing one or more keywords per line, separated by commas. You can also include a comment on any line by preceding it with the sequence space-period-space (' . '). The Language Support System (LSS) processes everything on the line up to this sequence and ignores everything after it. A comment on one line does not affect the next line in the element.

Example

```
NO-CODE,                    . No object module - syntax check only
SOURCE,REMARK
```

Note: *You can include comments only within the keyword option element, not on the compiler call line.*

In case of conflict, keyword options and letter options take precedence over keyword options listed in *keyword option element*.

keyword-option

Keyword options control the compilation and are described in this section. In case of conflict, keyword options take precedence over letter options and keyword options listed in *keyword option element*.

...

The ellipsis means you can list more than one keyword option, separating them by commas. The ellipsis is a notational symbol only; do not actually specify it.

Note: *In the following examples, qual*file.source is a symbolic element containing the source program and qual*file.om is the object module to be created. (In actual usage, the source program and object module need not be in the same file.)*

Example

Source program in a symbolic element, default options:

```
@UCOB    qual*file.source,qual*file.om
```

Example

Source program from the runstream (the letter option I is required):

```
@UCOB,I    ,qual*file.om
  .
  . { language statements }
  .
@EOF
```

Example

Compilation using keyword options WARNING and DOUBLESPACE:

```
@UCOB    qual*file.source,qual*file.om,,,WARNING,DOUBLESPACE
```

Example

Compilation using keyword options listed in the symbolic element qual*file.keywords:

```
@UCOB     qual*file.source,qual*file.om,,qual*file.keywords
```

In case of conflict, keyword options and letter options take precedence over keyword options listed in *keyword-option-element*.

Example

Compilation using

- The letter options D and E
- The keyword options listed in the symbolic element qual*file.keywords
- The additional keyword option XREF

```
@UCOB,DE    qual*file.source,qual*file.om,,qual*file.keywords,XREF
```

Example

Compilation using

- No object program name (the object module receives the source program name)
- The keyword options WARNING and DOUBLESPACE

```
@UCOB     qual*file.source,,,,WARNING,DOUBLESPACE
```

Additional Rules

- When you omit fields in the compiler call, use commas to indicate the missing fields.
- Compiler call lines are limited to 80 characters. Use a semicolon (;) at the end of a line to continue the compiler call on another line, for example:

  ```
  @UCOB     qual*file.source,qual*file.om,,,WARNING,DOUBLESPACE,;
            NO-CODE
  ```

  Use of the semicolon as a continuation marker is subject to various rules. See the *ECL End Use and Programming Reference Manual* for more information.
- Since UCS compilers use the Service Library (SLIB) symbolic read and symbolic write routines for input and output, correction lines are not accepted. You must apply correction lines using the ELT processor before to compiling the source program.

2.2. Specifying Compiler Call Options

You can control the compilation of your source program using options on the compiler call in any of the following ways:

- Specify letter options on the compiler call (see 2.2.1 and 2.2.2).

```
@UCOB,SR qual*file.source,qual*file.om
```

- Specify keyword options on the compiler call (see 2.2.1 and 2.2.3).

```
@UCOB qual*file.source,qual*file.om,,,CODE,FULL-DEBUG,OPTIONS,WARNING
```

- Specify an element name that contains keyword options (see 2.2.1 and 2.2.3).

```
@UCOB qual*file.source,qual*file.om,,qual*file.keywords
```

If a conflict occurs between letter options, keyword options on the compiler call, or keyword options in an element, the compiler assigns priority in the following order:

1. Keyword options on the compiler call (highest priority)
2. Letter options
3. Keyword options in the element

2.2.1. Commonly Used Sets of Options

Keyword and letter options give you considerable flexibility in controlling the compilation. However, certain sets of keyword or letter options are typically used for certain types of compilations. Table 2–1 gives examples of such compilations and the options typically used to perform them. The compilation types and corresponding options are meant as examples only and are not intended to limit your choice. Note that default values are not listed, since the compiler applies them automatically.

Table 2–1. Typical Compilations and the Options to Perform Them

Compilation Type	Options Used
Checking for syntax errors (no object module)	NO-CODE, SOURCE, REMARK
Program development and debugging	CLEAR, DEBUG/FULL, DEBUG/MONITOR, MONITOR, NO-OPTIM, RUNCHECK, PARMCHECK
Optimal execution	COMPAT/ARGUMENTS, OPTIM/*option*
Optimal compilation (wall time) for large programs	CACHE
Displaying nonstandard features	LEVEL/STANDARD
Providing compatibility with ASCII COBOL	CLEAR, COMPAT
Providing compatibility with C programs	CALIGN

Table 2–1. Typical Compilations and the Options to Perform Them

Compilation Type	Options Used
Producing the least information (output) about the compilation	N option (implies NO-WARNING)
Producing the most information (output) about the compilation	L option (implies LISTCOPY, ALLOC, OBJECT, OPTIONS, SOURCE, XREF, LEVEL/STANDARD, REMARK)
Producing the least information (output) about the compilation	N option (implies NO-WARNING)
Producing the most information (output) about the compilation	L option (implies LISTCOPY, ALLOC, OBJECT, OPTIONS, SOURCE, XREF, LEVEL/STANDARD, REMARK)

The Linker Information Listing specifies linker information for the object module. It is produced by the LINKINFO keyword option. (LINKINFO is the default if you have specified OBJECT or OBJECT-ONLY.) The compiler produces a Linker Information Listing for each object module produced. Linker Information Listing is composed of the following sections:

- Header
- EXTERNAL REFERENCES
- EXTERNAL DEFINITIONS
- COMMON BLOCKS
- ALS INFORMATION
- LOGICAL BANK

Header Section

The header specifies the

- Object module name
- Bank group name for this compilation

  Bank group names are created by appending the suffix $ to the object module name.

EXTERNAL REFERENCES Section

This section lists the names of all external references (calls) made in the object module. Only the first 12 characters of the names are printed. Typical UCS object modules contain external references to the following, in addition to references to user subprograms:

- Entry points with the prefix RTS$, which are UCS Runtime System entry points
- TCA$$$, a UCS Runtime System entry point for the task control area

EXTERNAL DEFINITIONS Section

This section lists all external definitions (entry points) contained in the object module and their addresses. Each entry starts on a new line and has four subfields when the NARROW option is in effect. If the WIDE option is used, the entries contain the fifth field 'LINE NUMBER'. External definitions include

- Internal Name

  This is the entry point name as defined in the source program. It is truncated at 32 characters.

- External Name

  This is the entry point name used for LINK resolution.

- LC

  This is the location counter of the defined entry point.

- Offset

  This is the address of the definition, relative to the start of the given location counter.

- Line Number

  This is the line number in the source program where the definition occurred. It is truncated at 53 characters.

COMMON BLOCKS Section

This section lists all common blocks contained in the object module. Each entry starts on a new line and has two subfields:

- Common block name
  - All EXTERNAL user names appear in this section.
  - All names listed in the COMMON-STORAGE section appear under the single name $BLANK$COMMON$.
- Logical bank number of the bank containing the common block

ALS INFORMATION Section

This section lists the sizes of the activity local stack (ALS) for each procedure in the source program. Each entry starts on a new line and has the following three subfields:

- Internal name

  The procedure name found in the source program.

- External name

  The EXTERNAL name of the procedure used for linker resolution.

- Size

  The number of words needed for the procedure's ALS.

LOGICAL BANK Section

This section describes the logical banks created by the compilation. It includes the following subfields:

- Logical bank name

  Logical bank names are created by appending the logical bank number as a suffix to the bank group name.

- Location counter number allocated for the bank

  This is the same as the logical bank number.

- Size

  This gives the size of the bank in words, in both octal and decimal values.

  The compiler allocates one extra word for each common block and each data bank declared in the program. This permits the compiler to generate more efficient coding sequences in some cases. Thus the size of the common storage is larger than both of the following:

  – The size of the corresponding common block declared in the program

  – The size of the corresponding common block in the Allocation and Cross-Reference Listings

- Bank

  Bank types are code, data, link vector, and so forth. See the *Linking System Programming Reference Manual* for all possible values.

- Bank mode

  Bank modes are EM (extended mode). See the *Linking System Programming Reference Manual* for all possible values.

- Storage class

  This gives the type of storage the bank is loaded into. MSU means ordinary main storage. See the *Linking System Programming Reference Manual* for all possible values.

2.2.2. Specifying Letter Options

Letter options are a convenient way to briefly specify compiler options and listings produced during compilation. They also let you control

- Whether warning and remark messages are produced during compilation
- Whether source input is being provided from the runstream
- Whether the compilation should terminate in error if major errors are encountered

Table 2–2 lists letter options.

Table 2–2. Compiler Call Letter Options

Letter	Description
D	Prints the Allocation Listing. This is a listing of all program symbols for which storage is allocated, including information about data types, storage allocation, and line number where a symbol is declared. Same as keyword option ALLOC.
E	Prints warning and remark messages. Same as keyword options WARNING and REMARK.
I	Specifies that source input is from the runstream. If you specify the I option, the source program must directly follow the compiler call. If you also specify a source element in the compiler call, the source program is saved in a file assigned that source element name. @EOF or some other system statement must follow the source to signal its termination.
L	Specifies all of the listing keyword options. Same as specifying the keyword options ALLOC, LINKINFO, LITERAL, OBJECT, OBJ-PKT, OPTIONS, REMARK, SOURCE, and XREF.
N	Specifies no listing options are to take effect. Same as specifying the keyword options NO-ALLOC, NO-LINKINFO, NO-LITERAL, NO-OBJECT, NO-OBJ-PKT, NO-OPTIONS, NO-REMARK, NO-SOURCE, NO-WARNING, and NO-XREF. The N option is the opposite of L. If D, E, O, R, or S (which conflict with N) are specified with N, they take precedence over N.
O	Prints the Object Code Listing. This is a listing of the generated object code with related source lines interspersed. Same as keyword option OBJECT.
R	Prints the Cross-Reference Listing. This is an alphabetized list of symbols defined in the program and the line numbers of statements that reference the symbols. Same as the keyword option XREF.
S	Prints a listing of the source program. Same as the keyword options SOURCE and REMARK. If you also specify WARNING, the default diagnostic control option, warning messages are printed.
X	Specifies that the termination routine checks for major errors at the end of compilation; if any have occurred, it terminates the compilation in error. Same as the keyword option ERREXIT.

2.2.3. Specifying Keyword Options

Keyword options give you more control over compilation than letter options. The following categories of keyword options exist:

- Source control

  Controls how the compiler searches for copied source text. See 2.2.4.

- Compilation control

  Primarily controls how errors affect the compilation and whether an object module should be produced. See 2.2.5.

- Listing control

  Controls which compiler listings are produced. See 2.2.6.

- Program control

  Controls the nature of the object program code generated by the compiler. These keyword options are extremely important because they let you create code that is designed for special purposes or situations; for example, code that

  – Provides the best compilation or execution performance

  – Can efficiently handle large programs

  – Is specifically designed for debugging or production

  See 2.2.7.

- Diagnostics control

  Controls whether the compiler issues warning, remark, and level diagnostic messages during compilation. See 2.2.8.

Table 2–3 alphabetically lists each keyword option and the subsection containing detailed information. (Note that only one keyword option name of a set of related names is listed. For example, CODE also represents its opposite, NO-CODE.) Within each table grouping, keyword options are listed alphabetically.

Table 2–3. List of Keyword Options

Keyword Option	Reference
ALLOC	2.2.6
APPLICATION/*name*	2.2.7
AUXPROLOG/*n*	2.2.7
CACHE/*spec*	2.2.7
CALIGN	2.2.7
CLEAR	2.2.7
CODE	2.2.5
CODE-LVE	2.2.7

Table 2–3. List of Keyword Options

Keyword Option	Reference
COMPAT	2.2.7
COMP-BIN	2.2.7
D$WORK-WS	2.2.7
DEBUG/option	2.2.7
ERRCHECK	2.2.5
ERREXIT	2.2.5
EXTENDED	2.2.7
IR-DIFF	2.2.8
I18N-IO	2.2.7
LEVEL options	2.2.8
LEVEL/RPW	2.2.8
LEVEL/SEG	2.2.8
LEVEL/SEG2	2.2.8
LEVEL/SQL	2.2.8
LINKINFO	2.2.6
LISTCOPY	2.2.5
LITERAL	2.2.6
MAIN-PROGRAM and SUBPROGRAM	2.2.7
MAXERRORS/*n*	2.2.5
MAX-SIZE/*m* *and MAXMEM/m*	2.2.7
MIN-ALS	2.2.7
MIN-DBANK	2.2.7
MONITOR	2.2.7
MULTI-PF	2.2.4
NARROW or WIDE	2.2.6
OBJECT	2.2.6
OBJ-PKT	2.2.6
OPTIM	2.2.7
OPTIONS	2.2.6
PARMCHECK	2.2.7
*double-or-single*QUOTE	2.2.6
REL18	2.2.7

Table 2–3. List of Keyword Options

Keyword Option	Reference
REMARK	2.2.8
REORDER	2.2.7
RUNCHECK	2.2.7
SEARCH-*order*	2.2.4
SEGCODE	2.2.7
SHIFT	2.2.7
SIGNON	2.2.6
SOURCE	2.2.6
*double-or-single*SPACE	2.2.6
TIPMALLOC	2.2.7
TRACECOPY	2.2.6
TRACEMSG	2.2.6
TRANSFORM	2.2.7
TRIM-CRLF	2.2.5
UREP-XREF	2.2.7
URTS$TABLES/*n*	2.2.7
WARNING	2.2.8
XML/*name*	2.2.7
XREF	2.2.6

Only one of the possible alternatives for a keyword option can be in effect at a given time. For example, if you specify the listing keyword option NARROW, WIDE cannot also be in effect. If conflicting keyword options appear on the same compiler call, the compiler issues a warning diagnostic message and uses the last keyword option specified.

Most keyword options have a default value, which is identified in the table entry. The default for some keyword options depends on the specification of other keyword options. These cases are explained in the text for those keyword options.

The following subsections list keyword options and defaults according to the categories they control.

2.2.4. Compiler Call Keyword Options for Source Control

Table 2–4 summarizes the source control keyword options.

Table 2–4. Source Control Compiler Keyword Options

Keyword Option	Description
MULTI-PF	Determines whether you can have multiple special program files of copied source text.
SEARCH-*order*	Determines the search order for copied source text.

MULTI-PF

Format

```
MULTI-PF
NO-MULTI-PF
```

Default: MULTI-PF

These keyword options determine whether you can have multiple special program files of copied source text, used by the COPY statement.

MULTI-PF means you can have multiple special program files, with a maximum of 51. These files must have an actual file name or an @USE name of the following format:

COBPF, COBPF1, COB$PF2, ... COB$PF50

The compiler searches all such files for copied source text, until one of the following occurs:

- COB$PF50 is searched.
- A gap in the sequence of files occurs.

For example, in both of the following cases, COB$PF3 is the last file searched:

COB$PF, COBPF1, COB$PF2, COB$PF3

COBPF, COBPF1, COB$PF2, COB$PF3, COB$PF5

If these files are not assigned, the compiler dynamically assigns them to the run and frees them at the end of compilation (provided the files exist).

NO-MULTI-PF means the compiler searches only the single file COB$PF for copied source text (provided it exists).

SEARCH-*order*

Format

```
SEARCH-PF-SI
SEARCH-SI-PF
```

Default: SEARCH-PF-SI

These keyword options determine which files are searched by the compiler for COPY libraries when you have not specified the file containing the library. (These options have no effect when you do specify the file.)

In these options, PF stands for program file, meaning the special program file COB$PF. SI stands for source input (that is, the file specified in source-program on the compiler call). SEARCH-PF-SI searches the special program file first; SEARCH-SI-PF searches the source input file first.

Other files are also searched if the copied text is not found in these files. The complete search order is as follows for SEARCH-PF-SI:

1. The special program file COB$PF is searched. (This can be the actual file name or an @USE name.) If the file is not assigned, the compiler dynamically assigns it to the run and frees it at the end of compilation (provided the file exists).

If you use the MULTI-PF keyword option, you can use up to 51 such files.

2. The file specified in source-program on the compiler call (source input) is searched (provided you do not use the I option).

   If you do use the I option, the file specified in object-program on the compiler call (object module output) is searched.

3. The program file TPF$ is searched.
4. The system library file SYSLIB*PROC$ is searched.
5. If the proc or element is not found in any of these files, the compiler issues an error message; compilation continues as if the COPY statement is not present.

SEARCH-SI-PF reverses the order of the first two files.

2.2.5. Compiler Call Keyword Options for Compilation Control

Table 2–5 summarizes the compilation control keyword options.

Table 2–5. Compilation Control Compiler Keyword Options

Keyword Option	Description
CODE	Determines whether the compiler produces an object module or merely checks syntax.
ERRCHECK	Determines whether an object module is produced if there are major errors.
ERREXIT	Determines whether the compilation is terminated if there are major errors.
LISTCOPY	Determines whether text of included files or copied source lines is listed.
MAXERRORS/*n*	Terminates compilation after *n* major errors.
TRIM-CRLF	Causes CR and/or LF (carriage-return line-feed) characters at the end of a source input line to be ignored. Only one of each will be ignored.

CODE

Format

```
CODE
NO-CODE
```

Default: CODE

CODE causes the compiler to produce an object module. NO-CODE means the compiler does not produce an object module or generate the Allocation, Cross-Reference, or Object Code listings. However, the compiler does check syntax and semantics and generates the Source Listing if requested. NO-CODE is useful for syntax checking when you first compile your program. lt prevents you from wasting system resources by trying to produce an object module when your program has errors.

ERRCHECK

Format

```
ERRCHECK
NO-ERRCHECK
STD-ERRCHECK
```

ERRCHECK means an object module and most listings are not produced if the source element has any major errors. This effectively cancels the ALLOC, LINKINFO, and OBJECT options.

NO-ERRCHECK causes the compiler to produce an object module even if the source element has major errors, unless ERREXIT is specified. (Fatal or internal errors, however, are nonrecoverable and prevent generation of an object module, even with NO-ERRCHECK.)

Note that ERREXIT has precedence over NO-ERRCHECK and can suppress the generation of an object module.

STD-ERRCHECK has the same effect as ERRCHECK, but it also changes the default of the LEVEL option to LEVEL/STANDARD and produces no object module if the program contains any nonstandard features (extensions).

ERREXIT

Format

```
ERREXIT
NO-ERREXIT
```

Default: NO-ERREXIT

ERREXIT causes the compiler to check for major errors. If it finds any, the compiler calls the termination routine to terminate the compilation in error. For ERREXIT, ERRCHECK is in effect unless you specify STD-ERRCHECK.

NO-ERREXIT means the compiler does not terminate the compilation in error.

If you use NO-ERREXIT and NO-ERRCHECK, and your object module has major errors, there is no guarantee that the object module will execute correctly.

Neither ERREXIT nor NO-ERREXIT has an effect if other conditions cause premature error termination.

LISTCOPY

Format

```
LISTCOPY
NO-LISTCOPY
```

Default: NO-LISTCOPY

LISTCOPY lists the text of all included files or copied source lines. NO-LISTCOPY prevents the copied source from being included in the source listing.

These options are subordinate to the SOURCE option and the S letter option; SOURCE specifies LISTCOPY and NO-SOURCE specifies NO-LISTCOPY.

MAXERRORS/*n*

Format

```
MAXERRORS/n
```

MAXERRORS/*n* specifies that the compilation terminates immediately if more than n major errors occur in the compilation. If you specify MAXERRORS, you must specify /*n*, with no spaces on either side of the slash.

If you do not specify MAXERRORS, the default for *n* is 100,000.

Since UCS compilers are multiple-pass compilers, the order in which the system lists error messages is not necessarily the order in which it detected them. Therefore, after the compilation terminates as a result of reaching the MAXERRORS limit, additional error messages can appear in the listing because they were detected in an earlier pass of the compilation.

TRIM-CRLF

Formats

```
TRIM-CRLF
NO-TRIM-CRLF
```

Default: TRIM-CRLF

TRIM-CRLF automatically strips a CR and/or LF from the end of a source line when a program is compiled. Only one CR and/or one LF is stripped, and they must be at the very end of the line (or followed only by blanks).

When source programs are edited or created on workstations, there are usually CR-LF (Carriage-Return Line-Feed) characters at the end of each line to signal the end of a line. When these programs are copied to OS 2200 files with products such as cpFTP or TAS, these trailing sequences are usually removed when the OS 2200 SDF images are created for the lines. If the sequences are NOT removed, the UCS compilers usually give errors or warnings during compilation when they encounter the 'extra' CR or LF characters at the ends of the lines.

In addition, one can edit OS 2200 files directly from workstations using a mapped drive handled by CIFS. CIFS removes a line feed character and terminates the SDF image for a line at that point. It does NOT remove the carriage return if one is there. You must manually run a utility to remove the extra CR that CIFS leaves at the end of the images. (You must do a @CIFSUT *cp* command with the *–w* option.) For more information on this, see the *CIFS for ClearPath OS 2200 User, Programmer, and Administrator Reference Manual*. Using the TRIM-CRLF option means that you do not have to manually remove these trailing CR characters.

NO-TRIM-CRLF will not strip the CR and/or LF from the end of the source lines.

2.2.6. Compiler Call Keyword Options for Listing Control

Table 2–6 summarizes the listing control keyword options. An example of each type of compiler listing is in 2.2.3.

Table 2–6. Summary of Listing Control Compiler Keyword Options

Keyword Option	Description
ALLOC	Determines whether the Allocation Listing is produced.
*double-or-single*QUOTE	Determines whether the figurative constant QUOTE contains " or '.
*size*SPACE	Determines whether the Source Listing is double or single spaced.
LINKINFO	Determines whether the Linker Information Listing is produced.
LITERAL	Determines whether Japanese literals are listed.
NARROW or WIDE	Determines whether listings are 79 or 132 characters per line.
OBJECT	Determines whether the Object Code Listing is produced.
OBJ-PKT	Determines whether a more detailed Object Code Listing is produced.
OPTIONS	Determines whether the Options Listing is produced.
SIGNON	Determines whether the sign-on line, bank sizes line, and sign-off line are produced.
SOURCE	Determines whether the Source Listing is produced.
TRACECOPY	Determines whether the Tracecopy Listing is produced.
TRACEMSG	Determines whether diagnostic messages indicate the location (qual*file.elt) of the source line.
XREF	Determines whether the Cross-Reference Listing is produced.

ALLOC

Format

```
ALLOC
NO-ALLOC
```

Default: NO-ALLOC

ALLOC produces the Allocation Listing (see 2.3.6). This lists all program symbols for which storage is allocated, including information about data types, storage allocation, and line number in which the symbol was declared.

NO-ALLOC suppresses the listing of program symbols.

The Allocation Listing is not printed in the following cases:

- You specify NO-CODE.
- You specify ERRCHECK and your program has a major error.

DOUBLEQUOTE

Format

```
DOUBLEQUOTE
SINGLEQUOTE
```

Default: DOUBLEQUOTE

DOUBLEQUOTE causes the figurative constant QUOTE to contain the character quotation mark (").

SINGLEQUOTE causes the figurative constant QUOTE to contain the character apostrophe (').

DOUBLESPACE

Format

```
DOUBLESPACE
SINGLESPACE
```

Default: SINGLESPACE

DOUBLESPACE inserts a blank line between source lines on the Source Listing. Blank lines are not inserted between other lines, including diagnostic message groups. The double spacing is visible only when the source program is printed.

Until then, it appears to be single-spaced even when the output is redirected to a print file and then displayed on a terminal.

SINGLESPACE causes the source lines on the listing to be single-spaced.

LINKINFO

Format

```
LINKINFO
NO-LINKINFO
```

LINKINFO produces the Linker Information Listing (see 2.3.9). This lists the external definitions, external references, common block names, and logical bank sizes. LINKINFO is the default when you specify OBJECT or OBJECT-ONLY.

NO-LINKINFO suppresses the listing and is the default for NO-OBJECT.

The Linker Information Listing is not printed in the following cases:

- You specify NO-CODE.
- You specify ERRCHECK and your program has a major error.

LITERAL

Format

```
LITERAL
NO-LITERAL
```

Default: NO-LITERAL

LITERAL lists all Japanese literals used in the program source.

NO-LITERAL does not list the Japanese literals used in the program source.

NARROW

Format

```
NARROW
WIDE
```

NARROW prints listings with 79 characters per line. NARROW is the default in demand mode with print directed to the terminal.

WIDE prints listings with 132 characters per line. WIDE is the default in batch mode or in demand mode with print redirected by a @BRKPT statement.

OBJECT

Format

```
OBJECT
OBJECT-ONLY
NO-OBJECT
```

Default: NO-OBJECT

OBJECT produces the Object Code Listing (see 2.3.8). This lists the generated object code with related source lines interspersed.

OBJECT-ONLY lists the generated object code without interspersed source lines.

Both OBJECT and OBJECT-ONLY invoke LINKINFO.

NO-OBJECT suppresses the listing of generated object code.

The Object Code Listing is not printed in the following cases:

- You specify NO-CODE.
- You specify ERRCHECK and your program has a major error.

OBJ-PKT

Format

```
OBJ-PKT
NO-OBJ-PKT
```

Default: NO-OBJ-PKT

OBJ-PKT produces a more complete Object Code Listing by printing additional information for I/O packets, parameter packets, and array descriptors. OBJ-PKT is the default with the letter option L.

To use OBJ-PKT, you must use OBJECT or OBJECT-ONLY.

NO-OBJ-PKT suppresses the additional information on the Object Code Listing.

OPTIONS

Format

```
OPTIONS
NO-OPTIONS
```

OPTIONS produces the Options Listing (see 2.3.2). This lists all applicable compiler options and the values assigned, including default values. OPTIONS is the default in batch mode or in demand mode with print redirected by a @BRKPT statement.

NO-OPTIONS does not print a list and is the default in demand mode with print directed to the terminal.

SIGNON

Format

```
SIGNON
NO-SIGNON
```

Default: SIGNON

SIGNON produces the sign-on line, the bank sizes line, and the sign-off line.

NO-SIGNON suppresses the printing of the sign-on line, the bank sizes line, and the sign-off line.

SOURCE

Format

```
SOURCE
NO-SOURCE
```

Default: NO-SOURCE

SOURCE produces the Source Listing (see 2.3.4). This prints the source program in a format based on other listing control keyword options you select (for example, DOUBLESPACE, WIDE, and so forth).

NO-SOURCE suppresses printing of the source program.

TRACECOPY

Format

```
TRACECOPY
NO-TRACECOPY
```

Default: NO-TRACECOPY

TRACECOPY produces the Tracecopy Listing which lists the locations of COPY files found during a compilation.

NO-TRACECOPY suppresses printing of the Tracecopy Listing.

TRACEMSG

Format

```
TRACEMSG
NO-TRACEMSG
```

Default: NO-TRACEMSG

TRACEMSG determines whether diagnostic messages indicate the location (*qual***file.elt/ver*) of the source line.

NO-TRACEMSG suppresses printing of the location of the source line in diagnostic messages.

XREF

Format

```
XREF
SHORT-XREF
NO-XREF
```

XREF produces the Cross-Reference Listing (see 2.3.7). This alphabetically lists symbols defined in the program and the line numbers of any statements that reference the symbols.

SHORT-XREF prints the same listing as XREF but does not list the unreferenced symbols. Short-XREF also prints DMS symbols internally referenced by the compiler.

NO-XREF suppresses printing of a Cross-Reference Listing.

2.2.7. Compiler Call Keyword Options for Program Control

Table 2–7 summarizes the program control keyword options.

Table 2–7. Program Control Compiler Keyword Options

Keyword Option	Description
APPLICATION/*name*	Specifies the repository to be used by UREP invoke directives and embedded SQL statements within the program, and by cross-reference processing initiated by the UREP-XREF keyword option.
AUXPROLOG/*n*	Determines whether the compiler generates additional prologue and epilogue code for each user program and subprogram in a compilation unit. This prologue and epilogue code will call a user-supplied prologue subprogram and a user-supplied epilogue subprogram when this feature has been activated during program execution by a call to the service subprogram UCS$ACTAUX.
CACHE/*spec*	Can improve wall-time compilation performance for large programs.
CALIGN	Perform actions to allocate data items with USAGE COMP-1, COMP-2, and COMP-5 in the same manner as variables with same internal format defined in a C program.
CLEAR	Determines whether static variables are initialized to zero.
CODE-LVE	Determines whether creators of fixed-gate shared subsystems need to define fixed gates for internal entry points.
COMPAT	Performs actions to provide compatibility with ASCII COBOL.
COMP-BIN	Helps promote compatibility between ASCII COBOL and UCS COBOL by causing a USAGE-COMP item to be treated as USAGE BINARY.
D$WORK-WS	Tells the compiler to allocate the DMS D$WORK data structure in WORKING-STORAGE, even if the INVOKE clause places the DMCA into COMMON. Use of this option is recommended for all DMS programs.
DEBUG/*option*	Determines whether (and how much) debugging code is included in the object module.
EXTENDED/*n*	Determines whether code is generated for execution on any extended mode system or a specific extended mode system.

Table 2–7. Program Control Compiler Keyword Options

Keyword Option	Description
I18N-IO	Determine whether calls to I18NLIB routines should be made in order to determine the CODE TYPE or LOCALE that is currently in force.
MAIN-PROGRAM and SUBPROGRAM	Determine whether the object module is a main program or subprogram and thus where execution begins when multiple object modules are statically linked into a single object module.
MAX-SIZE and MAXMEM/*m*	Limits the amount of memory used by the compiler's internal paging area.
MIN-ALS	Can reduce the amount of ALS storage required when the INITIAL clause is used on the PROGRAM-ID statement.
MIN-DBANK	Reduces the size of $0 static storage, at the expense of additional executable code, by allocating some packets on the ALS instead of in $0.
MONITOR	Helps debug a program by printing the contents of a data item each time the data item is referenced.
OPTIM	Determines whether optimizations are performed to improve program performance.
PARMCHECK	Determines whether warning messages are issued when the number of parameters in the calling and called procedure do not match.
REL18	Determines whether the more efficient and space saving 18-bit relocation method is used.
REORDER	Improves program performance by rearranging instructions to better utilize the hardware.
RUNCHECK	Determines whether various run-time error conditions are tested for, and terminates program execution if such conditions occur.
SEGCODE	Determines whether multiple code banks are generated, with the total size allowed to exceed 262,143 words. This keyword is useful for large programs.
SHIFT	Determines whether the program may have kanji shift code images.
TIPMALLOC	Determines whether to reference and initialize the TIPMALLOC$$$ bank at the main prologue execution. The TIPMALLOC$$$ bank will be used instead of the URTS$TABLES bank for the UC malloc, calloc, or realloc memory allocation functions. The COBOL RunTime routines utilize the UC memory allocation functions. This option should only be used when compiling the main program.
TRANSFORM	Determines whether the compiler performs transformations that improve execution performance but may also cause precision differences.

Table 2–7. Program Control Compiler Keyword Options

Keyword Option	Description
UREP-XREF	Determines whether cross-reference information is stored in UREP.
URTS$TABLES/*n*	Determines how much space to reserve in the URTS$TABLES heap bank for those things that must be in the same bank. Can be used to eliminate runtime error 37001 (attempt to expand URTS$TABLES beyond its maximum upper limit – all basic mode addressable space has been used). This option is only effective when used on the compilation of the main program for your executable.
XML/*name*	Determines whether an XML document is created for a user specified data record.

APPLICATION/*name*

Format

```
APPLICATION/name
APPLICATION/alias
```

APPLICATION/*name* specifies

- The repository UREP uses to bring source input into the program when a UREP invoke is specified.
- The repository in which cross-reference relationships are created when the UREP-XREF option is in effect.
- The repository used for embedded SQL commands for both compile-time and run-time actions.

APPLICATION/*alias* specifies an alias for the name to be used for the same functions.

Both the name and alias consist of 1 to 12 characters. If you do not specify APPLICATION, the default is (in order of precedence)

- The application name on an optional DMS 2200 INVOKE statement
- The application name UDSSRC

See Section 7 for more information about RDMS and programs containing embedded SQL statements. See Section 8 for more information about UREP.

AUXPROLOG/*n*

Format

```
AUXPROLOG/n
NO-AUXPROLOG
```

Default: NO-AUXPROLOG

AUXPROLOG/*n* generates additional prologue and epilogue code for each user routine in a compilation unit. This generated code determines if user-supplied auxiliary prologue (USER$PROLOG) and epilogue (USER$EPILOG) subprograms should be called during the execution of the prologue and epilogue code for a program or subprogram. The calling of these subprograms is governed by a flag that is set at program execution time by calling the service subprogram UCS$ACTAUX and cleared by calling the service subprogram UCS$DEACTAUX.

AUXPROLOG/*n* also creates an auxiliary stack frame on the activity-local stack (ALS) with a size of n words, with a minimum size of two words. The first two words of the stack frame are reserved. The first word contains the virtual address (VA) of the previous auxiliary stack frame (if one is active on the stack frame) or zero. The second word contains the virtual address of the entry point of the routine that called the user-supplied USER$PROLOG or USER$EPILOG routine. These two reserved words can be accessed by the user-supplied USER$PROLOG and USER$EPILOG routines but should not be modified by them. The remaining words can be used and modified.

USER$PROLOG and USER$EPILOG are the external names of the user-supplied routines that are called from the auxiliary prologue and epilogue code generated by the compiler. They each have one parameter, which is the auxiliary stack frame. These subprograms are usually written in a system software language such as UCS C, which can manipulate virtual addresses. See the keyword AUXPROLOG/*n* in the *C Compiler Programming Reference Manual Volume 2* for an example of a user-supplied auxiliary prologue and epilogue subprogram.

NO-AUXPROLOG inhibits the generation of additional auxiliary prologue and epilogue code by the compiler. It also inhibits the creation of the auxiliary stack frame.

CACHE/*spec*

Formats

```
CACHE/D0
CACHE/Dn
CACHE/Sn
CACHE/n
```

Default: CACHE/D0.

The purpose of CACHE is to improve wall-time compilation performance by using the available real memory on the system to reduce the number of I/O CACHE/D*n* requests. (Real memory is much faster than issuing I/O requests to peripheral devices.) You may want to use this option in the following circumstances:

- You have a very large program. (Cache storage is especially useful for very large programs.)
- Compilation is unacceptably slow.
- Your system has extra memory that you can use during compilation.

The compiler's storage manager has an internal paging area that holds the intermediate form of your program. (This internal paging area is approximately 15 million words.) When this paging area overflows, the compiler's storage manager uses peripheral I/O storage to manage this space.

When the CACHE/*n* option is used, the storage manager uses up to *n* 262,144 word cache banks as intermediate buffers to manage its paging area. It only uses peripheral I/O if it runs out of space in the cache banks. (The CACHE option is probably unnecessary on most compilations on current 2200 systems due to their large memories.)

If the system gives you the memory to hold the cache banks, cache storage decreases wall time and I/O SUPs. If real memory is not available or the operating system does not give your run the space the cache banks needed, the compilation wall time increases, system swap file activity increases, and general system throughput can suffer. If your system is short on memory, heavy use of the CACHE option should be reserved for batch runs at off-peak times.

CACHE/D0 (the default) means cache storage is not used during compilation; the compilation uses a mass storage file, if needed.

For CACHE/D*n* or CACHE/*n*, *n* represents an upper limit on the number of banks that can be used in the compilation. (The absolute upper limit is 256.) The compiler allocates only as many banks as it needs. The larger the program, the greater the value for *n*. You should use one cache bank for every 1000 to 2000 lines of code.

CACHE/S*n* is similar to CACHE/D*n* except that some required memory is obtained immediately, instead of only as needed. This method decreases ER SUPS for large compilations, but also can allocate more memory than needed for your compilation.

CALIGN

Format

```
CALIGN
NO-CALIGN
```

Default: NO-CALIGN

CALIGN assures that all data names with the following definitions and group items containing such data names will be word-aligned:

- USAGE COMP-1
- USAGE COMP-2
- USAGE COMP-5 PIC S9(y) or PIC 9(*y*); where *y* = 5 through 10

CALIGN assures that all data names with the definition USAGE COMP-5 PIC S9(z) or PIC 9(*z*) (where *z* = 1 through 4) will be half-word-aligned. Group items containing such data names will be word-aligned.

CLEAR

Format

```
CLEAR
NO-CLEAR
```

Default: NO-CLEAR

CLEAR sets any static variables (that have no initial value specified) to zero.

NO-CLEAR does not specify an initial value for these variables.

CODE-LVE

Format

```
CODE-LVE
NO-CODE-LVE
```

Default: NO-CODE-LVE

CODE-LVE eliminates the need for a subsystem creator to define fixed gates for routines (entry points) internal to the subsystem. It generates a one-word link vector entry in the code bank for each imported procedure, in addition to the normal two-word link vector entry in the link vector bank.

When the subsystem creator splits subsystem code from data during static linking with @LINK,Y, the Linking System uses the appropriate link vector entry to resolve the call to the internal entry point.

NO-CODE-LVE suppresses generation of the one-word link vector entry in the bank; subsystems using @LINK,Y must still define fixed gates for internal entry points.

For complete information on use of this feature, see the *Linking System Subsystems Programming Guide*.

COMPAT

Format

```
COMPAT
COMPAT/FULL
COMPAT/FULLHV377
COMPAT/HV177
NO-COMPAT
COMPAT/ARGUMENTS
```

Default: NO-COMPAT

COMPAT makes the following features compatible with ASCII COBOL:

- HIGH-VALUE is octal 0177 (all bits set for a 7-bit machine).
- The SYNCHRONIZED clause allocates compatible data.
- Word alignment for COMP-1 and COMP-2 data items is compatible.
- COMP data items are allocated as binary.
- COMP-3 data items are allocated as PACKED-DECIMAL. COMP-3 requires COMPAT or COMP-BIN.
- Exact binary items can be described the same. For example, PIC 1 without a USAGE BINARY-1 clause defines an exact binary item.
- PROGRAM-ID can be an integer.
- No decimal truncation occurs when the value of the sender contains more significant digits than the receiver, if the receiver is a binary, character numeric, edited or packed-decimal item.
- Moving a floating-point item or performing an arithmetic calculation done in floating point where the receiver is a binary, character numeric, edited or packed-decimal item will produce the same results as ASCII COBOL.

COMPAT/FULL is the same as COMPAT, except that all static variables are allocated, whether or not they are ever referenced. Also, all static variables are placed in one bank except for DMCA (DML), which is defined in working storage section or common storage section depending on the specifications you specified in the INVOKE clause. In addition, database data names (USAGE DATABASE-KEY, AREA-NAME, RECORD-NAME ...) will be placed in a separate bank. COMPAT/FULL sets COMP-BIN.

COMPAT/FULLHV377 is the same as COMPAT, except that HIGH-VALUES is set to octal 0377 (all bits set for an 8-bit machine). COMPAT/FULLHV377 sets COMP-BIN.

COMPAT/HV177 sets HIGH-VALUES to octal 0177. It does nothing else. It does not set COMP-BIN.

NO-COMPAT performs no compatibility actions.

COMPAT/ARGUMENTS restricts parameters passed BY REFERENCE to 01 and 77 level item. This allows the compiler to assume that all incoming parameters are word aligned and thus allows better code to be generated for parameter references.

COMP-BIN

Format

```
COMP-BIN
NO-COMP-BIN
```

Default: NO-COMP-BIN

COMP-BIN promotes compatibility between ASCII COBOL and UCS COBOL source programs.

COMP-BIN causes an item declared as USAGE COMP to be treated as though it was declared USAGE BINARY. COMP-BIN also causes items declared as USAGE COMP-3 to be allocated as USAGE PACKED-DECIMAL. The COMPAT, COMPAT/FULL, and COMPAT/FULLHV377 options set this option. Programs that use the COMPAT, COMPAT/FULL, and COMPAT/FULLHV377 options cannot override COMP-BIN during compilation.

NO-COMP-BIN does not cause any difference in treatment of COMP data items.

D$WORK-WS

Format

```
D$WORK-WS

NO-D$WORK-WS
```

Default: NO-D$WORK-WS

This option is for use by DMS programs. When a DMS program places the DMCA into COMMON using the INVOKE clause, the UCS COBOL compiler normally puts the DMS D$WORK data structure at the end of COMMON-STORAGE, after all the user variables. If COMMON-STORAGE layout is not the same in all program units in an executable program, user variables can overlay the D$WORK data structure and problems can occur. The D$WORK-WS keyword option causes the compiler to put the DMS D$WORK data structure into WORKING-STORAGE, thereby eliminating any potential problems.

It is recommended that all COBOL programs that use DMS use this keyword option in their compilations since it eliminates the potential problems that results if D$WORK overlays user variables in COMMON-STORAGE. The option is **not** on by default.

DEBUG/*option*

Format

```
DEBUG
DEBUG/option
NO-DEBUG
```

DEBUG generates a PADS symbolic debugging dictionary (SDD) that eliminates all information about variables that are declared but not referenced in the program. However, if any member of a group is referenced, information on all group members and any REDEFINES of the group are included in the SDD. This is useful when using PADS to debug programs that copy source code but actually use only a few of the shared items declared in the copied text.

DEBUG/*option* controls the amount and type of debugging information available for a program. The following options are available:

- DEBUG/FULL
- DEBUG/MONITOR
- DEBUG/REFERENCE
- DEBUG/STATEMENT
- DEBUG/WALKBACK (default)

DEBUG/FULL generates a PADS symbolic debugging dictionary (SDD) in the object module, including declared but unused variables, line numbers, and procedure information.

DEBUG/MONITOR generates information in the symbolic debugging dictionary, in addition to the information generated by the DEBUG, DEBUG/REFERENCE, and DEBUG/FULL options. This information allows for the monitoring of input and output for each line of source code executed during a PADS debugging session. This option invokes the DEBUG keyword option if the DEBUG, DEBUG/REFERENCE, or DEBUG/FULL keyword options are not specified.

DEBUG/REFERENCE generates a PADS symbolic debugging dictionary (SDD) similar to what is generated for DEBUG, except information on more group members is eliminated. Information on variables that are group items or group members is included in the SDD when any of the following rules apply:

1. An item is included in the SDD when it is referenced in the program.
2. A group item is included in the SDD when at least one of its members is referenced in the program.
3. A group member is included in the SDD when at least one of the group items of which it is a member is referenced in the program.

4. An overlaid item (both redefining and redefined) is included in the SDD when it overlays any item that is included in the SDD according to rules 1 through 3.

Exceptions to these rules include the following:

- When a group item is only referenced in an APPLY EXDEF/EXREF statement, RENAMES clause, or PROCEDURE DIVISION USING statement, the group item is included in the SDD but rule 3 does not apply to its group members.
- The appearance of an item in a DATA RECORD clause is not considered to be a reference unless the associated file name is referenced in the program.
- When a DATA RECORD clause contains more than one record name, these records are treated as REDEFINES by the compiler and rule 4 applies.
- The appearance of an item in a REDEFINES clause is not considered to be a reference.
- When the INITIAL clause is specified on the PROGRAM-ID, all items declared in WORKING-STORAGE that contain a VALUE clause are considered to be referenced.

DEBUG/REFERENCE Example

```
01 A.
     02 B.
        03 B1 PIC X(2).
        03 B2 PIC X(2).
     02 C PIC X(4).
     02 D.
        03 D1 PIC 9(4).
        03 D2 PIC 9(2).
        03 D3 PIC 9(2).
     02 E PIC 9(4).
     02 F PIC X(4).
     02 G REDEFINES F.
        03 G1 PIC X(2).
        03 G2 PIC X(2).
            .
            .
     MOVE 10 TO D2.
     MOVE 'ABCD'TO F.
     MOVE F TO B.
```

When the DEBUG/REFERENCE keyword option is used to compile this program, information on variables A, B, B1, B2, D, D2, F, and G is included in the SDD. Information on variables C, D1, D3, E, G1, and G2 is not included in the SDD.

DEBUG/STATEMENT generates an SDD that contains information about procedures, functions, and line numbers. The SDD will not contain any information about variables or labels.

DEBUG/WALKBACK generates an SDD that contains information only about procedures and functions, allowing the use of such PADS commands as INQUIRE WALKBACK and TRACE ON FOR PROCEDURE. This type of SDD provides the minimum level of PADS high-level debugging.

NO-DEBUG suppresses generation of the SDD and is recommended for production programs.

EXTENDED/*n*

Format

```
EXTENDED
EXTENDED/n
```

Default: EXTENDED.

Code is targeted for the class of machine that performs the compilation.

EXTENDED/*n* specifies the machine class of the system where the object module produced by the compiler will execute. The parameter *n* can have one of the following values, depending on the machine:

0	Both XPA and non-XPA systems
2	1100/90, 2200/400, or 2200/600 system
4	2200/100, 2200/200 system
6	2200/XPA machine (CMOS systems)
8	4000 system and above (SAIL-based systems)

If you want the most efficient code possible for a particular machine, select the appropriate value for *n*. Code targeted for one class of machine can run on a different class. However, it might not be as efficient as code targeted for that class.

EXTENDED, the default value, means code is targeted for the class of machine that performs the compilation.

I18N-IO

Format

```
I18N-IO
NO-I18N-IO
```

Default: NO-I18N-IO

When the I18N-IO option is specified, processing of CODE TYPE and LOCALE is done the same as the original implementation in UCOB 8R1 and 8R1A (i.e., calls to I18NLIB routines are generated to get the CODE TYPE number and LOCALE number of the locale in force). If the I18N-IO option is not set (NO-I18N-IO) then these calls to I18NLIB are not generated, and a CODE TYPE number of 1 (ASCII) and LOCALE number of 0 (no locale) are always used.

The use of the NO-I18N-IO option eliminates the overhead of the I18NLIB calls for users who simply want to use the ASCII CODE TYPE and LOCALE number.

MAIN-PROGRAM

Format

```
MAIN-PROGRAM
SUBPROGRAM
```

Default: MAIN-PROGRAM

These keyword options allow the Linking System to identify which program unit is the main program and which are subprograms when it statically links multiple program units into a single object module. The Linking System uses these keyword options to determine where program execution begins when the statically linked object module is executed.

MAIN-PROGRAM creates an object module that can be executed by @XQT or called by another object module as a subprogram.

SUBPROGRAM creates an object module that can be called as a subprogram by another object module only. If the program PROCEDURE DIVISION statement includes a USING clause, the compiler assumes SUBPROGRAM.

MAX-SIZE/*m* and MAXMEM/*m*

Format

```
MAX-SIZE/m
MAX-SIZE/mK
MAX-SIZE/mM
MAX-SIZE/DEFAULT

MAXMEM/m
MAXMEM/mK
MAXMEM/mM
MAXMEM/DEFAULT
```

Default: MAX-SIZE/DEFAULT or MAXMEM/DEFAULT

where:

m

specifies a memory limit with a granularity of 1 word. The minimum value of *m* is 1. The maximum value of *m* is 16,777,216 (2**24).

*m*K

specifies a memory limit with a granularity of 1,000 decimal words. The minimum value of *m* is 1. The maximum value of *m* is 16,777.

*m*M

specifies a memory limit with a granularity of 1,000,000 decimal words. The minimum value of *m* is 1. The maximum value of *m* is 16.

The MAX-SIZE and MAXMEM keyword options have the same meaning; each can be considered an alias of the other. These options can be used to put a limit on the amount of memory used by the compiler's internal paging area.

The compiler's internal paging area is approximately 200,000 words and is in a 262,144 bank. The paging area is automatically expanded as needed during a compilation. This expansion can grow to 16 million words on a large compilation.

Using the MAX-SIZE or MAXMEM option bypasses these defaults and limits the paging area size to the *m* value. Note that the *m* value is rounded up to the nearest 262,144 boundary for expansion purposes (for example, 1 is the same as 262,144 and 300K is the same as 524,288).

Using the MAX-SIZE/DEFAULT option nullifies the effect of any previous MAX-SIZE/*m* option (such as one supplied in a keyword options element). The paging area maximum size is then determined by the machine type and CACHE/*n* values as described in the previous paragraphs.

Examples

```
@UCOB PROG1,,,,SEGCODE,MAX-SIZE/4000000

@UCOB PROG2,,,,SEGCODE,CACHE/D100,MAXMEM/1M
```

The first compilation limits its paging area size to approximately 4.2 million words. Note that this is an upper limit, and the compiler grows to this size only if needed. Four million words is a large portion of the available memory if this compilation is performed on a non-2200/XPA-class machine. Memory quota limits could abort operations if quota limits are in effect.

The paging area size is limited to approximately 1 million words for the second compilation. If you do not use the MAX-SIZE or MAXMEM option, the machine type determines the paging area limit. The limit for a 2200/XPA system is 16 million words. The maximum paging area for a non-2200/XPA-class machine without the MAX-SIZE or MAXMEM option specified is determined as a function of the CACHE/*n* value and is about 3.5 million words for CACHE/D100 ((*n*/8)*262,144 + 262,144).

In rare situations, the compiler must expand past the limit dictated by the MAX-SIZE, MAXMEM, or CACHE options. Such an expansion is limited to one extra 262,144 expansion.

The compiler's paging area is not the only memory the UCS compilers use. There are code banks, other D-banks, and up to *n* 262,144 cache banks if a CACHE/*n* keyword option is specified for the compilation. The cache banks are not used until the main paging area is filled up and has expanded to its allowable limit. The size given by an @@CONS RC [*runid*] transparent command is a subset of all these banks. The subset is generally the total of the banks currently swapped into memory for your run. On a 2200/XPA system, this is usually all the banks that your program has used, because these machines have a large real memory.

See also the description of the CACHE/*n* keyword option in this section.

MIN-ALS

Format

```
MIN-ALS
NO-MIN-ALS
```

Default: NO-MIN-ALS

The program variables in the WORKING-STORAGE section are normally allocated in static storage in UCS COBOL. However, if the INITIAL clause is used on the PROGRAM-ID statement, then variables in the WORKING-STORAGE section are instead allocated on the Activity Local Stack (ALS). (Exception: Those 01 variables having the EXTERNAL clause are made into named COMMON blocks, which are always allocated in static storage). The ALS stack is a small bank, with a maximum size of only 229,376 words (0700000 octal). If you have large buffers in the WORKING-STORAGE section that are causing ALS overflows (the ALS might run out of space due to nested calls), you can reduce the amount of ALS used in your COBOL programs by using the MIN-ALS keyword option on your UCOB compilations. This keyword option causes the compiler to allocate those local WORKING-STORAGE variables that do not have VALUE clauses on them in static storage instead of allocating them on the ALS. If the large buffers causing the overflow problem have VALUE clauses on them, remove the VALUE clauses and add code at the beginning of your procedures to initialize them. See also the description of the Nested Procedure Calls in Appendix C.

Note: *The MIN-ALS keyword option has no effect on programs that do not have the INITIAL clause on their PROGRAM-ID statements.*

MIN-DBANK

Format

```
MIN-DBANK
NO-MIN-DBANK
```

Default: NO-MIN-DBANK

MIN-DBANK keyword option causes the compiler to reduce the amount of local static D-bank storage that is generated in the output object module. Specifically, it minimizes the number of constants and packets that are generated into the $0 location counter. The affected items are as follows:

- Parameter lists are generated on the activity-local stack (ALS) instead of in static $0 storage.
- Some addresses are created by executing extra code, that is AA,U, rather than as address constants in $0 storage.
- Code for Embedded SQL (ESQL) statements will not use writable $0 storage for packets.

Reducing the size of the $0 static storage is at the expense of additional executable code. The additional code is larger than the reduction in $0 storage. This can have a negative impact on execution time performance.

The MIN-DBANK option does not affect the location where the WORKING-STORAGE variable is allocated. WORKING-STORAGE variables by default are allocated in static local location counters, for example $0, $2, $4, and so on, in the compilation.

If your COBOL program has the INITIAL clause on the PROGRAM-ID statement, then the MIN-DBANK keyword option has no effect. The INITIAL clause results in the same code for the items listed earlier. The INITIAL clause also causes all variables in WORKING-STORAGE variable to be allocated on the ALS instead of in static local storage.

The INITIAL clause is often used when a COBOL application is to be linked into an HVTIP transaction. This clause reduces the Dbank size as the WORKING-STORAGE variable is reinitialized on each entry to a subprogram. See 3.1.3. The INITIAL clause is also used to create fixed-gate subsystems. Due to the INITIAL clause, there is little incentive to use the MIN-DBANK keyword option on UCOB compilations.

MONITOR

Format

```
MONITOR
MONITOR/LABEL
NO-MONITOR
```

Default: NO-MONITOR

MONITOR helps debug a program by writing the following information to the program print file:

- Each time execution encounters a paragraph or section label, the compiler writes the name (label) of the paragraph or section to the print file. This lets you know what part of the program is being executed. The monitor output contains:
  - The line number of the paragraph or section
  - The name of the paragraph or section
  - The identifier “PROC”
- Each time a main program or subprogram is entered, the compiler writes the entry point name to the print file. This lets you know what subprogram is currently being executed. The monitor output contains
  - The line number of the PROGRAM-ID line
  - The entry point name
  - The identifier "SUBPROGRAM-ENTRY"

  The monitor output for an alternate entry point (a label named in a WITH ENTRY POINTS clause on the PROCEDURE DIVISION statement) will NOT have a line number.

- Each time a subprogram or main program returns to its caller, the compiler again writes the subprogram's entry point name(s) to the print file. This lets you know that the subprogram is no longer in execution and that its caller is now again in execution. The monitor output contains
  - The line number of the last statement in the subprogram
  - The entry point name of the subprogram (this is a list of entry point names if the subprogram has multiple entry points)
  - The identifier "RETURN"

  Note that an EXIT PROGRAM statement executed in a main program does NOT cause the main program to stop execution or return to the operating system. You can only exit a main program by executing a STOP RUN statement or by branching to a label at the end of the program. An EXIT PROGRAM statement in a subprogram, however, does cause a return to be done and its caller to resume execution.
- Each time a STOP RUN is executed, the compiler writes a line to the print file. The monitor output contains
  - The line number of the STOP RUN statement
  - The entry point name of the subprogram (this is a list of entry point names if the subprogram has multiple entry points, and will be the string "(DECLARATIVE)" if the STOP RUN is in a DECLARATIVE)
  - The identifier "STOP RUN"
- Each time a data item is referenced, the compiler writes the contents of the data item to the print file. This lets you know the value of the data item at a particular point in program execution.

  For data references, the monitor output line is printed before the execution of the statement containing the reference. When referenced data items are possibly changed, as in arithmetic operations, moves, and so forth, monitor output is printed both before and after the execution of the statement. For each statement, the monitor output contains
  - The line number of the statement being executed
  - The name of the data item
  - The value of the data item

  If reference modification is specified for a data item, the entire data item is displayed, not just the portion specified by the reference modifier.

  If a subscripted data item is specified, the subscript is not displayed as part of the data item in the monitor output.

MONITOR/LABEL does not print data item values to the print file. Only output for subprogram entries and exits, STOP RUNs, and paragraph and section labels is put into the print file.

NO-MONITOR, which is the default, suppresses the generation of the debugging code.

Note that all MONITOR output is limited to 76 columns. If a list of entry point names causes this limit to be exceeded, the list is truncated.

MONITOR Example

The following is an example of a subprogram compiled with the MONITOR keyword option:

```
 1    IDENTIFICATION DIVISION.
 2    PROGRAM-ID. subex.
 3    DATA DIVISION.
 4    WORKING-STORAGE SECTION.
 5    01  data-item-1 pic s9(10) usage binary value 10.
 6    01  data-item-2 pic s9(10) usage binary value 20.
 7    01  data-item-3 pic s9(10) usage binary value 0.
 8    linkage section.
 9    01  res pic s9(10) usage binary.
10    01  parm1 pic s9(10) usage binary.
11     PROCEDURE DIVISION  using parm1  res.
12     ST.
13         if parm1 is equal to 22
14            move 2 to res
15            exit program.
16     PARA-1.
17         ADD data-item-1 TO data-item-2 GIVING data-item-3.
18
19         compute parm1 = parm1 + data-item-3.
20         compute res = parm1 + 1    .
```

The following is an example of the monitor output for this subprogram when called with the first parameter having a value of -10 and the second parameter with a value of 2:

```
2      SUBEX  SUBPROGRAM-ENTRY
12     ST  PROC
13     PARM1 = -0000000010
16     PARA-1  PROC
17     DATA-ITEM-1   = +0000000010
       DATA-ITEM-2   = +0000000020
       DATA-ITEM-3   = +0000000000
       DATA-ITEM-3   = +0000000030
19     PARM1 = -0000000010
       DATA-ITEM-3   = +0000000030
       PARM1 = -0000000010
       PARM1 = +0000000020
20     PARM1 = +0000000020
       RES   = +0000000002
       RES   = +0000000021
20     SUBEX  RETURN
```

Example Explanation

First, the line number (2) of the PROGRAM-ID and the name of the subprogram (SUBEX) are displayed along with the SUBPROGRAM-ENTRY identifier.

Next, the line number (12) and label (ST) are displayed for paragraph ST along with the identifier PROC. This is the first executable statement in the program.

Then the line number (13) and value of the PARM1 data item in the IF statement are displayed.

Since PARM1 is not equal to 22, the next output is the line number (16) and label (PARA-1) of the next paragraph to be executed along with the identifier PROC.

The line number (17), each data item specified (DATA-ITEM-1, DATA-ITEM-2, DATA-ITEM-3), and the current value of each specified data item in the ADD statement are displayed next.

Next, the values of any data items that are changed as a result of the ADD statement (DATA-ITEM-3) are displayed. Note that the line number is displayed only once for all data items in a given statement.

The same is done for the COMPUTE statements for lines 19 and 20. The line number and current values are given, followed by the before and after values of the data items that have their values changed.

The subprogram then returns to its caller, and the line number of the last line in the subprogram (20) is displayed along with the subprogram's name (SUBEX) and the identifier RETURN.

MCFLAG

You can dynamically control monitor output by explicitly setting and clearing a data item called MCFLAG. MCFLAG lets you selectively turn monitor output on and off during the execution of the program; you can produce monitor output for only the part of the program you are interested in.

If MCFLAG has a nonzero value, all monitor output is displayed. If MCFLAG is zero, the monitor output is suppressed. It is your responsibility to ensure that the value of MCFLAG is properly set.

You can explicitly define MCFLAG as follows:

```
01 MCFLAG PIC 9(10) BINARY.
```

If you explicitly define MCFLAG in a program containing nested subprograms, you must define it in the main program with the GLOBAL attribute in order for it to be used in the subprograms.

If MCFLAG is present and not defined as shown, UCS COBOL issues a warning and monitor output cannot be suppressed by the use of MCFLAG.

If you do not explicitly define MCFLAG, and you specify the keyword option MONITOR or MONITOR/LABEL, the compiler implicitly defines MCFLAG. The compiler gives MCFLAG an initial value of 1 (monitor output is displayed).

Effects of Using MONITOR and MONITOR/LABEL

MONITOR and MONITOR/LABEL significantly increase the size of the object module produced. If the program I-bank becomes larger than 65,535, you must use the keyword option SEGCODE. In addition, it is recommended that you use the keyword option NO-OPTIM for the compilation of large programs (greater than 10,000 lines) to speed up compilation time.

OPTIM

Format

```
OPTIM
OPTIM/GLOBAL
OPTIM/LOCAL
NO-OPTIM
```

Default: OPTIM

OPTIM, OPTIM/GLOBAL, and OPTIM/LOCAL increase the optimizations performed by LSS. These optimizations apply a series of transformations to an intermediate form of the program in an attempt to eliminate redundant operations and find the most efficient way of performing the operations that remain. Optimizations are optional for the following reasons:

- Additional overhead in compilation cost
- In the generated object code, some operations are moved away from the statements that use them, making it more difficult to use the PADS interactive debugging system. (Source code is not affected.)

An example of such an effect is moving load and store operations out of loops. If variable X is modified within a loop, it can be kept in a register throughout the loop, returning its value to storage only at the end of the loop.

In such cases, attempts to use PADS to display the value of X within the loop instead display the initial value of X and do not reflect any changes made within the loop.

OPTIM should be used only for well-tested programs in a production environment when efficiency takes precedence over debugging capabilities.

OPTIM/GLOBAL is equivalent to OPTIM.

OPTIM/LOCAL causes LSS to perform a subset of the optimizations performed for OPTIM. The compiler gathers less information and performs optimizations only on blocks of straight-line code. (For example, no optimizations are performed between code preceding an IF statement and code controlled by the IF statement.) The compiler also does not recognize loops in the code and cannot move invariant calculations out of loops, as occurs for OPTIM.

OPTIM/LOCAL has the following advantages:

- For some programs, OPTIM requires a large amount of compilation-time overhead. OPTIM/LOCAL can reduce this overhead.
- Code generated by OPTIM/LOCAL is generally more efficient than code generated by NO-OPTIM and less efficient than code generated by OPTIM and OPTIM/GLOBAL.

NO-OPTIM causes optimization to be bypassed, producing code that is less efficient but easier to debug. Therefore, it is preferable during program development when debugging takes precedence over efficiency.

The NO-OPTIM and TRANSFORM options are incompatible.

PARMCHECK

Format

```
PARMCHECK
NO-PARMCHECK
```

Default: NO-PARMCHECK

PARMCHECK generates extra code to compare the number of parameters (arguments) existing in the called procedure to the number of parameters existing in the calling procedure. It issues a warning message when the number of parameters does not match.

When the PARMCHECK option is used in conjunction with the COMPAT/ARGUMENTS option, extra code is generated to verify that each incoming parameter is word aligned. An error message is issued if a parameter is not word aligned.

The extra code produced by PARMCHECK has a negative effect on compilation and execution performance. PARMCHECK is typically used for program development.

NO-PARMCHECK suppresses parameter checking. It is typically used for production programs, after a program has been tested and debugged (using PARMCHECK).

REL18

Format

```
REL18
NO-REL18
```

Default: NO-REL18

REL18 causes the code location counters in a compiled object module to be marked as having *code offset 18 bit* relocation, a relocation type introduced in system release 6.0. NO-REL18 causes the code location counters to be marked as having *code offset 16 bit* relocation. The relocation type indicates how the linker handles the relocation of JUMP instructions in the code.

The UCS compilers generate full 18-bit JUMPs, which means that a code location counter can be up to 262,144 words in size. When the REL18 option is used, the compilers can generate the more efficient bit-map relocation for the JUMPs. When the REL18 option is **not** used, the compilers must use a more general relocation mechanism called PCODE to do JUMP relocation since the *code offset 16 bit* mechanism can only handle 16-bit JUMPs. The PCODE mechanism takes more internal space in the object module, and a very large static link could even conceivably overflow the PCODE table in the resulting ZOOM or object module. Therefore, using the REL18 option can be more efficient, and may also prevent the overflow of internal tables within a large statically linked object module.

Another benefit of using REL18 is that the static linker will merge the code banks to be after the code banks that result from MASM. (MASM code banks have a relocation type of code offset 16 bit, and generally cannot be linked at over 65,535 addresses.) This means that MASM object modules from the URTS Service Library (or user libraries) will be merged first, and should then not get the truncation errors that occur when they are linked at over 65,535. (HVTIP static links often have this problem.)

Older linkers (prior to Link level 10R1) cannot process an object module having the code offset 18-bit relocation type, and for this reason, REL18 is currently not the default. You should **not** use the REL18 keyword option if the resulting object module may be used on a system with a Linking system earlier than level 10R1 (system release 6.0).

REORDER

Format

```
REORDER
NO-REORDER
```

Default: REORDER on CMOS systems
NO-REORDER on SAIL-based systems (EXTENDED/8)

REORDER causes instruction reordering to be performed. Reordering improves program performance by rearranging instructions to better utilize the hardware.

NO-REORDER suppresses instruction reordering.

When NO-OPTIM is specified, no instruction reordering is performed unless REORDER is specified.

RUNCHECK

Format

```
RUNCHECK
NO-RUNCHECK
```

Default: NO-RUNCHECK

RUNCHECK generates extra code to test for the following error conditions and terminates program execution when an error condition exists:

- Range checking for array element references
- Range checking for reference modifications

NO-RUNCHECK suppresses error testing.

Specifying RUNCHECK with OPTIM causes a warning message because the restructuring of the program and elimination of statements by optimization can suppress the issuing of certain messages. A program compiled with NO-RUNCHECK usually runs faster and contains less object code than the same program compiled with RUNCHECK. However, it does not contain the advanced error detection mechanisms of RUNCHECK and should be used only in a production environment with a well-tested program.

SEGCODE

Format

```
SEGCODE
NO-SEGCODE
```

Default: NO-SEGCODE

By default (NO-SEGCODE), the compiler generates one instruction bank (I-bank) for a compilation unit. The size of an I-bank produced by the compiler can be up to 262,143 words. If you are using EXTENDED/0, EXTENDED/2, or EXTENDED/4 options to target your code for a non-XPA system, the size of an I-bank produced by the compiler can be up to 65,535 words.

Non-XPA systems include all systems released before the 2200/900 such as the 2200/600 and 2200/400. XPA systems include the 2200/900, 2200/500 and all subsequent systems. Be aware that Unisys no longer supports non-XPA systems.

If you reach these limits, you can split the program into smaller, separate program units or use the SEGCODE option. The SEGCODE option causes the compiler to generate separate I-banks for each program unit group and selected program segments, up to a maximum of 99 I-banks. If you compile more than 99 program units or program segments, the compiler cycles back to the first I-bank and combines the generated code for additional program units or program segments into existing I-banks.

The size of each I-bank produced by the compiler when the SEGCODE option is present can be up to 262,143 words, or 65,535 words if the EXTENDED/0, EXTENDED/2, or EXTENDED/4 keyword options are used.

If you exceed these limits or the compiler is unable to logically divide a program into a code segment that is less than the limit, the compilation terminates in error. You must then break the compilation units into smaller units and recompile.

You can use the Linking System to merge banks. The size of a merged l-bank produced by the Linking System can be 262,143 words when merging l-banks that were compiled with the SEGCODE option. A special LINK directive, SET MERGED_CODE_UL = 262143 is required when merging l-banks over 65,535 words.

See the *Linking System Programming Reference Manual* for more information about merging banks.

SHIFT

Format

```
SHIFT
NO-SHIFT
```

Default: NO-SHIFT

SHIFT allows kanji shift code images to be accepted as part of the source input. This option is required for programs having kanji symbolic names, kanji literals, or literals without the prefix cc.

NO-SHIFT specifies that kanji shift code images are not part of the source input.

TIPMALLOC

Formats

```
TIPMALLOC
NO-TIPMALLOC
```

Default: NO-TIPMALLOC

If needed, this option should only be used when compiling the main program.

The primary purpose of the TIPMALLOC option is to prevent URTS$TABLES from expanding to its maximum upper limit in a transaction (TIP) environment, which will cause the transaction to create another URTS$TABLES bank and therefore lose its TIP sticking.

This option should be used when a TIP transaction requires more than approximately 164,000 words of UC memory allocation. Note that the memory allocation request can come from the transaction or any subsystem that the transaction calls.

This option should not be used if the program is going to execute in an HVTIP environment since other means are already in place to handle memory allocation (see the Pooled Banks clause of the OUTPUT HVTIP2 link statement in the *Linking System Programming Reference Manual*).

Batch and demand runs acquire activity level banks dynamically as they are needed unless the TIPMALLOC option is specified. The UC malloc, calloc, or realloc functions obtain memory allocation from the TIPMALLOC$$$ bank. The TIPMALLOC$$$ bank is set to a maximum upper limit of 16,777,216 words with the current upper limit set to 262,144 words. If memory allocation exceeds 262,144 words and you want to preserve TIP sticking, you must expand the current upper limit of the bank by adding the SET SIZE statement to your LINK after the RESOLVE statement. The form of the statement is SET SIZE = SIZE + *xxxxx* FOR TIPMALLOC$$$ (where *xxxxx* is the size you wish to expand the bank beyond 262,144 words).

If the TIPMALLOC$ bank fills up to its limit of 16,777,216 words, the runtime system will allocate more banks as needed and allocate memory from them. NO-TIPMALLOC causes the UC malloc, calloc, or realloc memory allocation functions to acquire space from URTS$TABLES in a transaction (TIP) environment. The URTS$TABLES bank is a small bank and cannot be expanded above its maximum upper limit which is 262,144 words. Once it fills up, another URTS$TABLES bank will be created. and TIP sticking will be lost.

TRANSFORM

Format

```
TRANSFORM
NO-TRANSFORM
```

Default: NO-TRANSFORM

TRANSFORM performs some transformations that cause execution-time improvements but can also cause precision differences. For example, some division operations are transformed into multiplication with different results due to approximation of floating-point numbers.

NO-TRANSFORM produces program results that are the same for both OPTIM and NO-OPTIM compilations.

The TRANSFORM and NO-OPTIM options are incompatible.

UREP-XREF

Format

```
UREP-XREF
NO-UREP-XREF
```

Default: NO-UREP-XREF

UREP-XREF causes cross-reference information to be stored into UREP. This information includes the relationship between the source program and the file names and data names that the program introduces with INVOKE or COPY statements. The values in the UREP entity UCS OPTIONS determine the type of cross-reference information that is stored. See Section 8 for more information.

Use this option with programs that use the following:

- INVOKE statements
- COPY statements
- Embedded SQL statements

The keyword option APPLICATION specifies into which repository UREP-XREF stores cross-reference information for the UREP-XREF option.

If you specify NO-UREP-XREF, cross-reference information is not stored into UREP.

URTS$TABLES

Format

```
URTS$TABLES/n
```

Default: URTS$TABLES/0

URTS$TABLES/*n* can be used to reserve an area in the URTS$TABLES heap bank for those things that must be in the same bank (I/O and SORT are the primary users of the same bank request). *n* is an octal number indicating the size (in words) of the reserved space. The default value of *n* is zero, the maximum value is 0500000, and the recommended value is 050000. If runtime error 37001 (attempt to expand URTS$TABLES beyond its maximum upper limit - all basic mode addressable space has been used) persists, try increasing the reserve in increments of 010000 until the error goes away. Note that the URTS$TABLES option is only effective when used on the compilation of the main program for your executable.

URTS$TABLES is a small bank, and beginning with URTS level 12R1, once the bank is filled, another URTS$TABLES heap bank is created. This results in an open-ended size for this heavily used space. Several internal data structures are allocated in URTS$TABLES, some of which must be in the same bank, and some of which can be in different banks. Applications that use several concurrently opened files, multitasking, or sorts, may need more than one URTS$TABLES heap bank due to their aggregate size requirements. If your application requires more than one URTS$TABLES heap bank, this could result in a situation where everything that must be in the same bank does not fit in the same URTS$TABLES bank. If the memory manager cannot acquire the space from the URTS$TABLES bank that is required, the fatal runtime error 37001 (attempt to expand URTS$TABLES beyond its maximum upper limit - all basic mode addressable space has been used) is issued. This means you must use the URTS$TABLES option to reserve some “same-bank” space for these requests.

Note: *Once a second bank is dynamically acquired, it prevents Checkpoint/Restart from working and inhibits TIP sticking for transactions. The default reserve area in URTS$TABLES is set to zero to prevent applications from suddenly needing a second URTS$TABLES heap bank due to the reserve area. For HVTIP, if a large enough pooled bank is available, it is acquired rather than creating a new bank; therefore sticking is not inhibited.*

XML/*name*

Format

XML/*name*
NO-XML

Default: NO-XML

The XML/*name* keyword option invokes the COBOL to XML (Extensible Markup Language) generation tool. The primary purpose of the COBOL to XML generation tool is to produce an XML description of a COBOL record layout (sometimes referred to as an "01 record layout"). The XML/name option causes the compiler to create an XML document that describes the layout of one or more data records that are defined in the program.

In order to use the XML/name option, a text file must first be created that contains one or more 01 data record names that are defined in the COBOL program. This file must be placed into the CIFS current working directory (cwd). The name of this file is specified in the name portion of the XML/name option. There is one data record per line in this text file. In addition to the data record name, each line may contain the CIFS location and name of where the resulting XML file (.xsd) will be placed. See Section 18 XML Generator for additional details of how to create an XML file. An example of the contents of this file is as follows:

```
cobrec1
cobrec2 /os2200/bek/xmlwork/xyz
cobrec3 xmlfile/abc
```

In this example, .xsd files for the COBOL record names cobrec1, cobrec2 and cobrec3 will be generated. Since no CIFS path was specified for cobrec1, by default, the file cobrec1.xsd will be placed into the CIFS current working directory. The record name cobrec2 has an absolute CIFS path associated with it so, the file xyz.xsd will be created in the CIFS path /os2200/bek/xmlwork and will contain the information for the COBOL record cobrec2. The record name cobrec3 has a relative CIFS path associated with it, so the file abc.xsd will be created and placed into the CIFS current working directory in the path xmlfile.

If the file specified above was given the name *recordnames* and placed into the CIFS current working directory, it can be used in the compilation as follows:

@ucob qual*file.cobolprog,om,,,XML/*recordnames*

The compiler will find the *recordnames* file in the CIFS current working directory and will create the three .xsd files in the specified CIFS locations.

You must ensure that the CIFS cwd of the run doing the compilations is not at the OS 2200 qualifier level. The .xsd extension is applied to the XML files that are created and extensions cannot be used on OS 2200 file names. The CIFS cwd should be set to either an OS 2200 program file or the CIFS directory created by CIFSUT. The default CIFS cwd is the qualifier for the run and that will not allow for the generation of an .xsd XML file.

For additional information about creating XML files, see Section 18 XML File Generation.

2.2.8. Compiler Call Keyword Options for Diagnostic Message Control

Table 2–8 summarizes the diagnostic message control keyword options.

Table 2–8. Diagnostic Message Control Compiler Keyword Options

Keyword Option	Description
IR-DIFF	Determines whether messages inform you when the results of UCS COBOL calculations may differ from those of ASCII COBOL.
LEVEL *options*	Issue messages for nonstandard or obsolete features.
LEVEL/RPW	Issues messages for syntax that belong to the optional Report Writer module.
LEVEL/SEG	Issues messages for syntax that belong to the optional Segmentation module.
LEVEL/SEG2	Issues messages for syntax that belong to level 2 of the Segmentation module.
LEVEL/SQL	Issues warning messages for nonstandard syntax or host-variable references within an embedded SQL command.
REMARK	Determines whether REMARK messages are issued.
WARNING	Determines whether some subclasses of WARNING messages are issued.

IR-DIFF

Format

```
IR-DIFF
NO-IR-DIFF
```

Default: NO-IR-DIFF

IR-DIFF issues remark messages each time the intermediate result in an arithmetic expression assumes maximum precision. Calculations on the intermediate result are then performed in floating point. Thus, the higher accuracy of UCS COBOL produces results that may differ from the results of the same calculation in ASCII COBOL. IR-DIFF also sets the REMARK option.

NO-IRR-DIFF suppresses remark messages for results that may differ when using ASCII COBOL.

LEVEL/options

Format

```
LEVEL/STANDARD
NO-LEVEL
LEVEL
LEVEL/LOW
LEVEL/INTERMEDIATE
LEVEL/HIGH
LEVEL/OBSOLETE
```

Default: NO-LEVEL

LEVEL/STANDARD issues warning messages for nonstandard or obsolete features used within the program.

NO-LEVEL suppresses level checking and warning messages.

LEVEL issues warning messages for nonstandard or obsolete features.

LEVEL/LOW issues warning messages for nonstandard, obsolete, intermediate, high, and standard features.

LEVEL/INTERMEDIATE issues warning messages for nonstandard, obsolete, high, and standard features.

LEVEL/HIGH issues warning messages for nonstandard, obsolete, and standard features.

LEVEL/OBSOLETE issues warning messages for obsolete features.

The option STD-ERRCHECK sets LEVEL/STANDARD. Otherwise, the default is NO-LEVEL.

LEVEL/RPW

Format

```
LEVEL/RPW
```

Default: NO-LEVEL

LEVEL/RPW issues warning messages for syntax that belongs to the Report Writer module. It is separate from the other LEVEL options and can be used in addition to these options. The LEVEL/RPW option does not cancel out other LEVEL options.

LEVEL/SEG

Format

```
LEVEL/SEG
```

Default: NO-LEVEL

LEVEL/SEG issues warning messages for syntax that belongs to the Segmentation module. It is separate from the other LEVEL options and can be used in addition to these options. The LEVEL/SEG option does not cancel out other LEVEL options.

LEVEL/SEG2

Format

```
LEVEL/SEG2
```

Default: NO-LEVEL

LEVEL/SEG2 issues warning messages for syntax that is above level 1 of the Segmentation module. It is separate from the other LEVEL options and can be used in addition to these options. The LEVEL/SEG2 option does not cancel out other LEVEL options.

LEVEL/SQL

Format

```
LEVEL/SQL
```

Default: LEVEL/SQL

LEVEL/SQL issues warning messages for nonstandard syntax or host-variable references within an embedded SQL command. It is separate from the other LEVEL options and can be used in addition to these options. The LEVEL/SQL option does not cancel out other LEVEL commands.

REMARK

Format

```
REMARK
NO-REMARK
```

Default: NO-REMARK

REMARK lists messages of class REMARK.

NO-REMARK does not list REMARK messages.

The E, L, and S letter options set REMARK.

For further information about EXPLAIN statement output, see subsection 7.9 in this manual and the description of EXPLAIN in the *SQL Programming Reference Manual*.

WARNING

Format

```
WARNING
NO-WARNING
```

Default: WARNING

WARNING lists all compiler warning messages of subclass USAGE, ARCHAIC USAGE, or other compiler-specified subclasses.

NO-WARNING suppresses warning diagnostics except those requested by a leveling (LEVEL/) option.

2.3. Composition of a Compiler Listing

When you compile your program, you receive a listing describing the compilation. This listing is sent to your terminal or print file, depending on your execution mode, for example, demand mode, batch mode, or demand mode controlled by a @BRKPT statement.

A compiler listing is actually composed of many separate listings. Using compiler keyword options or letter options, you can control which listings are produced and how detailed the listings are. By default, the sign-on line, bank sizes line, and sign-off line are produced. (The options listing is produced by default if you are executing in batch mode or demand mode controlled by a @BRKPT statement.)

The following subsections describe the components of a compiler listing. Table 2–9 lists each component, the keyword option that creates it (if applicable), and the subsection you can reference for its description.

Table 2–9. Composition of a Compiler Listing

Listing Component	Keyword Option	Reference
Sign-on line	SIGNON	2.3.1
Options Listing	OPTIONS	2.3.2
Diagnostic messages	WARNING, REMARK, LEVEL (error messages always generated)	2.3.3
Source Listing	SOURCE	2.3.4
Tracecopy Listing	TRACECOPY	2.3.5
Allocation Listing	ALLOC	2.3.6
Cross-Reference Listing	XREF, SHORT-REF	2.3.7

Table 2–9. Composition of a Compiler Listing

Listing Component	Keyword Option	2.3.8
Object Code Listing	OBJECT, OBJECT-ONLY, OBJ-PKT	2.38
Linker Information Listing	LINKINFO	2.3.9
Bank sizes line	SIGNON	2.3.10
Sign-off line	SIGNON	2.3.11

The compiler listing is 79 characters wide when you specify the NARROW keyword option and 132 characters wide when you specify the WIDE keyword option (see Table 2–6).

Several listings generated by the compiler (for example, the Allocation Listing, Cross-Reference Listing, and Object Code Listing) have headings generated on each page when you generate the listings in breakpoint (@BRKPT) mode. The compiler uses the system print file defaults to determine where to insert the headings in the listings. If the print file is to be printed on a printer that has a page size different from the system default page size, you can override these defaults through the use of the @HDG command. For example:

```
@HDG    heading-text.M,length,top,bottom,lpi
```

For information on the @HDG statement, see the *ECL End Use and Programming Reference Manual.*

2.3.1. Sign-On Line

The system prints the sign-on line at the beginning of each compiler listing, immediately after the compiler call. It is produced by the SIGNON keyword option. (SIGNON is the default.)

Example

```
UCOB- 12R2 / EM (131127) LSS- 14R4 / EM (131126) 2014 Jul 01 1139:13
```

Format

```
UCOB- ssRii/genid1 (yymmdd) LSS- lssRii/genid2 (yymmdd) yyyy mmm dd hhmm:ss
```

where:

UCOB-

identifies the UCS COBOL compiler that performs the compilation.

*ss*R*ii*

is the level number of the compiler, where:

- *ss* is the symbolic update number.
- *ii* is the incremental update number. A letter after *ii* can be included to indicate minor modifications to the release level.

/genid1

is the COMUS generation identifier, up to 12 characters long. It is present only if a COMUS generation of UCS COBOL has been done at your site.

(yymmdd)

is the year, month, and day UCS COBOL was created by the development center or generated by the user site.

LSS-

stands for the Language Support System. It is the back-end code generator that creates the object module.

*lss*R*ii*

is the level number of LSS, where:

- *lss* is the symbolic update number.
- *ii* is the incremental update number. A letter after *ii* can be included to indicate minor modifications to the release level.

/genid2

is the COMUS generation identifier, up to 12 characters long. It is present only if a COMUS generation of LSS has been done at your site.

(yymmdd)

is the year, month, and day LSS was created by the development center or generated by the user site.

yyyy mmm dd

is the year, month, and day the compilation occurs.

hhmm:ss

is the hour, minute, and second the compilation begins.

2.3.2. Options Listing

The Options Listing follows the sign-on line. It is created by the OPTIONS keyword option. The Options Listing displays the compiler keyword options in effect for this compilation.

Example 2–1 shows an Options Listing with the NARROW option in effect.

```
OPTIONS: NO-ALLOC, APPLICATION/UDSSRC, NO-AUXPROLOG, CACHE/D0,
NO-CALIGN, NO-CLEAR, CODE, NO-CODE-LVE, NO-COMPAT, NO-COMP-BIN,
DEBUG/WALKBACK, DOUBLEQUOTE, SINGLESPACE, ERRCHECK, NO-ERREXIT,
EXTENDED, NO-I18N-IO, NO-IR-DIFF, NO-LEVEL, NO-LINKINFO, NO-LISTCOPY,
NO-LITERAL, MAIN-PROGRAM, MAXERRORS/100000, NO-MONITOR, MULTI-PF,
NO-OBJECT, NO-OBJ-PKT, OPTIM, OPTIONS, NO-PARMCHECK, NO-REMARK,
REORDER, NO-RUNCHECK, SEARCH-PF-SI, NO-SEGCODE, NO-SHIFT, SIGNON,
NO-SOURCE, NO-TRACECOPY, NO-TRACEMSG, NO-TRANSFORM, NO-UREP-XREF,
WARNING, NARROW, NO-XREF
```

Example 2–1. Options Listing (NARROW)

2.3.3. Diagnostic Messages

The compiler may print diagnostic messages informing you of errors or other noteworthy conditions relating to your program or its compilation. The following types of messages exist:

- Error messages
- Warning messages (includes level messages indicating conformance to language standards)
- Remark messages

The compiler displays messages according to the following rules:

- The compiler always displays error messages; there is no keyword or letter option to suppress printing of error messages.
- The compiler displays warning messages if you use the keyword option WARNING (which is the default value) or a LEVEL keyword option.
- The compiler displays remark messages if you use the keyword option REMARK.

The letter option E also displays warning and remark messages.

The location of diagnostic messages depends on whether or not you have specified production of the Source Listing (SOURCE keyword option or S letter option):

- If you have specified production of the Source Listing, messages appear in the Source Listing, on a line preceding the source line to which they apply.
- If you have not specified production of the Source Listing, messages appear following the Options Listing. (The source line causing the error is also printed.)

See Appendix A and Appendix B for more information on diagnostic messages, including

- Definitions of message types
- Information on message subclasses (major, minor, internal, and so forth)
- A list of all messages and, where appropriate, descriptions of particular messages

2.3.4. Source Listing

The Source Listing primarily displays your program's source statements. It is produced by the SOURCE keyword option. (NO-SOURCE is the default.) When the WIDE keyword option is in effect, the Source Listing consists of the following fields:

- Line number
- Procedure nesting level number
- Source lines

When the NARROW keyword option is in effect, the Source Listing consists of the following fields:

- Line number
- Source lines

Fields of the Source Listing

Line number

A line number appears in column 1 for each line of source text. If the source input element contains CTS or IPF 1100 line numbers, these line numbers appear. Otherwise, the compiler uses sequential line numbers (1, 2, 3...).

If the number is too large to fit in the allotted space, it is printed on a separate line preceding the source line.

Procedure nesting level number

This field helps you recognize different levels of subprograms. The main program is always level 1. This field appears on the line for the Identification Division and all lines in the Procedure Division.

Source lines

This field displays your program's source statements. If a source line exceeds the length of an output line, the compiler prints the remainder of the source line on the next line.

If something in your program is incorrect or noteworthy, the compiler issues an error, warning, or remark message informing you (see 2.3.3). If the Source Listing is created, such messages appear in the source lines field.

Examples

Example 2–2 shows a Source Listing with the WIDE option in effect.

```
1                     IDENTIFICATION DIVISION.
2                     PROGRAM-ID. PROGRAM-EXAMPLE.
3                    *
4                    * THIS PROGRAM READS IMAGES FROM THE CARD-READER
5                    * AND PRINTS THEM ON THE PRINTER.
6                    *
7                     ENVIRONMENT DIVISION.
8                     CONFIGURATION SECTION.
9                     SOURCE-COMPUTER. UNISYS-2200.
10                    OBJECT-COMPUTER. UNISYS-2200.
11                    SPECIAL-NAMES.
12                       PRINTER IS PRT.
13                    INPUT-OUTPUT SECTION.
14                    FILE-CONTROL.
15                       SELECT CARD-FILE ASSIGN TO CARD-READER.
16                       SELECT PRINT-FILE ASSIGN TO PRINTER.
17                    DATA DIVISION.
18                    FILE SECTION.
19                    FD  CARD-FILE
20                       LABEL RECORD OMITTED
21                       DATA RECORD IS IN-REC.
22                    01  IN-REC.
23                       05  CARD-IMAGE PIC X(80).
24                    FD  PRINT-FILE
25                    LABEL RECORD OMITTED
26                       DATA RECORD IS PRT-REC.
27                    01  PRT-REC.
28                       05  PRINT-IMAGE PIC X(132).
29                    WORKING-STORAGE SECTION.
30                    01  GROUP-DATA.
31                       05  T-TEST-VALUE PIC 9(5) USAGE BINARY.
32                       05  P-TEST-VALUE COMPUTATIONAL-2 VALUE +1.2825.
33                       05  R-TEST-VALUE PIC X(5) VALUE "PRINT".
34                       05  X-TEST-VALUE PIC 99 USAGE COMP VALUE 2.
35                       05  Z-TEST-VALUE PIC 99 USAGE PACKED-DECIMAL.
36              1     PROCEDURE DIVISION.
37              1     INIT-SECTION.
38              1     BEGIN.
39              1        OPEN INPUT CARD-FILE.
40              1        OPEN OUTPUT PRINT-FILE.
41              1     PRINT-DETAIL-LINES.
42              1        READ CARD-FILE AT END GO TO ALL-DONE.
43              1        MOVE CARD-IMAGE TO PRINT-IMAGE.
44              1        WRITE PRT-REC AFTER ADVANCING X-TEST-VALUE LINES.
45              1        GO TO PRINT-DETAIL-LINES.
46              1     ALL-DONE.
```

```
47              1          CLOSE CARD-FILE, PRINT-FILE.
48              1          STOP RUN.
```

Example 2–2. Source Listing (WIDE)

Example 2–3 shows a Source Listing with the NARROW option in effect.

```
1              IDENTIFICATION DIVISION.
2              PROGRAM-ID. PROGRAM-EXAMPLE.
3             *
4             * THIS PROGRAM READS IMAGES FROM THE CARD-READER
5             * AND PRINTS THEM ON THE PRINTER.
6             *
7              ENVIRONMENT DIVISION.
8              CONFIGURATION SECTION.
9              SOURCE-COMPUTER. UNISYS-2200.
10             OBJECT-COMPUTER. UNISYS-2200.
11             SPECIAL-NAMES.
12                 PRINTER IS PRT.
13             INPUT-OUTPUT SECTION.
14             FILE-CONTROL.
15                 SELECT CARD-FILE ASSIGN TO CARD-READER.
16                 SELECT PRINT-FILE ASSIGN TO PRINTER.
17             DATA DIVISION.
18             FILE SECTION.
19             FD  CARD-FILE
20                 LABEL RECORD OMITTED
21                 DATA RECORD IS IN-REC.
22             01  IN-REC.
23                 05  CARD-IMAGE PIC X(80).
24             FD  PRINT-FILE
25                 LABEL RECORD OMITTED
26                 DATA RECORD IS PRT-REC.
27             01  PRT-REC.
28                 05  PRINT-IMAGE PIC X(132).
29             WORKING-STORAGE SECTION.
30             01  GROUP-DATA.
31                 05  T-TEST-VALUE PIC 9(5) USAGE BINARY.
32                 05  P-TEST-VALUE COMPUTATIONAL-2 VALUE +1.2825.
33                 05  R-TEST-VALUE PIC X(5) VALUE "PRINT".
34                  05  X-TEST-VALUE PIC 99 USAGE COMP VALUE 2.
35                 05  Z-TEST-VALUE PIC 99 USAGE PACKED-DECIMAL.
36             PROCEDURE DIVISION.
37             INIT-SECTION.
38             BEGIN.
39                 OPEN INPUT CARD-FILE.
40                 OPEN OUTPUT PRINT-FILE.
41             PRINT-DETAIL-LINES.
42                 READ CARD-FILE AT END GO TO ALL-DONE.
43                 MOVE CARD-IMAGE TO PRINT-IMAGE.
44                 WRITE PRT-REC AFTER ADVANCING X-TEST-VALUE LINES.
45                 GO TO PRINT-DETAIL-LINES.
```

```
46            ALL-DONE.
47                CLOSE CARD-FILE, PRINT-FILE.
48                STOP RUN.
```

Example 2–3. Source Listing (NARROW)

2.3.5. Tracecopy Listing

The Tracecopy Listing lists the locations of COPY files found during a compilation. The location of the Tracecopy Listing line depends on whether or not you have specified production of the Source Listing with either the SOURCE keyword option or the S letter option.

If you have specified production of the Source Listing, each Tracecopy Listing line appears in the Source Listing on a line preceding the source line to which it applies. If you have not specified production of the Source Listing, Tracecopy Listing lines appear following the Options Listing.

Format

*qual*file.elt* (*proc-name*)

where:

*qual*file.elt*

identifies the name of the element that contains a PROC specified by the COPY statement.

proc-name

is the name of the PROC specified by the COPY statement.

Example 2–4 shows a Tracecopy Listing when the source code is being produced.

```
1             IDENTIFICATION DIVISION.
2             PROGRAM-ID. EXAMPLE-PROGRAM.
3             PROCEDURE DIVISION.
4             PP.


QUAL*FILE.PROC (MYCOBPROC)
5                 COPY MYCOBPROC.
5.C1.2       * This comment is brought into the program via a COPY
5.C1.3       * statement.  The file from which it came is listed above
5.C1.4       * as the result of using the keyword option TRACECOPY.
6                 STOP RUN.
```

Example 2–4. Tracecopy Listing

2.3.6. Allocation Listing

The Allocation Listing gives storage information about

- Variables
- Labels

The listing is composed of machine-dependent information. Items described on the listing are presented according to the order of their allocation. Within structures, subitems are presented according to the order in which they occur in the source input. This ordering scheme is most useful for interpreting dumps.

The compiler generates a single listing for all program units. This listing shows only variables for which the compiler has actually allocated storage.

The Allocation Listing is produced by the ALLOC keyword options. (NO-ALLOC is the default.) It consists of the following fields:

- NAME
- DECLARED
- ST/LC
- OFFSET
- SB
- SIZE
- MACH TYPE
- DM
- SPAN SIZE
- LV
- ATTRIBUTES

Fields of the Allocation Listing

NAME

This field contains the name of the item defined in the source program. If the name is longer than the allotted space, it appears alone on the line; the remainder of the information is on the next line.

DECLARED

This field contains the line number of the definition of the data item. If the line number extends past the end of the field, the next field begins in the appropriate position on the next line.

ST/LC

This field contains one of the following for the item:

- Storage type

  If the item's storage is dynamically allocated (created at run time), the listing specifies the type of storage containing the item. Possible values are

  – BASED (dynamically allocated storage)

  – AUTO (automatically allocated storage)

  – TEMP (temporary storage)

  – PARM (the item is a parameter for a call)

  – RETURN (the item is a return value from a call)

- Logical bank number (location counter)

  If the item's storage is statically allocated (created at compilation), the listing specifies the logical bank number containing the item.

OFFSET

This field identifies where the item is located within the logical bank or dynamically-allocated storage described in the ST/LC field. This field contains the relative word offset (in octal) of the item within the location counter or dynamically-allocated storage.

SB

SB stands for "starting bit." This field identifies the number of the bit where the item's storage begins in the word indicated by the OFFSET field. Numbers are decimal values and are numbered left to right within a word, beginning with bit zero.

SIZE

This field contains the item's storage size in bits (BIT), bytes (BYT), or words (WRD). For arrays, only the size of an individual element appears.

MACH TYPE

This field contains an abbreviation of the item's machine type. The machine type reflects the internal storage representation of the data. Meanings for most abbreviations are self explanatory, for example:

- CHR_STR means character string.
- CMN_BLK means common block.
- BIN_S means binary signed.
- BFLOT_S means floating binary signed.

DM

This field identifies which dimension of an array is being described in the SPAN SIZE field. A 1 means the first dimension is being described; a 2 means the second, and so forth.

Each dimension is described on a separate line. If the item is not an array, this field is blank.

SPAN SIZE

This field contains the amount of internal storage space from the beginning of one occurrence of an item to the next occurrence, for the array dimension specified in the DM field. That is, it tells how far away in storage is the next array cell, if you increment the dimension (specified in the DM field) by one.

Size is described in one of the following ways:

- Number of bits (BIT), bytes (BYT), or words (WRD)
- EXPR (meaning the size is determined by an expression)
- UNKNOWN (meaning the compiler cannot determine the size)

Filler bits are included in the size. If the item is not an array, this field is blank.

LV

This field contains the logical structure level of a data item:

- For the most inclusive group items, this field contains a 1. The next most inclusive group (or elementary) items receive a 2 in this field, and so forth.
- For level 01 elementary items or level 77 items, this field contains SC (meaning scalar).

ATTRIBUTES

This field contains declaration information specific to COBOL, such as the procedure name and data attributes as expressed by the COBOL language.

If the information does not fit in this field, it is printed in the same columns of the next line. This field appears only when the WIDE keyword option is in effect.

Example

Example 2–5 shows an Allocation Listing with the NARROW keyword option in effect.

```
A L L O C A T I O N    L I S T I N G

NAME      DECLARED  ST/LC   OFFSET SB      SIZE MACH    DM    SPAN LV
                                              TYPE
SIZE
GROUP-DATA
             30         0 00000005  0     19BYT CHR_STR        1
T-TEST-VALUE
             31         0 00000005  0      2BYT BIN_S          2
P-TEST-VALUE
             32         0 00000005 18      2WRD BFLOT_S        2
R-TEST-VALUE
             33         0 00000007 18      5BYT CHR_STR        2
X-TEST-VALUE
             34         0 00000010 27      2BYT CNUM_U         2
Z-TEST-VALUE
             35         0 00000011  9      2BYT DEC54_U        2
CARD-FILE    19         0 00000012  0      23WRD FILE          2
PRINT-FILE
             24         0 00000062  0      23WRD FILE          2
IN-REC       22    BASED  00000000  0      20WRD CHR_STR       1
CARD-IMAGE
             23    BASED  00000000  0      20WRD CHR_STR       2
PRT-REC      27    BASED  00000000  0      33WRD CHR_STR       1
PRINT-IMAGE
             28    BASED  00000000  0      33WRD CHR_STR       2
PROGRAM-EXAMPLE
             2          1 00001024  0            ENTRY        SC
INIT-SECTION
             37         1 00001045  0            LABEL        SC
BEGIN        38         1 00001045  0            LABEL        SC
PRINT-DETAIL-LINES
             41         1 00001075  0            LABEL        SC
ALL-DONE     46         1 00001124  0            LABEL        SC
```

Example 2–5. Allocation Listing (NARROW)

If the NARROW keyword option is in effect, the print line has the following differences:

- If the name is longer than the allotted space, it appears alone on the line; the remainder of the information is on the next line.
- If the line number in the DECLARED field extends past the end of the field, the next field begins in the appropriate position on the next line.
- The ATTRIBUTES field appears if the WIDE keyword option is in effect, but does not appear if the NARROW keyword is in effect.

2.3.7. Cross-Reference Listing

The Cross-Reference Listing lets you examine

- Where identifiers are referenced in your program
- The nature of the storage containing the identifiers

It displays all references in your source program to the following items:

- Statement labels
- Subprograms
- Data items

The Cross-Reference Listing lists them alphabetically. The compiler generates a single Cross-Reference Listing for all program units.

The Cross-Reference Listing is produced by the XREF or SHORT-XREF keyword options. (NO-XREF is the default.) It consists of the following fields (which appear with both the NARROW and WIDE keyword options):

- NAME
- ST/LC
- OFFSET
- SB
- SIZE
- ATTRIBUTES

Fields of the Cross-Reference Listing

NAME

This field contains the name of the item being cross-referenced. If the name is longer than the allotted space, it appears alone on the line; the remainder of the information is on the next line.

Printed beneath the name are the following:

- DCL

  DCL stands for "declared." This entry indicates line numbers in the program where the item is declared.

- SET

  This entry indicates line numbers where there is a reference that initializes or changes the contents of the item.

  A label is marked SET when it is used to prefix a statement.

- USED

  This entry indicates line numbers where there is a reference that does not initialize or change the contents of the item.

ST/LC

This field contains one of the following for the item:

- Storage type

  If the item's storage is dynamically allocated (created at run time), the listing specifies the type of storage containing the item. Possible values are

  – BASED (dynamically allocated storage)
  – AUTO (automatically allocated storage)
  – TEMP (temporary storage)
  – PARM (the item is a parameter for a call)
  – RETURN (the item is a return value from a call)

- Logical bank number (location counter)

  If the item's storage is statically allocated (created at compilation), the listing specifies the logical bank number containing the item.

OFFSET

This field identifies where the item is located within the logical bank or dynamically-allocated storage described in the ST/LC field. This field contains the relative word offset (in octal) of the item within the location counter or dynamically-allocated storage.

SB

SB stands for "starting bit." This field identifies the number of the bit where the item's storage begins in the word indicated by the OFFSET field. Numbers are

- Decimal values
- Numbered left to right within a word, beginning with bit zero

SIZE

This field contains the item's storage size in bits (BIT), bytes (BYT), or words (WRD). For arrays, only the size of an individual element appears.

ATTRIBUTES

This field contains declaration information.

If the information exceeds the length of the line, the excess is printed on the next line in the same columns.

The same Cross-Reference Listing fields appear with the NARROW and WIDE keyword options. The NARROW and WIDE keyword options use the same rules for overflow of each field. Some fields are narrower with NARROW, however. The DCL, SET, and USED line numbers are displayed in the same manner for both keyword options.

Examples

Example 2–6 shows a Cross-Reference Listing with the NARROW keyword option in effect.

```
C R O S S   R E F E R E N C E   L I S T I N G

NAME          ST/LC   OFFSET SB         SIZE   ATTRIBUTE
ALL-DONE          1 00001124  0                PROCEDURE NAME
     DCL  46
     USED 42
BEGIN             1 00001045  0                PROCEDURE NAME
     DCL  38
CARD-FILE         0 00000012  0        23WRD   FILE NAME
     DCL  19
     USED 15        39        42        47
CARD-IMAGE   BASED  00000000  0        20WRD   DATA NAME IN FILE SECTION
     DCL  23
     USED 43
GROUP-DATA        0 00000005  0        19BYT   DATA NAME
                                                IN WORKING-STORAGE SECTIO
     DCL  30
IN-REC       BASED  00000000  0        20WRD   DATA NAME IN FILE SECTION
     DCL  22
INIT-SECTION      1 00001045  0                PROCEDURE NAME
     DCL  37
P-TEST-VALUE      0 00000005 18         2WRD   DATA NAME
                                                IN WORKING-STORAGE SECTIO
     DCL  32
PRINT-DETAIL-LINES
                  1 00001075  0                PROCEDURE NAME
     DCL  41
     USED 45
PRINT-FILE        0 00000062  0        23WRD   FILE NAME
     DCL  24
     USED 16        40        47
PRINT-IMAGE  BASED  00000000  0        33WRD   DATA NAME IN FILE SECTION
     DCL  28
     SET  43
PROGRAM-EXAMPLE
                  1 00001024  0                PROGRAM NAME
     DCL  2
PRT          NOT REFERENCED/NOT ALLOCATED      MNEMONIC NAME
     DCL  12
PRT-REC      BASED  00000000  0        33WRD   DATA NAME IN FILE SECTION
     DCL  27
     USED 44
R-TEST-VALUE      0 00000007 18         5BYT   DATA NAME
                                                IN WORKING-STORAGE SECTIO
     DCL  33
T-TEST-VALUE      0 00000005  0         2BYT   DATA NAME
                                                IN WORKING-STORAGE SECTIO
     DCL  31
X-TEST-VALUE      0 00000010 27         2BYT   DATA NAME
                                                IN WORKING-STORAGE SECTIO
```

```
     DCL  34
     USED 44
Z-TEST-VALUE      0 00000011  9         2BYT   DATA NAME
                                               IN WORKING-STORAGE SECTIO
     DCL  35
```

Example 2–6. Cross-Reference Listing (NARROW)

2.3.8. Object Code Listing

The Object Code Listing shows prologue, procedure, and epilogue object code (machine--level code) generated by the compiler. The code is shown in octal and mnemonic (symbolic) form. This listing is designed to look like an assembly code listing.

The following keyword options control printing of the Object Code Listing:

- OBJECT or OBJECT-ONLY
- OBJ-PKT

OBJECT or OBJECT-ONLY Keyword Options

The Object Code Listing as a whole is produced by the OBJECT or OBJECT-ONLY keyword options. (NO-OBJECT is the default.) If you specify OBJECT-ONLY, the listing does not contain the program's source statements.

An Object Code Listing is composed of the following fields:

- RELATIVE ADDRESS
- INSTRUCTION SUBFIELDS (F,J,A,X,H,I,B,D)
- D FLD LC
- LABEL
- SYMBOLIC INSTRUCTION (F,J,A,U,X,B)
- SKELETON NUMBER
- ACTIVE PROCEDURE

Fields of the Object Code Listing

RELATIVE ADDRESS

This field gives the octal address of the instruction or data word, relative to the beginning of the current location counter.

INSTRUCTION SUBFIELDS

This field gives the contents of the instruction word, in instruction format. Each subfield (F,J,A,X,H,I,B,D) is printed in octal. For data words, this field is printed as a single 12-digit octal value preceded by "+".

For a complete description of all of the subfields of instruction format, refer to the *AIM Programming Reference Manual*.

D FLD LC

This field gives the following Relocation information:

- R means the instruction needs relocation by the Linking System.
- X means the instruction is an external reference requiring resolution by the Linking System.
- The location counter of the D subfield. It is represented without the usual parentheses and $. For example, $(1) appears as 1.

LABEL

This field gives one of the following:

- The label name used by another instruction to refer to the current instruction
- The location counter of the instruction itself (for example, $(1))

SYMBOLIC INSTRUCTION

This field gives the instruction in mnemonic form as it might appear in an assembly language program. Symbolic names for user variables are shown in the mnemonic code as follows:

- ASCII characters are shown in the mnemonic code when the program uses constants.
- The mnemonic form is separated into instruction format for instructions, parameter format for parameters, and so on. For a complete description of all of the subfields of instruction format, refer to the *AIM Programming Reference Manual*.
- Unless you specify OBJECT-ONLY, source lines and line numbers are interspersed in the appropriate place within the listing.
- Source lines and line numbers corresponding to object code moved for optimization or other reasons are displayed with the moved code.

SKELETON NUMBER

This field gives the number of the code generation template used to create the sequence of instructions that begins with this instruction. This information is used by Unisys for debugging purposes. It is not useful to you.

ACTIVE PROCEDURE

This field gives the name of the program, compilation unit, procedure, or function that corresponds to this instruction. This information is provided only when the WIDE keyword option is in effect.

If the NARROW keyword option is in effect, the Object Code Listing has the same format as with the WIDE keyword option, except for the following:

- The name of the active procedure is not printed.
- The listing is compressed.
- The D FLD LC and LABEL fields are combined into a single field called LC/LABEL.
- Labels are displayed on a separate line.
- Variable names in the SYMBOLIC INSTRUCTION field may be truncated.
- Line numbers and source lines that exceed their field sizes are displayed on multiple lines.

Example

Example 2–7 shows excerpts of an Object Code Listing with the NARROW keyword option in effect.

```
O B J E C T   C O D E   L I S T I N G

Relative Instruction Subfields         Symbolic Instruction            Skeleton
Address F  J  A  X HI B  D   Lc/Label  F,J      A,U,X,B                  Number

                              $(0)
000000 +  0000000000000     R      0   VA Pointer
                              $(1)
001000 +  0000000001000     R      1   Base Address
000012 +  0011000000027      $(0)      Input/Output Packet
000013 +  0020001000000                Input/Output Packet
000014 +  0000000000000                Input/Output Packet
000015 +  0000000000000                Input/Output Packet
000016 +  0000000000042     R      0   Input/Output Packet
000017 +  0000000000000                Input/Output Packet
000020 +  0000000000000                Input/Output Packet
                                       .
                                       .
                                       .
Main Program Prolog           $(1)
001001 23 16 04 00    000002           LR,U     R4,2                        1154
001002 23 16 05 00    000000           LR,U     R5,0
001003 27 00 02 00 0 00 0100           LX       X2,R0
001004 75 00 10 00 0 00 0002           LBU      B8,X2
001005 27 00 06 00 0 10 0001           LX       X6,01,,B8
001006 10 16 00 00    000000           LA,U     A0,0
000353                      X$(0)      UCS$CODELVE 'ALT$EMLIB' 'RTS$PROLOG
                                                      . Call to  RTS$PROLOGUE
001007 23 00 00 00 0 10 0354           LR       R0,0354,,B8
001010 07 16 13 00 0 10 0353           CALL     0353,,B8
001011 10 16 00 00    000000           LA,U     A0,0
001012 10 16 01 00    000000           LA,U     A1,0
001013 23 00 00 12 1 01 0003           LR       R0,*03,X10,B1
                                                      . LOCL to  PROGRAM-EXAM
001014 07 16 00 00 0  001024R      1   LOCL     PROGRAM-EXAMPLE
001015 27 16 01 00    001000R      1   LX,U     X1,$(1)+01000               1746
Main Program Epilog
001016 27 16 15 00    001016R      1   LX,U     A1,$(1)+01016               1153
001017 46 02 15 01 1 00 0000           LXI,H1   A1,*0,X1,B0
001020 10 16 00 00    000000           LA,U     A0,0
```

```
000355                      X$(0)      UCS$CODELVE 'ALT$EMLIB' 'RTS$EPILOG
                                       .
                                       .
                                       .
001166 23 00 00 00 0 10 0374           LR       R0,0374,,B8
001167 07 16 13 00 0 10 0373           CALL     0373,,B8
001170 27 00 11 11 1 01 0002           LX       X9,*02,X9,B1              1160
001171 24 16 12 00    000024           AX,U     X10,20
001172 73 17 03 00 0 00 0000           RTN
```

Example 2–7. Object Code Listing (NARROW)

OBJ-PKT Keyword Option

If you use the OBJECT or OBJECT-ONLY keyword options, you can also use the OBJ-PKT keyword option. (NO-OBJ-PKT is the default.) The OBJ-PKT keyword option produces a more complete Object Code Listing by printing additional information for I/O packets, parameter packets, and array descriptors. The following example shows a line on the Object Code Listing produced by specifying OBJ-PKT.

Example

```
000007      000000000030      R  0     Input/Output Packet
INPUT
```

2.3.9. Linker Information Listing

The Linker Information Listing specifies linker information for the object module. It is produced by the LINKINFO keyword option. (LINKINFO is the default if you have specified OBJECT or OBJECT-ONLY.) The compiler produces a Linker Information Listing for each object module produced.

A Linker Information Listing is composed of the following sections:

- HEADER
- EXTERNAL REFERENCES
- EXTERNAL DEFINITIONS
- COMMON BLOCKS
- ALS INFORMATION
- LOGICAL BANK

HEADER Section

The header specifies the

- Object module name
- Bank group name for this compilation

Bank group names are created by appending the suffix $ to the object module name.

EXTERNAL REFERENCES Section

This section lists the names of all external references (calls) made in the object module. Only the first 12 characters of the names are printed. Typical UCS object modules contain external references to the following, in addition to references to user subprograms:

- Entry points with the prefix RTS$, which are UCS Runtime System entry points
- TCA$$$, a UCS Runtime System entry point for the task control area

EXTERNAL DEFINITIONS Section

This section lists all external definitions (entry points) contained in the object module and their addresses. Each entry starts on a new line and has four subfields when the NARROW option is in effect. If the WIDE option is used, the entries contain the fifth field 'LINE NUMBER'. External definitions include

- Internal Name

  This is the entry point name as defined in the source program. It is truncated at 32 characters.

- External Name

  This is the entry point name used for LINK resolution.

- LC

  This is the location counter of the defined entry point.

- Offset

  This is the address of the definition, relative to the start of the given location counter.

- Line Number

  This is the line number in the source program where the definition occurred. It is truncated at 53 characters.

COMMON BLOCKS Section

This section lists all common blocks contained in the object module. Each entry starts on a new line and has two subfields:

- Common block name
  - All EXTERNAL user names appear in this section.
  - All names listed in the COMMON-STORAGE section appear under the single name $BLANK$COMMON$.
- Logical bank number of the bank containing the common block

ALS INFORMATION Section

This section lists the sizes of the activity-local stack (ALS) for each procedure in the source program. Each new entry starts on a new line and has the following three subfields:

- Internal name

  The procedure name found in the source program.

- External name

  The EXTERNAL name of the procedure used for linker resolution.

- Size

  The number of words needed for the procedure's ALS.

LOGICAL BANK Section

This section describes the logical banks created by the compilation. It includes the following subfields:

- Logical bank name

  Logical bank names are created by appending the logical bank number as a suffix to the bank group name.

- Location counter number allocated for the bank

  This is the same as the logical bank number.

- Size

  This gives the size of the bank in words, in both octal and decimal values.

  The compiler allocates one extra word for each common block and each data bank declared in the program. This permits the compiler to generate more efficient coding sequences in some cases. Thus the size of the common storage is larger than both of the following:

  – The size of the corresponding common block declared in the program

  – The size of the corresponding common block in the Allocation and Cross-Reference Listings

- Bank Type

  Bank types are code, data, link vector, and so forth. See the *Linking System Programming Reference Manual* for all possible values.

- Bank Mode

  Bank modes are features such as EM (extended mode). See the *Linking System Programming Reference Manual* for all possible values.

- Storage Class

  This gives the type of storage the bank is loaded into. MSU means ordinary main storage. See the *Linking System Programming Reference Manual* for all possible values.

Example

Example 2–8 shows a Linker Information Listing.

```
L I N K E R   I N F O R M A T I O N   L I S T I N G

 OBJECT MODULE NAME: OM
 BANK GROUP NAME: OM$

EXTERNAL REFERENCES

 TCA$$$       RTS$PROLOGUE RTS$EPILOGUE RTS$LBRELOAD RTS$OPEN     RTS$RECREAD
 RTS$RECWRITE RTS$CLOSE    RTS$STOP     RTS$FILECANC

EXTERNAL DEFINITIONS

 Internal Name        External Name  LC            Offset

                       CN$$PROGRAM-   $(1)       000001146
                       PROGRAM-EXAM   $(1)       000001024

COMMON BLOCKS

 No Common Blocks

ALS INFORMATION
 Internal Name        External Name                  Size
 SAMPLE               SAMPLE                          20
 MY_INTERNAL                                          20
 CN$$PROGRAM-         CN$$PROGRAM-                    20
 PROGRAM-EXAM                                         20


LOGICAL BANK                Size           Bank        Bank Storage
 Location Counter     Octal       Decimal  Type        Mode Class
 OM$0
  $(0)                 0400           256  Data        EM   MSU
 OM$1
  $(1)                 0173           123  Code        EM   MSU
 OM$3
  $(3)                 0113            75  SDD         EM   MSU

SIZES: CODE(EM2): 123 DATA: 256
END UCOB- 0 ERRORS(MAJOR) 0 ERRORS(MINOR)
@BRKPT PRINT$
```

Example 2–8. Linker Information Listing

2.3.10. Bank Sizes Line

The bank sizes line precedes the sign-off line. It is produced by the SIGNON keyword option (SIGNON is the default). The bank sizes line

- Lists storage usage
- Identifies the class of machine for which the compiled code has been generated

Example

```
SIZES  CODE(EM2):  117 DATA:  153
```

Format

```
SIZES: CODE(m):  x  DATA:  y  COMMON:  z
```

where:

m

represents the mode of the code bank (I-bank). This appears in the form

EM*n*

This means extended mode, where *n* is a value representing the class of machine for which the compiled code has been generated.

Possible values for *n* are:

Value	Machine
0	Any extended mode machine (generic)
2	1100/90, 2200/400, or 2200/600 system
4	2200/100 or 2200/200 system
6	2200/XPA machine (CMOS systems)
8	4000 system and above (SAIL-based systems)

x

is the number of words of storage used in the code bank for this mode.

y

is the number of words of storage used in the data banks, including link vectors.

z

is the number of words of storage used in common blocks. This field only appears if your program contains common blocks.

The compiler allocates one extra word for each common block and each data bank declared in the program.

2.3.11. Sign-Off Line

The system prints the sign-off line at the end of each compiler listing. It is produced by the SIGNON keyword option. (SIGNON is the default.)

Example

```
END UCOB- 0 ERRORS(MAJOR) 1 ERROR(MINOR) 1 WARNING(USAGE) 1
WARNING(LEVEL)
```

Note: *Specific level numbers and dates in the example can differ from the ones at your site.*

Format

```
END UCOB- n class(subclass)...
```

where:

END UCOB-

identifies the end of the UCS COBOL compilation.

n

is the number of errors or messages for each subclass.

class

is the type of error or message: ERROR, WARNING, or REMARK. (See Appendix A for descriptions of these types.) If there are no WARNING or REMARK messages, no message count appears for that class. The count of WARNING and REMARK messages appears even if NO-WARNING and NO-REMARK keyword options suppress printing of the messages.

(*subclass*)

is the severity of the error or message: MAJOR, MINOR, USAGE, or LEVEL. (See Appendix A for descriptions of these severity types.)

...

means that more than one *n class*(*subclass*) group may appear.

2.4. Separate Compilations for a Single Source Element (Back-to-Back COBOL Programs)

Under the new COBOL standard, it is possible for you to pass to the compiler, in one source element, a set of separate COBOL programs. The compiler does not determine their existence until COPY, REPLACING and the REPLACE directives are resolved, which can actually change the position at which separate programs begin and end.

If the source element (after COPY and REPLACE) looks like this:

```
IDENTIFICATION DIVISION
PROGRAM-ID. A.
   .
   .
   .
IDENTIFICATION DIVISION
PROGRAM-ID. B.
   .
   .
   .
END PROGRAM B.
END PROGRAM A.
```

then program B is nested within program A. The entire source element is considered to be one compilation unit. If the source element looks like this:

```
IDENTIFICATION DIVISION
PROGRAM-ID. A.
   .
   .
   .
END PROGRAM A.
IDENTIFICATION DIVISION
PROGRAM-ID. B.
   .
   .
   .
END PROGRAM B.
```

then program A and program B are considered to be separate programs. This means there are separate object modules and separate listings for each compilation, with no relationship between a definition of an identifier in one program and a reference in another, and so forth. The options on the processor call and default options will be the same for all compilations, but a new naming convention must be defined since there is only one RO field for *n* object modules. The first program will use the RO field, including any version specified; subsequent programs use the element name specified in the RO field, with a version "$$$$*nn*," where *nn* is incremented by one for each separate compile from one up to a maximum of 999.

The sign-on line with the newly created object module name, the options in effect (if requested), and a separate sign-off line will be displayed for each program. The final sign-off line is the same as the sign-off line when there is one compilation per source element. The diagnostic totals are for the entire source element. The SIZE information will follow each separate compilation sign-off line.

The separate compilation sign-off line is produced by the SIGNON keyword. (SIGNON is the default.) The separate compilation sign-off line is as follows:

- After first separate compilation

  ```
  **OBJECT MODULE ro-name/version GENERATED
  ```

- After second separate compilation

  ```
  **OBJECT MODULE ro-name/$$$$1 GENERATED
  ```

- After MAJOR errors in a separate compile with the ERRCHECK option, with STD-ERRCHECK and WARNING-LEVEL errors in a separate compilation, or if there are any MAJOR errors in tokenization (for example, a missing copy).

  ```
  **NO OBJECT MODULE GENERATED
  ```

2.4.1. Example of Separate Compilations

Example 2–9 shows separate compilations where the source element is PRMSPEX. It has 77 source lines.

```
@UCOB,S PRMSPEX,OM,,,NARROW,NO-OPTIONS
UCOB- 12R1E (100402) LSS- 14R3 (100816) 100930 07:02:36
1   IDENTIFICATION DIVISION.
2   PROGRAM-ID. MAIN1.
3   ENVIRONMENT DIVISION.
4   CONFIGURATION SECTION.
5   SOURCE-COMPUTER. UNISYS-1100.
6   OBJECT-COMPUTER. UNISYS-1100.
7   DATA DIVISION.
8   WORKING-STORAGE SECTION.
9   01   DN14 PIC S999.
10  PROCEDURE DIVISION WITH ENTRY POINTS SUB1.
11  PPP.
12       DISPLAY '*'.
13       DISPLAY 'XQT OF SEPARATE MAIN PROGRAM MAIN1'.
14       DISPLAY '*'.
15       GO TO PARA2.
16  SUB1.
17       DISPLAY
18       'SEPARATE PROGRAM MAIN1 CALLED AS SUBPROGRAM SUB1'.
19       DISPLAY '*'.
20  PARA2.
21       EXIT PROGRAM.
22  *EXIT PROGRAM IS A NO-OP IF IN A MAIN PROGRAM (@XQT)
```

```
*WARNING(USAGE) UCOB3053: program flow cannot reach this statement or these
statements
         v

23       DISPLAY 'SEPARATE PROGRAM MAIN1 XQTED AS A MAIN PROGRAM'.
24       DISPLAY ' ELSE THIS IS A FAILURE'.
25       DISPLAY '*'.
26       STOP RUN.
27   END PROGRAM MAIN1.
** OBJECT MODULE OM GENERATED
SIZES:  CODE(EM): 130 DATA: 160
UCOB- 10R1 (000601) LSS- 12R1 (000601) 000625 09:57:12
28   IDENTIFICATION DIVISION.
29   PROGRAM-ID. MAIN2.
30   ENVIRONMENT DIVISION.
31   CONFIGURATION SECTION.
32   SOURCE-COMPUTER. UNISYS-1100.
33   OBJECT-COMPUTER. UNISYS-1100.
34   DATA DIVISION.
35   WORKING-STORAGE SECTION.
36   01  DN14 PIC S999.
37   PROCEDURE DIVISION.

38   PPP.

39       DISPLAY '*'.
40       DISPLAY 'MAIN PROGRAM MAIN2 CALLING SUBPROGRAM SUB2'.
41       DISPLAY '*'.
42       CALL 'SUB2'.
43       DISPLAY 'MAIN PROGRAM MAIN2 CALLING NESTED SUBPROGRAM SUB1'.
44       DISPLAY '*'.
45  *THE CALL TO SUB1 WILL BE RESOLVED WITHIN THIS COMPILATION SINCE
46  *SUB1 IS CONTAINED DIRECTLY WITHIN MAIN2.
47       CALL 'SUB1'.
48       STOP RUN.
49   IDENTIFICATION DIVISION.
50   PROGRAM-ID. SUB2.
51   DATA DIVISION.
52   WORKING-STORAGE SECTION.
53   01  DN14 PIC S999.
54   PROCEDURE DIVISION.
55   PPP.
56       DISPLAY 'SUBPROGRAM SUB2 CALLING SEPARATE SUBPROGRAM SUB1'.
57       DISPLAY
58       'BECAUSE NESTED SUBPROGRAM SUB1 IS NOT ACCESSIBLE TO SUB2'.
59       DISPLAY '*'.
60  *THE CALL TO SUB1 WILL BE RESOLVED EXTERNAL TO THIS COMPILATION
61  *BECAUSE SUB1 IN THIS COMPILATION IS NOT CONTAINED WITHIN SUB2.
62       CALL 'SUB1'.
63       EXIT PROGRAM.
64   END PROGRAM SUB2.
*WARNING(USAGE) UCOB3053: program flow cannot reach this statement or these
```

```
statements
65   IDENTIFICATION DIVISION.
66   PROGRAM-ID. SUB1.
67   DATA DIVISION.
68   WORKING-STORAGE SECTION.
69   01  DN14 PIC S999.
70   PROCEDURE DIVISION.
71   PPP.

72       DISPLAY 'ENTRY TO NESTED SUBPROGRAM SUB1'.
73
74       DISPLAY '*'.
75       EXIT PROGRAM.
76   END PROGRAM SUB1.
77   END PROGRAM MAIN2.
** OBJECT MODULE OM/$$$$1 GENERATED
SIZES: CODE(EM): 216 DATA: 332
END UCOB- 0 ERRORS(MAJOR) 0 ERRORS(MINOR) 1 WARNING(USAGE)
```

Example 2–9. Separate Compilations

2.4.2. Execution of the Object Modules Produced

The following example shows the execution statements to execute the preceding programs, and the output from them.

```
@XQT OM
*
XQT OF SEPARATE MAIN PROGRAM MAIN1
*
SEPARATE PROGRAM MAIN1 XQTED AS A MAIN PROGRAM
   ELSE THIS IS A FAILURE
*
@XQT OM/$$$$1
*
MAIN PROGRAM MAIN2 CALLING SUBPROGRAM SUB2
*
SUBPROGRAM SUB2 CALLING SEPARATE SUBPROGRAM SUB1
BECAUSE NESTED SUBPROGRAM SUB1 IS NOT ACCESSIBLE TO SUB2
*
SEPARATE PROGRAM MAIN1 CALLED AS SUBPROGRAM SUB1
*
MAIN PROGRAM MAIN2 CALLING NESTED SUBPROGRAM SUB1
*
ENTRY TO NESTED SUBPROGRAM SUB1
```

2.5. Using the Run-Condition Word to Detect Compilation Errors

The run-condition word is established by the Executive system. You can use it to determine whether your program had severe errors during compilation. This can be useful in a batch environment to stop the runstream if a previous step contained this type of error.

The compiler uses bit 9 (numbering from bit 1 on the left) of the run-condition word to mean this compilation has an error of subclass MAJOR, FATAL, or INTERNAL. The compiler sets bit 9 to 1 at the beginning of the compilation. If it does not detect any of these errors during compilation, it sets bit 9 to zero, just before termination. You can test the value of this bit when the compilation ends.

The compiler also uses bit 8 of the run-condition word to mean that some previous task in this runstream set bit 9. At the beginning of the compilation, the compiler sets bit 8 if either bit 8 or 9 is already set. It then sets bit 9 for this compilation and proceeds as in the previous paragraph.

For more information on the run-condition word, refer to Section 10 and the *ER Programming Reference Manual*.

Section 3
Linking, Executing, and Debugging Your Program

3.1. Linking Your Program

Linking is the process of converting a program from the form produced by a compiler or assembler into a form that computer hardware can execute. This consists mainly of uniting separately compiled subprograms into a whole program.

Object modules produced by UCS compilers are immediately executable after compilation. This means that, unlike the collection process, no explicit linking phase is needed. When you execute an object module, the Linking System

1. Locates the object module
2. Loads it into main storage for execution

Some programs may consist of more than one object module (compilation unit). If all of a program's object modules reside in the same program file, you do not have to do anything to link them. When one object module executes a reference (call) to another, the following sequence occurs:

1. The operating system suspends execution and calls the Linking System.
2. The Linking System locates and loads the called object module.
3. The operating system executes the called object module.

This capability to link during execution is called dynamic linking.

If your program's object modules reside in more than one file, however, you need to do one of the following:

- Copy them into a single file. (See 3.1.1.)
- Create a LINK$PF.SSDEF$ element so the Linking System can link them during execution. (See 3.1.1.)
- Statically link them. (See 3.1.2.)

The Linking System allows one program to use all three types of linking in a single execution, as described in the following table.

Linking Type	When It Occurs	Advantages and Typical Use
Preload linking (also called collection, binding, or static linking)	Before loading	Fast execution, because linking is completed before execution. Typically used for production programs.
Initial load linking	At load time	Can be used in cases where dynamic linking is not possible. Typically used to load necessary data before a program begins execution. (In required cases, compilers specify this type of linking.)
Dynamic linking	During execution	Avoids a separate preload linking phase. Typically used during program development because linking is not required after each recompilation.

Figure 3–1 illustrates how an object module can be

- Executed immediately after compilation and linked during execution (dynamic linking)
- Explicitly linked before execution to other object modules (static linking)

Note: *In both cases, some references may require initial load linking.*

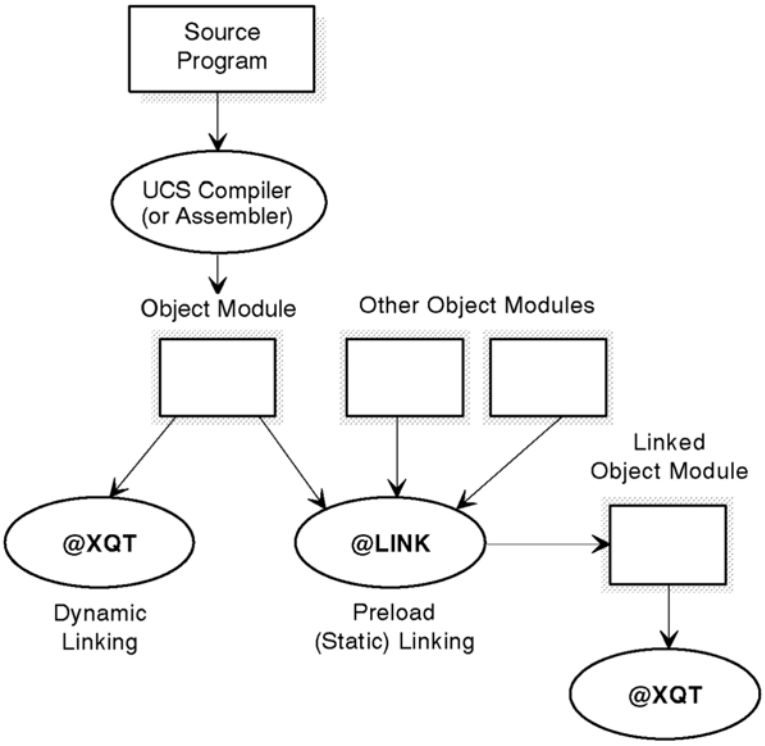


Figure 3–1. Program Development under the Linking System

The following subsections briefly discuss dynamic and static linking. For complete information on the Linking System and static and dynamic linking, see the following documents:

- *Linking System Programming Guide*
- *Linking System Programming Reference Manual*

3.1.1. Dynamic Linking

If all of your program's object modules are in the same file, dynamic linking occurs automatically. If all of your program's object modules are not in the same file, you must create a LINK$PF.SSDEF$ element so that the Linking System can find the required object modules during execution. Use the following process:

1. Call the Subsystem Definition Processor (part of the Linking System) with the @SSDP statement. On the call line, specify the name of the element to be produced (preceded by a comma): It can have any file name, but the element name must be SSDEF$. For example:

```
@SSDP    ,any*file.SSDEF$
```

2. Specify the command DEFINE LSC $LOCAL:

```
DEFINE LSC $LOCAL
```

3. In a SEARCH command, list all of the files containing object modules belonging to your program, in the order you want the files searched for entry point definitions. File names must have periods. Separate file names with commas:

```
SEARCH qual*file-1.,qual*file-2.
```

4. Use the @EOF statement to exit the Subsystem Definition Processor:

```
@EOF
```

5. Give the file specified in the @SSDP statement (step 1) the @USE name LINK$PF:

```
@USE LINK$PF,any*file.
```

6. Execute the program. During execution, the Linking System searches the files listed in the SEARCH command (step 3) for called object modules.

Note: *Initial load linking can be viewed as a special case of dynamic linking. Transparently to you, the compiler and Linking System use it as needed. From your perspective, references using initial load linking are resolved as if they were linked dynamically. However, you can also specifically request initial load linking by using static linking commands.*

3.1.2. Static Linking

Static linking means linking together separate object modules before execution. It is thus similar to collection. Unlike collection, however, you can subsequently link the statically linked object module to another object module.

Many programs can use the following general static linking runstream to link separately compiled object modules into a single object module. See the Linking System documentation for more information on static linking.

1. Call the LINK Processor (part of the Linking System) with the @LINK statement. On the call line, specify the name of the object module to be produced (preceded by a comma). For example:

```
@LINK    ,my*file.obj-module
```

2. Specify the object modules to be linked in an INCLUDE command. Separate object module names with commas:

```
INCLUDE qual*file-1.om-1,qual*file-2.om-2
```

3. Resolve references (calls) with a RESOLVE command. (This is especially useful for resolving references to system library routines.)

```
RESOLVE ALL REFERENCES USING LOCAL_DEFS,LCN
```

4. Structure the program for efficient execution with a PROCESS command:

```
PROCESS FOR EXTENDED
```

5. Delete all unneeded definitions (entry points) except the program starting address:

```
DELETE ALL DEFINITIONS EXCEPT START$
```

   (This command improves linking and execution performance. However, it means other programs cannot link to such entry points because they have been deleted from the object module.)

6. End static linking with the @EOF statement:

```
@EOF
```

7. Execute the program:

```
@XQT  my*file.obj-module
```

3.1.3. Creating Fixed-Gate Subsystems

You can package the code as a subsystem to make it available to other runs and transactions without embedding the code in other applications. All applications can share the subsystem code using a reference (call) to the subsystem. This results in

- Applications referring to the subsystem will have lesser code to load.
- You do not have to relink the applications, when shared libraries are used.

For information on how to select a subsystem type, for statically linking it, and installing the subsystems, refer to the *Linking System Subsystems Programming Guide*.

Most of the COBOL programs are packaged as fast-load (FLSS) or self-contained (SCSS) subsystems. These types of subsystems will load thread-local storage for each activity. However, it is sometimes possible to package COBOL code as a fully application-level fixed-gate subsystem if the code is written to adhere to some rules and restrictions.

A fully application-level fixed-gate subsystem has all the code and data in its object modules loaded at application level. The only thread-local storage available is the activity local stack (ALS) and any storage obtained through malloc or CREATE$BANK calls. By default, the UCS COBOL compiler (UCOB) generates static writable storage for WORKING-STORAGE variables and for packets for CALLs, SORT-MERGE, IO, DMS, ENTER MASM, and for certain ESQL statements such as FETCH commands. If your program is packaged as a fully application-level fixed-gate subsystem, the data being loaded at application-level can cause data corruption problems or a write-protect violation (IGDM). The following problems might occur:

- A protected fixed-gate subsystem might experience data corruption; multiple execution threads simultaneously access the application-level data if locks are not used to protect the data.
- A chameleon fixed-gate subsystem might experience write-protect violations as application-level data is read-only.
- A chameleon subsystem might also experience the data corruption problem if its application-level data is made writable.

With FLSS or SCSS subsystems, since the data is loaded as thread-local storage for the subsystem types, the data corruption or write-protect violation does not occur.

If you add the INITIAL clause to the PROGRAM-ID statement of a COBOL program, the compiler allocates the WORKING-STORAGE variables and parameter packets for CALL and ENTER MASM statements on the ALS instead in the static storage. This eliminates the possible data corruption and write-protect violation problems when accessing this subset of data. The code must be packaged as a fully application-level fixed-gate subsystem.

However, the INITIAL clause by itself is **not** sufficient to totally eliminate these potential subsystem problems. Code generated for IO statements, such as DISPLAY, for SORT, MERGE, and for CDML (DMS) statements will be stored in the static storage that is generated as part of the output object module. COMMON storage is also allocated in static storage. If you want to package COBOL code as a fully application-level fixed-gate subsystem you must do the following:

- Use the INITIAL clause on the PROGRAM-ID statements.
- Do not do any COBOL IO. You can call C code to execute printf and other IO operations.
- Do not use any COBOL SORT or MERGE statements.
- Do not use any CDML statements.
- Do not use any COMMON-STORAGE section (including the EXTERNAL clause for 01 level variables in the WORKING-STORAGE section).

Since the UCS COBOL system was not designed expecting its data areas to be made read-only, there might be more restrictions needed for situations that could cause problems but have not yet been encountered at this time. COBOL programs are rarely packaged as fully application-level fixed-gate subsystems because of these open ended restrictions; they are usually packaged as FLSS or SCSS subsystems, which give each activity its own data.

3.2. Executing Your Program

You can execute an object module immediately after compilation; programs consisting of more than one object module may require linking before execution (see 3.1).

You can execute an object module in the following ways:

- Ordinary object modules can be executed with the Executive control language @XQT statement:

```
@XQT  qual*file.obj-module
```

- Ordinary object modules developed and executed using the Interactive Processing Facility (IPF 1100) can be executed with the following statement:

```
XQT  qual*file.obj-module
```

You can invoke PADS from your program with the following COBOL statement:

```
CALL 'PADS$'.
```

When you finish with PADS, control returns to the next statement in your program unless, while using PADS, you direct it to cause execution to resume at some other point.

3.3. Debugging with PADS

The Programmers Advanced Debugging System (PADS) is the common debugging product for extended mode programs. PADS is a powerful, language-independent, interactive debugging tool.

Note: *Although PADS is your primary debugging tool, other compiler keyword options can also be useful when the execution of your program results in a storage limits violation, unexpected values in the output, or unexpected program flow. Table 2-1 summarizes the compiler keyword options recommended for debugging.*

Interactive debugging allows you to suspend execution of your program and enter PADS commands from a terminal. For example, PADS commands permit you to set traps in your program, examine or modify program variables, and resume program execution.

The PADS command set is flexible, making it suitable for users at all levels of skill. PADS provides a uniform way to debug programs written in any supported language. PADS is independent of the program language it is debugging.

References to program entities, such as statement labels, procedure names, or variables, generally take the same form as in the language used to code the program.

By compiling with the appropriate compiler keyword option (see 3.3.2), you can debug at a symbolic level without having to know the low-level details of the hardware. Low-level hardware-dependent debugging is also available with PADS.

You do not need to include special debugging statements in your program to use PADS. A production TIP transaction that wishes to have TIP sticking and RESTART should not have PADS as its contingency routine. PADS does a CREATE$BANK call to the Exec and this kills TIP sticking and RESTART for the transaction. One way to keep PADS out is to name the URTS runtime library in the RESOLVE statement in the static link of your transaction. This results in PADS stub object module C$NPADS being linked into your program.

```
RESOLVE ALL REFERENCES USING L_D,SYS$LIB$*EMOMRTS.,LCN
```

Another way is to specifically include the C$NPADS object module from the URTS library along with using the LOCAL_DEFS (or L_D) option in the RESOLVE statement in your static link:

```
INCLUDE SYS$LIB$*EMOMRTS.C$NPADS
RESOLVE ALL REFERENCES USING L_D,LCN
```

For more information on using PADS, see the *PADS Programming Guide*.

3.3.1. PADS Features

The following are some of the features provided by PADS:

- Program component display

  The DISPLAY command displays program constants, variables, or expressions that you specify. You can also display all variables in a compilation unit in alphabetical order or by program structure order.

  You can use the DUMP command to display storage locations of your program that do not correspond to any variables. The storage location is defined by a low-level address.

- Program component modification

  The ASSIGN command changes the contents of program variables.

  The ALTER command changes the contents of a storage location defined by a low-level address.

  The SET command changes access permission attributes for a bank in an object module.

- Program execution control

  You can control program flow by using the STEP command. You can direct PADS to execute a specific number of program statements, suspend program execution, and then solicit additional commands. Also, you can specify commands to be executed when the specified number of program statements have executed, by placing the commands in an optional THEN clause.

  You can suspend program execution using the following PADS commands. After the program is suspended, you can enter additional debug commands.

  – TRAP CONDITION

    This command sets a trap when a program condition (such as a divide fault or illegal operation) occurs.

  – TRAP EXECUTION

    This command sets a trap at a specific program location. The program location can be defined by program label, line number, procedure name, or by a low-level address.

  - TRAP CHANGE

    This command sets a trap when a specific program variable changes. The variable can be defined by program name or by a low-level address. Only commands that actually change the program value are trapped.

Using the optional THEN clause with a TRAP command allows you more debugging flexibility.

- Program execution monitoring

  You can trace program flow using the TRACE command to display a message for

  - Each program statement executed.
  - Each program label executed. PADS also identifies the calling statement, if possible.
  - Each procedure or subroutine executed. PADS also identifies the calling statement, if possible.

  Tracing can be turned on for an entire program or for specific elements.

  By using the INQUIRE WALKBACK command, you can trace the flow of all preceding subprogram calls. You can specify the number of calls to be traced or trace all calls back to the main program.

- PADS procedures

  PADS allows you to define command sequences, which are called procedures. PADS procedures can optionally have parameters. You call PADS procedures during a debugging session by using a CALL command.

  You can save PADS procedures in a program file by using a PUT command, modify them by using a text editor, and retrieve the procedures by using a GET command.

- PADS control commands

  PADS contains a set of conditional commands that increase the power of the command set. These control commands allow

  - Conditional execution of blocks of PADS commands (using the IF command)
  - Iterative execution of PADS commands (using the DO or DO WHILE commands)

- PADS variables

  PADS provides internal variables (declare symbols) that you define during the debugging session. You can use declare symbols in place of program variables. Declare symbols can be arrays and can be local or global.

- Dump capabilities

  PADS supplies standard dump procedures to produce dumps that closely resemble the output of the @PMD processor.

- Online help

  At any point during a PADS debugging session, you can request help about the PADS language by entering the HELP command. The HELP command provides a brief description of the definition, usage, and syntax of PADS commands. The HELP command can also provide definitions for the terms used in the PADS syntax.

3.3.2. Compiler Option Considerations

The compiler keyword options for debugging provide symbolic support for the following high-level program entities:

- Procedure names
- Labels
- Line numbers
- Variable names

Symbolic support means that the compiler generates a symbolic debugging dictionary (SDD) that enables PADS to locate and refer to program entities during program execution. PADS needs an SDD to provide a high-level environment for displaying or altering program entities.

The following are the compiler keyword options that affect debugging with PADS. These options are included in 2.2.7.

- DEBUG

  This option generates a restricted SDD. The SDD does not contain information about variables that are not referenced by the program. However, if any member of a group is referenced, information on all group members and any overlays of the group is included in the SDD.

- DEBUG/FULL

  This option generates a PADS SDD in the object module, including declared but unused variables, line numbers, and procedure information

- DEBUG/MONITOR

  This option generates information in the DEBUG/REFERENCE debugging dictionary, in addition to the information generated by the DEBUG and DEBUG/FULL options. This information allows for the monitoring of input and output for each line of source code executed during a PADS debugging session. This option invokes the DEBUG keyword option if the DEBUG, DEBUG/REFERENCE, or DEBUG/FULL keyword options are not specified.

- DEBUG/REFERENCE

  This option generates a restricted SDD similar to what is generated for DEBUG, except information on more group members is eliminated.

- DEBUG/STATEMENT

  This option generates an SDD that contains information about procedures, functions, and line numbers. The SDD will not contain any information about variables or labels.

- DEBUG/WALKBACK

  This option generates an SDD that contains information only about procedures and functions, allowing the use of such PADS commands as INQUIRE WALKBACK and TRACE ON FOR PROCEDURE. This type of SDD provides the minimum level of PADS high-level debugging.

- NO-DEBUG

  This option suppresses generation of the SDD and is recommended for production programs. PADS is functional, but only in a low-level environment. For example, to change the value of a variable, you must know the low-level address of that variable. Any PADS commands that refer to high-level entities generate diagnostic messages.

DEBUG generates a symbolic debugging dictionary that eliminates all information about variables that are declared but not referenced in the program. This is useful when using PADS to debug programs that copy source but actually use only a few of the shared items declared in the copied text.

The various object modules that make up your program can be compiled with different compiler keyword options for debugging. This allows the level of symbolic debugging to vary from one object module to another.

PADS uses only the value in the storage area for a variable to determine the value of that variable. Program optimization through the use of the OPTIM compiler keyword option (which is the default) may affect the debugging of program variables. For example, optimization may cause stores to the storage area for a variable to be omitted. Optimization may also delete some program statements.

For these reasons, if you use PADS for debugging, it recommended that you specify the NO-OPTIM compiler keyword option until debugging is complete. Otherwise, debugging results may be unpredictable.

Table 2-1 lists the compiler keyword options recommended for program debugging.

3.3.3. Excluding PADS

For some production application programs it is desirable to exclude PADS in order to get the highest performing code. This is especially true for TIP transactions, since you cannot get TIP sticking and RESTART when PADS is included in your executable. The tradeoff is that PADS cannot be used to debug the application if it has been excluded (postmortem debugging using a DIAG$ file can be done). One way to exclude PADS from the program is to name the URTS runtime library in the RESOLVE statement in the static link of your program. The result is a PADS stub object module, C$NPADS, being linked into your program.

RESOLVE ALL REFERENCES USING L_D, your-libraries., SYSLIB*EMOMRTS., LCN

The SYSLIB*EMOMRTS file name should follow the LOCAL_DEFS, and must precede LIBCODENAME on the resolve command. If you have library files that you want the linker to search, you can put those file names after LOCAL_DEFS.

Note: *Note LOCAL_DEFS can be abbreviated as L_D, and LIBCODENAME can be abbreviated as LCN.*

Another way is to specifically include the C$NPADS object module from the URTS library along with using the LOCAL_DEFS option in the RESOLVE statement in your static link:

INCLUDE SYSLIB*EMOMRTS.C$NPADS
RESOLVE ALL REFERENCES USING L_D, your-libraries., LCN

3.3.4. Activating PADS

PADS can be activated by the following methods.

@PADS Activation

You enter a @PADS ON processor call statement preceding the @XQT statement for your program.

PADS is then activated and accepts command input before any program statements are executed. PADS is also activated and accepts command input at program termination.

@@X C Activation

At any time during program execution, you may enter the @@X C statement. If there is output coming to your terminal, you must first press the Message Waiting key to break the output so you can enter @@X C.

The Exec treats the @@X C statement as a contingency to be handled by the contingency handler assigned to your program. The default contingency handler for UCS languages is the run-time library. The run-time library stops program execution and activates PADS. PADS then solicits your command input.

Compilation-Time Activation

You can invoke PADS from your program with the following COBOL statement:

```
CALL 'PADS$'
```

When you finish debugging with PADS, control returns to the next statement in your program, unless while using PADS you direct execution to resume at some other point in your program.

Postmortem Activation

You enter a @PADS DIAG or @PADS TIPDIAG processor call statement to debug the program state file or the error history file produced by UCS.

When activated for postmortem debugging, PADS does not accept commands that modify the saved program state or control the program flow, since the program is not executing.

Error Activation

The UCS Runtime System calls PADS, if your program causes a contingency.

Section 4
Changing I/O System Limits and I/O Optimization

4.1. System I/O Overview

File input/output for UCS programs uses certain default values. There are several such defaults; you can override the defaults at run time using the Define File Processor (DFP) if the defaults are not well suited to your application. For information on DFP, see the *DFP Administration and Programming Reference Manual.* This approach lets you override defaults without source code changes, recompiling, or relinking. However, each time a file is opened, your execution stops and the system calls DFP to override the defaults involved. This can degrade performance, perhaps significantly, in applications that frequently open files and that need the default I/O values overridden.

However, you also have the option to change some of the default values used during I/O operations. The defaults you can change include

- Minimum block size (PREPFACTOR)
- Default block size (BLOCKSIZE)
- Runtime System heap area size (HEAPSIZE)
- Conversion table default

Changing these default values does require relinking. However, you do not have to change your source code or recompile your object modules. And you may see substantial improvements in performance and reductions in I/O standard units of processing (SUP) charges.

The default values are defined in the UCS Runtime System object module URTS$TABLES. URTS$TABLES is an object module residing in the file SYSLIB*EMOMRTS. It is searched by the Linking System default library search chain defined in SYS$*DATA$.SSDEF$.

The URTS$TABLES element contains the primary data structures for URTS. If you generate a custom URTS$TABLES element, it should be regenerated to match the URTS level installed on your system before linking the custom URTS$TABLES element into your application. Failure to keep the custom URTS$TABLES element up-to-date could limit the availability of new features in URTS and the UCOB compiler.

To change the defaults in URTS$TABLES and use the new defaults in your application, you must

1. Generate an alternate URTS$TABLES object module
2. Place the new object module in an alternate file
3. Generate an alternate SSDEF$ element that searches the alternate file before it searches SYS$*DATA$.SSDEF$
4. Relink your application's object modules so that the new defaults are applied

A Symbolic Stream Generator (SSG) skeleton has been provided to build an alternate URTS$TABLES object module and place the new object module in a file you specify. (See 4.2.)

Changing these values is a part of the process of optimizing I/O in your application. Generally, you should modify these default values only if one of the following occurs:

- Your I/O SUP charges seem excessive for your application

  Changing URTS$TABLES can reduce I/O SUP charges by reducing the I/O needed to look up DFP modifications when a file is opened.

- Your application's performance appears to be I/O bound

  If your application is using prep factors or block sizes that are not well matched to the data involved in I/O operations, it may result in more I/O operations being needed than would otherwise be necessary. This can degrade performance as well as drive I/O SUP charges up.

  Since DFP overrides must be retrieved each time a file is opened, changing defaults to reduce the number of DFP overrides will also reduce the time spent in I/O operations.

- You find that the URTS$TABLES bank is being expanded automatically as a result of an Exec request (ER) issued by the UCS Runtime System
- Your program fails with error number 37001, indicating that the URTS$TABLES bank has no more available space

The remedies discussed fall into three categories:

- Optimizing the I/O process so that it is performed most efficiently. This is restricted to SDF sequential file I/O. You use the PREPFACTOR and BLOCKSIZE SGSs either singly or in combination for I/O optimization.
- Specifying a larger size for URTS$TABLES so it needs no automatic adjustment. This may depend on the amount of main storage available on your system. You control the default beginning size for URTS$TABLES with the HEAPSIZE SGS.

- Reducing the number of files opened concurrently. For example
  - Each SDF sequential file requires two buffers in URTS$TABLES, as well as other overhead space.
  - Each MSAM file requires five buffers in URTS$TABLES, as well as other overhead space.

  For more information about buffer requirements for various file types, see the *PCIOS Administration and Programming Reference Manual* and the *UDS SFS Administration and Support Reference Manual*.

You can also modify the default conversion table for IBM tape file I/O using the CONVERSION SGS.

4.2. Regenerating the URTS$TABLES Object Module

Use the SSG skeleton, UPDATETABLE, to create or alter an alternate URTS$TABLES object module. This skeleton reads stream generation statements (SGS) you enter and those contained in the UNITTAB element in SYSLIB*URTS. The UNITTAB element contains the default SGSs that the skeleton interprets to create an alternate URTS$TABLES object module.

UPDATETABLE reads in those default SGSs, replaces the ones for which you provide an alternate SGS in the runstream, and writes a new UNITTAB element on the file you specify in the FILE SGS. UPDATETABLE then uses the values in the new UNITTAB element to create an alternate URTS$TABLES object module. It then writes this new URTS$TABLES on a file with the new UNITTAB. The following example calls the skeleton:

Note: *The file name SYSLIB*URTS is different if your site uses a different name to hold the main UCS Runtime System product file.*

```
@SSG,L SYS$LIB$*URTS.UPDATETABLE
SGS
    .
    .  (Enter SGSs to change the defaults as needed)
    .
@EOF
@EOF
```

The following SGSs are allowed. The defaults shown are those Unisys provides when the products are shipped. Your site can reconfigure these to different values on installation; consult your site administrator for local changes in these defaults.

- FILE *file-name* [NEW]

  This SGS is required with each use of UPDATETABLE; *file-name* is the file where the alternate URTS$TABLES object module and UNITTAB element are placed. Thus it is this file that you list in the SEARCH command for your SSDEF$ element (see 4.3, step 2b).

  – If you specify NEW, *file-name* is cataloged, and UPDATETABLE uses the UNITTAB element in SYSLIB*URTS. Any SGSs you enter replace the corresponding SGS in the new UNITTAB that is written to the file you specify. UPDATETABLE then uses this new UNITTAB element to create a new URTS$TABLES object module in the same file with the new UNITTAB element.

    Specify NEW if this is the first time you have created an alternate URTS$TABLES object module or if you wish to use the default UNITTAB element as a starting point instead of one you previously created.

  – If you do not specify NEW, *file-name* is assumed to be an alternate URTS$TABLES object module and also a UNITTAB element. The new UNITTAB element and URTS$TABLES are rewritten to the same file, but with incremented cycle numbers.

    Do not specify NEW if you wish to begin your revision of the URTS$TABLES object module from the last set of defaults you specified.

- PREPFACTOR *integer*

  This SGS is optional. It defines a new value for the SDF sequential prepping factor. The prep factor specifies the minimum block size used in an I/O operation on an SDF sequential file. All SDF sequential file buffers are a multiple of the SDF prepping factor.

  Use PREPFACTOR (along with BLOCKSIZE) to control the amount of buffer space allocated for SDF sequential file I/O. Large I/O buffers will usually reduce I/O SUP charges because large I/O operations are more efficient than more numerous small ones.

  But if too much space is allocated for SDF sequential file I/O buffers, some applications can experience a shortage of heap bank space (URTS$TABLES). This is a fatal condition marked by error number 37001. (If you need large buffers but are running out of heap bank space, try reducing the number of files open concurrently.)

  Valid values for *integer* are 112, 224, 448, 896, 1792, 3584, 7168, and 14336. The initial default value for PREPFACTOR is 1792.

- BLOCKSIZE *integer*

  This SGS is optional. BLOCKSIZE defines a new value for the default SDF sequential block size used during I/O operations if

  – You do not specify a block size when you open an SDF sequential file

  – The UCS Runtime System opens an SDF sequential file for its own purposes

  BLOCKSIZE must be larger than PREPFACTOR if you specify both. If you specify BLOCKSIZE but do not specify PREPFACTOR, BLOCKSIZE must be larger than the system default prepping factor. If you specify a value for BLOCKSIZE that is less than or equal to PREPFACTOR or the system default prepping factor (whichever is in use), BLOCKSIZE is ignored.

  Valid values for *integer* are 112, 224, 448, 896, 1792, 3584, 7168, and 14336. The initial default value for BLOCKSIZE is 1792.

- HEAPSIZE *integer*

  This SGS is optional. HEAPSIZE defines the UCS Runtime System heap area used for record and buffer areas, Executive request (ER) packet areas, I/O tables, General Syntax Analyzer (GSA) messages, and data staging for other basic mode routines.

  The Exec expands the size of URTS$TABLES on receipt of an ER from the UCS Runtime System, up to the maximum allowable on your system. However, this ER is expensive and degrades performance since execution must be halted while URTS$TABLES is expanded. You can reduce costs and improve performance by specifying a HEAPSIZE large enough to avoid automatic expansion.

  Determining whether URTS$TABLES has been expanded can be difficult since URTS$TABLES can also be reduced in size as well. One method is to use PADS entry points at various places in your object module, then use PADS to find the current size of URTS$TABLES. One place to try an entry point is immediately after a file is opened.

  Depending on system limitations, URTS$TABLES can expand to something over 170,000 words. If more space is called for than the system can provide, your program will fail with error number 37001.

  Valid values for *integer* are 1000 to 170000. The initial default for HEAPSIZE is 10000.

  Note: *The recommended way to accomplish this task is to add the following to the program's static LINK:*

  ```
  SET SIZE = SIZE + increased-size FOR URTS$TABLES
  ```

  This command must follow the RESOLVE command and precede the PROCESS command in the static LINK.

  ```
  @LINK     ,filename. Element
  INCLUDE   filename.acb
  INCLUDE   filename.xyz
  RESOLVE ALL REFERENCES USING LCN
  SET SIZE = SIZE + increased-size FOR URTS$TABLES
  PROCESS FOR EXTENDED ZOOM
  ```

- CONVERSION *integer*

  This SGS is optional. Use this SGS to set the value for the default conversion table. If your application makes extensive use of tapes in EBCDIC in any of the character sets shown below, you may find it useful to change the default conversion value. (See 14.4.6.). The following *integer* values are valid.

Integer value	Type of Conversion
0 (default)	ASCII to/from ISO EBCDIC with the 128 character set
1	ASCII to/from ISO EBCDIC with the lowercase alphabetic character set
2	ASCII to/from IBM EBCDIC with the KATAKANA character set
3	ASCII to/from IBM EBCDIC with the lowercase alphabetic character set
4	ASCII to/from ISO EBCDIC with the KATAKANA character set

4.3. Using the Alternate URTS$TABLES Object Module

Note: *You must regenerate the URTS$TABLES object module every time a new release level of URTS is installed onto your system. You cannot statically link an older level of URTS$TABLES with newer levels of the URTS Library. If you have a statically linked object module that includes the regenerated URTS$TABLES along with the URTS Library from the same release level, it is acceptable to continue executing with this object module. However, if you perform a link, you must use the regenerated URTS$TABLES object module from the currently installed level along with the currently installed level of the URTS library.*

To access the new URTS$TABLES object module generated by UPDATETABLE, you must

1. Create an alternate subsystem definition element (SSDEF$)
2. Place the @USE name of LINK$PF on the file containing the alternate SSDEF$ element before static links or executions of object modules

The following describes how to generate and use an alternate SSDEF$ element.

1. List the default SSDEF$ element SYS$*DATA$.SSDEF$. Use the @LIST processor to produce a symbolic version of the SSDEF$ element. For more information on using @LIST with the Linking System, see the *Linking System Programming Reference Manual*. For example

```
@LIST,A SYS$*DATA$.SSDEF$
```

2. Modify the symbolic SSDEF$ element.
   a. Locate the definition of LSC RTS$EMLIB.
   b. Insert a search of the file which contains the new URTS$TABLES object module before the search of the file SYSLIB*EMOMRTS. For example

```
DEFINE LSC RTS$EMLIB
  SEARCH qual*file.
  SEARCH SYS$LIB$*EMOMRTS.
```

   where *qual*file* is the file that contains the alternate URTS$TABLES object module.

3. Generate an absolute SSDEF$ element from the symbolic input:

```
@SSDP  symbolic*file.SSDEF$,absolute*file.SSDEF$
```

4. Put the @USE name of LINK$PF on the file which contains the absolute alternate SSDEF$ element:
   a. For static link

```
@USE LINK$PF.,absolute*file.
@LINK qual*file.linksource,qual*file.om
@XQT qual*file.om
```

   where:

   *qual*file.linksource*

   contains static linking commands. (You can also issue static linking commands from the runstream.)

   *qual*file.om*

   is the object module created by static linking. This is the object module being executed for which you want to change the I/O system limits.

   b. For dynamic link

```
@USE LINK$PF.,absolute*file.
@XQT qual*file.om
```

   where *qual*file.om* is the object module being executed for which you want to change the I/O system limits.

4.4. UCS Runtime I/O

UCOB performs run-time input/output (I/O) operations using the Processor Common Input/Output System (PCIOS). PCIOS consists of a number of common bank modules that access the various file types and a number of processor interface common bank modules for the various UCS compilers such as COBOL, FORTRAN, and C.

PCIOS is used by the UCS Runtime System for I/O compatibility between the various language processors. See the *PCIOS Administration and Programming Reference Manual* for more information.

A UCOB program accesses sequential, relative, and indexed system data format (SDF) files through PCIOS. UCOB handles all formatting and conversions. The actual I/O is done in PCIOS. However, a restriction of PCIOS allows I/O processing to be done to sector-formatted mass storage only, not to word-addressable mass storage.

The default block size of 1792 words is used for all PCIOS related files.

A UCOB program accesses symbiont files through SAR$.

Section 5
Using System Interfaces

UCS COBOL programs have access to many features available through other products. The interfaces described in this section provide the means to do the following:

- Prepare UCS COBOL libraries
- Assign and control the attributes of UCS COBOL files
- Interpret messages that appear on your console

The following table indicates where this manual describes several additional interfaces that are available to UCS COBOL programs.

Interface	Description
DMS 2200	Section 6
RDMS 2200	Section 7
UDS UREP	Section 8
PCFP	Section 9
Executive Requests	Section 10
Interlanguage Calls	Section 15

5.1. Preparing Your UCS COBOL Library

To prepare a UCS COBOL library, source images are introduced to the Procedure Definition Processor (@PDP). The PDP output produces a UCS COBOL library that COBOL programs can access.

Several procedures (text items) can be grouped together in one element, but each separate text name must appear in the label field of a procedure statement (PROC).

The format for a text item input to PDP is:

```
text-name PROC
     (COBOL statements to be copied)
     END
```

The parameter *text-name* can consist only of the characters A through Z, 0 through 9, and -, except that the - cannot appear as the first or last character. The parameter *text-name* must begin in column 1 of the source image and END must begin in column 2. Library text must conform to the rules for COBOL reference format.

5.2. Assigning a UCS COBOL File

The purpose of this assignment procedure is to provide a link between the UCS COBOL-defined file name and the File Control System name (the actual file facilities).

The formats given here are specific examples of the @ASG control statements and are not intended as samples of general format. Consult the *ER Programming Reference Manual* for general rules and formats. If an @ASG is not present in the runstream for executing the object program, an implicit @ASG may be automatically done by the run-time system. See the "Availability of a file" table in the OPEN statement section for the appropriate file type.

5.2.1. Assigning Nonsymbiont Files

Format

```
@ASG,T  DF1,F///200             (disk)

@ASG,T  TAPE1,HICL,1234         (tape)

@ASG,T  TAPE3,HICL//////Q,9012  (Information Interchange Tape or IBM Tape)

@ASG,TJ TAPE3,HICL//////Q,9012  (Information Interchange Tape or IBM Tape)

@ASG,T  DF2.                    (disk)
```

All reel numbers must be specified for input tape files.

The links between COBOL-defined files and these File Control System files are provided in the following COBOL source language statements:

```
SELECT PAYROLL ASSIGN TO DISC DF1.
```

The COBOL file PAYROLL is now associated with a maximum size of 200 tracks with the File Control System name DF1.

```
SELECT OLD-MASTER ASSIGN TO TAPE TAPE1.
```

The COBOL file OLD-MASTER is now associated with tape number 1234 with the File Control System name TAPE1.

```
SELECT INTERCHANGE-FILE ASSIGN TO INTERCHANGE TAPE3.
```

The Information Interchange file INTERCHANGE-FILE is now associated with tape number 9012, which has the File Control System name TAPE3. The second format (@ASG,TJ) indicated an unlabeled tape number 9012, in which case LABEL RECORDS OMITTED must be stated.

When an 8-bit prelabeled reel is required for output (files assigned to TAPE or INTERCHANGE and LABEL RECORDS ARE STANDARD) and an 8-bit prelabeled reel is not mounted, the system issues a console message if the Executive Tape Labeling System is not configured to write 8-bit labels on non-J-option reels.

```
SELECT IBM TAPE ASSIGN TO TAPE TAPE3.
```

If the IBM TAPE is labeled, the system must be set up for EBCDIC tape labeling (see Section 13).

This is an example of element I/O.

```
SELECT FILE1 ASSIGN TO DISC 'QUAL*DF2.ELE'.
```

The COBOL FILE1 is associated with the File Control System QUAL*DF2. This is an example of element I/O.

Device Independency for ANSI Tape Files

You can assign ANSI tape files (for example, SELECT filename ASSIGN TO TAPE) to a mass storage device at execution time. In this case, the system creates an ANSI SDF mass storage file that is the same as if the file originally was assigned to DISC in the source program.

5.2.2. Assigning Symbiont Files

Format

No @ASG control statement is necessary.

General Rules

1. Assignments are handled automatically by the system.
2. The COBOL links are

```
SELECT file-name ASSIGN TO PRINTER.

SELECT file-name ASSIGN TO CARD-READER.

SELECT file-name ASSIGN TO CARD-PUNCH.
```

where *file-name* identifies the file name for use within the COBOL program.

5.2.3. Assigning Alternate Symbiont Files

Format

```
@ASG,CP DF1,F///200
```

General Rules

1. At the time the file is closed, an @BRKPT statement on that file is issued.
2. In order to control the printing or punching of an alternate file explicitly, that file must be assigned by the @ASG statement shown in this subsection and output by a @SYM statement.
3. For output, if a file is not assigned, the system attempts to assign it dynamically, and output and release it at the termination of the run. If the file cannot be assigned because of an assignment conflict, the run error terminates with an appropriate symbiont message. The automatic @ASG, @FREE, and @SYM are for batch runs only.
4. The COBOL links are

```
SELECT file-name ASSIGN TO PRINTER DF1.

SELECT file-name ASSIGN TO CARD-READER DF1.

SELECT file-name ASSIGN TO CARD-PUNCH DF1.
```

   where *file-name* identifies the file name for use within the COBOL program, and DF1 identifies the file name referenced on the @ASG statement.

5.3. Using the Define File Processor (DFP)

Use the Define File Processor to specify attributes for a file at run time without changing the COBOL source statements, recompiling, and relinking.

You create a define file block (DFB) with the statement @DFP,E. When the system opens the file, the UCS Runtime System library accesses the DFB. If a file DFP$ is assigned to the run or you gave a file the use (@USE) name DFP$, the UCS Runtime System looks in it (through ER PFS$) for an omnibus element that has the same name specified as the external name in the ASSIGN clause of the SELECT statement for the file in the COBOL source program. The version name must be spaces.

For more information about the Define File Processor (DFP) call and keyword parameters, refer to the *DFP Administration and Programming Reference Manual.*

5.3.1. Blocking Factors for Non-MSAM Files

The BLOCK keyword parameter list is as follows:

```
BLOCK = bs/bo/bn/bl//bu
```

where:

bs

is the BLOCK CONTAINS clause value of *integer-2.* It is an integer in the range 0 ≤ *bs* ≤ 131072.

bo

is the buffer offset parameter. This parameter is valid only for input tape files assigned the INTERCHANGE. It is an integer in the range 0 ≤ *bo* ≤ 99.

bn

is the number of buffers parameter. It is an integer in the range of 0 < *bn* < 64. However, values of *bn* greater than 2 are meaningful only for indexed files.

bl

is the replacement for the value of *integer-1* in the BLOCK CONTAINS clause. It is an integer in the range 0 < *bl* < 131072. This parameter is meaningful only for files that are indexed. Also, this parameter is meaningful only when the *bs* parameter is specified and *bl* ≤ *bs*.

bu

is the BLOCK CONTAINS clause RECORDS or CHARACTERS option. It is coded either as RECORDS or CHARACTERS.

In the absence of a *bu* parameter, CHARACTERS is assumed. This parameter is meaningful only when the *bs* parameter is specified.

5.3.2. Blocking Factors for MSAM Files

You can use the BLOCK keyword parameter to alter the compiler defaults for buffer size and number of buffers for 1974 Standard Indexed files (that is, PCIOS MSAM files). Difficulties can occur when running UCS COBOL programs that access MSAM files, especially where several MSAM files are accessed simultaneously or where the file records are very large. This occurs because the UCS COBOL compiler tries to optimize the number of buffers and the size of the buffers so that the PCIOS MSAM processor can operate more efficiently.

You can adjust the current operating environment without recompiling or remapping by using the BLOCK keyword parameter to override the compiler generated parameters. The BLOCK keyword parameter list is as follows:

```
BLOCK = bs//bn/bl/bu
```

where:

bs

buffer size (see "Specifying the Buffer Size" in the following discussion).

bn

number of buffer parameters ($4 \leq bn \leq 63$); for OUTPUT and I-O mode, *bn* must be ≥ 5 (see "Specifying the Number of Buffers" in the following discussion).

bl

block loading factor (see "Specifying the Block Loading Factor" in the following discussion).

bu

blocking unit, WORDS, CHARACTERS or RECORDS (ASCII characters are assumed).

Specifying the Buffer Size

Through DFP, you can alter the compiler generated buffer size by specifying *bs* or *bs* plus *bu.* If:

bu = CHARACTERS or *bu* parameter absent

the buffer size used is *bs* rounded up to words then rounded up again to a multiple of 448 words.

bu = WORDS

the buffer size used is *bs* rounded up to a multiple of 448 words.

bu = RECORDS

the buffer size used is (rounded up to a multiple of 448 words):

```
bs * (mrl + 2 + ndk) + 6
```

where:

mrl

is the maximum record length (in words)

ndk

is the number of keys with duplicates in an indexed file

This is the same formula that the COBOL compiler uses; however, no minimum size check is made. Thus, you can specify fewer than 10 records per block.

Specifying the Number of Buffers

Through DFP, you can alter the compiler generated number of buffers by specifying a DFP BLOCK *bn* parameter in the range 5 through 63 for OUTPUT and I-O and 4 through 63 for INPUT. If *bn* is less than the compiler generated value, only the top level index block for the RECORD key is locked into main storage at run-time. If *bn* is 4, no index blocks are locked into main storage.

If the BLOCK *bn* parameter is out of range, the value is ignored. You will obtain the default (4 buffers), plus one buffer for each key, plus the number specified on the SELECT statement RESERVE *n* AREAS, minus one.

Block Loading Factor

The DFP *bl* parameter is equivalent to the *integer-1* field in a BLOCK CONTAINS *integer-1* TO *integer-2* ... clause.

The block loading factor is the number of words reserved in a block at file creation time and is calculated as

```
((bs - bl) / bs) * buffer size
```

If *bs* is not present, or if *bl* is not less than *bs,* the *bl* parameter is ignored.

5.3.3. Record Formats

The *rf* parameter of the RECORD keyword parameter list provided by the DFP is allowed for INTERCHANGE and System Data Format (SDF) files only. No validation checks on the parameter values specified are made at run time.

```
RECORD = /rf
```

where *rf* can be

Parameter Value	Meaning	FCB Value
U	Undefined	01
F	Fixed	02
D	Variable	03

Parameter Value	Meaning	FCB Value
FB	Fixed-blocked	04
DB	Variable-blocked	05
FS	Fixed-segmented	06
DS	Variable-segmented	07
FBS	Fixed-blocked-segmented	010
VBS	Variable-blocked-segmented	011

5.3.4. Prep Factors

The PREP keyword parameter is valid only for mass storage files.

```
PREP = pf
```

For RELATIVE files (direct access), the *pf* parameter can be coded as 112, 224, 448, 896, or 1792.

Note: *Any other values of pf are rounded down to the next value allowed for RELATIVE files.*

5.3.5. Label Records

The LABEL keyword parameter list is

```
LABEL = lt/li
```

lt is the label type parameter. lt can be coded as

Label Type Parameter	FCB Value
OMITTED	1
STANDARD	3

Although this parameter is provided via DFP, complete freedom of choice is not possible because of the label routine collection requirements. This parameter can change the amount of label handling performed on a file only to a lower state than was declared in the source program. More precisely:

Stated Source Program	Allowable It Value
STANDARD	STANDARD,OMITTED
OMITTED	OMITTED

If this parameter is used to change the label processing functions specified in the source program, the physical files that are to be processed must conform to the "new" mode of processing stated, where *li* is the file identifier parameter. It can be from 1 to 17 characters in length. It is left justified and space filled.

Note: *The li parameter can only be specified for SDF sequential and relative disk files.*

5.3.6. EBCDIC Translate

The OPTIONS keyword parameter is

```
OPTIONS = A
```

where *A* indicates that for an EBCDIC file, a 256-character EBCDIC/ASCII translate table is used. Absence of the *A* parameter indicates a 128-character translate is used.

5.3.7. OPEN Parameter

The OPEN parameter gives the status of the file at open time. It is coded as

```
OPEN = os
```

where *os* is the open status and is interpreted as

Parameter	FCB Value	Meaning
NEW	01	Removes the effect of the OPEN statement EXTEND option if specified for a standard TAPE file.
EXTEND	03	If specified for a standard TAPE file opened for output, the action is the same as if the OPEN statement had the EXTEND option specified.

5.3.8. CLOSE Parameter

The CLOSE parameter specifies the disposition of the file at close time. It is coded as:

```
CLOSE = cd
```

where *cd* is the close disposition parameter and is interpreted as

Parameter	FCB Value	Meaning
REWIND	01	If specified for 1974 Standard TAPE or INTERCHANGE file, the file is rewound regardless of a no rewind option on the CLOSE statement.
LEAVE	02	Is specified for 1974 Standard TAPE or INTERCHANGE file, the file is not rewound at close time.
FREE	03	If specified for 1974 Standard TAPE or INTERCHANGE file, or SDF file, the file is freed (@FREE) at close time.

5.3.9. SHARE Parameter

The SHARE keyword parameter specifies that a file is shared (processed by the Shared File System). This parameter is only allowed for RELATIVE and INDEXED I-O files. The format is

```
SHARE = shr
```

where the value of *shr* equals one of the following values:

YES the file is to be processed by the Shared File System (SFS).

NO the file is to be processed by PCIOS.

See 5.5 for more information about specifying shared files.

5.3.10. Concatenation of Data Files

Concatenation of data files allows the system to process separate UCS COBOL physical files as one UCS COBOL logical file. Accomplish this by linking the DFB of each file. The first and subsequent DFBs must be available through the general interface mechanism described at the beginning of this subsection. All DFB keyword parameters are interpreted from the first DFB; only LABEL-ID, OPEN, and CLOSE parameters are interpreted from each subsequent DFB.

File concatenation is limited to the input (forward only) mode of ANSI TAPE, INTERCHANGE, and SDF sequential DISC files. Concatenation of a set of UCS COBOL files is allowed only on component files of the same file format, and uniform labeling conventions must exist for all component files. A component file is one of several files that compose a concatenated file, and a concatenated file is the entire collection of COBOL files that are processed as a single logical file.

When a data file of the type just mentioned is opened for input, the concatenation field of the DFB is checked if a DFB exists for that file. If the field is zero, concatenation terminates. If nonzero, the field contains the element name for the DFB of the next file in the concatenation chain. All DFBs must be accessible in a file DFP$ either by assigning the file as DFP$ or by linking it to DFP$ with a @USE statement.

When an end-of-file is detected on an input file that has a DFB, the concatenation field of the DFB is checked. If concatenation is specified, the system automatically closes the present file observing the DFB CLOSE parameter, retrieves the next DFB, and opens the data file specified in the DFB. If the DFP OPEN parameter for this file is OPEN=OLD and the file was cataloged, it is also automatically assigned. Processing of the input file continues as if no end-of-file had occurred. A true end-of-file condition is not raised until the processing of the last record of the final component file is complete.

The SELECT statement in the COBOL program for the concatenation file can specify the OPTIONAL clause but, if the first component file is present, all other component files must be present. If the first component file is not present, the entire concatenation file is treated as not present.

The following example shows how to use the Define File Processor to produce the define file blocks required to concatenate three component files:

```
@DFP,E  F1.,DFP$.FA,DFP$.FB   Keyword parameters describing F1*
@DFP,E  F2.,DFP$.FB,DFP$.FC   Keyword parameters describing F2*
@EOF
@DFP,E  F3.,DFP$.FC           Keyword parameters describing F3*
@EOF
```

You must assign file DFP$. Files F1, F2, and F3 must be preassigned, or they must be cataloged with OPEN parameters of OPEN=OLD, in which case the run dynamically assigns them.

The COBOL source lines to process these files as a single logical file have the form:

```
SELECT FA ASSIGN TO implementor-name.
   .
   .
   .
OPEN INPUT FA.
READ FA AT END imperative-statement.
```

The system does not execute the *imperative-statement* until it reaches an end-of-file on file F3.

5.4. Accessing the Processor Common Input/Output System (PCIOS)

The Processor Common Input/Output System (PCIOS) serves as the default link between UCS COBOL programs and data files. For a complete discussion of PCIOS, refer to the *PCIOS Administration and Programming Reference Manual*.

5.5. Accessing the Shared File System (SFS)

The Shared File System (SFS) can be used for Relative I-O (DSDF) and Indexed I-O (MSAM) to serve as a link between UCS COBOL programs and input/output devices. For a complete discussion of SFS, refer to the *SFS Administration and Support Reference Manual*.

This interface allows multithread access to files. These files can be processed singly or concurrently in a multithread environment and designated as recoverable. Use one of the following methods to designate a file as shared:

- At compilation time, specify A-SHARED-FILE as the *implementor-name-1* on the ASSIGN clause in the SELECT clause. This method causes all DFP processing for the file to be bypassed.
- At run time, use the SHARE parameter for DFP. (See 5.3.)

Rollback and Deadlock Situations

The Shared File System provides concurrent file access, as well as file recovery. When the Shared File System returns a rollback or deadlock status, UCS COBOL takes error processing actions, depending on whether the user program has done either or both of the following:

- Specified a FILE STATUS clause in the file control entry of the shared file
- Defined a USE procedure for the shared file in the declarative portion of the Procedure Division

The UCS COBOL error processing actions are summarized in Table 5–1.

Caution

When an internal rollback or deadlock occurs, and the file is declared as recoverable, all updates not yet committed are lost.

If you specify a FILE STATUS to detect rollback or deadlock, it must be defined as follows:

```
01   F-STATUS
     02   STATUS-KEY-1 PIC X.
     02   STATUS-KEY-2 PIC 99 BINARY.
```

If STATUS-KEY-1 contains a value of 9, error processing is handled as described in Table 5–1.

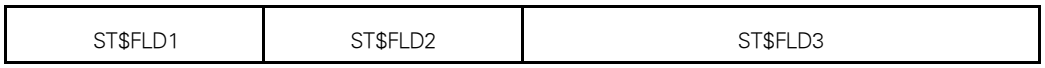
Table 5–1. Rollback/Deadlock Error Processing

	FILE STATUS Specified	No FILE STATUS Specified
USE Procedure Specified	1. Define file status as: Status Key 1 = 9; Status Key 2 = 20 (deadlock) or 21 (rollback) 2. Close all open shared files. 3. Process the declarative. 4. Normal return.	1. Close all open shared files. 2. Process the declarative. 3. Normal return.
No USE Procedure Specified	1. Define file status as: Status Key 1 = 9; Status Key 2 = 20 (deadlock) or 21 (rollback) 2. Close all open shared files. 3. Normal return.	1. Issue the proper error message. 2. Terminate the program execution.

5.6. Retrieving Shared File System (SFS) Status Field 3 (SFSSTATUS$)

Function

Retrieves the previous I/O to the Shared File System (SFS) status field 3 (ST$FLD3) error code. This field contains additional, usually implementation-specific, error information. See the *SFS Administration and Support Reference Manual* for error code conditions.



Format

```
CALL 'SFSSTATUS$' USING ARG.
```

where ARG is a half word binary variable in which the SFS field 3 error code is stored.

Example

```
01  SFSFIELD3   PIC 9(5) BINARY.
                        :
  CALL 'SFSSTATUS$' USING SFSFIELD3.
  DISPLAY "SFSFIELD3 = ", SFSFIELD3.
```

5.7. Passing Procedure Name Arguments to Non-COBOL Programs

If procedure name arguments are being passed to a called non-COBOL program, the called program is treated as a function and a parameter is returned that represents the procedure name to which the called program requires the calling program to jump.

For example, if the called program is a UCS FORTRAN program, the FORTRAN command RETURN *n* is used, where *n* is the parameter number of the procedure name to which the UCS FORTRAN program returns.

Example

The following example shows a COBOL program passing a procedure name parameter to a UCS FORTRAN subprogram.

This is the COBOL main program:

```
 IDENTIFICATION DIVISION.
 PROGRAM-ID. CALLUFTN.
*
*  The CALL to the UCS FORTRAN routine passes one data name and
*  two procedure names.  The UCS FORTRAN routine returns to
*  the first procedure name when the data name is equal to
*  one and to the second procedure name when the data name
*  is equal to two.  For all other values of data name the program
*  uses the normal return.
*
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER. UNISYS-2200.
 OBJECT-COMPUTER. UNISYS-2200.
 DATA DIVISION.
 FILE SECTION.
 WORKING-STORAGE SECTION.
 01   DATA-NAME       PIC 9(10) BINARY VALUE 1.
 PROCEDURE DIVISION.
 STARTING SECTION.
 DO-THE-SETUP.
     DISPLAY  '*  The CALL to the UCS FORTRAN routine '
                  'passes one data name and'.
     DISPLAY  '*  two procedure names.  The UCS FORTRAN routine '
                  'returns to'.
     DISPLAY  '*  the first procedure name when the '
                  'data name is equal to'.
     DISPLAY  '*  one and to the second procedure name '
                  'when the data name'.
     DISPLAY  '*  is equal to two.  For all other values of '
                  'data name it'.
     DISPLAY  '*  uses the normal return.'.
 LOOP-OVER-CALL.
```

```
            DISPLAY  ' '.
            DISPLAY  DATA-NAME.
            CALL 'UFTNRETURN' USING DATA-NAME
                        PROCEDURE-FOR-DATA-NAME-IS-ONE
                        PROCEDURE-FOR-DATA-NAME-IS-TWO.
            DISPLAY  'RETURNED WITH DATA-NAME = ' DATA-NAME.
            STOP RUN.
        PROCEDURE-FOR-DATA-NAME-IS-ONE.
            DISPLAY  'UFTNRETURN HAD DATA-NAME-IS-ONE'.
            MOVE 2 TO DATA-NAME.
            GO TO LOOP-OVER-CALL.
        PROCEDURE-FOR-DATA-NAME-IS-TWO.
            DISPLAY  'UFTNRETURN HAD DATA-NAME-IS-TWO'.
            MOVE 3 TO DATA-NAME.
            GO TO LOOP-OVER-CALL.
```

This is the UCS FORTRAN subprogram.

```
*
*    This demonstrates how a COBOL program can CALL a FORTRAN program
*    passing an integer constant and two procedure names.  The RETURN 1
*    returns to the first * on the SUBROUTINE line which represents the first
*    procedure name and RETURN 2 returns to the second * on
*    the SUBROUTINE line which represents the second procedure name passed.
*
     SUBROUTINE UFTNRETURN ( I,*,* )
*
*
     IF ( I .EQ. 1 )  GO TO 6
     IF ( I .EQ. 2 )  GO TO 7
     RETURN
6     RETURN 1
7     RETURN 2
    END
```

The following is the output generated by executing the preceding COBOL main program and calling the UCS FORTRAN subprogram:

```
*  The CALL to the UCS FORTRAN routine passes one data name and
*  two procedure names.  The UCS FORTRAN routine returns to
*  the first procedure name when the data name is equal to
*  one and to the second procedure name when the data name
*  is equal to two.  For all other values of data name it
*  uses the normal return.

0000000001
UFTNRETURN HAD DATA-NAME-IS-ONE

0000000002
UFTNRETURN HAD DATA-NAME-IS-TWO

0000000003
RETURNED WITH DATA-NAME = 0000000003
```

5.8. Changing the Sort-Merge Main Storage Size

The execution of a sort or merge requires additional main storage starting at the point where the compiler encounters the SORT or MERGE statement. The compiler dynamically allocates additional space when it encounters the SORT or MERGE statement, and releases the space at the completion of the SORT or MERGE statement. Thus, if more than one sort or merge is present in a program, the additional space is needed only when a sort or merge is executing.

You can dynamically change the amount of main storage needed for a sort or merge by using the R$CORE feature. Use this feature by doing a mass storage assignment of a file called R$CORE. The system uses the maximum granularity on this assign to determine the amount of main storage that the sort or merge requires. The UCS COBOL library multiplies the maximum granularity on the @ASG,T R$CORE statement by 1,000 to determine the actual amount of storage to dynamically assign. For example, specify a 14K sort-merge area as follows:

```
@ASG,T R$CORE.,f///14
```

In the absence of an R$CORE statement, the system uses 30K as the storage assignment.

5.9. System Constraints for DENSITY Clause

The DENSITY clause sets the number of lines per inch for printer output. The value specified in the clause is subject to the following system constraints:

1. If DENSITY IS *identifier* clause is specified, *identifier* can contain only the values 6, 8, or 12. Any other value causes the printer default to be used.
2. Following is a list of rules governing the relationship between DENSITY and LINAGE values:
   a. If the resultant DENSITY is 6, the resultant LINAGE must be a value less than 84.
   b. If the resultant DENSITY is 8, the resultant LINAGE must be a value less than 109.
   c. If the resultant DENSITY is 12, the resultant LINAGE must be a value less than 167.

See the *UCS COBOL Programming Reference Manual Volume 1* for more information about the DENSITY clause.

5.10. Interpreting Operator Console Messages

Several UCS COBOL statements send messages to the operator console during execution. The following subsections list and describe these messages.

5.10.1. ACCEPT . . . FROM CONSOLE

While running in the batch mode, an ACCEPT command displays a console message

```
"ENTER DATA"- - - - -
```

for every 50 characters of data that must be entered.

5.10.2. Blank Tape

A blank tape, assigned without a specified reel number that must have a label, displays a console message

```
ENTER NUMBER OF BLANK ON /NN
```

You can use any 6-character reply.

5.10.3. DISPLAY . . . UPON CONSOLE

While using batch mode, a DISPLAY command displays a console message for every 50 characters displayed.

5.10.4. Unlabeled Tape

When a file is opened, the following message displays on the console if you specify label records that are standard for a file assigned to an output tape reel but that are not 8-bit labeled:

```
LOAD PROPERLY LABELED REEL { llllll } file-name AE
```

The *llllll* specifies the logical device name on which the prelabeled reel is to be mounted. The *file-name* is the external file name specified in the ASSIGN clause for the file.

The operator is given an opportunity to mount a properly labeled tape. The operator can then respond with an A or an E. A response of A causes the ER TLBL$ request to be issued again. A response of E causes a diagnostic message 36001 to be issued and the program to abort.

5.10.5. VALUE OF CARTRIDGE-ID

If a file has specified a VALUE OF CARTRIDGE-ID clause, execution of either the OPEN statement for a nonreport file or the INITIATE statement for a report file can produce the following console messages:

```
sname xx IN,yy REQUESTED USE IN? YN

sname MOUNT CARTRIDGE yy A
```

A complete description of these messages can be found in the *Exec Operations Reference Manual Volume 1*.

5.10.6. VALUE OF FORMS-REQUEST

If a file has specified a VALUE OF FORMS-REQUEST clause, execution of either the OPEN statement for a nonreport file or the INITIATE statement for a report file produces the following console message:

```
x - SPECIAL FORMS REQ
```

where *x* is the 24-character alphanumeric field specified in the VALUE OF FORMS-REQUEST clause. The field cannot contain any periods.

Section 6
Accessing a DMS Database

UCS COBOL programs can access a network database through the interface to the Data Management System (DMS). This access is possible by inserting COBOL Data Manipulation Language (CDML) commands into the Data and Procedure Divisions of your COBOL program. The UCS COBOL compiler then syntactically analyzes these commands as it compiles your program.

UCS COBOL also has an interface to the Unisys Repository Manager (UREP), which complements the DMS interface. This additional access provides your program with a means of

- Generating UCS COBOL code from schema and subschema definitions that are stored in a UREP repository
- Inserting the generated code into your program

This section describes the UCS COBOL interface with the Data Management System (DMS). See the *COBOL Data Manipulation Language Operations and Programming Reference Manual* for the syntax and examples of the CDML statements. Also, see Section 8 for a description of the UCS COBOL interface to UREP, and Section 7 for information about accessing a DMS database.

6.1. The UCS COBOL Interface

To compile and link a UCS COBOL program containing CDML statements, perform the following steps:

1. Invoke the Data Definition Language (DDL) processor.

   Input to DDL includes your source schema describing the application database both physically and logically. DDL and SDDL use internal metaschemas and a meta-database to record information needed to define object subschema tables for the source schema.

   Output from DDL is the object schema for your application.

2. Invoke the Subschema Data Definition Language (SDDL) calling the processor.

   Input to SDDL includes your source subschema, which is a subset of the schema giving a logical view of all or part of the database. Again, SDDL uses internal metaschemas and the meta-database during processing.

   Output from SDDL consists of the following:

   - Object subschema tables

     The tables are analogous to the DDL object schema for the application.

   - Symbolic element S$PROC/*subschema-name*

     This element describes the records and database names that you can use in CDML statements. It is included automatically in the UCS COBOL compilation. See F.2.27 for rules on duplicate names.

   - Object module S$WORK/*subschema-name*

     Include this module as input to the Linking System.

3. Compile the UCS COBOL program with embedded CDML statements. Note that an INVOKE clause can appear only in the first program of a series of separately compiled programs in a single source element (back-to-back programs). In addition, the INVOKE can occur only in the outermost program of nested subprograms.

   Input to the UCS COBOL compiler consists of the following:

   - UCS COBOL program
   - Object subschema tables created by the SDDL processor
   - S$PROC/*subschema-name*, created by the SDDL processor

   Output from the UCS COBOL compiler is an object module that you can execute directly or use as input to the Linking System.

   Note: *If the schema and subschema source (DDL and SDDL) are processed with COBOL85 syntax, then the COMPAT and COMP-BIN keyword options cannot be used on the UCOB compilation.*

4. Link the program with information from SDDL.

   Input consists of the following:

   - UCS COBOL output module
   - S$WORK from SDDL

   Output is a statically linked object module Data Manipulation Language (DML) program.

The DML program then executes through the Data Management Routine (DMR), using the object schema and subschema to access and update data in the database.

For more information, see the *DMS DDL Administration, Operations, and Programming Guide* and the *DMS SDDL Administration, Operations, and Programming Guide.*

Figure 6–1 gives a conceptual overview of how a UCS COBOL program interfaces with DMS.

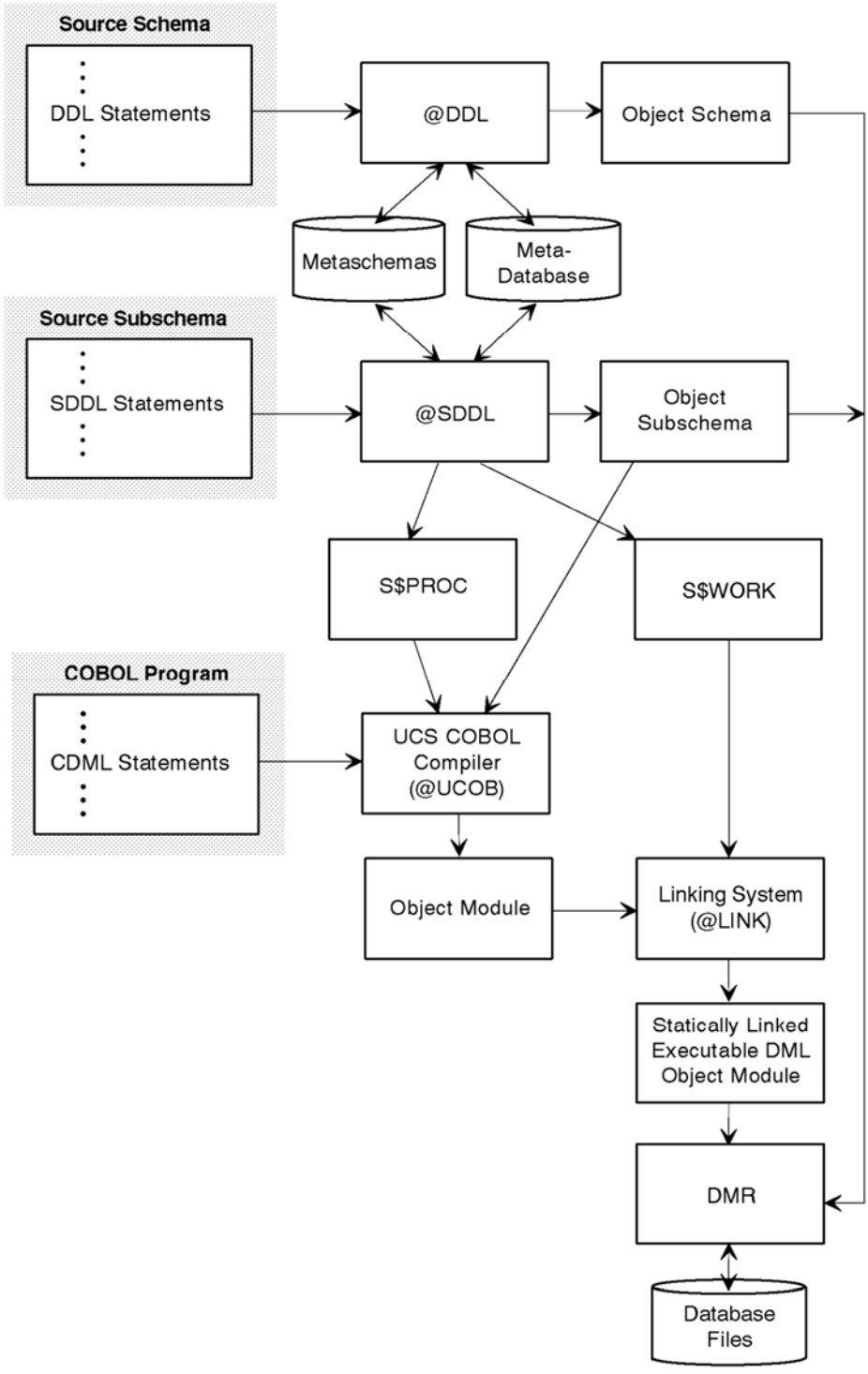


Figure 6–1. Calling DMS from UCS COBOL

6.2. Sample Program

You can use CDML commands directly in a UCS COBOL program with no source changes. Example 6–1 shows a UCS COBOL program using CDML commands. Note that the schema, S41, and the subschema, S41SS1, must be created with the DDL and SDDL processors before the program will successfully compile. See the *CDML Operations and Programming Reference Manual* for a complete description of this example.

When a COBOL program containing DMS statements is compiled, it is recommended that the D$WORK-WS keyword option be used, especially if the DMCA is placed into COMMON using the INVOKE clause. Refer to Section 2.2.7, "Compiler Call Keyword Options for Program Control," for a description of this keyword option.

```
IDENTIFICATION DIVISION
PROGRAM-ID. IMPDEP.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
SPECIAL-NAMES.
     PRINTER IS PRESS.
DATA DIVISION.
SUBSCHEMA SECTION.
INVOKE SUBSCHEMA S41SS1 IN FILE TESTSYM OF SCHEMA S41
     RUN-UNIT-ID IS IMPDEP
     ROLLBACK IS ROLL-BACK
     ERROR IS ERR-PARA.
WORKING-STORAGE SECTION.
77   ITEMFD       PIC 9(10) USAGE IS BINARY.
01   ERCODE.
     02    NUM    PIC 9(10) VALUE IS 0 USAGE IS BINARY.
     02    CODLST PIC X(6)  OCCURS 4 TIMES.
PROCEDURE DIVISION.
BEGIN.
     IMPART.
     DISPLAY 'IMPART COMPLETE' UPON PRESS.
     DEPART.
     DISPLAY 'DEPART COMPLETE' UPON PRESS.
     STOP RUN.
ROLL-BACK.
     DISPLAY 'ROLLBACK' UPON PRESS.
ERR-PARA.
     DISPLAY 'ERROR' UPON PRESS.
     DISPLAY 'ERROR-STATUS = ' ERROR-STATUS UPON PRESS.
     DISPLAY 'ERROR-NUM = ' ERROR-NUM OF DMCA UPON PRESS.
     STOP RUN.
```

Example 6–1. Sample UCS COBOL Program Accessing DMS

6.3. Database Key Usage

Care should be taken when manipulating values within data items that are declared as USAGE DATABASE-KEY. The data type of an item declared as a DATABASE-KEY is the same as if it had been declared as PIC 9(10) BINARY. A data item declared as 9(10) BINARY is stored internally as 1 word (36 bits).

Even though a database key is declared as 9(10) BINARY, it can actually hold a value that is more than 10 digits in length. As long as the database key is moved to/from another database key or data item declared as 9(10) BINARY, the full value is moved (no truncation occurs). If you move a data item not declared as 9(10) BINARY (e.g. PIC 1(36) BINARY-1) to a database key, truncation of the left most digits can occur. This is described in F.2.1. If you do not want the decimal truncation to occur, the keyword option COMPAT should be used. See COMPAT in 2.2.7 for additional details.

If a user needs to manipulate various portions of a database key, first move it to a field that is declared as 9(10) BINARY and is redefined into the individual fields. Update the individual fields as desired and then move the data item declared as 9(10) BINARY back to the database key. For example:

```
   01 DBK1 USAGE DATABASE-KEY.
   01 D1 PIC 9(10) BINARY.
   01 D2 REDEFINES D1.
      02 D2-1 PIC 1(9)  BINARY-1.
      02 D2-2 PIC 1(18) BINARY-1.
      02 D2-3 PIC 1(9)  BINRAY-1.

  MOVE DBK1 TO D1.
  ADD 5 TO D2-3.
  MOVE D1 TO DBK1.
```

The above example works the same whether or not the COMPAT keyword option is used. Since the database key was moved to and from a data item declared as 9(10) BINARY, no decimal truncation occurs.

Section 7
Accessing a Relational Database

UCS COBOL programs can query, retrieve, and update data from a relational database. You accomplish this by inserting Structured Query Language (SQL) commands into your UCS COBOL program.

The UCS COBOL compiler accepts SQL instructions and calls the Relational Data Management System (RDMS 2200), which includes the processor that

1. Interprets the SQL commands and Unisys SQL extensions in your UCS COBOL program
2. Performs the requested task

RDMS is part of the Universal Data System (UDS), which provides a unified system for developing, processing, and managing UCS language applications using databases. UDS also contains the Unisys Repository (UREP), which you use to create and manage the definitions for the relational tables accessed with RDMS (see the *Enterprise Servers Repository for ClearPath OS 2200 Programming Reference Manual*).

UCS COBOL programs can access a relational database using the following methods:

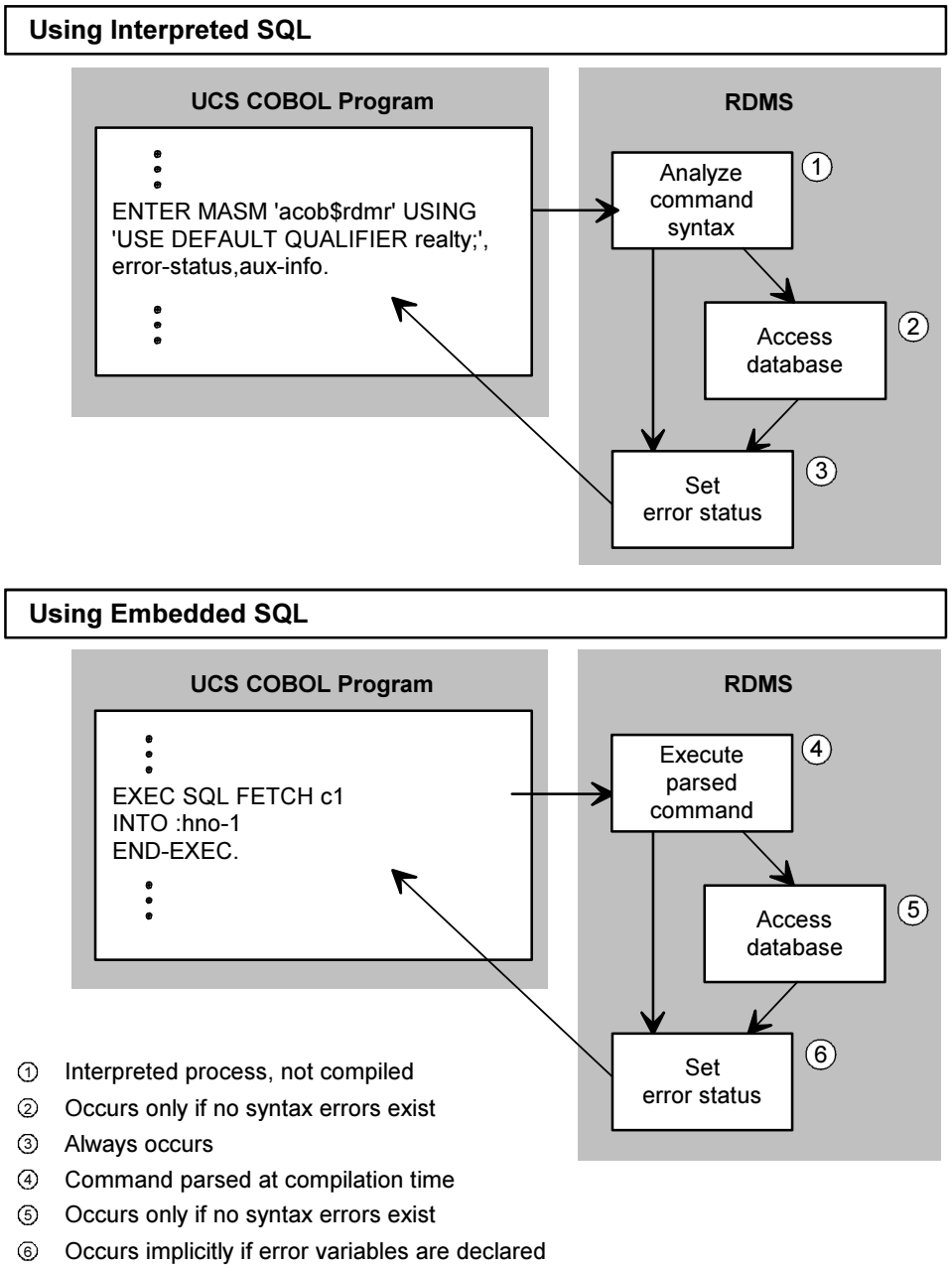
- Embedded SQL

  Include SQL commands directly in the program. RDMS parses the embedded SQL syntax as each command is compiled. Each command is then executed at run time.

- Interpreted SQL

  Code SQL commands in an ENTER MASM statement. RDMS parses the SQL syntax as each command is executed. After parsing the syntax, RDMS processes the requested task.

Figure 7–1 illustrates these interfaces. Both methods pass values to and from the relational database with UCS COBOL variables that are declared in the UCS COBOL program. See 7.3 for the UCS COBOL equivalents to the types that RDMS uses to accept or store data. See the *SQL Programming Reference Manual* for more information about using SQL to access an RDMS database.



① Interpreted process, not compiled
② Occurs only if no syntax errors exist
③ Always occurs
④ Command parsed at compilation time
⑤ Occurs only if no syntax errors exist
⑥ Occurs implicitly if error variables are declared

Figure 7–1. Accessing a Relational Database from UCS COBOL

7.1. Using SQL Commands in a UCS COBOL Program

To use SQL commands in your UCS COBOL program, you must perform the following steps:

1. Declare any variables needed by SQL commands in your host program.

   See 7.2 for a description of the required variables and how to declare them. See 7.3 for the correspondence of data types between RDMS and UCS COBOL.

2. Code the SQL commands in your program source.

   - For embedded SQL, code the commands directly in your source program (see 7.4).
   - For interpreted SQL, code the commands within ENTER MASM statements in your source program (see 7.5).

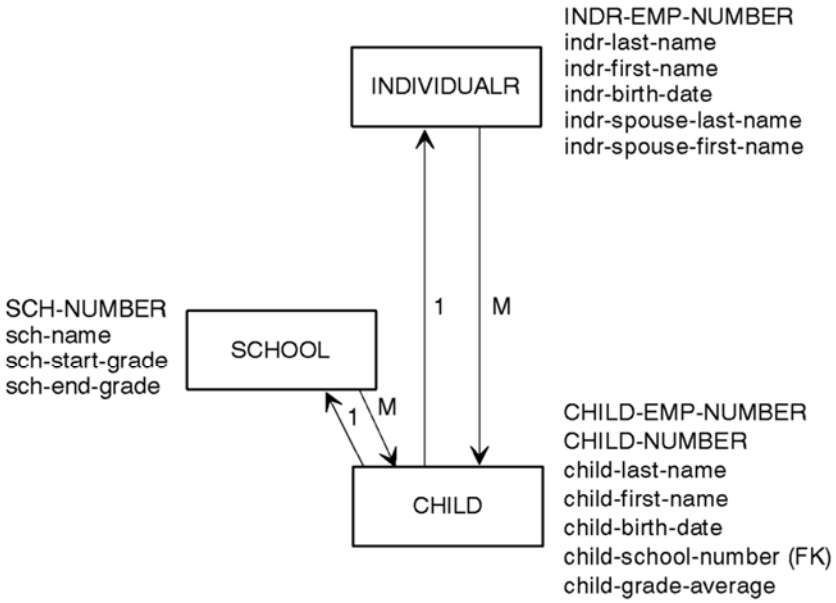
3. Compile the program (see 7.7). For embedded SQL, use the APPLICATION compiler keyword option to specify an application group. The UCS COBOL compiler produces an object module that contains your UCS COBOL instructions and the processed SQL statements. The processed SQL statements are ready for execution, using the same application group specified in the APPLICATION keyword option.

4. Link the program object module with the Linking System, if you want to statically link your program (see 7.8).

   Alternatively, you can dynamically link your program.

5. Execute your program.

This section contains an example coded both in embedded SQL. The program accepts an employee number as input and displays the information contained in a database about the employee's family. The following tables are used in the example.



Legend
1 One-to-one relationship
M One-to-many relationship
FK Foreign key

where:

INDIVIDUALR

is a table whose primary key is the individual's employee number (INDR-EMP-NUMBER). The table has a secondary index consisting of the individual's name (indr-last-name, indr-first-name).

SCHOOL

is a table whose primary key is the school's unique number (SCH-NUMBER).

CHILD

is a table whose primary key is the individual's employee number (CHILD-EMP-NUMBER) and the sequence of the child within the family (CHILD-NUMBER). The table has a secondary index consisting of the child's name (child-last-name, child-first-name).

The child's school number (child-school-number) is a foreign key to the SCHOOL table.

7.2. Declaring Variables for SQL Commands

In most database applications involving programming languages, you need to pass information between your program and RDMS. The variables in your program are called *host program variables*. They are UCS COBOL variable names and are declared according to their context in the program. They are not part of the SQL statement but are used to pass data to and from RDMS, including error information.

The mechanism and procedure for passing variables depend on the interface, as follows:

- Embedded SQL

  Host program variables are embedded directly in the command string and become known as *embedded variables*. See 7.2.1 and 7.2.2 for the requirements of embedded SQL data and error variables.

- Interpreted SQL

  The command string contains placeholder variables (see 7.2.3) that correspond to host program variables, which contain the values to be passed.

This subsection describes how to declare the necessary variables appropriately. See 7.4 and 7.5 for the actual coding of embedded and interpreted SQL statements.

7.2.1. Embedded SQL Data Variables

Any data-item that can be referenced as an identifier in a COBOL statement can be referenced in the same manner as an embedded SQL command, subject to the restrictions in 7.3.2 and 7.3.3. However, the SQL standard requires you to declare all COBOL variables that are used by embedded SQL commands within the WORKING-STORAGE SECTION or COMMON-STORAGE SECTION of the host program. The format for this declaration section is

```
      EXEC SQL BEGIN DECLARE SECTION END-EXEC.

  01  sql-variable             data-description.

      EXEC SQL END DECLARE SECTION END-EXEC.
```

where:

EXEC SQL BEGIN DECLARE SECTION END-EXEC

defines the beginning of the SQL declaration section.

sql-variable

is a variable data item declared as level 01 or 77. The permitted UCS COBOL types are listed in 7.3.2 and 7.3.3.

data-description

is the UCS COBOL syntax that defines the data item (for example, PIC S9(9) COMP).

EXEC SQL END DECLARE SECTION END-EXEC

defines the end of the SQL declaration section.

In an embedded SQL command, it is nonstandard to reference variables that are not declared in the SQL declaration section. The compiler issues warning messages for nonstandard syntax or host-variable references within an embedded SQL command when you specify the compiler keyword option LEVEL/SQL at compilation time. The messages have the following form:

```
WARNING(LEVEL-SQL)
```

The LEVEL/SQL keyword option is separate from the other LEVEL keyword options. LEVEL/SQL can be used in addition to other LEVEL keyword options and does not cancel them.

In compliance with the SQL standard, a program can have multiple DECLARE sections, which are limited to the WORKING-STORAGE SECTION or COMMON-STORAGE SECTION. However, DECLARE sections cannot be nested and they cannot overlap.

7.2.2. Embedded SQL Error Variables

RDMS uses special variables to pass information to the program about errors that occur during the execution of an embedded SQL command. The only exception is the GETERROR command; it does not affect the contents of these variables.

To be able to use these variables in the Procedure Division to check for errors, you must declare them in the Data Division. You can use either of the following data structures:

- SQLSTATE
- SQLCODE
- RDMCA variables ERROR-STATUS and AUX-INFO

If the keyword option LEVEL/SQL is in effect, the absence of the proper declaration of either SQLCODE or SQLSTATE will be flagged.

Refer to the *SQL Programming Reference Manual* for SQL coding techniques to access these variables and for the meaning of the values in them.

Error Code Variable SQLSTATE

The variable SQLSTATE is always a part of the embedded SQL in your program. RDMS uses it to return the status of each embedded SQL command, except the GETERROR command. To access the value of SQLSTATE, include either of the following definitions in the declaration section for SQL:

```
01 SQLSTATE PIC X(5) USAGE DISPLAY.

77 SQLSTATE PIC X(5) USAGE DISPLAY.
```

This declaration conforms to the SQL 92 standard. If SQLSTATE is not declared in this way, a MINOR diagnostic error will be produced stating that SQLSTATE will not contain the RDMS return status after the execution of an embedded SQL command.

Error Code Variable SQLCODE

The variable SQLCODE is always a part of the embedded SQL in your program. RDMS uses it to return the status of each embedded SQL command, except the GETERROR command. To access the value of SQLCODE, include the following definition in the declaration section for SQL:

```
01 SQLCODE  PIC S9(9) USAGE COMPUTATIONAL.
```

This declaration conforms to the 1989 SQL standard. For efficiency, you can declare SQLCODE with a usage of BINARY; however, the compiler flags this usage as nonstandard when you specify the LEVEL/SQL compiler keyword option.

Note: *Standard SQL states that SQLSTATE is the preferred status variable and SQLCODE will be removed at a later date from a revised version of standard SQL.*

RDMCA Variables ERROR-STATUS and AUX-INFO

In addition to SQLCODE and SQLSTATE, you can use the variables ERROR-STATUS and AUX-INFO in the structure RDMCA. RDMS uses these variables to return the status and auxiliary information of each embedded SQL command, except the GETERROR command, analogous to the way the interpreter interface uses the second and third variables on the ENTER MASM statement (see 7.2.3). You may find them easier to use than SQLCODE if you are migrating to embedded SQL from interpreted SQL.

These variables must be defined to meet the following criteria:

- Defined with a parent of RDMCA
- Defined in the SQL declaration section of the program
- Defined without other clauses

Use the following declaration for these variables in your program:

```
01  RDMCA.
    02  ERROR-STATUS PIC 9(4).
    02  AUX-INFO     PIC S9(9) BINARY.
```

where:

ERROR-STATUS

is the variable into which RDMS places the status of each command.

AUX-INFO

is the variable that receives the column position if RDMS detects an error. It also receives auxiliary information for a DELETE, INSERT, LOCATE, REVOKE, UNLOAD, or UPDATE command.

To interpret the values that RDMS assigns to these error variables, see the *SQL Programming Reference Manual*.

7.2.3. Interpreted SQL Variables

The interpreter interface requires markers, called *placeholder variables*, in the command string for any values that must be passed to or from RDMS. Placeholder variables simply mark a place in the command string corresponding to a value the command expects. When you call the interpreter interface and pass the command string with its placeholder variables, you also pass a list of host program variables containing the actual values the command must use. A host program variable must correspond to each placeholder variable in the command string.

The interpreter interface also uses two variables in the statement for reporting the command status, including both normal completion and error information.

Use the ENTER MASM statement to call the interpreter interface. It has the following format, where variable names must occur sequentially in the order specified.

Format

```
ENTER MASM 'ACOB$RDMR'
  USING sql-command
        error-status
        auxiliary-information
      [ program-variable-1
        program-variable-2
                  .
                  .
                  .
        program-variable-n ] .
```

where:

sql-command

is a thread control or SQL command. Refer to 7.5.1 for more information on the contents of this variable and an example.

error-status

contains the completion status of the SQL command on exit from the ENTER MASM statement. It is the second field in the ENTER MASM statement. It is declared as a PIC 9999 DISPLAY or COMP item in the UCS COBOL program. If its value is '0000' the command completed normally.

auxiliary-information

contains useful information about *error-status* on exit from the ENTER MASM statement. It is the third field in the ENTER MASM statement. It is declared as a PIC S1(36) BINARY-1 item in the UCS COBOL program.

program-variable-n

is a UCS COBOL variable name (a host variable) that is associated with an SQL placeholder variable. The number of these host variables must be equal to or greater than the number of distinct placeholder variables required for the SQL command. The list of host variable names follows *auxiliary-information* in the ENTER MASM statement. You must declare each *program-variable-n* in your UCS COBOL program.

A value of '0000' for *error-status* means the command completed normally. Refer to the *SQL Programming Reference Manual* for the possible values for *error-status* and the associated values for *auxiliary-information*, depending on the SQL command used.

7.2.4. Binary Large Objects – BLOBs

From a database perspective, a Binary Large Object (BLOB) is a binary string whose format is known only to the application that stores or retrieves the value. Typical applications may be scanned documents, photographed images, and audio messages.

BLOB variables can be used in both the embedded and interpreted methods.For both methods, the BLOB variable must be declared within the EXEC SQL BEGIN DECLARE SECTION described in 7.2.1.

Program host BLOB variables can be inserted into or fetched from the database. The equivalent SQL data type (column definition) is BLOB.

USAGE SQL TYPE IS BLOB

USAGE SQL is a special USAGE for passing information to and from a relational item defined as a BLOB. It can be used to define a COBOL item used as a host-variable in either an embedded SQL statement or an interpreted SQL statement. This data type may be used wherever a host-variable may be specified, but must be used only on a variable declared as a level 01 item and may not be used with any other USAGE clause.

Format

Integer n[K | M] is the maximum length for the BLOB host program variable, in bytes. The maximum length can be specified as a simple integer representing the number of bytes or as an integer followed by one of the following suffixes:

- K, kilobytes

  2^{10} or 1,024 bytes; 2K is interpreted as 2,048 bytes

- M, megabytes

  2^{20} or 1,048,576 bytes

```
01  blobname USAGE IS SQL  TYPE IS BLOB {(n[K]) }
```

or

```
01  blobname USAGE IS SQL  TYPE IS BLOB {(n[M]) }
```

General Rules:

If USAGE SQL is specified, the following structure is created by the compiler:

```
01 blobname.
      02   blobname-reserved          PIC S9(9) BINARY.
      02   blobname-length            PIC S9(9) BINARY.
      02   blobname-data              PIC X(blob-name-specifier).
```

where:

blob-name-specifier = n

if only integer specified

*blob-name-specifier = n*1024*

if n*K* is specified

*blob-name-specifier = n*1048576*

if n*M* is specified

The compiler expands the BLOB declaration into a structure that consists of two words (8 bytes) of control information and an alphanumeric field that holds the BLOB data. The declaration as expanded by the compiler consists of the following items:

- Structure name, *blobname*

  The same as the original *blobname* from the SQL declaration. The generated structure tag name in the program can be used as you would with any standard UCS COBOL group name. The *blobname* can be used as a host variable in a SQL statement to match the corresponding RDMS column definition of BLOB.

- The structure member, *blobname-reserved*

  The original *blobname* with a suffix of *-reserved*. This is reserved for future use. This field must be set to zero when a BLOB is used as an input item to the database (for example, used in an INSERT command).

- Structure member, *blobname-length*

  The original *blobname* with a suffix of *-length*. You are responsible for setting this field to the byte length of the alphanumeric field that holds the BLOB data when the BLOB is used as an input item in a SQL statement (for example, an INSERT command). This size is used by RDMS when transferring the BLOB from the BLOB program variable to the database.

  When a BLOB is used as an output item in a SQL statement, (for example, a FETCH command) RDMS sets the *blobname* length field to the byte length of the BLOB. This field defines the byte length of the BLOB data, *blobname-data*. The length does not include the 8-byte size of the BLOB control information.

- Structure member, *blobname-data*

  The original *blobname* with a suffix of *-data*. You are responsible for setting this field when the BLOB is used as in input item in an SQL statement. When a BLOB is used as an output item in a SQL statement, RDMS sets this field. If truncation occurs on output from the database to the host program variable, the BLOB's corresponding indicator variable is set to the declared byte length of the BLOB column definition.

Examples

```
01 blob1 USAGE IS SQL TYPE IS BLOB (5000).
01 blob2 USAGE IS SQL TYPE IS BLOB (500K).
01 blob3 USAGE IS SQL TYPE IS BLOB (200M).
```

These declarations result in the following data structures:

```
01 blob1.
   02 blob1-reserved      PIC S9(9) BINARY.
   02 blob1-length    PIC S9(9) BINARY.
   02 blob1-data          PIC X(5000).

01 blob2.
   02 blob2-reserved      PIC S9(9) BINARY.
   02 blob2-length    PIC S9(9) BINARY.
   02 blob2-data         PIC X(512000).

01 blob3.
   02 blob3-reserved     PIC S9(9) BINARY.
   02 blob3-length    PIC S9(9) BINARY.
   02 blob3-data          PIC X(209715200).
```

BLOB Size Limits

Static BLOBs

For a BLOB, *blob-name-specifier* can be in the range 1 to 67,108,852 bytes. The limit is based on the largest single object that is addressable by compiler-generated code. The largest a single object can be is 2^{24}-1 words or 67,108,860 bytes. The actual BLOB size allocated by the compiler is *blob-name-specifier + 8* where 8 represents control information (the reserved word and the length word).

Automatic BLOBs

For a BLOB allocated in automatic storage, *blob-size-specifier* is dependent on how many other automatic storage variables are declared in the function. The size of the activity level stack (ALS) is 262,143 words. However, the compiler limits the total automatic storage size for a given function to 196,608 words or 786,432 bytes. In addition to program variables, compiler-generated temporaries are allocated on the ALS. The execution flow of the program must also be taken into account since each invocation of a function obtains a stack frame on the ALS. Based on these facts, an exact limit cannot be documented for BLOBs allocated in automatic storage. As with any large object such as an array or structure, you must use your own judgment when using automatic storage.

BLOB Example

```
IDENTIFICATION DIVISION.
PROGRAM-ID. cobol-update2.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER.  Unisys-2200.
OBJECT-COMPUTER.  Unisys-2200.
SPECIAL-NAMES.
   PRINTER IS PRINTER.

DATA DIVISION.

WORKING-STORAGE SECTION.

      EXEC SQL BEGIN DECLARE SECTION END-EXEC.

   01 n-c1             PIC 9(10) BINARY.
   01 n-b1             SQL TYPE IS BLOB(10K).
   01 n-b2             SQL TYPE IS BLOB(10K).
   01 n-b3             SQL TYPE IS BLOB(10K).
   01 n-c2             PIC 9(10) BINARY.
   01 n-char1          PIC X(20).

   01 d-c1             PIC 9(10) BINARY.
   01 d-b1             SQL TYPE IS BLOB(10K).
   01 d-b2             SQL TYPE IS BLOB(10K).
   01 d-b3             SQL TYPE IS BLOB(10K).
   01 d-c2             PIC 9(10) BINARY.
```

```
01 d-char1          PIC X(20).

01 SQLCODE          PIC S9(9) USAGE COMPUTATIONAL.
01 SQLSTATE         PIC X(5).
01 RDMCA.
   02 ERROR-STATUS  PIC 9(4).
   02 AUX-INFO      PIC S9(9) BINARY.

   EXEC SQL END DECLARE SECTION END-EXEC.

01 indc1             PIC S9999 USAGE BINARY.
01 indb1             PIC S9999 USAGE BINARY.
01 indb2             PIC S9999 USAGE BINARY.
01 indb3             PIC S9999 USAGE BINARY.
01 indc2             PIC S9999 USAGE BINARY.
01 indchar1          PIC S9999 USAGE BINARY.

01 WORK-AREA-SIZE   PIC 1(36) USAGE BINARY-1 VALUE 2000.
01 at-end-thread    PIC 9.
01 ERROR-TEXT.
   05 ERR           PIC X(132) OCCURS 5 TIMES.
01 ERR-INDEX        PIC 9.

01 COMMAND-AREA.
   05 CM-LINE       PIC X(60) OCCURS 15 TIMES.

01 print-line1.
   05 label1        PIC X(6).
   05 indc1-value   PIC +9.
   05 label2        PIC X(8).
   05 c1-value      PIC 9(10).

01 print-line2.
   05 label3        PIC X(6).
   05 indb1-value   PIC +9.
   05 label4        PIC X(8).
   05 b1-value      PIC X(80).

01 print-line3.
   05 label5        PIC X(6).
   05 indb2-value   PIC +9.
   05 label6        PIC X(8).
   05 b2-value      PIC X(80).

01 print-line4.
   05 label7        PIC X(6).
   05 indb3-value   PIC +9.
   05 label8        PIC X(8).
   05 b3-value      PIC X(80).

01 print-line5.
   05 label9        PIC X(6).
```

```
       05 indc2-value   PIC +9.
       05 label10       PIC X(8).
       05 c2-value      PIC 9(10).

    01 print-line6.
       05 label11       PIC X(6).
       05 indchar1-value   PIC +9.
       05 label12       PIC X(8).
       05 char1-value   PIC X(40).
*
*
 PROCEDURE DIVISION.
 Start-test.

 in-proc1.
     CALL 'RSA$INIT' USING WORK-AREA-SIZE.
*
     MOVE 0 TO at-end-thread.
     MOVE 20               TO n-b1-length.
     MOVE 21               TO n-b2-length.
     MOVE 22               TO n-b3-length.

     MOVE SPACES TO print-line1.
     MOVE SPACES TO print-line2.
     MOVE SPACES TO print-line3.
     MOVE SPACES TO print-line4.
     MOVE 'indc1: '  TO label1.
     MOVE 'indb1: '  TO label3.
     MOVE 'indb2: '  TO label5.
     MOVE 'indb3: '  TO label7.
     MOVE 'indc2: '  TO label9.
     MOVE 'indch1:'  TO label11.
     MOVE ' c1:    ' TO label2.
     MOVE ' b1:    ' TO label4.
     MOVE ' b2:    ' TO label6.
     MOVE ' b3:    ' TO label8.
     MOVE ' c2:    ' TO label10.
     MOVE ' char1: ' TO label12.

     DISPLAY
       'start UCOB static embedded SQL interface test'
       UPON PRINTER.

     MOVE SPACES TO COMMAND-AREA.
     MOVE 'BEGIN THREAD FOR udssrc UPDATE;' TO CM-LINE(1).
     ENTER MASM 'ACOB$RDMR' USING COMMAND-AREA,
         ERROR-STATUS, AUX-INFO.
     IF ERROR-STATUS IS NOT EQUAL TO ZERO
        DISPLAY 'Error on BEGIN THREAD' UPON PRINTER
        PERFORM RDMS-ERROR-PARA2
     END-IF.
```

```
           MOVE 'cobol b1 501 len= 20' TO n-b1-data.
           MOVE 'cobol b2 501  len= 21' TO n-b2-data.
           MOVE 'cobol b3 501   len= 22' TO n-b3-data.
           MOVE 'cobol char1 501' TO n-char1.
           MOVE 99501 TO n-c2.

      *
      *  ###############################################################
      *
      *   DO INSERTS
      *
      *  ###############################################################
      *

           MOVE 501 TO n-c1.
           MOVE 'cobol b1 501 len= 20' TO n-b1-data.
           MOVE 'cobol b2 501  len= 21' TO n-b2-data.
           MOVE 'cobol b3 501   len= 22' TO n-b3-data.
           MOVE 'cobol char1 501' TO n-char1.
           MOVE 99501 TO n-c2.
           DISPLAY 'insert 501' UPON PRINTER.
           EXEC SQL
           INSERT INTO blob_test.blob_table_50
                 VALUES(:n-c1, :n-b1, :n-b2, :n-c2, :n-b3, :n-char1)
           END-EXEC.
           IF ERROR-STATUS IS NOT EQUAL TO ZERO
              DISPLAY 'Error on INSERT 501' UPON PRINTER
              PERFORM RDMS-ERROR-PARA2
           END-IF.

           MOVE 502 TO n-c1.
           MOVE 'cobol b1 502 len= 20' TO n-b1-data.
           DISPLAY 'insert 502 b2, b3, c2 default value' UPON PRINTER.
           EXEC SQL
           INSERT INTO blob_test.blob_table_50 (c1, b1)
                VALUES(:n-c1, :n-b1)
           END-EXEC.

           IF ERROR-STATUS IS NOT EQUAL TO ZERO
              DISPLAY 'Error on INSERT 502' UPON PRINTER
              PERFORM RDMS-ERROR-PARA2
           END-IF.

           MOVE 1  TO indc1.
           MOVE 1  TO indb1.
           MOVE -1 TO indb2.
           MOVE 0  TO indb3.
           MOVE -1 TO indchar1.
           MOVE 503 TO n-c1.
           MOVE 'cobol b1 503 len= 20' TO n-b1-data.
           MOVE 'cobol b2 503  len= 21' TO n-b2-data.
           MOVE 'cobol b3 503   len= 22' TO n-b3-data.
```

```
MOVE 99503 TO n-c2.
DISPLAY 'insert 503 indb2 = -1, indchar1 = -1'
   UPON PRINTER.
EXEC SQL
INSERT INTO blob_test.blob_table_50 (c1, c2, b2, b1)
      VALUES(:n-c1, :n-c2 :indc2, :n-b2 :indb2,
         :n-b1 :indb1)
END-EXEC.
IF ERROR-STATUS IS NOT EQUAL TO ZERO
   DISPLAY 'Error on INSERT 503' UPON PRINTER
   PERFORM RDMS-ERROR-PARA2
END-IF.
MOVE 0 TO indb2.
MOVE 0 TO indchar1.

MOVE 1  TO indc1.
MOVE 2  TO indb1.
MOVE 3  TO indb3.
MOVE -1  TO indc2.
MOVE 504 TO n-c1.
MOVE 'cobol b1 504 len= 20' TO n-b1-data.
MOVE 'cobol b2 504  len= 21' TO n-b2-data.
MOVE 'cobol b3 504   len= 22' TO n-b3-data.
MOVE 'cobol char1 504' TO n-char1.
MOVE 99504 TO n-c2.
DISPLAY 'insert 504 indc2 for c2= -1' UPON PRINTER.
EXEC SQL
INSERT INTO blob_test.blob_table_50 (b2, b1, c1, c2, char1)
 VALUES(:n-b2, :n-b1 :indb1,:n-c1 :indc1,
    :n-c2 :indc2, :n-char1)
END-EXEC.
IF ERROR-STATUS IS NOT EQUAL TO ZERO
   DISPLAY 'Error on INSERT 504' UPON PRINTER
   PERFORM RDMS-ERROR-PARA2
END-IF.
MOVE 0 TO indc2.

MOVE 505 TO n-c1.
MOVE 0 TO n-b2-length.
MOVE 'cobol b1 505 len= 20' TO n-b1-data.
MOVE 'cobol b2 505  len= 21' TO n-b2-data.
MOVE 'cobol b3 505   len= 22' TO n-b3-data.
MOVE 99505 TO n-c2.
DISPLAY 'insert 505 b2 len = 0' UPON PRINTER.
EXEC SQL
INSERT INTO blob_test.blob_table_50
  (b2, c2, b1, c1, b3, char1)
  VALUES(:n-b2, :n-c2, :n-b1, :n-c1, :n-b3, 'abc 505')
END-EXEC.
IF ERROR-STATUS IS NOT EQUAL TO ZERO
   DISPLAY 'Error on INSERT 505' UPON PRINTER
   PERFORM RDMS-ERROR-PARA2
```

```
           END-IF.
           MOVE 21 TO n-b2-length.

           MOVE 506 TO n-c1.
           MOVE 0 TO n-b2-length.
           MOVE 'cobol b1 506 len= 20' TO n-b1-data.
           MOVE 'cobol b2 506  len= 21' TO n-b2-data.
           MOVE 'cobol b3 506   len= 22' TO n-b3-data.
           MOVE 'char1 506' TO n-char1.
           MOVE 99506 TO n-c2.
           DISPLAY 'insert 506 b2 len = 0' UPON PRINTER.
           EXEC SQL
           INSERT INTO blob_test.blob_table_50 (b3, char1, c1, b1)
                 VALUES(:n-b3,:n-char1,:n-c1,:n-b1)
           END-EXEC.
           IF ERROR-STATUS IS NOT EQUAL TO ZERO
              DISPLAY 'Error on INSERT 506' UPON PRINTER
              PERFORM RDMS-ERROR-PARA2
           END-IF.
           MOVE 21 TO n-b2-length.

           EXEC SQL
                COMMIT
           END-EXEC.
           DISPLAY 'commit' UPON PRINTER.


           PERFORM PRINT-DATA-PARA.

      *
      * ################################################################
      *
      *   DO UPDATES
      *
      * ################################################################
      *
           DISPLAY '########## UPDATE SEARCHED ##################'
              UPON PRINTER.
      *
      *  ############  504  ###############
      *
           DISPLAY '-- update multiple blob values in multiple rows'
              UPON PRINTER.
           MOVE -1 TO indb2.
           EXEC SQL
              UPDATE blob_test.blob_table_50
              SET b1 = '504 search b1', b2= :n-b2 :indb2, b3=:n-b3
           END-EXEC.
           MOVE 0 TO indb2.

           PERFORM PRINT-DATA-PARA.
```

```
      DISPLAY 'commit thread' UPON PRINTER.
      EXEC SQL
         COMMIT THREAD
      END-EXEC.

      PERFORM PRINT-DATA-PARA.

      DISPLAY 'update' UPON PRINTER.
      EXEC SQL
         UPDATE blob_test.blob_table_50
         SET b1 = :n-b1, b2=NULL, b3=null
      END-EXEC.

      DISPLAY 'commit thread' UPON PRINTER.
      EXEC SQL
         COMMIT THREAD
      END-EXEC.

      PERFORM PRINT-DATA-PARA.

      MOVE 0  TO indc1.
      MOVE 0  TO indb1.
      MOVE 0  TO indb2.
      MOVE 0  TO indb3.
      MOVE 0  TO indchar1.
      MOVE 'cobol b1 601 len= 20' TO n-b1-data.
      MOVE 'cobol b2 601  len= 21' TO n-b2-data.
      MOVE 'cobol b3 601   len= 22' TO n-b3-data.
      MOVE 99501 TO n-c2.

      DISPLAY '########## UPDATE VALUES ##################'
         UPON PRINTER.
*
*  ############  601  ###############
*
      DISPLAY '-- update multiple blob values in single row'
         UPON PRINTER.
      EXEC SQL
         UPDATE blob_test.blob_table_50
         VALUES (1, '601 values b1', '601 values b2',
           601, '601 values b3', '601 char1 value')
         WHERE c1=1
      END-EXEC.

      PERFORM PRINT-DATA-PARA.


      MOVE SPACES TO COMMAND-AREA.
      MOVE 'OMIT THREAD;'                    TO CM-LINE(1).
      DISPLAY 'omit thread' UPON PRINTER.
      ENTER MASM 'ACOB$RDMR' USING COMMAND-AREA,
          ERROR-STATUS, AUX-INFO.
```

```
           IF ERROR-STATUS IS NOT EQUAL TO ZERO
              DISPLAY 'Error on omit thread' UPON PRINTER
              PERFORM RDMS-ERROR-PARA2
           END-IF.

           PERFORM PRINT-DATA-PARA.

           DISPLAY 'update' UPON PRINTER.
           EXEC SQL
              UPDATE blob_test.blob_table_50
              VALUES(2, :n-b1, :n-b2, :n-c2, :n-b3, :n-char1)
              WHERE c1=2
           END-EXEC.

           DISPLAY 'commit thread' UPON PRINTER.
           EXEC SQL
              COMMIT THREAD
           END-EXEC.

           PERFORM PRINT-DATA-PARA.

           DISPLAY '########## UPDATE CURSOR ##################'
              UPON PRINTER.
      *
      *  ###########  703  ##############
      *
           MOVE 'cobol b1 703 len= 20' TO n-b1-data.
           MOVE 'cobol b2 703  len= 21' TO n-b2-data.
           MOVE 'cobol b3 703   len= 22' TO n-b3-data.
           MOVE 99703 TO n-c2.

           DISPLAY
            '--update multiple blob values in single row - NULL test'
              UPON PRINTER.

           DISPLAY 'declare cursor' UPON PRINTER.
           EXEC SQL
              DECLARE curs703 CURSOR
              FOR SELECT c1, b1, b2, c2, b3, char1
              FROM blob_test.blob_table_50
              FOR UPDATE OF b1, b2, b3, char1
           END-EXEC.

           DISPLAY 'open cursor' UPON PRINTER.
           EXEC SQL
              OPEN CURSOR curs703
           END-EXEC.
           DISPLAY 'fetch first' UPON PRINTER.
           EXEC SQL
           FETCH FIRST curs703 INTO :d-c1 :indc1,:d-b1 :indb1,
                  :d-b2 :indb2,:d-c2 :indc2,:d-b3 :indb3,
                  :d-char1 :indchar1
```

```
           END-EXEC.

           DISPLAY 'fetch next' UPON PRINTER.
           EXEC SQL
           FETCH NEXT curs703 INTO :d-c1 :indc1,:d-b1 :indb1,
                  :d-b2 :indb2,:d-c2 :indc2,:d-b3 :indb3,
                  :d-char1 :indchar1
           END-EXEC.

           DISPLAY 'update' UPON PRINTER.
           EXEC SQL
              UPDATE blob_test.blob_table_50
              SET b1= '703 cursor b1', b2=NULL, char1 = :n-char1,
               b3= null
              WHERE CURRENT OF curs703
           END-EXEC.

           DISPLAY 'commit thread' UPON PRINTER.
           EXEC SQL
              COMMIT THREAD
           END-EXEC.

           PERFORM PRINT-DATA-PARA.
       *
       *    the COMMIT closes all open cursors
       *
           DISPLAY 'open cursor' UPON PRINTER.
           EXEC SQL
              OPEN CURSOR curs703
           END-EXEC.

           DISPLAY 'fetch first' UPON PRINTER.
           EXEC SQL
           FETCH FIRST curs703 INTO :d-c1 :indc1,:d-b1 :indb1,
                  :d-b2 :indb2,:d-c2 :indc2,:d-b3 :indb3,
                  :d-char1 :indchar1
           END-EXEC.

           DISPLAY 'fetch next' UPON PRINTER.
           EXEC SQL
           FETCH NEXT curs703 INTO :d-c1 :indc1,:d-b1 :indb1,
                  :d-b2 :indb2,:d-c2 :indc2,:d-b3 :indb3,
                  :d-char1 :indchar1
           END-EXEC.

           DISPLAY 'update' UPON PRINTER.
           EXEC SQL
              UPDATE blob_test.blob_table_50
              SET b1 = '703 cursor b1', b2=:n-b2,
               b3= '703 cursor b3'
              WHERE CURRENT OF curs703
           END-EXEC.
```

```
           DISPLAY 'commit thread' UPON PRINTER.
           EXEC SQL
              COMMIT THREAD
           END-EXEC.

           PERFORM PRINT-DATA-PARA.

           PERFORM END-THREAD-TO-UDS.
           PERFORM TERMINATION-PARA.
       *
       *  other procedure paragraphs
       *
        PRINT-DATA-PARA.
           MOVE SPACES TO COMMAND-AREA.
           MOVE 'DECLARE c1 CURSOR '                TO CM-LINE(1).
           MOVE 'SELECT * '                         TO CM-LINE(2).
           MOVE 'FROM blob_test.blob_table_50;'      TO CM-LINE(3).
           DISPLAY 'declare cursor' UPON PRINTER.
           ENTER MASM 'ACOB$RDMR' USING COMMAND-AREA,
               ERROR-STATUS, AUX-INFO.
           IF ERROR-STATUS IS NOT EQUAL TO ZERO
              DISPLAY 'Error on DECLARE CURSOR' UPON PRINTER
              PERFORM RDMS-ERROR-PARA2
           END-IF.

           MOVE SPACES TO COMMAND-AREA.
           MOVE 'OPEN c1;'                          TO CM-LINE(1).
           DISPLAY 'open cursor' UPON PRINTER.
           ENTER MASM 'ACOB$RDMR' USING COMMAND-AREA,
               ERROR-STATUS, AUX-INFO.
           IF ERROR-STATUS IS NOT EQUAL TO ZERO
              DISPLAY 'Error on OPEN CURSOR' UPON PRINTER
              PERFORM RDMS-ERROR-PARA2
           END-IF.

           PERFORM FETCH-AND-PRINT-PARA
               WITH TEST AFTER UNTIL ERROR-STATUS = 6001.

           MOVE SPACES TO COMMAND-AREA.
           MOVE 0000 TO ERROR-STATUS.
           MOVE 'CLOSE c1;'                         TO CM-LINE(1).
           DISPLAY 'close cursor' UPON PRINTER.
           ENTER MASM 'ACOB$RDMR' USING COMMAND-AREA,
               ERROR-STATUS, AUX-INFO.
           IF ERROR-STATUS IS NOT EQUAL TO ZERO
              DISPLAY 'Error on CLOSE CURSOR' UPON PRINTER
              PERFORM RDMS-ERROR-PARA2
           END-IF.

           MOVE SPACES TO COMMAND-AREA.
           MOVE 'DROP CURSOR c1;'                   TO CM-LINE(1).
```

```
           DISPLAY 'drop cursor' UPON PRINTER.
           ENTER MASM 'ACOB$RDMR' USING COMMAND-AREA,
               ERROR-STATUS, AUX-INFO.
           IF ERROR-STATUS IS NOT EQUAL TO ZERO
              DISPLAY 'Error on DROP CURSOR' UPON PRINTER
              PERFORM RDMS-ERROR-PARA2
           END-IF.


       FETCH-AND-PRINT-PARA.
           MOVE SPACES TO COMMAND-AREA.
           MOVE SPACES TO d-b1-data.
           MOVE SPACES TO d-b2-data.
           MOVE 'FETCH NEXT c1 '                    TO CM-LINE(1).
           MOVE
           'INTO $p1 $p2,$p3 $p4,$p5 $p6,$p7 $p8,$p9 $p10, $p11 $p12;'
                                                     TO CM-LINE(2).
           ENTER MASM 'ACOB$RDMR' USING COMMAND-AREA,
               ERROR-STATUS, AUX-INFO, d-c1, indc1, d-b1, indb1,
                   d-b2, indb2, d-c2, indc2, d-b3, indb3, d-char1,
                     indchar1.

           MOVE indc1 TO indc1-value.
           MOVE indb1 TO indb1-value.
           MOVE indb2 TO indb2-value.
           MOVE indb3 TO indb3-value.
           MOVE indc2 TO indc2-value.
           MOVE indchar1 TO indchar1-value.
           MOVE ZERO TO c1-value.
           IF indc1 IS >= ZERO
              MOVE d-c1 TO c1-value.
           MOVE SPACES TO b1-value.
           IF indb1 IS >= ZERO
              MOVE d-b1-data(1:d-b1-length) TO b1-value
           END-IF.
           MOVE SPACES TO b2-value.
           IF indb2 IS >= ZERO
              MOVE d-b2-data(1:d-b2-length) TO b2-value
           END-IF.
           MOVE ZERO TO c2-value.
           IF indc2 IS >= ZERO
              MOVE d-c2 TO c2-value.
           MOVE SPACES TO b3-value.
           IF indb3 IS >= ZERO
              MOVE d-b3-data(1:d-b3-length) TO b3-value
           END-IF.
           MOVE SPACES TO char1-value.
           IF indchar1 IS >= ZERO
              MOVE d-char1 TO char1-value.

           IF ERROR-STATUS IS EQUAL TO ZERO
              DISPLAY print-line1 UPON PRINTER
```

```
           DISPLAY print-line2 UPON PRINTER
           DISPLAY print-line3 UPON PRINTER
           DISPLAY print-line4 UPON PRINTER
           DISPLAY print-line5 UPON PRINTER
           DISPLAY print-line6 UPON PRINTER
       ELSE
           IF ERROR-STATUS IS NOT EQUAL TO 6001
              DISPLAY 'Error on FETCH' UPON PRINTER
              PERFORM RDMS-ERROR-PARA2
           ELSE
              CONTINUE
       END-IF.

   END-THREAD-TO-UDS.
       MOVE 1 TO at-end-thread.
       END THREAD.

   TERMINATION-PARA.
       DISPLAY 'END UCOB static embedded SQL interface update test'
            UPON PRINTER.
       STOP RUN.

   RDMS-ERROR-PARA.
       EXEC SQL WHENEVER SQLERROR CONTINUE END-EXEC.
       DISPLAY '*** SQLSTATE = ' SQLSTATE UPON PRINTER.
       DISPLAY '*** RDMCA AUX-INFO = ' AUX-INFO UPON PRINTER.
       DISPLAY '*** ERROR STATUS = ' ERROR-STATUS UPON PRINTER.
       GETERROR INTO :ERR(1), :ERR(2), :ERR(3), :ERR(4), :ERR(5).
       PERFORM VARYING ERR-INDEX FROM 1 BY 1 UNTIL ERR-INDEX = 6
           DISPLAY ERR (ERR-INDEX) UPON PRINTER
       END-PERFORM.
       IF at-end-thread IS EQUAL TO 0
          PERFORM END-THREAD-TO-UDS.
       PERFORM TERMINATION-PARA.

   RDMS-ERROR-PARA2.
       EXEC SQL WHENEVER SQLERROR CONTINUE END-EXEC.
       DISPLAY '*** ERROR STATUS = ' ERROR-STATUS UPON PRINTER.
       DISPLAY '*** RDMCA AUX-INFO = ' AUX-INFO UPON PRINTER.
       MOVE 'GETERROR INTO $P1, $P2, $P3, $P4, $P5 ;'
          TO COMMAND-AREA.
       ENTER MASM 'ACOB$RDMR' USING COMMAND-AREA, ERROR-STATUS,
              AUX-INFO, ERR(1), ERR(2), ERR(3), ERR(4), ERR(5).
       PERFORM VARYING ERR-INDEX FROM 1 BY 1 UNTIL ERR-INDEX = 6
           DISPLAY ERR (ERR-INDEX) UPON PRINTER
       END-PERFORM.
       IF at-end-thread IS EQUAL TO 0
          PERFORM END-THREAD-TO-UDS.
       PERFORM TERMINATION-PARA.
```

7.3. Data Types for Passing Values between RDMS and UCS COBOL

This subsection describes the correspondence of data types for passing values between RDMS and a UCS COBOL program.

7.3.1. Equivalent Data Types

RDMS and UCS COBOL each define specific data types. Some data types require conversion when passed; other data types are equivalent and do not require conversion. Table 7–1 identifies the equivalent data types for UCS COBOL and RDMS. To avoid unnecessary conversion and the resulting overhead, use equivalent types for corresponding data items.

Table 7–1. Equivalent Data Types for UCS COBOL and RDMS

RDMS Data Types	UCS COBOL Data Declarations
CHARACTER(*n*)	X(*n*) A(*n*)
NCHARACTER(*n*)	X(*n*) usage disp-2[1]
DECIMAL(*d,s*)	S9(*d-s*-1)v9(*s*) sign leading separate[2]
NUMERIC(*d,s*)	S9(*d-s*)v9(*s*) usage binary[3]
INTEGER	S9(10) usage binary S9(9) usage comp-5 S1(36) usage binary-1
SMALLINT	S9(5) usage binary S9(4) usage comp-5 S1(18) usage binary-1
REAL	usage comp-1
DOUBLE PRECISION	usage comp-2
FLOAT(*p*)	usage comp-1[4] usage comp-2[4]
DATE, TIME, TIMESTAMP	X(*n*)[5]
BLOB	usage is SQL, type is BLOB.

Legend

1. Kanji characters
2. UCS COBOL supports only a value of $d \leq 19$.
3. UCS COBOL supports only a value of $d \leq 18$
4. Depends on the value of p: if $p \leq 27$, use usage comp-1; if $p > 27$ or not specified, use usage comp-2.
5. See the *Enterprise Relational Database Server for ClearPath OS 2200 SQL Programming Reference Manual* for appropriate values for *n*.

7.3.2. Passing Values to RDMS from Program Variables

If host program variables provide data to RDMS, they can be any of the following data types:

- BINARY (signed or unsigned)
- BINARY-1 (signed or unsigned)
- BLOB
- COMP-1
- COMP-2
- COMP-5 (signed)
- PACKED-DECIMAL (signed or unsigned)
- Numeric DISPLAY or COMP
  - For embedded SQL, these data types can be signed (with any sign position) or unsigned.
  - For interpreted SQL, these data types must be signed with the LEADING SEPARATE clause.
- Alphanumeric (can have the JUSTIFIED attribute)
- ALPHABETIC (can have the JUSTIFIED attribute)

  This data type is supported for embedded SQL only.
- Kanji variables

The following arguments are not supported:

- Edited numeric program variable arguments
- Edited alphanumeric arguments
- Index arguments

An alphabetic program variable argument must correspond to a CHARACTER value; that is, a column of type CHARACTER. An alphanumeric program variable argument must either correspond to a CHARACTER value, or contain date or time data. If the variable contains datetime data, an SQL CAST function must be used. See the *Enterprise Relational Database Server for ClearPath OS 2200 SQL Programming Reference Manual* for more information.

A numeric program variable argument of type BINARY, BINARY-1, COMP-1, COMP-2, COMP-5, or PACKED-DECIMAL must correspond to a numeric value of type NUMERIC, DECIMAL, INTEGER, REAL, or DOUBLE PRECISION. For numeric variable arguments, the UCS Runtime System performs appropriate conversions if the data types are not equivalent.

When used as a host-variable to pass data to RDMS, *blobname-data* must contain the data to be passed and *blobname-length* must contain the length of the data in bytes.

Alternate form of qualification:

If a host variable is declared within a group within the SQL declare section, normal UCOB qualification can be used to reference that particular variable. UCOB also allows embedded SQL statements to contain a host variable group name followed by a period and a data-name. The new name (A.B) is equivalent to using qualification (B of A). The alternate qualification method can only be used on two levels of qualification.

This can be illustrated by the following example:

```
 IDENTIFICATION DIVISION.
 PROGRAM-ID. ESQL-ALT-QUAL.
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER. UNISYS-2200.
 OBJECT-COMPUTER. UNISYS-2200.
 DATA DIVISION.
 WORKING-STORAGE SECTION.
*****************************************************************
*    EMBEDDED SQL HOST VARIABLE DECLARATION                     *
*****************************************************************
     EXEC SQL BEGIN DECLARE SECTION END-EXEC
 01 captain.
     02 qcode                   PIC X(2).
     EXEC SQL END DECLARE SECTION END-EXEC
*
 PROCEDURE DIVISION.
 BEGIN.
*
     EXEC SQL DECLARE c9 CURSOR FOR RANDOM FOR
     SELECT * FROM alpha.seed WHERE
     qcode < 'OZ'  END-EXEC.
*
     DISPLAY 'EXEC SQL OPEN c9 END-EXEC'.
     EXEC SQL OPEN c9 END-EXEC.
*
```

```
*****************************************************************
*    SQL FETCH OF A.B HOST VARIABLE                             *
*****************************************************************
     EXEC SQL FETCH FIRST c9 INTO :captain.qcode  END-EXEC.
*****************************************************************
*    SAME SQL FETCH WITH QUALIFICATION B of A HOST VARIABLE     *
*****************************************************************
     EXEC SQL FETCH FIRST c9 INTO :qcode of captain  END-EXEC.
     STOP RUN.
```

7.3.3. Passing Values from RDMS to Program Variables

RDMS stores data into the following types of host program variables:

- BINARY (signed or unsigned)
- BINARY-1 (signed or unsigned)
- BLOB
- COMP-1
- COMP-2
- COMP-5 (signed)
- PACKED-DECIMAL (signed or unsigned)
- Numeric DISPLAY or COMP
  - For embedded SQL, these data types can be signed (with any sign position) or unsigned.
  - For interpreted SQL where the receiving data item is signed, the sign must be a separate, leading character.
- Alphanumeric (can have the JUSTIFIED attribute)
- ALPHABETIC (can have the JUSTIFIED attribute)

  This data type is supported for embedded SQL only.
- Kanji variables
- Edited numeric arguments

  This data type is supported for embedded SQL only.
- Edited alphanumeric arguments

  This data type is supported for embedded SQL only.

RDMS does not support any attempt to store into a constant.

If the host program variable argument is an unsigned numeric item, RDMS strips any sign before storing a value into it.

You can store any data type in an alphanumeric or edited alphanumeric program variable argument. Numeric DISPLAY items are stored as a string with a decimal point inserted at the proper position.

You can store alphanumeric values (RDMS data type CHAR or NCHAR) only in alphabetic, alphanumeric, or edited alphanumeric program variable arguments.

For numeric variable arguments, the UCS Runtime System performs appropriate conversions if the data types are not equivalent.

When used as a host-variable to receive data from RDMS, *blobname-data* will contain the data passed from RDMS and *blobname-length* will contain the length of the data that was passed from RDMS. If the size of *blobname-data* is not large enough to receive the data passed, a truncation error will occur.

7.4. Coding Embedded SQL Commands

You can place embedded SQL commands directly into your program. The UCS COBOL compiler allows RDMS to parse these commands when the program is compiled.

The two types of embedded SQL commands, static and dynamic, affect the execution of your program in different ways. See the *Enterprise Relational Database Server for ClearPath OS 2200 SQL Programming Reference Manual* for a listing of the SQL commands you can embed in your program as well as a comparison of the static and dynamic commands.

7.4.1. Marking Embedded SQL Commands for the Compiler

Embedded SQL commands must appear in Area B of the COBOL source line. Except for commands that are compiler directives, you can embed SQL commands in the Procedure Division wherever a COBOL imperative statement is allowed. The format of the embedded commands is the following:

```
EXEC SQL sql-command END-EXEC.
```

where:

EXEC SQL

is the delimiter that defines the beginning of the embedded SQL command.

sql-command

is an embedded SQL command.

END-EXEC

is the delimiter that terminates the embedded command.

You can use the following embedded SQL commands without the beginning and ending delimiters:

- BEGIN THREAD
- END THREAD
- COMMIT
- GETERROR
- ROLLBACK

Each of the previous commands must be the only statement in a sentence. Thus, the sentence containing one of these commands must end with a period, as follows:

```
BEGIN THREAD T1
   FOR APPLICATION APPSVN READ.
GETERROR INTO T1ERR.
```

If you use these commands with the delimiters EXEC SQL and END-EXEC and you specify the keyword option LEVEL/SQL at compilation time, the compiler flags the statement as nonstandard. If you do not specify the keyword option LEVEL/SQL, the compiler does not issue a message.

7.4.2. Example of Using Embedded SQL Commands

Example 7–1 illustrates the use of embedded SQL commands and variables in a UCS COBOL program. See 7.1 for a description of the tables used in the program.

```
 IDENTIFICATION DIVISION.
 PROGRAM-ID.  DSPFAMESQL.
*****************************************************************
*****************************************************************
*                                                               *
* SIMPLE BATCH COBOL RDMS EMBEDDED SQL PROGRAM - DSPFAMESQL     *
*                                                               *
*        reads the input data                                   *
*        begins thread with UDS                                 *
*        retrieves the requested information from the database  *
*        displays this information on the printer               *
*        ends thread with UDS                                   *
*                                                               *
*****************************************************************
*****************************************************************
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER.  UNISYS-2200.
 SPECIAL-NAMES.
     PRINTER IS PRINTER.
 DATA DIVISION.
 WORKING-STORAGE SECTION.
```

```
*************************************************************
*    USER INPUT ERROR MESSAGE
*************************************************************
 01  INVALID-EMPLOYEE-NUMBER    PIC X(40)
       VALUE 'NO EMPLOYEE EXISTS WITH EMPLOYEE NUMBER '.
*************************************************************
*   VARIABLE TO HOLD EMPLOYEE NUMBER INPUT
*************************************************************
 01  EMPLOYEE-NUMBER    PIC X(5).
*************************************************************
*   UDS THREAD STATUS FLAG
*************************************************************
 01  THREAD-FLAG        PIC 9(1) VALUE 0.
*************************************************************
*   RELATIONAL DATABASE INFORMATION
*************************************************************
 01  INDR-LAST-NAME      PIC X(20).
 01  INDR-FIRST-NAME     PIC X(12).
 01  CHILD-LAST-NAME     PIC X(20).
 01  CHILD-FIRST-NAME    PIC X(12).
*************************************************************
*    ESQL STATEMENT STATUS
*************************************************************
 01  SQLSTATE            PIC X(5).
*************************************************************
*    RELATIONAL COMMAND ERROR BUFFERS
*************************************************************
     EXEC SQL BEGIN DECLARE SECTION END-EXEC.
 01  RDMCA.
      02 ERROR-STATUS  PIC 9999.
      02 AUX-INFO      PIC S9(9) USAGE BINARY.
     EXEC SQL END DECLARE SECTION END-EXEC.
*************************************************************
*    GETERROR BUFFERS
*************************************************************
 01  ERROR-TEXT.
     05  ERR             PIC X(132) OCCURS 5 TIMES.
*************************************************************
*    INDEX FOR GETERROR DISPLAYS
*************************************************************
 01  ERR-INDEX           PIC 9.
*
 PROCEDURE DIVISION.
*************************************************************
*    PROGRAM FLOW
*************************************************************
 MAIN-PROGRAM-FLOW.
     DISPLAY 'ENTER EMPLOYEE NUMBER' UPON PRINTER.
     ACCEPT EMPLOYEE-NUMBER.
*************************************************************
*    GENERAL ERROR PROCESSING FOR A NEGATIVE SQL ERROR
*************************************************************
```

```
     EXEC SQL
          WHENEVER SQLERROR GO TO RDMS-ERROR-PARA
     END-EXEC.
     PERFORM BEGIN-THREAD-TO-UDS.
     PERFORM SINGLETON-SELECT-INDRDATA.
     IF SQLSTATE =  02000
          DISPLAY INVALID-EMPLOYEE-NUMBER, EMPLOYEE-NUMBER
            UPON PRINTER
          PERFORM END-THREAD-TO-UDS
          PERFORM TERMINATION-PARA
     ELSE CONTINUE
     END-IF.
     PERFORM DISPLAY-INDIVIDUAL-DATA.
     PERFORM SETUP-CHILDDATA-CURSOR.
     PERFORM FETCH-FROM-CHILDDATA-CURSOR
          WITH TEST AFTER UNTIL SQLSTATE =  02000.
     PERFORM END-THREAD-TO-UDS.
     PERFORM TERMINATION-PARA.
*************************************************************
*    INITIALIZE WITH THE DATABASE SYSTEM
*    SET DEFAULT QUALIFIER ON ALL TABLES IN THIS THREAD
*************************************************************

BEGIN-THREAD-TO-UDS.
     BEGIN THREAD DSPDATA FOR APPSVN RETRIEVE.
     MOVE 1 TO THREAD-FLAG.
     EXEC SQL

      USE DEFAULT QUALIFIER FAMILY

     END-EXEC.
*************************************************************
*    SINGLETON SELECT OF THE INDIVIDUAL'S DATA
*************************************************************

  SINGLETON-SELECT-INDRDATA.
     EXEC SQL
        SELECT INDR_LAST_NAME, INDR_FIRST_NAME
            INTO :INDR-LAST-NAME, :INDR-FIRST-NAME
                        FROM INDIVIDUALR
                        WHERE INDIVIDUALR.INDR EMP_NUMBER = :EMPLOYEE-
NUMBER
     END-EXEC
*************************************************************
*    DISPLAY THE INDIVIDUALÆS INFORMATION TO THE PRINTER
*************************************************************
 DISPLAY-INDIVIDUAL-DATA.
     DISPLAY 'INDIVIDUAL DATA' UPON PRINTER.
     DISPLAY '  EMPLOYEE NUMBER IS '
     EMPLOYEE-NUMBER UPON PRINTER.
     DISPLAY '    LAST NAME: ' INDR-LAST-NAME UPON PRINTER.
     DISPLAY '    FIRST NAME: ' INDR-FIRST-NAME UPON PRINTER.
```

```
*************************************************************
*    DEFINE CURSOR TO BE USED FOR DATA RETRIEVAL
*    OPEN CURSOR
*************************************************************
 SETUP-CHILDDATA-CURSOR.
     EXEC SQL
          DECLARE CHILDDATA CURSOR
            SELECT CHILD_LAST_NAME, CHILD_FIRST_NAME
            FROM CHILD
            WHERE CHILD.CHILD-EMP-NUMBER = :EMPLOYEE-NUMBER
     END-EXEC.
     EXEC SQL
          OPEN CHILDDATA
     END-EXEC.
*************************************************************
*    RETRIEVE DATA FROM THE CURSOR
*************************************************************
 FETCH-FROM-CHILDDATA-CURSOR.
     EXEC SQL
          FETCH NEXT CHILDDATA INTO
          :CHILD-LAST-NAME, :CHILD-FIRST-NAME
     END-EXEC.
     IF SQLSTATE NOT =  '02000'
          DISPLAY '        CHILD-NAME: '  CHILD-FIRST-NAME, ' ',
          CHILD-LAST-NAME UPON PRINTER
     ELSE CONTINUE
     END-IF.
*************************************************************
*    TERMINATE WITH THE DATABASE SYSTEM
*************************************************************
 END-THREAD-TO-UDS.
     END THREAD.
     MOVE 0 TO THREAD-FLAG.
*************************************************************
*   TERMINATE PROGRAM
*************************************************************
 TERMINATION-PARA.
     DISPLAY 'PROGRAM HAS COMPLETED' UPON PRINTER.
     STOP RUN.
*
*
*************************************************************
*    RETURN AN ERROR MESSAGE ON THE PRINTER IF A
*    DATABASE ERROR OCCURS
*************************************************************
 RDMS-ERROR-PARA.
     EXEC SQL  WHENEVER SQLERROR CONTINUE  END-EXEC.
     DISPLAY '***SQLSTATE = ' SQLSTATE UPON PRINTER.
     DISPLAY '***RDMCA AUX INFO = ' AUX-INFO UPON PRINTER.
     DISPLAY '***ERROR STATUS = ' ERROR-STATUS UPON PRINTER.
     GETERROR INTO :ERR(1), :ERR(2), :ERR(3),  :ERR(4), :ERR(5).
     PERFORM VARYING ERR-INDEX FROM 1 BY 1 UNTIL ERR-INDEX = 6
```

```
            DISPLAY ERR (ERR-INDEX) UPON PRINTER
        END-PERFORM.
        PERFORM END-THREAD-TO-UDS.
        PERFORM TERMINATION-PARA.
```

Example 7–1. UCS COBOL Program Using Embedded SQL

7.5. Coding Interpreted SQL

To use the RDMS interpreter interface, embed SQL commands in your UCS COBOL program with the ENTER MASM statement. In this method, RDMS parses and processes SQL commands as the system encounters them during execution.

7.5.1. Calling the RDMS Interpreter Interface (ENTER MASM)

Use the following format of the ENTER MASM statement to call the interpreter interface, where variable names must occur sequentially in the order specified:

```
ENTER MASM 'ACOB$RDMR'
    USING sql-command
        error-status
        auxiliary-information
      [ program-variable-1
        program-variable-2
                .
                .
                .
        program-variable-n ] .
```

where:

sql-command

is a thread control or SQL command, coded in either of the following ways:

- As a string constant. This item contains the complete command and any required placeholder variables within apostrophes.
- As a COBOL variable. This is a nonnumeric DISPLAY item containing the complete command and any required placeholder variables within apostrophes.

error-status, auxiliary-information, and program-variable-n

are variables declared in the UCS COBOL program. See 7.2.3 for information on declaring these variables.

Example: BEGIN THREAD Command

To submit a BEGIN THREAD command to the interpreter interface from a UCS COBOL program, use the following ENTER MASM statement. The names of the application, variables, and thread names are applicable to this example only.

```
ENTER MASM 'ACOB$RDMR'
    USING 'BEGIN THREAD T1 FOR APPLICATION APPSVN READ; '
        ERRSTAT
        COLPOS.
*
    IF ERRSTAT NOT ZERO
        PERFORM ERRCHK.
```

In the previous example,

- ERRSTAT is the variable declared in the UCS COBOL program to receive the completion status.
- COLPOS is the variable declared in the UCS COBOL program to receive the auxiliary information about the outcome of the executed command.
- If the value of ERRSTAT is not zero, the program branches to an error-handling subprogram called ERRCHK.

7.5.2. Example of Using the Interpreter Interface

Example 7–2 illustrates how to use the interpreter interface and variables in a UCS COBOL program. See 7.1 for a description of the tables used in the program.

```
 IDENTIFICATION DIVISION.
 PROGRAM-ID.  DSPFAMSQL.
 DATE-WRITTEN.  JUNE 1990.
 DATE-COMPILED.
*REMARKS.
*****************************************************************
*****************************************************************
*                                                               *
* SIMPLE BATCH COBOL RDMS NONEMBEDDED SQL PROGRAM - DSPFAMSQL    *
*                                                               *
*        reads the input data                                   *
*        begins thread with UDS                                 *
*        retrieves the requested information from the database  *
*        displays this information on the printer               *
*        ends thread with UDS                                   *
*                                                               *
*****************************************************************
*****************************************************************
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER.  UNISYS-2200.
 OBJECT-COMPUTER.  UNISYS-2200.
 SPECIAL-NAMES.
```

```
    PRINTER IS PRINTER.
 DATA DIVISION.
 WORKING-STORAGE SECTION.
*************************************************************
*    USER INPUT ERROR MESSAGE
*************************************************************
 01  INVALID-EMPLOYEE-NUMBER    PIC X(40)
       VALUE 'NO EMPLOYEE EXISTS WITH EMPLOYEE NUMBER '.
*************************************************************
*    VARIABLE TO HOLD EMPLOYEE NUMBER INPUT
*************************************************************
 01  EMPLOYEE-NUMBER    PIC X(5).
*************************************************************
*    RELATIONAL DATABASE INFORMATION
*************************************************************
 01  INDR-LAST-NAME      PIC X(20).
 01  INDR-FIRST-NAME     PIC X(12).
 01  CHILD-LAST-NAME     PIC X(20).
 01  CHILD-FIRST-NAME    PIC X(12).
*************************************************************
*    RELATIONAL COMMAND LINE
*************************************************************
 01  RCOM-AREA.
     10  R-LINE         PIC X(40) OCCURS 30 TIMES.
*************************************************************
*    RELATIONAL COMMAND ERROR BUFFERS
*************************************************************
 01  ERROR-STATUS       PIC 9999.
 01  AUX-INFO           PIC S1(36) USAGE BINARY-1.

01  ERROR-TEXT.
     05  ERR-S          PIC X(132) OCCURS 5 TIMES.
*************************************************************
*    UDS THREAD STATUS FLAG
*************************************************************
 01  THREAD-FLAG        PIC 9(1) VALUE 0.
*************************************************************
*    INDEX FOR THE ERROR-DISPLAY PERFORM
*************************************************************
 01  ERR-INDEX          PIC 9.
*
 PROCEDURE DIVISION.
*************************************************************
*    PROGRAM FLOW
*************************************************************
 MAIN-PROGRAM-FLOW.
     DISPLAY 'ENTER EMPLOYEE NUMBER' UPON PRINTER.
     ACCEPT EMPLOYEE-NUMBER.
     PERFORM BEGIN-THREAD-TO-UDS.
     IF ERROR-STATUS = ZERO
        MOVE 1 TO THREAD-FLAG.
        PERFORM SET-DEFAULT-QUALIFIER.
     IF ERROR-STATUS = ZERO
        PERFORM SINGLETON-SELECT-INDRDATA.
     IF ERROR-STATUS = 6001
```

```
           DISPLAY INVALID-EMPLOYEE-NUMBER, EMPLOYEE-NUMBER
              UPON PRINTER
           PERFORM END-THREAD-TO-UDS
           PERFORM TERMINATION-PARA.
        IF ERROR-STATUS = ZERO
           PERFORM DISPLAY-INDIVIDUAL-DATA
           PERFORM DEFINE-CHILDDATA-CURSOR.
        IF ERROR-STATUS = ZERO
           PERFORM OPEN-CHILDDATA-CURSOR.
        IF ERROR-STATUS = ZERO
           PERFORM FETCH-FROM-CHILDDATA-CURSOR
           WITH TEST AFTER UNTIL ERROR-STATUS NOT = ZERO.
        IF ERROR-STATUS NOT = 6001 AND ERROR-STATUS NOT = ZERO
           PERFORM RDMS-ERROR-PARA.
        PERFORM END-THREAD-TO-UDS.
        PERFORM TERMINATION-PARA.
*********************************************************
*    INITIALIZE WITH THE DATABASE SYSTEM
*********************************************************
 BEGIN-THREAD-TO-UDS.
        ENTER MASM 'ACOB$RDMR' USING
            'BEGIN THREAD DSPDATA FOR APPSVN RETRIEVE ; '
            ERROR-STATUS, AUX-INFO.
*********************************************************
*    SET DEFAULT QUALIFIER ON ALL TABLES IN THIS THREAD
*********************************************************
 SET-DEFAULT-QUALIFIER.
        ENTER MASM 'ACOB$RDMR' USING
            'USE DEFAULT QUALIFIER FAMILY ; '
            ERROR-STATUS, AUX-INFO.
*********************************************************
*    SINGLETON SELECT OF THE INDIVIDUAL'S DATA
*********************************************************
 SINGLETON-SELECT-INDRDATA.
        MOVE SPACES TO RCOM-AREA.
        MOVE 'SELECT                                  ' TO R-LINE (1).
        MOVE ' INDR_LAST_NAME, INDR_FIRST_NAME        ' TO R-LINE (2).
        MOVE ' INTO $P1, $P2                          ' TO R-LINE (3).
        MOVE ' FROM INDIVIDUALR                       ' TO R-LINE (4).
        MOVE ' WHERE INDIVIDUALR.INDR_EMP_NUMBER      ' TO R-LINE (5).
        MOVE ' = $P3 ;                                ' TO R-LINE (6).
        ENTER MASM 'ACOB$RDMR' USING RCOM-AREA,
            ERROR-STATUS, AUX-INFO,
            INDR-LAST-NAME, INDR-FIRST-NAME, EMPLOYEE-NUMBER.
*********************************************************
*    DISPLAY THE INDIVIDUAL'S INFORMATION TO THE PRINTER
*********************************************************
 DISPLAY-INDIVIDUAL-DATA.
        DISPLAY 'INDIVIDUAL DATA' UPON PRINTER.
        DISPLAY '  EMPLOYEE NUMBER IS '  EMPLOYEE-NUMBER UPON PRINTER.
        DISPLAY '    LAST NAME:  ' INDR-LAST-NAME UPON PRINTER.
        DISPLAY '    FIRST NAME: ' INDR-FIRST-NAME UPON PRINTER.
*********************************************************
*    DEFINE CURSOR TO BE USED FOR DATA RETRIEVAL
*********************************************************

 DEFINE-CHILDDATA-CURSOR.
        MOVE SPACES TO RCOM-AREA.
```

```
        MOVE 'DECLARE CHILDDATA CURSOR              ' TO R-LINE (1).
        MOVE ' SELECT                               ' TO R-LINE (2).
        MOVE '    CHILD_LAST_NAME, CHILD_FIRST_NAME ' TO R-LINE (3).
        MOVE ' FROM CHILD                           ' TO R-LINE (4).
        MOVE ' WHERE CHILD.CHILD_EMP_NUMBER         ' TO R-LINE (5).
        MOVE ' = $P1 ;                              ' TO R-LINE (6).
        ENTER MASM 'ACOB$RDMR' USING RCOM-AREA,
            ERROR-STATUS, AUX-INFO, EMPLOYEE-NUMBER.
   ************************************************************
   *    OPEN CURSOR
   ************************************************************
    OPEN-CHILDDATA-CURSOR.
        ENTER MASM 'ACOB$RDMR' USING
            'OPEN CHILDDATA USING $P1 ; '
            ERROR-STATUS, AUX-INFO, EMPLOYEE-NUMBER.
   ************************************************************
   *    RETRIEVE DATA FROM THE CURSOR
   ************************************************************
    FETCH-FROM-CHILDDATA-CURSOR.
        ENTER MASM 'ACOB$RDMR' USING
            'FETCH NEXT CHILDDATA INTO $P1, $P2, ; '
            ERROR-STATUS, AUX-INFO,
            CHILD-LAST-NAME, CHILD-FIRST-NAME.
        IF ERROR-STATUS = ZERO
            DISPLAY '        CHILD NAME:  ' CHILD-FIRST-NAME, ' ',
                CHILD-LAST-NAME UPON PRINTER
        END-IF.

************************************************************
*    TERMINATE WITH THE DATABASE SYSTEM
************************************************************
 END-THREAD-TO-UDS.
     ENTER MASM 'ACOB$RDMR' USING
         'END THREAD ; '
          ERROR-STATUS, AUX-INFO.
     MOVE 0 TO THREAD-FLAG.
************************************************************
*    TERMINATE PROGRAM
************************************************************
 TERMINATION-PARA.
      DISPLAY 'PROGRAM HAS COMPLETED' UPON PRINTER.
      STOP RUN.
*
*
************************************************************
*    RETURN AN ERROR MESSAGE ON THE PRINTER IF A
*    DATABASE ERROR OCCURS
************************************************************
 RDMS-ERROR-PARA.
     DISPLAY 'ERROR STATUS = ' ERROR-STATUS UPON PRINTER.
     DISPLAY 'AUXILIARY INFORMATION = ' AUX-INFO UPON PRINTER.
     MOVE 'GETERROR INTO $P1, $P2, $P3, $P4, $P5 ; ' TO RCOM-AREA.
     ENTER MASM 'ACOB$RDMR' USING RCOM-AREA,
         ERROR-STATUS, AUX-INFO, ERR-S (1), ERR-S (2),
         ERR-S (3), ERR-S (4), ERR-S (5).
     PERFORM ERROR-DISPLAY
         VARYING ERR-INDEX FROM 1 BY 1 UNTIL ERR-INDEX = 6.
```

```
ERROR-DISPLAY.
    DISPLAY ERR-S (ERR-INDEX) UPON PRINTER.
```

Example 7–2. UCS COBOL Program Using the Interpreter Interface

7.6. Allocating and Initializing the RDMS Work Area (RSA$INIT)

RDMS provides the subroutine RSA$INIT, which allows you to allocate and initialize all the required space for the RDMS work area. Using RSA$INIT reduces the overhead of programs that require a large work area. Without the use of RSA$INIT, RDMS initially allocates 2,000 words of space for the work area, dynamically acquiring more space as it is needed. The overhead incurred to obtain this additional space is expensive and should be reduced.

To specify the size of the RDMS work area, do the following steps:

1. Determine the work area size required for your program (see 7.6.1).
2. Pass this size to RDMS with the RSA$INIT subroutine (see 7.6.2).

7.6.1. Determining the Work Area Size

To determine the size of the work area, temporarily code a DEBUG DUMP SIZES command (in either embedded or interpreted SQL format) immediately after the program's END THREAD command.

The following code shows the syntax for an embedded SQL DEBUG DUMP SIZES command in UCS COBOL:

```
EXEC SQL EXECUTE IMMEDIATE
     'DEBUG DUMP SIZES'
END-EXEC.
```

The following code shows the syntax for an interpreted SQL DEBUG DUMP SIZES command in UCS COBOL:

```
ENTER MASM 'ACOB$RDMR' USING
     'DEBUG DUMP SIZES; '
     ERROR-STATUS, AUX-INFO.
```

Compile and execute the program and check the output of the DEBUG DUMP SIZES command. It has the following format:

RSA Dbank Size is 13970 words.

Use the value listed in the call to RSA$INIT (see 7.6.2)

Note: *After determining the work area size, remove the DEBUG DUMP SIZES command from the program.*

See the *Enterprise Relational Database Server for ClearPath OS 2200 SQL Programming Reference Manual* for more information about the DEBUG DUMP SIZES command.

7.6.2. Calling RSA$INIT

Your program should call RSA$INIT only once, and should make the call before calling the subroutine ACOB$RDMR. If the default size of the work area (2,000 words) is sufficient for your program, no call is necessary in batch or demand mode.

The call to RDMS has the following format:

Format

```
CALL "RSA$INIT" [ USING work-area-size ]
```

where *work-area-size* specifies the number of words that RDMS allocates and initializes for the work area, as determined by the DEBUG DUMP SIZES command. You must specify this value as a data item with the following data description:

```
PICTURE 1(36) USAGE BINARY-1.
```

The value of *work-area-size* must be within the range 2,000 ≤ *work-area-size* ≤ 100,000. If you do not specify *work-area-size*, the default size of the work area is 2,000 words. If you specify a value less than 2,000, RDMS allocates 2,000 words of space. If you specify a value greater than 100,000, RDMS allocates 100,000 words of space.

7.7. Compiling Programs That Access RDMS

Compiling UCS COBOL programs containing SQL is straightforward. The following paragraphs describe the minor differences in compiling that exist for the two RDMS interfaces:

- Programs using the interpreter interface (the ENTER MASM statement)

  Simply call the UCS COBOL compiler. No special keyword options or preprocessing is necessary.

- Programs containing embedded SQL

  To access RDMS from the UCOB compilation, RDMS must be installed for the default application specified by the APPLICATION keyword option (UDSSRC).

If your program references table and view definitions contained in an application group other than the default (UDSSRC), you must identify the correct application group. To do this, specify the APPLICATION keyword option on the compiler call (see 2.2.7). The compiler uses the RDMS and repository software located in this application group repository when parsing the syntax. Programs with embedded SQL do not require any other keyword options or preprocessing.

The run unit used in any RDMS qualification of SQL names will be the name of the UCS COBOL source program.

Note: *You name application groups when you install them, using either the default names or others you select. See the* UDS Planning and Migration Overview *for information on installing application groups.*

Example 1

The following example shows a compiler call for a UCS COBOL program using SQL. If the program uses embedded SQL, it runs in the default application group, UDSSRC, since no APPLICATION keyword option is specified.

```
@UCOB,S qual*file.sql/src,qual*file.sql-prog/om
```

This call produces the object module qual*file.sql-prog/om. The run unit used by RDMS will be qual*file.sql/src.

Example 2

The following example is the same as Example 1, except that it compiles a UCS COBOL program using embedded SQL that must run in the application group APPSVN (specified by the APPLICATION keyword option).

```
@UCOB,S qual*file.sql/src,qual*file.sql-prog/om,,,APPLICATION/APPSVN
```

This call produces the object module qual*file.sql-prog/om, which contains the processed SQL statements. This object module is in a form ready for execution by RDMS at run time.

7.8. Linking Programs That Access RDMS

You can use either static or dynamic linking to link batch and demand programs containing SQL. You must, however, statically link Transaction Processing (TIP) and High-Volume Transaction Processing (HVTIP) programs containing SQL. Static linking provides better performance for all production environments.

When you use interpreted or embedded SQL in a program, RDMS uses the UCS C malloc function to acquire dynamic memory allocations. In batch/demand, the malloc space is acquired by creating an activity-level bank. For TIP, the malloc space is acquired from the URTS$TABLES heap bank. This is a small bank and once the bank is filled, another URTS$TABLES heap bank is created.

7.8.1. Statically Linking Applications

The CREATE$BANK call causes your transaction to lose its TIP sticking and TIP restart. To prevent this from occurring, the main program must use the keyword option TIPMALLOC on the compilation of the program. This results in the TIPMALLOC$$$ bank being included within your link. The use of the TIPMALLOC option allows TIP sticking and restart to function because a dynamic bank creation does not take place.

Using the keyword option TIPMALLOC on the compilation of the main program allows RDMS and any other subsystem that is called to acquire the memory heap space from the TIPMALLOC$$$ heap bank instead of the URTS$TABLES heap bank. A fast load zoom cannot be created with the TIPMALLOC option because the total (collective) size is more than 262K words. (This is not the actual size, but the potential growth size when the program requires more memory.)

The keyword TIPMALLOC can also be used for batch/demand executions, to eliminate a bank from being created at execution time. This would allow TIP sticking and restart to function because a dynamic bank creation does not take place.

TIPMALLOC is not allowed in HVTIP. Under HVTIP, C malloc space is acquired from the HVTIP D-bank, program level BDI 5. When heap space runs out, the malloc calls fail, returning NULL and the program execution fails. However, if the HVTIP ICP static link has an OUTPUT HVTIP2 statement instead of an OUTPUT HVTIP statement, C malloc space is then taken from separate activity level banks called "pooled" banks. If your static link and VTBUTL VALTAB settings also supply activity level pooled banks, those banks are then used for malloc space. If you do not supply any pooled banks, CREATE$BANK calls are done to dynamically acquire activity level banks which are used for C malloc space. Those system calls result in your transaction losing its TIP sticking and restart. See the *Transaction Processing Administration and Operations Reference Manual* for more details on how to supply extra banks to the transaction. See the *Linking System Programming Reference Manual* for a description of the OUTPUT HVTIP2 statement.

See the TIPMALLOC keyword explanation in 2.2.7.

To identify the library search chain for the application group, include the following command in the runstream:

```
@USE LINK$PF,SYS$LIB$*APP$n .
```

where *n* is the number of the application group in which the RDMS software resides.

Example

The following example is a batch or demand runstream that statically links a program containing SQL commands to the application group APP$7. It produces an efficient zero overhead object module (ZOOM).

```
@USE LINK$PF,SYS$LIB$*APP$7.
@LINK,S ,qual*file.sql-prog/zm
INCLUDE qual*file.sql-prog/om
RESOLVE ALL REFERENCES USING SYS$LIB$*EMOMRTS.,LCN
```

```
PROCESS FOR EXTENDED ZOOM
DELETE ALL DEFINITIONS EXCEPT START$
@EOF
```

Accessing a Relational Database

The following statements apply to the preceding example:

- The @USE LINK$PF statement ensures that the program uses the RDMS interface for the application group (APP$7 in this example). Any ESQL programs input to this static link must have been compiled for this application group using the matching APPLICATION/appname keyword option.
- The @LINK,S statement calls the LINK processor. Notice that the name used for the output object module of the static link is different from the compiler output object module. Otherwise, the statically linked object module overwrites the object module produced by the compiler. This is because standard object modules (the compiler output), bound object modules, and ZOOMs are all subtypes of an absolute; and the system allows only one version of an absolute to exist per name. Giving the statically linked output a unique name assures that the object module produced by the compiler remains available for debugging with PADS.
- The INCLUDE command following the call to the LINK processor specifies the object module produced by the compiler, which is the input to the Linking System.
- The RESOLVE command eliminates references for the Programmers Advanced Debugging System (PADS), greatly reducing ZOOM size and aiding performance.
- The PROCESS command produces the most efficiently banked ZOOM for this static link.
- The DELETE command deletes all unnecessary definitions (entry points), retaining only the required entry point, START$. This produces a ZOOM that executes efficiently in a production environment.

For more information about linking and the statements and commands used in this runstream, see the *Linking System Programming Reference Manual*.

7.8.2. Dynamically Linking Applications

The following runstream dynamically links a program containing SQL commands and executes it for the application group APP$7:

```
@USE LINK$PF,SYS$LIB$*APP$7.
@XQT   qual*file.sql-prog/om
```

The library search chain for the application group is identified through the @USE LINK$PF statement before execution (see 7.8.1). During execution, the Linking System dynamically resolves references as they are encountered.

Note that if any of the programs use Embedded SQL, those programs must have been compiled for this application group with a matching APPLICATION/appname keyword option. ESQL programs are targeted for a specific application group at compilation time.

7.9. Compile Time Explain Reports

RDMS explain reports can be generated adjacent to ESQL DECLARE CURSOR statements. The explain report shows the access path chosen by the RDMS optimizer for the specified query.

Compile time explain reports are a subset of the output of the execution time SQL EXPLAIN statement.

You can produce these reports by including the REMARK or SOURCE keyword option, or the E, S, or L letter option, on the compiler call. Suppress the reports by including the NO-REMARK keyword option or the N option on the compiler call.

Section 8
Accessing the Unisys Repository Manager (UREP)

The Unisys Repository Manager (UREP) is the product that you directly use to create, update, and store definitions for the following types of Universal Data System (UDS) databases:

- Network (DMS)
- Relational (RDMS)
- Conventional file (SFS)

In addition to creating and storing database definitions, you can use UREP directly from your UCS COBOL programs to perform the following functions:

- Insert code into your UCS COBOL source program with the INVOKE command. The INVOKE command causes UREP to generate file descriptions, data declarations, arbitrary source code sequences, and screen definitions from definitions that you previously created and stored in UREP. UREP places the generated code in the source program at the point of the INVOKE command.
- Store cross-reference information into UREP, using the UREP-XREF keyword option, to track definitions that UREP or COPY procedures insert into the UCS COBOL source program.

UCS COBOL provides the following features that allow you to use information from and store information into UREP:

- The INVOKE clause
- The UREP-XREF compiler keyword option

The INVOKE clause and UREP-XREF keyword option have interrelated functions. The INVOKE clause causes information that you previously defined and stored in UREP to be processed, producing source code for your program.

For example, you create and store the definition for a File Description using UREP. During compilation, an INVOKE clause references the definition stored in UREP. At the point that the INVOKE clause occurs, UREP generates a File Description from information stored in a repository and then inserts it into your source program.

The UREP-XREF keyword option has a related function. It processes the source code in your program and stores cross-reference information in the repository. This allows you to track and control changes to your organization's common source code. For example, when you change the File Description that UREP inserts into your program, you can use the UREP-XREF keyword option to store information in UREP about what programs reference the stored definition.

Thus, you can use information stored in UREP to generate source code for your program and you can use the UREP-XREF keyword option to track the use of the generated code and store it in UREP.

The type and detail of cross-reference information placed into the repository depend on a UREP system-defined entity and its associated attribute values (system-defined entity equals Options) that are assigned.

See the *Enterprise Servers Repository for ClearPath OS 2200 Programming Reference Manual* and the *Enterprise Servers Repository for ClearPath OS 2200 Administration Guide* for more information about setting up your code definitions and storing cross-reference information.

8.1. Specifying an Application Group

When creating definitions with UREP, you can store information for a related group of programs in the same repository. Each repository is identified by an application name that defines which repository contains the information needed for the INVOKE clauses in a UCS COBOL source program.

Designate which repository to use in one of the following ways:

- Specify the compiler keyword option APPLICATION/*name* on the call to the compiler (see 2.2.7).
- Do not use an APPLICATION keyword option or a DMS INVOKE statement, causing the application name to default to the default repository, UDSSRC.

8.2. Generating Code for Your Source Program (INVOKE)

You can store common attributes of your UCS COBOL applications in UREP entities. The INVOKE...FROM REPOSITORY clause uses information that is stored in UREP and generates it into UCS COBOL declarations and definitions. UREP places the generated code into the source program at the point of the INVOKE clause. The compiler then treats the INVOKE clause like a comment line (similar to the treatment of a COPY statement).

The key steps that UREP requires for a program to use the INVOKE clause are as follows:

1. Create an entity in a UREP repository (using the UREP processor or the FORMGEN processor to define DPS forms) that defines the information pertaining to your application. The *Enterprise Servers Repository for ClearPath OS 2200 Administration Guide* describes this process.
2. Use the INVOKE statement in your program. This causes UREP at compilation time to generate the stored information into source code that UREP then inserts into your program at the point of the INVOKE statement (see 8.2.1, 8.2.2, and 8.2.3).
3. At compilation time, specify the APPLICATION keyword option to designate which UREP application name to associate with the INVOKE clause (see 8.1).

The following subsections describe the different types of code that you can generate by placing INVOKE statements in a UCS COBOL program.

8.2.1. Inserting SELECT Statement Information with INVOKE

Function

You can generate a SELECT statement in the File-Control Paragraph of your source program from the definition of a SELECT statement that is stored in UREP. The following syntax for the INVOKE clause causes UREP to generate and place this code in your source program.

Format

```
INVOKE SELECT [ file-name-2 = ]  file-name-1
     FROM REPOSITORY [ ASSIGN TO { implementor-name-1 }... ] .
```

where:

file-name-1

is the source code being incorporated into the program. This file must exist in UREP.

file-name-2

provides an optional alias for *file-name-1*; if you specify *file-name-2*, use it to reference the file in the SELECT statement in the FILE SECTION and the PROCEDURE SECTION (including references to a file name in the INVOKE ALL FILE...FROM REPOSITORY statement).

implementor-name-1

is a file name, 1 to 12 characters long, that corresponds to a file name on an @ASG statement. You can use this variable to assign *file-name-1* to as many devices as needed. The ASSIGN clause must be present on the INVOKE SELECT statement to use *implementor-name-1*.

Apart from the optional ASSIGN clause, you specify all other information for the SELECT statement, including ORGANIZATION, ACCESS, and KEY, when you create the definition with UREP.

Result

The INVOKE statement generates a SELECT statement, with its appropriate clauses, from the associated information in UREP.

If the compiler keyword option UREP-XREF is in effect for the compilation, UREP tracks the use of *file-name-2*; UREP, however, always stores the information using the actual file name, *file-name-1*.

Example

Example 8–1 shows the results of generating a SELECT statement with the INVOKE statement.

```
21           ************************************************************
22           * INVOKE SELECT
23           ************************************************************
24           *
25             invoke select options1-f1 from repository.
25.C1.1            SELECT OPTIONS1-F1 ASSIGN TO DISC.
26           *
27           ************************************************************
28           * INVOKE SELECT
29           ************************************************************
30           *
31             invoke select options1-f1a = options1-f1 from repository.
31.C1.1            SELECT OPTIONS1-F1 ASSIGN TO DISC.
32           *
33           ************************************************************
34           * INVOKE SELECT
35           ************************************************************
36           *
37             invoke select options1-f2 from repository assign to exfile.
37.C1.1            SELECT OPTIONS1-F2 ASSIGN TO DISC.
38           *
```

Example 8–1. Invoking a SELECT Statement

8.2.2. Inserting File Descriptions with INVOKE

Function

If a file has an INVOKE...FROM REPOSITORY clause in the SELECT statement of the source program (see 8.2.1), you must generate its FILE DESCRIPTION in the FILE SECTION using the following format. Note that you cannot use any of the other clauses associated with a FILE DESCRIPTION if it is generated with the INVOKE ALL FILE... FROM REPOSITORY statement.

Format

```
INVOKE ALL FILE file-name-1 FROM REPOSITORY .
```

where *file-name-1* is the file name or the alias file name if one was given (see 8.2.1) that is specified in the INVOKE SELECT clause.

Result

The INVOKE statement causes UREP to generate a complete File Description (FD or SD) and all Record Descriptions that define the specified file, and place them into the source program.

The following example shows the results of generating a SELECT statement with the INVOKE statement.

Example

Example 8–2 shows the results of generating a File Description with the INVOKE statement.

```
40                              file section.
41                             *
42                             ****************************************
43                             * INVOKE FILE
44                             ****************************************
45                             *
46                              invoke all file options1-f1 from
repository.
46.C1.1                         FD  OPTIONS1-F1.
46.C1.2                        *    DATA RECORD IS OPTIONS1-F1-R1
46.C1.3                         01  OPTIONS1-F1-R1.
46.C1.4                             02  FILE-NAME PIC x(12).
46.C1.5                             02  FRECORD-NUMBER PIC 9(6).
46.C1.6                             02  FPROGRAM-NAME PIC x(12).
47                             *
48                             ****************************************
49                             * INVOKE FILE
50                             ****************************************
51                             *
52                              invoke all file options1-f1a from
repository.
52.C1.1                         FD  OPTIONS1-F1.
52.C1.2                        *    DATA RECORD IS OPTIONS1-F1-R1
52.C1.3                         01  OPTIONS1-F1-R1.
52.C1.4                             02  FILE-NAME PIC x(12).
52.C1.5                             02  FRECORD-NUMBER PIC 9(6).
52.C1.6                             02  FPROGRAM-NAME PIC x(12).
53                             *
54                             ****************************************
55                             * INVOKE FILE
56                             ****************************************
57                             *
58                              invoke all file options1-f2 from
repository.
58.C1.1                         FD  OPTIONS1-F2.
58.C1.2                        *    DATA RECORDS ARE OPTIONS1-F2-R1
OPTIONS1-F2-R2
58.C1.3                         01  OPTIONS1-F2-R1.
58.C1.4                             02  F2-DATA1 PIC x(12).
58.C1.5                             02  FILLER PIC x(8).
58.C1.6                             02  F2-DATA2 PIC 9(8).
58.C1.7                         01  OPTIONS1-F2-R2.
58.C1.8                             02  F2-DATA3 PIC x(12).
58.C1.9                             02  FILLER PIC x(8).
58.C1.10                            02  F2-DATA4 PIC 9(8).
59                             *
```

Example 8–2. Invoking a File Description.

8.2.3. Inserting Data Descriptions with INVOKE

Function

You can use an INVOKE statement to generate descriptions for the following:

- COBOL data items
- Relational data
- Screen definitions

Use these data descriptions in these sections of the Data Section:

- WORKING-STORAGE SECTION
- COMMON-STORAGE SECTION
- LINKAGE SECTION

Result

UREP generates the correct description clauses within the UCS COBOL program for the physical location of the INVOKE statement. The line numbering for the new source lines is the same as for a COPY statement.

The INVOKE statement uses the formats described in the following subsections.

Inserting COBOL Data Item Definitions

Format

```
INVOKE  { RECORD [ data-name-2 = ] data-name-1  } FROM REPOSITORY .
```

where:

data-name-1

must exist as a definition of a level 01 record or a level 77 item in a UREP repository. You cannot use the INVOKE RECORD clause if you use an INVOKE ALL FILE statement to generate the file description for the file (see 8.2.2).

data-name-2

provides an optional alias for *data-name-1*; if you specify *data-name-2*, use it for references to the data name in the FILE SECTION and the PROCEDURE SECTION of the source program. If the compiler keyword option UREP-XREF is in effect for the compilation, UREP tracks the use of *data-name-2*. UREP, however, always stores the information using the actual data name, *data-name-1*.

Example

Example 8–3 shows the results of generating record definitions with the INVOKE statement.

```
93              ************************************************************
94              *  INVOKE RECORD
95              ************************************************************
96              *
97                invoke record options1-r1 from repository.
97.C1.1           01  OPTIONS1-R1.
97.C1.2               02  OPTIONS1-R1-D1 OCCURS 10 times.
97.C1.3                   03  RECORD-NAME PIC x(12).
97.C1.4                   03  RECORD-NUMBER PIC 9(6).
97.C1.5                   03  RPROGRAM-NAME PIC x(12).
98              *
99              ************************************************************
100             *  INVOKE RECORD
101             ************************************************************
102             *
103              invoke record alt-file-rec = options1-f2-r1 from repository.
103.C1.1            01  OPTIONS1-F2-R1.
103.C1.2                02  F2-DATA1 PIC x(12).
103.C1.3                02  FILLER PIC x(8).
103.C1.4                02  F2-DATA2 PIC 9(8).
104             *
```

Example 8–3. Invoking Data Descriptions

Inserting Relational Data Definitions

Format



where:

table-name

is a user-defined word that identifies a repository definition for a record description of an RDMS relational table. You can specify *table-name* only after the TABLE subclause. Underscores and dollar signs are allowed in this name.

view-name

is a user-defined word that identifies a repository definition for a record description that describes the columns in an RDMS relational view. You can specify *view-name* only after the VIEW subclause. Underscores and dollar signs are allowed in this name.

schema-name

is a user-defined word that identifies the UREP definition of a schema that contains the description for the table or view. Underscores and dollar signs are allowed in this name.

Result

The INVOKE statement generates source variable declarations for all column names associated with a table definition. Each column name is assigned a 01 level declaration with its associated data type and size assigned as the object. The column name defined in the table definition becomes the name of the variable (column underscores are consistent with COBOL hyphens).

The compiler then places these variables into the repository as cross-referenced data-items only if they are referenced by embedded SQL statements in the procedure division. If they are not referenced by embedded SQL statements, they are not cross-referenced (that is, source generation of these variable declarations through the INVOKE statement does not generate cross-referencing for the columns brought into the source).

For more information about relational cross-reference information, see the *Enterprise Servers Repository for ClearPath OS 2200 Administration Guide*, the *Enterprise Relational Database Server for ClearPath OS 2200 Administration Guide*, and the *Enterprise Relational Database Server for ClearPath OS 2200 SQL Programming Reference Manual*.

Example

Example 8–4 shows the results of generating definitions of relational tables and views with the INVOKE statement.

```
107             ********************************************************
108             *  INVOKE TABLE
109             ********************************************************
110             *
111              invoke table table1 in ureptest from repository.
111.C1.1         01  TABLE-NAME PIC X(12).
111.C1.2         01  TRECORD-NUMBER PIC S9(5) SIGN LEADING SEPARATE.
111.C1.3         01  TPROGRAM-NAME PIC X(12).
112             *
113             ********************************************************
114             *  INVOKE VIEW
115             ********************************************************
116             *
117              invoke view view1 in ureptest from repository.
117.C1.1         01  VIEW-NAME PIC X(12).
117.C1.2         01  VRECORD-NUMBER PIC S9(5) SIGN LEADING SEPARATE.
117.C1.3         01  VPROGRAM-NAME PIC X(12).
118             *
```

Example 8–4. Invoking Table and View Descriptions

Inserting Screen Definitions

Format

$$\text{INVOKE}\left\{\text{FORM}\left\{\begin{matrix}\text{OF}\\\text{IN}\end{matrix}\right\}\text{FORMLIB}\ formlib-name\left[\text{WITH}\left\{\begin{matrix}\text{VALUES}\\\text{PREFIXES}\\\text{REDEFINES}\\\text{NONE}\end{matrix}\right\}\right]\right\}\text{FROM REPOSITORY.}$$

where:

form-desc

is a user-defined word that identifies a repository definition for a collection of data descriptions that describe a screen format. You can specify more than one attribute (VALUES, PREFIXES, REDEFINES, NONE) on the WITH clause; however, do not use the same value more than once in the same INVOKE statement. Also, do not use the attributes REDEFINES and VALUE in the same INVOKE statement.

If you do not specify the WITH subclause, VALUES and PREFIXES are the defaults. Underscores and dollar signs are allowed in this name.

formlib-name

is a user-defined word that identifies the UREP definition of a form library that contains the *form-desc*. Underscores and dollar signs are allowed in this name.

Inserting Screen Definitions with the INVOKE Statement

Example 8–5 shows the results of generating a screen definition with the INVOKE statement. This form was created without initial values and without repeating images. Default initial values are given as follows:

- Alphabetic and alphanumeric data items with spaces
- Numeric data items with zero

The program shows two ways of invoking the form, both of which have the WITH clause. The first has WITH VALUES PREFIXES (the default WITH clause). The second has only WITH VALUES. Currently, VALUES is required in the WITH clause. Without PREFIXES in the WITH clause, all defined images that the program uses will begin with 'S1-'.

```
40          invoke form lspi1-1 in formlib forms
41                      with values prefixes from repository.
41.C1.1     01  SCREEN-LSPI1-1.
41.C1.2         02  SCREEN-LSPI1-1-HEADER.
41.C1.3             05  FILLER PIC X(2) VALUE SPACES.
41.C1.4             05  FILLER PIC 9(5)  BINARY VALUE ZERO.
41.C1.5             05  FILLER PIC X(4) VALUE SPACES.
41.C1.6             05  S1-FILLER PIC X(8) VALUE '$DPS$SWS'.
41.C1.7              05  S1-NUMBER PIC 9(4) VALUE 1.
41.C1.8              05  S1-NAME PIC X(8)  VALUE 'LSPI1'.
41.C1.9              05  S1-CHECK-NUMBER PIC 9(10) BINARY VALUE 43807.
```

Example 8–5. Inserting Screen Definitions with the INVOKE Statement

```
41.C1.10            05  S1-FCA-LENGTH PIC 9(5)  BINARY VALUE 32.
41.C1.11            05  S1-FCA-COUNT PIC 9(5)  BINARY VALUE 8.
41.C1.12            05  FILLER PIC 9(2)  BINARY VALUE 24.
41.C1.13            05  FILLER PIC 9(2)  BINARY VALUE 0.
41.C1.14            05  S1-NUMBER-ROWS PIC 9(2)  BINARY VALUE 24.
41.C1.15            05  S1-NUMBER-COLS PIC 9(2)  BINARY VALUE 80.
41.C1.16            05  FILLER PIC 9(2)  BINARY VALUE 5.
41.C1.17            05  FILLER PIC 9(2)  BINARY VALUE 0.
41.C1.18            05  FILLER PIC 9(2)  BINARY VALUE ZERO.
41.C1.19            05  S1-QUERY-STATUS PIC 9(2)  BINARY VALUE 0.
41.C1.20            05  FILLER PIC 9(5)  BINARY VALUE 56.
41.C1.21            05  S1-WSDATA-LENGTH PIC 9(5)  BINARY VALUE 12.
41.C1.22            05  S1-SOE-YCO PIC 9(2)  BINARY VALUE ZERO.
41.C1.23            05  S1-SOE-XCO PIC 9(2)  BINARY VALUE ZERO.
41.C1.24            05  S1-FUNCTIONKEY PIC 9(5)  BINARY VALUE ZERO.
41.C1.25            05  S1-OUT-YCO PIC 9(2)  BINARY VALUE 9.
41.C1.26            05  S1-OUT-XCO PIC 9(2)  BINARY VALUE 16.
41.C1.27            05  S1-OUT-FID PIC 9(5)  BINARY VALUE 4.
41.C1.28            05  S1-IN-YCO PIC 9(2)  BINARY VALUE 0.
41.C1.29            05  S1-IN-XCO PIC 9(2)  BINARY VALUE 0.
41.C1.30            05  S1-IN-FID PIC 9(5)  BINARY VALUE 0.
41.C1.31            05  S1-VALID PIC X(1) VALUE 'V'.
41.C1.32            05  S1-DYNAMIC PIC X(1) VALUE 'D'.
41.C1.33            05  S1-BACKGROUND PIC X(1) VALUE 'E'.
41.C1.34            05  S1-FOREGROUND PIC X(1) VALUE 'W'.
41.C1.35            05  S1-INTENSITY PIC X(1) VALUE 'N'.
41.C1.36            05  S1-HIGHLIGHT PIC X(1) VALUE 'N'.
41.C1.37            05  S1-FONT PIC X(1) VALUE 'N'.
41.C1.38            05  S1-EMPHASIS PIC X(1) VALUE 'N'.
41.C1.39            05  S1-CNUM-VALIDATE PIC X(1) VALUE 'Y'.
41.C1.40            05  FILLER PIC X(3) VALUE SPACES.
41.C1.41            05  S1-SCREENFILE PIC X(12) VALUE 'FORMS'.
41.C1.42            05  FILLER PIC X(12) VALUE SPACES.
41.C1.43        02  SCREEN-LSPI1-1-FCA.
41.C1.44            05  S1-IMAGE3-STAT PIC 9(2)  BINARY VALUE 0.
41.C1.45            05  S1-IMAGE3-TYPE PIC 9(2)  BINARY VALUE 1.
41.C1.46            05  S1-IMAGE3-YCO PIC 9(2)  BINARY VALUE 6.
41.C1.47            05  S1-IMAGE3-XCO PIC 9(2)  BINARY VALUE 18.
41.C1.48            05  S1-IMAGE3-CHAR PIC 9(5)  BINARY VALUE ZERO.
41.C1.49            05  S1-IMAGE3-FID PIC 9(5)  BINARY VALUE 1.
41.C1.50            05  S1-IMAGE3-VAL PIC X(1) VALUE 'V'.
41.C1.51            05  S1-IMAGE3-DYN PIC X(1) VALUE 'U'.
41.C1.52            05  S1-IMAGE3-BACK PIC X(1) VALUE 'E'.
41.C1.53            05  S1-IMAGE3-FORE PIC X(1) VALUE 'W'.
41.C1.54            05  S1-IMAGE3-INT PIC X(1) VALUE 'N'.
41.C1.55            05  S1-IMAGE3-HIGH PIC X(1) VALUE 'N'.
41.C1.56            05  S1-IMAGE3-FONT PIC X(1) VALUE 'N'.
41.C1.57            05  S1-IMAGE3-EMPH PIC X(1) VALUE 'N'.
41.C1.58            05  S1-IMAGE4-STAT PIC 9(2)  BINARY VALUE 0.
41.C1.59            05  S1-IMAGE4-TYPE PIC 9(2)  BINARY VALUE 2.
41.C1.60            05  S1-IMAGE4-YCO PIC 9(2)  BINARY VALUE 7.
41.C1.61            05  S1-IMAGE4-XCO PIC 9(2)  BINARY VALUE 21.
41.C1.62            05  S1-IMAGE4-CHAR PIC 9(5)  BINARY VALUE ZERO.
41.C1.63            05  S1-IMAGE4-FID PIC 9(5)  BINARY VALUE 2.
41.C1.64            05  S1-IMAGE4-VAL PIC X(1) VALUE 'V'.
41.C1.65            05  S1-IMAGE4-DYN PIC X(1) VALUE 'D'.
41.C1.66            05  S1-IMAGE4-BACK PIC X(1) VALUE 'E'.
41.C1.67            05  S1-IMAGE4-FORE PIC X(1) VALUE 'W'.
```

Example 8–5. Inserting Screen Definitions with the INVOKE Statement (contd.)

```
41.C1.68            05  S1-IMAGE4-INT PIC X(1) VALUE 'N'.
41.C1.69            05  S1-IMAGE4-HIGH PIC X(1) VALUE 'N'.
41.C1.70            05  S1-IMAGE4-FONT PIC X(1) VALUE 'N'.
41.C1.71            05  S1-IMAGE4-EMPH PIC X(1) VALUE 'N'.
41.C1.72            05  S1-IMAGE5-STAT PIC 9(2)  BINARY VALUE 0.
41.C1.73            05  S1-IMAGE5-TYPE PIC 9(2)  BINARY VALUE 1.
41.C1.74            05  S1-IMAGE5-YCO PIC 9(2)  BINARY VALUE 8.
41.C1.75            05  S1-IMAGE5-XCO PIC 9(2)  BINARY VALUE 21.
41.C1.76            05  S1-IMAGE5-CHAR PIC 9(5)  BINARY VALUE ZERO.
41.C1.77            05  S1-IMAGE5-FID PIC 9(5)  BINARY VALUE 3.
41.C1.78            05  S1-IMAGE5-VAL PIC X(1) VALUE 'V'.
41.C1.79            05  S1-IMAGE5-DYN PIC X(1) VALUE 'U'.
41.C1.80            05  S1-IMAGE5-BACK PIC X(1) VALUE 'E'.
41.C1.81            05  S1-IMAGE5-FORE PIC X(1) VALUE 'W'.
41.C1.82            05  S1-IMAGE5-INT PIC X(1) VALUE 'N'.
41.C1.83            05  S1-IMAGE5-HIGH PIC X(1) VALUE 'N'.
41.C1.84            05  S1-IMAGE5-FONT PIC X(1) VALUE 'N'.
41.C1.85            05  S1-IMAGE5-EMPH PIC X(1) VALUE 'N'.
41.C1.86            05  S1-IMAGE6-STAT PIC 9(2)  BINARY VALUE 0.
41.C1.87            05  S1-IMAGE6-TYPE PIC 9(2)  BINARY VALUE 1.
41.C1.88            05  S1-IMAGE6-YCO PIC 9(2)  BINARY VALUE 9.
41.C1.89            05  S1-IMAGE6-XCO PIC 9(2)  BINARY VALUE 16.
41.C1.90            05  S1-IMAGE6-CHAR PIC 9(5)  BINARY VALUE ZERO.
41.C1.91            05  S1-IMAGE6-FID PIC 9(5)  BINARY VALUE 4.
41.C1.92            05  S1-IMAGE6-VAL PIC X(1) VALUE 'V'.
41.C1.93            05  S1-IMAGE6-DYN PIC X(1) VALUE 'U'.
41.C1.94            05  S1-IMAGE6-BACK PIC X(1) VALUE 'E'.
41.C1.95            05  S1-IMAGE6-FORE PIC X(1) VALUE 'W'.
41.C1.96            05  S1-IMAGE6-INT PIC X(1) VALUE 'N'.
41.C1.97            05  S1-IMAGE6-HIGH PIC X(1) VALUE 'N'.
41.C1.98            05  S1-IMAGE6-FONT PIC X(1) VALUE 'N'.
41.C1.99            05  S1-IMAGE6-EMPH PIC X(1) VALUE 'N'.
41.C1.100           05  S1-IMAGE7-STAT PIC 9(2)  BINARY VALUE 0.
41.C1.101           05  S1-IMAGE7-TYPE PIC 9(2)  BINARY VALUE 1.
41.C1.102           05  S1-IMAGE7-YCO PIC 9(2)  BINARY VALUE 10.
41.C1.103           05  S1-IMAGE7-XCO PIC 9(2)  BINARY VALUE 16.
41.C1.104           05  S1-IMAGE7-CHAR PIC 9(5)  BINARY VALUE ZERO.
41.C1.105           05  S1-IMAGE7-FID PIC 9(5)  BINARY VALUE 5.
41.C1.106           05  S1-IMAGE7-VAL PIC X(1) VALUE 'V'.
41.C1.107           05  S1-IMAGE7-DYN PIC X(1) VALUE 'U'.
41.C1.108           05  S1-IMAGE7-BACK PIC X(1) VALUE 'E'.
41.C1.109           05  S1-IMAGE7-FORE PIC X(1) VALUE 'W'.
41.C1.110           05  S1-IMAGE7-INT PIC X(1) VALUE 'N'.
41.C1.111           05  S1-IMAGE7-HIGH PIC X(1) VALUE 'N'.
41.C1.112           05  S1-IMAGE7-FONT PIC X(1) VALUE 'N'.
41.C1.113           05  S1-IMAGE7-EMPH PIC X(1) VALUE 'N'.
41.C1.114           05  S1-IMAGE8-STAT PIC 9(2)  BINARY VALUE 0.
41.C1.115           05  S1-IMAGE8-TYPE PIC 9(2)  BINARY VALUE 1.
41.C1.116           05  S1-IMAGE8-YCO PIC 9(2)  BINARY VALUE 11.
41.C1.117           05  S1-IMAGE8-XCO PIC 9(2)  BINARY VALUE 13.
41.C1.118           05  S1-IMAGE8-CHAR PIC 9(5)  BINARY VALUE ZERO.
41.C1.119           05  S1-IMAGE8-FID PIC 9(5)  BINARY VALUE 6.
41.C1.120           05  S1-IMAGE8-VAL PIC X(1) VALUE 'V'.
41.C1.121           05  S1-IMAGE8-DYN PIC X(1) VALUE 'U'.
41.C1.122           05  S1-IMAGE8-BACK PIC X(1) VALUE 'E'.
41.C1.123           05  S1-IMAGE8-FORE PIC X(1) VALUE 'W'.
41.C1.124           05  S1-IMAGE8-INT PIC X(1) VALUE 'N'.
41.C1.125           05  S1-IMAGE8-HIGH PIC X(1) VALUE 'N'.
```

Example 8–5. Inserting Screen Definitions with the INVOKE Statement (contd.)

```
41.C1.126           05  S1-IMAGE8-FONT PIC X(1) VALUE 'N'.
41.C1.127           05  S1-IMAGE8-EMPH PIC X(1) VALUE 'N'.
41.C1.128           05  S1-IMAGE9-STAT PIC 9(2)  BINARY VALUE 0.
41.C1.129           05  S1-IMAGE9-TYPE PIC 9(2)  BINARY VALUE 1.
41.C1.130           05  S1-IMAGE9-YCO PIC 9(2)  BINARY VALUE 12.
41.C1.131           05  S1-IMAGE9-XCO PIC 9(2)  BINARY VALUE 20.
41.C1.132           05  S1-IMAGE9-CHAR PIC 9(5)  BINARY VALUE ZERO.
41.C1.133           05  S1-IMAGE9-FID PIC 9(5)  BINARY VALUE 7.
41.C1.134           05  S1-IMAGE9-VAL PIC X(1) VALUE 'V'.
41.C1.135           05  S1-IMAGE9-DYN PIC X(1) VALUE 'U'.
41.C1.136           05  S1-IMAGE9-BACK PIC X(1) VALUE 'E'.
41.C1.137           05  S1-IMAGE9-FORE PIC X(1) VALUE 'W'.
41.C1.138           05  S1-IMAGE9-INT PIC X(1) VALUE 'N'.
41.C1.139           05  S1-IMAGE9-HIGH PIC X(1) VALUE 'N'.
41.C1.140           05  S1-IMAGE9-FONT PIC X(1) VALUE 'N'.
41.C1.141           05  S1-IMAGE9-EMPH PIC X(1) VALUE 'N'.
41.C1.142           05  S1-IMAGE10-STAT PIC 9(2)  BINARY VALUE 0.
41.C1.143           05  S1-IMAGE10-TYPE PIC 9(2)  BINARY VALUE 1.
41.C1.144           05  S1-IMAGE10-YCO PIC 9(2)  BINARY VALUE 13.
41.C1.145           05  S1-IMAGE10-XCO PIC 9(2)  BINARY VALUE 21.
41.C1.146           05  S1-IMAGE10-CHAR PIC 9(5)  BINARY VALUE ZERO.
41.C1.147           05  S1-IMAGE10-FID PIC 9(5)  BINARY VALUE 8.
41.C1.148           05  S1-IMAGE10-VAL PIC X(1) VALUE 'V'.
41.C1.149           05  S1-IMAGE10-DYN PIC X(1) VALUE 'U'.
41.C1.150           05  S1-IMAGE10-BACK PIC X(1) VALUE 'E'
41.C1.151           05  S1-IMAGE10-FORE PIC X(1) VALUE 'W'.
41.C1.152           05  S1-IMAGE10-INT PIC X(1) VALUE 'N'.
41.C1.153           05  S1-IMAGE10-HIGH PIC X(1) VALUE 'N'.
41.C1.154           05  S1-IMAGE10-FONT PIC X(1) VALUE 'N'.
41.C1.155           05  S1-IMAGE10-EMPH PIC X(1) VALUE 'N'.
41.C1.156       02  SCREEN-LSPI1-1-FCA-END REDEFINES SCREEN-LSPI1-1-FCA.
41.C1.157           05  SCREEN-LSPI1-1-FCA-REDEF OCCURS 8 TIMES.
41.C1.158               15  S1-FCA-STAT PIC 9(2)  BINARY.
41.C1.159               15  S1-FCA-TYPE PIC 9(2)  BINARY.
41.C1.160               15  S1-FCA-YCO PIC 9(2)  BINARY.
41.C1.161               15  S1-FCA-XCO PIC 9(2)  BINARY.
41.C1.162               15  S1-FCA-CHAR PIC 9(5)  BINARY.
41.C1.163               15  S1-FCA-FID PIC 9(5)  BINARY.
41.C1.164               15  S1-FCA-VAL PIC X(1).
41.C1.165               15  S1-FCA-DYN PIC X(1).
41.C1.166               15  S1-FCA-BACK PIC X(1).
41.C1.167               15  S1-FCA-FORE PIC X(1).
41.C1.168               15  S1-FCA-INT PIC X(1).
41.C1.169               15  S1-FCA-HIGH PIC X(1).
41.C1.170               15  S1-FCA-FONT PIC X(1).
41.C1.171               15  S1-FCA-EMPH PIC X(1).
41.C1.172       02  SCREEN-LSPI1-1-DATA.
41.C1.173           05  S1-IMAGE3 PIC A(6) VALUE SPACES.
41.C1.174           05  FILLER PIC X(2).
41.C1.175           05  S1-IMAGE4 PIC S1(18) BINARY-1 VALUE ZERO.
41.C1.176           05  FILLER PIC 1(18).
41.C1.177           05  S1-IMAGE5 PIC 9(6) BINARY VALUE ZERO.
41.C1.178           05  FILLER PIC X(1).
41.C1.179           05  S1-IMAGE6 PIC 9(10) BINARY VALUE ZERO.
41.C1.180           05  S1-IMAGE7 PIC 9(6) VALUE ZERO.
41.C1.181           05  FILLER PIC X(2).
41.C1.182           05  S1-IMAGE8 COMP-1 VALUE ZERO.
```

Example 8–5. Inserting Screen Definitions with the INVOKE Statement (contd.)

```
41.C1.183            05  S1-IMAGE9 COMP-2 VALUE ZERO.
41.C1.184            05  S1-IMAGE10 PIC X(6) VALUE SPACES.
41.C1.185            05  FILLER PIC X(2).
42           *
43            invoke form lspi1-1 in formlib forms
44                        with values from repository.
44.C1.1       01  SCREEN-LSPI1-1.
44.C1.2           02  SCREEN-LSPI1-1-HEADER.
44.C1.3               05  FILLER PIC X(2) VALUE SPACES.
44.C1.4               05  FILLER PIC 9(5)  BINARY VALUE ZERO.
44.C1.5               05  FILLER PIC X(4) VALUE SPACES.
44.C1.6               05  S1-FILLER PIC X(8) VALUE '$DPS$SWS'.
44.C1.7               05  S1-NUMBER PIC 9(4) VALUE 1.
44.C1.8               05  S1-NAME PIC X(8) VALUE 'LSPI1'.
44.C1.9               05  S1-CHECK-NUMBER PIC 9(10) BINARY VALUE 43807.
44.C1.10              05  S1-FCA-LENGTH PIC 9(5)  BINARY VALUE 32.
44.C1.11              05  S1-FCA-COUNT PIC 9(5)  BINARY VALUE 8.
44.C1.12              05  FILLER PIC 9(2)  BINARY VALUE 24.
44.C1.13              05  FILLER PIC 9(2)  BINARY VALUE 0.
44.C1.14              05  S1-NUMBER-ROWS PIC 9(2)  BINARY VALUE 24.
44.C1.15              05  S1-NUMBER-COLS PIC 9(2)  BINARY VALUE 80.
44.C1.16              05  FILLER PIC 9(2)  BINARY VALUE 5.
44.C1.17              05  FILLER PIC 9(2)  BINARY VALUE 0.
44.C1.18              05  FILLER PIC 9(2)  BINARY VALUE ZERO.
44.C1.19              05  S1-QUERY-STATUS PIC 9(2)  BINARY VALUE 0.
44.C1.20              05  FILLER PIC 9(5)  BINARY VALUE 56.
44.C1.21              05  S1-WSDATA-LENGTH PIC 9(5)  BINARY VALUE 12.
44.C1.22              05  S1-SOE-YCO PIC 9(2)  BINARY VALUE ZERO.
44.C1.23              05  S1-SOE-XCO PIC 9(2)  BINARY VALUE ZERO.
44.C1.24              05  S1-FUNCTIONKEY PIC 9(5)  BINARY VALUE ZERO.
44.C1.25              05  S1-OUT-YCO PIC 9(2)  BINARY VALUE 9.
44.C1.26              05  S1-OUT-XCO PIC 9(2)  BINARY VALUE 16.
44.C1.27              05  S1-OUT-FID PIC 9(5)  BINARY VALUE 4.
44.C1.28              05  S1-IN-YCO PIC 9(2)  BINARY VALUE 0.
44.C1.29              05  S1-IN-XCO PIC 9(2)  BINARY VALUE 0.
44.C1.30              05  S1-IN-FID PIC 9(5)  BINARY VALUE 0.
44.C1.31              05  S1-VALID PIC X(1) VALUE 'V'.
44.C1.32              05  S1-DYNAMIC PIC X(1) VALUE 'D'.
44.C1.33              05  S1-BACKGROUND PIC X(1) VALUE 'E'.
44.C1.34              05  S1-FOREGROUND PIC X(1) VALUE 'W'.
44.C1.35              05  S1-INTENSITY PIC X(1) VALUE 'N'.
44.C1.36              05  S1-HIGHLIGHT PIC X(1) VALUE 'N'.
44.C1.37              05  S1-FONT PIC X(1) VALUE 'N'.
44.C1.38              05  S1-EMPHASIS PIC X(1) VALUE 'N'.
44.C1.39              05  S1-CNUM-VALIDATE PIC X(1) VALUE 'Y'.
44.C1.40              05  FILLER PIC X(3) VALUE SPACES.
44.C1.41              05  S1-SCREENFILE PIC X(12) VALUE 'FORMS'.
44.C1.42              05  FILLER PIC X(12) VALUE SPACES.
44.C1.43              02  SCREEN-LSPI1-1-FCA.
44.C1.44              05  IMAGE3-STAT PIC 9(2)  BINARY VALUE 0.
44.C1.45              05  IMAGE3-TYPE PIC 9(2)  BINARY VALUE 1.
44.C1.46              05  IMAGE3-YCO PIC 9(2)  BINARY VALUE 6.
44.C1.47              05  IMAGE3-XCO PIC 9(2)  BINARY VALUE 18.
44.C1.48              05  IMAGE3-CHAR PIC 9(5)  BINARY VALUE ZERO.
44.C1.49              05  IMAGE3-FID PIC 9(5)  BINARY VALUE 1.
44.C1.50              05  IMAGE3-VAL PIC X(1) VALUE 'V'.
44.C1.51              05  IMAGE3-DYN PIC X(1) VALUE 'U'.
```

Example 8–5. Inserting Screen Definitions with the INVOKE Statement (contd.)

```
44.C1.52              05  IMAGE3-BACK PIC X(1) VALUE 'E'.
44.C1.53              05  IMAGE3-FORE PIC X(1) VALUE 'W'.
44.C1.54              05  IMAGE3-INT PIC X(1) VALUE 'N'.
44.C1.55              05  IMAGE3-HIGH PIC X(1) VALUE 'N'.
44.C1.56              05  IMAGE3-FONT PIC X(1) VALUE 'N'.
44.C1.57              05  IMAGE3-EMPH PIC X(1) VALUE 'N'.
44.C1.58              05  IMAGE4-STAT PIC 9(2)  BINARY VALUE 0.
44.C1.59              05  IMAGE4-TYPE PIC 9(2)  BINARY VALUE 2.
44.C1.60              05  IMAGE4-YCO PIC 9(2)  BINARY VALUE 7.
44.C1.61              05  IMAGE4-XCO PIC 9(2)  BINARY VALUE 21.
44.C1.62              05  IMAGE4-CHAR PIC 9(5)  BINARY VALUE ZERO.
44.C1.63              05  IMAGE4-FID PIC 9(5)  BINARY VALUE 2.
44.C1.64              05  IMAGE4-VAL PIC X(1) VALUE 'V'.
44.C1.65              05  IMAGE4-DYN PIC X(1) VALUE 'D'.
44.C1.66              05  IMAGE4-BACK PIC X(1) VALUE 'E'.
44.C1.67              05  IMAGE4-FORE PIC X(1) VALUE 'W'.
44.C1.68              05  IMAGE4-INT PIC X(1) VALUE 'N'.
44.C1.69              05  IMAGE4-HIGH PIC X(1) VALUE 'N'.
44.C1.70              05  IMAGE4-FONT PIC X(1) VALUE 'N'.
44.C1.71              05  IMAGE4-EMPH PIC X(1) VALUE 'N'.
44.C1.72              05  IMAGE5-STAT PIC 9(2)  BINARY VALUE 0.
44.C1.73              05  IMAGE5-TYPE PIC 9(2)  BINARY VALUE 1.
44.C1.74              05  IMAGE5-YCO PIC 9(2)  BINARY VALUE 8.
44.C1.75              05  IMAGE5-XCO PIC 9(2)  BINARY VALUE 21.
44.C1.76              05  IMAGE5-CHAR PIC 9(5)  BINARY VALUE ZERO.
44.C1.77              05  IMAGE5-FID PIC 9(5)  BINARY VALUE 3.
44.C1.78              05  IMAGE5-VAL PIC X(1) VALUE 'V'.
44.C1.79              05  IMAGE5-DYN PIC X(1) VALUE 'U'.
44.C1.80              05  IMAGE5-BACK PIC X(1) VALUE 'E'.
44.C1.81              05  IMAGE5-FORE PIC X(1) VALUE 'W'.
44.C1.82              05  IMAGE5-INT PIC X(1) VALUE 'N'.
44.C1.83              05  IMAGE5-HIGH PIC X(1) VALUE 'N'.
44.C1.84              05  IMAGE5-FONT PIC X(1) VALUE 'N'.
44.C1.85              05  IMAGE5-EMPH PIC X(1) VALUE 'N'.
44.C1.86              05  IMAGE6-STAT PIC 9(2)  BINARY VALUE 0.
44.C1.87              05  IMAGE6-TYPE PIC 9(2)  BINARY VALUE 1.
44.C1.88              05  IMAGE6-YCO PIC 9(2)  BINARY VALUE 9.
44.C1.89              05  IMAGE6-XCO PIC 9(2)  BINARY VALUE 16.
44.C1.90              05  IMAGE6-CHAR PIC 9(5)  BINARY VALUE ZERO.
44.C1.91              05  IMAGE6-FID PIC 9(5)  BINARY VALUE 4.
44.C1.92              05  IMAGE6-VAL PIC X(1) VALUE 'V'.
44.C1.93              05  IMAGE6-DYN PIC X(1) VALUE 'U'.
44.C1.94              05  IMAGE6-BACK PIC X(1) VALUE 'E'.
44.C1.95              05  IMAGE6-FORE PIC X(1) VALUE 'W'.
44.C1.96              05  IMAGE6-INT PIC X(1) VALUE 'N'.
44.C1.97              05  IMAGE6-HIGH PIC X(1) VALUE 'N'.
44.C1.98              05  IMAGE6-FONT PIC X(1) VALUE 'N'.
44.C1.99              05  IMAGE6-EMPH PIC X(1) VALUE 'N'.
44.C1.100             05  IMAGE7-STAT PIC 9(2)  BINARY VALUE 0.
44.C1.101             05  IMAGE7-TYPE PIC 9(2)  BINARY VALUE 1.
44.C1.102             05  IMAGE7-YCO PIC 9(2)  BINARY VALUE 10.
44.C1.103             05  IMAGE7-XCO PIC 9(2)  BINARY VALUE 16.
44.C1.104             05  IMAGE7-CHAR PIC 9(5)  BINARY VALUE ZERO.
44.C1.105             05  IMAGE7-FID PIC 9(5)  BINARY VALUE 5.
44.C1.106             05  IMAGE7-VAL PIC X(1) VALUE 'V'.
44.C1.107             05  IMAGE7-DYN PIC X(1) VALUE 'U'.
44.C1.108             05  IMAGE7-BACK PIC X(1) VALUE 'E'.
44.C1.109             05  IMAGE7-FORE PIC X(1) VALUE 'W'.
```

Example 8–5. Inserting Screen Definitions with the INVOKE Statement (contd.)

```
44.C1.110             05  IMAGE7-INT PIC X(1) VALUE 'N'.
44.C1.111             05  IMAGE7-HIGH PIC X(1) VALUE 'N'.
44.C1.112             05  IMAGE7-FONT PIC X(1) VALUE 'N'.
44.C1.113             05  IMAGE7-EMPH PIC X(1) VALUE 'N'.
44.C1.114             05  IMAGE8-STAT PIC 9(2)  BINARY VALUE 0.
44.C1.115             05  IMAGE8-TYPE PIC 9(2)  BINARY VALUE 1.
44.C1.116             05  IMAGE8-YCO PIC 9(2)  BINARY VALUE 11.
44.C1.117             05  IMAGE8-XCO PIC 9(2)  BINARY VALUE 13.
44.C1.118             05  IMAGE8-CHAR PIC 9(5)  BINARY VALUE ZERO.
44.C1.119             05  IMAGE8-FID PIC 9(5)  BINARY VALUE 6.
44.C1.120             05  IMAGE8-VAL PIC X(1) VALUE 'V'.
44.C1.121             05  IMAGE8-DYN PIC X(1) VALUE 'U'.
44.C1.122             05  IMAGE8-BACK PIC X(1) VALUE 'E'.
44.C1.123             05  IMAGE8-FORE PIC X(1) VALUE 'W'.
44.C1.124             05  IMAGE8-INT PIC X(1) VALUE 'N'.
44.C1.125             05  IMAGE8-HIGH PIC X(1) VALUE 'N'.
44.C1.126             05  IMAGE8-FONT PIC X(1) VALUE 'N'.
44.C1.127             05  IMAGE8-EMPH PIC X(1) VALUE 'N'.
44.C1.128             05  IMAGE9-STAT PIC 9(2)  BINARY VALUE 0.
44.C1.129             05  IMAGE9-TYPE PIC 9(2)  BINARY VALUE 1.
44.C1.130             05  IMAGE9-YCO PIC 9(2)  BINARY VALUE 12.
44.C1.131             05  IMAGE9-XCO PIC 9(2)  BINARY VALUE 20.
44.C1.132             05  IMAGE9-CHAR PIC 9(5)  BINARY VALUE ZERO.
44.C1.133             05  IMAGE9-FID PIC 9(5)  BINARY VALUE 7.
44.C1.134             05  IMAGE9-VAL PIC X(1) VALUE 'V'.
44.C1.135             05  IMAGE9-DYN PIC X(1) VALUE 'U'.
44.C1.136             05  IMAGE9-BACK PIC X(1) VALUE 'E'.
44.C1.137             05  IMAGE9-FORE PIC X(1) VALUE 'W'.
44.C1.138             05  IMAGE9-INT PIC X(1) VALUE 'N'.
44.C1.139             05  IMAGE9-HIGH PIC X(1) VALUE 'N'.
44.C1.140             05  IMAGE9-FONT PIC X(1) VALUE 'N'.
44.C1.141             05  IMAGE9-EMPH PIC X(1) VALUE 'N'.
44.C1.142             05  IMAGE10-STAT PIC 9(2)  BINARY VALUE 0.
44.C1.143             05  IMAGE10-TYPE PIC 9(2)  BINARY VALUE 1.
44.C1.144             05  IMAGE10-YCO PIC 9(2)  BINARY VALUE 13.
44.C1.145             05  IMAGE10-XCO PIC 9(2)  BINARY VALUE 21.
44.C1.146             05  IMAGE10-CHAR PIC 9(5)  BINARY VALUE ZERO.
44.C1.147             05  IMAGE10-FID PIC 9(5)  BINARY VALUE 8.
44.C1.148             05  IMAGE10-VAL PIC X(1) VALUE 'V'.
44.C1.149             05  IMAGE10-DYN PIC X(1) VALUE 'U'.
44.C1.150             05  IMAGE10-BACK PIC X(1) VALUE 'E'.
44.C1.151             05  IMAGE10-FORE PIC X(1) VALUE 'W'.
44.C1.152             05  IMAGE10-INT PIC X(1) VALUE 'N'.
44.C1.153             05  IMAGE10-HIGH PIC X(1) VALUE 'N'.
44.C1.154             05  IMAGE10-FONT PIC X(1) VALUE 'N'.
44.C1.155             05  IMAGE10-EMPH PIC X(1) VALUE 'N'.
44.C1.156         02  SCREEN-LSPI1-1-FCA-END REDEFINES SCREEN-LSPI1-1-FCA.
44.C1.157             05  SCREEN-LSPI1-1-FCA-REDEF OCCURS 8 TIMES.
44.C1.158                 15  FCA-STAT PIC 9(2)  BINARY.
44.C1.159                 15  FCA-TYPE PIC 9(2)  BINARY.
44.C1.160                 15  FCA-YCO PIC 9(2)  BINARY.
44.C1.161                 15  FCA-XCO PIC 9(2)  BINARY.
44.C1.162                 15  FCA-CHAR PIC 9(5)  BINARY.
44.C1.163                 15  FCA-FID PIC 9(5)  BINARY.
44.C1.164                 15  FCA-VAL PIC X(1).
44.C1.165                 15  FCA-DYN PIC X(1).
44.C1.166                 15  FCA-BACK PIC X(1).
```

Example 8–5. Inserting Screen Definitions with the INVOKE Statement (contd.)

```
44.C1.167                15  FCA-FORE PIC X(1).
44.C1.168                15  FCA-INT PIC X(1).
44.C1.169                15  FCA-HIGH PIC X(1).
44.C1.170                15  FCA-FONT PIC X(1).
44.C1.171                15  FCA-EMPH PIC X(1).
44.C1.172         02  SCREEN-LSPI1-1-DATA.
44.C1.173             05  IMAGE3 PIC A(6) VALUE SPACES.
44.C1.174             05  FILLER PIC X(2).
44.C1.175             05  IMAGE4 PIC S1(18) BINARY-1 VALUE ZERO.
44.C1.176             05  FILLER PIC 1(18).
44.C1.177             05  IMAGE5 PIC 9(6) BINARY VALUE ZERO.
44.C1.178             05  FILLER PIC X(1).
44.C1.179             05  IMAGE6 PIC 9(10) BINARY VALUE ZERO.
44.C1.180             05  IMAGE7 PIC 9(6) VALUE ZERO.
44.C1.181             05  FILLER PIC X(2).
44.C1.182             05  IMAGE8 COMP-1 VALUE ZERO.
44.C1.183             05  IMAGE9 COMP-2 VALUE ZERO.
44.C1.184             05  IMAGE10 PIC X(6) VALUE SPACES.
44.C1.185             05  FILLER PIC X(2).
```

Example 8–5. Inserting Screen Definitions with the INVOKE Statement

Inserting Screen Definitions with Specific Initial Values

The form in Example 8–6 is the same as the previous one, except that it was created with specific initial values for the alphabetic and alphanumeric data items. The user cannot specify initial values for numeric data items.

This example is similar to Example 8–5, except that the values in the WITH clause are reversed to show that order is not important.

```
56            *
57             invoke form lspi2-2 in formlib forms
58                         with prefixes values from repository.
58.C1.1        01  SCREEN-LSPI2-2.
58.C1.2            02  SCREEN-LSPI2-2-HEADER.
58.C1.3                05  FILLER PIC X(2) VALUE SPACES.
58.C1.4                05  FILLER PIC 9(5)  BINARY VALUE ZERO.
58.C1.5                05  FILLER PIC X(4) VALUE SPACES.
58.C1.6                05  S2-FILLER PIC X(8) VALUE '$DPS$SWS'.
58.C1.7                05  S2-NUMBER PIC 9(4) VALUE 2.
58.C1.8                05  S2-NAME PIC X(8) VALUE 'LSPI2'.
58.C1.9                05  S2-CHECK-NUMBER PIC 9(10) BINARY VALUE 210092.
58.C1.10               05  S2-FCA-LENGTH PIC 9(5)  BINARY VALUE 32.
58.C1.11               05  S2-FCA-COUNT PIC 9(5)  BINARY VALUE 8.
58.C1.12               05  FILLER PIC 9(2)  BINARY VALUE 24.
58.C1.13               05  FILLER PIC 9(2)  BINARY VALUE 0.
58.C1.14               05  S2-NUMBER-ROWS PIC 9(2)  BINARY VALUE 24.
58.C1.15               05  S2-NUMBER-COLS PIC 9(2)  BINARY VALUE 80.
58.C1.16               05  FILLER PIC 9(2)  BINARY VALUE 5.
58.C1.17               05  FILLER PIC 9(2)  BINARY VALUE 0.
58.C1.18               05  FILLER PIC 9(2)  BINARY VALUE ZERO.
58.C1.19               05  S2-QUERY-STATUS PIC 9(2)  BINARY VALUE 0.
58.C1.20               05  FILLER PIC 9(5)  BINARY VALUE 56.
58.C1.21               05  S2-WSDATA-LENGTH PIC 9(5)  BINARY VALUE 12.
58.C1.22               05  S2-SOE-YCO PIC 9(2)  BINARY VALUE ZERO.
```

Example 8–6. Inserting Screen Definitions with Specific Initial Values

```
58.C1.23              05  S2-SOE-XCO PIC 9(2)  BINARY VALUE ZERO.
58.C1.24              05  S2-FUNCTIONKEY PIC 9(5)  BINARY VALUE ZERO.
58.C1.25              05  S2-OUT-YCO PIC 9(2)  BINARY VALUE 9.
58.C1.26              05  S2-OUT-XCO PIC 9(2)  BINARY VALUE 16.
58.C1.27              05  S2-OUT-FID PIC 9(5)  BINARY VALUE 4.
58.C1.28              05  S2-IN-YCO PIC 9(2)  BINARY VALUE 0.
58.C1.29              05  S2-IN-XCO PIC 9(2)  BINARY VALUE 0.
58.C1.30              05  S2-IN-FID PIC 9(5)  BINARY VALUE 0.
58.C1.31              05  S2-VALID PIC X(1) VALUE 'V'.
58.C1.32              05  S2-DYNAMIC PIC X(1) VALUE 'D'.
58.C1.33              05  S2-BACKGROUND PIC X(1) VALUE 'E'.
58.C1.34              05  S2-FOREGROUND PIC X(1) VALUE 'W'.
58.C1.35              05  S2-INTENSITY PIC X(1) VALUE 'N'.
58.C1.36              05  S2-HIGHLIGHT PIC X(1) VALUE 'N'.
58.C1.37              05  S2-FONT PIC X(1) VALUE 'N'.
58.C1.38              05  S2-EMPHASIS PIC X(1) VALUE 'N'.
58.C1.39              05  S2-CNUM-VALIDATE PIC X(1) VALUE 'Y'.
58.C1.40              05  FILLER PIC X(3) VALUE SPACES.
58.C1.41              05  S2-SCREENFILE PIC X(12) VALUE 'FORMS'.
58.C1.42              05  FILLER PIC X(12) VALUE SPACES.
58.C1.43           02  SCREEN-LSPI2-2-FCA.
58.C1.44              05  S2-IMAGE3-STAT PIC 9(2)  BINARY VALUE 0.
58.C1.45              05  S2-IMAGE3-TYPE PIC 9(2)  BINARY VALUE 1.
58.C1.46              05  S2-IMAGE3-YCO PIC 9(2)  BINARY VALUE 6.
58.C1.47              05  S2-IMAGE3-XCO PIC 9(2)  BINARY VALUE 18.
58.C1.48              05  S2-IMAGE3-CHAR PIC 9(5)  BINARY VALUE ZERO.
58.C1.49              05  S2-IMAGE3-FID PIC 9(5)  BINARY VALUE 1.
58.C1.50              05  S2-IMAGE3-VAL PIC X(1) VALUE 'V'.
58.C1.51              05  S2-IMAGE3-DYN PIC X(1) VALUE 'U'.
58.C1.52              05  S2-IMAGE3-BACK PIC X(1) VALUE 'E'.
58.C1.53              05  S2-IMAGE3-FORE PIC X(1) VALUE 'W'.
58.C1.54              05  S2-IMAGE3-INT PIC X(1) VALUE 'N'.
58.C1.55              05  S2-IMAGE3-HIGH PIC X(1) VALUE 'N'.
58.C1.56              05  S2-IMAGE3-FONT PIC X(1) VALUE 'N'.
58.C1.57              05  S2-IMAGE3-EMPH PIC X(1) VALUE 'N'.
58.C1.58              05  S2-IMAGE4-STAT PIC 9(2)  BINARY VALUE 0.
58.C1.59              05  S2-IMAGE4-TYPE PIC 9(2)  BINARY VALUE 2.
58.C1.60              05  S2-IMAGE4-YCO PIC 9(2)  BINARY VALUE 7.
58.C1.61              05  S2-IMAGE4-XCO PIC 9(2)  BINARY VALUE 21.
58.C1.62              05  S2-IMAGE4-CHAR PIC 9(5)  BINARY VALUE ZERO.
58.C1.63              05  S2-IMAGE4-FID PIC 9(5)  BINARY VALUE 2.
58.C1.64              05  S2-IMAGE4-VAL PIC X(1) VALUE 'V'.
58.C1.65              05  S2-IMAGE4-DYN PIC X(1) VALUE 'D'.
58.C1.66              05  S2-IMAGE4-BACK PIC X(1) VALUE 'E'.
58.C1.67              05  S2-IMAGE4-FORE PIC X(1) VALUE 'W'.
58.C1.68              05  S2-IMAGE4-INT PIC X(1) VALUE 'N'.
58.C1.69              05  S2-IMAGE4-HIGH PIC X(1) VALUE 'N'.
58.C1.70              05  S2-IMAGE4-FONT PIC X(1) VALUE 'N'.
58.C1.71              05  S2-IMAGE4-EMPH PIC X(1) VALUE 'N'.
58.C1.72              05  S2-IMAGE5-STAT PIC 9(2)  BINARY VALUE 0.
58.C1.73              05  S2-IMAGE5-TYPE PIC 9(2)  BINARY VALUE 1.
58.C1.74              05  S2-IMAGE5-YCO PIC 9(2)  BINARY VALUE 8.
58.C1.75              05  S2-IMAGE5-XCO PIC 9(2)  BINARY VALUE 21.
58.C1.76              05  S2-IMAGE5-CHAR PIC 9(5)  BINARY VALUE ZERO.
58.C1.77              05  S2-IMAGE5-FID PIC 9(5)  BINARY VALUE 3.
58.C1.78              05  S2-IMAGE5-VAL PIC X(1) VALUE 'V'.
```

Example 8–6. Inserting Screen Definitions with Specific Initial Values (contd.)

```
58.C1.79              05  S2-IMAGE5-DYN PIC X(1) VALUE 'U'.
58.C1.80              05  S2-IMAGE5-BACK PIC X(1) VALUE 'E'.
58.C1.81              05  S2-IMAGE5-FORE PIC X(1) VALUE 'W'.
58.C1.82              05  S2-IMAGE5-INT PIC X(1) VALUE 'N'.
58.C1.83              05  S2-IMAGE5-HIGH PIC X(1) VALUE 'N'.
58.C1.84              05  S2-IMAGE5-FONT PIC X(1) VALUE 'N'.
58.C1.85              05  S2-IMAGE5-EMPH PIC X(1) VALUE 'N'.
58.C1.86              05  S2-IMAGE6-STAT PIC 9(2)  BINARY VALUE 0.
58.C1.87              05  S2-IMAGE6-TYPE PIC 9(2)  BINARY VALUE 1.
58.C1.88              05  S2-IMAGE6-YCO PIC 9(2)  BINARY VALUE 9.
58.C1.89              05  S2-IMAGE6-XCO PIC 9(2)  BINARY VALUE 16.
58.C1.90              05  S2-IMAGE6-CHAR PIC 9(5)  BINARY VALUE ZERO.
58.C1.91              05  S2-IMAGE6-FID PIC 9(5)  BINARY VALUE 4.
58.C1.92              05  S2-IMAGE6-VAL PIC X(1) VALUE 'V'.
58.C1.93              05  S2-IMAGE6-DYN PIC X(1) VALUE 'U'.
58.C1.94              05  S2-IMAGE6-BACK PIC X(1) VALUE 'E'.
58.C1.95              05  S2-IMAGE6-FORE PIC X(1) VALUE 'W'.
58.C1.96              05  S2-IMAGE6-INT PIC X(1) VALUE 'N'.
58.C1.97              05  S2-IMAGE6-HIGH PIC X(1) VALUE 'N'.
58.C1.98              05  S2-IMAGE6-FONT PIC X(1) VALUE 'N'.
58.C1.99              05  S2-IMAGE6-EMPH PIC X(1) VALUE 'N'.
58.C1.100             05  S2-IMAGE7-STAT PIC 9(2)  BINARY VALUE 0.
58.C1.101             05  S2-IMAGE7-TYPE PIC 9(2)  BINARY VALUE 1.
58.C1.102             05  S2-IMAGE7-YCO PIC 9(2)  BINARY VALUE 10.
58.C1.103             05  S2-IMAGE7-XCO PIC 9(2)  BINARY VALUE 16.
58.C1.104             05  S2-IMAGE7-CHAR PIC 9(5)  BINARY VALUE ZERO.
58.C1.105             05  S2-IMAGE7-FID PIC 9(5)  BINARY VALUE 5.
58.C1.106             05  S2-IMAGE7-VAL PIC X(1) VALUE 'V'.
58.C1.107             05  S2-IMAGE7-DYN PIC X(1) VALUE 'U'.
58.C1.108             05  S2-IMAGE7-BACK PIC X(1) VALUE 'E'.
58.C1.109             05  S2-IMAGE7-FORE PIC X(1) VALUE 'W'.
58.C1.110             05  S2-IMAGE7-INT PIC X(1) VALUE 'N'.
58.C1.111             05  S2-IMAGE7-HIGH PIC X(1) VALUE 'N'.
58.C1.112             05  S2-IMAGE7-FONT PIC X(1) VALUE 'N'.
58.C1.113             05  S2-IMAGE7-EMPH PIC X(1) VALUE 'N'.
58.C1.114             05  S2-IMAGE8-STAT PIC 9(2)  BINARY VALUE 0.
58.C1.115             05  S2-IMAGE8-TYPE PIC 9(2)  BINARY VALUE 1.
58.C1.116             05  S2-IMAGE8-YCO PIC 9(2)  BINARY VALUE 11.
58.C1.117             05  S2-IMAGE8-XCO PIC 9(2)  BINARY VALUE 13.
58.C1.118             05  S2-IMAGE8-CHAR PIC 9(5)  BINARY VALUE ZERO.
58.C1.119             05  S2-IMAGE8-FID PIC 9(5)  BINARY VALUE 6.
58.C1.120             05  S2-IMAGE8-VAL PIC X(1) VALUE 'V'.
58.C1.121             05  S2-IMAGE8-DYN PIC X(1) VALUE 'U'.
58.C1.122             05  S2-IMAGE8-BACK PIC X(1) VALUE 'E'.
58.C1.123             05  S2-IMAGE8-FORE PIC X(1) VALUE 'W'.
58.C1.124             05  S2-IMAGE8-INT PIC X(1) VALUE 'N'.
58.C1.125             05  S2-IMAGE8-HIGH PIC X(1) VALUE 'N'.
58.C1.126             05  S2-IMAGE8-FONT PIC X(1) VALUE 'N'.
58.C1.127             05  S2-IMAGE8-EMPH PIC X(1) VALUE 'N'.
58.C1.128             05  S2-IMAGE9-STAT PIC 9(2)  BINARY VALUE 0.
58.C1.129             05  S2-IMAGE9-TYPE PIC 9(2)  BINARY VALUE 1.
58.C1.130             05  S2-IMAGE9-YCO PIC 9(2)  BINARY VALUE 12.
58.C1.131             05  S2-IMAGE9-XCO PIC 9(2)  BINARY VALUE 20.
58.C1.132             05  S2-IMAGE9-CHAR PIC 9(5)  BINARY VALUE ZERO.
58.C1.133             05  S2-IMAGE9-FID PIC 9(5)  BINARY VALUE 7.
58.C1.134             05  S2-IMAGE9-VAL PIC X(1) VALUE 'V'.
58.C1.135             05  S2-IMAGE9-DYN PIC X(1) VALUE 'U'.
58.C1.136             05  S2-IMAGE9-BACK PIC X(1) VALUE 'E'.
```

Example 8–6. Inserting Screen Definitions with Specific Initial Values (contd.)

```
58.C1.137              05  S2-IMAGE9-FORE PIC X(1) VALUE 'W'.
58.C1.138              05  S2-IMAGE9-INT PIC X(1) VALUE 'N'.
58.C1.139              05  S2-IMAGE9-HIGH PIC X(1) VALUE 'N'.
58.C1.140              05  S2-IMAGE9-FONT PIC X(1) VALUE 'N'.
58.C1.141              05  S2-IMAGE9-EMPH PIC X(1) VALUE 'N'.
58.C1.142              05  S2-IMAGE10-STAT PIC 9(2)  BINARY VALUE 0.
58.C1.143              05  S2-IMAGE10-TYPE PIC 9(2)  BINARY VALUE 1.
58.C1.144              05  S2-IMAGE10-YCO PIC 9(2)  BINARY VALUE 13.
58.C1.145              05  S2-IMAGE10-XCO PIC 9(2)  BINARY VALUE 21.
58.C1.146              05  S2-IMAGE10-CHAR PIC 9(5)  BINARY VALUE ZERO.
58.C1.147              05  S2-IMAGE10-FID PIC 9(5)  BINARY VALUE 8.
58.C1.148              05  S2-IMAGE10-VAL PIC X(1) VALUE 'V'.
58.C1.149              05  S2-IMAGE10-DYN PIC X(1) VALUE 'U'.
58.C1.150              05  S2-IMAGE10-BACK PIC X(1) VALUE 'E'.
58.C1.151              05  S2-IMAGE10-FORE PIC X(1) VALUE 'W'.
58.C1.152              05  S2-IMAGE10-INT PIC X(1) VALUE 'N'.
58.C1.153              05  S2-IMAGE10-HIGH PIC X(1) VALUE 'N'.
58.C1.154              05  S2-IMAGE10-FONT PIC X(1) VALUE 'N'.
58.C1.155              05  S2-IMAGE10-EMPH PIC X(1) VALUE 'N'.
58.C1.156        02  SCREEN-LSPI2-2-FCA-END REDEFINES SCREEN-LSPI2-2-FCA.
58.C1.157              05  SCREEN-LSPI2-2-FCA-REDEF OCCURS 8 TIMES.
58.C1.158                  15  S2-FCA-STAT PIC 9(2)  BINARY.
58.C1.159                  15  S2-FCA-TYPE PIC 9(2)  BINARY.
58.C1.160                  15  S2-FCA-YCO PIC 9(2)  BINARY.
58.C1.161                  15  S2-FCA-XCO PIC 9(2)  BINARY.
58.C1.162                  15  S2-FCA-CHAR PIC 9(5)  BINARY.
58.C1.163                  15  S2-FCA-FID PIC 9(5)  BINARY.
58.C1.164                  15  S2-FCA-VAL PIC X(1).
58.C1.165                  15  S2-FCA-DYN PIC X(1).
58.C1.166                  15  S2-FCA-BACK PIC X(1).
58.C1.167                  15  S2-FCA-FORE PIC X(1).
58.C1.168                  15  S2-FCA-INT PIC X(1).
58.C1.169                  15  S2-FCA-HIGH PIC X(1).
58.C1.170                  15  S2-FCA-FONT PIC X(1).
58.C1.171                  15  S2-FCA-EMPH PIC X(1).
58.C1.172        02  SCREEN-LSPI2-2-DATA.
58.C1.173              05  S2-IMAGE3 PIC A(6) VALUE 'abcxyz'.
58.C1.174              05  FILLER PIC X(2).
58.C1.175              05  S2-IMAGE4 PIC S1(18) BINARY-1 VALUE ZERO.
58.C1.176              05  FILLER PIC 1(18).
58.C1.177              05  S2-IMAGE5 PIC 9(6) BINARY VALUE ZERO.
58.C1.178              05  FILLER PIC X(1).
58.C1.179              05  S2-IMAGE6 PIC 9(10) BINARY VALUE ZERO.
58.C1.180              05  S2-IMAGE7 PIC 9(6) VALUE ZERO.
58.C1.181              05  FILLER PIC X(2).
58.C1.182              05  S2-IMAGE8 COMP-1 VALUE ZERO.
58.C1.183              05  S2-IMAGE9 COMP-2 VALUE ZERO.
58.C1.184              05  S2-IMAGE10 PIC X(6) VALUE 'abc123'.
58.C1.185              05  FILLER PIC X(2).
59               *
60             invoke form lspi2-2 in formlib forms
61                         with values from repository.
61.C1.1        01  SCREEN-LSPI2-2.
61.C1.2            02  SCREEN-LSPI2-2-HEADER.
61.C1.3                05  FILLER PIC X(2) VALUE SPACES.
61.C1.4                05  FILLER PIC 9(5)  BINARY VALUE ZERO.
61.C1.5                05  FILLER PIC X(4) VALUE SPACES.
```

Example 8–6. Inserting Screen Definitions with Specific Initial Values (contd.)

```
61.C1.6                 05  S2-FILLER PIC X(8) VALUE '$DPS$SWS'.
61.C1.7                 05  S2-NUMBER PIC 9(4) VALUE 2.
61.C1.8                 05  S2-NAME PIC X(8) VALUE 'LSPI2'.
61.C1.9                 05  S2-CHECK-NUMBER PIC 9(10) BINARY VALUE 210092.
61.C1.10                05  S2-FCA-LENGTH PIC 9(5)  BINARY VALUE 32.
61.C1.11                05  S2-FCA-COUNT PIC 9(5)  BINARY VALUE 8.
61.C1.12                05  FILLER PIC 9(2)  BINARY VALUE 24.
61.C1.13                05  FILLER PIC 9(2)  BINARY VALUE 0.
61.C1.14                05  S2-NUMBER-ROWS PIC 9(2)  BINARY VALUE 24.
61.C1.15                05  S2-NUMBER-COLS PIC 9(2)  BINARY VALUE 80.
61.C1.16                05  FILLER PIC 9(2)  BINARY VALUE 5.
61.C1.17                05  FILLER PIC 9(2)  BINARY VALUE 0.
61.C1.18                05  FILLER PIC 9(2)  BINARY VALUE ZERO.
61.C1.19                05  S2-QUERY-STATUS PIC 9(2)  BINARY VALUE 0.
61.C1.20                05  FILLER PIC 9(5)  BINARY VALUE 56.
61.C1.21                05  S2-WSDATA-LENGTH PIC 9(5)  BINARY VALUE 12.
61.C1.22                05  S2-SOE-YCO PIC 9(2)  BINARY VALUE ZERO.
61.C1.23                05  S2-SOE-XCO PIC 9(2)  BINARY VALUE ZERO.
61.C1.24                05  S2-FUNCTIONKEY PIC 9(5)  BINARY VALUE ZERO.
61.C1.25                05  S2-OUT-YCO PIC 9(2)  BINARY VALUE 9.
61.C1.26                05  S2-OUT-XCO PIC 9(2)  BINARY VALUE 16.
61.C1.27                05  S2-OUT-FID PIC 9(5)  BINARY VALUE 4.
61.C1.28                05  S2-IN-YCO PIC 9(2)  BINARY VALUE 0.
61.C1.29                05  S2-IN-XCO PIC 9(2)  BINARY VALUE 0.
61.C1.30                05  S2-IN-FID PIC 9(5)  BINARY VALUE 0.
61.C1.31                05  S2-VALID PIC X(1) VALUE 'V'.
61.C1.32                05  S2-DYNAMIC PIC X(1) VALUE 'D'.
61.C1.33                05  S2-BACKGROUND PIC X(1) VALUE 'E'.
61.C1.34                05  S2-FOREGROUND PIC X(1) VALUE 'W'.
61.C1.35                05  S2-INTENSITY PIC X(1) VALUE 'N'.
61.C1.36                05  S2-HIGHLIGHT PIC X(1) VALUE 'N'.
61.C1.37                05  S2-FONT PIC X(1) VALUE 'N'.
61.C1.38                05  S2-EMPHASIS PIC X(1) VALUE 'N'.
61.C1.39                05  S2-CNUM-VALIDATE PIC X(1) VALUE 'Y'.
61.C1.40                05  FILLER PIC X(3) VALUE SPACES.
61.C1.41                05  S2-SCREENFILE PIC X(12) VALUE 'FORMS       '.
61.C1.42                05  FILLER PIC X(12) VALUE SPACES.
61.C1.43                02  SCREEN-LSPI2-2-FCA.
61.C1.44                05  IMAGE3-STAT PIC 9(2)  BINARY VALUE 0.
61.C1.45                05  IMAGE3-TYPE PIC 9(2)  BINARY VALUE 1.
61.C1.46                05  IMAGE3-YCO PIC 9(2)  BINARY VALUE 6.
61.C1.47                05  IMAGE3-XCO PIC 9(2)  BINARY VALUE 18.
61.C1.48                05  IMAGE3-CHAR PIC 9(5)  BINARY VALUE ZERO.
61.C1.49                05  IMAGE3-FID PIC 9(5)  BINARY VALUE 1.
61.C1.50                05  IMAGE3-VAL PIC X(1) VALUE 'V'.
61.C1.51                05  IMAGE3-DYN PIC X(1) VALUE 'U'.
61.C1.52                05  IMAGE3-BACK PIC X(1) VALUE 'E'.
61.C1.53                05  IMAGE3-FORE PIC X(1) VALUE 'W'.
61.C1.54                05  IMAGE3-INT PIC X(1) VALUE 'N'.
61.C1.55                05  IMAGE3-HIGH PIC X(1) VALUE 'N'.
61.C1.56                05  IMAGE3-FONT PIC X(1) VALUE 'N'.
61.C1.57                05  IMAGE3-EMPH PIC X(1) VALUE 'N'.
61.C1.58                05  IMAGE4-STAT PIC 9(2)  BINARY VALUE 0.
61.C1.59                05  IMAGE4-TYPE PIC 9(2)  BINARY VALUE 2.
61.C1.60                05  IMAGE4-YCO PIC 9(2)  BINARY VALUE 7.
61.C1.61                05  IMAGE4-XCO PIC 9(2)  BINARY VALUE 21.
61.C1.62                05  IMAGE4-CHAR PIC 9(5)  BINARY VALUE ZERO.
61.C1.63                05  IMAGE4-FID PIC 9(5)  BINARY VALUE 2.
```

Example 8–6. Inserting Screen Definitions with Specific Initial Values (contd.)

```
61.C1.64              05  IMAGE4-VAL PIC X(1) VALUE 'V'.
61.C1.65              05  IMAGE4-DYN PIC X(1) VALUE 'D'.
61.C1.66              05  IMAGE4-BACK PIC X(1) VALUE 'E'.
61.C1.67              05  IMAGE4-FORE PIC X(1) VALUE 'W'.
61.C1.68              05  IMAGE4-INT PIC X(1) VALUE 'N'.
61.C1.69              05  IMAGE4-HIGH PIC X(1) VALUE 'N'.
61.C1.70              05  IMAGE4-FONT PIC X(1) VALUE 'N'.
61.C1.71              05  IMAGE4-EMPH PIC X(1) VALUE 'N'.
61.C1.72              05  IMAGE5-STAT PIC 9(2)  BINARY VALUE 0.
61.C1.73              05  IMAGE5-TYPE PIC 9(2)  BINARY VALUE 1.
61.C1.74              05  IMAGE5-YCO PIC 9(2)  BINARY VALUE 8.
61.C1.75              05  IMAGE5-XCO PIC 9(2)  BINARY VALUE 21.
61.C1.76              05  IMAGE5-CHAR PIC 9(5)  BINARY VALUE ZERO.
61.C1.77              05  IMAGE5-FID PIC 9(5)  BINARY VALUE 3.
61.C1.78              05  IMAGE5-VAL PIC X(1) VALUE 'V'.
61.C1.79              05  IMAGE5-DYN PIC X(1) VALUE 'U'.
61.C1.80              05  IMAGE5-BACK PIC X(1) VALUE 'E'.
61.C1.81              05  IMAGE5-FORE PIC X(1) VALUE 'W'.
61.C1.82              05  IMAGE5-INT PIC X(1) VALUE 'N'.
61.C1.83              05  IMAGE5-HIGH PIC X(1) VALUE 'N'.
61.C1.84              05  IMAGE5-FONT PIC X(1) VALUE 'N'.
61.C1.85              05  IMAGE5-EMPH PIC X(1) VALUE 'N'.
61.C1.86              05  IMAGE6-STAT PIC 9(2)  BINARY VALUE 0.
61.C1.87              05  IMAGE6-TYPE PIC 9(2)  BINARY VALUE 1.
61.C1.88              05  IMAGE6-YCO PIC 9(2)  BINARY VALUE 9.
61.C1.89              05  IMAGE6-XCO PIC 9(2)  BINARY VALUE 16.
61.C1.90              05  IMAGE6-CHAR PIC 9(5)  BINARY VALUE ZERO.
61.C1.91              05  IMAGE6-FID PIC 9(5)  BINARY VALUE 4.
61.C1.92               05  IMAGE6-VAL PIC X(1) VALUE 'V'.
61.C1.93               05  IMAGE6-DYN PIC X(1) VALUE 'U'.
61.C1.94               05  IMAGE6-BACK PIC X(1) VALUE 'E'.
61.C1.95               05  IMAGE6-FORE PIC X(1) VALUE 'W'.
61.C1.96               05  IMAGE6-INT PIC X(1) VALUE 'N'.
61.C1.97               05  IMAGE6-HIGH PIC X(1) VALUE 'N'.
61.C1.98               05  IMAGE6-FONT PIC X(1) VALUE 'N'.
61.C1.99               05  IMAGE6-EMPH PIC X(1) VALUE 'N'.
61.C1.100              05  IMAGE7-STAT PIC 9(2)  BINARY VALUE 0.
61.C1.101              05  IMAGE7-TYPE PIC 9(2)  BINARY VALUE 1.
61.C1.102              05  IMAGE7-YCO PIC 9(2)  BINARY VALUE 10.
61.C1.103              05  IMAGE7-XCO PIC 9(2)  BINARY VALUE 16.
61.C1.104              05  IMAGE7-CHAR PIC 9(5)  BINARY VALUE ZERO.
61.C1.105              05  IMAGE7-FID PIC 9(5)  BINARY VALUE 5.
61.C1.106              05  IMAGE7-VAL PIC X(1) VALUE 'V'.
61.C1.107              05  IMAGE7-DYN PIC X(1) VALUE 'U'.
61.C1.108              05  IMAGE7-BACK PIC X(1) VALUE 'E'.
61.C1.109              05  IMAGE7-FORE PIC X(1) VALUE 'W'.
61.C1.110              05  IMAGE7-INT PIC X(1) VALUE 'N'.
61.C1.111              05  IMAGE7-HIGH PIC X(1) VALUE 'N'.
61.C1.112              05  IMAGE7-FONT PIC X(1) VALUE 'N'.
61.C1.113              05  IMAGE7-EMPH PIC X(1) VALUE 'N'.
61.C1.114              05  IMAGE8-STAT PIC 9(2)  BINARY VALUE 0.
61.C1.115              05  IMAGE8-TYPE PIC 9(2)  BINARY VALUE 1.
61.C1.116              05  IMAGE8-YCO PIC 9(2)  BINARY VALUE 11.
61.C1.117              05  IMAGE8-XCO PIC 9(2)  BINARY VALUE 13.
61.C1.118              05  IMAGE8-CHAR PIC 9(5)  BINARY VALUE ZERO.
61.C1.119              05  IMAGE8-FID PIC 9(5)  BINARY VALUE 6.
61.C1.120              05  IMAGE8-VAL PIC X(1) VALUE 'V'.
61.C1.121              05  IMAGE8-DYN PIC X(1) VALUE 'U'.
```

Example 8–6. Inserting Screen Definitions with Specific Initial Values (contd.)

```
61.C1.122              05  IMAGE8-BACK PIC X(1) VALUE 'E'.
61.C1.123              05  IMAGE8-FORE PIC X(1) VALUE 'W'.
61.C1.124              05  IMAGE8-INT PIC X(1) VALUE 'N'.
61.C1.125              05  IMAGE8-HIGH PIC X(1) VALUE 'N'.
61.C1.126              05  IMAGE8-FONT PIC X(1) VALUE 'N'.
61.C1.127              05  IMAGE8-EMPH PIC X(1) VALUE 'N'.
61.C1.128              05  IMAGE9-STAT PIC 9(2)  BINARY VALUE 0.
61.C1.129              05  IMAGE9-TYPE PIC 9(2)  BINARY VALUE 1.
61.C1.130              05  IMAGE9-YCO PIC 9(2)  BINARY VALUE 12.
61.C1.131              05  IMAGE9-XCO PIC 9(2)  BINARY VALUE 20.
61.C1.132              05  IMAGE9-CHAR PIC 9(5)  BINARY VALUE ZERO.
61.C1.133              05  IMAGE9-FID PIC 9(5)  BINARY VALUE 7.
61.C1.134              05  IMAGE9-VAL PIC X(1) VALUE 'V'.
61.C1.135              05  IMAGE9-DYN PIC X(1) VALUE 'U'.
61.C1.136              05  IMAGE9-BACK PIC X(1) VALUE 'E'.
61.C1.137              05  IMAGE9-FORE PIC X(1) VALUE 'W'.
61.C1.138              05  IMAGE9-INT PIC X(1) VALUE 'N'.
61.C1.139              05  IMAGE9-HIGH PIC X(1) VALUE 'N'.
61.C1.140              05  IMAGE9-FONT PIC X(1) VALUE 'N'.
61.C1.141              05  IMAGE9-EMPH PIC X(1) VALUE 'N'.
61.C1.142              05  IMAGE10-STAT PIC 9(2)  BINARY VALUE 0.
61.C1.143              05  IMAGE10-TYPE PIC 9(2)  BINARY VALUE 1.
61.C1.144              05  IMAGE10-YCO PIC 9(2)  BINARY VALUE 13.
61.C1.145              05  IMAGE10-XCO PIC 9(2)  BINARY VALUE 21.
61.C1.146              05  IMAGE10-CHAR PIC 9(5)  BINARY VALUE ZERO.
61.C1.147              05  IMAGE10-FID PIC 9(5)  BINARY VALUE 8.
61.C1.148              05  IMAGE10-VAL PIC X(1) VALUE 'V'.
61.C1.149              05  IMAGE10-DYN PIC X(1) VALUE 'U'.
61.C1.150              05  IMAGE10-BACK PIC X(1) VALUE 'E'.
61.C1.151              05  IMAGE10-FORE PIC X(1) VALUE 'W'.
61.C1.152              05  IMAGE10-INT PIC X(1) VALUE 'N'.
61.C1.153              05  IMAGE10-HIGH PIC X(1) VALUE 'N'.
61.C1.154              05  IMAGE10-FONT PIC X(1) VALUE 'N'.
61.C1.155              05  IMAGE10-EMPH PIC X(1) VALUE 'N'.
61.C1.156       02  SCREEN-LSPI2-2-FCA-END REDEFINES SCREEN-LSPI2-2-FCA.
61.C1.157              05  SCREEN-LSPI2-2-FCA-REDEF OCCURS 8 TIMES.
61.C1.158                  15  FCA-STAT PIC 9(2)  BINARY.
61.C1.159                  15  FCA-TYPE PIC 9(2)  BINARY.
61.C1.160                  15  FCA-YCO PIC 9(2)  BINARY.
61.C1.161                  15  FCA-XCO PIC 9(2)  BINARY.
61.C1.162                  15  FCA-CHAR PIC 9(5)  BINARY.
61.C1.163                  15  FCA-FID PIC 9(5)  BINARY.
61.C1.164                  15  FCA-VAL PIC X(1).
61.C1.165                  15  FCA-DYN PIC X(1).
61.C1.166                  15  FCA-BACK PIC X(1).
61.C1.167                  15  FCA-FORE PIC X(1).
61.C1.168                  15  FCA-INT PIC X(1).
61.C1.169                  15  FCA-HIGH PIC X(1).
61.C1.170                  15  FCA-FONT PIC X(1).
61.C1.171                  15  FCA-EMPH PIC X(1).
61.C1.172         02  SCREEN-LSPI2-2-DATA.
61.C1.173              05  IMAGE3 PIC A(6) VALUE 'abcxyz'.
61.C1.174              05  FILLER PIC X(2).
61.C1.175              05  IMAGE4 PIC S1(18) BINARY-1 VALUE ZERO.
61.C1.176              05  FILLER PIC 1(18).
61.C1.177              05  IMAGE5 PIC 9(6) BINARY VALUE ZERO.
61.C1.178              05  FILLER PIC X(1).
61.C1.179              05  IMAGE6 PIC 9(10) BINARY VALUE ZERO.
```

Example 8–6. Inserting Screen Definitions with Specific Initial Values (contd.)

```
61.C1.180               05  IMAGE7 PIC 9(6) VALUE ZERO.
61.C1.181               05  FILLER PIC X(2).
61.C1.182               05  IMAGE8 COMP-1 VALUE ZERO.
61.C1.183               05  IMAGE9 COMP-2 VALUE ZERO.
61.C1.184               05  IMAGE10 PIC X(6) VALUE 'abc123'.
61.C1.185               05  FILLER PIC X(2).
```

Example 8–6. Inserting Screen Definitions with Specific Initial Values

Inserting Screen Definitions with Repeating Image

The form in Example 8–7 is the same as the form in Example 8–5, except that it was created with a repeating image to show what DPS generates (the image is repeated three times). Also, the example does not include the WITH clause, causing the program to use the default values of VALUES and PREFIXES.

```
71            invoke form lspi3-3 in formlib forms
72                                        from repository.
72.C1.1       01  SCREEN-LSPI3-3.
72.C1.2           02  SCREEN-LSPI3-3-HEADER.
72.C1.3               05  FILLER PIC X(2) VALUE SPACES.
72.C1.4               05  FILLER PIC 9(5)  BINARY VALUE ZERO.
72.C1.5               05  FILLER PIC X(4) VALUE SPACES.
72.C1.6               05  S3-FILLER PIC X(8) VALUE '$DPS$SWS'.
72.C1.7               05  S3-NUMBER PIC 9(4) VALUE 3.
72.C1.8               05  S3-NAME PIC X(8) VALUE 'LSPI3'.
72.C1.9               05  S3-CHECK-NUMBER PIC 9(10) BINARY VALUE 246452.
72.C1.10              05  S3-FCA-LENGTH PIC 9(5)  BINARY VALUE 44.
72.C1.11              05  S3-FCA-COUNT PIC 9(5)  BINARY VALUE 11.
72.C1.12              05  FILLER PIC 9(2)  BINARY VALUE 24.
72.C1.13              05  FILLER PIC 9(2)  BINARY VALUE 0.
72.C1.14              05  S3-NUMBER-ROWS PIC 9(2)  BINARY VALUE 24.
72.C1.15              05  S3-NUMBER-COLS PIC 9(2)  BINARY VALUE 80.
72.C1.16              05  FILLER PIC 9(2)  BINARY VALUE 5.
72.C1.17              05  FILLER PIC 9(2)  BINARY VALUE 0.
72.C1.18              05  FILLER PIC 9(2)  BINARY VALUE ZERO.
72.C1.19              05  S3-QUERY-STATUS PIC 9(2)  BINARY VALUE 0.
72.C1.20              05  FILLER PIC 9(5)  BINARY VALUE 68.
72.C1.21              05  S3-WSDATA-LENGTH PIC 9(5)  BINARY VALUE 18.
72.C1.22              05  S3-SOE-YCO PIC 9(2)  BINARY VALUE ZERO.
72.C1.23              05  S3-SOE-XCO PIC 9(2)  BINARY VALUE ZERO.
72.C1.24              05  S3-FUNCTIONKEY PIC 9(5)  BINARY VALUE ZERO.
72.C1.25              05  S3-OUT-YCO PIC 9(2)  BINARY VALUE 9.
72.C1.26              05  S3-OUT-XCO PIC 9(2)  BINARY VALUE 16.
72.C1.27              05  S3-OUT-FID PIC 9(5)  BINARY VALUE 4.
72.C1.28              05  S3-IN-YCO PIC 9(2)  BINARY VALUE 0.
72.C1.29              05  S3-IN-XCO PIC 9(2)  BINARY VALUE 0.
72.C1.30              05  S3-IN-FID PIC 9(5)  BINARY VALUE 0.
72.C1.31              05  S3-VALID PIC X(1) VALUE 'V'.
72.C1.32              05  S3-DYNAMIC PIC X(1) VALUE 'D'.
72.C1.33              05  S3-BACKGROUND PIC X(1) VALUE 'E'.
72.C1.34              05  S3-FOREGROUND PIC X(1) VALUE 'W'.
72.C1.35              05  S3-INTENSITY PIC X(1) VALUE 'N'.
72.C1.36              05  S3-HIGHLIGHT PIC X(1) VALUE 'N'.
72.C1.37              05  S3-FONT PIC X(1) VALUE 'N'.
72.C1.38              05  S3-EMPHASIS PIC X(1) VALUE 'N'.
```

Example 8–7. Inserting Screen Definitions with Repeating Image

```
72.C1.39              05  S3-CNUM-VALIDATE PIC X(1) VALUE 'Y'.
72.C1.40              05  FILLER PIC X(3) VALUE SPACES.
72.C1.41              05  S3-SCREENFILE PIC X(12) VALUE 'FORMS'.
72.C1.42              05  FILLER PIC X(12) VALUE SPACES.
72.C1.43          02  SCREEN-LSPI3-3-FCA.
72.C1.44              05  S3-IMAGE3-STAT PIC 9(2)  BINARY VALUE 0.
72.C1.45              05  S3-IMAGE3-TYPE PIC 9(2)  BINARY VALUE 1.
72.C1.46              05  S3-IMAGE3-YCO PIC 9(2)  BINARY VALUE 6.
72.C1.47              05  S3-IMAGE3-XCO PIC 9(2)  BINARY VALUE 18.
72.C1.48              05  S3-IMAGE3-CHAR PIC 9(5)  BINARY VALUE ZERO.
72.C1.49              05  S3-IMAGE3-FID PIC 9(5)  BINARY VALUE 1.
72.C1.50              05  S3-IMAGE3-VAL PIC X(1) VALUE 'V'.
72.C1.51              05  S3-IMAGE3-DYN PIC X(1) VALUE 'U'.
72.C1.52              05  S3-IMAGE3-BACK PIC X(1) VALUE 'E'.
72.C1.53              05  S3-IMAGE3-FORE PIC X(1) VALUE 'W'.
72.C1.54              05  S3-IMAGE3-INT PIC X(1) VALUE 'N'.
72.C1.55              05  S3-IMAGE3-HIGH PIC X(1) VALUE 'N'.
72.C1.56              05  S3-IMAGE3-FONT PIC X(1) VALUE 'N'.
72.C1.57              05  S3-IMAGE3-EMPH PIC X(1) VALUE 'N'.
72.C1.58              05  S3-IMAGE4-STAT PIC 9(2)  BINARY VALUE 0.
72.C1.59              05  S3-IMAGE4-TYPE PIC 9(2)  BINARY VALUE 2.
72.C1.60              05  S3-IMAGE4-YCO PIC 9(2)  BINARY VALUE 7.
72.C1.61              05  S3-IMAGE4-XCO PIC 9(2)  BINARY VALUE 21.
72.C1.62              05  S3-IMAGE4-CHAR PIC 9(5)  BINARY VALUE ZERO.
72.C1.63              05  S3-IMAGE4-FID PIC 9(5)  BINARY VALUE 2.
72.C1.64              05  S3-IMAGE4-VAL PIC X(1) VALUE 'V'.
72.C1.65              05  S3-IMAGE4-DYN PIC X(1) VALUE 'D'.
72.C1.66              05  S3-IMAGE4-BACK PIC X(1) VALUE 'E'.
72.C1.67              05  S3-IMAGE4-FORE PIC X(1) VALUE 'W'.
72.C1.68              05  S3-IMAGE4-INT PIC X(1) VALUE 'N'.
72.C1.69              05  S3-IMAGE4-HIGH PIC X(1) VALUE 'N'.
72.C1.70              05  S3-IMAGE4-FONT PIC X(1) VALUE 'N'.
72.C1.71              05  S3-IMAGE4-EMPH PIC X(1) VALUE 'N'.
72.C1.72              05  S3-IMAGE5-STAT PIC 9(2)  BINARY VALUE 0.
72.C1.73              05  S3-IMAGE5-TYPE PIC 9(2)  BINARY VALUE 1.
72.C1.74              05  S3-IMAGE5-YCO PIC 9(2)  BINARY VALUE 8.
72.C1.75              05  S3-IMAGE5-XCO PIC 9(2)  BINARY VALUE 21.
72.C1.76              05  S3-IMAGE5-CHAR PIC 9(5)  BINARY VALUE ZERO.
72.C1.77              05  S3-IMAGE5-FID PIC 9(5)  BINARY VALUE 3.
72.C1.78              05  S3-IMAGE5-VAL PIC X(1) VALUE 'V'.
72.C1.79              05  S3-IMAGE5-DYN PIC X(1) VALUE 'U'.
72.C1.80              05  S3-IMAGE5-BACK PIC X(1) VALUE 'E'.
72.C1.81              05  S3-IMAGE5-FORE PIC X(1) VALUE 'W'.
72.C1.82              05  S3-IMAGE5-INT PIC X(1) VALUE 'N'.
72.C1.83              05  S3-IMAGE5-HIGH PIC X(1) VALUE 'N'.
72.C1.84              05  S3-IMAGE5-FONT PIC X(1) VALUE 'N'.
72.C1.85              05  S3-IMAGE5-EMPH PIC X(1) VALUE 'N'.
72.C1.86              05  S3-IMAGE6-STAT PIC 9(2)  BINARY VALUE 0.
72.C1.87              05  S3-IMAGE6-TYPE PIC 9(2)  BINARY VALUE 1.
72.C1.88              05  S3-IMAGE6-YCO PIC 9(2)  BINARY VALUE 9.
72.C1.89              05  S3-IMAGE6-XCO PIC 9(2)  BINARY VALUE 16.
72.C1.90              05  S3-IMAGE6-CHAR PIC 9(5)  BINARY VALUE ZERO.
72.C1.91              05  S3-IMAGE6-FID PIC 9(5)  BINARY VALUE 4.
72.C1.92              05  S3-IMAGE6-VAL PIC X(1) VALUE 'V'.
72.C1.93              05  S3-IMAGE6-DYN PIC X(1) VALUE 'U'.
72.C1.94              05  S3-IMAGE6-BACK PIC X(1) VALUE 'E'.
72.C1.95              05  S3-IMAGE6-FORE PIC X(1) VALUE 'W'.
72.C1.96              05  S3-IMAGE6-INT PIC X(1) VALUE 'N'.
```

Example 8–7. Inserting Screen Definitions with Repeating Image (contd.)

```
72.C1.97              05  S3-IMAGE6-HIGH PIC X(1) VALUE 'N'.
72.C1.98              05  S3-IMAGE6-FONT PIC X(1) VALUE 'N'.
72.C1.99              05  S3-IMAGE6-EMPH PIC X(1) VALUE 'N'.
72.C1.100             05  S3-IMAGE7-STAT PIC 9(2)  BINARY VALUE 0.
72.C1.101             05  S3-IMAGE7-TYPE PIC 9(2)  BINARY VALUE 1.
72.C1.102             05  S3-IMAGE7-YCO PIC 9(2)  BINARY VALUE 10.
72.C1.103             05  S3-IMAGE7-XCO PIC 9(2)  BINARY VALUE 16.
72.C1.104             05  S3-IMAGE7-CHAR PIC 9(5)  BINARY VALUE ZERO.
72.C1.105             05  S3-IMAGE7-FID PIC 9(5)  BINARY VALUE 5.
72.C1.106             05  S3-IMAGE7-VAL PIC X(1) VALUE 'V'.
72.C1.107             05  S3-IMAGE7-DYN PIC X(1) VALUE 'U'.
72.C1.108             05  S3-IMAGE7-BACK PIC X(1) VALUE 'E'.
72.C1.109             05  S3-IMAGE7-FORE PIC X(1) VALUE 'W'.
72.C1.110             05  S3-IMAGE7-INT PIC X(1) VALUE 'N'.
72.C1.111             05  S3-IMAGE7-HIGH PIC X(1) VALUE 'N'.
72.C1.112             05  S3-IMAGE7-FONT PIC X(1) VALUE 'N'.
72.C1.113             05  S3-IMAGE7-EMPH PIC X(1) VALUE 'N'.
72.C1.114             05  S3-IMAGE8-STAT PIC 9(2)  BINARY VALUE 0.
72.C1.115             05  S3-IMAGE8-TYPE PIC 9(2)  BINARY VALUE 1.
72.C1.116             05  S3-IMAGE8-YCO PIC 9(2)  BINARY VALUE 11.
72.C1.117             05  S3-IMAGE8-XCO PIC 9(2)  BINARY VALUE 13.
72.C1.118             05  S3-IMAGE8-CHAR PIC 9(5)  BINARY VALUE ZERO.
72.C1.119             05  S3-IMAGE8-FID PIC 9(5)  BINARY VALUE 6.
72.C1.120             05  S3-IMAGE8-VAL PIC X(1) VALUE 'V'.
72.C1.121             05  S3-IMAGE8-DYN PIC X(1) VALUE 'U'.
72.C1.122             05  S3-IMAGE8-BACK PIC X(1) VALUE 'E'.
72.C1.123             05  S3-IMAGE8-FORE PIC X(1) VALUE 'W'.
72.C1.124             05  S3-IMAGE8-INT PIC X(1) VALUE 'N'.
72.C1.125             05  S3-IMAGE8-HIGH PIC X(1) VALUE 'N'.
72.C1.126             05  S3-IMAGE8-FONT PIC X(1) VALUE 'N'.
72.C1.127             05  S3-IMAGE8-EMPH PIC X(1) VALUE 'N'.
72.C1.128             05  S3-IMAGE9-STAT PIC 9(2)  BINARY VALUE 0.
72.C1.129             05  S3-IMAGE9-TYPE PIC 9(2)  BINARY VALUE 1.
72.C1.130             05  S3-IMAGE9-YCO PIC 9(2)  BINARY VALUE 12.
72.C1.131             05  S3-IMAGE9-XCO PIC 9(2)  BINARY VALUE 20.
72.C1.132             05  S3-IMAGE9-CHAR PIC 9(5)  BINARY VALUE ZERO.
72.C1.133             05  S3-IMAGE9-FID PIC 9(5)  BINARY VALUE 7.
72.C1.134              05  S3-IMAGE9-VAL PIC X(1) VALUE 'V'.
72.C1.135              05  S3-IMAGE9-DYN PIC X(1) VALUE 'U'.
72.C1.136              05  S3-IMAGE9-BACK PIC X(1) VALUE 'E'.
72.C1.137              05  S3-IMAGE9-FORE PIC X(1) VALUE 'W'.
72.C1.138              05  S3-IMAGE9-INT PIC X(1) VALUE 'N'.
72.C1.139              05  S3-IMAGE9-HIGH PIC X(1) VALUE 'N'.
72.C1.140              05  S3-IMAGE9-FONT PIC X(1) VALUE 'N'.
72.C1.141              05  S3-IMAGE9-EMPH PIC X(1) VALUE 'N'.
72.C1.142              05  S3-IMAGE10-STAT PIC 9(2)  BINARY VALUE 0.
72.C1.143              05  S3-IMAGE10-TYPE PIC 9(2)  BINARY VALUE 1.
72.C1.144              05  S3-IMAGE10-YCO PIC 9(2)  BINARY VALUE 13.
72.C1.145              05  S3-IMAGE10-XCO PIC 9(2)  BINARY VALUE 21.
72.C1.146              05  S3-IMAGE10-CHAR PIC 9(5)  BINARY VALUE ZERO.
72.C1.147              05  S3-IMAGE10-FID PIC 9(5)  BINARY VALUE 8.
72.C1.148              05  S3-IMAGE10-VAL PIC X(1) VALUE 'V'.
72.C1.149              05  S3-IMAGE10-DYN PIC X(1) VALUE 'U'.
72.C1.150              05  S3-IMAGE10-BACK PIC X(1) VALUE 'E'.
72.C1.151              05  S3-IMAGE10-FORE PIC X(1) VALUE 'W'.
72.C1.152              05  S3-IMAGE10-INT PIC X(1) VALUE 'N'.
72.C1.153              05  S3-IMAGE10-HIGH PIC X(1) VALUE 'N'.
72.C1.154              05  S3-IMAGE10-FONT PIC X(1) VALUE 'N'.
```

Example 8–7. Inserting Screen Definitions with Repeating Image (contd.)

```
72.C1.155              05  S3-IMAGE10-EMPH PIC X(1) VALUE 'N'.
72.C1.156              05  S3-REPEAT-1-FCA-ST.
72.C1.157                  15  FILLER PIC 9(2)  BINARY VALUE 0.
72.C1.158                  15  FILLER PIC 9(2)  BINARY VALUE 1.
72.C1.159                  15  FILLER PIC 9(2)  BINARY VALUE 14.
72.C1.160                  15  FILLER PIC 9(2)  BINARY VALUE 18.
72.C1.161                  15  FILLER PIC 9(5)  BINARY VALUE ZERO.
72.C1.162                  15  FILLER PIC 9(5)  BINARY VALUE 9.
72.C1.163                  15  FILLER PIC X(1) VALUE 'V'.
72.C1.164                  15  FILLER PIC X(1) VALUE 'U'.
72.C1.165                  15  FILLER PIC X(1) VALUE 'E'.
72.C1.166                  15  FILLER PIC X(1) VALUE 'W'.
72.C1.167                  15  FILLER PIC X(1) VALUE 'N'.
72.C1.168                  15  FILLER PIC X(1) VALUE 'N'.
72.C1.169                  15  FILLER PIC X(1) VALUE 'N'.
72.C1.170                  15  FILLER PIC X(1) VALUE 'N'.
72.C1.171                  15  FILLER PIC 9(2)  BINARY VALUE 0.
72.C1.172                  15  FILLER PIC 9(2)  BINARY VALUE 1.
72.C1.173                  15  FILLER PIC 9(2)  BINARY VALUE 14.
72.C1.174                  15  FILLER PIC 9(2)  BINARY VALUE 26.
72.C1.175                  15  FILLER PIC 9(5)  BINARY VALUE ZERO.
72.C1.176                  15  FILLER PIC 9(5)  BINARY VALUE 10.
72.C1.177                  15  FILLER PIC X(1) VALUE 'V'.
72.C1.178                  15  FILLER PIC X(1) VALUE 'U'.
72.C1.179                  15  FILLER PIC X(1) VALUE 'E'.
72.C1.180                  15  FILLER PIC X(1) VALUE 'W'.
72.C1.181                  15  FILLER PIC X(1) VALUE 'N'.
72.C1.182                  15  FILLER PIC X(1) VALUE 'N'.
72.C1.183                  15  FILLER PIC X(1) VALUE 'N'.
72.C1.184                  15  FILLER PIC X(1) VALUE 'N'.
72.C1.185                  15  FILLER PIC 9(2)  BINARY VALUE 0.
72.C1.186                  15  FILLER PIC 9(2)  BINARY VALUE 1.
72.C1.187                  15  FILLER PIC 9(2)  BINARY VALUE 14.
72.C1.188                  15  FILLER PIC 9(2)  BINARY VALUE 34.
72.C1.189                  15  FILLER PIC 9(5)  BINARY VALUE ZERO.
72.C1.190                  15  FILLER PIC 9(5)  BINARY VALUE 11.
72.C1.191                  15  FILLER PIC X(1) VALUE 'V'.
72.C1.192                  15  FILLER PIC X(1) VALUE 'U'.
72.C1.193                  15  FILLER PIC X(1) VALUE 'E'.
72.C1.194                  15  FILLER PIC X(1) VALUE 'W'.
72.C1.195                  15  FILLER PIC X(1) VALUE 'N'.
72.C1.196                  15  FILLER PIC X(1) VALUE 'N'.
72.C1.197                  15  FILLER PIC X(1) VALUE 'N'.
72.C1.198                  15  FILLER PIC X(1) VALUE 'N'.
72.C1.199          05  S3-REPEAT-1-FCA-END REDEFINES S3-REPEAT-1-FCA-ST.
72.C1.200                  10  S3-HREPEAT-1-FCA OCCURS 3 TIMES
72.C1.201                          INDEXED BY S3-R-INDEX-FCA.
72.C1.202                  15  S3-IMAGE1-STAT PIC 9(2)  BINARY.
72.C1.203                  15  S3-IMAGE1-TYPE PIC 9(2)  BINARY.
72.C1.204                  15  S3-IMAGE1-YCO PIC 9(2)  BINARY.
72.C1.205                  15  S3-IMAGE1-XCO PIC 9(2)  BINARY.
72.C1.206                  15  S3-IMAGE1-CHAR PIC 9(5)  BINARY.
72.C1.207                  15  S3-IMAGE1-FID PIC 9(5)  BINARY.
72.C1.208                  15  S3-IMAGE1-VAL PIC X(1).
72.C1.209                  15  S3-IMAGE1-DYN PIC X(1).
72.C1.210                  15  S3-IMAGE1-BACK PIC X(1).
72.C1.211                  15  S3-IMAGE1-FORE PIC X(1).
72.C1.212                  15  S3-IMAGE1-INT PIC X(1).
```

Example 8–7. Inserting Screen Definitions with Repeating Image (contd.)

```
72.C1.213                15  S3-IMAGE1-HIGH PIC X(1).
72.C1.214                15  S3-IMAGE1-FONT PIC X(1).
72.C1.215                15  S3-IMAGE1-EMPH PIC X(1).
72.C1.216        02  SCREEN-LSPI3-3-FCA-END REDEFINES SCREEN-LSPI3-3-FCA.
72.C1.217            05  SCREEN-LSPI3-3-FCA-REDEF OCCURS 11 TIMES.
72.C1.218                15  S3-FCA-STAT PIC 9(2)  BINARY.
72.C1.219                15  S3-FCA-TYPE PIC 9(2)  BINARY.
72.C1.220                15  S3-FCA-YCO PIC 9(2)  BINARY.
72.C1.221                15  S3-FCA-XCO PIC 9(2)  BINARY.
72.C1.222                15  S3-FCA-CHAR PIC 9(5)  BINARY.
72.C1.223                15  S3-FCA-FID PIC 9(5)  BINARY.
72.C1.224                15  S3-FCA-VAL PIC X(1).
72.C1.225                15  S3-FCA-DYN PIC X(1).
72.C1.226                15  S3-FCA-BACK PIC X(1).
72.C1.227                15  S3-FCA-FORE PIC X(1).
72.C1.228                15  S3-FCA-INT PIC X(1).
72.C1.229                15  S3-FCA-HIGH PIC X(1).
72.C1.230                15  S3-FCA-FONT PIC X(1).
72.C1.231                15  S3-FCA-EMPH PIC X(1).
72.C1.232        02  SCREEN-LSPI3-3-DATA.
72.C1.233            05  S3-IMAGE3 PIC A(6) VALUE SPACES.
72.C1.234            05  FILLER PIC X(2).
72.C1.235            05  S3-IMAGE4 PIC S1(18) BINARY-1 VALUE ZERO.
72.C1.236            05  FILLER PIC 1(18).
72.C1.237            05  S3-IMAGE5 PIC 9(6) BINARY VALUE ZERO.
72.C1.238            05  FILLER PIC X(1).
72.C1.239            05  S3-IMAGE6 PIC 9(10) BINARY VALUE ZERO.
72.C1.240            05  S3-IMAGE7 PIC 9(6) VALUE ZERO.
72.C1.241            05  FILLER PIC X(2).
72.C1.242            05  S3-IMAGE8 COMP-1 VALUE ZERO.
72.C1.243            05  S3-IMAGE9 COMP-2 VALUE ZERO.
72.C1.244            05  S3-IMAGE10 PIC X(6) VALUE SPACES.
72.C1.245            05  FILLER PIC X(2).
72.C1.246            05  S3-REPEAT-1-DATA-ST.
72.C1.247                   15  FILLER PIC X(6) VALUE SPACES.
72.C1.248                   15  FILLER PIC X(2).
72.C1.249                   15  FILLER PIC X(6) VALUE SPACES.
72.C1.250                   15  FILLER PIC X(2).
72.C1.251                   15  FILLER PIC X(6) VALUE SPACES.
72.C1.252                   15  FILLER PIC X(2).
72.C1.253         05  S3-REPEAT-1-DATA-END REDEFINES S3-REPEAT-1-DATA-ST.
72.C1.254                   10  S3-HREPEAT-1-DATA OCCURS 3 TIMES
72.C1.255                       INDEXED BY S3-R-INDEX-DATA.
72.C1.256                   15  S3-IMAGE1 PIC X(6).
72.C1.257                   15  FILLER PIC X(2).
```

Example 8–7. Inserting Screen Definitions with Repeating Image

8.2.4. Inserting Source Code with INVOKE

Function

You can use an INVOKE statement to bring source code (data declarations or executable statements) that are stored in UREP into a program. You can bring this source code into any location of the program if the syntax of the code is valid for that location.

Format

```
INVOKE {SOURCE source-name} FROM REPOSITORY.
```

where *source-name* is the name associated with a block of UCS COBOL code that exists in the UREP repository.

Result

The INVOKE SOURCE command causes UREP to place the source code with the specified *source-name* into the source program at the location of the INVOKE. The line numbering for the new source lines is the same as for a COPY statement.

Example

Example 8–8 shows the results of generating a source code sequences with the INVOKE statement.

```
1                              invoke source identiv from repository.
1.C1.1          1              IDENTIFICATION  DIVISION.
1.C1.2                         PROGRAM-ID.     IDDS-TST4.
2                              INVOKE SOURCE ENVDIV   FROM REPOSITORY.
2.C1.1                         ENVIRONMENT  DIVISION.
2.C1.2                         CONFIGURATION  SECTION.
2.C1.3                         SOURCE-COMPUTER.  UNISYS-2200.
2.C1.4                         OBJECT-COMPUTER.  UNISYS-2200.
2.C1.5                         SPECIAL-NAMES.
2.C1.6                             PRINTER IS PRT.
3                              ***
4                              DATA  DIVISION.
5                              *
6                                  INVOKE SOURCE WKSTORE    FROM
REPOSITORY.
6.C1.1                         WORKING-STORAGE SECTION.
6.C1.2                         01  AA  PIC  X(3).
6.C1.3                         01  BB.
6.C1.4                             05  CC  PIC X(5).
6.C1.5                             05  DD  PIC S9(9)V9(8).
6.C1.6                         77  EE      PIC 99.
7                              ***
8               1              PROCEDURE DIVISION.
9                                  INVOKE SOURCE PROCDIV FROM REPOSITORY.
9.C1.1          1              BEGIN.
9.C1.2          1                  DISPLAY "WELCOME TO UREP TESTING."
UPON PRT.
9.C1.3          1                  MOVE "XYZ" TO AA.
9.C1.4          1                  MOVE "STUVW" TO CC.
```

```
9.C1.5          1                   DISPLAY "DONE WITH THIS SECTION." UPON
PRT.
10              1                   STOP RUN.
```

Example 8–8. Invoking a SOURCE Statement

8.3. Updating UREP (UREP-XREF)

If the UREP-XREF keyword option is in effect during compilation, UCS COBOL updates the repository using program source code. UCS COBOL checks an entity of type OPTIONS and name UCS to see how much cross-reference information it should store for the following attributes:

- DMS-LEVEL

  DMS items inserted in the program with the INVOKE statement
- RDMS-LEVEL

  RDMS relational database items referenced in embedded SQL commands, as well as tables and views inserted into the program with the INVOKE statement
- DPS-LEVEL

  DPS forms inserted in the program with the INVOKE statement
- INVOKE-LEVEL

  All other items inserted into the program with the INVOKE statement
- COPY-LEVEL

  All items inserted into the program with the COPY statement

The amount of information stored in UREP is controlled by the values specified for each of these attributes. The possible values for the attributes are NONE, PARTIAL, or COMPLETE. Each attribute assumes the default value PARTIAL if either of the following is true:

- You do not specify a value for the attribute.
- You do not create an OPTIONS UCS entity in UREP (all of the attributes take the default value).

UREP tracks one or more of the following relationships between the UCS COBOL source program (MODULE) and individual items, and stores this information in the repository for the application:

- WRITES (changes were made to the contents of an item)
- REFERENCES (references were made to an item; however, there were no changes)
- CREATES (DMS STORE, RDMS INSERT)

If you specify the value PARTIAL for an attribute, UREP consolidates all of the activity that occurs for subitems of a higher-level item named in an INVOKE or COPY statement as activity for the higher-level item. Thus, UREP tracks the WRITES and REFERENCES relationships to the source program as if they are for the high-level item.

For example, the UCS COBOL program P1 contains the following statement:

```
MOVE 1 TO data-name-a
```

where *data-name-a* is a level 02 subitem of RECORD-B that is updated during execution of the program. With PARTIAL in effect, UREP tracks this relationship for the high-level item, as follows:

```
MODULE P1 WRITES TO RECORD-B
```

If you specify the value COMPLETE for an attribute, all of the subitems are tracked individually.

The INVOKE or COPY statement is considered to be a REFERENCES relationship to the higher-level named item.

See the *Enterprise Servers Repository for ClearPath OS 2200 Administration Guide* for more information about storing cross-reference information.

The compiler stores the UREP-XREF information after successful generation of an object module (no major errors during compilation). Errors in the storing process or in accessing UREP have no effect on the production of the object module if there are no major errors; however, UREP does not store the cross-reference information.

You can recover this cross-reference information by recompiling the program with the NO-CODE compiler keyword option. This keyword causes the compiler to store the cross-reference information in UREP for programs that successfully compile, but suppresses the creation of an object module.

Section 9
Accessing Program-Callable FURPUR (PCFP)

Program-Callable FURPUR (PCFP) performs most of the file and element handling functions of the FURPUR statements. It can be called from both user programs and system processors. PCFP gives you much more capability in specifying partial names than FURPUR does; it can match both single characters or variable length strings by using wild card characters.

To assist you in calling it, PCFP contains definition elements for all the packets that can be passed to it, and for all return entries and structures that it returns to the caller. There is a set of definition elements for each of the following UCS languages:

- C
- COBOL
- FORTRAN

For complete information on this product, refer to the *PCFP Programming Reference Manual*.

Section 10
Using Executive Requests

By using the UCS Runtime System to call Executive requests (ER), your UCS COBOL programs can call the Exec to perform functions such as obtain system times, print the date, or terminate a program. Figure 10–1 illustrates how UCS COBOL programs make these calls. The UCS COBOL compiler accepts these calls within your source program and the Linking System resolves their references during linking.

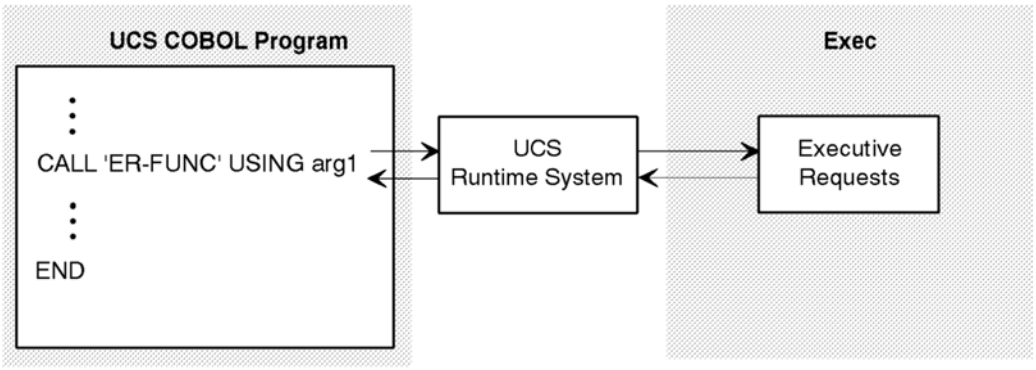


Figure 10–1. Using Executive Requests from UCS COBOL

Accessing the ERs from a UCS COBOL program executing in extended mode is nontrivial because ERs execute only in basic mode. This places restrictions on overlapping addresses and multiple banks in UCS programs. Most of the complexity in accessing the ERs, however, is hidden by the interface in the UCS Runtime System. The system accomplishes this for most of the ERs by:

- Placing all of the data and packets into a bank that is always accessible in basic mode
- Performing ASCII to Fieldata and Fieldata to ASCII conversions where necessary

Several of the ER calls require you to provide additional information or meet constraints (presented later). You should also note the following requirements when calling ERs from a UCS COBOL program:

- You must use the supplied packet building routine UCSSYMBPK$ (see 10.6.1) for the FSYMB subroutine.
- Integer arguments must be declared as PIC 9(10) BINARY to ensure that a full word is allocated.

This section provides information about using standard UCS Runtime System subroutine calls to access ERs in a UCS COBOL program. You must have prior knowledge of how to use ERs to fully understand how to implement them in your UCS COBOL program (see the *ER Programming Reference Manual* for specific information about how to use ERs).

Table 10–1 lists the Executive requests that UCS COBOL explicitly supports. The "Subsection" column indicates where you can find more information about the requests listed in the table.

Table 10–1. Executive Requests

Executive Request Function	Subsection
ABORT$	10.5
ACSF$	10.1
CLIST$	10.13
CNVTIM$	10.19
COND$	10.2
DATE$	10.3
DWTIME$	10.18
EABT$	10.5
ERR$	10.5
EXIT$	10.5
FITEM$	10.14
INFO$	10.15
INITREG$	10.21
IO$	10.6
IOW$	10.6
OPT$	10.8
PCT$	10.9
PFD$	10.7
PFI$	10.7
PFS$	10.7

Table 10–1. Executive Requests

Executive Request Function	Subsection
PFUWL$	10.7.3
PFWL$	10.7.3
SETC$	10.2
SYMB$	10.6
TDATE$	10.3
TIME$	10.17
TINTL$	10.11
TLBL$	10.10
TSWAP$	10.12
UNLCK$	10.6
WAIT$	10.6
WANY$	10.6

Additional ERs are supported through UCSGENERAL$. UCS COBOL does not support the following ER functions:

BRD$
BSD$
BSRD$
PSA$
RDL$
RRSD
SRD$
SW$
TSA$
TSF$

10.1. Dynamic Control Statement Requests (ERACSF, FACSF, FACSF2, IACSF, IACSF2)

Function

Permits your program to submit an Exec control statement image (arg1) for interpretation and processing during program execution. FACSF, FACSF2, IACSF, and IACSF2 all refer to the Executive request function ACSF$.

Format

$$\text{CALL} \left\{ \begin{matrix} \text{'FACSF'} \\ \text{'FACSF2'} \\ \text{'IACSF'} \\ \text{'IACSF2'} \end{matrix} \right\} \text{USING } arg1[\,, arg2]$$

$$\text{CALL ['ERACSF'] USING } arg1, arg3$$

where:

arg1

is a character scalar expression or contiguous character array containing one of the control statements from the list below. The expression *arg1* must not be longer than 84 characters and must be terminated by the character sequence: blank, period, blank.

arg2

is an integer scalar expression giving the length of the control image in characters. Define *arg2* as usage BINARY.

arg3

is an integer expression in which the error status is returned. Define *arg3* as PIC 1(36) BINARY-1.

Description

Valid control statements for *arg1* are as follows:

Statement	Use
@ADD	Add to runstream
@ASG	Assign a file
@BRKPT	Breakpoint symbiont output files
@CAT	Catalog a file
@CKPT	Produce checkpoint dump of this run

Statement	Use
@FREE	Deassign a file
@LOG	Log message to the master log file
@MODE	Set mode and/or noise constant for tape file
@QUAL	Specify a file qualifier
@RSTRT	Restart run whose checkpoint dump is saved by @CKPT
@START	Schedule an independent run
@SYM	Queue files for symbiont processing
@USE	Associate internal to external file name

If *arg2* is present, the control image length can be up to 240 characters. If the value of *arg2* is greater than 240, the system uses 240 as the length.

If *arg2* is missing or the program uses CALL 'ERACSF', the default length for the control image is 84 characters. If the control image is actually less than 84 characters, it should be terminated by the character sequence: blank, period, blank. Otherwise, extraneous characters following the actual control statement within the 84 character image could cause an Exec error.

If the length of *arg1* is less than the length to be used for the control image, the system pads the difference with blanks. For example, the following code passes 84 characters to the ER with the last 68 being blanks:

```
01  IMAGE PIC X(16) VALUE
                    '@ASG,A TEMPFILE.'.
    CALL 'FACSF' USING IMAGE.
```

The following code sequence passes '@ASG' to the ER:

```
01  IMAGE.
     02  ASG OCCURS 4 TIMES PIC X(4).
     MOVE '@ASG,A TEMPFILE2' TO IMAGE.

     CALL 'FACSF' USING ASG(1), 16.
```

FACSF2 is the same as FACSF, except that FACSF prints an error message and terminates execution if the system rejects the ACSF$ Executive request. FACSF2 ignores the status returned by the ACSF$ Executive request and continues execution.

IACSF and IACSF2 are the same entry points as FACSF and FACSF2.

Example

```
01  ASG PIC X(20) VALUE '@ASG,A FILENAME. . '.
     CALL 'FACSF' USING ASG.
```

```
* This call would attempt to assign the file FILENAME by referring to
* the Executive request function ACSF$.
```

10.2. Setting and Retrieving the Condition Word (FSETC, FCOND)

Function

Provides routines to modify and retrieve the condition word.

FSETC

Places (sets) the contents of the lower third (bits 25-36) and bits 8 and 9 of *arg* in the corresponding run condition word. You can use the run condition word as a flag that can be tested by the control statement @TEST or retrieved by the COBOL call CALL 'FCOND' USING *arg* and then tested (see 2.5). FSETC refers to the ER function SETC$. See the *ER Programming Reference Manual* for more information for bit settings. Note that UCS documentation refers to the condition word as bits 1 to 36 while the Exec documentation refers to 0 to 35.

FCOND

Retrieves the condition word and makes it available to you in *arg.* FCOND refers to the Executive request function COND$.

Formats

```
CALL 'FSETC' USING arg.
CALL 'FCOND' USING arg.
```

where *arg* is an integer scalar expression for FSETC; it is an integer variable or array element for FCOND.

Prior to the execution of the following example, the following command must be performed:

```
@SETC 7777
```

Example

```
IDENTIFICATION DIVISION.
PROGRAM-ID.
    er-fsetc-fcond.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
SPECIAL-NAMES.
    PRINTER IS PRT.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
DATA DIVISION.
```

```
FILE SECTION.
WORKING-STORAGE SECTION.
01  bits.
    02 b0    pic 1 binary-1.
    02 b1    pic 1 binary-1.
    02 b2    pic 1 binary-1.
    02 b3    pic 1 binary-1.
    02 b4    pic 1 binary-1.
    02 b5    pic 1 binary-1.
    02 b6    pic 1 binary-1.
    02 b7    pic 1 binary-1.
    02 b8    pic 1 binary-1.
    02 b9    pic 1 binary-1.
    02 b10   pic 1 binary-1.
    02 b11   pic 1 binary-1.
    02 b12   pic 1 binary-1.
    02 b13   pic 1 binary-1.
    02 b14   pic 1 binary-1.
    02 b15   pic 1 binary-1.
    02 b16   pic 1 binary-1.
    02 b17   pic 1 binary-1.
    02 b18   pic 1 binary-1.
    02 b19   pic 1 binary-1.
    02 b20   pic 1 binary-1.
    02 b21   pic 1 binary-1.
    02 b22   pic 1 binary-1.
    02 b23   pic 1 binary-1.
    02 b24   pic 1 binary-1.
    02 b25   pic 1 binary-1.
    02 b26   pic 1 binary-1.
    02 b27   pic 1 binary-1.
    02 b28   pic 1 binary-1.
    02 b29   pic 1 binary-1.
    02 b30   pic 1 binary-1.
    02 b31   pic 1 binary-1.
    02 b32   pic 1 binary-1.
    02 b33   pic 1 binary-1.
    02 b34   pic 1 binary-1.
    02 b35   pic 1 binary-1.
PROCEDURE DIVISION.
BEGIN.
    display ' ' upon prt.

    call 'fcond' using bits.
    if b12 = 1 and b13 = 1 and b14 = 1 and b15 = 1 and b16 = 1
       and b17 = 1 and b18 = 1 and b19 = 1 and b20 = 1 and
       b21 = 1 and b22 = 1 and b23 = 1
       display 'Test #1 Passes' upon prt
    else
       display 'Test #1 Fails: ' upon prt
       display 'Expected t2 = 111111111111' upon prt
       display 'Actual t2 = '
```

```
            b12 b13 b14 b15 b16 b17 b18 b19 b20 b21 b22 b23
            upon prt.

   move all zeros to bits.
   move 1 to b33 b34 b35.
   call 'fsetc' using bits.
   move all zeros to bits.
   call 'fcond' using bits.
   if b33 = 1 and b34 = 1 and b35 = 1
      display 'Test #2 Passes' upon prt
   else
      display 'Test #2 Fails:' upon prt
      display 'Actual bits = ' b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10
           b11 b12 b13 b14 b15 b16 b17 b18 b19 b20 b21 b22 b23
           b24 b25 b26 b27 b28 b29 b30 b31 b32 b33 b34 b35 upon
                                                        prt.
   stop run.
```

10.3. Retrieving Date and Time in ASCII (ADATE)

Function

Retrieves the time and date in ASCII. ADATE refers to the Executive request function DATE$.

Format

```
CALL 'ADATE' USING arg1, arg2
```

where:

arg1, arg2

are character variables, contiguous character arrays, or character array elements of 6 or more characters in length into which the date and time are stored in ER DATE$ format.

The data in *arg1* is the ASCII character form MMDDYY, as follows:

- MM represents the month. Values returned are 01 through 12.
- DD represents the day. Values returned are 01 through 31.
- YY represents the last two digits of the year. Values returned are 00 through 99. (00-99).

The data in *arg2* is in the ASCII character form HHMMSS, as follows:

- HH represents hours. Values returned are 00 through 23.
- MM represents minutes. Values returned are 00 through 59.

- SS represents seconds. Values returned are 00 through 59.

The first six character positions of each variable, array, or array element are filled with the ASCII character form of the date and time. The remaining characters are space-filled.

Example

```
 01 DATE-1 PIC X(6).
 01 TIME-1 PIC X(6).
*This call supplies the date ('MMDDYY') in DATE-1 and the time
*('HHMMSS') in TIME-1.
     CALL 'ADATE' USING DATE-1 TIME-1.
     DISPLAY 'DATE' DATE-1 'TIME' TIME-1.
```

Note: *Timestamp Overflow - The DATE$ format is not a linear format, so overflow is not applicable in the same manner as a linear time. However, the DATE$ format only provides the last two digits of the year, which might not be sufficient to uniquely identify the time.*
For example, 1964-01-01 cannot be distinguished from 2064-01-01. Due to this limitation and the local time base used, you might want to consider other timestamp formats.

10.4. Retrieving Time and Date (UCSTDATE$)

Retrieves the time and date in binary. UCSTDATE$ refers to the ER function TDATE$.

Format

```
CALL 'UCSDATE$' USING arg
```

where:

arg

is a one-word variable or array element into which the binary date and time are stored in ER TDATE$ format.

Note: *Timestamp Overflow - The year portion of the TDATE$ timestamp format will wrap after 2027-12-31 23:59:59 resulting in times which are ambiguous and considered invalid. Unisys strongly recommends that you do not use ER TDATE$ and the TDATE$ timestamp format within new programs or routines. Further, we recommend that you should examine your existing use of the ER and the TDATE$ timestamp format and consider using other timestamp formats. For more information, refer to the Time Considerations for ClearPath OS 2200 Systems.*

10.5. Activity Termination (FEXIT, FERR, FABORT, FEABT)

Function

Provides the following forms of activity termination:

FEXIT

Terminates the current activity. Refers to the Executive request function EXIT$.

FERR

Terminates the activity in error. One of two entry points for the Executive request function ERR$.

FABORT

Unconditionally terminates all activities and the run if the run is batch mode. Refers to the Executive request function ABORT$.

FEABT

Unconditionally terminates all activities but allows error diagnostics. FEABT is similar to FABORT except that register dumps are provided and PMD requests are honored. One of two entry points for the Executive request function EABT$.

The following are alternate entry points for the Executive requests ERR$ and EABT$.

ERERR$

Performs the same function as FERR. Terminates the activity in error. Refers to the Executive request function ERR$.

EREABT$

Performs the same function as FEABT. Unconditionally terminates all activities but allows error diagnostics. EREABT$ is similar to FABORT except that register dumps are provided and PMD requests are honored. Refers to the Executive request function EABT$.

Formats

```
CALL 'FEXIT'.
CALL 'FERR'.
CALL 'FABORT'.
CALL 'FEABT'.
CALL 'ERERR$'.
CALL 'EREABT$'.
```

10.6. Input/Output ERs (FIO, FIOW, FSYMB, FUNLCK, FWANY, FWST)

Function

The following table lists the COBOL I/O Executive requests and their descriptions.

Executive Request	Description
FIO	Requests an operation on an I/O device. Returns control to the executing program without waiting for completion of the I/O operation.
FIOW	Requests an operation on an I/O device. Returns control upon completion of the I/O operation.
FSYMB	Provides a means to request certain symbiont Executive request functions, character transfer, and some expanded READ$ capacity.
FUNLCK	Enables an input/output interrupt activity to reduce its switching priority to the priority of the activity that initiated the input/output request.
FWANY	Delays execution until any current input/output operation is completed.
FWST	Delays execution until the I/O operation controlled by the specified I/O packet has completed.

Formats

```
CALL 'FIO' USING pkt.
CALL 'FIOW' USING pkt.
CALL 'FSYMB' USING pkt.
CALL 'FWST' USING pkt.

CALL 'FUNLCK'.
CALL 'FWANY'.
```

where *pkt* is a 13-word table filled with input/output packet information. See 10.6.4 for examples of these ERs.

Description

Two basic steps are necessary when using these routines:

1. Use the UCS COBOL subroutine UCSMAKEIOPK$ (see 10.6.2) to construct the packets for FIO, FIOW, and FWST. Use the UCSSYMBPK$ subroutine to construct the packet for FSYMB (see 10.6.1). You do not need to construct a packet for FWANY and FUNLCK.
2. Call the I/O Executive request function, passing the I/O packet as the first argument of the call.

   Note: *If the I/O routines are called from a shared subsystem, the data packets and buffers must reside in a data bank with a LOW BDI.*

The Executive request is done with register A0 pointing to the program-constructed input/output packet. This provides you with full control over the input/output operation and complete error analysis capability.

There are no buffers external to these routines. They are all reentrant at both the program and the activity level.

The following describes the activity that occurs when each procedure is called.

FIO

1. Obtains the parameter *pkt*, which points to the I/O packet
2. References the ER function IO$
3. Returns control immediately to the caller

Construct the I/O packet with UCSMAKEIOPK$ (see 10.6.2). It should contain 125 words to allow I/O by bank descriptor index (BDI), but it never needs more than 125 words.

The I/O packet must be in static storage preceded by at least 128 words, which can contain anything other than I/O packets. The following example illustrates a constructed I/O packet with 125 words (500 bytes), preceded by 128 words (512 bytes).

```
*  The following common block declaration declares 125 words of static
*  storage preceded by the required 128 words of unused storage:

 01  XYZ.
     02 FILLER PIC X(512).
     02 IOPAK  PIC X(500).
```

FIOW

1. Obtains the parameter *pkt*, which points to the I/O packet
2. References the ER function IOW$
3. Waits for completion before returning to the caller

Construct the I/O packet with UCSMAKEIOPK$ (see 10.6.2). It should contain 125 words to allow I/O by bank descriptor index (BDI), but it never needs more than 125 words. UCSMAKEIOPK$ tells you the number of words the generated I/O packet actually uses.

FSYMB

1. Obtains the parameter *pkt*, which points to the I/O packet
2. References the ER function SYMB$
3. Returns control immediately to the caller

Construct the symbiont I/O packet with UCSSYMBPK$ (see 10.6.1). It must be 13 words long.

This procedure does not support Fieldata records.

FUNLCK

1. References the ER function UNLCK$, which enables an I/O interrupt activity to reduce its switching priority to the priority of the activity that initiated the I/O request
2. Returns control immediately to the caller

FWANY

1. References the ER function WANY$
2. Waits for completion of any outstanding input or output activity before returning to the caller

FWST

1. Obtains the parameter *pkt*, which points to the I/O packet
2. References the ER function WAIT$
3. Waits for completion of any outstanding input or output activity before returning to the caller

10.6.1. Building a Symbiont I/O Packet (UCSSYMBPK$)

Function

This subroutine builds a SYMB$ packet that can be used for one or more FSYMB calls (see 10.6). I/O packets for FSYMB must be created by UCSSYMBPK$.

Format

```
CALL 'UCSSYMBPK$' USING packet, file, function, modes, character count1,
buffer1
[ , line spacing [ , character count2, buffer2 ] ]
```

where:

packet

is a 13-word contiguous array in which the symbiont packet is built.

file

is a character scalar expression or contiguous character array that contains the file name to be put in the symbiont packet. The standard symbionts (READ$, PRINT$, and PUNCH$) are specified as ASCII character strings (for example, 'READ$' means READ$).

function

is an integer scalar expression with the following permissible values.

Value	Description
8	Write.
16	Read.
34	Submit print-control or punch-control image. See the *ER Programming Reference Manual*.
37	Write followed by read.
45	Submit print-file control image. See the *ER Programming Reference Manual*.

modes

is an integer scalar expression with its value formed as the sum of any of the four integers defined as follows:

Value	Description
1	ASCII data to be transmitted.
2	Truncate if image read is longer than the buffer.
8	Used on first write call to an alternate punch file to inform the Executive that the file being written is an alternate punch file and not an alternate print file.
16	Special translation information. If you use this, you must fill in the translation table number and the character type in the symbiont packet after the UCSSYMBPK$ call is done.

character count1, character count2

are integer scalar expressions that specify the number of characters to be transferred. *Character count2* is only meaningful when *function* is FWR; it indicates the maximum number of characters accepted for the input buffer.

buffer1, buffer2

are character variables, contiguous character arrays, or character array elements (or a constant, in the case of *buffer1*), that contain the image transferred. *Buffer2* is only meaningful when *function* is FWR, in which case it is the input buffer.

line spacing

is an integer scalar expression that specifies the line spacing. It is meaningful only for print files; the default value is 1.

Example

See10.6.4.

10.6.2. Building an I/O Packet (UCSMAKEIOPK$)

The UCSMAKEIOPK$ subroutine builds an I/O packet to be used with the I/O ER functions described in 10.6. It must be used to build packets for FIO, FIOW, and FWST.

The I/O packet built can be used for more than one subroutine call. It must not be changed for use with I/O Executive requests, except for some fields that can be changed for use with FIO and FIOW.

Format

```
CALL 'UCSMAKEIOPK$' USING I/O-packet, file-name, function,
mass-storage-address, number-of-buffers, directions, word-counts, buffers ,
number-of-words-used [,MR-functions, MR-flags, MR-data-format ]
```

where:

I/O-packet

is a contiguous array of 125 words in which the I/O packet is built.

For UCS COBOL, addressing is a problem when using ERs. The 125-word buffer area is used for both the I/O packet and access word pairs, because all read and write operations are converted to I/O by BDI. You should not alter any I/O packet created by UCSMAKEIOPK$, except to change function, mass-storage-address, and MR-data-format for FIO, FIOW, and FWST.

For FIO, the I/O packet must be in static storage preceded by at least 128 words.

Example

```
*  The following common block declaration produces 125 words of static
*  storage preceded by 128 words of unused storage.
 01  XYZ.
     02 Filler PIC X(512).
     02 IOPAK  PIC X(500).
```

To reference individual fields within the I/O packet, you can define IOPAK as a structure with the elementary items referenced to match the packet as defined in the ER Programming Reference Manual. For example, if the I/O status returned was to be referenced, IOPAK could be identified as follows:

```
02   IOPAK
     03 PIC X(12).
     03 IOSTAT PIC 1(6) BINARY-1.
     03 PIC 1(3) BINARY-1.
     03 PIC X(487).
```

file-name

is a character scalar expression or contiguous character array describing the internal file name, as defined in the *ER Programming Reference Manual.*

function

is an integer scalar expression describing the I/O function to be performed from the following table. Any I/O function not in this list causes UCSMAKEIOPK$ to set the error flag; that is, it sets *number-of-words-used* to zero or negative.

I/O Executive Request	Value
EACQ$ (Extended acquire) You must fill in words 6, 7, and 8 of the I/O packet. Uses *word-counts*(1) for the length to acquire. Does not use the other array parameters. *number-of-buffers* must be 1. ABSW$ is not supported.	15
ACQ$ (Acquire) Uses *word-counts*(1) for the length to acquire. Does not use the other array parameters. *number-of-buffers* must be 1.	14
BDR$ (Read by BDI)	44
BDRB$ (Read backward by BDI)	46
BDW$ (Write by BDI)	4
BSF$ (Backspace file)	43
EREL$ (Extended release) Similar to REL$ except word-counts (1) specifies track granularity rather than a word count. 65,535 tracks can be released on a single I/O request.	23
FSAFE$ (File update wait)	59

I/O Executive Request	Value
FSF$ (Forward space file)	42
GW$ (Gather write; converted to FBDW)	13
MB$ (Move backward)	41
MF$ (Move forward)	40
MR$ (Multirequest)	53
MS$ (Mode set) For this function, there can only be one buffer and it must contain five words. The first word of the buffer contains the MS$ flags; refer to the *ER Programming Reference Manual*. The second and third words contain the mnemonic for the processor data format (in ASCII, left-justified and space-filled). The fourth and fifth words contain the mnemonic for the tape data format (in ASCII, left-justified and space-filled).	45
R$ (Read; converted to BDR$)	16
RB$ (Read backward; converted to BDRB$)	17
REL$ (Release) Only one buffer can be used; at most 65,535 words can be released in one call to FIO or FIOW. Note that RR$ is not supported. Uses *word-counts*(1) for the length to release. Does not use the other array parameters. *number-of-buffers* must be 1.	19
REW$ (Rewind)	32
REWI$ (Rewind with interlock)	33
SCR$ (Scatter read; converted to BDR$)	35
SCRB$ (Scatter read backward; converted to BDRB$)	36
SM$ (Set mode) Only 1 one-word buffer is used; see the *ER Programming Reference Manual* for details.	34
UNL$ (Unlock) At most 65,535 words can be unlocked in one call to FIO or FIOW. Uses *word-counts*(1) for the length to acquire. Does not use the other array parameters. *number-of-buffers* must be 1.	22
W$ (Write; converted to BDW$)	8
WEF$ (Write end-of-file marker)	9

mass-storage-address

an integer scalar expression indicating the mass storage address at which the I/O transfer starts. If you use the I/O packet with a tape file, the recommended value is 0. You can alter this field (word 5) for use with FIO, FIOW, and FWST.

number-of-buffers

is an integer scalar expression indicating the number of buffers to be transferred in the I/O operation. It must be greater than 0 and less than 51.

directions

is a one-dimensional contiguous integer array whose one-word elements give the direction in which addresses are to be changed for each new word going to or from the buffer. The number of elements in the array must equal *number-of-buffers.* Each array element is an integer with a value between 0 and 3, as follows:

Value	Description
0	Transfer starts with the first word of the buffer and continues in a forward direction. The second word is transferred to or from the second word of the buffer, and so on.
1	All words are transferred to or from the first word of the buffer. This direction value can be used to clear an area of mass storage.
2	Transfer starts at the first word of the buffer and continues in a backward direction. The second word is transferred to or from the word before the first word of the buffer; that is, the address of the second word is one lower than the address of the first word. The third word is transferred to or from the word that is two words before the first word of the buffer, and so on. This direction value is normally useful only when reading a tape backwards.
3	This is the same as a value of 1, except that input operations cannot access storage, depending on how the operating system is configured at your site.

word-counts

is a one-dimensional contiguous integer array whose elements give the number of words to be transferred to the corresponding buffer. The number of elements in the array must equal *number-of-buffers*. Each array element is an integer greater than zero.

Except for MR$, the total number of words transferred cannot be greater than 229,376 (which is 128 1792-word tracks). For further information, refer to "Description" in this subsection. For functions that do not allow multiple buffers (that is, those other than MR$, BDR$, BDRB$, BDW$, and the functions converted to one of these), the value of the first element in the array must be less than 65,536.

buffers

is a one-dimensional contiguous integer array whose elements specify the storage address at which each I/O transfer starts. (You can use the LOC$ subroutine to retrieve an address located in working storage or common storage (see 10.6.3).) The number of elements in the array must equal *number-of-buffers*.

Each I/O buffer must start on a word boundary.

Except for the MR$ function, each call to FIOW or FIO writes one block to or reads one block from the external medium, independent of how many buffers were specified in the call to UCSMAKEIOPK$. For more information, refer to "Description" in this subsection.

For MS$, the buffer contains 5 words, as described under *function.*

number-of-words-used

is an integer variable or array element that contains the number of words needed for this I/O packet on exit from UCSMAKEIOPK$. The value of this variable is never set greater than 125.

A zero or negative value for this variable indicates an error was discovered in the parameters passed, as shown in the following table.

Value	Description
0	This value can mean any of the following errors: • *function* contains an invalid code. • *number-of-buffers* not equal to 1 for ACQ$, EACQ$, MS$, REL$, SM$, or UNL$. • *word-counts* less than 1 or greater than 65,536 for one ACW. • The wrong data format is used for MR$.
-1	*number-of-buffers* is out of range (less than 1 or greater than 50).
-2	The value of directions is not 0, 1, 2, or 3.
-3	An element of *word-counts* exceeds 229,376 and the function is not MR$.
-4	The buffers specified occupy more banks than can be based at the point of execution of an I/O ER.
-5	One of the elements of *buffers* has an address that cannot be expressed in basic mode. It cannot be used with a basic mode ER.
-6	Automatic buffer splitting has produced more than 50 access-word pairs. For more information, refer to "Description" in this subsection.

MR-functions

is a one-dimensional contiguous integer array of at least *number-of-buffers* integer values that gives the I/O function for each buffer. Refer to the *ER Programming Reference Manual* for the permissible values.

Pass this argument only if *function* is MR$.

MR-flags

is a one-dimensional contiguous integer array of at least *number-of-buffers* integer values. Bits 31 through 36 of each element, which are numbered as in BITS, are used as the MR$ flags for the corresponding buffer. For a description of these flags, refer to the *ER Programming Reference Manual.*

Pass this argument only if *function* is MR$.

MR-data-format

is an integer scalar expression indicating the data format for all the buffers in the MR$ I/O operation.

Value	Description
1	Quarter-word format
3	8-bit packed format

Any other value causes UCSMAKEIOPK$ to set the error flag. For a description of these formats, see the *ER Programming Reference Manual.*

Pass this argument only if *function* is MR$.

Description

For functions that support more than one buffer (MR$, BDR$, BDRB$, BDW$, and the functions that are converted to these), the buffers are split automatically in order to enforce the following I/O restrictions imposed by the operating system:

- No access-word pair can specify a transfer of more than 65,535 words, because the width of the field specifying the data-transfer length to the I/O ER is 16 bits.
- The set of access-word pairs (for MR$, a single set of data-chained access-word pairs) cannot transfer more than 262,143 words. Mass-storage I/O considerations reduce this limit to 229,376 words, which is a block of 128 1792-word tracks.
- The data transfer specified by an access-word pair cannot span the addresses associated by more than one BDI.

Each buffer specified on a call to UCSMAKEIOPK$ is automatically split into two or more access-word pairs if it violates any of the previous restrictions. The Exec accumulates a running total of words transferred for any sequence of buffers specified with data chaining. When the total reaches 229,376, a split occurs with command chaining set at the split.

The maximum number of words that can be specified for the buffers of an MR$ function depends on various conditions, such as the following:

- Number of banks containing the buffers
- Locations of the buffers within the banks
- Number of buffers

All the BDIs containing the buffers must be based when either Executive request IO$ or IOW$ is performed. The required basing is performed automatically by FIO, FIOW, or FWST, up to a maximum of eight BDIs. A single BDI describes a maximum of 262,144 words, so the largest buffer that can be transferred in a single request is 2,097,152 words (262,144 multiplied by 8).

You can specify a larger data transfer by, for example, writing a buffer more than once. You could write a single 60,000-word buffer 50 times, resulting in the transfer of 3,000,000 words. This might be useful in initializing a mass-storage file.

If you write an array of more than 2,097,152 words, split it into multiple calls on boundaries that are a multiple of 229,376 words. Alternatively, split a large array into multiple records on 131,072-word boundaries. Either approach ensures that automatic buffer splitting does not change the record structure in the file. If automatic buffer splitting is allowed to create more than one record from a single buffer, you must use exactly the same sequence to read the data back in, to avoid altering the assumed record sizes.

Example

See 10.6.4.

10.6.3. Retrieving a Buffer Address (LOC$)

Function

The LOC$ subroutine provides the capability to retrieve the address of a buffer. It can be used in conjunction with UCSMAKEIOPK$ to build an I/O packet. LOC$ can also be used to set a pointer to reference based storage. Refer to the *UCS COBOL Programming Reference Manual Volume 1*, *COBOL Statements*, "The USING POINTER Clause" for more information.

The successful completion of LOC$ causes the address of *arg1* to be stored in *arg2*.

Format

```
CALL 'LOC$' USING arg1, arg2
```

where:

arg1

is the buffer from which the storage address is retrieved. *arg1* must be a 01 level item. This argument cannot reference an array. Also, you cannot use reference modification on this argument.

arg2

is the variable into which the address (*arg1* address) is stored. *arg2* must be a one-word binary item defined as PIC 1(36) BINARY-1 or USAGE POINTER. This argument cannot reference an array. Also, you cannot use reference modification on this argument. The contents of PIC 1(36) BINARY-1 one word VA pointer argument can be used by the UCSMAKEIOPK$ subroutine (see 10.6.2).

Example

See 10.6.4 for an example.

Note: *All addresses generated by UCS languages are extended mode virtual addresses.*

10.6.4. I/O Executive Request Examples

Example 1

The following example shows how the FSYMB Executive request and the UCSSYMBPK$ subroutine are used.

```
* This program writes 132 characters from BUFFER through SYMB$ if
printer
* allows 132 characters.

 01  PACKET.
     02  PCKT PIC 9(10) BINARY OCCURS 13 TIMES.
 01  BUF.
     02  STR-1 PIC X(110) VALUE 'TEST BUFFER LENGTH OF 132 CHARACTERS'.
     02  STR-2 PIC X(22) VALUE 'END OF 132 CHARACTERS'.
     02  FILLER PIC X(62) VALUE SPACES.
     02  STR-3 PIC X(11) VALUE 'END OF LINE'.
     02  BUFFER REDEFINES BUF PIC X(205).

 CALL 'UCSSYMBPK$' USING PACKET 'PRINT$' 8 1 132 BUFFER.

* CALL SYMB$

 CALL 'FSYMB' USING PACKET.
```

Example 2

The following example illustrates how to create an I/O packet using LOC$ and UCSMAKEIOPK$, and how to read a sector of a file using the FIOW Executive request.

```
IDENTIFICATION DIVISION.
PROGRAM-ID.
    locprog2.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. Unisys-2200.
OBJECT-COMPUTER. Unisys-2200.
```

```
 SPECIAL-NAMES.
     PRINTER IS PRT.
 INPUT-OUTPUT SECTION.
 FILE-CONTROL.
     select f-n assign to disc myfile.
 DATA DIVISION.
 FILE SECTION.
 fd  f-n.
 01  rec pic x(108).
 WORKING-STORAGE SECTION.
 01  number-of-buffers pic 9(10) binary value 1.
 01  number-of-packet-words-used pic 9(10) binary value 0.
 01  receiving-address pic 1(36) binary-1.
 01  directions pic 9(10) binary value 0.
 01  words-to-transfer pic 9(10) binary value 28.
 01  grp.
     02  filler pic  x(512).
 01  i-o-packet.
         03  filler pic x(500).
 01  fr  pic 9(10) binary value 16.
 01  receiving-buffer pic x(112) value all 'x'.
 01  starting-address pic 9(10) binary value 1.
 PROCEDURE DIVISION.
 BEGIN.
*
*  Create a file that contains 4 records
*
     open output f-n.
     perform 4 times
       move all 'A' to rec
       move 'E' to rec(108:1)
       write rec
     end-perform.
     close f-n.
*
*  Create I/O packet
*
     call 'loc$' using receiving-buffer, receiving-address.
     call 'ucsmakeiopk$' using i-o-packet, 'myfile', fr,
           starting-address, number-of-buffers, directions
           words-to-transfer, receiving-address,
           number-of-packet-words-used.
     if number-of-packet-words-used <= 0
         display 'error in building packet'
         stop run
     end-if.
     display 'i/o packet built ok' upon prt.
*
* Read sector of the file MYFILE using ER FIOW
*
     call 'fiow' using i-o-packet.
*
```

```
*   The first word of each sector that is read is control information
*   which is in octal.  It is not displayed here.
*
     display receiving-buffer(5:108) upon prt.
```

10.7. Program File Maintenance Executive Requests

The program file maintenance Executive requests through UCS COBOL fall into two categories: those that require a packet and those that do not.

The program file maintenance ERs that need a packet are

- PFD$ - mark element for deletion
- PFI$ - update element table
- PFS$ - search table of contents

Use these Executive request functions in the following manner:

1. Prepare a skeleton packet using UCSMAKEPFPK$ (see 10.7.1).
2. Fill in the remainder of the packet as described in the *ER Programming Reference Manual* by using BINARY-1 to define bits.
3. Call the appropriate UCS COBOL subroutine to access the Executive request function (see 10.7.2).

The program file maintenance routines that do not need a packet (see 10.7.3) are

- PFUWL$ - update next write location
- PFWL$ - retrieve next write location

10.7.1. Building a Skeleton Packet for Program File Maintenance ERs (UCSMAKEPFPK$)

Function

This subroutine builds a skeleton packet for the program file maintenance Executive request functions that require a packet - PFD$, PFI$, and PFS$ (see 10.7.2).

Format

```
CALL 'UCSMAKEPFPK$' USING packet,file,element,version.
```

where:

packet

is a one-dimensional contiguous array of one-word items that contains at least 12 elements; the skeleton packet is built in this parameter.

file

is a character scalar expression or contiguous character array giving the internal file name of the program file to use; only the first 12 characters are significant.

element

is a character scalar expression or contiguous character array specifying the element name; only the first 12 characters are significant.

version

is a character scalar expression or contiguous character array specifying the version of the element; only the first 12 characters are significant.

Example

```
01  PACKET.
    02  PCKT PIC 9(10) BINARY OCCURS 12 TIMES.
01  FILE-NAME PIC X(12) VALUE '$$$$FILETEST'.
01  NAME PIC X(12) VALUE 'ELEMENT-NAME'.
01  VERSION PIC X(12) VALUE 'ELT-VERSION'.

*  Build a packet for one of these ERs: PFD$, PFI$, PFS$.

     CALL 'UCSMAKEPFPK$' USING PACKET FILE-NAME NAME VERSION.
```

10.7.2. Deleting, Inserting, or Searching for an Element (UCSPFD$, UCSPFI$, UCSPFS$)

Function

For the following program file maintenance Executive request functions, a packet must be built prior to their execution (see 10.7.1):

UCSPFD$

Sets a delete flag in the element table item for the requested element. Refers to the Executive request function PFD$.

UCSPFI$

Inserts an entry into the program file's element table. Refers to the Executive request function PFI$.

UCSPFS$

Searches the program file's table of contents for a given item. Refers to the Executive request function PFS$.

Formats

$$\text{CALL} \left\{ \begin{matrix} \text{'UCSPFD\$'} \\ \text{'UCSPFI\$'} \\ \text{'UCSPFS\$'} \end{matrix} \right\} \text{USING } packet, status$$

where:

packet

is a packet built with the UCSMAKEPFPK$ subroutine (see 10.7.1) which you must complete.

status

is an integer variable or array element into which the status of the Executive request is stored. Define *status* as PIC 1(36) BINARY-1.

Example

```
   01  STAT PIC 9(10) BINARY.
* Set delete flag.
       CALL 'UCSPFD$' USING PACKET STAT.
* Insert entry.
       CALL 'UCSPFI$' USING PACKET STAT.
* Search for item.
       CALL 'UCSPFS$' USING PACKET STAT.
```

10.7.3. Updating and Retrieving Write Limits (UCSPFUWL$, UCSPFWL$)

Function

The following are program file maintenance Executive request functions that do not require a packet:

UCSPFUWL$

Updates the next available write limit in a program file to the address given in register A1. Refers to the Executive request function PFUWL$.

UCSPFWL$

Retrieves the next available write limit in the program file. Refers to the Executive request function PFWL$.

Formats

```
CALL 'UCSPFUWL$' USING file, new-limit, status.
CALL 'UCSPFWL$' USING file, current-limit, status.
```

where:

file

is a character scalar expression or contiguous character array specifying the internal file name of the file whose write limit is to be updated or retrieved (only the first 12 characters are significant).

new-limit

is an integer scalar expression specifying the new write limit for the file.

current-limit

is an integer scalar variable or array element into which the current write limit of the file is stored.

status

is an integer scalar variable or array element into which the status of the Executive request is stored. Define status as PIC 1(36) BINARY-1.

Example

```
    01  USE-FILE PIC X(12) VALUE 'FILE2T'.
    01  WRITE-LIMIT PIC 9(10) BINARY.
    01  STAT PIC 1(36) BINARY-1.
    01  CONSTANT PIC 9(10) BINARY.

  * Retrieve current limit.

        CALL 'UCSPFWL$' USING USE-FILE WRITE-LIMIT STAT.
        ADD CONSTANT TO WRITE-LIMIT.

  * Set new limit.

        CALL 'UCSPFUWL$' USE-FILE WRITE-LIMIT STAT.
```

10.8. Retrieving @XQT or Processor Call Statement Options (UCSOPT$)

Function

Makes available options specified on the @XQT or processor call statement. Refers to Executive request function OPT$.

Format

```
CALL 'UCSOPT$' USING options.
```

where *options* is an integer scalar variable or array element into which the options word (as returned by the Executive request OPT$) is stored.

Example

```
01  OPTRET PIC 9(10) BINARY VALUE ZERO.
    CALL 'UCSOPT$' USING OPTRET.
```

10.9. Retrieving Program Control Table (UCSPCT$)

Function

Makes all or specified portions of the information stored in the program control table (PCT) available to the requesting program. Refers to Executive request function PCT$.

Format

```
CALL 'UCSPCT$' USING number-of-words, start-word, buffer.
```

where:

number-of-words

is an integer scalar expression specifying how many words from the PCT are to be retrieved.

start-word

is an integer scalar expression specifying that word of the PCT at which the retrieval is to start; the first word of the PCT is word 0.

buffer

is a contiguous array of one-word elements into which the words copied from the PCT are stored. The number of elements in the array must be at least as large as *number-of-words.*

Example

```
    01  NUMB PIC 9(10) BINARY.
    01  START-WORD PIC 9(10) BINARY.
    01  BUFR.
        02  BUFR-1 PIC 1(36) BINARY-1 OCCURS 10 TIMES.

        CALL 'UCSPCT$' USING NUMB START-WORD BUFR.

  * Retrieve variable information from the PCT.
```

10.10. Reading and Writing Tape Label Blocks (UCSTLBL$)

Function

Reads or writes selected portions of HDR1, HDR2, EOF1, EOF2, EOV1, and EOV2. The function also reads and writes user labels before and after each file. Refers to Executive request function TLBL$, Version 1. See the *ER Programming Reference Manual* for details.

Format

```
CALL 'UCSTLBL$' USING file, function, buffer-length, buffer, status.
```

where:

file

is a character scalar expression or contiguous character array specifying the internal file name of the tape file for which labels are to be read or written; only the first 12 characters are significant.

function

is an integer scalar expression specifying the function to be performed. The valid function codes are described in the *ER Programming Reference Manual*.

buffer-length

is an integer scalar expression giving the number of elements of *buffer.*

buffer

is a one-dimensional contiguous array of one-word elements. The number of elements in the array must be equal to or greater than *buffer-length. Buffer* contains the Read/Write System Label Packet beginning with the fourth word (word 3) of the label packet. See the *ER Programming Reference Manual* for details.

status

is an integer variable or array element into which the status of the TLBL$ Executive request is stored. Define *status* as PIC 1(36) BINARY-1.

Example

```
01  TAPE-FILE PIC X(12) VALUE '$$$TAPE$FILE'.
01  STAT PIC 1(36) BINARY-1.
02  01  BUFFR1 PIC X(80) VALUE 'UHL1 Label to test TLBL$ interface'.

    CALL 'UCSTLBL$' USING TAPE 17 20 BUFFR1 STAT.
```

10.11. Tape File Initialization (UCSTINTL$)

Function

Logically rewinds a specified tape file so that a subsequent pass can be made from the load point of the first reel. Refers to Executive request function TINTL$.

Format

```
CALL 'UCSTINTL$' USING file, status.
```

where:

file

is a character scalar expression or contiguous character array specifying the internal file name (only the first 12 characters are significant) of the tape file that is to be logically rewound, in other words, positioned to the load point of the first reel of the file.

status

is an integer variable or array element that on execution contains the status returned by the TINTL$ Executive request. Define *status* as PIC 1(36) BINARY-1.

Example

```
     01  TAPE-FILE PIC X(12) VALUE '$$TAPE$FILE'.
     01  STAT PIC 1(36) BINARY-1.

         CALL 'UCSTINTL$' USING TAPE-FILE STAT.

*  Rewind tape.
```

10.12. Tape Swapping (UCSTSWAP$)

Function

Closes access to the current reel of a tape file and requests loading of the next reel of the file (unless special action is requested). Refers to the Executive request function TSWAP$.

Format

```
CALL 'UCSTSWAP$' USING file, status [ , function, reel ] .
```

where:

file

is a character scalar expression or contiguous character array indicating the internal file name (only the first 12 characters are significant) of the tape file that is to have a reel swapped.

status

is an integer variable or array element indicating the status returned by the TSWAP$ Executive request. Define *status* as PIC 1(36) BINARY-1.

The next two arguments are optional, but if one is given the other must be present.

function

is an integer scalar expression specifying the swap function to be performed. The permissible values are as follows:

Value	Description
+0 or -0	Standard reel swap. This is the action taken if *function* and *reel* are not given; *reel* is ignored.
+1 or -1	Standard reel swap. Returns the reel number of the reel swapped to in *reel*.
+2 or -2	Swap to the reel whose reel number is given in *reel*.
+3 or -3	Swap to the reel whose reel index is given in *reel*.

If *function* is negative, the request is not held on a reel-number conflict.

reel

The form of *reel* varies with the value of *function:*

Function	Reel Form
0	It can be any valid data item; it is ignored.
1	It is a scalar character variable into which the reel number of the new reel is stored.
2	It is a character scalar expression giving the reel to swap to; only the first 6 characters of reel are significant.
3	It is an integer scalar expression specifying the reel index of the new reel.

Example

```
01  TAPE-FILE PIC X(12) VALUE '$$$TAPE$FILE'.
01  STAT PIC 1(36) BINARY-1.

    CALL 'UCSTSWAP$' TAPE-FILE STAT 3 1.
```

10.13. Control Statement Listing (UCSCLIST$)

Function

Registers a set of control statement names with the Executive. Refers to Executive request function CLIST$. The system always returns control statements to you as ASCII images.

Format

```
CALL 'UCSCLIST$' USING number-of-names, names, status.
```

where:

number-of-names

is an integer scalar expression specifying the number of names that are to be registered with the EXEC. If this value is zero, CLIST$ mode is terminated and the rest of the parameters are ignored; however, they must still be provided.

names

is the first item in a one-dimensional contiguous character array. The number of elements in the array must be equal to or greater than *number-of-names.* Only the first 6 characters of each element are used; any element of all spaces is ignored (and the name count reduced accordingly).

status

is an integer scalar variable or array element into which the status of the CLIST$ Executive request is stored. Define *status* as PIC 1(36) BINARY-1.

10.14. Retrieving Facility Assignment (UCSFITEM$)

Function

Provides a method to obtain information on file or facility assignments. Refers to Executive request function FITEM$.

Format

```
CALL 'UCSFITEM$' USING internal-file, external-file, qualifier,
  rest-of-packet.
```

where:

internal-file

is a character scalar expression or contiguous character array that gives the internal file name (only the first 12 characters are significant) of the file for which information is wanted.

external-file

is a character scalar variable, contiguous character array, or character array element into which the external file name (only the first 12 characters are significant) is returned.

qualifier

is a character scalar variable, contiguous character array, or character array element into which the qualifier (only the first 12 characters are significant) of the file is returned.

rest-of-packet

is a one-dimensional contiguous array containing at least 7 elements of one-word data items. This array is filled with words 6 through 12 of the FITEM$ packet.

Example

```
01  USE-NAME PIC X(12) VALUE '$$$$FILETEST'.
01  ACTFIL PIC X(12).
01  ACTQUL PIC X(12).
01  REST.
    02  REST-1 PIC 9(10) BINARY OCCURS 7 TIMES.

    CALL 'UCSFITEM$' USE-NAME ACTFIL ACTQUL REST.
```

10.15. Miscellaneous Information Retrieval (UCSINFO$)

Function

Retrieves system, run, and program-oriented parameters in a manner independent of system level. Refers to the Executive request function INFO$.

Format

```
CALL 'UCSINFO$' USING argument-group1, argument-group2... .
```

where:

argument-group

pertains to a specific INFO$ Executive request function as described below. The format of each argument group is

function-code, status, arg1 [, *arg2...*]

where:

function-code

is the INFO$ function code as given in the *ER Programming Reference Manual* and briefly described below.

status

is a one-word item into which the status of the particular subfunction of the INFO$ Executive request is stored as a positive number. If an error has been detected in the argument group, a value of -1 is stored; this means that too few parameters were passed in this argument group. Define status as PIC 1(36) BINARY-1.

arg1...

are the parameters passed in the argument group. See the following information on UCSINFO argument groups for details.

UCSINFO$ Argument Groups

The following list describes the parameters that UCSINFO$ retrieves and the argument groups required for UCSINFO$ to obtain the value.

1. Retrieve user-id:

   Format

   ```
   1, status, user-id
   ```

   where *user-id* is a character variable, contiguous character array, or character array element into which the user-id of the run (at most 12 characters) is stored.

2. Retrieve site-id:

   Format

   ```
   2, status, site-id
   ```

   where *site-id* is a character variable, contiguous character array, or character array element into which the site-id of the run (at most 6 characters) is stored.

3. Give file name for use in subsequent functions:

   Format

   ```
   3, status, internal-file
   ```

   where *internal-file* is a scalar character expression or contiguous array giving the internal file name to be used in subsequent argument groups. For example, you can retrieve the file position for the tape assigned as internal file name 'A' as follows:

   ```
   CALL 'UCSINFO$' USING 3, status-1, 'A', 4, status-2, position.
   ```

   Once *internal-file* is given in an argument group, it remains valid during the rest of the UCSINFO$ call (or until another *internal-file* is given in a subsequent argument group).

4. Retrieve file position for a tape:

   Note: *You must provide an internal file name before using this function (see argument group 3 above).*

   Format

   ```
   4, status, position
   ```

   where *position* is an integer scalar variable or array element that is set to the number of tape marks between the load point and the tape's current position.

5. Retrieve block position for a tape:

   Note: *You must provide an internal file name before using this function (see argument group 3 above).*

Format

```
5, status, position
```

where *position* is an integer scalar variable or array element that is set to the number of blocks between the tape's current position and the nearest previous tape mark (or load point).

6. Retrieve reel index for a tape:

   Note: *You must provide an internal file name before using this function (see argument group 3 above).*

   Format

   ```
   6, status, reel-index
   ```

   where *reel-index* is an integer variable or array element that is set to the reel index of the tape file. The first reel of a tape file has index number 1.

7. Retrieve SUP values:

   Format

   ```
   7, status, results
   ```

   where *results* is a one-dimensional contiguous array with at least eight one-word elements into which the SUP values are stored.

8. Retrieve equipment mnemonic for a file:

   Note: *You must provide an internal file name before using this function (see argument group 3 above).*

   Format

   ```
   8, status, mnemonic
   ```

   where *mnemonic* is a character variable, contiguous character array, or character array element into which the mnemonic for the equipment on which the file resides is stored. An equipment mnemonic contains at most 6 characters.

9. Retrieve device names for a tape file:

   Note: *You must provide an internal file name before using this function (see argument group 3 above).*

   Format

   ```
   9, status, device-1, device-2
   ```

   where *device-1* and *device-2* are character variables, contiguous character arrays, or character array elements into which the device name for the tape file is stored. A device name contains at most 6 characters.

10. Retrieve mode information for a tape file:

    Note: *You must provide an internal file name before using this function (see argument group 3).*

    Format

    ```
    10, status, mode-word, processor-code, tape-code
    ```

    where:

    mode-word

    is an integer variable or array element into which the data transfer format, the density, and the translator index are stored.

    processor-code

    is a character variable, contiguous character array, or character array element into which the processor code (at most 6 characters) is stored.

    tape-code

    is a character variable, contiguous character array, or character array element into which the tape code (at most 6 characters) is stored.

11. Retrieve equipment index and file information:

    Note: *You must provide an internal file name before using this function (see argument group 3).*

    Format

    ```
    11, status, index
    ```

    where *index* is an integer variable or array element into which the equipment index and file-type information are stored.

12. Not applicable.

13. Retrieve information about current program:

    Format

    ```
    13, status, file, sector-address, element-name, version
    ```

    where:

    file

    is a character variable, contiguous character array, or character array element into which the internal file name (at most 12 characters) of the file containing the currently executing program is stored.

    sector-address

    is an integer variable or array element into which the sector address at which the text of the currently executing program begins is stored.

element-name

is a character variable, contiguous character array, or character array element into which the element name (at most 12 characters) of the currently executing program is stored.

version

is a character variable, contiguous character array, or character array element into which the version name (at most 12 characters) of the currently executing program is stored.

14. Retrieve assign count for a file:

    Note: *You must provide an internal file name before using this function (see argument group 3).*

    Format

    ```
    14, status, assign-count
    ```

    where *assign count* is an integer variable or array element into which the assign count for the file is stored.

15. Retrieve ring indicator for a tape file:

    Note: *You must provide an internal file name before using this function (see argument group 3 above).*

    Format

    ```
    15, status, ring-indicator
    ```

    where *ring-indicator* is an integer variable or array element into which the ring indicator value is stored.

16. Retrieve drive-related information for a tape file:

    Note: *You must provide an internal file name before using this function (see argument group 3).*

    Format

    ```
    16, status, drive-code
    ```

    where *drive-code* is an integer variable or array element into which information about the tape drive for this file is stored.

17. Retrieve fully-qualified name for a file:

    Note: *You must provide an internal file name before using this function (see argument group 3).*

    Format

    ```
    17, status, qualifier, file, absolute-cycle
    ```

where:

qualifier

is a character variable, contiguous character array, or character array element into which the qualifier (at most 12 characters) is stored.

file

is a character variable, contiguous character array, or character array element into which the external file name (at most 12 characters) is stored.

absolute-cycle

is an integer variable or array element into which the absolute cycle number is stored.

18. Retrieve hardware feature mask:

    Format

    ```
    18, status, feature-mask
    ```

    where *feature-mask* is a double-word variable or array element into which the hardware-feature mask is stored.

19. Not applicable.

20. Not applicable.

21. Not applicable.

22. Retrieve terminal type information:

    Format

    ```
    22, status, terminal-type
    ```

    where *terminal-type* is an integer scalar variable or array element that is set to the terminal type.

23. Retrieve printer indicator:

    Format

    ```
    23, status, printer-indicator
    ```

    where *printer-indicator* is an integer scalar variable or array element into which the printer indicator is stored.

24. Retrieves the CTL-pool name used on cartridge tape assignments:

    Note: *You must provide an internal file name before using this function (see argument group 3 above).*

Format

```
24, status, CTL-name
```

where CTL-*name* is a character variable, contiguous character array, or character array element into which the CTL-*name* used on cartridge tape assignments (at most 8 characters) is stored.

25. Not applicable.

26. Retrieves the media manager specification used on tape assignments:

    Note: *You must provide an internal file name before using this function (see argument group 3 above).*

    ### Format

    ```
    26, status, media-specification
    ```

    where *media-specification* is an integer variable or array element into which the mmspec value for the file is stored.

27. Retrieves the current standard unit of processing (SUP) values expressed in nanoseconds:

    ### Format

    ```
    27, status, nanosup
    ```

    where *nanosup* is a one-dimensional contiguous array with at least 31 one-word elements into which the nanosup values are stored.

    ### Example

```
01  USE-FILE PIC X(12).
01  FILE-QUAL PIC X(12).
01  FILE-NAME PIC X(12).
01  FILE-CYCLE PIC 9(10) BINARY.
01  HW-FEATURES.
    02  HW PIC 9(10) BINARY OCCURS 2 TIMES
01  ACT-USER PIC X(12).
01  ACT-SITE PIC X(16).
01  TAPE-NAME PIC X(12).
01  FILE-POSITION PIC 9(10) BINARY.
01  BLOCK-POSITION PIC 9(10) BINARY.
01  REEL-INDEX PIC 9(10) BINARY.
01  SUP-VALUES.
    02  SUPS PIC 9(10) BINARY OCCURS 8 TIMES.
01  EQUIPMENT-MNEMONIC PIC X(6).
01  DEVICE-1 PIC X(6).
01  DEVICE-2 PIC X(6).
01  MODE-WORD PIC 9(10) BINARY.
01  PROCESSOR-CODE PIC X(6).
01  TAPE-CODE PIC X(6).
01  EQUIPMENT-INDEX PIC 9(10) BINARY.
01  CEC-INFO.
```

```
    02  CEC PIC 9(10) BINARY OCCURS 3 TIMES.
01  PROG-FILE PIC X(12).
01  PROG-SECTOR PIC 9(10) BINARY.
01  PROG-NAME PIC X(12).
01  PROG-VERSION PIC X(12).
01  ASSIGN-COUNT PIC 9(10) BINARY.
01  RING PIC 9(10) BINARY.
01  DRIVE-CODE PIC 9(10) BINARY.
01  STATUS.
    02  STATS PIC 1(36) BINARY-1 OCCURS 23 TIMES.

    MOVE '$$$$TAPE$FILE' TO TAPE-NAME.
    MOVE '$$$$FILETEST' TO USE-FILE.

    CALL 'UCSINFO$' USING
         1  STATS(1)    ACT-USER
         2  STATS(2)    ACT-SITE
         3  STATS(3)    TAPE-NAME
         4  STATS(4)    FILE-POSITION
         5  STATS(5)    BLOCK-POSITION
         6  STATS(6)    REEL-INDEX
         7  STATS(7)    SUP-VALUES
         8  STATS(8)    EQUIPMENT-MNEMONIC
         9  STATS(9)    DEVICE-1 DEVICE-2
        10  STATS(10)   MODE-WORD PROCESSOR-CODE TAPE-CODE
        11  STATS(11)   EQUIPMENT-INDEX
        12  STATS(12)   CEC-INFO
        13  STATS(13)   PROG-FILE PROG-SECTOR PROG-NAME PROG-VERSION
        14  STATS(14)   ASSIGN-COUNT
        15  STATS(15)   RING
        16  STATS(16)   DRIVE-CODE
        17  STATS(17)   FILE-QUAL FILE-NAME FILE-CYCLE
        18  STATS(18)   HW-FEATURES.
```

28. Not applicable.

29. Retrieve the internal name of the file to which print has been directed.

    Format

    ```
    29, status, brk-name
    ```

    where *brk-name* is a character variable, contiguous character array, or character array element into which the internal name of the file to which print has been directed (12 characters) is stored.

30. Retrieve the type of media loaded.

    Note: *You must provide an internal file name before using this function (see argument group 3 above).*

Format

```
30, status, media-type
```

where *media-type* is an integer scalar variable or array element that is set to the type of media loaded.

31. Retrieve the run continuation log entry.

Format

```
31, status, run-ident
```

where *run-ident* is a one-dimensional contiguous array with at least 71 one-word elements into which the copy of the run continuation log entry (Log Entry 106, Subtype 2) is stored.

Example

```
01  USE-FILE PIC X(12).
01  FILE-QUAL PIC X(12).
01  FILE-NAME PIC X(12).
01  FILE-CYCLE PIC 9(10) BINARY.
01  HW-FEATURES.
02  HW PIC 9(10) BINARY OCCURS 2 TIMES
01  ACT-USER PIC X(12).
01  ACT-SITE PIC X(16).
01  TAPE-NAME PIC X(12).
01  FILE-POSITION PIC 9(10) BINARY.
01  BLOCK-POSITION PIC 9(10) BINARY.
01  REEL-INDEX PIC 9(10) BINARY.
01  SUP-VALUES.
02  SUPS PIC 9(10) BINARY OCCURS 8 TIMES.
01  EQUIPMENT-MNEMONIC PIC X(6).
01  DEVICE-1 PIC X(6).
01  DEVICE-2 PIC X(6).
01  MODE-WORD PIC 9(10) BINARY.
01  PROCESSOR-CODE PIC X(6).
01  TAPE-CODE PIC X(6).
01  EQUIPMENT-INDEX PIC 9(10) BINARY.
01  PROG-FILE PIC X(12).
01  PROG-SECTOR PIC 9(10) BINARY.
01  PROG-NAME PIC X(12).
01  PROG-VERSION PIC X(12).
01  ASSIGN-COUNT PIC 9(10) BINARY.
01  RING PIC 9(10) BINARY.
01  DRIVE-CODE PIC 9(10) BINARY.
01  STATUS.
    02  STATS PIC 1(36) BINARY-1 OCCURS 23 TIMES.

    MOVE '$$$$TAPE$FILE' TO TAPE-NAME.
    MOVE '$$$$FILETEST' TO USE-FILE.

    CALL 'UCSINFO$' USING
         1   STATS(1) ACT-USER
```

```
2   STATS(2) ACT-SITE
3   STATS(3) TAPE-NAME
4   STATS(4) FILE-POSITION
5   STATS(5) BLOCK-POSITION
6   STATS(6) REEL-INDEX
7   STATS(7) SUP-VALUES
8   STATS(8) EQUIPMENT-MNEMONIC
9   STATS(9) DEVICE-1 DEVICE-2
10  STATS(10) MODE-WORD PROCESSOR-CODE TAPE-CODE
11  STATS(11) EQUIPMENT-INDEX
13  STATS(13) PROG-FILE PROG-SECTOR PROG-NAME PROG-VERSION
14  STATS(14) ASSIGN-COUNT
15  STATS(15) RING
16  STATS(16) DRIVE-CODE
17  STATS(17) FILE-QUAL FILE-NAME FILE-CYCLE
18  STATS(18) HW-FEATURES.
```

10.16. General Executive Requests (UCSGENERAL$)

Function

Provides access to any Executive request function that satisfies the following criteria:

- The Executive request always returns to the instruction immediately after the ER itself. It can raise a contingency, but it cannot jump.
- The Executive request does not spin off a new user activity or deactivate an existing activity.
- The Executive request takes all its parameters from A registers, which can either contain the value or point to the data items (packets, for example) used by the ER.
- Addresses in A registers or ER packets are required by the ER to be in basic mode format.

  Note: *All addresses generated by UCS languages are extended mode virtual addresses. You must convert extended mode addresses to basic mode addresses for ERs that require basic mode address formats. The UCSEMTOBM$ service routine is provided for this purpose (see 10.23). If ER requires R1 register as a parameter, fill-in A13 register with the value in the R1 register.*
- The Executive request delivers its results in A registers or in packets pointed to by the A registers used as input parameters.

  Note: *If UCSGENERAL$ is called from a shared subsystem, the data packets and buffers must reside in a data bank with a LOW BDI.*

The Executive request functions that satisfy these criteria are the following:

- APRINT$
- IO$

  Spins off a new activity, but the user does not see it
- LOG$

- SYSLOG$

  Needs SSLOGER security privilege and you must convert extended mode addresses to basic mode addresses using the UCSEMTOBM$ service routine (see 10.23).
- SYMB$

The Executive request functions that do not satisfy these criteria are the following:

- AREAD$

  Jumps to the EOF address at the end of the input file
- FORK$

  Spins off a new activity
- II$

  Deactivates an activity
- IOI$

  Temporarily jumps to some out-of-line address
- TRON$

  Assigns the common bank name section instead of the subsystem name section.
- Some Transaction Processing (TIP) Executive requests

  Take parameters in R registers

A UCS COBOL program that calls an Executive request function that requires an address of a buffer to be passed as one of its parameters should use the LOC$ subroutine (see 10.6.3).

Format

```
CALL 'UCSGENERAL$' (ER-code, input-mask, output-mask, input-registers,
output-registers [,user-D-bank, ER-instruction-word ])
```

where:

ER-code

is an integer scalar expression (USAGE BINARY) specifying the ER to be performed. See the *ER Programming Reference Manual* for ER codes.

input-mask

is a one-word value (normally an integer) specifying which A-registers are to be loaded before the ER is performed. Any bits not used in this word must be zero. The convention for determining which registers to load is presented in the following table.

Bit Set	Register to Load
1	A0
2	A1
3	A2
4	A3
5	A6
6	A5
7	A6
8	A7
9	A8
10	A9
11	A10
12	A11
13	A12
14	A13 and R1
15	A14
16	A15

output-mask

is a one-word value (normally an integer) specifying which A-registers are to be stored to *output-registers* after the ER. Any bits not used in this word must be zero. The convention describing which registers to store is presented in the following table.

Bit Set	Register to Store
1	A0
2	A1
3	A2
4	A3
5	A6

Bit Set	Register to Store
6	A5
7	A6
8	A7
9	A8
10	A9
11	A10
12	A11
13	A12
14	A13
15	A14
16	A15

input-registers

is a one-dimensional contiguous array of one-word variables. The first element contains the value of the first A-register to be loaded, the second element contains the value of the second A-register to be loaded, and so forth. For example, if the *input-mask* specifies that A0 and A3 should be loaded, the first element of *input-registers* is loaded into A0 and the second element is loaded into A3. The value of A13 register is also moved to the R1 register.

output-registers

is a one-dimensional contiguous array of one-word variables. The first A-register to be stored is stored in the first element, the second A-register to be stored is stored in the second element, and so forth. For example, if the *output-mask* specifies that A1 and A5 are to be stored after the ER, A1 is stored to the first element of *output-registers* and A5 is stored to the second.

user-D-bank

is a variable, array element, or array (except an assumed noncontiguous array value) having storage in the same bank as all data used by the ER.

All data used from memory by the ER must reside in the same bank. This can be accomplished by placing all data items used by the ER in the same common block or by placing none of the data items in a common block. The UCSGENERAL$ subroutine bases the bank indicated by the address of this argument as a basic mode bank prior to execution of the ER. Basic mode banks reserve the first 128 words of the address space for use by registers. Thus, the address of any data item used from memory by the ER must not fall within the range 0 - 177 octal (within the first 128 words). This is usually accomplished by placing the data items in a common block with a filler of 128 words preceding it. Note the filler area can be used for any data items not used by the ER.

If the data items are contained in a large bank made up of several small banks, the address of the data items used by the ER cannot fall within the first 128 words of each small bank.

ER- instruction- word

is a required scratch word giving a location in the specified user D-bank written into by UCSGENERAL$. It must be a full word, and word-aligned. It must have a bank offset greater than 128 (octal 200). Its address must not be within the ER packet or other data in use during the ER. The offset must be greater than the general register set because the ER is executed in basic mode. You can get the correct offset by padding the bank. If you pass an offset of 128 or less, the service library ignores the scratch word parameter and prints the following error message:

```
19066:  INCORRECT NUMBER OF ARGUMENTS ON SERVICE LIBRARY CALL
```

If the ER uses data only from registers and not from memory (for example, TIME$), you can omit *user D-bank* and *ER instruction word*.

Note: *To ensure that the 128-word offset is allocated, you must allocate its variable names or use the COMPAT/FULL keyword option (see 2.2.3).*

Examples

The following program illustrates the use of UCSGENERAL$. It uses the Executive request function MODSWTIME$.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MODSWTIME-EXAMPLE.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
DATA DIVISION.
WORKING-STORAGE SECTION.
01  ER-CODE                     PIC 9(10) BINARY   VALUE 209.
01  INPUT-MASK.
      02 INPUT-NOUSE-A0-A35      PIC 1(36) BINARY-1 VALUE 0.
01  OUTPUT-MASK.
      02 OUTPUT-USE-A0           PIC 1(1)  BINARY-1 VALUE 1.
      02 OUTPUT-NOUSE-A1-A35     PIC 1(35) BINARY-1 VALUE 0.
01  ER-PACKET                   PIC 1(36) BINARY-1 VALUE 0.
01  OUTPUT-REG.
      02 TIME-IS-MODIFIED-SWTIME PIC 1(1)  BINARY-1.
      02 SEASONAL-SHIFT-ON       PIC 1(1)  BINARY-1.
      02 SECONDS-SINCE-1900      PIC 1(34) BINARY-1.
PROCEDURE DIVISION.
BEGIN.
     CALL 'UCSGENERAL$' USING ER-CODE, INPUT-MASK,
     OUTPUT-MASK, ER-PACKET, OUTPUT-REG.
* 209 IS THE ER CODE FOR MODSWTIME$
* Upon successful completion, OUTPUT-REG (A0) contains:
```

```
* TIME-IS-MODIFIED-SWTIME - This is always set to 1. It
*                          indicates a modified-swtime timestamp.
* SEASONAL-SHIFT-ON - This is 0 or 1 indicating the seasonal
*                   offset at the UTC time described in bits 2-35.
* SECONDS-SINCE-1900 - These 34 bits contain the number of
*                    seconds since 00:00:00 1899 Dec 31, UTC base.
      STOP RUN.
```

The following program illustrates the use of UCSGENERAL$. It uses the Executive request function DATE$.

```
01 DATEDOLLAR-ER.
    02 MMDDYY PIC 9(10) BINARY.
    02 HHMMSS PIC 9(10) BINARY.
01 D REDEFINES DATEDOLLAR-ER.
    02 D1 PIC 1(12) BINARY-1.
    02 D2 PIC 1(12) BINARY-1.
    02 D3 PIC 1(12) BINARY-1.
    02 D4 PIC 1(12) BINARY-1.
    02 D5 PIC 1(12) BINARY-1.
    02 D6 PIC 1(12) BINARY-1.
01 A0-A1.
    02 BIT1 PIC 1 BINARY-1 VALUE 1.
    02 BIT2 PIC 1 BINARY-1 VALUE 1.
    02 BIT3-36 PIC 1(34) BINARY-1 VALUE 0.
01 ER-CODE PIC 9(10) BINARY VALUE 18.
                    .
                    .
                    .
* CALL ER DATE$
    CALL 'UCSGENERAL$' USING ER-CODE, 0, A0-A1, 0, D.
* 18 is the ER code for DATE$
* D1 will contain (in Fieldata) the current month
* D2 will contain (in Fieldata) the current day
* D3 will contain (in Fieldata) the last two digits of the year
* D4 will contain (in Fieldata) the hour
* D5 will contain (in Fieldata) the minutes
* D6 will contain (in Fieldata) the seconds
```

The following example shows the use of UCSGENERAL$ using all 7 parameters. This program prints the line “HI THERE” with single spacing on the standard output device using the Executive request function APRINT$.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXAMPLE.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
DATA DIVISION.
COMMON-STORAGE SECTION.
01  BANK-OFFSET PIC X(512).
```

```
01  USER-D-BANK PIC X(4).
01  ER-INSTRUCTION PIC X(4).
01  CHAR-STRING PIC X(8) VALUE 'HI THERE'.
01  CHAR-STRING-ADDR PIC 1(36) BINARY-1.
01  INPUT-MASK.
    02  INPUT-USE-A0 PIC 1 BINARY-1 VALUE 1.
    02  INPUT-NOUSE-A1-A36 PIC 1(35) BINARY-1 VALUE 0.
01  OUTPUT-MASK.
    02  OUTPUT-NOUSE-A0-A36 PIC 1(36) BINARY-1 VALUE 0.
01  ER-PACKET.
    02  PF1 PIC 1(12) BINARY-1 VALUE 1.
    02  PF2 PIC 1(6)  BINARY-1 VALUE 2.
    02  PF3 PIC 1(18) BINARY-1 VALUE 0.
01  OUTAR PIC 1(36) BINARY-1.
PROCEDURE DIVISION.
BEGIN.
    CALL 'LOC$' USING CHAR-STRING, CHAR-STRING-ADDR.
    MOVE CHAR-STRING-ADDR TO PF3.
    CALL 'UCSGENERAL$'
            USING 56, INPUT-MASK, OUTPUT-MASK, ER-PACKET, OUTAR,
                  USER-D-BANK, ER-INSTRUCTION.
    STOP RUN.
```

10.17. Milliseconds Past Midnight (UCSTIME$)

Function

Provides the current local time in milliseconds past midnight, local time base, in integer form.

Format

```
CALL 'UCSTIME$' USING t
```

where *t* is a 1-word binary item.

Description

The UCSTIME$ subroutine returns the number of milliseconds past midnight in the argument *t*.

Example

```
 WORKING-STORAGE SECTION.
*
 01  TIM           PIC 9(10) BINARY.
 01  HOURS         PIC 9(10) BINARY.
 01  MINUTES       PIC 9(10) BINARY.
 01  SECONDS       PIC 9(10) BINARY.
 01  MILLISECONDS  PIC 9(10) BINARY.
*
 PROCEDURE DIVISION.
 BEGIN.
```

```
CALL 'UCSTIME$' USING TIM.
DISPLAY 'The number of milliseconds past midnight is ' TIM.
COMPUTE MILLISECONDS = FUNCTION REM (TIM, 1000).
COMPUTE TIM = TIM / 1000.
COMPUTE SECONDS = FUNCTION REM (TIM, 60).
COMPUTE TIM = TIM / 60.
COMPUTE MINUTES = FUNCTION REM (TIM, 60).
COMPUTE HOURS = TIM / 60.
DISPLAY 'This converts to a clock time of '
        hours ':' minutes ':' seconds ':' milliseconds.
```

Sample Output

```
The number of milliseconds past midnight is    0047331402
This converts to a clock time of 13:08:51:402
```

10.18. Retrieving Time (UCSDWTIME$)

Function

Provides the current local time in nanoseconds past 00:00:00.000 December 31, 1899, in integer form.

Format

```
CALL 'UCSDWTIME$' USING t
```

where *t* is a 2-word binary item.

Description

The UCSDWTIME$ subroutine returns the number of nanoseconds past 00:00:00.000 December 31, 1899, local time base, in the variable *t*. Because the value is so large, two words are required to hold it. Use UCSCNVTIM$ to convert the value in *t* to a 6-character date (see 10.19).

Example

```
 WORKING-STORAGE SECTION.
*
 01  TIMEID  PIC S1(72) BINARY-1.
 PROCEDURE DIVISION.
 BEGIN.
       CALL 'UCSDWTIME$' USING TIMEID.
       DISPLAY 'Nanoseconds since December 31, 1899 = ' TIMEID.
```

Sample Output

```
Nanoseconds since December 31, 1899 = 005511130006767822
```

10.19. Converting Time in UCSDWTIME$ Format (UCSCNVTIM$)

Function

The UCSCNVTIM$ subroutine converts a time in nanoseconds in UCSDWTIME$ format to a date in month, day, and year format (mmddyy).

Format

```
CALL 'UCSCNVTIM$' USING t, d
```

where:

t

is a 2-word binary item.

d

is an alphanumeric variable with a length of 6.

Description

The argument *t* must contain a time in nanoseconds in the format produced by UCSDWTIME$. UCSCNVTIM$ converts the value in *t* to a 6-character month, day, and year format (mmddyy). The converted value is returned in argument *d*.

Example

```
 WORKING-STORAGE SECTION.
*
 01  TIMEID     PIC S1(72) BINARY-1.
 01  DATEID    PIC X(6).
*
 PROCEDURE DIVISION.
 BEGIN.
   CALL 'UCSDWTIME$' USING TIMEID.
   CALL 'UCSCNVTIM$' USING TIMEID DATEID.
   DISPLAY 'The converted UCSDWTIME$ time = ' DATEID.
```

Sample Output

```
The converted UCSDWTIME$ time = 062790
```

10.20. Converting Time in UCSDWTIME$ Format to a Date (UCSCNVTIM$CC)

Function

The UCSCNVTIM$CC subroutine converts a time in nanoseconds in UCSDWTIME$ format to a date in a format determined by the length of *d*. It can also return the current date in the same formats.

Format

```
CALL 'UCSCNVTIM$CC' USING t, d
```

where:

t

is a 2-word binary item.

d

is an alphanumeric variable with a length of at least 8. The result will be returned in *d*.

Argument *t*, if present, must contain a time in nanoseconds in the format produced by UCSDWTIME$ (see 10.18). UCSCNVTIM$CC converts this value to an 8 to 17 character format holding the 4-digit year, 2-digit month, and 2-digit day and optionally, the 2-digit hour, 2-digit minute, 2-digit second, and 3-digit millisecond. The precision is determined from the size of the variable *d*.

Size of *d*	Precision	Format of *d*
8	Day	ccyymmdd
10	Hour	ccyymmddhh
12	Minute	ccyymmddhhmm
14	Second	ccyymmddhhmmss
15	1/10 Second	ccyymmddhhmmssm
16	1/100 Second	ccyymmddhhmmssmm
17	1/1000 Second	ccyymmddhhmmssmmm

If argument *t* is not present, the current date-time is returned into argument *d*.

Example

```
WORKING-STORAGE SECTION.
01  timeid pic s1(72) binary-1.
01  date-day pic x(8).
01  date-hour pic x(10).
```

```
01  date-minute pic x(12).
01  date-second pic x(14).
01  date-10th-second pic x(15).
01  date-1000th-second pic x(17).
PROCEDURE DIVISION.
BEGIN.
    call 'ucsdwtime$' using timeid.

    call 'ucscnvtim$cc' using timeid, date-day.
    display date-day.

    call 'ucscnvtim$cc' using timeid, date-hour.
    display date-hour.

    call 'ucscnvtim$cc' using timeid, date-minute.
    display date-minute.

    call 'ucscnvtim$cc' using timeid, date-second.
    display date-second.

    call 'ucscnvtim$cc' using timeid, date-1000th-second.
    display date-1000th-second.

    call 'ucscnvtim$cc' using date-10th-second.
    display date-10th-second.

Output

20000502
2000050215
200005021502
20000502150257
20000502150257967
200005021502580
```

10.21. Initial Register Contents (UCSINITREG$)

Function

The UCSINITREG$ subroutine provides the initial contents of the registers that are set up by the Exec when a program begins execution.

Format

```
CALL 'UCSINITREG$' USING regs
```

where *regs* is an array of type integer. The array must contain 12 elements.

Description

When a program begins execution, the Exec initializes a set of registers with information about the machine and the program's environment. The subroutine UCSINITREG$ returns the contents of the initialized registers in the argument *regs*. The UCS Runtime System saves the contents of the initialized registers in an internal save area so the call to UCSINITREG$ may take place any time after the program begins execution (the call does not have to be the first executable statement).

Table 10–2 lists the registers retrieved and their contents (bit 0 is the rightmost bit).

Table 10–2. Registers Retrieved

Register	Description	Bit	Contents
A2/A3	Hardware feature mask	71 through 36	Unused
		35 through 30	Numerical Host-ID, (1=A, 2=B …)
		29 through 8	Unused
		7	VERSION3E type hardware
		6	Unused
		5	Queuing Architecture supported
		4	Unused
		3	Unused
		2	Basic mode
		1	Extended mode
		0	Unused
A4	Program type		Octal value:
			04 Demand
			O5 Deadline batch
			06 Batch
			011 Self-destructive TIP
			012 Self-initializing TIP
			013 Reentrant TIP
			014 Online batch
			015 HVTIP
A5	@XQT options	35	Processor reloaded from QECL stack
		34	Output is directed to a print file with @BRKPT
		33	R11-R15 are valid

Table 10–2. Registers Retrieved

Register	Description	Bit	Contents
		25-0	Options A-Z
R1	Program start date in DATE$ format.		Fieldata date with a local time base. The format is MMDDYY where MM is the month (01=January), DD is day of the month and YY is the last two digits of the year. See *Time Considerations for ClearPath OS 2200 Systems* for details on the DATE$ format.
R2	Program start time in either TDATE$ format or Modified-SWTIME format.		Binary time. The setting of the configuration parameter R2MSWTIME determines the timestamp format. If true, the Modified-SWTIME format is used. If false, the TDATE$ format is used. See *Time Considerations for ClearPath OS 2200 Systems* for details on the timestamps formats and on the configuration parameter R2MSWTIME.
R3	Accumulated CPU time for the run in 200-microsecond increments		
R11/R12	Machine type in Fieldata (left-justified)		
R13/14	Operating system level in Fieldata (left-justified)		
R15	System site-id in Fieldata (left-justified)		

Example

```
01  REGS.
    02  A2   PIC  9(10) BINARY.
    02  A3   PIC  9(10) BINARY.
    02  A4   PIC  9(10) BINARY.
    02  A5   PIC  9(10) BINARY.
    02  R1   PIC  9(10) BINARY.
    02  R2   PIC  9(10) BINARY.
    02  R3   PIC  9(10) BINARY.
    02  R11  PIC  9(10) BINARY.
    02  R12  PIC  9(10) BINARY.
    02  R13  PIC  9(10) BINARY.
```

```
    02  R14  PIC  9(10) BINARY.
    02  A15  PIC  9(10) BINARY.
PROCEDURE DIVISION.
BEGIN.
    CALL 'UCSINITREG$' USING REGS.
```

10.22. Converting between Fieldata and ASCII Characters (UCSASCFD$, UCSFDASC$)

Function

These subroutines allow you to convert between Fieldata and ASCII characters, as follows:

- UCSFDASC$

  Converts Fieldata characters to ASCII characters
- UCSASCFD$

  Converts ASCII characters to Fieldata characters

Both routines are intended to be used with ER and system service routines only. Other uses of UCSASCFD$ are not supported since Fieldata is an obsolete data type.

Format

$$\text{CALL} \left\{ \begin{matrix} \text{'UCSFDASC\$'} \\ \text{'UCSASCFD\$'} \end{matrix} \right\} \text{USING } a, b, i$$

where:

a

is the source string to be converted. For UCSFDASC$, *a* is an expression of any type. For UCSASCFD$, *a* is a character expression.

b

is the target for the converted characters. For UCSFDASC$, *b* is a character variable, character array, character array element, or character substring. For UCSASCFD$, *b* is a variable, array, or array element of any type, or a character substring.

i

is an integer scalar expression that specifies the number of characters to convert. If *i* = 0, no characters are converted.

Description

Note: *In the following paragraphs, ASCII characters are assumed to be packed four characters per word, starting on a quarter word boundary. Fieldata characters are assumed to be packed six characters per word, starting on a half-word boundary only. Any other character alignment is not supported and could produce unpredictable results.*

UCSFDASC$ converts *i* Fieldata characters represented by *a* to ASCII characters and places the result into *b*. If *a* and *b* share the same storage, the conversion may fail.

UCSASCFD$ converts *i* ASCII characters represented by *a* to Fieldata characters and places the result into *b*. If *a* and *b* share the same storage, the source characters *a* are destroyed.

The entities *a* and *b* cannot be noncontiguous arrays.

Example

The following example illustrates the use of UCSFDASC$. The program calls the UCSPCT$ service routine to obtain the first word of the program control table (PCT), which is the run identifier (RUNID) in Fieldata characters. The program then calls UCSFDASC$ to convert the RUNID from Fieldata to ASCII characters.

```
 IDENTIFICATION DIVISION.
 PROGRAM-ID.  example.
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER. UNISYS-2200.
 OBJECT-COMPUTER. UNISYS-2200.
 SPECIAL-NAMES.
     PRINTER IS PRT.
 DATA DIVISION.
 WORKING-STORAGE SECTION.
 01  pct-buffer-runid PIC 9(10) BINARY.
 01  pct-words-to-read PIC 9(10) BINARY VALUE 1.
 01  pct-word-offset PIC 9(10) BINARY VALUE 0.
 01  ascii-runid PIC X(6).
 PROCEDURE DIVISION.
 BEGIN.
*
*  Retrieve the Fieldata RUNID from the program control table
*  (PCT) using the UCSPCT$ Executive Request service routine.
*  Convert the Fieldata RUNID to ASCII using UCSFDASC$.
*     CALL 'UCSPCT$'
             USING pct-words-to-read, pct-word-offset,
             pct-buffer-runid.
      CALL 'UCSFDASC$' USING pct-buffer-runid, ascii-runid, 6.
      DISPLAY ascii-runid UPON PRT.
      STOP RUN.
```

10.23. Converting Extended Mode Addresses to Basic Mode Addresses (UCSEMTOBM$)

Function

UCSEMTOBM$ converts an extended mode address of form

```
l, bdi, offset
```

to a basic mode address of form

```
e, bdi, offset
```

Format

```
CALL 'UCSEMTOBM$' USING em-addr, bm-addr, status
```

where:

em-addr

is an integer value representing the extended mode address to be converted. e*m-addr* is an extended mode address of form *l, bdi, offset*. This address is the result of the LOC$ service routine (see 10.6.3).

Note: *Most ERs require the packet and data accessed by the ER to be in PROGRAM level storage. Common blocks are allocated as PROGRAM level storage.* em-addr *should be the address of an item in a common block; if not, you should use the proper @LINK commands to declare the storage as PROGRAM level.*

bm-addr

is an integer scalar variable or array element that contains the result of the conversion upon exit from UCSEMTOBM$. *bm-addr* is a basic mode address of form *e, bdi, offset*. If *status* is nonzero, *bm-addr* is returned with a value of zero.

status

is an integer scalar variable or array element that contains the status of the conversion on exit from UCSEMTOBM$. *status* is zero if the conversion was successful. *status* is a nonzero value if the extended mode BDI value is greater than 4095 (basic mode limit). Define *status* as PIC 1(36) BINARY-1.

Description

The UCSEMTOBM$ subroutine converts an extended mode address to a basic mode address. UCSEMTOBM$ is supported for use with UCSGENERAL$ only, for performing ERs that require basic mode addresses for the contents of an A-register or ER packet.

Example

The following example illustrates use of UCSEMTOBM$ for performing ER SYSLOG$ through UCSGENERAL$. ER SYSLOG$ requires a basic mode address in the ER packet and in register A0. UCSEMTOBM$ is called to convert extended mode addresses (obtained through LOC$) to basic mode addresses.

```
 IDENTIFICATION DIVISION.
 PROGRAM-ID.
     perfsyslog.
*
*   This program illustrates the use of UCSEMTOBM$ and UCSGENERAL$.
*   ER SYSLOG$ is executed through UCSGENERAL$.  ER SYSLOG$ requires
*   a basic mode address form of (E, BDI, OFFSET) in the ER packet
*   and in register A0.  UCSEMTOBM$ is used to convert extended mode
*   addresses to basic mode addresses.
*
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER. UNISYS-2200.
 OBJECT-COMPUTER. UNISYS-2200.
 DATA DIVISION.
 WORKING-STORAGE SECTION.
 01  ucsemtobm-status pic 1(36) binary-1.
 01  bm-buff-addr     pic 1(36) binary-1.
 01  register-a0      pic 1(36) binary-1.
 01  input-mask.
     02 bit1     pic 1     binary-1 value 1.
     02 bit2-36  pic 1(35) binary-1 value 0.
 01  er-syslog pic 1(36) binary-1 value 186.
 01  em-addr pic 1(36) binary-1.
 01  filler pic x(512).
 01  scratch-word pic 1(36) binary-1.
*
*  Definition of SYSLOG$ packet
*
 01  syslog-pkt.
     02 word-1.
          03 filler         pic 1(6)  binary-1.
          03 packet-level   pic 1(6)  binary-1.
          03 packet-length  pic 1(6)  binary-1.
          03 log-entry-type pic 1(18) binary-1.
     02 word-2.
          03 filler         pic 1(18) binary-1.
          03 version-number pic 1(18) binary-1.

     02 word-3.
          03 buffer-address pic 1(36) binary-1.
     02 word-4.
          03 buffer-length  pic 1(18) binary-1.
          03 cycle-id       pic 1(18) binary-1.
     02 word-5.
```

```
          03 status-word.
               04 er-status   pic 1(18) binary-1.
               04 filler      pic 1(18) binary-1.
 01  buffer pic x(60) value
           '**Test for ER SYSLOG$ using UCSEMTOBM$ and UCSGENERAL$**'.
 PROCEDURE DIVISION.
 BEGIN.
*
*  Convert extended mode address of BUFFER to basic mode address
*
     call 'loc$' using buffer, em-addr.
     call 'ucsemtobm$'
          using em-addr, bm-buff-addr, ucsemtobm-status.
     if ucsemtobm-status not equal 0
          display 'ERROR1 - UCSEMTOBM$ ERROR STATUS ='
                  ucsemtobm-status
          stop run
     end-if.
*
*  Initialize SYSLOG$ packet
*
     move 0 to syslog-pkt.
     move 1 to packet-level.
     move 5 to packet-length.
     move 200101 to log-entry-type.
     move 1 to version-number.
     move bm-buff-addr to buffer-address.
     compute buffer-length = 15.
*
*  Convert extended mode address of SYSLOG-PKT to basic mode
*  address and place the result in REGISTER-A0
*
     call 'loc$' using syslog-pkt em-addr.
     call 'ucsemtobm$'
          using em-addr, register-a0, ucsemtobm-status.
     if ucsemtobm-status not equal 0
          display 'ERROR2 - UCSEMTOBM$ ERROR STATUS ='
                  ucsemtobm-status
          stop run
     end-if.
*
*  Execute ER SYSLOG$
*
     call 'ucsgeneral$' using er-syslog, input-mask, 0
                        register-a0, 0, buffer, scratch-word.
     if er-status = 0
       then
          display 'ER SYSLOG$ COMPLETE, CYCLE-ID=' cycle-id
       else
          display 'ERROR - ER SYSLOG$ STATUS=' status-word.
     STOP RUN.
```

10.24. Activating Auxiliary Prologue and Epilogue (UCS$ACTAUX and UCS$DEACTAUX)

Function

UCS$ACTAUX and UCS$DEACTAUX are service subprograms that allow you to activate and deactivate the calling of user-supplied auxiliary prologue and epilogue subprograms during program execution. See the description of the AUXPROLOG/*n* keyword option in 2.2.3.

Format

CALL $\left\{ \begin{matrix} \text{'UCS\$ACTAUX'} \\ \text{'UCS\$DEACTAUX'} \end{matrix} \right\}$

Description

The UCS$ACTAUX subroutine activates the calling of user-supplied auxiliary and epilogue subprograms by those programs and subprograms that were compiled with the AUXPROLOG/*n* keyword option.

The UCS$DEACTAUX subroutine deactivates the calling of user-supplied auxiliary and epilogue subprograms by those programs and subprograms that were compiled with the AUXPROLOG/*n* keyword option.

Example

These subprograms are usually written in a system software language such as UCS C, which can manipulate virtual addresses (VAs). See the keyword AUXPROLOG/*n* in the *C Compiler Programming Reference Manual Volume 2* for an example of a user-supplied auxiliary prologue and epilogue subprogram.

10.25. Subprograms for Performing System Level I/O

The I/O subprograms described in this section provide an interface to the OS 2200 Executive request ER IOW$ described in the *ER Programming Reference Manual*. The read and write subprograms use the FIOW and UCSMAKEIOPK$ subprograms described in the *COBOL Compiler Programming Reference Manual Volume 2.* Many of the status codes returned by the subprograms described in this section (such as BULKREAD$ and BULKWRITE$) are the same status codes as those returned by FIOW. The advantages of these subprograms compared to FIOW and UCSMAKEIOPK$ include

- Ability to transfer larger amounts of data than FIOW
- Simpler interfaces
- Files that are assigned and freed through OPEN and CLOSE functions
- No need to construct an I/O packet by calling UCSMAKEIOPK$

The input/output ER (FIO, FIOW) subprograms described in the *COBOL Compiler Programming Reference Manual Volume 2* are limited to reading and writing a maximum of 128 tracks (229,376 words) of storage per subprogram call.

The standard UCS COBOL I/O subprograms are limited to reading and writing a maximum of 2^{24}-1 (16,777,215) words.

The UCS COBOL compilers are limited to allocating objects (such as structures and arrays) up to 2^{24}-1 words in size. The UCS COBOL compilers generate code that can address a maximum of 2^{24}-1 words for a single object.

The low-level I/O subprograms described in this subsection provide the capability to perform low-level I/O operations of up to 2^{30} (1,073,741,824) words per subprogram call.

Note: *The 2200/XPA system (such as the 2200/500, 2200/900, or HMP IX systems) supports very large banks up to 2^{33} words. However, the operating system currently supports very large banks up to 2^{30} words. A very large bank is made up of storage having consecutive bank descriptor indexes (BDI). The system imposes limits on the number of BDIs according to the address tree level. See the Internal Limits appendix in the* Linking System Programming Reference Manual *for documented limits. For a given program, the Exec and the Linking System may allocate a few BDIs at each address tree level for internal use. The actual number of BDIs allocated for internal use may vary. In addition, the storage in your program requires BDIs. Consequently, the maximum size of a very large bank and the maximum number of words performed by one I/O operation is somewhat smaller than the limits specified.*

The BULKREAD$ and BULKWRITE$ subprograms described in this section use the UCSMAKIOPK$ and FIOW subprograms described in the *COBOL Compiler Programming Reference Manual*, *Volume 2*. The ER IOW$ status codes are defined in the *ER Programming Reference Manual*.

The functions described in this subsection are limited to FASTRAND-formatted mass storage files only. Tape files are not supported. Program files (files containing elements) are supported.

Generally, three function calls are required to perform a simple read or write operation:

- A call to open a file or program file element
- A call to perform the read or write
- A call to close the file

The I/O operation will stop on any I/O error including an I/O error type 5 (read from unassigned area of mass storage). A status is returned from the function call indicating the I/O status.

If you call the BULKREAD$ read or BULKWRITE$ routines more than once, it does not continue reading or writing from where you left off. Multiple calls will result in the I/O operation being performed using the same mass storage address. However, you can change the mass storage address using the BULKSETSECTOR$ routine that is described in this section.

All parameters passed to the BULKIO functions must be on word boundaries. If data items residing in structures are passed as parameters, you may have to insert FILLER items to ensure this.

10.25.1. BULKOPEN$

Function

The BULKOPEN$ subprogram opens the OS 2200 file whose name is the string referenced by *filename.* Upon a successful open, the subprogram returns an address of a file information packet (FIP) in *file-id*.

Format

```
CALL 'BULKOPEN$' USING file-id, filename, io-type, open-status
```

where:

file-id

is an integer variable into which the address of the FIP is returned. The *file-id* variable must be passed as an argument for later read, write, and close operations.

Define *file-id* as PIC 1(36) BINARY-1.

filename

is a character string that must represent a valid OS 2200 file name. Qualifiers and file cycles are allowed. File names having element and version names are allowed. File read/write keys are not allowed. The contents of *filename* must end with or space. Define *filename* as a quoted string or as PIC X(*n*). If the file already exists, it must be a FASTRAND-formatted mass storage file.

io-type

is a character string that consists of a mode specification. The *io-type* string has three fields separated by commas:

- Read/Write mode indicator
- An element type indicator (optional)
- An element subtype indicator (optional)

The *io-type* string can consist of any mix of uppercase and lowercase characters. The *io-type* string must be terminated by at least one trailing space. If the element subtype indicator is specified, it must be preceded by the type indicator.

Table 10–3 describes the type string mode, element type, and element subtype indicators.

Table 10–3. Permitted Values for the Type Argument on the BULKOPEN$ Subprogram

Specification	Description
Read/Write Mode Indicator (required)	
R	Open existing file for reading
W	Open existing file for writing, or if the file does not exist, create a temporary file
r+	Open existing file for update (reading and writing)
Element Type Indicator (optional for element I/O)	
[, [TYPE=] {*type-name* | *type-code*}]	Specify the element type using either a name or number.
Element Subtype Indicator (optional for element I/O)	
[, [SUBTYPE=] *subtype-code*]	Specify the element subtype using a number.

open-status

is set to a status constant upon return from a call to BULKOPEN$. Define *open-status* as PIC 1(36) BINARY-1. The status values are described in Table 10–5.

If you open a file that is not assigned before the BULKOPEN$ call, the BULKOPEN$ subprogram attempts to assign the file in the following way:

- If the file is cataloged, it is assigned.
- If the file does not exist and the type argument is
  - r, r+, then BULKOPEN$ returns a failed indication.
  - w, then BULKOPEN$ assigns a temporary file.

Cataloged files not assigned to the run before the BULKOPEN$ call are freed when the file is closed.

Temporary files assigned during the BULKOPEN$ call remain assigned until the run terminates or until you explicitly free the file with the @FREE command.

All files assigned before the BULKOPEN$ call remain assigned until the run terminates or until you explicitly free the file with the @FREE command.

All use names attached to a file before a BULKOPEN$ call remain attached to the file after the file is closed.

If a given file is opened more than once before being closed, the closes should be done in reverse order. If this is not done, the file may not be left in the original "assign" state at the end.

Element I/O

The optional element type and subtype are only used if the *filename* has an element specification in it. If you include the optional element type and subtype indicator, they are syntactically validated but are ignored if the *filename* does not have an element specification in it. If you include an element name in the *filename* but do not include an element type indicator or element subtype indicator, the default element type is SYMBOLIC (1) and subtype is ELT (1). If an element subtype is specified, it must be proceeded by an element type specification.

The values for *type-name* and *type-code* are defined in Table 10–4.

Table 10–4. Permitted Values for the *type-name* and *type-code* on the BULKOPEN$ Subprogram

Element Type	*type-name*	*type-code*
symbolic	SYMBOLIC SYM	1
assembler procedure	ASMP	2
COBOL procedure	COBP	3
Fortran procedure	FORP	4
relocatable	RELOCATABLE REL RB	5
absolute, object modules, ZOOMs, and SSDEF	ABSOLUTE ABS	6
omnibus	OMNIBUS OMN	7

The *subtype-code* must be an integer constant. If the integer constant starts with a leading zero, it is treated as an octal value. The *subtype-code* must be a value between 0 and 63 (octal 0 through 077). The subtype information is used when creating a new program file element (write).

Read/Write ("r+") is not allowed for element I/O. Elements can only be read or created anew (write); they cannot be updated ("R+"). If you want to update an element, you must open it for input, read it in, close it, and then open it for output, write it, and close it.

The bulk I/O routines do not require exclusive access to write to a file. If you want exclusive access when writing to a file, you must assign the file yourself exclusively before opening the file.

When you close an element opened for write, it creates a new program file element with a new table of contents (TOC) entry. The contents of the new element are not available to other programs or activities until the file is closed.

You must not have more than one element at a time opened for write in a given program file. The OS 2200 program file format writes new elements at the end of the file, and bad results are ensured if multiple element writes are attempted simultaneously. The bulk I/O routines cannot diagnose this error.

For reads, the ASMP, COBP, and FORP element types are all treated as symbolics (SYM).

Element subtypes are ignored on reads.

Element subtypes are valid for type SYMBOLIC, ABSOLUTE, and OMNIBUS elements. Other element types do not have subtypes.

You cannot open an ABSOLUTE or RELOCATABLE element for write. Bulk I/O does not create the required information needed for the table of contents (TOC) entry.

When you read file/file-element text, it is read-in in a raw I/O manner. If it consists of SDF symbolic images, the SDF information is in the read-in text. If the images are cycled, all the cycles are there. It is up to your program to interpret the information. If you are working with SDF images with cycle information, you should use regular high-level COBOL I/O and not the low-level bulk I/O complex.

The cycle information for a SYMBOLIC element opened for write is all set to zero. You should not have cycling SDF images in the text of the element. If you read in a symbolic element having cycling images and write it out to a new symbolic element using bulk I/O, the cycling will be at zero and all images with other cycle numbers may be ignored by any program or processor that reads the new element. The content of any files/file-elements written is your responsibility.

Examples of Valid Type Strings

```
"r"    "r,TYPE=OMNIBUS"          "w,TYPE=SYMBOLIC,SUBTYPE=046"
"R"    "r,7"                  "W,1,046"
"W"    "R+"                   "W,OMNIBUS,2"
```

Examples of Invalid Type Strings

```
"r,,7"     "r=9"              "w,SUBTYPE=046"      "r,TYPE="
"r,8"      "w,TYPE=0,1"       "w,1,SUBTYPE=99" "r,TYPE"
"W 1,1"       "R,TYPE=ELT"       "r,TYPE=OM"          "write"
```

Table 10–5. Status Values Set by the BULKOPEN$ Subprogram

Status Value	Description
0	Successful open.
12	Cannot acquire dynamic storage for the file information packet (FIP) .
5502	*filename* string is empty.
5503	*type* string is empty.
5504	*type* string is invalid .
5505	Cannot assign the file. The file is either rolled-out or exclusively assigned to another run.
5506	Cannot acquire dynamic storage for the FIP at the required offset of 125 words from the start of the bank.
5508	Invalid 2200 file name.
5509	Cannot get exclusive use of the file with *type* = w or r+.

Table 10–5. Status Values Set by the BULKOPEN$ Subprogram

Status Value	Description
5510	File is not cataloged and *type* is r or r+.
5511	An attempt to assign a temporary file failed.
5512	Unable to find assign name for *filename*.
5514	Syntax error in the *filename* on an open.
5515	Attempt to open an element for Read/Write (type = "R+").
5516	Error from URTS library function ucspfwl (get next write location).
5518	Error from URTS library function ucspf (program file search).
5520	You are not allowed to create (write) ABSOLUTE or RELOCATABLE program file elements.
Other integer value (error number)	The UCS C memory allocation routine (malloc) failed while attempting to allocate storage for the FIP. See B.1 for a description of the error number.

10.25.2. BULKCLOSE$ Subprogram

Function

The BULKCLOSE$ subprogram closes the OS 2200 file associated with *file-id*. The subprogram frees a file if both of the following are true:

- The file is a cataloged file.
- The file is not assigned before the BULKOPEN$ call.

All use names attached to the file before the BULKOPEN$ call remain attached to the file after the BULKCLOSE$ call.

Format

```
CALL 'BULKCLOSE$' USING file-id, close-status
```

where:

file-id

is an integer variable containing the address of the FIP for the file to be closed. *file-id* is set by a call to BULKOPEN$. Define *file-id* as PIC 1(36) BINARY-1.

open-status

is set to a status constant upon return from a call to BULKCLOSE$. Define *open-status* as PIC 1(36) BINARY-1.

Table 10–6. Status Values Set by the BULKCLOSE$ Subprogram

Status Value	Description
0	Successful close
5601	*file-id* is not set
5507	File is already closed.
5517	Error from URTS library function ucspfuwl (set next write location).
5519	Error from URTS library function ucspfi (program file insert).
5522	You attempted to create an element larger than 262,143 sectors in size (262143 * 28 words). This is an OS 2200 program file format limit based on the 18-bit size fields in the program file table of contents entries.
Other integer value (error number)	The UCS C memory allocation routine (free) failed while attempting to free storage for the FIP. See B.1 for a description of the error number.

10.25.3. BULKREAD$ Subprogram

Function

The BULKREAD$ subprogram reads *bytes-to-read* number of bytes into the area pointed to by *buf*, from the file pointed by *file-id*. The BULKREAD$ subprogram rounds *bytes-to-read* up to the next multiple of four bytes, that is, to a whole word (*bytes-to-read* + 3) / 4.

Format

```
CALL 'BULKREAD$' USING file-id, buf, bytes-to-read, bytes-xferred,
sentinel-flag, sentinel, sentinel-char-offset, read-status
```

where:

file-id

is an integer variable containing the address of the FIP for the file to be closed. *file-id* is set by a call to BULKOPEN$. Define *file-id* as PIC 1(36) BINARY-1.

bytes-to-read

contains the size in bytes to be transferred on the read. Define *bytes-to-read* as PIC 1(36) BINARY-1.

buf

is a word-aligned character string that contains the data read by the subroutine. Define *buf* as PIC X(*n*), where *n* is large enough to hold the extraneous bytes transferred by BULKREAD$ when the *bytes-to-read* argument is not a multiple of 4.

bytes-xferred

is set by BULKREAD$ to the number of bytes transferred on the read. It is always a multiple of 4. The number of bytes transferred on the read will never be greater than the value of *bytes-to-read* + 3. Define *bytes-xferred* as PIC 1(36) BINARY-1.

sentinel-flag

is an integer value specifying whether the BULKREAD$ subprogram will or will not search for a *sentinel* character. Define *sentinal-flag* as a numeric integer or as PIC 1(36) BINARY-1.

sentinel

represents a 1-character variable that contains the value of a sentinel character. If the *sentinel-flag* is nonzero, the BULKREAD$ subprogram performs the read operation and then searches the area pointed by *buf* for a byte containing the value of the *sentinel* argument. The search is performed for *bytes-xferred* number of bytes. Define *sentinel* as a 1-character nonnumeric literal, a PIC X data item OR as a PIC 1(9) BINARY-1 data item for a value not allowed as nonnumeric literal. For example, if *sentinel* is to be octal 0400, the VALUE of the PIC 1 data item specified should be 256.

sentinel-char-offset

is an integer value returned if *sentinel-flag* is nonzero. If the search finds the *sentinel* character, the integer pointed to by the *sentinel-char-offset* argument is set to the byte offset of the *sentinel* in *buf*. The byte offset starts at zero. If the *sentinel* character is not found, the integer pointed to by the *sentinel-char-offset* argument is set to –1. The search operation can be used for objects that are greater than the standard UCS limit of 2^{24}-1 (16,777,215) words in size. Define *sentinel-char-offset* as PIC S1(36) BINARY-1.

read-status

is set to a status constant upon return from a call to BULKREAD$. Define *read-status* as PIC 1(36) BINARY-1. The BULKREAD$ subprogram sets *read-status* to the value returned by the Executive request FIOW subprogram.

Table 10–7. Status Values Set by the BULKREAD$ Subprogram

Status Value	Description
0	Successful read
5601	*File-id* is not set.
5607	The file is closed and cannot be accessed until it is opened again.
Other integer value	The bulkio_read function returned the status returned by the Executive request ER IOW$ function. Refer to the *ER Programming Reference Manual* for a complete list of ER IOW$ status codes.

Reading to the End of File

The BULKREAD$ subprogram uses the Executive request IOW and the I/O subprogram "read by BDI" (054). The BULKREAD$ subprogram returns an I/O status of 5 when you attempt to read from an unassigned area of the file. The Exec normally assigns files with track granularity (1792 words per track). A write of only 1 word to a file with no initial tracks reserved results in one track assigned. If you read until you encounter an I/O status 5 condition, the number of words transferred includes all the words of the last assigned track. If you have a large number of initially reserved tracks, you may read many tracks until you encounter an I/O status 5.

The number of words read depends on the value of the *bytes-to-read* argument and how many tracks are assigned to the file. In general, the extra words read are not part of the original data written but are part of the file's track assigned area and contain unpredictable values. You are responsible for interpreting the data read in and knowing where the logical end of the file is. The *sentinel* arguments can help you determine the logical end of the file.

Reading Program Files

An OS 2200 program file has a table of contents (TOC) at the beginning. There usually is a relatively large unassigned area of mass storage between the TOC and the text of the first element. If you attempt to read the whole program file in one bulk I/O, you will get an I/O Error 5 and will only read the TOC portion. Bulk I/O is low-level I/O so you must be careful using it.

10.25.4. BULKWRITE$ Subprogram

Function

The BULKWRITE$ subprogram writes *bytes-to-write* number of bytes from the area pointed to by *buf* to the file pointed to by *file-id*. The BULKWRITE$ subprogram rounds *bytes-to-write* up to the next multiple of four bytes, that is, to a whole word (*bytes-to-write* + 3) / 4.

Format

```
CALL 'BULKWRITE$' USING file-id, buf, bytes-to-write, bytes-xferred,
write-status.
```

where:

file-id

is an integer variable containing the address of the FIP for the file to be closed. *file-id* is set by a call to BULKOPEN$. Define *file-id* as PIC 1(36) BINARY-1.

buf

is a word-aligned character string that contains the data to be written by the subroutine. Define *buf* as PIC X(*n*), where *n* is large enough to hold the number of bytes to be written plus any sentinel character to be appended at the end of the data.

bytes-to-write

is a numeric integer or an integer variable. The integer or the contents of *bytes-to-wri*te specify how many bytes that BULKWRITE$ should write to the file. Define *bytes-to-write* as PIC 1(36) BINARY-1.

If *bytes-to-write* is negative, the same 4 bytes pointed to by *buf* are written *n* times to the file referenced by *file-id,* where *n* = (ABS(*bytes-to-write*) + 3 / 4). Passing a negative value for *bytes-to-write* is useful for clearing the contents of a file to zero.

bytes-xferred

is set by BULKWRITE$ to the number of bytes transferred on the write. It is always a multiple of 4. The number of bytes transferred on the write will never be greater than the value of *bytes-to-write* + 3. Define *bytes-xferred* as PIC 1(36) BINARY-1.

write-status

is an integer that contains the status returned by BULKWRITE$. BULKWRITE$ will set *write-status* to the status returned by the Executive request IOW function. Define open-status a PIC 1(36) BINARY-1.

Table 10–8. Status Values Set by the BULKWRITE$ Subprogram

Status Value	Description
0	Successful write
5501	*File-id* is not set
5507	The file is closed and cannot be accessed until it is opened again.
5513	Write attempted on file opened with *type* = r
Other integer value	The BULKWRITE$ routine returned the status returned by the Executive Request IOW function. Refer to the *ER Programming Reference Manual* for a complete list of ER IOW$ status codes.

10.25.5. BULKGETFILESIZE$ Subprogram

Function

This subprogram determines the size in bytes of a file. The file must be opened. The intent of this function is to give a file size that can be used for buffer allocations and other approximate uses that do not need an exact byte count. The size will be greater than or equal to the size of the program file element's text.

Format

```
CALL 'BULKGETFILESIZE$' USING file-id, file-size, status.
```

where:

file-id

is an integer variable containing the address of the FIP for the file for which the size is to be determined. The value of *file-id* is set by a call to BULKOPEN$. Define *file-id* as PIC 1(36) BINARY-1. The file *file-id* must be associated with a program file element opened for input (read). The size is determined by the size in its table of contents entry. If *file-id* is not associated with a program file element opened for input, the *file-size* parameter will be set to zero and a status of 5523 returned.

file-size

is set to the size in bytes of the file pointed to by *file-id*. Define *file-size* as PIC 1(36) BINARY-1. *file-size* is in terms of bytes, but is based on the number of sectors, so it will be a multiple of 112.

status

is set to a status constant upon return from a call to BULKGETFILESIZE$. Define *status* as PIC 1(36) BINARY-1.

Table 10–9. Status Values Set by the BULKGETFILESIZE$ Subprogram

Status Value	Description
0	Successful operation.
5501	The *file-id* is not set.
5507	The file is closed and cannot be accessed.
5523	The file is not a Program File Element opened for Input.

10.25.6. BULKGETSECTOR$ Subprogram

Function

The BULKGETSECTOR$ subprogram gets the starting sector address of a file. This subprogram can be used in combination with the BULKSETSECTOR$ subprogram to perform I/O starting at a specific mass storage address in a file. It can also be used to implement a random-access I/O facility.

Format

```
CALL 'BULKGETSECTOR$' USING file-id, file-sector-offset, status.
```

where:

file-id

is an integer variable containing the address of the FIP for the file for which the size is to be determined. *file-id* is set by a call to BULKOPEN$. Define *file-id* as PIC 1(36) BINARY-1.

file-sector-offset

Define *file-sector-offset* as PIC 1(36) BINARY-1.

If *file-id* is associated with a program file element opened for input, *file-sector-offset* will be the sector offset in the file where the element resides. If *file-id* is associated with a program file element opened for output, *file-sector-offset* will be the sector address where the element is to be written.

If *file-id* is associated with a data file, the *file-sector-offset* will be set to zero since that is where the I/O is based from.

File-sector-offset is set to the value as determined at open time. It is not affected by BULKSETSECTOR$ calls.

status

is set to a status constant upon return from a call to BULKGETSECTOR$. Define *status* as PIC 1(36) BINARY-1.

Table 10–10. Status Values Set by the BULKGETSECTOR Subprogram

Status Value	Description
0	Successful operation.
5501	The *file-id* is not set.
5507	The file is closed and cannot be accessed.

10.25.7. BULKSETSECTOR$ Subprogram

Function

The BULKSETSECTOR$ subprogram sets the sector offset variable in the bulk I/O FIP for the associated file. The next I/O call (BULKREAD$ or BULKWRITE$) will be done using this value as the mass-storage address for I/O. Normally, the starting mass-storage address for bulk I/O is invariant and is set at the open to the beginning of the file or element (element if file is associated with a program file element). This function allows you to change the mass storage address that is used for the I/O calls on the file.

This routine can be used in combination with the BULKGETSECTOR$ subprogram to be able to perform I/O operations starting at a specific mass storage address in a file. It can be used to implement a random-access I/O facility.

Format

```
CALL  'BULKSETSECTOR$' USING file-id, file-sector-offset, status.
```

where:

file-id

is an integer variable containing the address of the FIP for the file for which the size is to be determined. *file-id* is set by a call to BULKOPEN$. Define *file-id* as PIC 1(36) BINARY-1.

file-sector-offset

Define *file-sector-offset* as PIC 1(36) BINARY-1.

status

is set to a status constant upon return from a call to BULKSETSECTOR$. Define *status* as PIC 1(36) BINARY-1.

Table 10–11. Status Values Set by the BULKSETSECTOR$ Subprogram

Status Value	Description
0	Successful operation.
5501	The *file-id* is not set.
5507	The file is closed and cannot be accessed.
5521	You supplied a negative *file-sector-offset*. The *file-sector-offset* must be positive.

10.25.8. Example of Bulk I/O

The following example program executes calls to the bulk I/O subprograms and tests the results. The program does the following:

- Initializes 500 bytes of BUF with integer values 1-500.
- Sets BFCHAR (11) = 'A' (for later search purposes.).
- Opens file BULKIOTEST1.
- Writes the contents of BUF to file BULKIOTEST
- Reads file BULKIOTEST, with search sentinel BYTEINT set to 256
- Reads file BULKIOTEST, with search sentinel quoted-string 'A'
- Closes file BULKIOTEST.

The expected results are:

```
Attempt open
Attempt write
xfer-count = 0000000500
Sent-off should be 255, it is= +0000000255
Sent-off should be 10, it is= +0000000010
Attempt close
```

Example

```
IDENTIFICATION DIVISION.
PROGRAM-ID. BULKIO.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS.
OBJECT-COMPUTER. UNISYS.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
DATA DIVISION.
FILE SECTION.
WORKING-STORAGE SECTION.
```

```
 01 BUF.
     02 BFGP OCCURS 1 TO 1000 DEPENDING ON BUFSIZE.
        03 BFCHAR PIC 1(9)   BINARY-1.
 01 BUFSIZE    PIC S9(10) BINARY VALUE 500.
 01 WS-GRP.
     02 FILLER  PIC X(3).
     02 PIC 1(9)    BINARY-1  VALUE 256.
 01 XFER-COUNT PIC 1(36)   BINARY-1  VALUE 0.
 01 MY-STATUS  PIC 1(36)   BINARY-1  VALUE 0.
 01 SENT-OFF   PIC S1(36) BINARY-1 VALUE 0.
 01 I          PIC s9(10) BINARY   VALUE 0.
 01 FP         PIC 1(36)  BINARY-1 VALUE 0.
 PROCEDURE DIVISION.
 PPPP.
     PERFORM BUFSIZE TIMES
       ADD 1 TO I
       MOVE I TO BFCHAR (I)
     END-PERFORM.
     MOVE 65 TO BFCHAR (11).
* Open the file
     DISPLAY 'Attempt open'.
     CALL 'BULKOPEN$'
       USING FP, 'bulkiotest1.', 'W ', MY-STATUS.
     IF ((FP = 0) OR (MY-STATUS NOT = 0))
       DISPLAY 'Open failed, fp = 0, my-status = ', MY-STATUS
       STOP RUN.
     DISPLAY 'Attempt write'.
     CALL 'BULKWRITE$'
       USING FP, BUF, 500, XFER-COUNT, MY-STATUS.
     IF MY-STATUS NOT = 0
       DISPLAY 'status from WRITE is = '  MY-STATUS  STOP RUN.
     IF XFER-COUNT NOT = BUFSIZE
       DISPLAY 'xfer-count = ', XFER-COUNT .
     MOVE SPACES TO BUF.
     DISPLAY 'Attempt read'.
* Do search using integer value of 256
     CALL 'BULKREAD$'
       USING FP, BUF, 500, XFER-COUNT, 1, BYTEINT, SENT-OFF,
             MY-STATUS.
     IF MY-STATUS NOT = 0
       DISPLAY 'status from READ is = ', MY-STATUS  STOP RUN.
     DISPLAY 'sent-off should be 255, it is = ', SENT-OFF.
* Do search using character form
     CALL 'BULKREAD$'
       USING FP, BUF, 500, XFER-COUNT, 1, 'A', SENT-OFF,
             MY-STATUS.
     IF MY-STATUS NOT = 0
       DISPLAY 'status from READ is = ',  MY-STATUS STOP RUN.
     DISPLAY 'sent-off should be 10, it is = ', SENT-OFF.


*Close the file
```

```
DISPLAY 'Attempt close'.
CALL 'BULKCLOSE$' USING FP, MY-STATUS.
IF MY-STATUS NOT = 0
  DISPLAY 'status from CLOSE is = ', MY-STATUS.
STOP RUN.
```

10.26. Subprograms for Dynamic Memory Allocation

The subprograms described in this section provide the capability of dynamically allocating and de-allocating storage for a based data-item within a COBOL program. UCS COBOL has two separate mechanisms for defining based storage and two kinds of pointers. The first method uses the USING POINTER clause on 01 level declarations in the WORKING-STORAGE SECTION to define both, a based item and the pointer it is based on. Those pointers must be declared as PIC 1(36) BINARY-1, and hold one-word VA (Virtual Address) addresses. They have word granularity and can only point at word boundaries. You give them a pointer value by calling LOC$ to point it at an existing 01 or 77 item, by calling UCOBMALLOC$ to allocate new storage, or by doing binary arithmetic on them.

Example:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. VAexample.
DATA DIVISION.
WORKING-STORAGE SECTION.
77 sz  PIC S9(10) BINARY.
77 stat  PIC S9(10) BINARY.
77 ptr1  PIC 1(36) BINARY-1.
77 ptrcur  PIC 1(36) BINARY-1.
77 i14   PIC S9(10) BINARY VALUE 14.
01 mynames USING POINTER ptr1.
    03 names  PIC X(48) OCCURS 100 TIMES.
01 current-name PIC X(48) USING POINTER ptrcur.
PROCEDURE DIVISION.
ST.
    MOVE FUNCTION LENGTH(mynames) TO sz.
    CALL "UCOBMALLOC$" USING sz  ptr1  stat.
    IF stat IS NOT EQUAL TO 0 OR ptr1 IS EQUAL TO 0 STOP RUN.
    MOVE "John Doe" TO names(i14).
* Remember, VA pointers can only point to word boundaries:
    COMPUTE ptrcur = ptr1 + ( 48 / 4 ) * (i14 - 1).
    IF current-name IS NOT EQUAL TO "John Doe"
       DISPLAY "*ERROR: " current-name.
```

The second method is compatible with the IBM COBOL pointer and based storage facility. You declare a pointer variable as having USAGE POINTER, and the based structure is declared in the LINKAGE SECTION. You SET the ADDRESS OF the based structure to either the ADDRESS OF some other structure or variable (at any level) or to a pointer that was declared as USAGE POINTER. This newer pointer form has byte granularity and can point at byte boundaries, and the UCS C two-word pointer form is used for the pointers. (They are compatible with UCS C pointers.) You give a pointer a value by calling LOC$ to point it at some other 01 structure, by calling UCOBMALLOC$2 to get new storage or by a SET statement. The SET statement can set a pointer to the value of another pointer or to the ADDRESS OF some other structure or substructure (any level). You cannot do arithmetic on these C-style pointers (adding 1 to one of these pointers would NOT cause it to point at the next byte).

Example:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. CPTRexample.
DATA DIVISION.
WORKING-STORAGE SECTION.
77 sz  PIC S9(10) BINARY.
77 stat  PIC S9(10) BINARY.
77 newptr  USAGE POINTER.
77 i14   PIC S9(10) BINARY VALUE 14.
LINKAGE SECTION.
01 mynames.
    03 names  PIC X(48) OCCURS 100 TIMES.
01 current-name PIC X(48).
PROCEDURE DIVISION.
ST.
    MOVE FUNCTION LENGTH(mynames) TO sz.
    CALL "UCOBMALLOC$2" USING sz  newptr  stat.
    IF stat IS NOT EQUAL 0 OR newptr is EQUAL TO NULL STOP RUN.
    SET ADDRESS OF mynames TO newptr.
    MOVE "John Doe" TO names(i14).
    SET ADDRESS OF current-name TO ADDRESS OF names(i14).
    IF current-name IS NOT EQUAL TO "John Doe"
       DISPLAY "*ERROR: '" current-name "'".
```

See the *UCS COBOL Programming Reference Manual Volume 1, COBOL Statements*, for more information on using the USING POINTER clause and the USAGE POINTER clause.

The memory allocation routines call the URTS runtime system to find space in data banks for the memory requests. The OS 2200 M-Series architecture supports the following databank types:

- Small banks (2^{18} words = 262,144 words)
- Large banks (2^{24} words = 16,777,216 words)
- Very large banks (2^{30} = 1,073,741,824 words)

Note: *The M-Series architecture supports very large banks, up to 2^{33} words in size. However, the Operating System currently only supports very large banks up to 2^{30} words in size. A very large bank (VLB) is made up of consecutive smaller banks. The system imposes limits on the number of banks according to the address tree level. See the* Linking System Programming Reference Manual *for documented limits. For a given program, the Exec and the Linking System might allocate a few additional banks at each address level for internal use. The actual number of banks allocated for internal use can vary. Consequently, the maximum size of a very large bank is smaller than the limits specified.*

The UCS COBOL compiler allocates automatic variables in a small bank (the ALS) and allocates static variables in a small or large bank depending on the size of the static data items. (In UCOB generated-code, WORKING-STORAGE local variables are put into automatic storage when the INITIAL clause is used on the PROGRAM-ID statement.) The memory allocation subprograms add the capability of creating a VLB through this allocation process. However, generated code from the UCS compilers (UCOB, UC, UFTN, UPLS) cannot easily reference data beyond the first 2^{24} words in a VLB, so use of VLBs is discouraged. No single data structure or array must exceed 2^{24} words in size. Blobs are an exception as they are handled as a unit.

There are two sets of memory allocation subprograms; one set working with one-word VA pointers and one set working with 2-word C-style pointers. The interfaces are identical with the exception of the pointer form. When an allocation call succeeds, the returned pointer can be used to access a based object in the space allocated until the space is explicitly freed or reallocated. Allocation fails if storage is not available to satisfy the request. You must check the value of the returned pointer. A zero or NULL pointer value indicates failure. A return status parameter will also be non-zero on a failure and provides additional information.

The memory allocation subprograms that work with the one-word VA pointers are:

- UCOBMALLOC$
- UCOBCALLOC$
- UCOBFREE$
- UCOBREALLOC$

The memory allocation subprograms that work with the two-word C-style pointers are:

- UCOBMALLOC$2
- UCOBCALLOC$2
- UCOBFREE$2
- UCOBREALLOC$2

10.26.1. UCOBMALLOC$

Function

The UCOBMALLOC$ subprogram allocates storage for the number of bytes specified by *number-of-bytes*, returns the address (VA) of the storage allocated in *address-name*, and the return status of the operation in *return-status*.

Format

```
CALL 'UCOBMALLOC$' USING number-of-bytes, address-name, return-status
```

where:

number-of-bytes

is an integer variable. Define *number-of-bytes* as PIC 1(36) BINARY-1.

address-name

is an integer variable. Define *address-name* as PIC 1(36) BINARY-1.

return-status

is an integer variable. Define *return-status* as PIC 1(36) BINARY-1. See Table 10–12 for possible values.

10.26.2. UCOBCALLOC$

Function

The UCOBCALLOC$ subprogram allocates storage for the number of bytes specified by *number-of-bytes*, clears the allocated area to binary zero, returns the address of the storage allocated in *address-name,* and the return status of the operation in *return-status*.

Format

```
CALL 'UCOBCALLOC$' USING number-of-bytes, address-name, return-status
```

where:

number-of-bytes

is an integer variable. Define *number-of-bytes* as PIC 1(36) BINARY-1.

address-name

is an integer variable. Define *address-name* as PIC 1(36) BINARY-1.

return-status

is an integer variable. Define *return-status* as PIC 1(36) BINARY-1. See Table 10–12 for possible values.

10.26.3. UCOBFREE$

Function

The UCOBFREE$ subprogram de-allocates the storage at the address contained in *address-name* and the return status of the operation in *return-status*. The address in *address-name* must refer to the address returned from a call to UCOBMALLOC$, UCOBCALLOC$, or UCOBREALLOC$.

Format

```
CALL 'UCOBFREE$' USING address-name, return-status
```

where:

address-name

is an integer variable. Define *address-name* as PIC 1(36) BINARY-1.

return-status

is an integer variable. Define *return-status* as PIC 1(36) BINARY-1. See Table 10–12 for possible values.

10.26.4. UCOBREALLOC$

Function

The UCOBREALLOC$ subprogram attempts to allocate storage within the storage pointed to by *address-name-1.* The contents of the *address-name-1* is set by a prior call to UCOBMALLOC$, UCOBREALLOC$, or UCOBCALLOC$. If the size of the storage previously allocated is equal or greater than the contents of *number-of-bytes*, the previously allocated storage is reused and the address of the storage is placed in *address-name-2*. If the size of the previously allocated storage is less than the contents of *number-of-bytes*, new storage is allocated and the address is placed in *address-name-2*. The *number-of-bytes* bytes of storage from the previous allocation is also moved to the new allocation and the previous storage is freed. The return status of the operation is placed in *return-status*.

Format

```
CALL 'UCOBREALLOC$' USING address-name-1, number-of-bytes, address-
name-2,
return-status
```

where:

address-name-1

is an integer variable. Define *address-name-1* as PIC 1(36) BINARY-1.

number-of-bytes

is an integer variable. Define *number-of-bytes* as PIC 1(36) BINARY-1.

address-name-2

is an integer variable. Define *address-name-2* as PIC 1(36) BINARY-1.

return-status

is an integer variable. Define *return-status* as PIC 1(36) BINARY-1. See Table 10–12 for possible values.

10.26.5. UCOBMALLOC$2

Function

The UCOBMALLOC$2 subprogram allocates storage for the number of bytes specified by *number-of-bytes*, returns the address of the storage allocated as a two-word C-style pointer in *address-name*, and the return status of the operation in *return-status*.

Format

```
CALL 'UCOBMALLOC$2' USING number-of-bytes, address-name, return-status
```

where:

number-of-bytes

is an integer variable. Define *number-of-bytes* as PIC 1(36) BINARY-1.

address-name

is a 2-word C-style pointer variable. Define *address-name* as USAGE POINTER.

return-status

is an integer variable. Define *return-status* as PIC 1(36) BINARY-1. See Table 10–12 for possible values.

10.26.6. UCOBCALLOC$2

Function

The UCOBCALLOC$2 subprogram allocates storage for the number of bytes specified by *number-of-bytes*, clears the allocated area to binary zero, returns the address of the storage allocated in *address-name*, and the return status of the operation in *return-status*.

Format

```
CALL 'UCOBCALLOC$2' USING number-of-bytes, address-name, return-status
```

where:

number-of-bytes

is an integer variable. Define *number-of-bytes* as PIC 1(36) BINARY-1.

address-name

is a 2-word C-style pointer variable. Define *address-name* as USAGE POINTER.

return-status

is an integer variable. Define *return-status* as PIC 1(36) BINARY-1. See Table 10–12 for possible values.

10.26.7. UCOBFREE$2

Function

The UCOBFREE$2 subprogram de-allocates the storage at the address contained in *address-name* and the return status of the operation in *return-status*. The address in *address-name* must refer to the address returned from a call to UCOBMALLOC$2, UCOBCALLOC$2, or UCOBREALLOC$2.

Format

```
CALL 'UCOBFREE$2' USING address-name, return-status
```

where:

address-name

is a 2-word C-style pointer variable. Define *address-name* as USAGE POINTER.

return-status

is an integer variable. Define *return-status* as PIC 1(36) BINARY-1. See Table 10–12 for possible values.

10.26.8. UCOBREALLOC$2

Function

The UCOBREALLOC$2 subprogram attempts to allocate storage within the storage pointed to by *address-name-1*. The contents of the *address-name-1* are set by a prior call to UCOBMALLOC$2, UCOBREALLOC$2, or UCOBCALLOC$2. If the size of the storage previously allocated is equal to or greater than the contents of *number-of-bytes*, the previously allocated storage is reused and the address of the storage is placed in *address-name-2*. If the size of the previously allocated storage is less than the contents of *number-of-bytes*, new storage is allocated and the address is placed in *address-name-2*. The *number-of-bytes* bytes of storage from the previous allocation is also moved to the new allocation and the previous storage is freed. The return status of the operation is placed in *return-status*.

Format

```
CALL 'UCOBREALLOC$2' USING address-name-1, number-of-bytes, address-
name-2,
return-status
```

where:

address-name-1

is a 2-word C-style pointer variable. Define *address-name-1* as USAGE POINTER.

number-of-bytes

is an integer variable. Define *number-of-bytes* as PIC 1(36) BINARY-1.

address-name-2

is a 2-word C-style pointer variable. Define *address-name-2* as USAGE POINTER. *return-status* is an integer variable. Define *return-status* as PIC 1(36) BINARY-1. See Table 10–12 for possible values.

Table 10–12. Status Values Returned in *return-status*

Status Value	Description
0	Successful execution
12	Not enough storage is available for the allocation request
Other integer value (error number)	See subsection B.1 *UCS Runtime System Messages (URTS)* of Volume 2 for a description of the error number.

10.27. Interface to Zip Utilities

COBOL users now have the ability to access data from a zip file on the OS 2200. In addition, they can compress data on the OS 2200 into a zip file. This interface requires the use of CIFS for ClearPath OS 2200 (see the *CIFS for ClearPath OS 2200 User, Programmer, and Administrator Reference Manual*). This interface allows COBOL applications to read data directly from the compressed zip file, as needed. In addition, this capability allows the COBOL user to take data from the OS 2200, compress it, and write it to a zip file.

This feature uses interfaces to the ZIPUT Processor that is a component of CIFS to compress and decompress the data.

In order to accomplish the desired task of reading/writing compressed data from a zip file; the COBOL user must have the following capabilities:

- The ability to determine the number of entries in a specified zip file.
- The ability to determine the names of each of the entries in a zip file.
- The ability to read data from a specified entry in a zip file.
- The ability to write data to a specified entry in a zip file.
- The ability to close a zip file once all the data has been written to it.
- The ability to set the compression level for subsequent zip file entries.

The user must also be able to determine if an error occurred during the processing of the data so that the appropriate action can be taken.

Parameters containing an error status, size, or number must start on a word boundary. These are 01/77 level variables declared, for example, as PIC 9 (10) BINARY, PIC S9 (10) BINARY.

The subsections that follow describe the COBOL callable entries that can be used to access zip files from the 2200.

10.27.1. ZIP$NBRENTRIES

Function

This routine is called to determine the number of entries that are contained in a specified zip file.

Format

```
CALL 'ZIP$NBRENTRIES'  USING zipfile-name, nbr-of-entries, error-
status.
```

where:

zipfile-name

is a variable that contains the name of the zip file that is to be checked. This field is described as PIC X(*n*) and must be followed by a null character (low-values).

nbr-of-entries

is a variable into which the number of entries that the zip file contains will be returned. This variable must be defined as PIC 9(10) BINARY.

error-status

is a variable into which the status of the operation is returned. This variable must be defined as PIC 9(10) BINARY.

The error statuses returned by this routine are as follows:

0	Normal return
1	Invalid name specified for the zip file
2	Zip file could not be found or in error
3	File name or entry could not be found
6	Out of memory
9	Internal error during initialization of the URTS zip utilities

Example

```
01  zipgrp.
    02  zipfile-name pic x(20) value 'test.zip'.
    02  nulterm  pic x value low-values.
01  nbr-of-entries pic 9(10) binary value 0.
01  error-status pic 9(10) binary value 0.

   CALL 'ZIP$NBRENTRIES'
        USING zipfile-name nbr-of-entries error-status.
```

10.27.2. ZIP$NAMEENTRIES

Function

This routine is called to retrieve the name of each of the entries in a specified zip file. It will also return the number of entries in the zip file.

The name of each entry is placed into the buffer that was passed to this routine. The size allowed for each name is also passed to this routine. If the name is smaller than the buffer being filled in, it is space padded. If the name is larger than the buffer that is being filled in, the name is truncated and an error status is returned.

Format

```
CALL 'ZIP$NAMEENTRIES' USING
     zipfile-name, buffer-for-names nbr-of-entries, entry-size, error-
status.
```

where:

zipfile-name

is a variable that contains the name of the zip file that is to be checked. This field is described as PIC X(*n*) and must be followed by a null character (low-values).

buffer-for-names

is the buffer into which entry names in the specified zip file will be placed.

nbr-of-entries

is a variable into which the number of entries that the zip file contains will be returned. This variable must be defined as PIC 9(10) BINARY.

entry-size

is a variable that contains the size of buffer into which the entry name will be stored. This variable must be defined as PIC 9(10) BINARY.

error-status

is a variable into which the status of the operation is returned. This variable must be defined as PIC 9(10) BINARY.

The error statuses returned by this routine are as follows:

0	Normal return
1	Invalid name specified for the zip file
2	Zip file could not be found or in error
3	Zip file not found
4	Entry name was truncated
5	Unexpected exception
6	Out of memory
9	Internal error during initialization of the URTS zip utilities

Example

```
01  zipgrp.
    02  zipfile-name  pic x(20) value 'test8.zip'.
    02  nulterm  pic x value low-values.
01  nbr-of-entries pic 9(10) binary.
01  entry-size pic 9(10) binary value 25.
01  buffer-for-names.
    02 name-of-entry pic x(25) occurs 30 times value space.
01  error-status pic 9(10) binary.

    CALL 'ZIP$NAMEENTRIES'
         USING zipfile-name buffer-for-names nbr-of-entries entry-size
error-status.
```

10.27.3. ZIP$READENTRY

Function

This routine is called to read a buffer of data from a specified entry that is contained in a zip file. The data read is placed into the buffer provided in the parameters.

Format

```
       CALL 'ZIP$READENTRY'
            USING zipfile-name, nametoread, namesize, buffer, buffer-
size, sizeread,
            error-status.
```

where:

zipfile-name

is a variable that contains the name of the zip file that is to be checked. Thisfield is described as PIC X(n) and must be followed by a null character (low-values).

nametoread

is a variable that contains the name of the entry within the zip file that is to be read. This variable is describes as PIC X(n).

namesize

is a variable that contains the length of *nametoread*. This variable must be defined as PIC 9(10) BINARY.

buffer

is the buffer into which the decompressed data is stored.

buffer-size

is a variable that contains the size of buffer. This variable must be defined as PIC 9(10) BINARY.

sizeread

is a variable that contains the number of characters that were returned into buffer. This variable must be defined as PIC 9(10) BINARY.

error-status

is a variable into which the status of the operation is returned. This variable must be defined as PIC 9(10) BINARY.

The error statuses returned by this routine are as follows:

0 Normal return
1 Zip file name could not be converted to string
2 Zip file could not be found or in error
3 File name or entry could not be found
4 Out of memory (call to malloc failed)
6 Out of memory
7 Unexpected exception
9 Internal error during initialization of the URTS zip utilities

Example

```
01  zipgrp.
    02  zipfile  pic x(20) value 'test.zip'.
    02  nulterm  pic x value low-values.
01  buffer pic x(40).
01  buffer-size pic 9(10) binary value 40.
01  sizeread pic 9(10) binary value 0.
01  nametoread pic x(20).
01  namesize pic 9(10) binary value 20.
01  error-status pic 9(10) binary.
    CALL 'ZIP$READENTRY'
          USING zipfile nametoread, namesize, buffer,
                buffer-size, sizeread error-status.
```

10.27.4. ZIP$WRITEENTRY

Function

This routine is called to write a buffer of data to an entry in a zip file. The utility will compress the data and write it to the appropriate zip file entry. If an error is detected, the appropriate error status is returned from this routine. The default compression level will be used unless the ZIP$SETCOMPRESSION utility has been called. See 10.27.7 for additional information on ZIP$SETCOMPRESSION.

Format

```
CALL 'ZIP$WRITEENTRY' USING
     zipfile-name, entry-name, buffer, buffer-size, error-status.
```

where:

zipfile-name

is a variable that contains the name of the zip file that is to be checked. This field is described as PIC X(n) and must be followed by a null character (low-values).

entry-name

is a variable that contains the name of the entry within the zip file that is to be written. This variable is describes as PIC X(n).

buffer

is the buffer that contains the data that is to be written to the zip file entry.

buffer-size

is a variable that contains the number of characters that are in buffer. This variable must be defined as PIC 9(10) BINARY.

error-status

is a variable into which the status of the operation is returned. This variable must be defined as PIC 9(10) BINARY.

The error statuses returned by this routine are as follows:

0 Normal return
1 Zip file name could not be converted to string
2 Zip file could not be found or in error
3 Attempt to write a buffer to an entry that has already been written to and is closed.
4 Zip file name or entry name are all spaces
5 Size of buffer to be written is less than or equal to 0.
6 Out of memory
7 Unexpected exception
9 Internal error during initialization of the URTS zip utilities

Example

```
01  zipgrp.
    02  zipfile-name pic x(20) value 'ziptest15.zip'.
    02  nulterm  pic x value low-values.
01  error-status pic 9(10) binary value 0.
```

```
01  buffer pic x(50) value all 'a'.
01  entry-name-group.
    02  entry-name pic x(25).
01  buffer-size pic 9(10) binary.

     CALL 'ZIP$WRITEENTRY'
       USING zipgrp entry-name-group, buffer,buffer-size, error-status.
```

10.27.5. ZIP$CLOSEENTRY

Function

This routine is called from the COBOL program in order to close the zip file that is being written to. This routine need only be called following the completion of the ZIP$WRITEENTRY routine in order for the last buffer to be written out.

Format

```
CALL 'ZIP$CLOSEENTRY' USING error-status.
```

where *error-status* is a variable into which the status of the operation is returned. This variable must be defined as PIC 9(10) BINARY.

The possible error statuses that are returned are as follows:

0 Normal return
2 Zip file could not be found or in error
3 Attempt to close zip file without ever writing to one
6 Out of memory
9 Internal error during initialization of the URTS zip utilities

Example

```
01  error-status pic 9(10) binary value 0.

   CALL 'ZIP$CLOSEENTRY' USING error-status.
```

10.27.6. ZIP$PRINTEXCEPTION

Function

This routine is called to print the error exception that occurred during the processing of a zip file. This utility should be called if a nonzero error status was returned from one of the other zip utilities. This routine will then print the message that is associated with the error status.

Format

```
CALL 'ZIP$PRINTEXCEPTION' USING error-status.
```

where *error-status* is the variable into which the status of the operation is returned. This variable must be defined as pic 9(10) binary.

The possible error statuses returned by this routine are as follows:

0 Normal return
1 No exception to print
9 Internal error during initialization of the URTS zip utilities

Example

```
01  error-status pic 9(10) binary value 0.
  CALL 'ZIP$PRINTEXCEPTION' USING error-status.
```

10.27.7. ZIP$SETCOMPRESSION

Function

This routine is called to set the compression level of subsequent zip file entries. The new compression level will not take effect until the first write to a new zip file entry. The new compression level will stay in effect until ZIP$SETCOMPRESSION is called again. Valid compression levels are -1 to 9 (-1 is the default, 0 is no compression, 1 is best speed, 9 is best compression).

Format

```
CALL 'ZIP$SETCOMPRESSION' USING compression-level error-status.
```

where:

compression-level

is a variable that contains the compression level to be used on subsequent zip file entries. This variable must be defined as pic s9(10) binary.

error-status

is the variable into which the status of the operation is returned. This variable must be defined as pic 9(10) binary.

The error statuses returned by this routine are as follows:

0 Normal return
1 Compressed level is invalid
2 Error occurred during setting of the compression level
9 Internal error during initialization of the URTS zip utilities

Example

```
01  compression-level pic s9(10) binary.
01  error-status pic 9(10) binary value 0.

    MOVE 4 TO compression-level.
    CALL 'ZIP$SETCOMPRESSION'
          USING compression-level error-status.
```

10.27.8. Example Program

The following example program will write one buffer to a zip file entry. This zip file entry is then read and verified.

```
 IDENTIFICATION DIVISION.
 PROGRAM-ID. ziptest12.
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER. UNISYS-2200.
 OBJECT-COMPUTER. UNISYS-2200.
 SPECIAL-NAMES.
     PRINTER IS PRT.
 DATA DIVISION.
 WORKING-STORAGE SECTION.
 01  zipgrp.
     02  zipfile-name pic x(20) value 'ziptest12.zip'.
     02  nulterm  pic x value low-values.
 01  nbr-of-entries pic 9(10) binary value 0.
 01  error-status pic 9(10) binary value 0.
 01  buffer pic x(50) value all 'a'.
 01  entry-name-group.
     02  entry-name pic x(25) value 'entry1.txt'.
     02  nullterm pic x value low-values.
 01  entry-size pic 9(10) binary value 40.
 01  buffer-for-names.
     02  name-of-entry  pic x(40) occurs 30 times value space.
 01  nametoread pic x(25).
 01  flag pic x value 'F'.
     88 eof value 'T'.
 01  namesize pic 9(10) binary value 25.
 01  readbuffer pic x(25).
 01  buffer-size pic 9(10) binary value 25.
 01  sizeread pic 9(10) binary.
 01  compression-level pic s9(10) binary.
 PROCEDURE DIVISION.
 BEGIN.
     display 'In COBOL main program Ziptest12' upon prt.
     move 0 to error-status.
*
* Set compression level and verify operation was successful
*
     move 4 to compression-level.
     call 'ZIP$SETCOMPRESSION'
          using compression-level error-status.

     if error-status not = 0
        display 'Non zero error status returned = '
                 error-status upon prt
        call 'ZIP$PRINTEXCEPTION' using  error-status.
*
```

```
* Write a buffer to entry 'entry1.txt' in 'ziptest12.zip'
*
     call 'ZIP$WRITEENTRY'
       using zipgrp entry-name-group , buffer, 50 error-status.

*
* Verify that the operation was successful and close the entry
*
     if error-status not = 0
        display 'Non zero error status returned = '
                 error-status upon prt
        call 'ZIP$PRINTEXCEPTION' using  error-status.
     call 'ZIP$CLOSEENTRY' using error-status.
     if error-status not = 0
        display 'Non zero error status returned = '
                 error-status upon prt
        call 'ZIP$PRINTEXCEPTION' using  error-status.
*
* Determine the number of entries in the zip file
*
     call 'ZIP$NBRENTRIES'
           using zipgrp nbr-of-entries error-status.
     if error-status not = 0
        display 'Non zero error status returned = '
                 error-status upon prt
        call 'ZIP$PRINTEXCEPTION' using  error-status.
     if nbr-of-entries = 1
          display 'Test 1 OK' upon prt
       else
          display 'Test 1 NG' upon prt.
*
* Verify the name of the entry that was created in the zip file
*
     call 'ZIP$NAMEENTRIES'
        using zipgrp buffer-for-names nbr-of-entries
              entry-size error-status.
     if error-status = 0
        display 'TEST 2  OK no error detected' upon prt
     else
        display 'TEST2 NG error status =' error-status upon prt
        call 'ZIP$PRINTEXCEPTION' using  error-status.
     if name-of-entry(1) = 'entry1.txt'
        display 'TEST 3 OK' upon prt
     else
        display 'TEST 3 NG' upon prt
     end-if.
*
* Read 25 characters at a time from 'entry1.txt'
* in 'ziptest12.zip' and verify the contents
*
     move space to readbuffer.
     move 'entry1.txt' to nametoread.
```

```
call 'ZIP$READENTRY'
  using zipgrp nametoread, namesize, readbuffer,
          buffer-size, sizeread error-status.
if error-status not = 0
   display 'Error status bad ' error-status upon prt
   call 'ZIP$PRINTEXCEPTION' using  error-status.

if sizeread = 25
   display 'TEST 4 OK' upon prt
else
   display 'TEST 4 NG' upon prt.
if readbuffer = 'aaaaaaaaaaaaaaaaaaaaaaaa'
   display 'TEST 5 OK' upon prt
else
   display 'TEST 5 NG' upon prt.

move space to readbuffer.
move 'entry1.txt' to nametoread.
call 'ZIP$READENTRY'
  using zipgrp nametoread, namesize, readbuffer,
          buffer-size, sizeread error-status.
if error-status not = 0
   display 'Error status bad ' error-status upon prt
   call 'ZIP$PRINTEXCEPTION' using  error-status.

if sizeread = 25
   display 'TEST 6 OK' upon prt
else
   display 'TEST 6 NG' upon prt.
if readbuffer = 'aaaaaaaaaaaaaaaaaaaaaaaa'
   display 'TEST 7 OK' upon prt
else
   display 'TEST 7 NG' upon prt.

move space to readbuffer.
move 'entry1.txt' to nametoread.
call 'ZIP$READENTRY'
  using zipgrp nametoread, namesize, readbuffer,
          buffer-size, sizeread error-status.
if error-status not = 0
   display 'Error status bad ' error-status upon prt
   call 'ZIP$PRINTEXCEPTION' using  error-status.

if sizeread = 0
   display 'TEST 8 OK' upon prt
else
   display 'TEST 8 NG' upon prt.
if readbuffer = space
   display 'TEST 9 OK' upon prt
else
   display 'TEST 9 NG' upon prt.
```

```
        Display 'Test complete' upon prt.
        STOP RUN.
```

Prior to the execution of this program, the user must point to the CIFS directory that contains the zip file that will be acted upon. An example of this follows:

```
@CIFSUT
cd /os2200/qual/filename
```

For more information on this, see the CIFS for ClearPath OS 2200 User Programmer, and Administrator Reference Manual.

The result of the execution of the preceding program should be as follows:

```
In COBOL main program Ziptest12
Test 1 OK
TEST 2  OK no error detected
TEST 3 OK
TEST 4 OK
TEST 5 OK
TEST 6 OK
TEST 7 OK
TEST 8 OK
TEST 9 OK
Test complete
```

10.28. Retrieving Coordinated Universal Time (UTC)

COBOL users now have the ability to retrieve the Coordinated Universal Time (UTC). This feature is available only if the machine architecture supports UTC.

This feature provides user interfaces to retrieve UTC time as well as the information necessary to determine the local time equivalent. The feature uses the Exec interface SYSTEM$TIME. For complete details on this interface, see the *System Services Programming Reference Manual*.

The subsections that follow describe the COBOL callable routines that can be used to access UTC time.

Note: *If you use the UTC time functions in HVTIP, you must use the INITIAL clause on the PROGRAM-ID.*

10.28.1. UTC$TIME

Function

This routine is called to return the UTC time in UCSDWTIME$ format. The routine UCSCNVTIM$CC can then be used to convert the UTC time into a character format.

Format

```
CALL 'UTC$TIME'  USING parm, returnstatus.
```

where:

parm

is a structure that will contain the result of the routine. It should be in the following format:

```
01  parm.
    02 version pic 9(5) binary value 0.
    02 filler pic 9(5) binary value 0.
    02 timeb
       03 dwtime pic s1(72) binary-1 value 0.

       03 daylight-savings pic 9(5) binary value 0.
       03 time-differential pic s9(5) binary value 0.
       03 time-zone pic xxxx value space.
```

where:

version is the version of the packet being used.

timeb is the structure that contains the following:

dwtime	is the UTC time expressed in UCSDWTIME$ format.
daylight-savings	is a flag indicating if daylight saving time is in effect.
time-differential	contains the differential between UTC and local time.
time-zone	is the mnemonic for the local time zone.

returnstatus

is a variable into which the status of the operation is returned. The status is returned in ELMS format and must be defined as follows:

```
01  returnstatus.
    02  elms.
        03 prod-id    pic 1(18) binary-1.
        03 res        pic 1(3)  binary-1.
        03 message-id pic 1(15) binary-1.
    02  aux    pic s9(10) binary.
```

The message-id variable can be examined to determine the status of the call. The following values can be returned:

0	is the UTC time expressed in UCSDWTIME$ format.
96	system configuration does not allow for UTC time.

Refer to the System Services Programming Reference Manual for all other values returned.

See 10.28.3 for an example of using the UTC$TIME routine.

10.28.2. TIMEB2DWTIM$

Function

This routine is called to convert the UTC time returned by UTC$TIME to local time. The time differential and daylight saving time flag will be used in doing the conversion. The local time will be returned in UCSDWTIME$ format.

The routine UCSCNVTIM$CC can then be used to convert the local time into a character format.

Format

```
CALL 'TIMEB2DWTIM$' USING timeb, localtime.
```

where:

timeb

is a variable that contains the UTC time returned by the UTC$TIME routine that needs to be converted to local time. This time is in UCSDWTIME$ format. See 10.28.1 for information on how to obtain this variable.

localtime

is a variable that will contain the local time that was derived from the UTC time. This local time will be in UCSDWTIME$ format. This variable should be defined as pic s1(72) binary-1.

See 10.28.3 for an example of using the TIMEB2DWTIM$ routine.

10.28.3. Example Program

The following program illustrates the use of the UTC$TIME and TIMEB2DWTIM$ routines.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. utctime.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
```

```
 OBJECT-COMPUTER. UNISYS-2200.
 DATA DIVISION.
 WORKING-STORAGE SECTION.
 01  parm.
     02 version pic 9(5) binary value 1.
     02 filler  pic 9(5) binary value 0.
     02 timeb

        03 dwtime pic s1(72) binary-1 value 0.
        03 daylight-savings pic 9(5) binary value 0.
        03 time-differential pic s9(5) binary value 0.
        03 time-zone pic xxxx value space.
 01  tim  pic x(17).
 01  returnstatus.
     02  elms.
         03 prod-id    pic 1(18) binary-1.
         03 res        pic 1(3)  binary-1.
         03 message-id pic 1(15) binary-1.
     02  aux    pic s9(10) binary.
 01  local-time pic s1(72) binary-1  value 0.
 PROCEDURE DIVISION.
 BEGIN.
* Call service routine to retrieve the UTC time.
* The UTC time is returned in UCSDWTIME$ format, therefore
* the routine UCSCNVTIM$CC is called to convert the UTC time
* to a character format.

     call 'utc$time' using parm, returnstatus.
     evaluate message-id
       when 0
          call 'ucscnvtim$cc' using dwtime tim
          display 'UTC Time Follows:'
          display 'Year = ' tim(1:4)
          display 'Month= ' tim(5:2)
          display 'Day  = ' tim(7:2)
          display 'Hour = ' tim(9:2)
          display 'Min  = ' tim(11:2)
          display 'Sec  = ' tim(13:2)
          display 'Mill = ' tim(15:3)
          display 'Zone = ' time-zone
       when 96
          display 'System configuration does not allow UTC time'
          display 'Local time is returned'
       when other
          display 'Returned error status = ' message-id
     end-evaluate.

     display '    '.

* Convert UTC time to local time by calling the
* routine TIMEB2DWTIM$.
```

```
call 'timeb2dwtim$' using timeb local-time.
call 'ucscnvtim$cc' using local-time tim.

display 'UTC Time Converted to local time follows:'
display 'Year = ' tim(1:4).
display 'Month= ' tim(5:2).
display 'Day  = ' tim(7:2).
display 'Hour = ' tim(9:2).
display 'Min  = ' tim(11:2).
display 'Sec  = ' tim(13:2).
display 'Mill = ' tim(15:3).
display 'Zone = ' time-zone.
```

The result of running this program will look as follows:

```
UTC Time Follows:
Year = 2003
Month= 03
Day  = 11
Hour = 15
Min  = 13
Sec  = 14
Mill = 205
Zone = CST

UTC Time Converted to local time follows:
Year = 2003
Month= 03
Day  = 11
Hour = 09
Min  = 13
Sec  = 14
Mill = 205
Zone = CST
```

Note: *This example was run on a system that is in the United States Central Time Zone (CST). This time zone is 6 hours behind UTC time.*

Section 11
Transaction Processing in UCS

Programs written in COBOL, FORTRAN, and C can be executed in a Transaction Processing (TIP) or High-Volume Transaction Processing (HVTIP or HVTIP2) environment within the Universal Compiling System (UCS). Table 11–1 shows the transaction processing interfaces supported for these three host languages.

Table 11–1. Transaction Processing Interfaces from UCS Languages

UCS COBOL	UCS FORTRAN	UCS C
DPS 2200	DPS 2200	DPS 2200
MCB	MCB	MCB
Primitives that do not handle messages	Primitives that do not handle messages	Primitives that do not handle messages
DMS 2200	DMS 2200	
RDMS 2200 SQL Embedded SQL	RDMS 2200 SQL	RDMS 2200 SQL
SFS 2200 Relative access (direct SDF) Indexed access (MSAM)	SFS 2200 Direct access (direct SDF)	SFS 2200 Binary access (direct SDF)
PCIOS	PCIOS	PCIOS
FCSS	FCSS	FCSS

Notice that Message Control Bank (MCB) and Display Processing System (DPS 2200) functions are supported for all three host languages. Most primitives that do not handle messages are supported by the UCS host languages. COMPOOL message-buffering primitives, however, are not supported in UCS programming for any language.

All three host languages can access the Universal Data System (UDS) relational database model (RDMS 2200) and the conventional Shared File System (SFS 2200). The network-hierarchical Database Management System (DMS 2200) is supported only for UCS COBOL and UCS FORTRAN.

UCS COBOL includes an embedded SQL interface for RDMS 2200.

A production TIP transaction that wishes to have TIP sticking and RESTART should not have PADS as its contingency routine. PADS does a CREATE$BANK call to the Exec and this kills TIP sticking and RESTART for the transaction. One way to keep PADS out is to name the URTS runtime library in the RESOLVE statement in the static link of your transaction. This results in PADS stub object module C$NPADS being linked into your program.

```
RESOLVE ALL REFERENCES USING L_D,SYS$LIB$*EMOMRTS.,LCN
```

Another way is to specifically include the C$NPADS object module from the URTS library along with using the LOCAL_DEFS (or L_D) option in the RESOLVE statement in your static link:

```
INCLUDE SYS$LIB$*EMOMRTS.C$NPADS
RESOLVE ALL REFERENCES USING L_D,LCN
```

For a more detailed description of transaction processing, see the following documents:

- *Application Development Solutions Application Development Programming Guide* (7831 4077)
- *Configuration and Operations Guide (7833 1550)*
- *ClearPath OS 2200 Display Processing System (DPS 2200) Run-Time Programming Reference Manual* (7830 7840)
- *Transaction Processing Applications Programming Guide* (7830 9986)

11.1. Using DPS 2200 Functions to Code Transactions

DPS 2200 supports transactions written in UCS COBOL. DPS 2200 provides a simple call interface to perform its work, as shown in the following skeleton:

```
CALL 'D$INIT' USING DPS-STATUS, INFO-BUFFER.
  .
  .
  .
CALL 'D$OPEN' USING DPS-STATUS, SCREEN-xxxxx-nn.
  .
  .
  .
CALL 'D$SEND' USING DPS-STATUS, SCREEN-xxxxx-nn.
  .
  .
  .
```

The statements used to call DPS 2200 in UCS programs are the same as in non-UCS programs. The same DPS 2200 software and forms files can be used in both environments simultaneously.

UCS TIP and HVTIP transactions that access DPS 2200 can use either MCB or COMPOOL message buffering, depending on the following conditions:

- VALTAB STATUS keyword setting
- Availability of the MCB software
- The designation of MCB message buffering in the SILAS configuration

MCB message buffering is used if the following conditions exist:

- STA,J is specified in the VALTAB entry.
- MCB software is available.
- MCB message buffering is designated in SILAS.

If the STA keyword is specified without J, then COMPOOL message buffering is used.

11.2. Using MCB Functions to Code Transactions

All UCS TIP and HVTIP transactions can directly call MCB functions. The host language UCS COBOL supports calls to MCB functions, using the following generic syntax:

```
CALL 'MCB$ENT' USING p-mcb-packet, p-mcb-status
                     [, msg-buffer[, multi-dest-list]].
```

Notice that a call to MCB in a UCS program differs from one in a non-UCS program because it includes two additional optional parameters, as follows:

msg-buffer

The parameter for the message buffer identifies the storage area into which messages are read and from which messages are sent by the transaction. This parameter eliminates the need for the CLOCTE call used in ASCII COBOL programs to set up this buffer address.

multi-dest-list

The fourth parameter, *multi-dest-list*, provides a simple method for identifying a multiple-destination list of position identifiers (PID). This parameter eliminates the need for the CLOCTE call used in ASCII COBOL programs to set up this buffer address.

Transactions that use MCB functions require the following:

- Declarations of the MCB functions
- Allocation of storage space for the MCB packet
- MCB error message handling buffers
- In some cases, a message buffer

To aid the UCS programmer in coding these requirements, the MCB release tape includes sample UCS COBOL procedures for these buffers that you can use as a template. The element containing these procedures, MCBDEF-UCOB, is found in the MCB installation file.

11.3. Using Primitives That Do Not Handle Messages to Code Transactions

UCS programming supports the following primitives that do not handle messages for UCS COBOL:

- COMMIT and ROLLBACK
- CONECT and DISCON
- All file-handling functions in the File Control Super Structure (FCSS)
- TIMER call primitives for scheduling transaction execution at a future time: TIMABS, TIMDEL, TIMREL, TIMUPA, and TIMUPR
- Primitives for the execution-option indicator: UOPAND, UOPGET, and UOPTOR

The COMPOOL message-handling primitives are not supported in UCS programming.

The coding of primitives has not changed in UCS programs, except the function names, as shown in the following table.

Previous Name	UCS Name
CCONET	CONECT
CDISCN	DISCON
CCMMIT	COMMIT
CRLBAK	ROLLBACK
CFCSS	FCSS
CUOPND	UOPAND
CUOPGT	UOPGET
CUOPTR	UOPTOR
CTMABS	TIMABS
CTMDEL	TIMDEL
CTMREL	TIMREL
CTMUPA	TIMUPA
CTMUPR	TIMUPR

The previous name of each call is supported in UCS COBOL transactions, but you should use the UCS name when coding new UCS transactions or upgrading existing transactions to UCS.

11.4. Retrieving the TIP RESTART Status (UCOB_RESTARTCHECK and UCS_RESTARTCHECK)

This routine returns a value that indicates whether the current execution is from an initial program load or from a TIP RESTART/BT-Load.

Format

```
CALL 'UCOB_RESTARTCHECK' USING ret-value.
```

where *ret-value* is a word binary integer that tells you how the currently executing program was loaded. The returned values are

0 is an initial program load

1 is a TIP RESTART

2 is an HVTIP RESTART

3 is a TIP BT-Load

UCOB_RESTARTCHECK is also available as the parameterless function UCS_RESTARTCHECK. Languages that can use functions (such as C and FORTRAN) can call this function instead of UCOB_RESTARTCHECK if they wish.

Description

A TIP or HVTIP program that has TIP sticking can experience a TIP RESTART. After the program's exit, if there is another transaction of the same trancode that is queued or that is scheduled within a short time of the exit, the Exec restarts execution of the transaction at its starting address. The state of the data banks of the transaction are as they were on program exit. In the case of an HVTIP transaction, the HVTIP loader reloads data as needed and the rest of the run-time system and user code not need to do anything special. If it is a self-initializing (PRGTYP 2) or reentrant (PRGTYP 3) TIP transaction, the UCS Runtime System reinitializes its run-time data. However, it is up to each individual user program to reinitialize any of its own data that will be needed for correct execution.

A TIP transaction can also be loaded by what is called a BT-Load (Block-Transfer Load). The D-banks of the transaction are loaded from another running copy of the transaction. BT-Loads can occur if the K and I options are on the STATUS VALTAB entry for the transaction and if the BTLOADING flagbox bit is on. However, due to the way BT-Loads are done, the data may not be consistent and correct reinitialization may be impossible to do. Incorrect program execution can result. For example, if a TIP transaction calls an FLSS (Fast-Load Self-Contained Subsystem), BT-Loads **must not** be enabled for the transaction. If your TIP transaction experiences random failures when loaded by TIP BT-Loads, then don't use them. TIP/HVTIP RESTARTs are much more reliable since data is in a consistent state from a clean program exit.

The reason for TIP RESTARTs and BT-Loads is performance. When a TIP RESTART or BT-Load occurs and the transaction is restarted

- The Exec does not have to allocate banks and load the transaction.
- The system does not have to allocate memory pages to the transaction since the memory has already been allocated by the previous execution.

To get TIP sticking and RESTART

- The program type (PRGTYP) must be self-initializing (2), reentrant (3), or HVTIP (5).
- The I option must be on the STATUS VALTAB entry.
- The STICKING flagbox bit must be on.

A number of factors can inhibit TIP sticking and RESTART:

- Any contingency
- Any symbiont prints (DISPLAY statements)
- Not detaching from MCB
- Having an active TERMREG at program exit
- Any create-bank or modify-bank calls to the Exec
- Not exiting via the normal URTS exit
- Any files that have been assigned must be freed before program exit

See "Using Performance Improvement Techniques" in the *Transaction Processing Programming Reference Manual* for information on TIP sticking and RESTART. For information on VALTAB entries and the VTBUTL utility see "Registering Transaction Programs" in the *Transaction Processing Administration and Operations Reference Manual*.

Example

```
IDENTIFICATION DIVISION.
PROGRAM-ID. RESTARTCHECK.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SPECIAL-NAMES.
    PRINTER IS PRINTER.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 restart-value pic s9(10) usage binary value 0.
PROCEDURE DIVISION.
ST.
     call 'UCOB_RESTARTCHECK' using restart-value.
     if restart-value is equal to 0
        display 'This execution was loaded by a normal program load' upon printer.
     if restart-value is equal to 1
        display 'This execution is a TIP RESTART' upon printer.
     if restart-value is equal to 2
        display 'This execution is an HVTIP RESTART' upon printer.
     if restart-value is equal to 3
        display 'This execution was loaded by a TIP BT-Load '
        upon printer.
    EXIT PROGRAM.
```

11.5. Considerations

You must observe the following restrictions in your UCS COBOL source program to execute it successfully in a transaction processing environment:

- The local variables of a program that is loaded by either a TIP BT-Load or a TIP RESTART will have the values left by the previous execution. Therefore, they may not have the initial values from any VALUE clauses. However, if you use the INITIAL clause on the PROGRAM-ID statement, then the local variables are put in their initial state each time the program is called. Use of the INITIAL clause is recommended if your transaction is loaded by TIP BT-Load or TIP RESTART.
- The CANCEL verb should not be used in the TIP environment, but it is supported in the HVTIP environment.
- Report writer modules are not allowed in a TIP environment.
- See the *UCS Application Development Programming Guide*, for details on the COBOL CALL identifier statement in the TIP/HVTIP environment.

Section 12
Contingency Handling

Contingency handling is a service that notifies you when abnormal, illegal, or unexpected conditions occur. Contingency handlers must be registered with the Exec. For UCS COBOL, the interfaces to contingency registration routines and handlers are written as UCS COBOL-callable language program constructs. This section describes these interfaces.

The URTS-supplied user contingency handling interfaces and PADS are not compatible. Do not link PADS into programs that have user contingency handling interfaces. However, if you have your own contingency handler, you still can use PADS to perform postmortem debugging by using the diagnostic file, DIAG$, created by the Exec. For example, after executing your program and capturing the storage limits error, you can issue a @PMD command and the Exec will call PADS and invoke a canned PADS dump procedure. This dump will include a register dump, walkback, and bank dump for the last contingency. If you want to use PADS for further debugging, reexecute the program followed by an @PADS DIAG command. This allows you to enter PADS commands. For example, to dump the jump stack, enter INQUIRE JUMPS.

To use the UCS COBOL interfaces to contingency handling, you must be familiar with

- Extended mode contingency handling
- Extended mode gate and bank management
- Extended mode operating environment, including the concepts of rings and domains

See the *System Services Programming Reference Manual* for more information.

Note that the UCS Runtime System does not verify either the contingency handling input parameters of the interface routines `or the contents of the contingency registration packet`.

Status return variables from the contingency interfaces are either 2-word or 3-word integer values as follows:

- 2-word values (including the last 2 words of the 3-word value) are the return status information from an extended mode Exec call, except where noted. See the *System Services Programming Reference Manual* for specific Exec interface returns in registers A0 and A1.
- 3-word values are for contingency registration. The first word is a prefix that describes the format of the remaining 2-word value.

Two versions for registering contingencies are available as follows:

1. If you are using any type of registration, you can use the REG4CGY-COB2, CREG4CGY-COB2, or PREG4CGY-COB2 interfaces. Unisys recommends using these interfaces.
2. If you are using the version zero or version 2 registration types without using additional registration attributes you can use REG4CGY-COB, CREG4CGY-COB, or PREG4CGY-COB interfaces.

12.1. User Contingency Handler Registration

The user contingency handler registration interfaces register the calling subsystem for contingency handling. You can call these interfaces from the home subsystem or from a shared subsystem. See the *Linking System Subsystems Programming Guide* for information on subsystem types and attributes.

The extended mode Exec interface used is REG4CGY$.

12.1.1. Contingency Registration Packet Interface

Interface Names

```
REG4CGY-COB2 (for home subsystems)
PREG4CGY-COB2 (for protected subsystems)
CREG4CGY-COB2 (for chameleon subsystems)
```

The parameters in the following call setup example are the same for each interface name.

Call Setup

Note: *A common parameter list is used by all interfaces.*

```
:
COPY CGY-REG.
:
01   name      PIC X(n).
01   namelen   PIC S9(10) USAGE BINARY VALUE n.
01   locality  PIC S9(10) USAGE BINARY.
01   priv      PIC S9(10) USAGE BINARY.
01   ring      PIC S9(10) USAGE BINARY.
01   status01.
     02 stat   PIC S9(10) USAGE BINARY OCCURS 3 TIMES.
:
     CALL "REG4CGY-COB2" USING name, namelen, cgy-reg-pkt, cgy-pkt-size, locality,
                               priv, ring, status01.
```

where:

name

The name of the user contingency handling routine. It is processed as a character string by the registration interface routine. (*n* is the length in characters of the contingency handling routine name.)

namelen

The length in bytes of the user contingency handling routine's name. It is processed as an integer value by the registration interface routine.

cgy-reg-pkt

The address of the contingency registration packet. See the *Exec Systems Services PRM* under contingency registration REG4CGY$ for details.

cgy-pkt-size

The size in words of the contingency registration packet.

locality

The address tree level of the gate created for the contingency handler. Allowable values are

0 Caller's level (default)

1 Application level

2 Program level

3 Activity level

5 Caller's level

priv

The processor privilege at which the contingency handler executes. Allowable values are

0 Ordinary user (default)

2 Proprietary user

3 Ordinary user

ring

The ring level of the gate created for the contingency handler. Allowable values are

0 Ring 3 (default)

2 Ring 2

3 Ring 3

status

A 3-word integer value containing the registration status that is returned. If the first word returned is zero, registration was successful. If the first word is not zero, registration failed, and the first word describes the format for the last 2-word value as shown in the table that follows.

First Word	Description
0	Registration completed normally.
1	The contingency gate could not be created. The 2-word value is the status returned from the Exec CREATE$GATE interface.
2	The contingency registration with the Exec failed. The 2-word value is the status returned from the Exec REG4CGY$ interface.
3	User contingency handling entry point address is invalid. The 2-word value is invalid.

Contingency Registration Packet

The following contingency registration packet definitions are found in the Unisys system PROC file (SYSLIB*PROC$). The PROC name is CGY-PKTS. See the *System Services Programming Reference Manual* for a complete description of field definitions and meanings.

```
CGY-REG PROC
       01  CGY-REG-PKT.
           05  cgy-version             PIC S9(5) USAGE BINARY VALUE 2.
           05  cgy-function            PIC S9(5) USAGE BINARY VALUE 30.
           05  cgy-penetrate           PIC 1(36) USAGE BINARY-1 VALUE 0.
           05  cgy-reserved            PIC 1(30) USAGE BINARY-1 VALUE 0.
           05  cgy-boundary            PIC 1(6) USAGE BINARY-1 VALUE 0.
       01  CGY-PKT-SIZE                PIC S9(10) USAGE BINARY VALUE 3.
 END
```

12.1.2. Contingency Registration Version or Function Interface

Interface Names

```
REG4CGY-COB (for home subsystems)
PREG4CGY-COB (for protected subsystems)
CREG4CGY-COB (for chameleon subsystems)
```

The parameters in the following call setup example are the same for each interface name.

Call Setup

Note: *A common parameter list is used by all interfaces.*

```
:
01   name      PIC X(n).
01   namelen   PIC S9(10) USAGE BINARY VALUE n.
01   func.
     05  version       PIC 9(5) USAGE BINARY
     05  function-code PIC 9(5) USAGE BINARY
01   locality  PIC S9(10) USAGE BINARY.
01   priv      PIC S9(10) USAGE BINARY.
01   ring      PIC S9(10) USAGE BINARY.
01   status01.
     02 stat   PIC S9(10) USAGE BINARY OCCURS 3 TIMES.
:
     CALL "REG4CGY-COB" USING name, namelen, func, locality,
                              priv, ring, status01.
```

where:

name

The name of the user contingency handling routine. It is processed as a character string by the registration interface routine. (*n* is the length in characters of the contingency handling routine name.)

namelen

The length in bytes of the user contingency handling routine's name. It is processed as an integer value by the registration interface routine.

func

The type of contingency registration. The only allowable values are

Value	Description
10	Extended mode contingency registration.
30	Extended mode contingency registration with additional coverage for unregistered subsystems and AFCBs that are called from the subsystem in which this registration is being done.

This field contains two half-word binary fields (9(5) BINARY). The first PIC 9(5) BINARY field is the version. Version 0 supports word 0 of the contingency packet only. Version 1 supports words 0 to 2 with words 1 and 2 being set to zero. Passing registration attributes are not allowed. Version 2 indicates that the registering subsystem is prepared to handle the additional contingency information that can be passed to an extended mode contingency routine in CGY_PKT1 and CGY_PKT2. See the *Systems Services Programming Reference Manual* under contingency registration REG4CGY$ for details.

The second PIC 9 (5) BINARY field is the function code.

locality

The address tree level of the gate created for the contingency handler. Allowable values are

0 Caller's level (default)

1 Application level

2 Program level

3 Activity level

5 Caller's level

priv

The processor privilege at which the contingency handler executes. Allowable values are

0 Ordinary user (default)

2 Proprietary user

3 Ordinary user

ring

The ring level of the gate created for the contingency handler. Allowable values are

0 Ring 3 (default)

2 Ring 2

3 Ring 3

status

A 3-word integer value containing the registration status that is returned. If the first word returned is zero, registration was successful. If the first word is not zero, registration failed, and the first word describes the format for the last 2-word value as shown in the table that follows.

First Word	Description
0	Registration completed normally.
1	The contingency gate could not be created. The 2-word value is the status returned from the Exec CREATE$GATE interface.
2	The contingency registration with the Exec failed. The 2-word value is the status returned from the Exec REG4CGY$ interface.
3	User contingency handling entry point address is invalid. The 2-word value is invalid.

12.1.3. Contingency Packet Processing

When a contingency occurs the Exec passes the registered contingency handler parameters that describe the execution environment at the time of the contingency.

The following contingency packet definitions are found in the Unisys system PROC file (SYSLIB*PROC$). The PROC name is CGY-PKTS. See the *System Services Programming Reference Manual* for a complete description of field definitions and meanings.

```
CGY-PKTS PROC
       01  CGY-PKT1.
           05  version                    PIC 1(18) USAGE BINARY-1.
           05 unused-1                    PIC 1(2)  USAGE BINARY-1.
           05 cp-par-at-wait              PIC 1     USAGE BINARY-1.
           05 sensitive-data              PIC 1     USAGE BINARY-1.
           05 default-contingency         PIC 1     USAGE BINARY-1.
           05 use-cgypkt2                 PIC 1     USAGE BINARY-1.
           05  prev-act                   PIC 1(6)  USAGE BINARY-1.
           05  cgy-state                  PIC 1(6)  USAGE BINARY-1.
           05  cgy-ss                     PIC 1(36) USAGE BINARY-1.
           05  severity-code              PIC 1(3)  USAGE BINARY-1.
           05  product-id                 PIC 1(15) USAGE BINARY-1.
           05  unused-2                   PIC 1(3)  USAGE BINARY-1.
           05  message-id                 PIC 1(15) USAGE BINARY-1.
```

```
05  unused-3                        PIC 1(18) USAGE BINARY-1.
05  error-type                      PIC 1(6)  USAGE BINARY-1.
05  error-code                      PIC 1(6)  USAGE BINARY-1.
05  cgy-type                        PIC 1(6)  USAGE BINARY-1.
05  auxiliary-status                PIC 1(36) USAGE BINARY-1.
05  unused-4                        PIC 1(18) USAGE BINARY-1.
05  bank-identifier                 PIC 1(18) USAGE BINARY-1.
05  bank-type                       PIC 1(18) USAGE BINARY-1.
05  base-algorithm-phase            PIC 1(18) USAGE BINARY-1.
05  interface-specification         PIC 1(18) USAGE BINARY-1.
05  base-register-avbr              PIC 1(18) USAGE BINARY-1.
05  signal-em-er-data               PIC 1(36) USAGE BINARY-1.
05  status-word-1                   PIC 1(36) USAGE BINARY-1.
05  status-word-2                   PIC 1(36) USAGE BINARY-1.
05  status-word-3                   PIC 1(36) USAGE BINARY-1.
05  data-word-1                     PIC 1(36) USAGE BINARY-1.
05  data-word-2                     PIC 1(36) USAGE BINARY-1.
05  data-word-3                     PIC 1(36) USAGE BINARY-1.
05  data-word-4                     PIC 1(36) USAGE BINARY-1.
05  access-privilege                PIC 1(36) USAGE BINARY-1.
05  penetrated-wait-condition       PIC 1(6)  USAGE BINARY-1.
05  ip-privilege                    PIC 1(6)  USAGE BINARY-1.
05  address-translation-mode        PIC 1(6)  USAGE BINARY-1.
05  base-register-cpbr              PIC 1(18) USAGE BINARY-1.
05  operand-address                 PIC 1(36) USAGE BINARY-1.
05  executing-instruction           PIC 1(36) USAGE BINARY-1.
05  unused-6                        PIC 1(25) USAGE BINARY-1.
05  basic-mode                      PIC 1(1)  USAGE BINARY-1.
05  integer-carry                   PIC 1(1)  USAGE BINARY-1.
05  integer-overflow                PIC 1(1)  USAGE BINARY-1.
05  characteristic-overflow         PIC 1(1)  USAGE BINARY-1.
05  characteristic-underflow        PIC 1(1)  USAGE BINARY-1.
05  divide-check                    PIC 1(1)  USAGE BINARY-1.
05  arithmetic-exception-enable     PIC 1(1)  USAGE BINARY-1.
05  floating-point-residue-enable   PIC 1(1)  USAGE BINARY-1.
05  bdr-select                      PIC 1(1)  USAGE BINARY-1.
05  quarter-word-select             PIC 1(1)  USAGE BINARY-1.
05  dp-fp-underflow-mask            PIC 1(1)  USAGE BINARY-1.
05  b-registers                     PIC 1(36) USAGE BINARY-1
                                             OCCURS 15 TIMES.
05  x-registers                     PIC 1(36) USAGE BINARY-1
                                             OCCURS 12 TIMES.
05  a-registers                     PIC 1(36) USAGE BINARY-1
                                             OCCURS 20 TIMES.
05  r-registers                     PIC 1(36) USAGE BINARY-1
                                             OCCURS 16 TIMES.
05  rcs-status                      PIC 1(36) USAGE BINARY-1.
05  rcs-flags                       PIC 1(18) USAGE BINARY-1.
05  rcs-function                    PIC 1(18) USAGE BINARY-1.
05  rcsres0                         PIC 1(18) USAGE BINARY-1.
05  rcs-offset                      PIC 1(18) USAGE BINARY-1.
05  rcsres1                         PIC 1(36) USAGE BINARY-1.
05  rcsres2                         PIC 1(36) USAGE BINARY-1.
05  rcsres3                         PIC 1(36) USAGE BINARY-1.
05  rcsres4                         PIC 1(36) USAGE BINARY-1.
05  cgy-routine-data                PIC 1(36) USAGE BINARY-1.
05  unused-7                        PIC 1(30) USAGE BINARY-1.
05  return-action                   PIC 1(6)  USAGE BINARY-1.
05  originating-ss-va               PIC 1(36) USAGE BINARY-1.
05  prev-cgy-parm-va                PIC 1(36) USAGE BINARY-1.
```

```
      01  CGY-PKT2.
          05  ip-conditions-pkt2           PIC 1(36) USAGE BINARY-1.
          05  b-registers-pkt2             PIC 1(36) USAGE BINARY-1
                                                 OCCURS 15 TIMES.
          05  x-registers-pkt2             PIC 1(36) USAGE BINARY-1
                                                OCCURS 12 TIMES.
          05  a-registers-pkt2             PIC 1(36) USAGE BINARY-1
                                                OCCURS 20 TIMES.
          05  r-registers-pkt2             PIC 1(36) USAGE BINARY-1
                                                OCCURS 16 TIMES.
          05  va-prev-resume-action        PIC 1(36) USAGE BINARY-1.
      01  CGY-REEN.
          05 reentry                       PIC 1(36) USAGE BINARY-1
                                                OCCURS 3 TIMES.
END
```

Example

Note: *Program 1 and 3 are one compilation unit or program 2 and 3 are one compilation unit.*

Program 1

```
IDENTIFICATION DIVISION.
PROGRAM-ID.
    S02061.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
SPECIAL-NAMES.
    PRINTER IS PR.
DATA DIVISION.
WORKING-STORAGE SECTION.
COPY CGY-REG.
01 name             PIC X(7) VALUE "TAKECGY".
01 namelen          PIC S9(10) USAGE BINARY VALUE 7.
01 locality         PIC S9(10) USAGE BINARY VALUE 5.
01 priv             PIC S9(10) USAGE BINARY VALUE 3.
01 ring             PIC S9(10) USAGE BINARY VALUE 3.
01 status01.
02 stat1            PIC S9(10) USAGE BINARY OCCURS 3 TIMES VALUE 0.
01 BAD-IMAGE        PIC X(12) VALUE "@ASG,A .F. .".


PROCEDURE DIVISION.
BEGIN.
    CALL "REG4CGY-COB2" USING name, namelen, cgy-reg-pkt,
                                  cgy-pkt-size, locality, priv, ring,
status01.
    DISPLAY " stat1(1) = " stat1(1) UPON PR.
    DISPLAY " stat1(2) = " stat1(2) UPON PR.
    DISPLAY " stat1(3) = " stat1(3) UPON PR.
    DISPLAY "Issue BADCSF contingency ..." UPON PR.
```

```
    CALL "FACSF" USING BAD-IMAGE.
    DISPLAY "IN MAIN AGAIN" upon PR.
    STOP RUN.
END PROGRAM S02061.
```

Program 2

```
IDENTIFICATION DIVISION.
PROGRAM-ID.
    ABC.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
SPECIAL-NAMES.
    PRINTER IS PRT.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
DATA DIVISION.
FILE SECTION.
WORKING-STORAGE SECTION.
01  name      PIC X(7) VALUE "TAKECGY".
01  namelen   PIC S9(10) USAGE BINARY VALUE 7.
01  func
    05  version         PIC 9(5) usage BINARY VALUE 2.
    05  function-code   PIC 9(5) USAGE BINARY VALUE 10.
01  locality  PIC S9(10) USAGE BINARY VALUE 5.
01  priv      PIC S9(10) USAGE BINARY VALUE 2.
01  ring      PIC S9(10) USAGE BINARY VALUE 3.
01  status01.
    02 stat   PIC S9(10) USAGE BINARY OCCURS 3 TIMES VALUE 0.
01  x         PIC S9(10) USAGE BINARY VALUE 3.
01  y         PIC S9(10) USAGE BINARY VALUE 0.
01  z         PIC S9(10) USAGE BINARY VALUE 3.
PROCEDURE DIVISION.
BEGIN.
    CALL "REG4CGY-COB" USING name, namelen, func, locality,
                                        priv, ring, status01.
    DISPLAY " stat(1) = " stat(1) UPON PRT.
    DISPLAY " stat(2) = " stat(2) UPON PRT.
    DISPLAY " stat(3) = " stat(3) UPON PRT.
    DIVIDE y INTO x GIVING z.
    DISPLAY "After divide fault" UPON PRT.
    STOP RUN.
END PROGRAM ABC.
```

Program 3

```
IDENTIFICATION DIVISION.
PROGRAM-ID.
    TAKECGY.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
SPECIAL-NAMES.
    PRINTER IS PRT.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
DATA DIVISION.
FILE SECTION.
LINKAGE SECTION.
COPY CGY-PKTS.
PROCEDURE DIVISION USING cgy-pkt1, cgy-reen, cgy-pkt2.
BEGIN.
    DISPLAY " error-type = " error-type UPON PRT.
    DISPLAY " error-code = " error-code UPON PRT.
    DISPLAY " cgy-type = " cgy-type UPON PRT.
    DISPLAY " reentry(1) = " reentry(1) UPON PRT.
    DISPLAY " reentry(2) = " reentry(2) UPON PRT.
    DISPLAY " reentry(3) = " reentry(3) UPON PRT.
    MOVE 20 TO return-action.
    EXIT PROGRAM.
END PROGRAM TAKECGY.
```

12.2. User Contingency Handler Deregistration

The user contingency handler deregistration interface deregisters a contingency handler.

The extended mode Exec interface used is REG4CGY$ with function code 20.

Interface Name

```
DREG4CGY-COB
```

Call Setup

```
:
01  status01.
    02  stat   PIC S9(10) USAGE BINARY OCCURS 2 TIMES VALUE 0.
:
    CALL "DREG4CGY-COB" USING status01.
```

where *status* is a 2-word integer value that contains the deregistration status that is returned. If the first word is zero, deregistration is successful. If the first word is not zero, deregistration failed. See the *System Services Programming Reference Manual* for specific REG4CGY$ status codes and definitions.

12.3. User Termination Handler Registration

The user termination handler registration interface registers a handler for program and activity termination. This interface can be called from the home subsystem or from a shared subsystem. The termination handler registration is performed on a per activity basis.

The extended mode Exec interface used is REG4TERM$.

Interface Name

```
REG4TERM-COB
```

Call Setup

```
:
01   name          PIC X(n).
01   namelen       PIC S9(10) USAGE BINARY VALUE n.
01   termid        PIC S9(10) USAGE BINARY.
01   termdata      PIC S9(10) USAGE BINARY.
01   termbuff.
     02   buff     PIC S9(10) USAGE BINARY OCCURS 11 TIMES VALUE 0.
01   termstat.
     02   stat     PIC S9(10) USAGE BINARY OCCURS 2 TIMES VALUE 0.
:
     CALL "REG4TERM-COB" USING name, namelen, termid, termdata,
                               termbuff, termstat.
```

where:

name

The name of the termination handling routine. It is processed as a character string by the termination interface routine. (*n* is the length in characters of the termination handling routine name.)

namelen

The length in bytes of the termination handling routine. It is processed as an integer value by the termination interface routine.

termid

A 1-word integer value that contains the termination registration identification. This parameter identifies the termination handler for this subsystem to the Exec. This value is passed to the user termination handling routine.

termdata

A 1-word binary value that is passed to the user termination handling routine.

termbuff

An 11-word contiguous data area used by the UCS Runtime System to construct the packet needed to invoke the user termination handling routine with the proper parameters and UCS execution environment. This is the buffer that must be passed to the termination registration interface routine as an input parameter. The UCS Runtime System constructs the contents of this buffer. Only the space (11 contiguous words) needs to be provided. The buffer should not be modified between registration and invocation of the termination routine.

termstat

A 2-word integer value that contains the termination registration status that is returned. If the first word returned is zero, termination registration was successful. If the first word is not zero, registration failed and the first word of the status field contains the following information:

H1 nonzero

Status value is from REG4TERM$ call. See the *System Services Programming Reference Manual* for REG4TERM$ status codes and definitions.

Status value is from REG4TERM$ call.

H1 zero

H2 contains the status from the Linking System interface LS$RES_NAME (see the *Linking System Programming Reference Manual*).

Example

Note: *The following two programs are one compilation unit.*

```
IDENTIFICATION DIVISION.
PROGRAM-ID.
    ABC.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
SPECIAL-NAMES.
    PRINTER IS PRT.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
DATA DIVISION.
FILE SECTION.
WORKING-STORAGE SECTION.
01   name          PIC X(8) VALUE "TAKETERM".
01   namelen       PIC S9(10) USAGE BINARY VALUE 8.
01   termid        PIC S9(10) USAGE BINARY VALUE 2.
01   termdata      PIC S9(10) USAGE BINARY VALUE 123456.
01   termbuff.
     02   buff     PIC S9(10) USAGE BINARY OCCURS 11 TIMES VALUE 0.
  01   termstat.
     02   stat     PIC S9(10) USAGE BINARY OCCURS 2 TIMES VALUE 0.
  PROCEDURE DIVISION.
  BEGIN.
```

```
    CALL "REG4TERM-COB" USING name, namelen, termid, termdata,
                    termbuff, termstat.
    DISPLAY ' STATUS = '( stat(1)  stat(2) UPON PRT.
    STOP RUN.
END PROGRAM ABC.

IDENTIFICATION DIVISION.
PROGRAM-ID.
    TAKETERM.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
SPECIAL-NAMES.
    PRINTER IS PRT.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
DATA DIVISION.
FILE SECTION.
LINKAGE SECTION.
01  uid      PIC S9(10) USAGE BINARY.
01  udata    PIC S9(10) USAGE BINARY.
PROCEDURE DIVISION USING uid, udata.
BEGIN.
    DISPLAY " We made it to the UCOB termination subroutine"
                    UPON PRT.
    DISPLAY " uid = " uid UPON PRT.
    DISPLAY " udata = " udata UPON PRT.
    EXIT PROGRAM.
END PROGRAM TAKETERM.
```

The storage allocation for *uid* and *udata* (found in the LINKAGE SECTION) is in *termbuff*. These data items are mapped to *termbuff* by Unisys internal intercept code when the Exec calls the termination.

12.4. User Termination Handler Deregistration

The user termination handler deregistration interface deregisters a termination handler.

The extended mode Exec interface used is REG4TERM$.

Interface Name

```
DREG4TERM-COB
```

Interface Setup

```
:
01   termid PIC S9(10) USAGE BINARY.
01   termstat.
     02   stat PIC S9(10) USAGE BINARY OCCURS 2 TIMES.
:
     CALL "DREG4TERM-COB" USING termid, termstat.
```

where:

termid

The termination handler that is to be deregistered. It must be the same value passed to the termination registration interface registration routine.

termstat

A 2-word integer value that contains the termination deregistration status that is returned. If the first word is zero, the termination deregistration was successful. If the first word is not zero, the termination deregistration failed. See the *System Services Programming Reference Manual* for REG4TERM$ status codes and definitions.

12.5. UCS Runtime System Contingency Handling

UCS Runtime System contingency handling is described in the following subsections:

- Home Subsystem Operation
- Shared Subsystem Restrictions

12.5.1. Home Subsystem Operation

UCS COBOL has processing or language features that depend on the contingency handling provided by the UCS Runtime System in the home subsystem. If you register for and handle home subsystem contingencies, it can have an effect on the operation of the program. In addition to the special contingency processing in the following list, the UCS Runtime System displays a contingency message for all other contingencies that occur and then performs standard action (program execution is not resumed).

The following list shows the areas of processing that are dependent on special URTS contingency handling, which includes processing a contingency interrupt and then resuming program execution:

- Arithmetic faults with ON SIZE ERROR clause. The UCS Runtime System sets flag indicating arithmetic fault occurrence and resumes program execution.
- CALL IDENTIFIER with ON OVERFLOW clause (non-ZOOM case). The UCS Runtime System sets flag for generated code in the NOFIND case and resumes program execution.

12.5.2. Shared Subsystem Restrictions

The areas of processing and language features listed as being dependent upon the UCS Runtime System contingency processing in the home subsystem will not function similarly in a shared subsystem environment. The UCS Runtime System must capture contingencies in order to process them. This is not possible since the UCS Runtime System is registered for the home subsystem and the actual contingency occurs in the shared subsystem.

Section 13
Processing Tapes for Exchanging Information

The UCS COBOL Information Interchange Tape Processing module aids the porting of files to or from a system manufactured by another company. This module processes tapes that adhere to the X3.27-1969 Magnetic Tape Labels for Information Interchange. This standard is the preferred format for exchanging data between the systems of different computer manufacturers.

This section contains the Unisys definition of rules and restrictions for the processing of tapes that adhere to the Magnetic Tape Labels standard. These formats are in addition to, and in some cases, replacements for the formats, syntax, and rules for sequential I-O that are described in the *UCS COBOL Programming Reference Manual Volume 1*.

The information in this section applies to the following UCS COBOL divisions:

- Environment Division
- Data Division

This section uses the same organization as the sequential I-O section to help you determine where the Magnetic Tape Labels standard differs from the standard for COBOL85.

13.1. Environment Division—Input-Output Section

The Input-Output Section is part of the Environment Division of a source program. The Input-Output Section deals with the information the system needs to control transmission and handling of data between external media and the object program. The Input-Output Section is optional in the Environment Division of a COBOL source program.

13.1.1. The FILE-CONTROL Paragraph

Function

The FILE-CONTROL paragraph names each file and allows specification of other file-related information.

General Format

```
FILE-CONTROL .  { file-control-entry } . . .
```

13.1.2. The File Control Entry

Function

The file control entry names a file and can specify other file-related information.

General Format

```
SELECT [ OPTIONAL ] file-name-1
   ASSIGN [ TO ] implementor-name-1 [ , implementor-name-2 ] ...
   [ RESERVE integer-1 [ AREA | AREAS ] ]
   [ ORGANIZATION [ IS ] SEQUENTIAL ]
   [ PADDING [ CHARACTERIS ] { data-name-1 | literal-1 } ]
   [ RECORDDELIMITER [ IS ] STANDARD-1 ]
   [ ACCESS [ MODE IS ] SEQUENTIAL ]
   [ [ FILE ] STATUS [ IS ] data-name-1 ] .
```

Syntax Rules

1. The SELECT clause must be specified first in the file control entry. The clauses that follow the SELECT clause can appear in any order.
2. Each file name in the Data Division must be specified only once in the FILE-CONTROL paragraph. Each file name specified in the SELECT clause must have a file description entry in the Data Division of the same program.
3. The only allowable *implementor-name-1* is INTERCHANGE.

4. If *implementor-name-2* is specified, it is a 1- to 12-character name used to link the COBOL file to OS 2200 Executive File Control and should appear on an @ASG or @USE statement in the runstream for executing the object program.
5. When *implementor-name-2* is not specified, the first 12 characters of *file-name-1* are used to link the COBOL file to the OS 2200 Executive File Control.
6. All recursions of *implementor-name-2* are for documentation only.
7. If the ACCESS MODE clause is not specified, sequential access is assumed.
8. *Data-name-1* must be defined in the Data Division as a 2-character data item of the category alphanumeric and must not be defined in the File Section, the Report Section, or the Communication Section. *Data-name-1* must be defined as USAGE DISPLAY.
9. *Data-name-1* can be qualified.
10. When the ORGANIZATION IS SEQUENTIAL clause is not specified, sequential organization is implied.
11. The OPTIONAL phrase applies only to files opened in the input mode. Its specification is required for input files that are not necessarily present each time the object program is executed.

General Rules

1. The ASSIGN clause specifies the association of the file referred to by *file-name-1* with a storage medium.
2. If the RESERVE clause is specified, it is for documentation only. Two input-output areas are always allocated.
3. The ORGANIZATION clause specifies sequential organization as the logical structure of a file. The file organization is established at the time a file is created and cannot subsequently be changed.
4. Records in the file are accessed in the sequence dictated by the file organization. This sequence is specified by predecessor-successor record relationship established by the execution of WRITE statements when the file is created or extended.
5. If the FILE STATUS clause is specified, the data item referred to by *data-name-1* is always updated to contain the value of the I-O status following an I-O operation.

13.2. Data Division—File Section

In a COBOL program the File Description (FD) entry represents the highest level of organization in the File Section. The File Section header is followed by a file description entry consisting of a level indicator (FD), a file name, and a series of independent clauses. The FD clauses specify the size of the logical and physical records, the presence or absence of label records, the value of label items defined by Unisys, and the names of the data records that comprise the file. The entry itself is terminated by a period.

13.2.1. The RECORDING MODE Clause

Function

The RECORDING MODE clause is used to specify a tape format specified in X3.27-1978 Magnetic Tape Labels and File Structures for Information Interchange.

General Format

$$\text{RECORDING [MODE IS] } \left\{ \begin{matrix} \text{D} \\ \text{F} \\ \text{S} \\ \text{U} \end{matrix} \right\}$$

General Rules

The structure of the physical data blocks can be defined by one of the following:

1. The D phrase specifies variable-length blocked records.
2. The F phrase specifies fixed blocked records.
3. The S phrase specifies spanned blocked records.
4. The U phrase specifies variable-length unblocked records.

13.2.2. Record Description Restrictions

Data items in a record description for an INTERCHANGE file abide by the following restrictions:

- The usage for each data item must be DISPLAY.
- The description of each signed numeric data item must include the SIGN IS LEADING SEPARATE CHARACTER clause.

Section 14
Processing IBM Tapes

Unisys tape formats differ from those used by IBM. Thus, if you use IBM tapes to input or output data, you need to attain compatibility between the two formats by adapting your program to use the IBM Tape Processing Module as described in this section.

This section contains the rules and restrictions for processing IBM tapes. These formats are in addition to, and in some cases, replacements for the formats, syntax, and rules for sequential I-O that are described in the *UCS COBOL Programming Reference Manual Volume 1*. The information applies to the following UCS COBOL divisions:

- Environment Division
- Data Division
- Procedure Division

This section uses the same organization as the sequential I-O section in the *UCS COBOL Programming Reference Manual Volume 1* to help you determine the special requirements for processing IBM tapes.

14.1. Function

The IBM Tape Processing module can access records of an IBM-formatted tape file in an established sequence. The sequence is established as a result of writing the records to the file. It also provides for the specification of rerun points and the sharing of storage areas among files.

14.2. Language Concepts

The following subsections describe the language considerations that apply to processing IBM tapes.

14.2.1. Organization

Each record of a sequential file except the first has a unique predecessor record, and each record except the last has a unique successor record. These predecessor-successor relationships are established by the order of WRITE statements when the file is created. Once established, the predecessor-successor relationships do not change.

14.2.2. Access Mode

In the sequential-access mode, the sequence in which records are accessed is the order in which the records were originally written.

14.2.3. File Position Indicator

The file position indicator is a conceptual entity used to specify the next record to be accessed within a given file. The concept of the file position indicator has no meaning for a file opened in the output mode. The setting of the file position indicator is affected only by the OPEN and READ statements.

14.2.4. The At-End Condition

The at-end condition can occur as a result of the execution of a READ statement. For details of the causes of the condition, see 14.5.3.

14.2.5. Allocation Listing

The Allocation Listing gives information about variables and labels. Storage allocation of programs that use IBM tape processing is converted when records are read or written. The Allocation Listing reflects the internal allocations of the UCS COBOL compiler, not the IBM record allocation.

14.3. Environment Division—Input-Output Section

The Input-Output Section is part of the Environment Division of a source program. The Input-Output Section deals with the information the system needs to control transmission and handling of data between external media and the object program. The Input-Output Section is optional in the Environment Division of a COBOL source program.

14.3.1. The FILE-CONTROL Paragraph

Function

The FILE-CONTROL paragraph names each file and allows specification of other file-related information.

General Format

```
FILE-CONTROL .  { file-control-entry } ...
```

14.3.2. The File Control Entry

Function

The file control entry names a file and can specify other file-related information.

General Format

SELECT [OPTIONAL] $file-name$

ASSIGN [TO] $implementor-name-1$ [, $implementor-name-2$] ...

$$\left[\text{RESERVE}\ \left\{\begin{matrix}\text{integer}-1\\ \text{NO}\end{matrix}\right\}\ \text{ALTERNATE}\ \left[\begin{matrix}\text{AREA}\\ \text{AREAS}\end{matrix}\right]\right]$$

[ACCESS [MODE IS] SEQUENTIAL]

[ORGANIZATION [IS] SEQUENTIAL] .

Syntax Rules

1. The SELECT clause must be specified first in the file control entry. The clauses following the SELECT clause can appear in any order.
2. Each file described in the Data Division must be named only once as *file-name* in the FILE-CONTROL paragraph. Each file specified in the file control entry must have a file description entry in the Data Division.
3. The *implementor-name-1* is TAPE.
4. If the *implementor-name-2* is specified, it is a 1- to 12-character name used to link the COBOL file to OS 2200 system Exec file control and should appear on an @ASG or @USE statement in the runstream for executing the object program.
5. If the *implementor-name-2* is specified, it is a 1- to 12-character name used to link the COBOL file to system Exec file control and should appear on an @ASG or @USE statement in the runstream for executing the object program.
6. When *implementor-name-2* is not specified, the first 12 characters of the file name are used to link the COBOL file to system Exec file control.
7. All recursions of *implementor-name-1* and *implementor-name-2* are for documentation only.
8. If the ACCESS MODE clause is not specified, the ACCESS MODE IS SEQUENTIAL clause is implied.
9. When the ORGANIZATION IS SEQUENTIAL clause is not specified, the ORGANIZATION IS SEQUENTIAL clause is implied.
10. The OPTIONAL phrase can only be specified for input files. Its specification is required for input files that are not necessarily present each time the object program is executed.

General Rules

1. The ASSIGN clause specifies the association of the file referenced by file name to a storage medium.
2. The RESERVE clause allows the user to affect the number of input-output areas allocated. If the RESERVE clause is not specified or if *integer-1* is specified, two input-output areas are allocated. If NO is specified, one input-output area is allocated.
3. The ORGANIZATION clause specifies the logical structure of a file. The file organization is established at the time a file is created and cannot subsequently be changed.
4. Records in the file are accessed in the sequence dictated by the file organization. This sequence is specified by predecessor-successor record relationships established by the execution of WRITE statements when the file is created.

14.3.3. The I-O-CONTROL Paragraph

Function

The I-O-CONTROL paragraph specifies the points at which rerun is to be established and the storage area that is to be shared by different files.

General Format

```
I-O-CONTROL .
  [
     RERUN ON implementor-name [ EVERY ]     { [ END OF ] REEL       }
                                             { integer-1 RECORDS     }

                   [ OF ] file-name-1 ...  ]

  [ SAME [ RECORD ] [ AREA FOR ] { file-name-2 } ...]...
```

Syntax Rules

1. The I-O-CONTROL paragraph is optional.
2. *File-name-1* can be either an input or output file.
3. The allowable *implementor-names* are TAPE and DISC.
4. More than one RERUN clause can be specified for a given *file-name-1,* subject to the following restrictions:
   a. When multiple *integer-1* RECORDS clauses are specified, no two of them can specify the same *file-name-1.*
   b. When multiple END OF REEL clauses are specified, no two of them can specify the same *file-name-1.*

5. The two forms of the SAME clause (SAME AREA, SAME RECORD AREA) are considered separately in the following:

   More than one SAME clause can be included in a program. However:

   a. A file name must not appear in more than one SAME AREA clause.

   b. A file name must not appear in more than one SAME RECORD AREA clause.

   c. If one or more file names of a SAME AREA clause appear in a SAME RECORD AREA clause, all of the file names in that SAME AREA clause must appear in that SAME RECORD AREA clause. However, additional file names not appearing in that SAME AREA clause can also appear in that SAME RECORD AREA clause. The rule that only one of the files mentioned in a SAME AREA clause can be open at any given time takes precedence over the rule that all files mentioned in a SAME RECORD AREA clause can be open at any given time.

6. The files referenced in the SAME AREA or SAME RECORD AREA clause need not all have the same organization or access.

General Rules

1. The RERUN clause specifies when and where the rerun information is recorded.
2. Rerun information is written in a separate file whose file name is RERUN$. The two forms of the RERUN clause are based on the following conditions under which rerun points can be established:

   a. When the END OF REEL clause is used. In this case, the rerun information is written whenever the END OF REEL condition occurs on *file-name-1.*

   b. When the *integer-1* RECORDS is used. In this case, the rerun information is written whenever *integer-1* records of *file-name-1* have been processed.

3. The SAME AREA clause specifies that two or more files that do not represent sort or merge files are to use the same main storage area during processing. The area being shared includes all storage areas assigned to the files specified; therefore, it is not valid to have more than one of the files open at the same time. (See the *UCS COBOL Programming Reference Manual Volume 1* for more information.)
4. The SAME RECORD AREA clause specifies that two or more files are to use the same storage area for processing the current logical record. All of the files can be open at the same time. A logical record in the SAME RECORD AREA is considered a logical record of each opened output file whose file name appears in this SAME RECORD AREA clause and of the most recently read input file whose file name appears in this SAME RECORD AREA clause. This is equivalent to an implicit redefinition of the area; that is, records are aligned on the leftmost character position.

14.4. Data Division—File Section

In a COBOL program, the File Description (FD) entry represents the highest level of organization in the File Section. The File Section header is followed by a file description entry consisting of a level indicator (FD), a file name, and a series of independent clauses. The FD clauses specify the size of the logical and physical records, the presence or absence of label records, the value of label items defined by Unisys, and the names of the data records that comprise the file. The entry itself is terminated by a period.

14.4.1. The File Description—Complete Entry Skeleton

Function

The file description furnishes information concerning the physical structure, identification, and record names pertaining to a given file.

General Format

FD $file-name$

$$\left[\text{ BLOCK [CONTAINS][}integer-1\text{ TO] }integer-2\ \left\{\begin{matrix}\text{RECORDS}\\\text{CHARACTERS}\end{matrix}\right\}\right]$$

$$\left[\underline{\text{RECORD}}\ \left\{\begin{matrix}\text{CONTAINS }integer-3\text{ CHARACTERS}\\\text{IS }\underline{\text{VARYING}}\text{ IN SIZE [[FROM }integer-4\text{][TO }integer-5\text{]CHARATCERS]}\\\left[\ \underline{\text{DEPENDING}}\text{ ON }data-name-1\ \right]\\\text{CONTAINS }integer-6\text{ TO }integer-7\text{ CHARACTERS}\end{matrix}\right\}\right]$$

$$\text{LABEL}\ \left\{\begin{matrix}\text{RECORD [IS]}\\\text{RECORD [ARE]}\end{matrix}\right\}\ \left\{\begin{matrix}\text{STANDARD}\\\text{OMITTED}\end{matrix}\right\}$$

$$\left[\text{VALUE OF}\ \left\{inplementor-name-1\text{ [IS] }\left\{\begin{matrix}data-name-1\\literal-1\end{matrix}\right\}\right\}\ldots.\right]$$

$$\left[\text{ DATA}\ \left\{\begin{matrix}\text{RECORD [IS]}\\\text{RECORD [ARE]}\end{matrix}\right\}\ \left\{data-name-2\right\}\ldots.\right]$$

$$\text{RECORDING [MODE IS]}\ \left\{\begin{matrix}\text{F}\\\text{U}\\\text{V}\end{matrix}\right\}\text{ [AN] .}$$

Syntax Rules

1. The level indicator FD identifies the beginning of a file description and must precede *file-name.*
2. The clauses that follow the name of the file are optional in many cases, and their order of appearance is immaterial.
3. One or more record description entries must follow the file description entry.

14.4.2. The BLOCK CONTAINS Clause

Function

The BLOCK CONTAINS clause specifies the size of a physical record.

General Format

```
BLOCK [CONTAINS ] [ integer-1 TO ] integer-2 { RECORDS    }
                                             { CHARACTERS }
```

General Rules

1. This clause is required except when:
   a. A physical record contains only one complete logical record.
   b. RECORDING MODE U is specified.
2. The size of the physical record can be stated in terms of RECORDS, unless one of the following situations exists, in which case the RECORDS phrase must not be used.
   a. The physical record contains padding (area not contained in a logical record).
   b. Logical records are grouped in such a manner that an inaccurate physical record size would be implied.
3. When the word CHARACTERS is specified, the physical record size is specified in terms of the number of ASCII character positions required to store the physical record, regardless of the types of characters used to represent the items within the physical record.
4. If only *integer-2* is shown, it represents the exact size of the physical record.

   If *integer-1* and *integer-2* are both shown, they refer to the minimum and maximum size of the physical record.

14.4.3. The DATA RECORDS Clause

Function

The DATA RECORDS clause serves only as documentation for the names of data records with their associated file.

General Format

```
DATA { RECORD [ IS ]  } { data-name-1 } ...
     { RECORDS [ARE ] }
```

Syntax Rule

Data-name-1 represents the names of the data records and must have 01-level-number record descriptions, with the same names associated with them.

General Rules

1. The presence of more than one *data-name* indicates that the file contains more than one type of data record. These records can be of different sizes, different formats, and so forth. The order in which they are listed is not significant.
2. Conceptually, all data records within a file share the same area. This is in no way altered by the presence of more than one type of data record within the file.

14.4.4. The LABEL RECORDS Clause

Function

The LABEL RECORDS clause specifies whether labels are present.

General Format

```
LABEL [ RECORD [ IS ] ] { STANDARD }
      [RECORDS [ ARE ]] { OMITTED  }
```

Syntax Rule

This clause is required when STANDARD is used.

General Rules

1. OMITTED specifies that no explicit labels exist for the file or the device to which the file is assigned.
2. STANDARD specifies that EBCDIC labels exist for the file or the device to which the file is assigned. (See 5.2.1)

14.4.5. The RECORD Clause

Function

The RECORD clause specifies the number of character positions in a fixed-length record, or specifies the range of character positions in a variable-length record. If the number of character positions does vary, the clause specifies the minimum and maximum number of character positions.

Format 1

```
RECORD CONTAINS integer-1 CHARACTERS
```

Format 2

```
RECORD IS VARYING IN SIZE [ [FROM integer-2][TO integer-3] CHARACTERS]

    [DEPENDING ON data-name-1]
```

Format 3

```
RECORD CONTAINS integer-4 TO integer-5 CHARACTERS
```

Syntax Rules

The following is a list of the syntax rules.

Format 1

The following syntax rule applies to format 1:

1. Format 1 is used to specify fixed length records; no other record description entry for the file can specify a number of character positions other than *integer-1.*

Format 2

The following syntax rules applies to format 2:

1. Record descriptions for the file must not describe records that contain a lesser number of character positions than that specified by *integer-2,* nor records that contain a greater number of character positions than that specified by *integer-3.*
2. *Integer-3* must be greater than *integer-2.*
3. *Data-name-1* must describe an elementary unsigned integer in the Working-Storage or Linkage Section.

General Rules

The following is a list of general rules.

All Formats

The following general rules apply to all formats:

1. If the RECORD clause is not specified, the size of each data record is completely defined in the record description entry.
2. If the associated file connector is an external file connector, all file description entries in the run unit that are associated with that file connector must specify the same values for *integer-1* or *integer-2* and *integer-3.* If the RECORD clause is not specified, all record description entries associated with this file connector must be the same length.

Format 2

The following general rules apply to format 2:

1. Format 2 is used to specify variable-length records. Integer-2 specifies the minimum number of character positions to be contained in any record of the file.

Integer-3 specifies the maximum number of character positions in any record of the file.

2. The number of character positions associated with a record description is determined by the sum of the number of character positions in all elementary data items excluding redefinitions and renamings, plus any implicit FILLER due to synchronization. If a table is specified
    a. The minimum number of table elements described in the record is used in the previous summation to determine the minimum number of character positions associated with the record description.
    b. The maximum number of table elements described in the record is used in the previous summation to determine the maximum number of character positions associated with the record description.
3. If *integer-2* is not specified, the minimum number of character positions to be contained in any record of the file is equal to the least number of character positions described for a record in that file.
4. If *integer-3* is not specified, the maximum number of character positions to be contained in any record of the file is equal to the greatest number of character positions described for a record in that file.
5. If *data-name-1* is specified, the results of a READ or WRITE are unpredictable, unless the AN option is used on the RECORDING MODE clause.
6. If *data-name-1* is specified, the number of character positions in the record must be placed into the data item referred to by *data-name-1* before any WRITE statement is executed for the file.
7. If *data-name-1* is specified, the execution of a WRITE statement or the unsuccessful execution of a READ statement does not alter the contents of the data item referred to by *data-name-1.*
8. During the execution of a WRITE statement, the number of character positions in the record is determined by the following conditions:
    a. If *data-name-1* is specified, by the content of the data item referred to by *data-name-1.*
    b. If *data-name-1* is not specified and the record does not contain a variable-occurrence data item, by the number of character positions in the record.
    c. If *data-name-1* is not specified and the record does contain a variable-occurrence data item, by the sum of the fixed portion and that portion of the table described by the number of occurrences at the time of execution of the output statement.
9. If *data-name-1* is specified, after the successful execution of a READ statement for the file, the contents of the data item referred to by *data-name-1* indicate the number of character positions in the record just read.
10. If the INTO phrase is specified in the READ statement, the number of character positions in the current record that participate as the sending data items in the implicit MOVE statement is determined by the following conditions:

a. If *data-name-1* is specified, by the content of the data item referred to by *data-name-1.*

b. If *data-name-1* is not specified, by the value that would have been moved into the data item referred to by *data-name-1* had *data-name-1* been specified.

Format 3

The following general rules apply to format 3:

1. When format 3 of the RECORD clause is used, *integer-4* and *integer-5* refer to the minimum number of characters in the smallest size data record and the maximum number of characters in the largest size data record, respectively. However, in this case, the size of each data record is completely defined in the record description entry.
2. The size of each data record is specified in terms of the number of character positions required to store the logical record, regardless of the types of characters used to represent the items within the logical record. The size of a record is determined by the sum of the number of characters in all fixed-length elementary items plus the sum of the maximum number of characters in any variable-length item subordinate to the record. This sum can be different from the actual size of the record. (See 3.5.15 and 3.5.16.)

14.4.6. The RECORDING MODE Clause

Function

The RECORDING MODE clause is required to specify IBM OS/360 9-track tape compatibility.

General Format

$$\text{RECORDING [MODE IS] } \left\{ \begin{matrix} \text{F} \\ \text{U} \\ \text{V} \end{matrix} \right\} \text{ [AN]}$$

General Rules

1. The structure of the physical data blocks can be defined by one of the following forms:
   a. The F phrase specifies fixed blocked records.
   b. The U phrase specifies variable-length unblocked records.
   c. The V phrase specifies variable-length blocked records.
2. When the AN option is specified, the records are assumed to consist entirely of ASCII characters internally and of EBCDIC characters on magnetic tape.
3. When the AN option is not specified, the following data conversions are performed:
   a. On input, all DISPLAY data items are converted from EBCDIC to ASCII.
   b. On output, all DISPLAY data items are converted from ASCII to EBCDIC.

c. When a record is read from tape, leading blanks are converted to leading zeros in every numeric DISPLAY field.

d. Signed numeric DISPLAY fields are written to tape with an overpunched sign on the low-order digit.

e. On input, all COMPUTATIONAL and PACKED-DECIMAL data items are converted from IBM OS/360 format to UCS COBOL internal format.

f. On output, all COMPUTATIONAL and PACKED-DECIMAL data items are converted from UCS COBOL internal format to IBM OS/360 format.

g. Records written to tape are adjusted such that slack bytes are inserted where necessary to ensure proper IBM OS/360 alignment.

h. For RECORDING MODES U and V files, format 2 of the OCCURS clause is allowed in the record description and the following processing is performed:

   1. A data item whose size is variable is expanded on input using the value of the data item referenced by *data-name-1* of the OCCURS clause.

   2. A data item whose size is variable is compressed on output using the value of the data item referenced by *data-name-1* of the OCCURS clause.

i. If a REDEFINES clause is used within a record description, then the original definition is used rather than the redefinition for the purposes of record conversion.

4. If AN is not specified and the file contains multiple record descriptions, then a record selector field within each record description is required. (See 14.4.9)

14.4.7. The VALUE OF Clause

Function

The VALUE OF clause details the description of an item in the label records associated with a file.

General Format

$$\text{VALUE OF } \left\{ impelentor-name-1 \text{ [IS] } \left\{ \begin{matrix} data-name-1 \\ literal-1 \end{matrix} \right\} \right\} \ldots\ldots$$

Syntax Rules

1. The allowable implementor names are FILE ACCESS, FILE-ID, and PURGE-DATE. These implementor names have the implied definition of:

FILE-ACCESS	PIC X.
FILE-ID	PIC X(17).
PURGE-DATE	PIC 9(6)

2. *Data-name-1* should be qualified when necessary, but may not be subscripted or indexed. It may not be an item described with the USAGE IS INDEX clause.

3. *Data-name-1* must be in the Working-Storage Section.

4. *Data-name-1* must be described as USAGE DISPLAY.

General Rules

1. For an input file, the appropriate label routine checks to see if the value of *implementor-name-1* is equal to the value of *literal-1* or of *data-name-1,* whichever has been specified. (See 14.5.2)

   For an output file, at the appropriate time, the value of *implementor-name-1* is made equal to the value of *literal-1* or of a *data-name-1,* whichever has been specified. (See 14.5.2)

2. A figurative constant can be substituted in the format wherever a *literal-1* is specified.

14.4.8. The Record Description—Complete Entry Skeleton

A record description consists of a set of data description entries describing the characteristics of a particular record. Each data description entry consists of a level number followed by a data name, if required, followed by a series of independent clauses as required. A record description has a hierarchical structure and therefore the clauses used with an entry can vary considerably, depending on whether or not they are followed by subordinate entries. The structure of a record description is defined in the *UCS COBOL Programming Reference Manual Volume 1*.

Function

A data description entry specifies the characteristics of a particular item of data.

General Format 1

$$level-number \left\{ \begin{matrix} data-name-1 \\ \text{FILLER} \end{matrix} \right\} \ [\ ; \text{REDEFINES}\ data-name-2\ ]$$

$$\left[\left\{ \begin{matrix} \text{PICTURE} \\ \text{PIC} \end{matrix} \right\} [\ \text{IS}\ ]\ character-string \right]$$

$$\left[\left[\text{USAGE}\ [\ \text{IS}\ ] \left\{ \begin{matrix} \text{BINARY} \\ \text{COMPUTATIONAL} \\ \text{COMP} \\ \text{COMP}-1 \\ \text{COMPUTATIONAL}-2 \\ \text{COMP}-2 \\ \text{COMPUTATIONAL}-4 \\ \text{COMP}-4 \\ \text{DISPLAY} \\ \text{DISP} \\ \text{PACKED}-\text{DECIMAL} \end{matrix} \right\} \right] \right]$$

$$\left[\left\{ \begin{matrix} \text{SYNCHRONIZED} \\ \text{SYNC} \end{matrix} \right\} [\ \text{LEFT} \\ \text{RIGHT}\ ] \right]$$

```
[ {JUSTIFIED} [ RIGHT ] ]
[ {JUST     }           ]

[ BLANK [ WHEN ] ZERO ]

[ {  VALUE [ IS ] } { literal-1 [ {THROUGH} literal-2 ] } .... ]
[ {VALUES [ ARE ] }               {THRU   }               ]

[ RANK IS integer-1 ]
```

General Format 2

```
66 data-name-1 RENAMES data-name-2  [ { THROUGH } data-name-3 ] .
                                    { THRU    }
```

General Format 3

```
88 condition-name { VALUE  [ IS ] } { literal-1 [ { THROUGH } literal-2 ] } ... .
                  { VALUES [ ARE ] }             { THRU    }
```

Syntax Rules

1. The clauses can be written in any order, with two exceptions. The *data-name*-1 or FILLER clause must immediately follow the *level-number*. The REDEFINES clause, when used, must immediately follow the *data-name-1* clause.
2. The PICTURE clause must be specified for every elementary item except floating-point data items, in which case use of this clause is prohibited.
3. The words THRU and THROUGH are equivalent.

General Rules

Format 1

1. A PICTURE *character-string* containing one or more 1s is not allowed.
2. The RANK, SYNCHRONIZED, USAGE, and VALUE (in the File Section) clauses are described in the remainder of this subsection.

Formats 2 and 3

See the *UCS COBOL Programming Reference Manual Volume 1.*

The RANK Clause

Function

The RANK clause increases the efficiency of the process of record selection.

General Format

```
RANK IS integer-1
```

Syntax Rules

1. The RANK clause is optional.
2. The RANK clause can be used only in the description of a record selector. Record selectors are described in 14.4.9.
3. *Integer-1* is an unsigned integer in the range 1 through 63.

General Rules

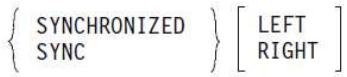
1. When the RANK clause is omitted, a value of 63 is assumed for the record selector.
2. When two or more record selectors in the same record description have the same RANK, the textual order determines the order in which the selectors are tested.
3. When a record description contains multiple record selectors, the value of the RANK assigned to each record selector need not be consecutive.
4. When a record description contains more than one record selector, the order in which the record selectors are tested is determined by the RANK of each selector. Record selectors of rank *n* are tested before any record selector of rank *n* + 1; for example, RANK 1 has a higher precedence than a record selector of RANK 2.
5. When a record description contains only one record selector, the RANK clause is permitted but it has no meaning.

The SYNCHRONIZED Clause

Function

The SYNCHRONIZED clause specifies the alignment of an elementary item on the natural boundaries of the computer main storage when IBM OS/360 allocation and alignment rules apply.

General Format



Syntax Rules

1. This clause can appear only with an elementary item.
2. SYNC is an abbreviation for SYNCHRONIZED.

General Rules

1. A full description of data allocation and alignment can be found in the *UCS COBOL Programming Reference Manual Volume 1.*
2. This clause specifies that the subject data item is to be aligned in the computer such that no other data item occupies any of the character positions between the leftmost and rightmost natural boundaries delimiting this data item. If the number of character positions required to store this data item is less than the number of character positions between those natural boundaries, the unused character

positions (or portions thereof) must not be used for any other data item. Such unused character positions, however, are included in:

a. The size of any group items to which the elementary item belongs.

b. The character positions redefined when this data item is the object of a REDEFINES clause.

3. The LEFT and RIGHT phrases serve for documentation only.

4. Whenever a SYNCHRONIZED item is referenced in the source program, the original size of the item, as shown in the PICTURE clause, is used in determining any action that depends on size, such as justification, truncation, or overflow.

5. If the data description of an item contains the SYNCHRONIZED clause and an operational sign, the sign of the item appears in the normal operational sign position, regardless of whether the item is SYNCHRONIZED LEFT or SYNCHRONIZED RIGHT.

6. When the SYNCHRONIZED clause is specified in a data description entry of a data item that also contains an OCCURS clause or in a data description entry of a data item subordinate to a data description entry that contains an OCCURS clause, then:

   a. Each occurrence of the data item is SYNCHRONIZED.

   b. Any implicit FILLER generated for other data items within that same table is generated for each occurrence of those data items.

7. The elementary items associated with this clause are handled as follows:

   Any necessary generation of implicit FILLER, if the elementary item immediately preceding an item containing the SYNCHRONIZED clause does not terminate at a word boundary, is not included as part of the elementary item. Such automatically generated FILLER positions are included in:

   a. The size of any group to which the FILLER item belongs.

   b. The number of character positions allocated when the group item of which the FILLER item is a part appears as the object of a REDEFINES clause.

8. The presence of the SYNCHRONIZED clause has no effect on the alignment of data items.

The USAGE Clause

Function

The USAGE clause specifies the format of a data item in the computer storage.

General Format

$$[\ \text{USAGE}\ [\ \text{IS}\ ]\ ]\ \left\{\begin{array}{c}\text{BINARY}\\ \text{COMPUTATIONAL}\\ \text{COMP}\\ \text{COMP}-1\\ \text{COMPUTATIONAL}-2\\ \text{COMP}-2\\ \text{COMPUTATIONAL}-4\\ \text{COMP}-4\\ \text{DISPLAY}\\ \text{DISP}\\ \text{PACKED}-\text{DECIMAL}\end{array}\right\}$$

Syntax Rules

1. The PICTURE *character-string* of a COMPUTATIONAL, and PACKED-DECIMAL item can contain only 9s, the operational sign character S, the implied decimal point character V, and one or more Ps.
2. COMP, COMP-*n*, and DISP are abbreviations for COMPUTATIONAL, COMPUTATIONAL-*n,* and DISPLAY, respectively.
3. COMPUTATIONAL-1 and COMPUTATIONAL-2 usage specifications cannot appear in a data description that has a PICTURE clause, because the computer storage format of these items is fixed.

General Rules

1. A full description of data allocation and alignment can be found in the *UCS COBOL Programming Reference Manual Volume 1*.
2. The USAGE clause can be written at any level. If the USAGE clause is written at a group level, it applies to each group or elementary item in the group. The USAGE clause of an elementary item cannot contradict the USAGE clause of a group to which the item belongs.
3. If the group usage is DISPLAY or COMPUTATIONAL, the group is considered to be an alphanumeric string of ASCII characters.
4. If the USAGE clause is not specified on a group item, the group is considered to be an alphanumeric string of ASCII characters. If the USAGE clause is not specified for an elementary item or for any group to which the item belongs, the usage is implicitly DISPLAY.
5. This clause specifies the manner in which a data item is represented in the storage of a computer. It does not affect the use of the data item, although the specifications for some statements in the Procedure Division can restrict the USAGE clause of the operands referred to. The USAGE clause can affect the radix or type of character representation of the item.

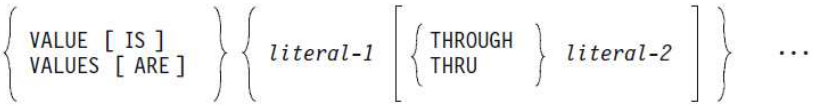
6. A COMPUTATIONAL or COMPUTATIONAL-*n* item is capable of representing a value to be used in computations and must be numeric. If a group item is described as COMPUTATIONAL or COMPUTATIONAL-*n,* the elementary items in the group are COMPUTATIONAL or COMPUTATIONAL-*n.* The group item itself is not COMPUTATIONAL or COMPUTATIONAL-*n* (cannot be used in computations).
7. The USAGE IS DISPLAY clause indicates that the format of the data is a standard data format.
8. The USAGE IS PACKED-DECIMAL clause specifies that the data item is represented in the storage of the computer with a radix of 10. PACKED-DECIMAL usage is equivalent to ASCII COBOL COMPUTATIONAL-3 usage.

The VALUE Clause

Function

The VALUE clause is used to specify a record selector. (See 14.4.9)

General Format



Syntax Rules

1. The words THRU and THROUGH are equivalent.
2. The VALUE clause cannot be stated for any item whose size is variable.
3. A signed numeric literal must be associated with a signed numeric PICTURE *character-string.*
4. All numeric literals in a VALUE clause of an item must have a value that is within the range of values indicated by the PICTURE clause, and not have a value that would require truncation of nonzero digits. Nonnumeric literals in a VALUE clause of an item must not exceed the size indicated by the PICTURE clause.

General Rules

1. The VALUE clause must not conflict with other clauses in the data description of the item or in the data description within the hierarchy of the item. The following rules apply:
   a. If the category of the item is numeric, all literals in the VALUE clause must be numeric. The literal is aligned according to the standard alignment rules.
   b. If the category of the item is alphabetic, alphanumeric, alphanumeric edited, or numeric edited, all literals in the VALUE clause must be nonnumeric literals. The literal is aligned as if the data item had been described as alphanumeric. If the THRU phrase is used, the native EBCDIC collating sequence is used for record selection comparison.

2. A figurative constant can be substituted wherever a literal is specified.
3. Wherever the THRU phrase is used, *literal-1* must be less than *literal-2*.
4. The VALUE clause must not be stated in a data description entry that contains an OCCURS clause or in an entry that is subordinate to an entry containing an OCCURS clause.
5. The VALUE clause must not be stated in a data description entry that contains a REDEFINES clause or in an entry that is subordinate to an entry containing a REDEFINES clause.
6. If the VALUE clause is used in an entry at the group level, the literal must be a figurative constant or a nonnumeric literal, and the group area is initialized without consideration for the individual elementary or group items contained within this group. The VALUE clause cannot be stated at the subordinate levels within this group.
7. The VALUE clause must not be written for a group containing items with descriptions including JUSTIFIED, SYNCHRONIZED, or USAGE (other than USAGE IS DISPLAY).

14.4.9. Record Selector Fields

A record selector field is identified to the compiler by the appearance of a VALUE clause in the record description. (See the VALUE clause in 14.4.8.)

When the file is read, the record selectors are tested to determine which record conversion must be performed. The record selectors in the first record are tested; if any of the selectors fail, the next record is considered. If all the record selectors match, the first record description is chosen. This process is repeated for all the data descriptions until a record description is encountered, all of whose selectors match. If no record description can be chosen, a diagnostic is issued.

In any given record description, the record selectors are listed in their textual order. This ordering can be changed by using the RANK clause. (See the RANK clause in 14.4.8.)

The RANK clause does not affect the record description chosen; its purpose is to increase the efficiency of the record selection process.

The number of record selectors in any single record description must not exceed 63. Each record description in a given file description need not contain the same number of selectors. In addition, the selectors need not be in the same position (internal or external) in each record description, and they need not have the same USAGE or PICTURE. However, it is the user's responsibility to ensure that this does not lead to inadvertent record selection.

14.5. Procedure Division

The Procedure Division contains declarative procedures when the USE statement from the Sequential I-O module is present in a COBOL source program.

14.5.1. The CLOSE Statement

Function

The CLOSE statement terminates the processing of reels/units and files with optional rewind and/or lock where applicable.

General Format

```
CLOSE file-name-1  [ { REEL } [ [ WITH ] NO REWIND ]                    ]
                   [ { UNIT }                                           ]
                   [                                                    ]
                   [ [ WITH ]  { NO REWIND }                            ]
                   [           { LOCK      }                            ]

[                  [ { REEL } [ [ WITH ] NO REWIND ]                    ]  ]
[ , file-name-2    [ { UNIT }                                           ]  ] ...
[                  [                                                    ]  ]
[                  [ [ WITH ]  { NO REWIND }                            ]  ]
[                  [           { LOCK      }                            ]  ]
```

Syntax Rule

The files referenced in the CLOSE statement need not all have the same organization or access.

General Rules

Except where otherwise stated in the general rules that follow, the terms “reel” and “unit” are synonymous and completely interchangeable in the CLOSE statement.

1. A CLOSE statement can be executed only for a file in an open mode.
2. For the purpose of showing the effect of various types of CLOSE statements as applied to various storage media, all files are divided into the following categories:
   a. Sequential single-reel (a sequential file that is entirely contained on one reel).
   b. Sequential multi-reel (a sequential file that is contained on more than one reel).
3. The results of executing each type of CLOSE for each category of file are summarized in Table 14–1.

Table 14–1. CLOSE Statement File Categories and Formats

CLOSE Statement Format	File Category	
	Sequential Single-Reel	Sequential Multi-Reel
CLOSE	F	C,F,A
CLOSE WITH LOCK	F,D	C,F,E,A
CLOSE WITH NO REWIND	C,B	C,B,A
CLOSE REEL/UNIT	X	E,F
CLOSE REEL/UNIT WITH NO REWIND	X	E

The definitions of the symbols in the table follow. Where the definition depends on whether the file is an input or output file, alternate definitions are given; otherwise, a definition applies to input and output files.

A

Previous Reels (Units) Unaffected

Input Files:

All reels in the file prior to the current reel are processed according to the standard reel swap procedure, except those reels controlled by a prior CLOSE REEL/UNIT statement. If the current reel is not the last in the file, the reels in the file following the current one are not processed.

Output Files:

All reels in the file prior to the current reel are processed according to the standard reel swap procedure, except those reels controlled by a prior CLOSE REEL/UNIT statement.

B

No Rewind of Current Reel (Unit)

The current reel is left in its current position.

C

Close File

Input Files:

If the file is positioned at its end and label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If the file is positioned at its end and label records are not specified for the file, label processing does not take place, but other closing operations are executed. If the file is positioned other than at its end, the closing operations are executed, but there is no ending label processing.

Output Files:

If label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If label records are not specified for the file, label processing does not take place, but other closing operations are executed.

D

File Lock

This file cannot be opened again during this execution of this run unit.

E

Close Reel (Unit)

Input Files:

The following operations take place:

1. A reel swap.
2. The standard beginning reel label procedure is executed.

   The next executed READ statement for that file makes available the next data record on the new reel.

Output Files:

The following operations take place:

1. The standard ending reel label procedure is executed.
2. A reel is swapped.
3. The standard beginning reel label procedure is executed.

The next executed WRITE statement that references that file directs the next logical data record to the next reel of the file.

F

Rewind

The current reel is positioned at its physical beginning.

X

Illegal

This is an illegal combination of a CLOSE option and a file category. The results at object time are undefined.

4. The action taken if the file is in the open mode when a STOP RUN statement is executed or the action taken for a file that has been opened in a called program and not closed in that program prior to the execution of a CANCEL statement for that program are undefined. No implicit CLOSE procedures will be initiated. This could cause the file to be unusable on subsequent accesses to it.

5. If the OPTIONAL phrase has been specified for the file in the FILE-CONTROL paragraph of the Environment Division and the file is not present, the standard end-of-file processing is not performed for that file.
6. If a CLOSE statement without the REEL or UNIT phrase has been executed for a file, no other statement (except the SORT or MERGE statements with the USING or GIVING phrases) can be executed that references that file, either explicitly or implicitly, unless an intervening OPEN statement for that file is executed.
7. Following the successful execution of a CLOSE statement without the REEL or UNIT phrase, the record area associated with the file name is no longer available. The unsuccessful execution of such a CLOSE statement leaves the availability of the record area undefined.

14.5.2. The OPEN Statement

Function

The OPEN statement initiates the processing of files. It also performs checking and writing of labels and other input-output operations.

General Format

```
OPEN { INPUT file-name-1   [ [ WITH ] NO REWIND ] [ file-name-2 [ [ WITH ]NO REWIND ]] ... } ...
     { OUTPUT file-name-3  [ [ WITH ] NO REWIND ] [ file-name-4 [ [ WITH ]NO REWIND ]] ... }
```

Rule

The files referenced in the OPEN statement need not all have the same organization or access.

General Rules

1. The successful execution of an OPEN statement determines the availability of the file and results in the file being in an open mode.
2. The successful execution of an OPEN statement makes the associated record area available to the program.
3. Prior to the successful execution of an OPEN statement for a given file, no statement (except for a SORT or MERGE statement with the USING or GIVING phrases) can be executed that references that file, either explicitly or implicitly.
4. An OPEN statement must be successfully executed prior to the execution of any of the permissible input-output statements. In Table 14–2, an X at an intersection indicates that the specified statement, used in the sequential access mode, can be used with the sequential file organization and open mode given at the top of the column.

Table 14–2. Permissible Statements

Statement	Open Mode	
	Input	Output
READ	X	
WRITE		X

5. A file can be opened with the INPUT and OUTPUT phrases in the same program. Following the initial execution of an OPEN statement for a file, each subsequent OPEN statement execution for that same file must be preceded by the execution of a CLOSE statement, without the REEL, UNIT, or LOCK phrase, for that file.
6. Execution of the OPEN statement does not obtain or release the first data record.
7. If label records are specified for the file, the beginning labels are processed as follows:
   a. When the INPUT phrase is specified, the execution of the OPEN statement causes the labels to be checked in accordance with the conventions for input label checking.
   b. When the OUTPUT phrase is specified, the execution of the OPEN statement causes the labels to be written in accordance with the conventions for output label writing.

   The behavior of the OPEN statement when label records are specified but not present, or when label records are not specified but are present, is undefined.
8. The file description entry for *file-name-1* or *file-name-2* must be equivalent to that used when this file was created.
9. If an input file is designated with the OPTIONAL phrase in its SELECT clause, the object program causes an interrogation for the presence or absence of this file. If the file is not present, the first READ statement for this file causes the at-end condition to occur. (See 14.5.3.)
10. The NO REWIND phrase can be used only with sequential single reel files. (See 14.5.1.)
11. Since the storage medium for the file permits rewinding, the following rules apply:
    a. When the NO REWIND phrase is not specified, execution of the OPEN statement causes the file to be positioned at its beginning.
    b. When the NO REWIND phrase is specified, execution of the OPEN statement does not cause the file to be repositioned; the file must be already positioned at its beginning prior to execution of the OPEN statement.
12. For files being opened with the INPUT phrase, the OPEN statement sets the file position indicator to the first record currently existing within the file. If no record exists in the file, the file position indicator is set such that the next executed READ statement for the file will result in an at-end condition.

13. Upon successful execution of an OPEN statement with the OUTPUT phrase specified, a file is created. At that time, the associated file contains no data records.

14.5.3. The READ Statement

Function

The READ statement makes available the next logical record from a file.

General Format

```
READ file-name [ RECORD ] [ INTO identifier ] [ AT ] END imperative-statement
```

Syntax Rule

The INTO phrase must not be used when the input file contains logical records of various sizes, as indicated by their record descriptions. The storage area associated with identifier and the record area associated with *file-name* must not be the same storage area.

General Rules

1. The associated file must be open in the INPUT mode at the time this statement is executed. (See 14.5.2)
2. The record to be made available by the READ statement is determined as follows:
   a. If the file position indicator was positioned by the execution of the OPEN statement, the record pointed to by the file position indicator is made available.
   b. If the file position indicator was positioned by the execution of a previous READ statement, the file position indicator is updated to point to the next existing record in the file and then that record is made available.
3. Regardless of the method used to overlap access time with processing time, the concept of the READ statement is unchanged in that a record is available to the object program prior to the execution of any statement following the READ statement.
4. When the logical records of a file are described with more than one record description, these records share the same storage area. This is equivalent to an implicit redefinition of the area. The contents of any data items that lie beyond the range of the current data record are undefined at the completion of the execution of the READ statement.
5. If the INTO phrase is specified, the record being read is moved from the record area to the area specified by identifier according to the rules specified for the MOVE statement without the CORRESPONDING phrase. The implied MOVE does not occur if the execution of the READ statement was unsuccessful. Any subscripting or indexing associated with the identifier is evaluated after the record has been read and immediately before it is moved to the data item.

6. When the INTO phrase is used, the record being read is available in both the input record area and the data area associated with identifier.
7. If, at the time of execution of a READ statement, the position of the file position indicator for that file is undefined, the execution of that READ statement is unsuccessful.
8. If the end of a reel is recognized during execution of a READ statement and the logical end of the file has not been reached, the following operations are executed:
   a. The standard ending reel label procedure is executed.
   b. A reel is swapped.
   c. The standard beginning reel label procedure is executed.
   d. The first data record of the new reel is made available.
9. If a file described with the OPTIONAL phrase is not present at the time the file is opened, then at the time of execution of the first READ statement for the file, the AT END condition occurs and the execution of the READ statement is unsuccessful. The standard end-of-file procedures are not performed. (See 14.3.1 and 14.5.2) Execution of the program then proceeds as specified in general rule 11.
10. If, at the time of the execution of a READ statement, no next logical record exists in the file, the at-end condition occurs and the execution of the READ statement is considered unsuccessful.
11. When the at-end condition is recognized, control is transferred to the AT END imperative-statement.

    When the at-end condition occurs, execution of the input-output statement that caused the condition is unsuccessful.
12. Following the unsuccessful execution of any READ statement, the contents of the associated record area and the position of the file position indicator are undefined.
13. When the at-end condition has been recognized, a READ statement for that file must not be executed without first executing a successful CLOSE statement followed by the execution of a successful OPEN statement for that file.

14.5.4. The WRITE Statement

Function

The WRITE statement releases a logical record for an output file.

General Format

```
WRITE record-name [ FROM identifier ]
```

Syntax Rules

1. *Record-name* and *identifier* must not reference the same storage area.
2. *Record-name* is the name of a logical record in the File Section of the Data Division and can be qualified.

General Rules

1. The associated file must be open in the OUTPUT mode at the time of the execution of this statement. (See 14.5.2)
2. The logical record released by the execution of the WRITE statement is no longer available in the record area unless the associated file is named in a SAME RECORD AREA clause or the execution of the WRITE statement was unsuccessful due to a boundary violation. The logical record is also available to the program as a record of other files referenced in the same SAME RECORD AREA clause as the associated output file, as well as to the file associated with *record-name.*
3. The results of the execution of the WRITE statement with the FROM phrase are equivalent to the execution of the statement

   ```
   MOVE identifier TO record-name
   ```

   according to the rules specified for the MOVE statement, followed by the same WRITE statement without the FROM phrase.

   The contents of the record area prior to the execution of the implicit MOVE statement have no effect on the execution of this WRITE statement.

   After execution of the WRITE statement is complete, the information in the area referenced by identifier is available, even though the information in the area referenced by *record-name* cannot be. (See general rule 2.)
4. The file position indicator is unaffected by the execution of a WRITE statement.
5. The maximum record size for a file is established at the time the file is created and must not subsequently be changed.
6. The execution of the WRITE statement releases a logical record to the operating system.
7. After the recognition of an end-of-reel or an end-of-unit of an output file that is contained on more than one physical reel, the WRITE statement performs the following operations:
   a. The standard ending reel label procedure is executed.
   b. A reel is swapped.
   c. The standard beginning reel label procedure is executed.

Section 15
Interlanguage Calls

Procedures written in one UCS language (currently UCS C, UCS COBOL, and UCS FORTRAN) can be called from any of the others. However, differences among the languages require certain restrictions for handling the following items:

- Data types
- Data alignments
- Data sizes
- Passing modes used by the languages
  - Pass by value (PBV)

    PBV is the convention of passing a parameter value directly as an immediate value.

    In the UCS standard calling sequence (SCS), the value can be immediate if its size is two words or less. If the language requires PBV and the parameter size is greater than two words, the compiler uses the pass-by-reference-value passing mode.
  - Pass by reference (PBR)

    PBR is the convention of passing the address of an object, which allows the called program to change the parameter, if the language rules allow it.
  - Pass by reference value (PBRV)

    PBRV is the convention that allows the compiler to automatically copy an object to a temporary variable and pass the address of the temporary variable.

    The called program cannot change the value of the object, only the value of the copy. This method is used by the compiler when the parameter to be passed is greater than two words in size (that is, it does not fit into the SCS packet as an immediate value).
- Contingency handling

  Each UCS language handles contingencies according to its own standard. However, once C signal handling is initialized, it takes precedence over the contingency handling method of any other language. This occurs whether C is the language of the main program or a called function. In a multilanguage environment, therefore, C signal handling can effectively disable the contingency handling routines of UCS COBOL, and UCS FORTRAN.

Each UCS language handles contingencies according to its own standard. However, once C signal handling is initialized, it takes precedence over the contingency handling method of any other language. This occurs whether C is the language of the main program or a called function. In a multilanguage environment, therefore, C signal handling can effectively disable the contingency handling routines of UCS COBOL.

If C signal handling does not provide a routine for a given contingency, the default action prints a message and aborts the program.

Caution

In a multilanguage environment, differences among language definitions, defaults, and control mechanisms can produce unpredictable results from contingencies.

The items in the previous list relating to data attributes and passing modes are discussed in the following subsections, which show how UCS COBOL calls a particular language. Each subsection contains a table that lists COBOL data types and the corresponding types of the other language, if any. Codes in the table identify the possible relations between corresponding types, as follows:

Y

The type is allowed with no restrictions.

N

The type is not allowed.

YL

The type is allowed, but the data item must have the proper storage alignment and size.

Refer to the *UCS COBOL Programming Reference Manual Volume 1* for the storage alignment and size needed by UCS COBOL data types. To determine storage alignment and size needed by other UCS languages, you may have to examine their allocation listings if the information is not available in the programming reference manual.

YT

The argument can be passed, but one of the following is true:

- COBOL does not have a type that directly corresponds to the type of the called language.
- The called language does not have a type that directly corresponds to the COBOL type.

You are responsible for setting up the proper layout and values expected by the called program. For example, the COMPLEX type in FORTRAN can be represented in COBOL as a structure.

You are responsible for setting up the proper layout and values expected by the called program.

YV

The type is allowed, but the value passed (such as the value for an enumeration type) must be in the range of values required by the called program.

When compiling with the COMPAT compiler keyword option, the size and alignment of data objects can change. You should examine the allocation listing produced by UCS COBOL to determine the exact size and alignment specifications and data types.

15.1. UCS COBOL Calling UCS FORTRAN

Table 15–1 lists the data types defined by COBOL and indicates the corresponding FORTRAN data type, if any. This table is based on COBOL default mode. If the compilation uses the compiler keyword option COMPAT, the size and alignment of data objects can change. You may have to examine the allocation listings produced by UCS COBOL and UCS FORTRAN to determine the exact storage layout of variables passed as parameters.

Table 15–1. COBOL vs. FORTRAN Data Types

COBOL Actual Argument	FORTRAN Argument	Relation
PIC S9(10) BINARY	INTEGER	YL
PIC S9(10) COMPUTATIONAL-4	INTEGER	YL
PIC S9(9) COMPUTATIONAL-5	INTEGER	YL
BINARY-1	(none)	YT
COMP-1 COMPUTATIONAL-1	REAL	YL
COMP-2 COMPUTATIONAL-2	DOUBLE PRECISION REAL	YL
COMP PIC 9(*n*) COMPUTATIONAL PIC 9(*n*)	(none)	YT
DISP PIC 9(*n*) DISPLAY PIC 9(*n*)	(none)	YT

Table 15–1. COBOL vs. FORTRAN Data Types

COBOL Actual Argument	FORTRAN Argument	Relation
DISP PIC X(*n*) DISPLAY PIC X(*n*)	CHARACTER*(*n*)	Y
PACKED-DECIMAL	(none)	N
INDEX	(none)	YT
Paragraph or section name	* (Asterisk is dummy argument)	Y
DISP-I18N	CHARACTER*(*n*)	Y
BYTE-I18N	(none)	N

Legend

N Argument type not allowed
Y Argument type allowed with no restrictions
YL Argument type allowed, but data item must have proper storage alignment and size
YT Argument type can be passed, but type may not exist in one of the languages and must be handled within called program.

You should be aware of the following considerations when calling a FORTRAN program from a COBOL program:

- I/O files

  Mass-storage and tape I/O files can be shared between different UCS languages, but they must be closed by the language that opened them before they can be accessed by the other language.

- Length of identifiers

  FORTRAN identifiers must begin with a letter and cannot contain a hyphen (-).

- Alignment of data

  FORTRAN expects all noncharacter arguments to be word-aligned.

- Character data

  COBOL cannot pass character data to FORTRAN variables declared with an asterisk length.

- Structures

  FORTRAN does not have structures.

- Functions

  A FORTRAN function cannot be called.

- Arrays

  FORTRAN arrays are allocated storage in column-major order. UCS COBOL allocates arrays in row-major order. If a COBOL array is passed as an argument to UCS FORTRAN, the FORTRAN program must declare and reference the array with the array subscripts in reverse order as compared to the array in COBOL.

- Assumed-shape arrays

  COBOL cannot pass arrays to FORTRAN assumed-shape arrays.

- Standard print file

  FORTRAN output to the standard print file operates in the reverse order of the COBOL DISPLAY statement. FORTRAN performs a line feed and then writes the items in the output list. By default, the COBOL DISPLAY statement first writes the items in the output list and then performs a line feed. Therefore, you may need special logic to avoid overprinting if both the main COBOL program and the FORTRAN subprogram print output. The example at the end of this subsection demonstrates this consideration.

Example

Example 15–1 and Example 15–2 demonstrate a call from a COBOL main program to a FORTRAN subprogram, with the following actions by the main program and subprogram:

1. The COBOL main program initializes an integer variable, a character string, and a one-dimensional floating-point array.
2. The contents of the COBOL data items are displayed to verify initialization.
3. The COBOL main program calls the FORTRAN subprogram, passing the three data items as actual arguments.
4. The FORTRAN subprogram displays the incoming dummy arguments to demonstrate that the dummy arguments were received correctly and that the subprogram interprets them correctly.
5. The subprogram updates the contents of the dummy arguments and returns control to the main program. The changes to the dummy arguments are straightforward.
6. The main program again displays the contents of the arguments to verify that the FORTRAN subprogram updated them correctly.

```
IDENTIFICATION DIVISION.
 PROGRAM-ID.
     COBMAIN.
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER. UNISYS-2200.
 OBJECT-COMPUTER. UNISYS-2200.
 SPECIAL-NAMES.
     PRINTER IS PRT.
 DATA DIVISION.
 WORKING-STORAGE SECTION.
 01  workrec1.
     02  parm1  PIC S9(10) USAGE BINARY.
 01  workrec2.
     02  parm2  PIC X USAGE DISPLAY.
 01  workrec3.
     02  parm3 USAGE COMP-1 OCCURS 3 TIMES VALUE 1.
 PROCEDURE DIVISION.
```

```
 BEGIN.
     DISPLAY 'Start in COBOL Main Program' UPON PRT.
     MOVE 123 to parm1.
     MOVE '* COBOL *' to parm2.
*
*  Call Fortran Subprogram
*
     Display 'Call Fortran Passing  Parm1 = ' parm1 UPON PRT.
     Display '                      Parm2 = ' parm2 UPON PRT.
     Display '                  Parm3(1) = ' parm3(1) UPON PRT.
     Display '                  Parm3(2) = ' parm3(2) UPON PRT.
     Display '                  Parm3(3) = ' parm3(3) UPON PRT.

     Call 'FORSUB' USING parm1, parm2, workrec3.

     Display 'After call to Fortran Parm1 = ' parm1 UPON PRT.
     Display '                      Parm2 = ' parm2 UPON PRT.
     Display '                  Parm3(1) = ' parm3(1) UPON PRT.
     Display '                  Parm3(2) = ' parm3(2) UPON PRT.
     Display '                  Parm3(3) = ' parm3(3) UPON PRT.
     STOP RUN.
```

Example 15–1. UCS COBOL Main Program to Call UCS FORTRAN Subprogram

```
subroutine FORSUB(iparm, cparm, aparm)
     integer iparm
     character cparm*12
     real aparm(3)
     print *,' '
     print *,'Inside Fortran        Parm1 = ', iparm
     print *,'                      Parm2 = ', cparm
     print *,'                      Parm3 = ', aparm
     print *,' '
     iparm = 456
     cparm = '* FORTRAN *'
     aparm = 5.0
     return
     end
```

Example 15–2. UCS FORTRAN Subprogram Called from UCS COBOL Main Program

Example 15–3 shows the output from executing this program.

```
Start in COBOL Main Program
Call Fortran Passing  Parm1 = +0000000123
                      Parm2 = * COBOL *
                    Parm3(1) =  0.10000000E+01
                    Parm3(2) =  0.10000000E+01
                    Parm3(3) =  0.10000000E+01

Inside Fortran        Parm1 =           123
                      Parm2 = * COBOL *
                      Parm3 =   1.0000000      1.0000000      1.0000000

After call to Fortran Parm1 = +0000000456
                      Parm2 = * FORTRAN *
                    Parm3(1) =  0.50000000E+01
                    Parm3(2) =  0.50000000E+01
                    Parm3(3) =  0.50000000E+01
```

Example 15–3. Output of UCS COBOL Program Calling UCS FORTRAN Subprogram

15.2. UCS COBOL Calling UCS C

Table 15–2 lists the data types defined by COBOL and indicates the corresponding C data type, if any. This table is based on COBOL default mode. If the compilation uses the compiler keyword option COMPAT, the size and alignment of data objects can change. You may have to examine the allocation listings produced by UCS COBOL and UCS C to determine the exact storage layout of variables passed as parameters.

Table 15–2. COBOL vs. C Data Types

COBOL Actual Argument	C Parameter	Relation
PIC S9(10) BINARY	char, unsigned char int, unsigned int long, unsigned long, short, unsigned short, long long	YL
PIC S9(10) COMP-4 COMPUTATIONAL-4	char, unsigned char int, unsigned int long, unsigned long, short, unsigned short, long long	YL
BINARY-1	char, unsigned char int, unsigned int long, unsigned long, short, unsigned short, long long	YL
COMP-1 COMPUTATIONAL-1	float	YL
COMP-2 COMPUTATIONAL-2	double, long double	YL
COMP-5 COMPUTATIONAL-5	short, long	YL
COMP PIC 9(*n*) COMPUTATIONAL PIC 9(*n*)	char	Y

Table 15–2. COBOL vs. C Data Types

COBOL Actual Argument	C Parameter	Relation
DISP PIC 9(*n*) DISPLAY PIC 9(*n*)	char	Y
DISP PIC X(*n*) DISPLAY PIC X(*n*)	char	Y
PACKED-DECIMAL	(none)	YT
INDEX	int, unsigned int long, unsigned long	YL
Paragraph or section name	(none)	N
DISP-I18N	char	Y
BYTE-I18N	(none)	YT
USAGE POINTER	Pointer	Y

Legend

N Argument type not allowed.
Y Argument type allowed with no restrictions.
YL Argument type allowed, but data item must have proper storage alignment and size.
YT Argument can be passed, but type may not exist in one of the languages and must be handled within called program.

You should be aware of the following considerations when calling a C program from a COBOL program:

- Parameter passing

  C passes arguments by value. However, applying the #pragma interface to a C function being called from COBOL will cause the compiler to convert reference parameters into C's pointer format. The C program must define all of its parameters as being pointers.

- Alignment of data

  When a parameter is passed by reference to a C function with the interface pragma applied, the parameter's alignment must match UCS C's alignment rules. When the C formal parameter is a pointer to a char, the data passed must be byte aligned. When the C formal parameter is a pointer to short, the data passed must be half-word aligned. When the C formal parameter is any other type of pointer, the data passed must be word aligned. If the compiler keyword option CALIGN is in effect, the compiler will assure that data items with USAGE COMP-1, COMP-2, and COMP-5 are aligned on the proper boundare (see 2.2.7).

- I/O files

  Mass-storage and tape I/O files can be shared between different UCS languages, but they must be closed by the language that opened them before they can be accessed by the other language.

- C run-time environment

  To ensure the proper C runtime environment, including I/O and contingency handling within URTS, any language transition from another UCS language to UCS C requires the compilation unit containing the called C function to be compiled with the AFA/FULL keyword option when the C function does not have the #pragma interface applied.

- Case sensitivity of names

  Since the C language is case-sensitive and COBOL converts external names to uppercase letters, be sure the external names of all C subprograms are declared in uppercase characters.

- Function names

  The names of C functions called from a COBOL program must start with a letter and cannot contain a hyphen (-).

- Functions

  COBOL does not support function calls and should not call a function written in C. In other words, COBOL cannot receive the return value from a C function. You must use arguments to pass a returned value from C to COBOL.

- Character strings

  The C convention for indicating the end of a character string (which C maintains as a single-dimension array) is to use a null character, which is a byte containing all binary zeros. C string functions, as well as the %s format descriptor, require the null character to terminate the string.

  If a COBOL program passes a character string to a C function, it must insert a null character in the appropriate place in the string. If a COBOL program receives a character string from a C program, the COBOL program should consider the length of the string to be the length of the text preceding the null character in the string, not the declared length of the string.

Example

Example 15–4 and Example 15–5 demonstrate a call from a COBOL main program to a C subprogram, with the following actions by the main program and subprogram:

1. The COBOL main program initializes an integer variable, a character string, and a one-dimensional floating-point array.
2. The contents of the COBOL data items are displayed to verify initialization.
3. The COBOL main program calls the C subprogram, passing the three data items as actual arguments.

   The second argument is a structure that represents a C character string, which has a terminating null character.
4. The C subprogram displays the incoming dummy arguments to demonstrate that the dummy arguments were received correctly and that the subprogram interprets them correctly.

5. The subprogram updates the contents of the dummy arguments and returns control to the main program. The changes to the dummy arguments are straightforward.
6. The main program again displays the contents of the arguments to verify that the C subprogram updated them correctly.

Example 15–4 shows the output from executing this program.

```
IDENTIFICATION DIVISION.
PROGRAM-ID.
    COBMAIN.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
SPECIAL-NAMES.
    PRINTER IS PRT
    SYMBOLIC CHARACTERS null IS 1.
DATA DIVISION.
WORKING-STORAGE SECTION.
01  workrec1.
    02  parm1  PIC S9(10) USAGE BINARY.
01  parm2.
    02  parm2-1 PIC X(6) USAGE DISPLAY VALUE 'STRING'.
    02  parm2-2 PIC X.
01  workrec3.
    02  parm3 USAGE COMP-1 OCCURS 3 TIMES VALUE 1.
PROCEDURE DIVISION.
BEGIN.
    DISPLAY 'Start in COBOL Main Program' UPON PRT.
    MOVE 123 to parm1.
    MOVE null to parm2-2.
*
*  Call C Subprogram
*
     Display 'Call C Passing  Parm1 = ' parm1 UPON PRT.
     Display '                Parm2 = ' parm2 UPON PRT.
     Display '             Parm3(1) = ' parm3(1) UPON PRT.
     Display '             Parm3(2) = ' parm3(2) UPON PRT.
     Display '             Parm3(3) = ' parm3(3) UPON PRT.

     Call 'CSUB' USING parm1, parm2, workrec3.

     Display 'After call to C Parm1 = ' parm1 UPON PRT.
     Display '                Parm2 = ' parm2 UPON PRT.
     Display '             Parm3(1) = ' parm3(1) UPON PRT.
     Display '             Parm3(2) = ' parm3(2) UPON PRT.
     Display '             Parm3(3) = ' parm3(3) UPON PRT.
     STOP RUN.
```

Example 15–4. UCS COBOL Main Program to Call UCS C Subprogram

```
#include <stdio.h>
#include <string.h>

#pragma interface CSUB
void CSUB(int *iptr,char *cptr,float (*fptr)[3])
{
 int i;

 printf("\n");
 printf("*Inside C      Parm1 =%d \n",*iptr);
 printf("               Parm2 =%s \n",cptr);
 for (i=0;i<3;i++)
   printf("               Parm3(%i) = %f\n",i+1,(*fptr)[i]) ;
 printf("\n") ;

 *iptr =456 ;
 strcpy(cptr,"CHANGE");
 (*fptr)[0] = (*fptr)[1] = (*fptr)[2] = 5.0 ;
}
```

Example 15–5. UCS C Subprogram Called from UCS COBOL Main Program

```
Start in COBOL Main Program
Call C Passing  Parm1 = +0000000123
                Parm2 = STRING
             Parm3(1) =  0.10000000E+01
             Parm3(2) =  0.10000000E+01
             Parm3(3) =  0.10000000E+01

Inside C     Parm1 = 123
             Parm2 = STRING
             Parm3 = 1.000000  1.000000   1.000000

After call to C Parm1 = +0000000456
                Parm2 = CHANGE
             Parm3(1) =  0.50000000E+01
             Parm3(2) =  0.50000000E+01
             Parm3(3) =  0.50000000E+01
```

Example 15–6. Output of UCS COBOL Program Calling UCS C Subprogram

Section 16
Internationalization

Internationalization (I18N) refers to the Unisys software features that enable you to develop and run applications that can be customized to meet the needs of a specific language, culture, or business environment. The internationalization features provide support for numerous character sets and locales.

16.1. I18NLIB Components

The Internationalization Library (I18NLIB) product forms the basis for the operating system's internationalization capabilities and provides services for system software and application programs to use when working with non-US-English operating environments.

I18NLIB has the following major parts:

1. Exec Environment Variables - The environment variables convey information between the application program and the service routines.
2. Locale definition files - These files define the processing rules for a region. Their related CHARMAP files define the allowable characters for the region's language(s). The locale definition files are provide by Unisys.
3. UREP entities and relationships - UREP is used to keep track of and report on locale, CHARMAP, translation, and other information available to the service routines and available to the system software and application programs.
4. Service Routines - These routines process data passed by the system software or application program according to the rules supplied by the locale definition.
5. COPY procs for PLUS, C, and COBOL which provide the declarations for the service routines and define templates for the structures used by the service routines.

16.2. Internationalization of Programs

In order to internationalize a UCS COBOL program, a user must insert CALL statements to the desired I18NLIB service routine (i.e. UCOB$SETLOC, UCOB$STRCOLL, UCOB$TOUPPER ...) and/or use the new data types defined by the USAGE DISP-I18N and USAGE BYTE-I18N clauses. The *Internationalization Planning and Implementation Overview* provides complete details on how to call each of the I18NLIB service routines.

The USAGE DISP-I18N data type has been added to allow the UCS COBOL compiler to do comparisons in a culturally sensitive manner. When a DISP-I18N data item is explicitly used in a comparison (i.e. IF, EVALUATE, SEARCH) the compiler will generate a call to the I18NLIB service routine string_collate in order to do the comparison based upon the locale that is currently in force. The use of this data type can eliminate the need for the user to insert CALL statements to the string_collate service routine.

The USAGE BYTE-I18N data type will be used to declare a string of bits that are obtained by calling the I18NLIB service routine string_transform (UCOB$STRXFRM). This string of bits can be compared to another string of bits obtained from another call to string_transform to determine the collating order of the items that are being compared. The two BYTE-I18N data items are compared as bit strings with no conversion being done. If the BYTE-I18N data items are of unequal length, then the shorter one will be padded with binary zeroes on the right until it is of equal size to the larger one.

Code type and locale information can be stored and retrieved from data files through the CODE TYPE and LOCALE clauses. These clauses allow you to store information about the data at the time the file is created and to retrieve the information about the data at the time the file is input. The setting and retrieving of CODE TYPE and LOCALE information is controlled by the use of the I18N-IO compiler option.

The *Internationalization Planning and Implementation Overview* provides complete details about all of the locales and service routines that can be used. Please refer to that document for questions concerning the internationalization of your UCS COBOL program.

Section 17
CIFS COBOL Program Interface

CIFS for ClearPath OS 2200, is an implementation of the Common Internet File System (CIFS) specification that allows access to OS 2200 files via standard Internet protocols. For complete details about CIFS, see *CIFS for ClearPath OS 2200 User, Programmer, and Administrator Reference Manual*.

CIFS permits direct access to files stored in OS 2200 through SMB clients like Windows Explorer. You can move, copy, delete, and display properties of their OS 2200 files. You can also edit files with Notepad, Word, image processors, and so on, and save changes to the files. Using Windows drag-and-drop functions, you can copy or move files between your workstation and OS 2200.

COBOL programmers can access CIFS services and application program interfaces (APIs) by calling them as subprograms from a COBOL program. See the *CIFS for ClearPath OS 2200 User, Programmer, and Administrator Reference Manual* for complete details and descriptions of all the available APIs.

17.1. CIFS Copy Procedures

The following table lists the UCOB COPY PROCs that are available to CIFS programs. These PROCs are in the element CIFS-DEF in the file SYSLIB*CIFS$LIB. These COPY PROCs should be used in programs that call the CIFS APIs.

Proc Name	Description
CIFS-AMODE	Contains definitions for CIFS$ACCESS
CIFS-API	Contains definitions for the CIFS status definitions
CIFS-DIR	Contains definitions for a DIR variable for directory service subprograms
CIFS-FCNTL	Contains definitions for the cifs-flock-struct variable for file locking and cifs-fcntl-cmd variable for file control operations
CIFS-FILE	Contains definitions for a FILE variable for stream service subprograms
CIFS-FPOS	Contains definitions for the cifs-fpos variable for file positioning
CIFS-MODE	Contains definitions for the cifs-mode variable for CIFS$OPEN, CIFS$CHMOD, and so on

Proc Name	Description
CIFS-OFLAG	Contains definitions for the cifs-oflag variable for CIFS$OPEN
CIFS-STAT	Contains definitions for the cifs-stat and cifs-istat data item for CIFS$STAT and CIFS$ISTAT
CIFS-XLATE	Contains definitions for the cifs-xlate variable for CIFS$XLATE commands

17.2. COBOL APIs

The following COBOL APIs are available for use in programs that need to perform CIFS functionality. For complete details on each API, see *CIFS for ClearPath OS 2200 User, Programmer, and Administrator Reference Manual*.

CIFS Call	Parameters
CALL "CIFS$ACCESS"	USING PATH, CIFS-AMODE, CIFS-STATUS
CALL "CIFS$CHDIR"	USING PATH, CIFS-STATUS
CALL "CIFS$CHMOD"	USING PATH, CIFS-MODE, CIFS-STATUS
CALL "CIFS$CHOWN"	USING PATH, OWNER, GROUP, CIFS-STATUS
CALL "CIFS$CLEARERR"	USING CIFS-FILE
CALL "CIFS$CLOSE"	USING FD, CIFS-STATUS
CALL "CIFS$CLOSEDIR"	USING CIFS-DIR, CIFS-STATUS
CALL "CIFS$CREAT"	USING PATH, CIFS-MODE, NEW-FD, CIFS-STATUS
CALL "CIFS$DUP"	USING OLD-FD, NEW-FD, CIFS-STATUS
CALL "CIFS$DUP2"	USING FD-1, FD-2, CIFS-STATUS
CALL "CIFS$FCLOSE"	USING CIFS-FILE, CIFS-STATUS
CALL "CIFS$FCNTL"	USING FD, FCNTL-CMD, CMD-PARAM, RETURN-VALUE, CIFS-STATUS
CALL "CIFS$FDATASYNC"	USING FD, CIFS-STATUS
CALL "CIFS$FDOPEN"	USING FD, TYPE, CIFS-FILE, CIFS-STATUS
CALL "CIFS$FEOF"	USING CIFS-FILE, EOF-FLAG
CALL "CIFS$FERROR"	USING CIFS-FILE, ERROR-FLAG
CALL "CIFS$FFLUSH"	USING CIFS-FILE, CIFS-STATUS
CALL "CIFS$FGETC"	USING CIFS-FILE, NEXT-CHAR, CIFS-STATUS
CALL "CIFS$FGETPOS"	USING CIFS-FILE, CIFS-FPOS, CIFS-STATUS
CALL "CIFS$FGETS"	USING STRING, MAX-CHARS, CIFS-FILE, CIFS-STATUS
CALL "CIFS$FILENO"	USING CIFS-FILE, FD, CIFS-STATUS

CIFS Call	Parameters
CALL "CIFS$FOPEN"	USING PATH, TYPE-STRING, CIFS-FILE, CIFS-STATUS
CALL "CIFS$FPRINTF"	USING CHARS-OUT, CIFS-FILE, FORMAT-STRING[, ITEM...]
CALL "CIFS$FPUTC"	USING CHAR, CIFS-FILE, CIFS-STATUS
CALL "CIFS$FPUTS"	USING STRING, CIFS-FILE, CHARS-OUT, CIFS-STATUS
CALL "CIFS$FREAD"	USING BUFFER, ELT-SIZE, ELT-COUNT, CIFS-FILE, ELTS-READ, CIFS-STATUS
CALL "CIFS$FREOPEN"	USING PATH, TYPE-STRING, CIFS-FILE, CIFS-STATUS
CALL "CIFS$FSCANF"	USING ITEMS-SCANNED, CIFS-FILE, FORMAT-STRING[, ITEM...]
CALL "CIFS$FSEEK"	USING CIFS-FILE, OFFSET, WHENCE, CIFS-STATUS
CALL "CIFS$FSETPOS"	USING CIFS-FILE, CIFS-FPOS, CIFS-STATUS
CALL "CIFS$FSTAT"	USING FD, CIFS-STAT, CIFS-STATUS
CALL "CIFS$FSYNC"	USING FD, CIFS-STATUS
CALL "CIFS$FTELL"	USING CIFS-FILE, OFFSET, CIFS-STATUS
CALL "CIFS$FTRUNCATE"	USING FD, LENGTH, CIFS-STATUS
CALL "CIFS$FWRITE"	USING BUFFER, ELT-SIZE, ELT-COUNT, CIFS-FILE, ELTS-WRITTEN, CIFS-STATUS
CALL "CIFS$GETC"	USING CIFS-FILE, NEXT-CHAR, CIFS-STATUS
CALL "CIFS$GETCHAR"	USING NEXT-CHAR, CIFS-STATUS
CALL "CIFS$GETCWD"	USING PATH-BUFFER, MAX-SIZE, CIFS-STATUS
CALL "CIFS$GETS"	USING STRING, CIFS-STATUS
CALL "CIFS$ISTAT"	USING INO, CIFS-ISTAT, SIZE, CIFS-STATUS
CALL "CIFS$LINK"	USING PATH1, PATH2, CIFS-STATUS
CALL "CIFS$LSEEK"	USING FD, OFFSET, WHENCE, NEW-OFFSET, CIFS-STATUS
CALL "CIFS$MASK"	USING NAME, MASK, RESULT
CALL "CIFS$MKDIR"	USING PATH, CIFS-MODE, CIFS-STATUS
CALL "CIFS$MKFIFO"	USING PATH, CIFS-MODE, CIFS-STATUS
CALL "CIFS$OPEN"	USING PATH, CIFS-OFLAG, CIFS-MODE, NEW-FD, CIFS-STATUS
CALL "CIFS$OPENDIR"	USING PATH, CIFS-DIR, CIFS-STATUS
CALL "CIFS$PERROR"	USING CIFS-STATUS, MSG-STR
CALL "CIFS$PIPE"	USING FD-PAIR, CIFS-STATUS
CALL "CIFS$PRINTF"	USING CHARS-OUT, FORMAT-STRING[, ITEM...]
CALL "CIFS$PUTC"	USING CHAR, CIFS-FILE, CIFS-STATUS
CALL "CIFS$PUTCHAR"	USING CHAR, CIFS-STATUS

CIFS Call	Parameters
CALL "CIFS$PUTS"	USING STRING, CIFS-STATUS
CALL "CIFS$READ"	USING FD, BUFFER, BYTES-TO-READ, BYTES-READ, CIFS-STATUS
CALL "CIFS$READDIR"	USING CIFS-DIR, ENTRY-INO, ENTRY-NAME, MAX-LEN, TRUE-LEN, CIFS-STATUS
CALL "CIFS$REMOVE"	USING PATH, CIFS-STATUS
CALL "CIFS$RENAME"	USING OLD-PATH, NEW-PATH, CIFS-STATUS
CALL "CIFS$REWIND"	USING CIFS-FILE, CIFS-STATUS
CALL "CIFS$REWINDDIR"	USING CIFS-DIR, CIFS-STATUS
CALL "CIFS$RMDIR"	USING PATH, CIFS-STATUS
CALL "CIFS$SCANF"	USING ITEMS-SCANNED, FORMAT-STRING[, ITEM...]
CALL "CIFS$SHARE"	USING PATH, SHARE-NAME, CIFS-STATUS
CALL "CIFS$SPRINTF"	USING CHARS-OUT, DEST-STRING, FORMAT-STRING[, ITEM…]
CALL "CIFS$SSCANF"	USING ITEMS-SCANNED, SCAN-STRING, FORMAT-STRING[, ITEM...]
CALL "CIFS$STAT"	USING PATH, CIFS-STAT, CIFS-STATUS
CALL "CIFS$STDERR"	USING CIFS-FILE
CALL "CIFS$STDIN"	USING CIFS-FILE
CALL "CIFS$STDOUT"	USING CIFS-FILE
CALL "CIFS$STRERROR"	USING ERRNUM, MESSAGE-STRING[, INSERT-STRING…]
CALL "CIFS$TMPFILE"	USING CIFS-FILE, CIFS-STATUS
CALL "CIFS$TMPNAM"	USING NAME-STRING, CIFS-STATUS
CALL "CIFS$TRUNCATE"	USING PATH, LENGTH, CIFS-STATUS
CALL "CIFS$UMASK"	USING NEW-CIFS-MODE, OLD-CIFS-MODE, CIFS-STATUS
CALL "CIFS$UNAME"	USING CIFS-SS-BDI, CIFS-SS-NAME, CIFS-STATUS
CALL "CIFS$UNGETC"	USING PREV-CHAR, CIFS-FILE, RTN-CHAR, CIFS-STATUS
CALL "CIFS$UNLINK"	USING PATH, CIFS-STATUS
CALL "CIFS$UTIME"	USING PATH, CIFS-UTIMBUF, CIFS-STATUS
CALL "CIFS$WRITE"	USING FD, BUFFER, BYTES-TO-WRITE, BYTES-WRITTEN, CIFS-STATUS
CALL "CIFS$XLATE"	USING PATH, CIFS-XLATE, CCS-IN, CCS-OUT, CIFS-STATUS

17.3. Example Program

The following example program can be used to list the contents of a file/directory that resides on the 2200 or it can be use to copy a file/directory from one location to another. It uses many of the service programs that were discussed earlier in this section. First, compile the program. Be sure to perform the following command prior to the compilation:

```
@use cob$pf.,sys$lib$*cifs$lib.

 IDENTIFICATION DIVISION.
 PROGRAM-ID.
     CIFS-TEST.
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER. UNISYS-2200.
 OBJECT-COMPUTER. UNISYS-2200.
*
*  Control characters commonly used in C strings.
*
 SPECIAL-NAMES.
     SYMBOLIC nul, lf IS 1, 11.
 DATA DIVISION.
 WORKING-STORAGE SECTION.
*
*  General CIFS definitions.
*
 COPY CIFS-API.
*
*  Definitions for OFLAG bits used on open() calls.
*
 COPY CIFS-OFLAG.
*
*  Definitions for MODE bits used when creating files.
*
 COPY CIFS-MODE.
*
*  Space for a pointer to the standard output and standard error stream
*  control structures.
*
 COPY CIFS-FILE REPLACING CIFS-FILE BY CIFS-STDERR-FILE.
 COPY CIFS-FILE REPLACING CIFS-FILE BY CIFS-STDOUT-FILE.
*
*  Command parsing control string, result fields, and error messages.
*
 01  COMMAND-FORMAT.
   03  FILLER                  PIC X(8) VALUE " %n%3s%n".
   03  FILLER                  PIC X(7) VALUE " %4095s".
   03  FILLER                  PIC X(7) VALUE " %4095s".
   03  FILLER                  PIC X(4) VALUE " %1s".
```

```
   03  FILLER                  PIC X VALUE nul.
 77  FIELD-COUNT               PIC S1(36) BINARY-1.
 01  COMMAND-LINE.
   03  COMMAND-BYTE            PIC X OCCURS 4.
 77  COMMAND-START             PIC S1(36) BINARY-1.
 77  COMMAND-STOP              PIC S1(36) BINARY-1.
 77  COMMAND-BYTES             PIC S1(36) BINARY-1.
 77  ARGUMENT-1                PIC X(4096).
 77  ARGUMENT-2                PIC X(4096).
 77  ARGUMENT-3                PIC X.
 01  COMMAND-ERROR-MESSAGE.
   03  FILLER                  PIC X(27)
         VALUE "*Error reading command line".
   03  FILLER                  PIC X VALUE nul.
 01  SYNTAX-ERROR-FORMAT.
   03  FILLER                  PIC X(24)
                               VALUE "Unrecognized input: <%s>".
   03  FILLER                  PIC X VALUE lf.
   03  FILLER                  PIC X VALUE nul.
*
*  Error and informational messages for file access.
*
 01  OPEN-ERROR-FORMAT.
   03  FILLER                  PIC X(21)
                               VALUE "*Error opening <%s>: ".
   03  FILLER                  PIC X VALUE nul.
 01  READ-ERROR-MESSAGE.
   03  FILLER                  PIC X(25)
         VALUE "*Error reading input file".
   03  FILLER                  PIC X VALUE nul.
 01  WRITE-ERROR-MESSAGE.
   03  FILLER                  PIC X(26)
         VALUE "*Error writing output file".
   03  FILLER                  PIC X VALUE nul.
 01  CLOSE-ERROR-MESSAGE.
   03  FILLER                  PIC X(26)
         VALUE "*Error closing output file".
   03  FILLER                  PIC X VALUE nul.
 01  COPY-FORMAT.
   03  FILLER                  PIC X(15)
                               VALUE "%d bytes copied".
   03  FILLER                  PIC X VALUE lf.
   03  FILLER                  PIC X VALUE nul.
*
*  Data fields used in reading and writing files.
*
 77  READ-FD                   PIC S1(36) BINARY-1.
 77  WRITE-FD                  PIC S1(36) BINARY-1.
 COPY CIFS-DIR.
 77  BYTES-READ                PIC S1(36) BINARY-1.
 77  BYTES-WRITTEN             PIC S1(36) BINARY-1.
 77  BYTES-TOTAL               PIC S1(36) BINARY-1.
```

```
 01  DATA-BUFFER.
   03  DATA-BYTE                PIC X OCCURS 131072.
*
*  Error and informational messages for listing directory entries.
*
 01  GETCWD-ERROR-MESSAGE.
   03  FILLER                   PIC X(35)
         VALUE "*Error retrieving current directory".
   03  FILLER                   PIC X VALUE nul.
 01  CHDIR-ERROR-FORMAT.
   03  FILLER                   PIC X(25)
         VALUE "*Error changing to <%s>: ".
   03  FILLER                   PIC X VALUE nul.
 01  DIRECTORY-ERROR-MESSAGE.
   03  FILLER                   PIC X(24)
                                VALUE "*Error reading directory".
 01  STAT-ERROR-FORMAT.
   03  FILLER                   PIC X(26)
         VALUE "*Error on stat() of <%s>: ".
   03  FILLER                   PIC X VALUE nul.
 01  STAT-ENTRY-FORMAT.
   03  FILLER                   PIC X(6) VALUE " %-18s".
   03  FILLER                   PIC X(6) VALUE " %012o".
   03  FILLER                   PIC X(4) VALUE " %8i".
   03  FILLER                   PIC X(4) VALUE " %5i".
   03  FILLER                   PIC X(5) VALUE " %11i".
   03  FILLER                   PIC X VALUE lf.
   03  FILLER                   PIC X VALUE nul.
 01  ISTAT-ERROR-FORMAT.
   03  FILLER                   PIC X(35)
         VALUE "*Error on istat() of <%s>, ino %d: ".
   03  FILLER                   PIC X VALUE nul.
 01  ISTAT-FILE-FORMAT.
   03  FILLER                   PIC X(11) VALUE "  %s#%s(%d)".
   03  FILLER                   PIC X VALUE nul.
 01  ISTAT-TYPE-FORMAT.
   03  FILLER                   PIC X(10) VALUE " (%hd.%hd)".
   03  FILLER                   PIC X VALUE nul.
 01  ISTAT-OWNER-FORMAT.
   03  FILLER                   PIC X(7) VALUE "  %s/%s".
   03  FILLER                   PIC X VALUE lf.
   03  FILLER                   PIC X VALUE nul.
*
*  Data fields used in listing directories.
*
 01  DOT-NAME.
   03  FILLER                   PIC X VALUE ".".
   03  FILLER                   PIC X VALUE nul.
 77  INODE-NUMBER               PIC S1(36) BINARY-1.
 01  ENTRY-NAME.
   03  ENTRY-BYTE               PIC X OCCURS 4096.
 COPY CIFS-STAT.
```

```
*
*
*
 PROCEDURE DIVISION.
 COMMAND-LOOP SECTION.
 GET-COMMAND.
*
*  Retrieve the FILE struct pointers for use on normal and
*  error messages.
*
     CALL "CIFS$STDOUT" USING CIFS-STDOUT-FILE.
     CALL "CIFS$STDERR" USING CIFS-STDERR-FILE.
*
*  Main loop to read, parse, and dispatch commands.
*
     PERFORM TEST AFTER UNTIL COMMAND-BYTES = 0
*
*  Read a command line from standard input.
*
         CALL "CIFS$READ" USING CIFS-STDIN-FD,
                                DATA-BUFFER,
                                131071,
                                COMMAND-BYTES,
                                CIFS-STATUS
         IF BYTES-READ < 0
             CALL "CIFS$PERROR" USING CIFS-STATUS,
                                      COMMAND-ERROR-MESSAGE
             STOP RUN
         END-IF
*
*  Insure the image is nul terminated for sscanf().
*
         IF COMMAND-BYTES > 0 AND DATA-BYTE(COMMAND-BYTES) = lf
             MOVE nul to DATA-BYTE(COMMAND-BYTES)
         ELSE
             MOVE nul TO DATA-BYTE(COMMAND-BYTES + 1)
         END-IF
*
*  Clear command field before parsing into it.
*
         MOVE SPACES TO COMMAND-LINE
         CALL "CIFS$SSCANF" USING FIELD-COUNT,
                                  DATA-BUFFER,
                                  COMMAND-FORMAT,
                                  COMMAND-START,
                                  COMMAND-LINE,
                                  COMMAND-STOP,
                                  ARGUMENT-1,
                                  ARGUMENT-2,
                                  ARGUMENT-3
*
*  If anything was input, replace the terminating nul with a space to
```

```
*  allow COBOL comparisons to work, then branch to the proper handler.
*
         IF FIELD-COUNT > 0
             SUBTRACT COMMAND-START FROM COMMAND-STOP GIVING
               COMMAND-BYTES
             MOVE SPACE TO COMMAND-BYTE(COMMAND-BYTES + 1)
             EVALUATE COMMAND-LINE ALSO FIELD-COUNT
                 WHEN "cp" ALSO 3 PERFORM COPY-FILE
                 WHEN "ls" ALSO 2 PERFORM LIST-DIRECTORY
                 WHEN OTHER PERFORM SYNTAX-ERROR
             END-EVALUATE
         END-IF
     END-PERFORM.
     STOP RUN.
*
*  Mumble something about inability to type.
*
 SYNTAX-ERROR.
     CALL "CIFS$FPRINTF" USING BYTES-WRITTEN,
                               CIFS-STDERR-FILE,
                               SYNTAX-ERROR-FORMAT,
                               DATA-BUFFER.
*
*  Section to copy a file to a new location.
*
 COPY-FILE SECTION.
*
*  Open the source file read-only. Display an error message and give
*  up if the open() fails.
*
 OPEN-INPUT-FILE.
     MOVE 0 TO CIFS-OFLAG.
     MOVE 1 TO CIFS-OFLAG-RDONLY.
     MOVE 0 TO CIFS-MODE.
     CALL "CIFS$OPEN" USING ARGUMENT-1,
                            CIFS-OFLAG,
                            CIFS-MODE,
                            READ-FD,
                            CIFS-STATUS.
     IF READ-FD < 0
         CALL "CIFS$FPRINTF" USING BYTES-WRITTEN,
                                   CIFS-STDERR-FILE,
                                   OPEN-ERROR-FORMAT,
                                   ARGUMENT-1
         CALL "CIFS$PERROR" USING CIFS-STATUS
         GO TO COPY-DONE
     END-IF.
*
*  Now try to create the output file. If the specifiec file already
*  exists in a writable state, it will be erased and overwritten.
*  Otherwise, a new file will be created. Any errors result in an
*  message being displayed and abandonment of the copy.
```

```
*
 OPEN-OUTPUT-FILE.
     MOVE 0 TO CIFS-MODE.
     COMPUTE CIFS-MODE-IRWXU = CIFS-MODE-IREAD +
                               CIFS-MODE-IWRITE.
     COMPUTE CIFS-MODE-IRWXG = CIFS-MODE-IREAD;
     COMPUTE CIFS-MODE-IRWXO = CIFS-MODE-IREAD;
     CALL "CIFS$CREAT" USING ARGUMENT-2,
                             CIFS-MODE,
                             WRITE-FD,
                             CIFS-STATUS.
     IF WRITE-FD < 0
         CALL "CIFS$FPRINTF" USING BYTES-WRITTEN,
                                   CIFS-STDERR-FILE,
                                   OPEN-ERROR-FORMAT,
                                   ARGUMENT-2
         CALL "CIFS$PERROR" USING CIFS-STATUS
         CALL "CIFS$CLOSE" USING READ-FD,
                                 CIFS-STATUS
         GO TO COPY-DONE
     END-IF.
*
*  Both input and output files are open, so start reading and writing.
*  Display a message and punt if any errors encountered.
*
 COPY-INPUT-TO-OUTPUT.
     MOVE 0 TO BYTES-TOTAL.
     PERFORM TEST AFTER UNTIL BYTES-READ = 0
         CALL "CIFS$READ" USING READ-FD,
                                DATA-BUFFER,
                                131072,
                                BYTES-READ,
                                CIFS-STATUS
         IF BYTES-READ < 0
             CALL "CIFS$PERROR" USING CIFS-STATUS,
                                      READ-ERROR-MESSAGE
             GO TO CLOSE-FILES
         END-IF
         IF BYTES-READ > 0
             CALL "CIFS$WRITE" USING WRITE-FD,
                                     DATA-BUFFER,
                                     BYTES-READ,
                                     BYTES-WRITTEN,
                                     CIFS-STATUS
             IF BYTES-WRITTEN < 0
                 CALL "CIFS$PERROR" USING CIFS-STATUS,
                                          WRITE-ERROR-MESSAGE
                 GO TO CLOSE-FILES
             END-IF
             ADD BYTES-WRITTEN TO BYTES-TOTAL
         END-IF
     END-PERFORM.
```

```
*
*  Blabber about how much we got done, then close both files. Note
*  that we ignore the status on closing the input file, but do report
*  a problem with the output, since that may result in the underlying
*  element not being created.
*
 CLOSE-FILES.
     CALL "CIFS$PRINTF" USING BYTES-WRITTEN,
                              COPY-FORMAT,
                              BYTES-TOTAL.
     CALL "CIFS$CLOSE" USING READ-FD,
                             CIFS-STATUS.
     CALL "CIFS$CLOSE" USING WRITE-FD,
                             CIFS-STATUS.
     IF CIFS-STATUS NOT = 0
         CALL "CIFS$PERROR" USING CIFS-STATUS,
                                  CLOSE-ERROR-MESSAGE
     END-IF.
 COPY-DONE.
     EXIT.
*
*  Section to display the entries in a directory.
*
 LIST-DIRECTORY SECTION.
*
*  Save the current directory, and switch to the requested one.
*
 CHANGE-CURRENT-DIRECTORY.
     CALL "CIFS$GETCWD" USING DATA-BUFFER,
                              131072,
                              CIFS-STATUS.
     IF CIFS-STATUS NOT = 0
         CALL "CIFS$PERROR" USING CIFS-STATUS,
                                  GETCWD-ERROR-MESSAGE
         GO TO LIST-DONE
     END-IF.
     CALL "CIFS$CHDIR" USING ARGUMENT-1,
                             CIFS-STATUS.
     IF CIFS-STATUS NOT = 0
         CALL "CIFS$FPRINTF" USING BYTES-WRITTEN,
                                   CIFS-STDERR-FILE,
                                   CHDIR-ERROR-FORMAT,
                                   ARGUMENT-1
         CALL "CIFS$PERROR" USING CIFS-STATUS
         GO TO LIST-DONE
     END-IF.
*
*  Open the requested directory. Display a message and get out if there
*  are any errors.
*
 OPEN-DIRECTORY.
     CALL "CIFS$OPENDIR" USING DOT-NAME,
```

```
                                CIFS-DIR,
                                CIFS-STATUS.
        IF CIFS-STATUS NOT = 0
            CALL "CIFS$FPRINTF" USING BYTES-WRITTEN,
                                      CIFS-STDERR-FILE,
                                      OPEN-ERROR-FORMAT,
                                      ARGUMENT-1
            CALL "CIFS$PERROR" USING CIFS-STATUS
            GO TO RESTORE-CURRENT-DIRECTORY
        END-IF.
*
*  Scan the directory, do stat() and istat() on each entry, and
*  display the info.
*
 READ-DIRECTORY.
        PERFORM TEST AFTER UNTIL BYTES-READ = 0
            CALL "CIFS$READDIR" USING CIFS-DIR,
                                INODE-NUMBER,
                                ENTRY-NAME,
                                4095,
                                BYTES-READ,
                                CIFS-STATUS
            MOVE nul to ENTRY-BYTE(BYTES-READ + 1)
            IF BYTES-READ <= 0
                IF CIFS-STATUS NOT = 0
                    CALL "CIFS$PERROR" USING CIFS-STATUS,
                                             DIRECTORY-ERROR-MESSAGE
                END-IF
                GO TO CLOSE-DIRECTORY
            END-IF
            CALL "CIFS$STAT" USING ENTRY-NAME,
                                   CIFS-STAT,
                                   CIFS-STATUS
            IF CIFS-STATUS = 0
                CALL "CIFS$PRINTF" USING BYTES-WRITTEN,
                                         STAT-ENTRY-FORMAT,
                                         ENTRY-NAME,
                                         CIFS-STAT-MODE,
                                         CIFS-STAT-INO,
                                         CIFS-STAT-NLINK,
                                         CIFS-sTAT-SIZE
                CALL "CIFS$ISTAT" USING CIFS-STAT-INO,
                                        CIFS-ISTAT,
                                        CIFS-ISTAT-SIZE,
                                        CIFS-STATUS
                IF CIFS-STATUS = 0
                    IF CIFS-ISTAT-CONTAIN > SPACES
                        CALL "CIFS$PRINTF" USING
                          BYTES-WRITTEN,
                          ISTAT-FILE-FORMAT,
                          CIFS-ISTAT-DIRECTORY,
                          CIFS-ISTAT-CONTAIN,
```

```
                    CIFS-ISTAT-CYCLE
                END-IF
                IF CIFS-ISTAT-ELEMENT-NAME > SPACES
                    CALL "CIFS$FPUTC" USING '.',
                                            CIFS-STDOUT-FILE,
                                            CIFS-STATUS
                    IF CIFS-STATUS NOT = 0
                        CALL "CIFS$PERROR" USING CIFS-STATUS
                        STOP RUN
                    END-IF
                    CALL "CIFS$FPUTS" USING
                      CIFS-ISTAT-ELEMENT-NAME,
                      CIFS-STDOUT-FILE,
                      BYTES-WRITTEN,
                      CIFS-STATUS
                    IF CIFS-ISTAT-ELEMENT-VERSION > SPACES
                        CALL "CIFS$FPUTC" USING '/',
                                                CIFS-STDOUT-FILE,
                                                CIFS-STATUS
                        CALL "CIFS$FPUTS" USING
                          CIFS-ISTAT-ELEMENT-VERSION,
                          CIFS-STDOUT-FILE,
                          BYTES-WRITTEN,
                          CIFS-STATUS
                    END-IF
                    CALL "CIFS$PRINTF" USING
                      BYTES-WRITTEN,
                      ISTAT-TYPE-FORMAT,
                      CIFS-ISTAT-ELEMENT-TYPE,
                      CIFS-ISTAT-ELEMENT-SUB-TYPE
                END-IF
                CALL "CIFS$PRINTF" USING BYTES-WRITTEN,
                                         ISTAT-OWNER-FORMAT,
                                         CIFS-ISTAT-OWNER,
                                         CIFS-ISTAT-ACR
            ELSE
                CALL "CIFS$FPRINTF" USING BYTES-WRITTEN,
                                          CIFS-STDERR-FILE,
                                          ISTAT-ERROR-FORMAT,
                                          ENTRY-NAME,
                                          CIFS-STAT-INO
                CALL "CIFS$PERROR" USING CIFS-STATUS
            END-IF
        ELSE
            CALL "CIFS$FPRINTF" USING BYTES-WRITTEN,
                                      CIFS-STDERR-FILE,
                                      STAT-ERROR-FORMAT,
                                      ENTRY-NAME
            CALL "CIFS$PERROR" USING CIFS-STATUS
        END-IF
    END-PERFORM.
*
```

```
*  Close the directory we were reading. Ignore any errors.
*
 CLOSE-DIRECTORY.
     CALL "CIFS$CLOSEDIR" USING CIFS-DIR,
                                CIFS-STATUS.
*
*  Reestablish the directory we were in when we started.
*
  RESTORE-CURRENT-DIRECTORY.
     CALL "CIFS$CHDIR" USING DATA-BUFFER,
                             CIFS-STATUS.
     IF CIFS-STATUS NOT = 0
         CALL "CIFS$FPRINTF" USING BYTES-WRITTEN,
                                   CIFS-STDERR-FILE,
                                   CHDIR-ERROR-FORMAT,
                                   DATA-BUFFER
         CALL "CIFS$PERROR" USING CIFS-STATUS
     END-IF.
 LIST-DONE.
     EXIT.
 END PROGRAM CIFS-TEST.
```

Following the compilation, the program can be executed. In order to list the contents of the OS 2200 file syslib*ucob, perform the following commands:

```
@xqt object-module
ls /os2200/sys$lib$/ucob
```

This should result in output that looks as follows:

```
.                  000000040777   455956     2        2983
 STD#sys$lib$*ucob(53)  lewis/PUBLIC
..                 000000040777        9  1062       38413
 cifs-admin/PRIVATE
ucob-fgss..abs     100001100777   767829     1     2907640
 STD#sys$lib$*ucob(53).ucob-fgss (6.1)  lewis/PUBLIC
ucob-fgssf..abs    100001100777   768187     1     2975736
 STD#sys$lib$*ucob(53).ucob-fgssf (6.1)  lewis/PUBLIC
ucob..abs          100001100777   767896     1       72248
 STD#sys$lib$*ucob(53).ucob (6.3)  lewis/PUBLIC
ivp                100002100777   767910     1       14224
 STD#sys$lib$*ucob(53).ivp (1.9)  lewis/PUBLIC
solar$sgs          100002100777   767894     1        6832
 STD#sys$lib$*ucob(53).solar$sgs (1.1)  lewis/PUBLIC
deact-scss         100002100777   767895     1        1568
 STD#sys$lib$*ucob(53).deact-scss (1.1)  lewis/PUBLIC
```

In order to copy the file syslib*ucob.ivp to another location, perform the following commands:

```
@xqt object-module
cp /os2200/sys$lib$/ucob/ivp /os2200/my/file/ivp
```

This should result in the following output:

```
12674 bytes copied
```

The file syslib*ucob.ivp should now be copied to the file my*file.ivp.

The following example illustrates what will happen if the file that you are attempting to copy from does not exist:

```
@xqt object-module
cp /os2200/invalid/file/ivp /os2200/my/file/ivp
```

This should result in the following output:

```
*Error opening </os2200/invalid/file/ivp>: *ERROR CFS000002: Object
with specified name does not exist.
```

Section 18
XML Generator

Generating an XML file that describes the layout of data in a COBOL 01 record can be used to process data when it is accessed outside of a COBOL program. For example, there might be a need to process record data written to an OS 2200 file on a Windows or Linux system. The information in the generated XML file can then be used to interpret the record data.

18.1. Creating an XML File

UCS COBOL will create an XML file for a specified 01 data record with the use of the keyword option XML/*name*, where *name* is the name of the file that contains the record names for which XML files are to be produced. Optionally, this file can contain the CIFS path location of the XML file and an alternate name (different from the record name).

18.1.1. Create Record Name File

Assume that the COBOL program, cobex, contains the following data descriptions in Working-Storage:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. cobex.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 cobrec1.
    02 dn1  PIC XXX.
    02 dn2  PIC 99.
01 cobrec2.
    02 dn3  PIC S99v99.
    02 dn4  PIC $$$9.9.
01 cobrec3.
    02 dn5  PIC 9(10) BINARY.
    02 dn6  PIC X.
PROCEDURE DIVISION.
BEGIN.
     :
     :
```

To create an XML file (it will have .xsd extension) for the three 01 data records in this program, a file must be created that specifies these record names. In addition, this file can optionally contain the CIFS location and alternate name of the resultant .xsd file. Let's create a text file called *recnames* and place the following information into it:

```
cobrec1
cobrec2 /os2200/bek/xmlwork/xyz
cobrec3 abc
```

In this example, .xsd files for the COBOL record names cobrec1, cobrec2 and cobrec3 will be generated. Since no CIFS path was specified for cobrec1, by default, the file cobrec1.xsd will be placed into the CIFS current working directory (as setup by CIFSUT). The CIFS current working directory can be determined by using the *pwd* or *cd* command in the CIFSUT utility. The record name cobrec2 has an absolute path (path starts with a '/') associated with it so, the file xyz.xsd will be created in the path /os2200/bek/xmlwork (this path must exist otherwise an error will be issued). This file will contain the information for the COBOL record cobrec2. The record name cobrec3 has a relative path (path does NOT start with a '/') associated with it, so the file abc.xsd will be created and placed into the CIFS current working directory.

18.1.2. Pre-compilation Setup

Before the compilation can be performed, the CIFS current working directory must be established and the file (created above) that contains the record names to process must be placed into it. To set the CIFS current working directory, do the following:

```
>@cifsut
CIFSUT 8R2Q1 (2014-Jul-29 20:27:13)  2015-Feb-03 Tue 11:41:59
>cd /os2200/bek/xmldump
>pwd
/os2200/bek/xmldump
>ls
cobex
```

The *pwd* command shows that the CIFS current working directory has been set to /os2200/bek/xmldump. In OS 2200 terminology this is the program file bek*xmldump. The *ls* command displays the contents of that directory, which is the COBOL source program cobex (see Section 18.1.2) that is to be compiled.

Now the file (recnames) that contains the data records to process must be copied into the CIFS current working directory. There are several ways to do this (@copy,s, CIFSUT *cp* command, or copy/paste from Windows Explorer). The following illustrates using the CIFSUT *cp* command:

```
>@cifsut
>cp /os2200/q/f/recnames  /os2200/bek/xmldump/recnames
>ls /os2200/bek/xmldump
cobex    recnames
```

As you can see, the directory now contains the COBOL source program (cobex) to compile as well as the file (recnames) that contains the record names to process.

18.1.3. Compile the COBOL Program

The program must now be compiled in order to get XML (.xsd) files created. The file containing the record names to process must be specified using the XML keyword option. This can be done as follows:

```
>@ucob bek*xmldump.cobex,om,,,xml/recnames
UCOB- 12.81/ EM (150126) LSS- 14.136/ 2200/8030 (150115) 2015 Feb 03 1155:17
SIZES: CODE(EM6): 72 DATA: 110END UCOB- 0 ERRORS(MAJOR) 0 ERRORS(MINOR)
```

Now, use CIFSUT to look at the contents of the directories:

```
>@cifsut
CIFSUT 8R2Q1 (2014-Jul-29 20:27:13)  2015-Feb-03 Tue 11:55:40
>ls /os2200/bek/xmldump
abc.xsd   cobex   COBREC1.xsd     recnames
>ls /os2200/bek/xmlwork
xyz.xsd
```

Upon completion of the compilation, three .xsd files were created as requested by the recnames file. The *ls* command above displays the contents of these directories. The .xsd file for the data record cobrec1 (COBREC1.xsd) is placed into the CIFS current working directory (/os2200/bek/xmldump) by default. The .xsd file for the data record cobrec3 (abc.xsd) is also placed into the CIFS current working directory because its location in the record name file was specified as a relative location within the CIFS current working directory. The .xsd file for the data record cobrec2 (xyz.xsd) is placed in directory /os2200/bek/xmlwork/xyz because the location in the record name file was specified as an absolute location.

NOTE: *The output object module of the compilation cannot be directed to the CIFS current working directory (cwd) since the compiler assigns it exclusively, making it unavailable to CIFS. CIFS I/O is used to read the record name text file and write the .xsd documents, so that no other run can have the program file, which is the CIFS current working directory, assigned exclusively. This includes the run doing the UCOB compilation.*

18.2. Example Output

The following is the contents of the .xsd file created for the data record cobrec1:

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
////////////////////////////////////////////////////////////////////////////////////////
//                                                                                    //
//                      Copyright © 2016 Unisys Corporation.                          //
//                                                                                    //
//                              All rights reserved.                                  //
//                                                                                    //
//                              UNISYS CONFIDENTIAL                                   //
```

```
//                                                                                    //
////////////////////////////////////////////////////////////////////////////////////////
-->
<!--*********************************************************-->
<!--* This document was generated by a tool.                *-->
<!--* Changes to this file may cause incorrect behavior and *-->
<!--*  will be lost if the document is regenerated.         *-->
<!--*********************************************************-->

<xs:schema xmlns:rs="urn:unisys.com.%7Becd49548-c8e3-4cb6-c981-0506041756ff%7D.record"
xmlns:cg="uri:unisys.com/Enterprise%20Portal/common-types"
targetNamespace="urn:unisys.com.%7Becd49548-c8e3-4cb6-c981-0506041756ff%7D.record"
elementFormDefault="qualified" attributeFormDefault="qualified" blockDefault="#all"
finalDefault="#all" version="1.5" xml:lang="en" xmlns:xs="http://www.w3.org/2001/XMLSchema">
<!--Common types and their attributes-->
 <xs:import namespace="uri:unisys.com/Enterprise%20Portal/common-types"
schemaLocation="../../../shared/common-types-schema.xsd"/>
<xs:annotation>
  <xs:documentation xml:lang="en-us">Schema generated for record COBREC1 in file working-
storage in program COBEX from source /os2200/BEK/XMLDUMP/COBEX Generated on  2015-02-
03T11:55:17.2734458-06:00</xs:documentation>
</xs:annotation>

 <!--Source record info-->
 <xs:complexType name="record_identity_type">
  <xs:simpleContent>
   <xs:extension base="xs:string">
    <xs:attribute name="unique_id" type="xs:string" fixed="{ecd49548-c8e3-4cb6-c981-
0506041756ff}"/>
    <xs:attribute name="source_path" type="xs:string" fixed="/os2200/BEK/XMLDUMP/COBEX"/>
<xs:attribute name="program_id" type="xs:string" fixed="COBEX"/>
    <xs:attribute name="file_name" type="xs:string" fixed="working-storage"/>
    <xs:attribute name="record_name" type="xs:string" fixed="COBREC1"/>
    <!--COMS usage = None-->
    <xs:attribute name="usage" type="xs:int" fixed="0"/>
    <xs:attribute name="record_size_list" type="xs:string" fixed="45"/>
    <xs:attribute name="generation_timestamp" type="xs:dateTime" fixed="2015-02-
03T11:55:17.2734458-06:00"/>
    <xs:attribute ref="cg:decimal_point_is_comma" fixed="false"/>
    <xs:attribute ref="cg:currency_sign" fixed="$"/>
   </xs:extension>
  </xs:simpleContent>
 </xs:complexType>
 <!--Generated types-->
 <!--Type for group COBREC1.-->
 <xs:complexType name="COBREC1_type">
<xs:annotation>
   <xs:documentation xml:lang="en-us">Complete type for group COBREC1.</xs:documentation>
</xs:annotation>
  <xs:sequence>
   <xs:element name="DN1" type="rs:DN1_type"/>
   <xs:element name="DN2" type="rs:DN2_type"/>
```

```
 </xs:sequence>
 <xs:attribute ref="cg:level" fixed="1"/>
 <xs:attribute ref="cg:size" fixed="45"/>
 <xs:attribute ref="cg:text" fixed=" "/>
 <xs:attribute ref="cg:offset" fixed="0"/>
</xs:complexType>
<!--Types for field DN1.-->
<xs:simpleType name="DN1_base_type">
 <xs:restriction base="xs:string">
  <xs:maxLength value="3"/>
  <xs:whiteSpace value="preserve"/>
 </xs:restriction>
</xs:simpleType>
<xs:complexType name="DN1_type">
 <xs:annotation>
  <xs:documentation xml:lang="en-us">Complete type for field DN1.</xs:documentation>
 </xs:annotation>
 <xs:simpleContent>
  <xs:extension base="rs:DN1_base_type">
   <xs:attribute ref="cg:level" fixed="2"/>
   <xs:attribute ref="cg:picture" fixed="2"/>
   <xs:attribute ref="cg:size" fixed="27"/>
   <xs:attribute ref="cg:text" fixed=" "/>
   <xs:attribute ref="cg:usage" fixed="5"/>
   <xs:attribute ref="cg:offset" fixed="0"/>
  </xs:extension>
 </xs:simpleContent>
</xs:complexType>
<!--Types for field DN2.-->
<xs:simpleType name="DN2_base_type">
 <xs:restriction base="xs:nonNegativeInteger">
  <xs:totalDigits value="2"/>
 </xs:restriction>
</xs:simpleType>
<xs:complexType name="DN2_type">
 <xs:annotation>
  <xs:documentation xml:lang="en-us">Complete type for field DN2.</xs:documentation>
 </xs:annotation>
 <xs:simpleContent>
  <xs:extension base="rs:DN2_base_type">
   <xs:attribute ref="cg:level" fixed="2"/>
   <xs:attribute ref="cg:picture" fixed="5"/>
   <xs:attribute ref="cg:size" fixed="18"/>
   <xs:attribute ref="cg:text" fixed=" "/>
   <xs:attribute ref="cg:usage" fixed="5"/>
   <xs:attribute ref="cg:offset" fixed="27"/>
  </xs:extension>
 </xs:simpleContent>
</xs:complexType>
<!--Type for record document.-->
<xs:element name="document">
 <xs:complexType>
```

```
    <xs:sequence>
     <xs:element name="COBREC1" type="rs:COBREC1_type"/>
    </xs:sequence>
   </xs:complexType>
  </xs:element>
</xs:schema>
```

18.3. XML File names

CIFS is used to create XML files. The CIFS file names may be based on a record name or on a CIFS path specified in the XML keyword option file. The XML files are ultimately kept in an OS 2200 program file as symbolic elements, always having a version name of xsd. The element name used by CIFS may not be exactly the same as the name you specified, instead, it may be truncated due to length or some characters may be changed due to element name restrictions. For example, a data name that contains a dash (-) will have the dash changed to an underscore (_). In addition, if a data name begins with a digit, an underscore is appended to the front of the name. A CIFSUT *ls* command on the directory will show the actual name that was created, but a FURPUR @prt,t command may show a truncated or modified name.

18.4. Back-to-Back COBOL Programs

If the COBOL source contains back-to-back programs (multiple compilations in a single source) the XML/*name* keyword option will be in effect for each compilation. If the same 01 record name exists in each compilation unit, then the PROGRAM-ID of the program currently being compiled will be appended to the record name. For example, if the record name being processed is rec1 and it exists in each compilation unit then two .xsd files will be generated. If the PROGRAM-ID of one compilation unit is PROG1 then the file rec1-prog1.xsd will be created. If the PROGRAM-ID of the second compilation unit is PROG2, then the file rec1-prog2.xsd will be created.

For additional information on back-to-back programs, see Section 2.4 Separate Compilations for a Single Source Element (Back-to-Back COBOL Programs).

18.5. Nested COBOL Programs

A COBOL source program can contain nested programs (one compilation unit with one or more programs embedded within the main program). If the same 01 record name is used in more than one of these programs, only the first occurrence in the source will cause an .xsd XML file to be generated.

Appendix A
Compilation-Time Diagnostic Messages

This appendix describes the diagnostic messages produced by the Universal Compiling System (UCS) compiler during the compilation process, as follows:

- UCS COBOL front-end compiler messages (see A.3)

  These messages have the prefix UCOB on listings.
- UCS Language Support System (LSS) messages (see A.4)

  These messages have the prefix LSS on listings.

This appendix also describes how to use the FIPS flagging facility to monitor how closely a program conforms to American National Standard COBOL (see A.2). Appendix B lists the diagnostic messages produced during program execution.

A.1. Diagnostic Message Types and Format

All messages are in one of the following three classes, in decreasing order of severity:

- Error
- Warning
- Remark

Each message class contains one or more subclasses. Table A-1 shows the classification of messages in decreasing order of severity and describes the meaning of each subclass.

Format

**message-class*(*message-subclass*) *product-name-message-number: message-text*

Examples

```
*ERROR(MAJOR) UCOB1200: An illegal combination of keywords has been
encountered

*WARNING(USAGE) UCOB1437: Numeric literal must be unsigned integer

*ERROR(FATAL) UCOB7012: RSA fatal error following
```

Note: *The message numbers have the prefix UCOB to distinguish them from execution messages produced by components of the Language Support System (LSS).*

```
*ERROR(MINOR) UCOB422: variable-name is illegally terminated with a
hyphen -
it is processed with the hyphen included
```

Note: *Italicized words in some messages indicate variables that are replaced by actual names or data when printed on the listing.*

Table A–1. Diagnostic Message Classes and Subclasses

Class	Subclass	Meaning
ERROR	INTERNAL	An internal error means the compilation terminated due to internal problems such as table overflow or the inability to allocate more storage. Ask your site administrator to submit a User Communication Form (UCF).
	FATAL	A fatal error is nonrecoverable. It is a user-caused error that means the compiler exceeded its limits, resulting in premature termination.
	MAJOR	A major error indicates that the compiler cannot reasonably assume your intent or correct the error.
	MINOR	A minor error indicates a usage or omission error. The compiler makes a reasonable assumption of your intent, corrects the problem, and continues compiling. However, further diagnostics can result.
WARNING	USAGE	A usage warning indicates an instance of questionable usage that nevertheless does not violate the language specifications.
	LEVEL-type	A level warning indicates the compiler detects a nonconforming feature. The compiler issues these messages only if you specify one of the compiler keyword options for level.
REMARK	CLARIFICATION	A clarification remark can be either informational or used to identify an anomaly which is not necessarily an error but may need to be modified. The compiler issues these messages if you specify the compiler letter option E, L, or S or the keyword option REMARK.

The compiler always issues messages of class ERROR. You cannot suppress printing them with a compiler keyword or letter option. The compiler uses the run-condition word to indicate the occurrence of major, fatal, or internal errors. See 2.5 for a description of the run-condition word in this context.

The compiler always issues messages of class ERROR. You cannot suppress printing them with a compiler keyword or letter option.

The compiler does not produce an object module in the following cases:

- The program has any fatal or nonrecoverable internal errors.
- The program has any major errors while either the ERREXIT or ERRCHECK compiler keyword option is in effect.

In all listings, diagnostic messages consist of groups of the following lines:

1. The diagnostic message
2. A line containing the pointer character "v" in the column where the error begins, if appropriate for the message
3. The source line that contains the error, if possible, including the line number (source lines that are too long are wrapped to subsequent lines)

If a source line contains more than one error, the first two lines of the group are repeated for each error, in column number order. Then the source line ends the group.

Note: *The message group contains the source line even if you specified the source not to be printed.*

Example

This example shows two errors in source statement 21.

```
*ERROR(MAJOR) UCOB1771: IDENTIFIER N2 is not a legal operand when used
in a
relational expression - expression ignored
              v
*ERROR(MAJOR) UCOB1602: IDENTIFIER NAME is undefined
                                  v
21    IF n1 = n2 THEN DISPLAY name.
```

For easier reference, this appendix lists only message numbers and text. It does not include the class and subclass, since they appear on the listing.

A.2. COBOL FIPS Flagging

The UCS COBOL compiler contains a flagging facility that allows you to optionally monitor a source program at compilation time. You can specify the monitoring for the following:

- Any COBOL subset
- Optional modules
- All obsolete language elements

The monitoring is a comparison of the syntax used in the source program against the syntax included in the user-selected COBOL subset. The compiler diagnoses any syntax in the source program that does not conform to that included in the user-selected COBOL subset. The compiler identifies these differences through messages on the source program listing. The compiler also diagnoses and identifies through a message on the source listing any syntax for an obsolete language element that is included in the processor and used in the source program.

The organization of COBOL specifications defines 12 modules. These specifications require eight modules:

- Nucleus
- Sequential
- Relative
- Indexed
- Interprogram Communication
- Sort-Merge
- Source Text Manipulation
- Intrinsic Functions

The COBOL specifications define four modules that are optional:

- Report Writer
- Segmentation
- Debug
- Communication

UCS COBOL implements all of the required modules and two of the four optional modules, Report Writer and Segmentation.

Most modules contain either one or two levels. All components in the *UCS COBOL Programming Reference Manual Volume 1* that are not boxed are considered level 1 unless stated otherwise. Volume 1 identifies level 2 components by enclosing them in solid boxing. UCS COBOL also monitors nonstandard extensions (as denoted in Volume 1 by broken-line boxes). The modules and levels are arranged to create three subsets and a standard level. Table A-2 summarizes the relationship of the modules, levels, and subsets.

Table A–2. Modules, Levels, and Subsets

Module	Level Low	Level Intermediate	Level High	Level Standard
Nucleus (NUC)	1	1	2	2
Sequential (SEQ)	1	1	2	2
Relative (REL)	-	1	2	2
Indexed (INX)	-	1	2	2
Interprogram Communication (IPC)	1	1	2	2
Sort-Merge (SRT)	-	1	1	1
Source Text Manipulation (STM)	-	1	2	2
Intrinsic Functions (ITR)	-	1	1	1
Report Writer (RPW)	-	-	-	1
Segmentation (SEG)	-	-	-	2
Debug (DEB)	-	-	-	2
Communication (COM)	-	-	-	2

You choose the level at which the source program is monitored by specifying one of the following compiler keyword options on the compiler call:

Option	Description
LEVEL/STANDARD LEVEL	LEVEL/STANDARD and LEVEL issue warning messages for nonstandard or obsolete features used within the program.
NO-LEVEL (default)	NO-LEVEL suppresses level checking and warning messages.
LEVEL/LOW	LEVEL/LOW issues warning messages for nonstandard, obsolete, intermediate, high, and standard features.
LEVEL/INTERMEDIATE	LEVEL/INTERMEDIATE issues warning messages for nonstandard, obsolete, high, and standard features.
LEVEL/HIGH	LEVEL/HIGH issues warning messages for nonstandard, obsolete, and standard features.
LEVEL/OBSOLETE	LEVEL/OBSOLETE issues warning messages for obsolete features. The option STD-ERRCHECK sets LEVEL/STANDARD. Otherwise the default is NO-LEVEL.
LEVEL/SQL	LEVEL/SQL issues warning messages for nonstandard syntax or host-variable references within an embedded SQL command.
LEVEL/RPW	LEVEL/RPW issues messages for syntax that belong to the optional Report Writer module.
LEVEL/SEG	LEVEL/SEG issues messages for syntax that belong to the optional Segmentation Module.
LEVEL/SEG2	LEVEL/SEG2 issues messages for syntax that belong to level 2 of the Segmentation module.

The compiler issues the following diagnostic messages:

```
*WARNING(LEVEL) UCOB1101:  (subset) level syntax is NON-CONFORMING
STANDARD
```

The syntax is nonconforming standard if the nonconforming syntax is not included within the user-selected COBOL subset or optional modules. SUBSET indicates the COBOL subset in which the current syntax is allowed.

```
*WARNING(NONSTANDARD) UCOB1102:  Syntax is non-conforming non-standard -
UCOB
```

The syntax is nonconforming nonstandard if the nonconforming syntax is a nonstandard extension included in the processor.

```
*WARNING(LEVEL) UCOB1103:  (subset) level syntax does not conform to the
level desired
```

Duplicate of UCOB1101.

```
*WARNING(LEVEL) UCOB1104:  This item is in the Obsolete Language list
```

The syntax is obsolete if the syntax identified is in the obsolete category within a COBOL subset or optional module included in the processor.

A.3. UCS COBOL Front-End Compiler Messages (UCOB)

0	The complexity of the statement causes the look-ahead buffer used for parsing to overflow - it will be necessary to simplify the statement before the program can be compiled
	The statement is too complex, please reduce the complexity of the statement and recompile the program.
1	The complexity of the statement causes the output buffer to overflow - it will be necessary to simplify the statement before the program can be compiled
	The parser has recognized a token and would place it on the stack; however, there is no room for it on the stack.
2	The complexity of the statement causes the control buffer to overflow - it will be necessary to simplify the statement before the program can be compiled
	The parser has tried to enter a production; however, there is not enough room on the stack to hold the production stack entry. This normally means that the production stack is too small for the application.
6	The error limit of *integer-value* specified on the compiler keyword option MAXERRORS, has been exceeded
	The keyword option MAXERRORS/n was specified on the compile line. A total of n+1 major errors has been detected by the compiler. The compilation is terminated.
410	Continuation of an alphanumeric literal does not begin with a quotation mark - one is assumed
	A continued alphanumeric literal does not have a quotation mark before the first nonblank character. The quotation mark is assumed.
411	Illegal character in column 7 of the line - a blank is assumed
	An *, /, -, or D are the only valid characters in column 7. A blank is assumed to be there.
415	Continuation of previous line is not necessary - the hyphen in column 7 is treated as a blank
	The continuation hyphen was unnecessary and therefore ignored. The source code on the previous line had no need to be continued.
416	The previous literal string was not properly continued - an ending quotation mark is assumed after column 72
	The last word on the line is an alphanumeric literal with no ending delimiter, and the next line has no continuation hyphen. An ending quotation mark is assumed after column 72.
417	Input character encountered - it is ignored

	An illegal character was encountered while reading in the source program. It is ignored.
418	Closing quotation mark missing - one is supplied at end-of-file or end-of-copy
	The last word on the line is an alphanumeric literal with no ending delimiter, and the next line has no continuation hyphen. An ending quotation mark is assumed after column 72.
419	Debugging lines are not allowed within comment entries - the debugging line is treated as part of the comment
	A debugging line has been encountered while processing a comment entry. The debugging line is considered to be part of the comment entry.
420	Hyphen continuation is not allowed within comment entries - the continuation line is treated as part of the comment
	A continuation line has been encountered while processing a comment entry. The continuation line is considered to be part of the comment entry.
422	'*character-string*' is illegally terminated with a hyphen - it is processed with the hyphen included
	A user name must not end with a hyphen. If the hyphen is intended as a minus sign, it must be preceded by a blank. The hyphen is accepted as part of the name.
425	'*character-string*' is not followed by a space or a separator - a space is assumed
	This item must be followed by a proper separator. A space is assumed to follow the item.
426	The floating constant '*character-string*' is not followed by an integer characteristic - zero is assumed
	The exponent in a floating-point constant must consist of at least one digit. A zero is assumed.
429	Length of item exceeds buffer size - item truncated
	The length of the current item is greater than the buffer size. The size of the item is truncated.
430	An illegal COBOL character has been encountered - this character is ignored
	Illegal character has been encountered. The set of characters used to form character strings and separators includes the letters of the alphabet, digits and special characters. The complete character set can be found in *UCS COBOL Programming Reference Manual Vol. 1*.
433	Continuation line must begin in area B - treated as though it begins in area B
	Any sentence, entry, phrase, or clause can be continued by starting subsequent lines in area B.
437	Two '*character-string*' characters required here - second '*character-string*' assumed - a '*character-string*' character inserted into the quoted string
	The second of two consecutive delimiters is missing at the beginning of a continuation to insert a delimiter into a literal. The second delimiter is assumed and is put into the literal.
439	Only 31 levels of nesting are permitted or having more than 4095 blocks is not permitted

	The number of levels of nested programs is limited to 31. There is a limit of 4095 total programs (blocks) in a nest.
440	Maximum of 999 separate programs in the same compilation unit has been detected
	UCS COBOL allows up to 999 separate programs in the same compilation unit. The 1000th program will cause this error.
454	Literal size of zero - size is set to one
	The given literal has no characters within the quotes. The size of the literal is set to 1 to help the compiler continue.
500	Attempt recovery here after previous error
	The compiler has attempted to recover from the error at the indicated location.
501	The word SUBSCHEMA expected - not found
	The word SUBSCHEMA on the INVOKE clause was not found.
502	The subschema name is too long - maximum is 12 characters
	The subschema character string on the INVOKE exceeded 12 characters.
503	The word FILE expected - not found
	The word FILE on the INVOKE clause was not found.
504	The word SCHEMA expected - not found
	The word SCHEMA on the INVOKE clause was not found.
505	The word INVOKE expected - not found
	The word INVOKE on the KEY clause was not found.
506	The word HVTIP expected - not found
	The word HVTIP on the ENVIRONMENT clause was not found.
507	DATANAMES or RECORDS expected - not found
	Neither DATANAMES or RECORDS on the COPYING clause was found.
508	WORKING, COMMON or LINKAGE expected - not found
	Either WORKING, COMMON or LINKAGE must be input here but was not found.
509	The RUN-UNIT-ID name is too long - maximum is 6 characters
	The RUN-UNIT-ID character string on the RUN-UNIT-ID clause exceeded 6 characters.
510	Integer expected - not found
	An integer is needed here but was not found.
511	The PRIORITY integer is too long - maximum 10 digits
	The PRIORITY digit string on the PRIORITY clause exceeded 10 digits.
512	COMMAND, RUN-UNIT or DEFERRED-UPDATES expected, not found
	Either COMMAND or RUN-UNIT or DEFERRED-UPDATES must be input here but was not found.
513	The word RUN-UNIT-STATISTICS expected, not found

	The word RUN-UNIT-STATISTICS on the DMCA clause was not found.
515	RUN-UNIT-ID clause already specified - ignored
	A duplicate RUN-UNIT-ID clause was found on the INVOKE clause; this one is not used.
516	PRIORITY clause already specified - ignored
	A duplicate PRIORITY clause was found on the INVOKE clause; this one is not used.
517	RECORD clause already specified - ignored
	A duplicate RECORD clause was found on the INVOKE clause; this one is not used.
518	SAVE clause already specified - ignored
	A duplicate SAVE clause was found on the INVOKE clause; this one is not used.
519	DMCA clause already specified - ignored
	A duplicate DMCA clause was found on the INVOKE clause; this one is not used.
520	Absolute subschema element not found
	The compiler was unable to locate the subschema element in the INVOKE file.
521	Schema invoked not equal to schema in subschema element
	The schema name on the INVOKE does not match the schema name in the absolute subschema SFD table.
522	Subschema invoked not equal to subschema in subschema element
	The subschema name on the INVOKE does not match the subschema name in the absolute subschema SFD table.
523	File invoked not equal to file in subschema element
	The file name on the INVOKE does not match the file name in the absolute subschema SFD table.
524	Invoke key not equal to subschema lock
	The invoke key on the INVOKE clause does not match the subschema lock in the SSCD table.
525	There is no record or data name entry point in the subschema S$PROC
	The entry point for either the subschema records or data names in S$PROC does not exist.
527	The WORKING option cannot be used, explicitly or implicitly, when the LINKAGE option is used in INVOKE clause - assume LINKAGE
	If the LINKAGE option is specified in any one of the COPYING, RECORD, or DMCA clauses of the INVOKE clause, the WORKING option, explicitly or implicitly, cannot be specified in any of these clauses.
528	The DMS interface feature (COBDMS) is not installed
	An illegal attempt is being made to use the DMS interface. Installation of the DMS interface (COBDMS) allows the use of DMS commands.
529	When the ENVIRONMENT IS HVTIP is used, the DMS records, datanames, DMCA and RDA all must be defined to be in Common-Storage - Common-Storage is assumed

	The ENVIRONMENT IS HVTIP clause was specified on the INVOKE statement and either the DMS records, data names, DMCA or RDA was not specified as Common.
1001	String '*character-string*' illegal in COPY statement
	An illegal or incorrect item was encountered while processing a COPY statement.
1002	More than 50 nested COPY statements are not allowed
	The number of nested COPY statements cannot exceed 50.
1004	Pseudo-text-1 is null or contains only COMMAS or SEMICOLON - COPY or REPLACE statement is ignored
	In a COPY REPLACING or REPLACE statement that contains pseudo-text, pseudo-text-1 must not be null or contain only periods, commas, or semicolons. Pseudo-text-1 is illegal in this statement, therefore the COPY REPLACING or REPLACE statement is ignored.
1005	Illegal use of a nested COPY
	A nested COPY statement has been illegally used. A COPY statement with the REPLACING phrase cannot contain another COPY statement. A COPY statement may not contain a REPLACING phrase if it is nested within another COPY statement. A COPY may not be used within the pseudo text of a REPLACING phrase.
1006	REPLACE statement must be preceded by a period - statement accepted
	A REPLACE statement can occur anywhere in the source program where a character string can occur. A separator period must precede it unless it is the first statement in a separately compiled program.
1007	'*character-string*' exceeds the implementation limit for procedure names - it is truncated
	The length of the procedure name is greater than the length allowed by the implementation. It is truncated to 30 characters.
1008	'*character-string*' exceeds the implementation limit for library names - it is truncated
	The length of the library name is greater than the length allowed by the implementation. It is truncated to 30 characters.
1009	The keyword BY was expected in the COPY or REPLACE statement - COPY or REPLACE statement is ignored
	The reserved word BY is expected at this point in a COPY REPLACING or REPLACE statement. The COPY REPLACING or REPLACE statement is ignored.
1013	COPY or REPLACE statement must end with a period - COPY or REPLACE statement is ignored
	A COPY or REPLACE statement must end with a period. A period is missing from this statement. The COPY or REPLACE statement is ignored.
1014	Pseudo-text required in REPLACE statement - REPLACE statement is ignored
	A REPLACE statement requires that pseudo-text be used for the replacement. The pseudo-text was missing from this REPLACE statement. The REPLACE statement is ignored.

1017	Incorrect request for a replacement line - request is ignored
	A COPY or REPLACE statement that is improper or has unrecognizable syntax.
1018	Incorrect qualification of identifier in the REPLACING clause of the COPY statement - the COPY statement is ignored
	The REPLACING clause of a COPY statement can use an identifier and that identifier can be qualified. In this situation, qualification is incorrectly specified resulting in an illegal identifier. The COPY statement is ignored.
1019	Illegal subscripting in a REPLACING clause - the REPLACING clause is ignored
	Illegal subscripting syntax was used on the BY identifier of the REPLACING clause.
1020	Closing delimiter for REPLACING pseudo-text is missing - REPLACING is ignored
	The closing delimiter, two equal signs (==), for a pseudo-text in a REPLACE or COPY REPLACING statement is missing. The REPLACING statement is ignored.
1021	Illegal use of ';', end-of-program or paragraph-name as end delimiter of embedded SQL statement; end delimiter assumed
	An embedded SQL statement must be terminated by an END-EXEC token. If the end of the program or a paragraph-name (characters in columns 8-11) are encountered for the same situation, processing is as if 'END-EXEC' were encountered. Any tokens following the semicolon or the paragraph name (including the paragraph name) are not considered to be within an embedded SQL statement (including an END-EXEC token) until another EXEC SQL token sequence is encountered.
1022	Missing syntax in COPY statement - COPY statement is ignored
	A COPY REPLACING statement was encountered that contained incomplete syntax. The statement should contain syntax for the replacement on each side of the word BY.
1023	Illegal data type - USAGE clause is ignored
	The use of USAGE BINARY-1 is illegal for an IBM tape file data item.
1101	*character-string* level syntax is NON-CONFORMING STANDARD
	The syntax used is not in the COBOL level specified by the user.
1102	Syntax is nonconforming nonstandard - UCOB
	This syntax is a nonstandard extension included in the compiler.
1103	*character-string* level syntax does not conform to the level desired - *character-string*
	The syntax used is not in the COBOL level specified by the user.
1104	This item is in the Obsolete Language list
	The items on the Obsolete Language list will be removed completely in a future standard.
1105	ALL literal to numeric/numeric edited is on the Obsolete Language list
	The items on the Obsolete Language list will be removed completely in a future standard.
1106	This statement is not implemented - statements within this section will be treated as comments

	The USE FOR DEBUGGING ON statement is an obsolete statement. It is treated as a comment line.
1107	A figurative constant cannot be used as an identifier in a CALL statement
	Figurative constants such as ZERO, SPACES, HIGH-VALUES, etc. cannot be used as an identifier in a CALL statement.
1108	RPW syntax is nonconforming STANDARD
	The specified syntax belongs to the optional Report Write Module
1109	SEG syntax is nonconforming STANDARD
	The specified syntax belongs to the optional Segmentation Module
1110	SEG2 syntax is nonconforming STANDARD
	The specified syntax belongs to the optional Segmentation Module and is above level 1 Segmentation.
1111	Missing or invalid program name - END PROGRAM '*character-string*' assumed
	There was not an END PROGRAM for the end of the nested program being compiled. The END PROGRAM is assumed.
1200	An illegal combination of keywords has been encountered
	An illegal combination of keywords has been encountered from the Identification or Environment Division.
1201	An ambiguity in the Identification Division has been detected
	The indicated syntax was not expected to occur in the Identification Division.
1202	An ambiguity in the Environment Division has been detected
	The indicated syntax was not expected to occur in the Environment Division.
1203	'*character-string*' is illegal in the sequence of clauses permitted in the Identification Division
	The clause is not allowed at this position in the Identification Division.
1204	'*character-string*' is illegal in the sequence of statements permitted in the Identification Division
	The clause does not belong in the Identification Division.
1205	The keyword DIVISION is missing - it is assumed
	The keyword DIVISION is expected after the keyword IDENTIFICATION, ENVIRONMENT, DATA, or PROCEDURE.
1206	The program name is missing - a dummy name is generated
	The PROGRAM-ID clause was expected here or was started and not completed. A dummy name is generated.
1207	The keywords COMMON or INITIAL must occur here - neither is assumed
	There is no period after the name of the PROGRAM-ID; therefore the next word is expected to be either COMMON or INITIAL. Neither is assumed.
1209	A nested program cannot contain a Configuration Section

	The Configuration Section cannot be in this program because it is nested and assumes the configuration of its parent.
1210	An ambiguous sequence of keywords has been detected
	This information is not expected in the Environment Division.
1211	The keyword SECTION is missing - it is assumed
	The keyword SECTION is expected here after section name like FILE or WORKING-STORAGE.
1212	An ambiguity in the SOURCE-COMPUTER clause has been detected
	This information is not expected in the SOURCE-COMPUTER clause.
1213	The keyword MODE is missing - it is assumed
	If DEBUGGING is used in the SOURCE-COMPUTER clause, the keyword MODE is expected also.
1214	An ambiguity in the OBJECT-COMPUTER clause has been detected
	This information is not expected in the OBJECT-COMPUTER clause.
1215	An error in the MEMORY clause has been detected - the clause is ignored
	The MEMORY clause is incorrect.
1216	One of the keywords WORDS, CHARACTERS or MODULES must occur in the MEMORY clause - the clause is ignored
	The MEMORY clause must have one of the keywords WORDS, CHARACTERS, or MODULES.
1217	An illegal SEGMENT-LIMIT has been encountered - the clause is ignored
	The SEGMENT-LIMIT clause does not contain an integer or the integer is not within the limits from 1 to 99.
1220	An error in the COLLATING SEQUENCE clause has been detected - it is ignored
	This information is not expected in the COLLATING SEQUENCE clause.
1221	The required segment number is missing - the clause is ignored
	The SEGMENT-LIMIT clause does not contain an integer. It is ignored.
1222	CONTROL DIVISION is illegal in the statements preceding IDENTIFICATION DIVISION - ignored
	The CONTROL DIVISION is no longer allowed in a COBOL program. It is ignored.
1223	PROGRAM-ID is not found - a dummy name is generated
	The PROGRAM-ID is not given for this program. A dummy name is generated.
1224	'*character-string*' is illegal in the sequence of clauses permitted in the Input-Output Section
	This keyword is out of sequence in the Input-Output Section or does not belong here.
1225	An ambiguity in the SPECIAL-NAMES clause has been detected
	This information does not belong in the SPECIAL-NAMES clause. The clause is deleted.

1226	Illegal literal in CLASS clause
	Literal associated with the CLASS clause must be unsigned if numeric. A non-numeric literal used in the THRU clause must be one character long. A Kanji literal can be used only with other Kanji literals.
1230	The keyword SECTION is missing - it is assumed
	The keyword SECTION is expected after the keyword INPUT-OUTPUT.
1231	The FILE-CONTROL paragraph is missing - accepted
	The FILE-CONTROL paragraph is expected in the Input-Output Section.
1232	An ambiguity in the FILE-CONTROL paragraph has been detected
	This information is not expected in the FILE-CONTROL paragraph. The paragraph is deleted.
1233	There is no SELECT clause in the FILE-CONTROL paragraph - accepted
	At least one SELECT clause is required in the FILE-CONTROL paragraph.
1235	An ambiguity in the I-O-CONTROL paragraph is detected
	This information is not expected in the I-O-CONTROL paragraph. The paragraph is deleted.
1236	'*character-string*' is not a legal ASSIGN name
	The name associated with the ASSIGN is not a legal name. Legal names are ANS74-PRINTER, CARD-PUNCH, CARD-READER, CONSOLE, DISC, INTERCHANGE, PRINTER, and TAPE.
1237	'*character-string*' is illegal in the sequence of clauses permitted in the Environment Division
	This clause is not expected or is out of order in the Environment Division.
1238	Causes an ambiguity in the Configuration Section
	This information is not expected in the Configuration Section. Recover at the next expected information.
1239	Causes an ambiguity in the Input-Output Section
	This information is not expected in the Input-Output Section. Recover at the next expected information.
1240	'*character-string*' illegal in the sequence of clauses permitted in the Configuration Section
	This information is not expected in the Configuration Section. Recover at the next expected information.

1242 '*character-string*' is illegal in the sequence of clauses permitted in the SPECIAL-NAMES paragraph

This information is not expected in the SPECIAL-NAMES paragraph. Recover at the next expected information.

1243 '*character-string*' is illegal in the sequence of clauses permitted in the FILE-CONTROL paragraph

This clause is not expected or is out of order in the FILE-CONTROL paragraph.

1244 '*character-string*' is illegal in the sequence of clauses permitted in the I-O-CONTROL paragraph

This clause is not expected or is out of order in the I-O-CONTROL paragraph.

1245 '*character-string*' is illegal in the sequence of keywords permitted in the SELECT clause

This information is not expected or out of order in the SELECT clause.

1246 A '.' must appear after here - it is assumed

A period is expected to follow this statement. Since a period was not found, one is assumed.

1247 '*character-string*' is illegal here - END-PERFORM assumed

END-PERFORM is only permitted after a series of imperative statements of an inline PERFORM.

1248 '*character-string*' causes an error at this position in the imperative statement

This information is not permitted at this position in an imperative statement.

1249 A '.' must appear before here - it is assumed

A period is required at this point. One is added.

1250 Causes an ambiguity in the Data Division

The information is not expected at this point in the Data Division.

1251 '*character-string*' is illegal in the sequence of sections permitted in the Data Division - it is ignored

The name for a section is not one that is permitted in the Data Division.

1253 Causes an ambiguity in the File Section

The information given is not expected at this point in the File Section.

1254 Causes an ambiguity in the WORKING-STORAGE SECTION

The information given is not expected at this point in the WORKING-STORAGE SECTION.

1255 Causes an ambiguity in the LINKAGE SECTION

The information given is not expected at this point in the LINKAGE SECTION.

1256 Causes an ambiguity in the COMMUNICATION SECTION

The information given is not expected at this point in the COMMUNICATION SECTION.

1257 Causes an ambiguity in the REPORT SECTION

The information given is not expected at this point in the REPORT SECTION.

1258 Causes an ambiguity in the COMMON-STORAGE SECTION

The information given is not expected at this point in the COMMON-STORAGE

SECTION.

1259 `COMMUNICATION SECTION not implemented, the source is scanned but not processed`

The Communication Section has not been implemented. This diagnostic will be removed or changed when it is implemented.

1260 `END-IF not allowed when NEXT SENTENCE specified`

The scope terminator END-IF is not allowed if the NEXT SENTENCE clause has been specified within the IF statement.

1300 `Symbolic character has no associated integer value - symbolic character ignored`

This symbolic character has no associated integer value.

1301 `Integer has no associated symbolic character - integer ignored`

The integer reference is treated as a comment.

1302 `Integer out of legal range - symbolic character ignored`

The integer given as the value is either negative or is larger than permitted.

1303 `Symbolic character '`*`character-string`*`' duplicate - ignored`

A symbolic character name cannot require qualification. This one does.

1304 `Alphabet name '`*`character-string`*`' invalid - STANDARD-1 assumed`

The alphabet name is invalid. STANDARD-1 is assumed.

1305 `A floating point literal must not have more than 3 digits in its exponent`

This literal cannot be more than three digits because it is used as an exponent.

1306 `A comma is illegal in the definition of a numeric item - assume a decimal point`

A comma is not allowed in the definition of a numeric item. It is not allowed as a decimal point when the program does not contain the DECIMAL- POINT IS COMMA clause.

1307 `Decimal point should be a comma`

A period is not allowed as a decimal point when the program contains the DECIMAL-POINT IS COMMA clause.

1308 `Maximum length of 160 exceeded for this nonnumeric literal - 160 assumed`

A maximum length of 160 characters is allowed for a nonnumeric literal. This literal exceeds that limit. A size of 160 is assumed for this literal.

1309 `Duplicate USAGE or SIGN - this occurrence overrides previous`

The USAGE or SIGN has already occurred for this level number; the last occurrence will be used.

1401 `Duplicate '`*`character-string`*`' - it is ignored`

The given clause has already occurred for this level number.

1402 '*character-string*' is illegal in data definition - it is ignored

At this point a record description is expected.

1403 A '.' must appear after the '*character-string*' - one is assumed

A period is expected at this point, one is assumed.

1404 '*character-string*' is not legal at this position in a COBOL statement

At this point, either a section or paragraph name is expected.

1405 A '.' is missing in the line - one is assumed

A period is expected here.

1406 The specified syntax is illegal in the sequence of statements preceding IDENTIFICATION DIVISION - ignored

This was expected to be IDENTIFICATION DIVISION.

1407 Extraneous END PROGRAM is found

There is an extra END PROGRAM at this point.

1408 IDENTIFICATION DIVISION header is not found - source skipped

The IDENTIFICATION DIVISION header is not found. The source is skipped.

1409 The specified syntax is illegal in the sequence of statements following the END PROGRAM header

An END PROGRAM can be followed by only another END PROGRAM or IDENTIFICATION DIVISION of the next program.

1410 '*character-string*' causes error in DECLARATIVES section

Either a section name or a declarative statement should be here.

1411 An END DECLARATIVES is missing or has been ignored

The END DECLARATIVES line is either missing from the source or has been ignored because previous errors in the DECLARATIVES caused the remainder of the DECLARATIVES to be treated as the main body of the program.

1412 '*character-string*' causes error in USE statement

The USE statement is incorrect or an unexpected statement follows.

1413 '*character-string*' is illegal in data definition - it is ignored

COMP-3 is not allowed as a data type. To use COMP-3, specify the COMPAT/FULL OR COMP-BIN option. This option allows for compatibility between UCS COBOL and ASCII COBOL.

1414 The reserved word PROCEDURE is required in the syntax of the USE statement - it is assumed

The keyword PROCEDURE is required at this position in the syntax of a USE statement.

1415 The reserved word ERROR or EXCEPTION is expected at this point - ERROR is assumed

The keyword ERROR or EXCEPTION is required at this point in the USE statement. The keyword ERROR is assumed.

1416 `Unexpected syntax was found at this point in the USE statement - the keyword INPUT is assumed`

A file name or the keyword INPUT, OUTPUT, I-O, or EXTEND was expected at this point in the USE statement. Since none of them are present, the keyword INPUT is assumed.

1417 `The reserved word REPORTING is expected at this point - REPORTING is assumed`

The keyword REPORTING is required at this point in the USE statement. The keyword REPORTING is assumed.

1418 `The section name is missing following the DECLARATIVES statement - DECLARATIVES statement is ignored`

A section name must immediately follow the DECLARATIVES statement. Since the section name is missing, the DECLARATIVES statement is ignored and the source is compiled as if no DECLARATIVES appeared.

1419 `This SECTION name is out of sequence. The rest of the source is skipped.`

The REPORT SECTION must be the last SECTION in the DATA DIVISION. One of the other section names has been encountered at this point. Since this may create major problems specially for DMS programs, only lexical analysis will be performed on the rest of the source program.

1420 `Double negation is illegal - accepted`

The double negation symbols are illegal.

1421 `A right parenthesis, ')', must precede '`*`character-string`*`' to complete the previous subscript list - one is assumed`

The closing parenthesis is required to end the subscript.

1422 `A ':' must separate the beginning and ending values for reference modification - one is assumed`

The colon is required for reference modification.

1423 `A ')' must follow the ending value for reference modification - one is assumed`

The closing parenthesis is required to end a reference modification.

1425 `'`*`character-string`*`' occurs where a nonnumeric literal is required - the value is assumed to be within quotation marks`

A nonnumeric literal is expected.

1426 `'`*`character-string`*`' occurs where a nonnumeric literal is required - the statement is ignored`

A nonnumeric literal is expected, and there is nothing to convert to a nonnumeric literal.

1427 `Syntax is not legal at this position in a COBOL statement`

The symbol is illegal in the allowable combinations of arithmetic operators and unary symbols.

1428 A ')' must precede '*character-string*' to complete the previous expression - one is assumed

The number of right parentheses must equal the number of left parentheses in an arithmetic expression.

1430 '*character-string*' is illegal in an arithmetic expression - the entire expression is deleted

This makes the statement something other than an arithmetic expression.

1431 '*character-string*' is illegal in a conditional expression - the entire expression is deleted

A conjunction is required at this point in a conditional expression.

1432 '*character-string*' is illegal where a data reference should occur - the entire reference is deleted

A data reference should have been given at this point.

1433 Reference modification found where a subscript is required - the ':' is ignored and data items are used as subscripts and reference modification is ignored

The subscript is required before any reference modification. The colon, ':', is ignored along with reference modification. The data items are used as subscripts.

1434 '*character-string*' is illegal as the object of an EVALUATE statement - the statement is ignored

The object of an EVALUATE statement must be an identifier; literal; condition; arithmetic expression; or the keyword ANY, TRUE, or FALSE.

1435 '*character-string*' is illegal where a data reference or literal should occur - the entire reference is deleted

The statement is not complete. A data reference is required at this point.

1436 Syntax is illegal where a literal should occur - the entire reference is deleted

A literal is required at this point to help complete the statement.

1437 Numeric literal must be unsigned integer

If the literal associated with the DISPLAY statement is numeric, it must be an unsigned integer.

1438 Illegal syntax found in the arithmetic expression

The arithmetic expression being processed did not contain a legal combination of data items, literals and operators.

1439 '*character-string*' is illegal where a paragraph or section name should occur - the entire reference is deleted

A paragraph or section name is expected here; the given phrase is ignored.

1440 '*character-string*' is illegal where a paragraph or section name should occur - the entire reference is deleted

A paragraph or section name is expected here; the given phrase is ignored.

1441 A section name should appear here - it is assumed

A section name should appear at this point in the program.

1442 A paragraph name should appear here - it is assumed

A paragraph name should appear at this point in the program.

1450 '*character-string*' cannot be used with reference modification

Reference modification does not apply at this point.

1451 '*character-string*' does not have a maximum length specified

The length is missing from this reference modification.

1452 '*character-string*' is not specified as USAGE IS DISPLAY and cannot be used with reference modification

Reference modification is legal only with DISPLAY items.

1453 '*character-string*' is not an integer constant - it will be converted to an integer and used as the starting location for reference modification

The starting location for reference modification must be an integer. This literal is not an integer.

1454 '*character-string*' is not an integer variable - it will be converted to an integer and used as the starting location for reference modification

The starting location for reference modification must be an integer. This data name is not an integer.

1455 '*character-string*' cannot be used to specify the starting location for reference modification - a 1 is assumed

This cannot be used to specify a starting location in reference modification.

1456 '*character-string*' is beyond the ending location of the string - a 1 is assumed as the starting location for reference modification

The given starting location is larger than the size of the data item being modified.

1457 '*character-string*' is less than 1 and does not represent a legal starting location - a 1 is assumed as the starting location for reference modification

The given starting location is less than 1 for the data item being modified.

1458 Record description entry expected after FD or SD

The record description (01 level) is expected here.

1459 Report group description entry expected after RD

The report group description is expected here.

1462 '*character-string*' is not an integer constant – a 1 is assumed for the length for reference modification

The constant is not an integer for the length; 1 is assumed.

1463 '*character-string*' is not an integer variable – a 1 is assumed for the length for reference modification

The length variable is not an integer; 1 is assumed.

1464 '*character-string*' is not an integer variable – and will be converted to an integer for use as the length for reference modification

The length variable is not an integer but is converted to an integer for the length.

1465 '*character-string*' specifies a length which exceeds the length of the referenced item – the specified length is reduced to the remaining size of the item

The length is larger than the remaining size of the item; the remaining size is used.

1466 '*character-string*' is less than 1 and does not represent a legal length for the referenced item – the specified length is replaced by the remaining size of the item

The length is less than 1; the remaining size of the item is used.

1491 Neither original definition nor the redefinition can include a variable occurrence item

Neither the original definition nor the redefinition can include a variable occurrence
data item. Further description of the REDEFINES clause can be obtained from the Universal Compiling System (UCS) COBOL Programming Reference Manual Volume 1.

1492 An integer or literal PROGRAM-ID is allowed with the COMPAT option

The COMPAT option allows PROGRAM-ID to be an integer or literal in order to be compatible with ASCII COBOL. There is a list of the differences between UCS COBOL and ASCII COBOL in Appendix F.

1493 The COMPAT option does not make UCS COBOL fully compatible with ASCII COBOL – Check UCS COBOL PRM for differences

There is a list of the differences between UCS COBOL and ASCII COBOL in Appendix F.

1494 With the selected COMPAT option, data allocation is on word boundaries rather than byte boundaries

With the selected COMPAT option, data allocation is the same as ASCII COBOL.

1495 HIGH-VALUE is set to 0177

HIGH-VALUE is set to 0177 when one of the following options is used: COMPAT, COMPAT/FULL or COMPAT/HV177.

1496 HIGH-VALUE is set to 0377

HIGH-VALUE is set to 0377 when the option COMPAT/FULL377 is used.

1497 `The COMP item is treated as USAGE BINARY rather than DISPLAY`

Use USAGE BINARY rather than USAGE COMPUTATIONAL (COMP) for a binary item.

1498 `The COMP-3 item is treated as USAGE PACKED-DECIMAL`

For UCOB use USAGE PACKED-DECIMAL rather than USAGE COMPUTATIONAL-3 (COMP-3).

1499 `PIC 1 is treated as if USAGE BINARY-1 was present`

A PIC 1 item requires a USAGE of BINARY-1. With the COMPAT option, it is assumed.

1500 `An invalid compatibility level was specified on the COMPAT option - the COMPAT/FULL option is assumed`

An invalid level for the COBOL compiler was specified on the COMPAT option. This invalid level is ignored, and the COMPAT/FULL option will be assumed.

1503 `An invalid language level was specified on the LEVEL option - the STANDARD option is assumed`

An invalid level for the COBOL compiler was specified on the LEVEL option. This invalid level is ignored, and the STANDARD option will be assumed.

1504 '*character-string*' `name exceeds 30 characters - it is truncated to 30`

COBOL names cannot exceed 30 characters. Characters more than 30 are truncated.

1599 `A DMS record name has been duplicately defined`

The name of a DMS record has been duplicately defined by another 01 structure. DMS record names must be unique and cannot have the same name as another 01 structure defined elsewhere in the program. This duplicate definition may cause problems at a later point in the program.

1600 '*character-string*' `is not data name - name is ignored`

This name does not have any of the characteristics of a COBOL data name. Name is not processed.

1601 '*character-string*' `is not unique - first definition accepted`

A procedure name reference requires qualification for uniqueness. The first occurrence of the procedure name in the Procedure Division is used.

1602 '*character-string*' `is undefined`

This identifier is not defined anywhere in the source program but is referenced in the Procedure Division.

1603 '*character-string*' `is not procedure name - name is ignored`

This name must be a label, a procedure name or a section name. None of the above conditions is satisfied for this name. The name is ignored.

1604 '*character-string*' `is illegally qualified - name is ignored`

Qualification for this name is not correct. It is not part of the structure mentioned.

1606 '*character-string*' has too many qualifiers – name is ignored

Maximum number of qualifiers for a COBOL name is 50. The number of qualifiers for this name exceeds the maximum. The name is ignored.

1607 '*character-string*' is not a legal qualifier for a data name or a procedure name – statement ignored

A valid qualifier for a data name or a procedure name was expected.

1609 Reference '*character-string*' is undefined

This forward reference name has not been defined anywhere in the source program. The name is ignored.

1610 Reference '*character-string*' is not a data name – name is ignored

This forward reference name has none of the characteristics of a data name. The name is ignored.

1611 Reference '*character-string*' is not unique – first definition accepted

This forward reference name has already been defined. The first definition is accepted, and the second one is ignored.

1613 Reference '*character-string*' is not a procedure name – name is _ ignored

This forward reference name must be a label, a procedure name or a section name. None of these conditions are true for this name. The name is ignored.

1614 Reference '*character-string*' is illegally qualified – name is ignored

The qualification of this forward reference name is not correct. It is not part of the structure mentioned.

1615 '*character-string*' illegal in VALUE OF clause – clause is ignored

Incorrect implementor name is used in the VALUE OF clause of the file description. VALUE OF clause is not processed.

1616 Invalid file or element specified in literal – ignored

The literal is greater than 12 characters and contains invalid characters for a file or element name.

1617 '*character-string*' : data name or literal expected in VALUE OF clause

A data name or a literal is expected at this position in the VALUE OF clause of the file description. The VALUE OF clause is ignored.

1618 '*character-string*' is illegal in the sequence of clauses permitted in the file description – it is ignored

An incorrect clause was found while processing legal clauses of the file description. The incorrect clause is ignored.

1619 '*character-string*' duplicate definition of file name – duplicate is ignored

A duplicate definition is found for a previously defined file.

1620 '*character-string*' is not correct name type - name is ignored

A data name with specific definition is used in a statement or clause which requires some other type of definitions.

1622 RECORD DELIMITER clause is illegal for file organization - clause is ignored

The RECORD DELIMITER clause is only legal for files with the SEQUENTIAL organization.

1623 '*character-string*' is greater than 12 characters - name is accepted

The external name used for a file in the SELECT clause is greater than 12 characters. The compiler accepts the name.

1624 Illegal USAGE and/or SIGN for file with CODE-SET clause - DISPLAY and/or SIGN LEADING SEPARATE assumed

If the CODE-SET clause is specified for a file, all data in that file must be described as USAGE IS DISPLAY. All signed numeric data must be described with the SIGN IS LEADING SEPARATE clause or SIGN IS TRAILING SEPARATE clause.

1626 GLOBAL file cannot be specified in a SAME RECORD AREA clause - GLOBAL clause is ignored

If the SAME RECORD AREA clause is specified for several files, the record description entries or the file description entries for these files must not include the GLOBAL clause.

1627 Illegal USAGE and/or SIGN for INTERCHANGE file - DISPLAY and/or SIGN LEADING SEPARATE assumed

If implementor name INTERCHANGE is specified for a file, all data in that file must be described as USAGE IS DISPLAY. All signed numeric data must be described with the SIGN IS LEADING SEPARATE clause.

1628 Record must have data name in GLOBAL/EXTERNAL file - GLOBAL/EXTERNAL clause is ignored

GLOBAL clause can be specified only in data description entries whose level number is 01 in the File Section or Working-Storage Section, file description entries or report description entries and cannot be a FILLER item.

1629 GLOBAL clause cannot be specified for a record of a file named in a SAME RECORD AREA clause - GLOBAL clause is ignored

If the SAME RECORD AREA clause is specified for several files, the record description entries or the file description entries for these files must not include the GLOBAL clause.

1631 '*character-string*' is greater than 12 characters - name is truncated

The file name in the SELECT clause is greater than 12 characters. It is truncated to 12 characters.

1632 '*character-string*' duplicate clause ignored

This error is issued when a duplicate clause is encountered in the Environment Division or file description entry.

1634 '*character-string*' duplicate clause ignored

This error is issued every time a duplicate clause is detected in the Environment Division.

1635 '*character-string*' invalid implementor name or literal – DISC is assumed

An incorrect implementor name was found while processing the SELECT clause. The invalid name is ignored and DISC is assumed.

1636 '*character-string*' implementor name is for documentation only

This warning is issued to advise the user that implementor name is for documentation only.

1637 '*character-string*' invalid number of areas – clause is ignored

An illegal identifier is used instead of an integer number in the RESERVE clause. The default number will be used. The maximum number of RESERVE areas is 63.

1638 '*character-string*' invalid implementor name or literal – name or literal is ignored

An incorrect implementor name is found. The invalid name is ignored.

1640 EXTERNAL file cannot be specified in a SAME AREA clause – EXTERNAL clause is ignored

File names used in a SAME AREA clause cannot refer to an EXTERNAL connector.

1641 LINAGE clause integer + '*character-string*' is greater than the system limit – limit assumed

The integer value is greater than the maximum allowed for this part of the LINAGE clause. The LINAGE integer is limited to 511. LINES AT TOP is limited to 99. LINES AT BOTTOM is limited to 99.

1642 REPORT clause must be associated with a sequential file

The REPORT clause that is specified in the file description (FD) can only be associated with a sequential file. The current file being processed is not a sequential file.

1643 EXTERNAL file cannot be specified in a MULTIPLE FILE TAPE clause – EXTERNAL clause is ignored

File names used in a SAME AREA clause cannot refer to file names used in a MULTIPLE FILE TAPE clause.

1644 '*character-string*' is not valid RECORD KEY – key is ignored

The record key must have the following characteristics: Must be alphanumeric within a record associated with the file name to which RECORD KEY clause is subordinate and must not refer to a group item that contains a variable occurrence data item.

1645 Maximum of '*integer-value*' ALTERNATE keys exceeded – key '*character-string*' is ignored

The number of alternate keys is greater than the maximum allowed.

1646 '*character-string*' is not valid ALTERNATE KEY – key is ignored

The alternate key must have the following characteristics: must be alphanumeric within a record associated with the file name to which ALTERNATE KEY clause is subordinate and must not refer to a group item that contains a variable occurrence data item.

1648 RELATIVE KEY '*character-string*' has an illegal description – accepted

A relative key must refer to an unsigned integer data item whose description does not contain the PICTURE symbol P.

1649 RELATIVE KEY '*character-string*' is defined in a record description of the associated file – accepted

A relative key must not be defined in a record description entry associated with that file name.

1650 FILE STATUS '*character-string*' is not legal – FILE STATUS is ignored

The FILE STATUS must have the following characteristics: must be defined in the Data Division as a two-character data item of the category alphanumeric and must not be defined in the File Section or the Report Section.

1651 BLOCK CONTAINS clause integer '*character-string*' is not valid – integer is ignored

The identifier or the integer numbers used to specify the size of a physical record in the BLOCK CONTAINS clause must be an integer or identifier with an integer value.

1652 RESERVE clause integer causes the number of buffers used to exceed the maximum allowed – RESERVE clause ignored

The maximum number of the buffer areas allowed has been exceeded. The RESERVE clause is ignored. The number of buffers used by UCS COBOL at run time is (4 + n + a), where n = integer specified on RESERVE clause, a = number of alternate keys. The maximum number of buffers that can be used is 63, therefore if 4 + n + a > 63 the integer specified on the RESERVE clause will be ignored.

1653 '*character-string*' is not a procedure name – this paragraph is ignored

This error is issued if the names used for the ROLLBACK and ERROR paragraph in the INVOKE clause are not labels.

1654 BLOCK CONTAINS clause integer-1 is greater than integer-2 – integer-1 is ignored

In the BLOCK CONTAINS clause, integer-1 cannot be greater than integer-2 if they are used to specify the physical size of the record for a certain file.

1655 BLOCK CONTAINS clause integer greater than maximum allowed – clause is ignored

In the BLOCK CONTAINS clause, the integer used to specify the physical size of the record cannot exceed the maximum of 2 ** 26 - 1. The BLOCK CONTAINS clause integer must not be greater than 1 when the U phrase is specified in the RECORDING MODE clause.

1656 `PADDING CHARACTER 'character-string' is not legal – clause is ignored`

The PADDING CHARACTER must be a 1-character nonnumeric literal.

1657 `PADDING CHARACTER 'character-string' is not legal – clause is ignored`

The PADDING CHARACTER must not be defined in the File Section, Report Section or Subschema Section.

1658 `RECORD DELIMITER clause 'character-string' invalid – clause is ignored`

This diagnostic is issued when in a RECORD DELIMITER clause a name other than STANDARD-1 is specified.

1659 `CODE-SET alphabet name 'character-string' is not valid – clause is ignored`

The alphabet name used in the CODE-SET clause does not have alphabet name characteristics.

1660 `File has no ASSIGN clause – DISC is assumed`

The file is not assigned to any device or implementor name. The default implementor name of DISC is assumed.

1661 `RANDOM/DYNAMIC ACCESS illegal for SEQUENTIAL files – SEQUENTIAL ACCESS is assumed`

For SEQUENTIAL ORGANIZATION files the only allowable ACCESS MODE is SEQUENTIAL.

1662 `PADDING CHARACTER clause is inapplicable – clause is ignored`

The PADDING CHARACTER clause is legal only for the sequential files.

1663 `Device type is illegal for relative/indexed files – DISC is assumed`

Only allowable implementor name for the relative and indexed files is DISC.

1664 `'character-string' CODE-SET clause is illegal for relative/indexed files – clause is ignored`

CODE-SET clause is allowed only with sequential file organization.

1665 `File 'character-string' not selected – FD is ignored`

No SELECT clause can be found for this file in the FILE-CONTROL paragraph of the Environment Division.

1666 `File 'character-string' not selected – SEQUENTIAL DISC file is assumed`

No SELECT clause could be found for this file. Compiler assumes it is selected with the default values. (organization: sequential, implementor-name: disc).

1667 `File 'character-string' is not unique name – name is accepted`

A duplicate file name has been found. Compiler accepts the name.

1668 `File must have RELATIVE KEY defined for RANDOM/DYNAMIC ACCESS – SEQUENTIAL ACCESS is assumed`

A file with relative organization and ACCESS MODE of DYNAMIC or RANDOM

requires a RELATIVE KEY clause. The compiler assumes the ACCESS MODE is SEQUENTIAL.

1669 RECORD clause integer '*character-string*' invalid – integer is ignored

The identifier or the integer numbers used to specify the size of a physical record in the RECORD CONTAINS clause must be an integer or an identifier with an integer value.

1670 RECORD clause integer greater than maximum allowed – maximum is assumed

In the RECORD CONTAINS clause, the integer used to specify the physical size of the record cannot exceed the maximum of 2 ** 26 - 1.

1671 RECORD clause integer-1 is greater than integer-2 – integer-1 is ignored

In the RECORD CONTAINS clause integer-1 cannot be greater than integer-2 if they are used to specify the physical size of the record for a certain file.

1672 RECORDING MODE '*character-string*' clause is not legal for this file – clause is ignored

For INTERCHANGE files the recording modes are D, F, S, and U. For IBM-TAPE files the recording modes are F, U, and V with an optional AN. No other files have recording modes.

1673 '*character-string*' REPORT and LINAGE clauses together are illegal – clause is accepted

The LINAGE clause should not be specified for the report description entries.

1674 '*character-string*' is not legal for RECORD DEPENDING ON data name – description is accepted

When DEPENDING ON data name clause is specified for the RECORD clause, the data name must describe an elementary unsigned integer in the Working-Storage or Linkage Section.

1675 DENSITY IS '*character-string*' clause inapplicable for non-printer files – clause is ignored

The DENSITY clause is not applicable for nonprinter files.

1676 DENSITY IS '*character-string*': integer or dataname is illegal – default assumed

The DENSITY must be an unsigned whole number or a data name with the same description.

1677 '*character-string*' is not report name – name is ignored

The identifier used to specify the report name in a REPORT/REPORTS clause is not the name of a report description entry.

1678 '*character-string*' report name is named in another file – name is accepted

This report name has already been defined in another file description entry.

1679 The IBM tape feature (COBIBM) is not installed – RECORDING MODE is

ignored

The attempt to use the IBM tape interface is illegal. Installation of the IBM tape interface (COBIBM) allows use of the IBM tape syntax.

1680 The RDMS interface feature (COBRDM) is not installed – ENTER MASM is ignored

The attempt to use the RDMS interface is illegal. Installation of the RDMS interface (COBRDM) allows use of ENTER MASM "ACOB$RDMR".

1681 VALUE OF clause '*character-string*' is not a valid literal or data name, or implementor name is illegal for file type – clause is ignored

The VALUE OF clause has an invalid implementor name or the data name is not defined in the WORKING-STORAGE SECTION, or has not been described as USAGE IS DISPLAY, or an illegal literal is used.

1682 VALUE OF '*character-string*' data name is not valid – name is ignored

The data name used in the VALUE OF clause is not defined in the WORKING-STORAGE SRCTION, or has not been described as USAGE IS DISPLAY, or an illegal literal is used.

1683 LINAGE clause integer '*character-string*' illegal – integer is ignored

A noninteger value is used in the LINAGE clause or the integer is negative.

1684 LINAGE clause '*character-string*' inapplicable to file type – clause is ignored

LINAGE clause is not legal for files with the relative or indexed organization.

1685 FOOTING is '*character-string*' integer greater than LINAGE – FOOTING is ignored

In the LINAGE clause, second integer or data item referred to by second data name must have a value greater than zero and cannot have a value greater than first integer or the data item referred to by first data name.

1686 '*character-string*' is not valid data name in LINAGE clause – name is ignored

The given data name is not an elementary item that is an unsigned numeric integer.

1688 LINAGE integer not compatible with DENSITY integer

The integer specified for the LINAGE clause does not meet the rules governing the relationship between the DENSITY clause and the LINAGE clause. The specifications will be accepted. See the rules pertaining to the system constraints for the DENSITY clause in order to determine the specific problem.

1689 File must have RECORD KEY defined for an INDEXED file

RECORD KEY clause is a requirement for indexed files.

1690 RECORDING MODE is incompatible with RECORD clause – RECORD is ignored

The F phrase of the RECORDING MODE clause specifies fixed blocked records. A RECORD clause for the same file cannot define variable length records.

1694 `Implementor name A-SHARED-FILE is illegal with a sequential organization file - DISC assumed`

A file assigned with implementor name A-SHARED-FILE is illegal with sequential organization.

1695 `RELATIVE KEY clause illegal for this file ORGANIZATION - RELATIVE ORGANIZATION is assumed`

Definition of the RELATIVE KEY for a file with the organization other than relative is illegal.

1696 `RECORD/ALTERNATE KEY clause illegal for this file ORGANIZATION - INDEXED ORGANIZATION is assumed`

Definition of the RECORD/ALTERNATE KEY for a file with the organization other than indexed is illegal.

1697 `RERUN clause '`*`character-string`*`' is not implemented - for documentation only`

This error message is issued when a file name or an implementor name other than DISC or TAPE is used in a RERUN clause.

1698 `RERUN clause '`*`character-string`*`' is invalid - clause is ignored`

A noninteger value is encountered where an integer is required.

1699 `RERUN clause '`*`character-string`*`' is greater than maximum allowed - maximum is used`

The integer value used in the RERUN clause exceeds the maximum of 999999999 (ten digits). Maximum is assumed.

1700 `Both RELATIVE & RECORD/ALTERNATE KEYS are specified - INDEXED ORGANIZATION is assumed & RELATIVE KEY is ignored`

Definition of both RELATIVE KEY and RECORD/ALTERNATE KEY for the same file is illegal.

1701 `No other clause is allowed with LINAGE IS SYSTEM - clause ignored`

If the LINAGE IS SYSTEM LINES option is specified within an FD clause, LINES AT TOP, LINES AT BOTTOM or FOOTING must not be specified.

1705 `CODE TYPE '`*`character-string`*`' is illegal - CODE TYPE clause is ignored`

The CODE TYPE data name is illegal. The data name must be an elementary data item defined in the Working-Storage Section as PIC 99 BINARY.

1706 `LOCALE '`*`character-string`*`' is illegal - LOCALE clause is ignored`

The LOCALE data name is illegal. The data name must be an elementary data item defined in the Working-Storage Section as alphanumeric. The maximum size of the data item is 16 characters.

1707 `CODE TYPE '`*`character-string`*`' clause associated with an illegal file type - clause is ignored.`

The CODE TYPE clause is valid only for sequential or relative files. The CODE

TYPE clause is ignored.

1708 `LOCALE '`*`character-string`*`' clause associated with an illegal file type - clause is ignored.`

The LOCALE clause is valid only for INDEXED (MSAM) files. The LOCALE clause is ignored.

1721 `'`*`character-string`*`' unrecognized binary expression operator - expression ignored`

An illegal operator was found in a conditional expression.

1723 `Missing AND or OR in implied relational`

The AND or OR operator is missing in an abbreviated combined relational condition.

1724 `No previous relational for implied relational`

The implied relational condition has no previous relational condition from which to get the subject for the current condition.

1727 `Token stack overflow during processing of an expression - complexity of the statement must be reduced`

The size of the expression being processed has caused the token stack to overflow. The complexity of the expression must be reduced before the compiler can continue.

1730 `'`*`character-string`*`' is illegal with a NOT condition`

The logical operator NOT must be followed by a class condition or a conditional expression.

1731 `The unary minus is performed on the nonarithmetic '`*`character-string`*`'`

A unary operator must be followed by an arithmetic expression.

1732 `The unary minus is not being performed on an arithmetic expression`

A unary operator must be followed by an arithmetic expression.

1733 `The unary minus is illegally performed on '`*`character-string`*`'`

A unary operator must be followed by an arithmetic expression.

1735 `The NOT is illegally performed on '`*`character-string`*`'`

The logical operator NOT must be followed by a class condition or a conditional expression.

1736 `Arithmetic operations cannot be performed on '`*`character-string`*`' because it is not an elementary item`

Only elementary numeric operands can be used as operands in an arithmetic expression.

1737 `Conditional expression required here - expression ignored`

The keywords IF, WHEN, and UNTIL must all be followed by a conditional expression; which must begin with a data item, a literal, a left parenthesis or a unary sign. The containing statement is omitted.

1740 '*character-string*' was incorrectly used as a condition variable

A data item was incorrectly used as a condition name.

1741 '*character-string*' causes an incorrect data item to be treated as a condition

The data name used as a class name or switch name was not a condition name.

1742 '*character-string*' is an illegal condition item

An incorrect data type was used in a conditional expression.

1745 `Identifier is USAGE DISPLAY, but it has incorrect data description for this class condition - accepted`

For class condition NUMERIC, the identifier must be USAGE DISPLAY or a group item and cannot be ALPHABETIC (PIC A), unless reference modification is applied for this statement. For class condition ALPHABETIC, ALPHABETIC-UPPER, ALPHABETIC-LOWER, and class name, the identifier must be USAGE DISPLAY with data description of nonnumeric, or must have reference modification applied.

1746 `Incorrect operand used in a class condition - statement ignored`

Operand used in a class condition was not an identifier.

1747 '*character-string*' must be an elementary numeric item

The data item referenced must be an elementary numeric data item.

1748 `Group item which contains elementary signed field is illegal for NUMERIC test`

A NUMERIC class test cannot be used with a group item composed of elementary items whose data description indicates the presence of an operational sign.

1751 `Intermediate result assumed maximum precision - the result precision can be different from ASCII COBOL.`

The size of intermediate operands retained in calculations is different between UCS COBOL and ASCII COBOL. This could lead to different program results. UCS COBOL uses a single consistent algorithm. While some differences in precision can result, UCS COBOL is as precise or more precise than ASCII COBOL.

1752 `The specified operand is not a valid arithmetic data item or constant`

An alphanumeric (including edited) item or an index item cannot appear in an algebraic expression.

1753 `Identifier used for the REMAINDER must be numeric or numeric edited - REMAINDER is ignored`

The identifier used for the REMAINDER is illegal. It must be a numeric or numeric edited data item.

1754 '*character-string*' is not an elementary data item

The data item referenced must be an elementary data item.

1755 `This is not a valid operand in the arithmetic expression`

An invalid operand was used in an arithmetic expression.

1756 `This operand is not valid in an arithmetic expression`

An invalid operand was used in an arithmetic expression.

1757 `Identifier used for the REMAINDER is not an elementary data item - REMAINDER is ignored`

The data item used for the REMAINDER must be an elementary data item.

1758 `A nonnumeric literal in an arithmetic expression is illegal - accepted`

Arithmetic expressions should contain only numeric literals.

1759 `An alphabetic item in an arithmetic expression is illegal`

Arithmetic expressions should contain only numeric data items.

1762 '*character-string*' is not a Boolean constant

A Boolean constant is expected at this position in a conditional expression.

1763 '*character-string*' is not a Boolean variable

A Boolean variable is expected at this position in a conditional expression.

1764 `The first operand is not a Boolean expression`

The first operand in the conditional expression is expected to be a Boolean expression.

1766 `The second operand is not a Boolean expression`

The second operand in the conditional expression is expected to be a Boolean expression.

1769 `Subject and object are both literals - accepted`

The relation condition must contain at least one reference to a variable. This comparison is accepted during compilation.

1770 `INDEX items cannot be used in the expression - expression ignored`

An alphanumeric (including edited) item or an index item should not appear in an algebraic expression.

1771 '*character-string*' is not a legal operand when used in a relational expressionexpression ignored

An invalid operand was used in a relational expression.

1772 `Comparison of a noninteger to an alphanumeric is illegal - expression ignored`

A noninteger numeric operand cannot be compared to a nonnumeric operand.

1773 '*character-string*' is not an INDEX and cannot be compared to an index data item - expression ignored

Index data items can be compared only to another index data item or an index name.

1774 '*character-string*' data type is not compatible with the data type of the operand to which it is being compared

The data types of the operands being compared must be compatible to one another.

1775 '*character-string*' is illegal - only EQUAL and NOT EQUAL comparisons are permitted on Boolean items

Two Boolean variables can be compared only with the EQUAL or NOT EQUAL operators.

1776 `Relative indexing is only permitted with INDEX items`

Only index items are allowed with relative indexing.

1777 `Only constants are permitted in expressions with INDEX items`

An alphanumeric (including edited) item or an index item cannot appear in an algebraic expression.

1778 `Expressions involving INDEX items are not allowed`

Index items cannot be used in arithmetic expressions.

1779 `Paragraph name or section name '`*`character-string`*`' cannot be uniquely referenced`

Either the same section name has been defined twice in the Procedure Division or the same paragraph name has been defined twice within a section. The duplicate definition is accepted, but can cause erroneous execution if referenced. Also, the paragraph name or section name can have already been used as a data name.

1780 `The comparison of a noninteger numeric to an alphanumeric item is illegal - the numeric will be treated as characters`

The comparison of a noninteger numeric to an alphanumeric item is illegal. The noninteger numeric item is treated as characters.

1781 `The data type, size, or level number of the data name MCFLAG is not in the proper form for use with the keyword option MONITOR - MCFLAG will be ignored by the keyword option MONITOR`

In order to control monitor output, the variable MCFLAG must be defined as a one word binary item (PIC 9(10) BINARY). MCFLAG must also be defined as an 01 or 77 level item. Monitor output will be produced, however it will not be controlled by the variable MCFLAG.

1782 `Operand length exceeds the maximum limit allowed - expression is ignored`

The maximum size of a USAGE DISP-I18N or USAGE BYTE-I18N item used in a comparison is 4096. The expression will be ignored.

1804 `'`*`character-string`*`' is not within a statement - ignored`

When a character string is encountered in a position where a paragraph name or section name definition is legally possible, and if the character string is not in area A (columns 8-11), a paragraph name definition is not created, results are unpredictable, and a diagnostic is produced.

1806 `Data name is followed by an illegal subscript list`

A syntax error has occurred in a subscript expression. The statement containing the subscript expression is omitted.

1807 `'`*`character-string`*`' must be followed by a subscript list`

A data reference which contains an OCCURS clause must be followed by a subscript list.

1810 '*character-string*' is an extra subscript - it is ignored

The number of subscript indicators in the subscript expression is greater than the number required by the dimensionality of the referenced item. The excess subscript indicators are truncated and the reference is accepted.

1811 '*character-string*' is outside the range of values permitted for the subscript - it is replaced by a 1

A numeric literal used as a subscript must not exceed the maximum number of occurrences stated in the OCCURS clause. The oversized literal is replaced by 1.

1812 '*character-string*' is not a legal integer value - it is replaced by the integer 1

The specified subscript is not an integer value. It is replaced by a subscript of 1.

1813 '*character-string*' is not an elementary item - it is replaced by a 1

A data item used as a subscript must be an integer, that is, its scale (number of decimal positions) must be zero. It is replaced by a subscript of 1.

1814 '*character-string*' is not in the list of permitted INDEX names - it is replaced by a 1

The subscript is not in the list of permitted INDEX names that are allowed for the array.

1815 Only relative indexing is permitted - it is replaced by a 1

Only relative indexing is permitted for this array.

1816 The subscript is not an elementary item or constant

The data item used as a subscript must be an elementary data item or a numeric constant.

1819 A USAGE IS INDEX data item is illegal as a subscript - it is allowed but can cause an error in execution

An index data item (USAGE IS INDEX) cannot be used as a subscript. The index data item is allowed, but the user should be aware of possible errors at execution time.

1820 '*character-string*' is not legal in a subscript list - it is replaced by a 1

A data item used as a subscript must be an integer, that is, its scale (number of decimal positions) must be zero. It is replaced by a subscript of 1.

1821 '*character-string*' does not have enough subscripts - 1 is assumed for each missing subscript

The number of subscripts must equal the number of OCCURS clauses in the description of the table element being referenced. The missing subscripts are assumed to be 1.

1823 '*character-string*' is not an integer value - it is converted and the integer result used

The item specified in the subscript reference is not an integer. It is converted to an integer and treated as such.

1824 '*character-string*' is illegal in the sequence of clauses permitted in the sort description - it is ignored

The specified reference is not legal in a sort description.

1825 '*character-string*' is illegal in the sequence of clauses permitted in the report description - it is ignored

The specified reference is not legal in a report description.

1828 '*character-string*' is illegal in the sequence of clauses permitted in the communication description - it is ignored

The specified reference is not legal in a communication description.

1829 '*character-string*' is illegal in the sequence of clauses permitted in the File Section - it is ignored

The specified reference is not legal in the File Section.

1830 '*character-string*' is illegal in the sequence of clauses permitted in the Working-Storage Section - it is ignored

The specified reference is not legal in the Working-Storage Section.

1831 '*character-string*' is illegal in the sequence of clauses permitted in the Common-Storage Section - it is ignored

The specified reference is not legal in the Common-Storage Section.

1832 '*character-string*' is illegal in the sequence of clauses permitted in the Linkage Section - it is ignored

The specified reference is not legal in the Linkage Section.

1833 '*character-string*' is illegal in the sequence of clauses permitted in the Communication Section - it is ignored

The specified reference is not legal in the Communication Section.

1834 '*character-string*' is illegal in the sequence of clauses permitted in the Report Section - it is ignored

The specified reference is not legal in the Report Section.

2000 Illegal level number - 01 assumed

Allowable level numbers include 01 through 49, 66, 77, and 88. Initial levels must be either 01 or 77. Initial level in Report Section must be 01. Level 77 in File Section is illegal. Levels 66, 77 and 88 can have only certain level numbers following them.

2001 Illegal PICTURE clause - clause omitted

The PICTURE character string of an alphanumeric edited item is restricted to certain combinations of A, X, 9, B, 0 and /, and must contain at least one A or X and at least one B, 0 or /. The size of the edited characters is also restricted. The maximum replication on an editing character (B, 0, /) is 999. The maximum number of editing characters allowed in a PICTURE string is 2000.

2002 Level 66 must have data name

Level number of 66 must have data names associated with them, FILLER cannot

be used.

2003 `Literal-2 not greater than literal-1 - OCCURS clause is ignored`

In the format of OCCURS integer-1 TO integer-2, integer-1 must be greater than or equal to zero, and integer-2 must be greater than integer-1.

2004 '*character-string*' is illegal in a PICTURE clause - character is ignored

The allowable characters in a PICTURE clause are A, B, P, S, V, X, Z, CR, DB and their lower case equivalents, (,), /, 0, 1, 9, +, -, *, ,, ., $, and integers within parentheses.

2005 '*character-string*' causes illegal order of characters - PICTURE is ignored

In a PICTURE string, symbol S, if specified, must be first; a floating + or - cannot be preceded by *, 1, Z, 9 or P; trailing + or - must be the last character, + or - cannot be preceded by a floating $; the character P can appear only to the left or the right as a contiguous string of P's.

2006 `No allocation for this elementary item`

The PICTURE clause must be specified for every elementary item except an index data item and the subject of the RENAMES clause.

2007 `Illegal USAGE type - USAGE clause is ignored`

The USAGE clause for a report group item can specify only USAGE IS DISPLAY. It can be written in any data description entry with a level number other than 66 or 88, must contain one of the allowable USAGE types.

DMS usages, such as DATABASE-KEY, RECORD-NAME, SET-NAME, AREA-NAME, AREA-KEY, and ACQUIRE-LIST are not allowed in the FILE SECTION.

2008 Maximum number of '*integer-value*' characters allowed in a PICTURE is exceeded - PICTURE is truncated

The number of characters that can be described by a PICTURE string is exceeded.

2009 `PICTURE and USAGE conflict - USAGE is ignored`

An elementary data item, or an elementary data item subordinate to a group item whose declaration contains a USAGE clause specifying BINARY, BINARY-1, COMPUTATIONAL, COMPUTATIONAL-5 or PACKED-DECIMAL must be declared with a PICTURE string that describes a numeric item. For USAGE COMPUTATIONAL-5/ COMP-5, the PICTURE string must also describe item being integer, SIGNED, and length 9(4) or 9(9).

2010 `Level number does not reflect correct hierarchy - assume previous level`

A data description entry that contains the OCCURS DEPENDING ON clause can be followed within that record description only by data description entries which are subordinate to it or a level 01 item. The previous data description had a PICTURE clause but should be parent to the current item.

2011 `Illegal JUSTIFIED clause - clause is ignored`

The JUSTIFIED clause can be specified only at the elementary level and cannot be

specified for a data item described as numeric or for which editing has been specified. Also, JUSTIFIED cannot be specified for an index data item.

2012 `Maximum number of decimal digits is 18 - treated as PIC X`

The number of digit positions that can be described by a numeric PICTURE string must range from 1 to 18 inclusive. Item is treated as PIC X .

2013 `Illegal RENAMES clause - clause is ignored`

The RENAMES clause is illegal. All RENAMES entries referring to data items within a given record must immediately follow the last data description entry of the associated record description entry. A 66 level entry cannot rename another 66 level entry nor can it rename a 77, 88, or 01 level entry. Data-name-2 and data-name-3 must be names of elementary items or groups of elementary items in the same logical record.

2014 `BLANK WHEN ZERO incompatible with zero suppression - BLANK WHEN ZERO clause is ignored`

The asterisk, when used as the zero suppression symbol, and the clause BLANK WHEN ZERO cannot appear in the same entry.

2015 `Item USAGE and group USAGE differ - item USAGE is used`

If the USAGE clause is written in the data description entry for a group item, it can also be written in the data description entry for any subordinate elementary item or group item, but the same USAGE must be specified in both entries.

2016 `VALUE clause not permitted here - VALUE clause is ignored`

The VALUE clause must not be stated in a data description entry that contains a REDEFINES clause, or in an entry that is subordinate to an entry containing a REDEFINES clause. The VALUE clause must also not be specified for a group item containing items subordinate to it with descriptions including JUSTIFIED, SYNCHRONIZED, or USAGE (other than USAGE IS DISPLAY). It also must not be specified in any entry which is part of the description or redefinition of an external data record. The record selector value is not permitted when the translation is a whole record. The VALUE clause may only be used in an FD of an IBM tape file.

2017 `VALUE clause missing here`

In a condition name entry, the VALUE clause is required.

2018 `SIGN clause and USAGE conflict - SIGN clause is ignored`

The numeric data description entries to which the SIGN clause applies must be described, implicitly or explicitly, as USAGE IS DISPLAY or BINARY-1.

2019 `OCCURS at this level is illegal - OCCURS clause is ignored`

The OCCURS clause must not be specified in a data description entry that has a level number of 01, 66, 77, or 88 or that has a variable occurrence data item subordinate to it.

2020 `Level 01 REDEFINES in File Section illegal - REDEFINES ignored`

The REDEFINES clause must not be used in level 01 entries in the File Section.

2021 `Invalid RANK clause`

The RANK clause is in error. The value in the RANK clause must be from 0 to 63. The RANK clause without the VALUE clause is meaningless. The RANK clause

might have been used twice here. The RANK is to be used only with an IBM-TAPE file. The RANK clause is not permitted when the translation is a whole record.

2022 `Elementary clause at group level - clause is ignored`

A clause that is required to be at an elementary level has been specified at a group level.

2023 '*character-string*' is illegal in this PICTURE string - PICTURE is ignored

The character string in the PICTURE clause must consist of certain allowable combinations of characters in the COBOL character set. The allowable combinations determine the category of the elementary item.

2024 `CR, DB, S, V, or . can only appear once in PICTURE - second specification is ignored`

The following symbols can appear only once in a given PICTURE clause: S, V, ., CR, DB.

2025 '*character-string*' must be rightmost within PICTURE string - PICTURE is ignored

When the symbols CR or DB are used, they must represent the rightmost character positions that are counted in the size of the item. If Ps occupy the right-most part of the PICTURE string, the symbol V, if specified, must be in the rightmost position.

2026 `Illegal PICTURE clause - PICTURE is ignored`

At least one of the symbols A, X, Z, 1, 9 or *, or at least two occurrences of one of the symbols +, - or 'currency sign' must be present in a PICTURE string. If the COMPAT/FULL option was not specified, PIC 1 without the USAGE BINARY-1 clause is illegal. In addition, USAGE BINARY-1 is illegal for an INTERCHANGE tape.

2027 `PICTURE string too long - string is truncated to 30 characters`

The maximum number of characters allowed in a PICTURE string is 30.

2028 `Item prior to 88 level cannot have JUST, SYNC, or USAGE clause`

A condition name, 88 level, can be associated with any data description entry which contains a level number except a group containing items with descriptions including JUSTIFIED, SYNCHRONIZED or USAGE(other than DISPLAY).

2029 `Group with SIGN must contain at least one signed USAGE DISPLAY elementary item`

The SIGN clause can be specified only for a numeric data description entry whose PICTURE contains the character S and whose USAGE is DISPLAY, or a group item containing at least one such numeric data description entry.

2030 `No PICTURE or USAGE clause on an elementary item - assume size 1`

The PICTURE clause must be specified for every elementary item except an index data item and the subject of the RENAMES clause.

2031 `Illegal SIGN clause - SIGN clause is ignored`

The SIGN clause applies only to numeric data description entries whose PICTURE clause contains the character S. The SIGN requested must be an allowable type, that is SIGN LEADING.

2032 `Illegal GLOBAL clause - GLOBAL clause is ignored`

The GLOBAL clause: must be in the File Section or Working-Storage Section, must not be on the data name FILLER, must not have a parent with a GLOBAL clause, must not be subordinate to an item with an OCCURS clause or have an OCCURS clause in its data description entry. Two identical user names cannot both have the GLOBAL clause associated with them.

2033 `Illegal EXTERNAL clause - EXTERNAL clause is ignored`

The EXTERNAL clause must be on a file description entry (level FD) or on a level 01 entry in the WORKING-STORAGE SECTION. It must not have a data name of FILLER. The level 01 entry must not contain a REDEFINES clause. Two identical user names cannot both have the EXTERNAL clause associated with them.

2034 `Illegal OCCURS number - OCCURS clause is ignored`

Integers specified in the OCCURS clause must be unsigned and greater than 0. Where two integers are used, the first must be greater than or equal to 0, the second must be greater than the first.

2035 `Illegal SYNC clause - SYNC clause is ignored`

The SYNCHRONIZED clause can appear only in an elementary item.

2036 `Illegal BLANK WHEN ZERO clause - clause is ignored`

The BLANK WHEN ZERO clause can be specified only for an elementary item whose PICTURE is specified as numeric or numeric edited. Also, the item must be described either implicitly or explicitly as USAGE DISPLAY.

2037 `Illegal REDEFINES clause - REDEFINES clause is ignored`

The level numbers of the REDEFINES data name and the subject of the entry must be identical but must not be 66 or 88. The REDEFINES must immediately follow the subject of the entry. The REDEFINES data name cannot contain an OCCURS clause or be an INDEX item.

2038 `VALUE specified is not legal with item's PICTURE clause - VALUE clause ignored`

If the category of the item is numeric, all literals in the VALUE clause must be numeric. If the category is alphabetic, alphanumeric, alphanumeric edited or numeric edited, all literals in the VALUE clause must be nonnumeric. A floating VALUE can be used only with a USAGE COMP-1 or COMP-2 picture.

2039 `VALUE specified must not be numeric - VALUE clause ignored`

If the VALUE clause is used in an entry at the group level, the literal must be a figurative constant or a nonnumeric literal.

2040 `CURRENCY SIGN must be 1 character long - first character is used`

The literal which appears in the CURRENCY SIGN clause is used in the PICTURE clause to represent the currency symbol. It must be nonnumeric and limited to a single character.

2041 '*character-string*' illegal character for CURRENCY SIGN – clause is ignored

The literal which appears in the CURRENCY SIGN clause can be any character from the computers character set except the digits 0 through 9, alphabetic characters consisting of the uppercase letters A, B, C P, R, S, V, X, Z, and the special characters *, +, -, ,, ., ;, (,), ", =, /.

2042 Illegal THROUGH phrase on VALUE clause – THROUGH phrase is ignored

The THRU associated with the VALUE clause can be specified only on 88 level items. There must be two literals associated with a THRU, the first must be less than the second.

2043 Index '*character-string*' is not unique name – name is accepted

A data item must be associated with the INDEX clause and must be a unique word within the program.

2044 OCCURS DEPENDING follows or is followed by illegal item – ODO ignored, integer-2 occurrences assumed

A data description entry that contains the OCCURS DEPENDING ON clause can be followed within that record description only by data description entries which are subordinate to it or a level 01 item. It also must not be subordinate to a group item with an OCCURS clause.

2045 Data names associated with OCCURS clause cannot be Boolean

The data name associated with the DEPENDING ON phrase of the OCCURS clause must describe an integer.

2046 Maximum size for replication of a character is exceeded

The maximum number of replications in a PICTURE clause is 999999. The item is given the size of one.

2047 OCCURS DEPENDING ON data name must be an undimensioned scalar integer

The data name associated with the DEPENDING ON phrase of the OCCURS clause must describe an integer. It must also not be dimensioned.

2048 EXTERNAL clause missing on OCCURS DEPENDING ON data name

If the OCCURS DEPENDING ON clause is specified in a data description entry containing the EXTERNAL clause, the data name following DEPENDING ON must refer to a data item possessing the external attribute which is described in the same Data Division.

2049 OCCURS DEPENDING ON data name must be in same Data Division as GLOBAL OCCURS item

If the OCCURS clause is specified in a data description entry subordinate to one containing the GLOBAL clause, the data name following the DEPENDING ON phrase, if specified, must be a global name and must refer to a data item which is described in the same Data Division.

2050 `GLOBAL clause missing on OCCURS DEPENDING ON data name`

If the OCCURS clause is specified in a data description entry subordinate to one containing the GLOBAL clause, the data name following the DEPENDING ON phrase, if specified, must be a global name and must refer to a data item which is described in the same Data Division.

2051 `ASCENDING/DESCENDING KEY must be subordinate to OCCURS item`

The data name associated with the ASCENDING/DESCENDING KEY phrase of the OCCURS clause must be subordinate to the entry containing the OCCURS clause.

2052 `ASCENDING/DESCENDING KEY must be at same OCCURS level as referencing OCCURS item`

The data name associated with the ASCENDING/DESCENDING KEY phrase of the OCCURS clause must not contain an OCCURS clause except when it is the subject of the entry.

2053 `Only first ASCENDING/DESCENDING KEY can be name of entry containing OCCURS`

The first specification of the data name associated with the ASCENDING/ DESCENDING KEY phrase must be the name of either the entry containing the OCCURS clause or an entry subordinate to the entry containing the OCCURS clause.

2054 `Segment numbers must range from 00 to 99 and are not allowed in a nested program`

The segment number is limited to two positive digits. Only the top parent in a nested program can be segmented.

2055 `Paragraph and section names must begin in area A`

Each paragraph name or section name must start in area A.

2056 `A paragraph is in a section so all paragraphs must be in sections`

If any paragraph is in a section, all paragraphs must be in sections.

2057 `COMMON clause can be used only if program is contained within another program`

The COMMON clause specifies that a program is common. A common program is contained within another program but can be called from programs other than the one containing it.

2058 `Two nested programs cannot have the same PROGRAM-ID`

The compiler cannot always tell which program is being called when two of them have the same name.

2060 `Item subordinate to group item with VALUE clause cannot have JUST, SYNC or USAGE (other than DISP) clause - clause accepted`

The VALUE clause must not be specified for a group item containing items subordinate to it with descriptions including JUSTIFIED, SYNCHRONIZED, or USAGE (other than USAGE IS DISPLAY).

2061 `The VALUE clause literal exceeds the PICTURE size of the variable. Truncation of significant digits may occur — accepted`

The size of the data specified on the VALUE clause of an 88 level item or a data item is larger than the associated data item.

2062 `Reserved word illegally begins in area B - accepted`

Reserved words which are division headers or section headers or paragraphs must begin in area A (columns 8 thru 11). If they do not start in area A, they are diagnosed but accepted.

2063 `Illegal reserved word in area A - accepted`

Reserved words, except for division headers, section headers, paragraph names, FD, RD, SD, or CD must begin in area B (columns 12 thru 72). If not, they are diagnosed but accepted.

2064 `A maximum of 7 dimensions is allowed - OCCURS ignored`

The OCCURS clause cannot have more than 7 dimensions.

2065 `When the characters 'V' and 'P' are in the same PICTURE, it must start with 'VP' or end with 'PV' - the 'V' is ignored`

In a PICTURE clause containing Ps, a V is redundant and not required. If a V is present, it must precede the Ps if the clause starts with Ps or it must trail the Ps if the clause ends with Ps.

2066 `Size of numeric literal is larger than the maximum allowed`

The size of the numeric literal that is specified is larger than the maximum allowed by the compiler.

2067 `Illegal USAGE and PICTURE combination - USAGE clause is ignored`

USAGE DISP-I18N is only valid for elementary data items described as alphanumeric (PIC X) or alphabetic (PIC A). USAGE BYTE-I18N data items are only valid for elementary data items described as alphanumeric (PIC X).

2068 `Both operands in this comparison or assignment must be declared as USAGE BYTE-I18N`

If one of the operands in a comparison or in an assignment statement is declared as USAGE BYTE-I18N, then the other operand must also be declared as USAGE BYTE-I18N. The exception is either LOW-VALUE or HIGH-VALUE may be used with a BYTE-I18N item in a comparison or assignment statement.

2069 `Illegal value associated with VALUE clause for BYTE-I18N data item`

A data item declared as USAGE BYTE-I18N can only have a figurative constant LOW-VALUE or HIGH-VALUE VALUE clause associated with it (i.e. VALUE LOW-VALUE).

2070 `Illegal device name in SPECIAL-NAMES paragraph - specification is ignored`

The device name following the SPECIAL-NAMES paragraph must be one of the allowable types.

2071 `Operand length exceeds the maximum length allowed (4096 bytes) - expression is ignored`

The maximum size of a BYTE-I18N item used in a comparison or assignment is 4096 bytes.

2076 `REDEFINES object name not the same as previous item - REDEFINES is ignored`

The REDEFINES clause, when specified, must immediately follow the subject of the entry and the level numbers must be identical.

2077 `Level 88 or 66 item illegal with USAGE - USAGE is ignored`

The USAGE clause can be written in any data description entry with a level number other than 66 or 88.

2081 `Level numbers 77 and 01 must be in area A`

Level numbers 01 and 77 must appear in area A of the source line (columns 8 - 11). Other level numbers can be indented for purposes of readability.

2082 `Level numbers 66 and 88 must not have PICTURE clauses - 01 is assumed`

The level numbers 66 and 88 cannot have a PICTURE clause contained in them, 66 can have only the RENAMES clause and 88 only the VALUE clause.

2087 `Integer-1 missing - value of 1 is assumed`

The OCCURS DEPENDING ON clause must contain two integers. The first integer is assumed missing and is given the assumed value 1.

2088 `Integer-1 illegal without DEPENDING clause`

If the OCCURS clause contains two integers, it must also contain the DEPENDING ON phrase.

2089 `Maximum number of binary digits is exceeded - excess truncated`

Each symbol 1 in the PICTURE string represents a bit position in an exact binary data item and is counted in the size of the item. A maximum of 72 bit positions is allowed for signed strings. A maximum of 71 bit positions is allowed for unsigned strings. The compiler accepts this item and allocates 72 bits; however unpredictable results can occur if the leftmost bit is set. The item is unsigned unless the 'S' was used.

2090 `More than 18 scaling positions - truncated`

The number of digits plus the number of Ps in the PICTURE clause is greater than 18. The number of Ps is reduced to make the sum equal to 18.

2091 `Peripheral device and usage conflict`

All the items in a symbiont file should be alphabetic, alphanumeric or alphanumeric edited.

2100 `The CODE clause literal must be a 2-character nonnumeric literal - clause is ignored`

The CODE clause specifies a 2-character nonnumeric literal that identifies each print line as belonging to a specific report.

2101 `Integers cannot exceed 3 digits in the PAGE, LINE,COLUMN or NEXT GROUP clause`

Integers specified in the PAGE, LINE or NEXT GROUP clause must not exceed 3 significant digits in length and not greater than 511. The COLUMN cannot exceed 999. If the COLUMN number is greater than 999, it is reset to 999.

2102 `Illegal integer in PAGE clause`

Each integer in the PAGE clause must be larger than or equal to the integer preceding it in the clause, with the exceptions that the integer following HEADING must be greater than or equal to 1 and the integer following PAGE must be greater than or equal to the largest integer in the PAGE clause.

2103 `LINE NUMBER of 0 not allowed`

The integer following LINE NUMBER specifies an absolute line number. An absolute line number specifies the line number on which the print line is presented.

2104 `Illegal clause with level number - clause is ignored`

The NEXT GROUP and TYPE clauses must be associated with the 01 level number.

2105 `First position of print line is column 1`

The leftmost position of the print line as specified in the CONTROL NUMBER clause is considered to be column 1.

2106 `SOURCE identifier must be LINE-COUNTER, PAGE-COUNTER, or a sum counter from current report`

The identifier in the SOURCE clause can be defined in any section of the Data Division. If it is a Report Section item, it can only be a PAGE-COUNTER or LINE-COUNTER or a sum counter of the report within which the SOURCE clause appears.

2107 `SOURCE identifier must be defined in the Data Division`

The identifier in the SOURCE clause can be defined in any section of the Data Division.

2108 `CONTROL clause data name is undefined or defined in the Report Section`

The data name in the CONTROL clause must not be defined in the Report Section. It can be qualified.

2109 `CONTROL clause data name cannot be variable in size`

The data name associated with the CONTROL clause must not have a variable occurrence data item subordinate to it.

2110 `Data name or FINAL must be previously specified in CONTROL clause of RD`

Data names and FINAL, if present in the TYPE or SUM RESET must be specified in the CONTROL clause of the corresponding report description entry.

2111 `Data item with SUM clause must be numeric`

The data item that is the subject of the report group description entry in which the SUM clause appears must not be defined as alphabetic.

2112 `A SUM clause can appear only in a CONTROL FOOTING report group`

Within a report group description a SUM clause can appear only in a CONTROL

FOOTING report group.

2113 `SUM clause identifiers must be numeric`

The value in each data item referred to by the SUM identifier is added to the sum counter; this addition is consistent with the rules for arithmetic statements.

2114 `Only printable items can have the GROUP INDICATE clause`

The GROUP INDICATE clause must be specified only in a DETAIL report group entry that defines a printable item.

2115 `The GROUP INDICATE clause is only allowed in a DETAIL report group`

The GROUP INDICATE clause must be specified only in a DETAIL report group entry that defines a printable item.

2116 `Illegal LINE NUMBER clause - clause is ignored`

Illegal LINE NUMBER clause. The compiler tried to make an assumption to correct the absolute LINE NUMBER coming after the relative LINE NUMBER but failed.

2117 `An entry with a VALUE clause must also have a COLUMN NUMBER clause`

Within a report group description an entry that contains a VALUE clause must also have a COLUMN NUMBER clause.

2118 `Illegal use of NEXT PAGE phrase of LINE NUMBER clause - NEXT PAGE ignored`

A NEXT PAGE phrase can appear only once and, if present, must be in the first LINE NUMBER clause and can appear only in the description of body groups and in a REPORT FOOTING report group of a given report group description entry.

2119 `Illegal COLUMN NUMBER clause - clause is ignored`

The data item referred to by the COLUMN NUMBER identifier must not have data items subordinate to it.

2120 `Illegal GROUP INDICATE clause - clause is ignored`

The data item referred to by the GROUP INDICATE identifier must not have data items subordinate to it.

2121 `Illegal SOURCE clause - clause is ignored`

The data item referred to by the SOURCE identifier must not have data items subordinate to it.

2122 `Illegal SUM clause - clause is ignored`

The data item referred to by the SUM identifier must not have data items subordinate to it.

2123 `This entry has a LINE NUMBER clause and is subordinate to a LINE NUMBER`

A report group description entry that contains a LINE NUMBER clause must not have a subordinate entry that also contains a LINE NUMBER clause.

2124 `This report TYPE can only appear once within a RD - TYPE is ignored`

REPORT HEADING, PAGE HEADING, CONTROL HEADING FINAL, CONTROL FOOTING FINAL, PAGE FOOTING, and REPORT FOOTING report groups can each appear no

more than once in the description of a report.

2125 `PAGE clause required in TYPE PAGE FOOTING and PAGE HEADING groups`

PAGE HEADING and PAGE FOOTING report groups can be specified only if a PAGE clause is specified in the corresponding report description entry.

2126 `NEXT GROUP value outside PAGE layout - value adjusted`

The integer of the absolute NEXT GROUP clause must specify a line number that is not less than that specified in the FIRST DETAIL phrase of the PAGE clause, and that is not greater than the specification in the FOOTING phrase of the PAGE clause.

2127 `LINE NUMBER is less than upper LIMIT - upper LIMIT assumed`

Integers in the LINE NUMBER clause cannot be specified in such a way as to cause any line of a report group to be presented outside the vertical subdivision of the page designated for that report group type as defined by the PAGE clause.

2128 `LINE NUMBER is greater than lower LIMIT - lower LIMIT assumed`

Integers in the LINE NUMBER clause cannot be specified in such a way as to cause any line of a report group to be presented outside the vertical subdivision of the page designated for that report group type as defined by the PAGE clause.

2129 `LINE NUMBER exceeds groups printable area - PAGE LIMIT assumed`

Integers in the LINE NUMBER clause cannot be specified in such a way as to cause any line of a report group to be presented outside the vertical subdivision of the page designated for that report group type as defined by the PAGE clause.

2130 `PAGE LIMIT specification rule error - specification is altered`

One of the following PAGE clause phrases has been specified illegally: FIRST DETAIL, PAGE HEADING, LAST DETAIL, FOOTING.

2131 `NEXT GROUP clause illegal with REPORT FOOTING or PAGE HEADING`

The NEXT GROUP clause must not be specified in a REPORT FOOTING report group or in a PAGE HEADING report group.

2132 `NEXT GROUP IS NEXT PAGE clause illegal for PAGE FOOTING - clause is ignored`

The NEXT PAGE phrase of the NEXT GROUP clause must not be specified in a PAGE FOOTING report group.

2133 `The report name is not specified in a REPORT clause`

The report name specified on the RD clause must have appeared in at least one REPORT clause in a file description.

2134 `SIGN clause must have SEPARATE CHARACTER phrase in Report Section - SIGN clause is ignored`

When the SIGN clause is included in a report group description entry, the SEPARATE CHARACTER phrase must be specified.

2135 `LINE NUMBER clause missing`

The COLUMN NUMBER clause, if present, must appear in or be subordinate to an entry that contains a LINE NUMBER clause.

2136 `LINE NUMBER exceeds groups printable area - PLUS 1 assumed`

A relative LINE NUMBER cannot exceed the area specified for a report group. The LINE NUMBER is handled as if a PLUS 1 had been stated.

2138 `Only a relative NEXT GROUP clause can be specified if PAGE LIMIT missing - 1 is assumed`

If the PAGE clause is omitted from a given report group description entry, only relative LINE NUMBER clauses can be specified in any report description entry within that report. The report consists of a single page of indefinite length.

2139 `Only a relative LINE NUMBER is legal if PAGE LIMIT is missing - 1 is assumed`

If the PAGE clause is omitted from the report description entry, only a relative NEXT GROUP clause can be specified in any report group description entry within that report. The report consists of a single page of indefinite length.

2140 `An absolute LINE NUMBER is illegal after a relative LINE NUMBER - defaults to PLUS 1`

Within a given report group description entry, all absolute LINE NUMBER clauses must precede all relative LINE NUMBER clauses.

2141 `NEXT PAGE is legal only in the first LINE of a group - 1 is assumed`

Within a given report group description entry a NEXT PAGE phrase can appear only once and, if present, must be in the first LINE NUMBER clause in that report group description entry.

2142 `First LINE NUMBER within a PAGE FOOTING must be absolute - FOOTING + 1 assumed`

The first LINE NUMBER clause specified within a PAGE FOOTING report group must be an absolute LINE NUMBER clause.

2143 `COLUMN numbers are not in ascending order - current COLUMN number is assumed to be previous column + 1`

Within a given print line, the printable items must be defined in ascending column number order such that each printable item defined occupies a unique sequence of contiguous character positions.

2144 `No TYPE clause on 01 report group - group is omitted`

A TYPE clause must be on the 01 level of the report group description.

2145 `Absolute LINE NUMBER is less than the previous LINE NUMBER in the group - 1 is assumed`

Within a given report group description entry, successive absolute LINE NUMBER clauses must specify integers that are in ascending order but need not be consecutive. Absolute lines must precede relative lines.

2149 `Data name or FINAL is missing from TYPE clause`

The data name or keyword FINAL that must be specified with a CONTROL HEADING or CONTROL FOOTING clause is missing.

2150 `Name missing after RD - no report is generated`

The first word after the keyword RD must be a user name. The rest of the report is scanned but the report cannot be written at object time since it has no associated file.

2151 `Keyword DETAIL is missing in FIRST DETAIL or LAST DETAIL clause - one is assumed`

In a FIRST DETAIL or LAST DETAIL phrase of the PAGE clause, the keyword DETAIL was not found, it was assumed.

2152 `Keyword ZERO is missing in BLANK WHEN ZERO clause - one is assumed`

In a BLANK WHEN ZERO clause, the keyword ZERO was not found. It is assumed.

2153 `Keyword HEADING or FOOTING is missing - HEADING is assumed`

Neither the keyword HEADING nor the keyword FOOTING was found after the keyword CONTROL, PAGE, or REPORT in a TYPE clause. HEADING is arbitrarily assumed.

2154 `Syntax error in TYPE clause - TYPE DETAIL is assumed`

The TYPE clause must have a report group type associated with it. TYPE DETAIL is assumed.

2155 `Keyword GROUP is missing in NEXT GROUP clause - one is assumed`

The keyword GROUP in a NEXT GROUP clause was not found. It is assumed.

2156 `Keyword SECTION is missing after keyword REPORT - one is assumed`

The keyword SECTION was not found after the keyword REPORT. It is assumed.

2157 `Duplicate CONTROL specified - duplicate CONTROL ignored`

Each occurrence of the data name in the CONTROL clause of an RD must identify a different item.

2158 `CONTROL data name cannot be subscripted - data name ignored`

A data name that is specified in the CONTROL clause must not be subscripted or indexed.

2159 `Illegal data name in CONTROL FOOTING or HEADING`

The data names and FINAL, if present in the CONTROL HEADING or CONTROL FOOTING phrase of the TYPE clause, must be specified in the CONTROL clause of the corresponding report description entry.

2160 `Must be sum counter if defined in the Report Section`

The identifiers associated with the SUM clause must be the names of the sum counters when defined in the Report Section.

2161 `UPON data name must be a DETAIL report group within the current report`

The data name following the UPON phrase of the SUM clause must be the name of a DETAIL report group described in the same report as the CONTROL FOOTING report group in which the SUM clause appears.

2162 `SUM identifiers must not be sum counters when UPON is specified`

If the UPON phrase is specified, any identifiers in the associated SUM clause must not be sum counters.

2163 `Report group must contain at least one body group`

In the Report Section of the Data Division, a 01 level number entry must contain at least one of the clauses TYPE DETAIL, CONTROL HEADING, or CONTROL FOOTING.

2164 `CONTROL data name is referenced in more than one CONTROL HEADING`

At most, one CONTROL HEADING report group can be specified for each data name or FINAL in the CONTROL clause of the report description entry.

2165 `CONTROL data name is referenced in more than one CONTROL FOOTING`

At most, one CONTROL HEADING report group can be specified for each data name or FINAL in the CONTROL clause of the report description entry.

2166 `CODE clause must be specified on all or none of the report names specified in the REPORT clause for a file`

If the CODE clause is specified for any report in a file, it must be specified for all reports in that file.

2167 `SOURCE clause in PAGE HEADING or PAGE FOOTING report group cannot be a CONTROL data item`

In PAGE HEADING and PAGE FOOTING report groups, SOURCE clauses and USE statements must not refer to CONTROL data names.

2168 `SOURCE clause in CF, PH, PF, or RF cannot be a group item which has a CONTROL item subordinate to it`

In CONTROL FOOTING, PAGE HEADING, PAGE FOOTING, and REPORT FOOTING report groups, SOURCE clauses and associated USE statements must not refer to group data items containing a CONTROL data item.

2169 `SOURCE clause in CF, PH, PF, or RF cannot be a data item which is subordinate to a CONTROL data item`

In CONTROL FOOTING, PAGE HEADING, PAGE FOOTING, and REPORT FOOTING report groups, SOURCE clauses and associated USE statements must not refer to data items subordinate to a CONTROL data item.

2170 `Unexpected input following reserved word LINE - integer 1 assumed to follow it`

An integer or PLUS integer was not found following the keyword LINE. Default integer 1 is assumed.

2171 `SUM identifier is a sum counter in a higher level`

A SUM object which is a sum counter must be defined either in the same report group that contains the SUM clause or in a report which is at a lower level in the CONTROL hierarchy of this report.

2172 `PICTURE clause expected - report line not processed`

The PICTURE clause must be stated in every elementary report item description.

2180 `Only printable items can have USAGE DISPLAY clause`

In the Report Section, the USAGE IS DISPLAY clause is used only to declare the usage of printable items.

2182 `COLUMN number too small - it is ignored`

The integer specified with the COLUMN NUMBER clause is too small to allow for the previous COLUMN clause.

2183 `PAGE/LINE/SUM COUNTER used illegally as a subscript - accepted`

A sum counter or the data names LINE-COUNTER and PAGE-COUNTER cannot be used as subscripts within the Report Section.

2184 `SOURCE clause in CF, PH, PF, or RF cannot be a redefinition or renaming of a CONTROL name`

In CONTROL FOOTING, PAGE HEADING, PAGE FOOTING, and REPORT FOOTING report groups of the TYPE clause, SOURCE clauses and associated USE statements must not redefine or rename any part of a CONTROL data item.

2185 `The CODE clause is meaningless when the report is associated with a print file`

The CODE clause is ignored if the report file is assigned to a printer file.

2186 `An identifier defined in the Report Section is illegal in this statement`

A data name defined in a report group description in the Report Section can be referred to only by a GENERATE statement, the UPON phrase of a SUM clause, a USE BEFORE REPORTING sentence, or as a sum counter qualifier.

2200 `'`*`character-string`*`' - file name expected - clause is ignored`

A file name was expected to appear at this position in the statement.

2202 `RERUN file + '`*`character-string`*`' not selected or invalid - clause is ignored`

The name associated with the RERUN clause must be a file name that appeared in a SELECT clause.

2203 `RERUN clause '`*`character-string`*`' is not implemented - for documentation only`

The specified phrase in the RERUN clause is for documentation purposes only.

2204 `File '`*`character-string`*`' already named in RERUN clause of same type - clause is ignored`

The file name already appears in a RERUN clause of the same type.

2205 `MULTIPLE FILE TAPE '`*`character-string`*`' invalid - clause is ignored`

The integer specified in the MULTIPLE FILE clause is illegal.

2207 `File name already is specified in a MULTIPLE FILE TAPE clause - file is ignored in this clause`

The same file name must not be specified more than once in a MULTIPLE FILE TAPE

clause.

2208 `RERUN file 'character-string' is not SEQUENTIAL ORGANIZATION - clause is ignored`

The file associated with the RERUN clause must be sequential.

2209 `MULTIPLE FILE TAPE 'character-string' not selected or invalid - clause is ignored`

The name associated with the MULTIPLE FILE TAPE clause did not appear in a SELECT clause.

2210 `SAME clause file name 'character-string' not valid - clause is ignored`

The name associated with the SAME clause is not a valid file name. Either the name does not reference a file or the file was not selected in this program. It is also illegal for a shared file (i.e. ASSIGN TO A-SHARED-FILE) to be specified in the SAME AREA clause.

2211 `'character-string' : file name already is specified in a SAME clause of this type - file is ignored in this clause`

A file name must not appear in more than one SAME RECORD AREA clause.

2212 `'character-string' is invalid phrase in SAME clause - phrase is ignored`

The specified phrase is illegal for use in a SAME AREA clause.

2213 `Conflict in use of SAME clause`

The file specified in the SAME AREA clause is not valid.

2214 `FD or SD already specified for this file - clause is ignored`

A FD or SD clause has already been processed for this statement.

2215 `ORGANIZATION clause illegal for sort files - clause is ignored`

The ORGANIZATION clause is not valid within a SD for a sort file.

2216 `ACCESS MODE clause illegal for sort files - clause is ignored`

The ACCESS MODE clause is not valid within a SD for a sort file.

2218 `RESERVE clause illegal for sort files - clause is ignored`

The RESERVE clause is not valid within a SD for a sort file.

2219 `RERUN clause illegal for sort files - clause is ignored`

The RERUN clause is not valid within a SD for a sort file.

2220 `EXTEND phrase must only be used for files in the sequential access mode - the statement is ignored`

The file name specified in the OPEN EXTEND statement must have ACCESS SEQUENTIAL specified in its SELECT clause.

2221 `PADDING CHARACTER clause illegal for sort files - clause is ignored`

The PADDING CHARACTER clause is not valid within a SD for a sort file.

2222 `FILE STATUS clause illegal for sort files - clause is ignored`

The FILE STATUS clause is not valid within a SD for a sort file.

2223 `RECORD DELIMITER clause illegal for sort files - clause is ignored`

The RECORD DELIMITER clause is not valid within a SD for a sort file.

2224 `MULTIPLE FILE TAPE clause illegal for sort files - clause is ignored`

The MULTIPLE FILE TAPE clause is not valid within a SD for a sort file.

2225 `RELATIVE KEY clause illegal for sort files - clause is ignored`

The RELATIVE KEY clause is not valid within a SD for a sort file.

2226 `RECORD or ALTERNATE KEY clause illegal for sort files - clause is ignored`

The RECORD or ALTERNATE KEY clause is not valid within a SD for a sort file.

2227 `No FD or SD specified for file name`

No FD or SD clause was processed for the specified file name.

2228 `File has both record descriptions & reports specified - accepted`

The description of a file that has the REPORTS clause associated with it should not also have a record description.

2229 `Sort file is illegally named in a SAME AREA clause - file is ignored`

Files that are specified in a SD clause cannot appear in the SAME AREA clause.

2230 `Only 1 file is specified in a SAME clause - clause is ignored`

Two or more files must be specified in the SAME AREA clause.

2231 `No sort file is specified in a SAME SORT-MERGE clause`

The file names specified in the SAME SORT-MERGE AREA clause must have been defined in a sort-merge file description entry.

2232 `Item is illegal in this statement - statement is ignored`

The data name used to specify a file was not defined anywhere in the program.

2233 '*character-string*' `not file name - statement is ignored`

The data name used to specify a file was not defined as a file in the Data Division.

2234 '*character-string*' `has no environment entry - statement is ignored`

No environment entry was allocated for the file, resulting in the file not being processed.

2235 '*character-string*' `: statement options are inapplicable to file type - options are ignored`

The specified clause is not appropriate with the given file type.

2236 '*character-string*' `: ACCESS MODE is inapplicable to file type - statement is ignored`

The ACCESS MODE clause should not be used with the given file type.

2237 `INVALID KEY/NOT INVALID KEY phrase is illegal with SEQUENTIAL ACCESS MODE - phrase is ignored`

The INVALID KEY/NOT INVALID KEY phrase should not be used with a sequential file.

2238 '*character-string*' : statement is inapplicable to file type - statement is ignored

The specified clause should not be used with the given file type.

2239 '*character-string*' : READ NEXT statement is illegal with a RANDOM ACCESS file - NEXT phrase is ignored

The NEXT phrase in a READ statement cannot be used when the file being read is a RANDOM ACCESS file.

2240 '*character-string*' is object of an APPLY EXREF clause - USE statement is ignored

The file name used in an USE statement cannot be an object of an APPLY EXREF clause. If a file is shared using APPLY EXREF/APPLY EXDEF, any input/output reference to the file name in a USE statement must be in a separate compilation. The reference must be in the main program and have the file name as the object of the APPLY EXDEF clause.

2241 AT END/EOP phrase is illegal with file ORGANIZATION &/or ACCESS MODE specified - phrase is ignored

The AT END or END-OF-PAGE phrase cannot be used with a file that has the type of ORGANIZATION or ACCESS MODE specified.

2242 OPEN OUTPUT of an OPTIONAL file is illegal - accepted

The OPTIONAL phrase applies only to files opened in the input, I-O, or extend mode.

2243 '*character-string*' operand is unrecognizable - FROM/INTO phrase is ignored

The operand associated with the FROM or INTO phrase is invalid.

2244 '*character-string*' illegal FROM/INTO identifier - statement ignored

The operand associated with the FROM or INTO phrase is invalid.

2245 '*character-string*' no file records are defined - FROM/INTO phrase is ignored

No records have been defined for the specified file; therefore the FROM or INTO phrase is ignored.

2246 READ file name INTO phrase is illegal, more than one file record and all records not alphanumeric - statement ignored

The INTO phrase is not legal in the given file. If there is more than one record described for the file, all records must be either a group or elementary alphanumeric.

2247 '*character-string*' read key is illegal - KEY IS phrase is ignored (Not INDEXED file, sequential READ, or KEY is not a file RECORD KEY)

The key specified in the KEY IS phrase is not valid.

2251 '*character-string*' - START statement is illegal on a RANDOM ACCESS file - statement is ignored

A START statement is not valid on a file that has RANDOM ACCESS specified on it.

2252 START statement is illegal on a relative file with no RELATIVE KEY specified - statement is ignored

A START statement is not legal on a relative file unless the RELATIVE KEY phrase has been specified.

2253 '*character-string*' : START KEY is illegal - KEY is ignored

The key specified in a START statement is illegal.

2254 '*character-string*' not defined - statement is ignored

The data name specified is undefined.

2255 '*character-string*' not record name - statement is ignored

A record name was expected at this point in the statement.

2258 '*character-string*' mnemonic name is illegal operand with LINAGE clause

The ADVANCING mnemonic name phrase cannot be specified when writing a record to a file which is associated with a file description entry containing a LINAGE clause.

2259 '*character-string*' ADVANCING phrase operand is illegal - ADVANCING phrase is ignored

The operand specified with the ADVANCING phrase in a WRITE statement is illegal.

2260 AT EOP phrase is illegal without LINAGE clause - phrase is accepted

The END-OF-PAGE or the NOT END-OF-PAGE cannot be specified unless the LINAGE clause is specified in the file description entry of the associated file.

2261 ADVANCING phrase and AT EOP phrase are illegal with a non print file - phrase is ignored

The END-OF-PAGE phrase must be specified only with a print file. The ADVANCING phrase must be specified only with a print file.

2262 AT EOP phrase is illegal with ADVANCING PAGE phrase - phrase is accepted

The phrases ADVANCING PAGE and END-OF-PAGE must not both be specified in a single WRITE statement.

2263 Illegal ACCEPT identifier - statement is ignored

The identifier used in the ACCEPT statement is not valid. The ACCEPT statement is ignored.

2264 '*character-string*' illegal ACCEPT mnemonic name - phrase is ignored and default device is used

The mnemonic name must be specified in the SPECIAL-NAMES paragraph of the Environment Division and must be associated with the implementor names CARD-READER or CONSOLE.

2265 '*character-string*' illegal : mnemonic name must be associated with CARD-READER or CONSOLE - name is ignored, default device is used

The mnemonic name must be specified in the SPECIAL-NAMES paragraph of the Environment Division and must be associated with the implementor names CARD-READER or CONSOLE.

2266 `INVALID KEY phrase is illegal with SEQUENTIAL ORGANIZATION - phrase is ignored`

The INVALID KEY phrase is illegal on a sequential file.

2267 '*character-string*' illegal DISPLAY mnemonic name - phrase is ignored and default device is used

The mnemonic name must be specified in the SPECIAL-NAMES paragraph of the Environment Division and must be associated with the implementor names CARD-PUNCH, PRINTER, or CONSOLE.

2268 '*character-string*' illegal : mnemonic name must be associated with CARD-PUNCH, PRINTER or CONSOLE - name is ignored, default device is used

The mnemonic name must be specified in the SPECIAL-NAMES paragraph of the Environment Division and must be associated with the implementor names CARD-PUNCH, PRINTER, or CONSOLE.

2269 `Illegal DISPLAY identifier or literal - operand is ignored`

The specified operand is not legal in a DISPLAY statement.

2270 '*character-string*' : READ/WRITE/REWRITE statement is not allowed for a report file - statement is accepted

The READ/WRITE/REWRITE statements must not refer to a file that is described in the Report Section.

2271 '*character-string*': illegal to have two USE statements for the same access mode or for the same file name in the same program - this declarative statement is ignored for this access mode or this file

The appearance of a file name cannot occur in more than one USE statement.

2272 `RELATIVE KEY should be described as EXTERNAL for an EXTERNAL file - accepted`

If a file has been described as EXTERNAL, then its RELATIVE KEY should also be described as EXTERNAL.

2273 `PADDING CHARACTER data name should be described as EXTERNAL for an EXTERNAL file - accepted`

If a file has been described as EXTERNAL, then its PADDING CHARACTER should also be described as EXTERNAL.

2274 '*character-string*' illegal ACCEPT identifier, an identifier with USAGE DISPLAY-2 cannot be used with device CONSOLE

An identifier with USAGE DISPLAY-2 cannot be used in an ACCEPT statement when CONSOLE is used as the input device. If no device is specified by the FROM clause in the ACCEPT statement, then CONSOLE is used as the default.

2275 `This file is assigned to DISC, REEL or UNIT option is not a valid clause - statement is accepted`

The REEL or UNIT phrase must be used only for files assigned to TAPE.

2276 `AT END/INVALID KEY phrase is required for this statement - statement accepted`

The INVALID KEY phrase or the AT END phrase must be specified if no applicable USE AFTER STANDARD EXCEPTION procedure is specified for the file. No AT END, INVALID KEY, or DECLARATIVE code is present.

2277 `This procedure name cannot be contained in the declaratives portion of the program`

Procedure name used in the WITH ENTRY POINTS option of the Procedure Division cannot be contained in the declaratives portion of the program.

2278 `This file's organization must be SEQUENTIAL or INDEXED and be assigned to DISC - statement is ignored`

In format 2 of the ENTER statement, the file name must refer to an indexed or sequential file assigned to DISC in the program.

2279 `This item must be 01/77 level elementary item with PIC 9(10) USAGE IS BINARY - statement is ignored`

The data name must be 01/77 level PIC 9(10) and the USAGE IS BINARY item. For sequential files, this has the number of the active records after the execution of the ENTER statement.

2280 `No RECORD KEY present or it was in error - statement is ignored`

No RECORD key was given for the file or the RECORD KEY given for the file was in error.

2281 `The data item containing USING POINTER clause is illegal`

The data item associated with the USING POINTER must be an 01 item that is declared in the Working-Storage section. In addition, a REDEFINES clause might not be used with this data declaration.

2282 `EXTERNAL file cannot be specified with USING POINTER`

The data item associated with the USING POINTER might not also specify the EXTERNAL clause.

2283 `The USING POINTER data item is invalid`

The data item used with the USING POINTER clause must be declared as PIC 1(36) BINARY-1. In addition, it must not be defined subordinate to the data item that contains the USING POINTER clause.

2300 `'`*`character-string`*`' must be only 1 character in length when associated with a THROUGH or ALSO phrase - clause ignored`

If the literal associated with the THROUGH or ALSO phrase in the ALPHABET or CLASS clause is nonnumeric, then it must be 1 character in length.

2301 `ALPHABET name '`*`character-string`*`' must be greater than zero - ALPHABET clause ignored`

If the literal associated with the THROUGH or ALSO phrase in the ALPHABET clause is numeric, then it must be within the range of 1 to the maximum number of characters in the native character set.

2302 `ALPHABET name '`*`character-string`*`' must be unsigned - ALPHABET clause ignored`

If the literal associated with the THROUGH or ALSO phrase in the ALPHABET clause is numeric, then it must be unsigned.

2303 `ALPHABET name '`*`character-string`*`' must be either an integer, figurative constant or a quoted string - ALPHABET clause ignored`

The specified operand is not valid in the ALPHABET clause.

2304 `ALPHABET name '`*`character-string`*`' is longer than 256 characters - ALPHABET clause ignored`

The specified character in the ALPHABET clause is not within the range of the character set.

2305 `ALPHABET name '`*`character-string`*`' is defined out of range (1 to 256) - ALPHABET clause ignored`

The specified character in the ALPHABET clause is not within the range of the character set.

2306 `ALPHABET name '`*`character-string`*`' appears more than once - ALPHABET clause ignored`

If the literal phrase of the ALPHABET clause is specified, a given character must not be specified more than once.

2310 `ALPHABET name '`*`character-string`*`' previously defined - ALPHABET clause ignored`

The ALPHABET name has already been defined.

2401 `RDML ending delimeter ]$ is missing - assumed`

A RDML statement must have an ending delimiter of]$.

2402 `Use of $ for variables within qualification expression prohibited`

The use of a $ in a qualified expression is illegal since it can be interpreted incorrectly by RDML.

2501 `Data name associated with INITIATE or TERMINATE statement must be report name - statement is ignored`

The report name associated with an INITIATE or TERMINATE statement must be defined by a report description entry in the Report Section of the Data Division.

2507 `Report name not specified in REPORT clause of a file - statement is omitted`

The REPORT clause specifies the names of reports that comprise a report file. A report name must appear in at least one REPORT clause.

2509 `CLOSE file name REEL/UNIT WITH NO REWIND is not permitted - clause ignored`

The CLOSE of a REEL/UNIT WITH NO REWIND is not a permitted. The clause is ignored.

2510 `Identifier associated with GENERATE statement must be a DETAIL report group or a report description`

The data name in GENERATE statement must be a TYPE DETAIL report group and can be qualified by a report name.

2511 `Report name on a GENERATE statement must have a CONTROL clause specified with it`

The report name of GENERATE statement can be used only if the referenced report description contains a CONTROL clause.

2512 `Report name on a GENERATE statement must have at least one body group`

The report name of GENERATE statement can be used only if the referenced report description contains at least one body group.

2513 `Report name on a GENERATE statement must have no more than one DETAIL report group in its report description`

The report name of GENERATE statement can be used only if the referenced report description contains not more than one DETAIL report group.

2514 `RESET must not specify a lower level CONTROL than the CONTROL FOOTING associated with it`

The data name associated with RESET must be one of the data names specified in the CONTROL clause for this report. It must not be a lower level control than the associated control for the report group in which the RESET phrase appears.

2515 `USE BEFORE REPORTING data name must reference a report group`

The identifier in the USE BEFORE REPORTING statement must refer to a report group.

2516 `SUPPRESS can only appear in a USE BEFORE REPORTING procedure`

The SUPPRESS statement can appear only in a USE BEFORE REPORTING procedure.

2517 `USE BEFORE REPORTING data name must not appear in more than one USE BEFORE REPORTING statement`

The identifier in the USE BEFORE REPORTING statement must not appear in more than one USE BEFORE REPORTING statement.

2518 `INITIATE/GENERATE/TERMINATE cannot appear in a USE BEFORE REPORTING procedure`

The GENERATE, INITIATE, or TERMINATE statements must not appear in a paragraph within a USE BEFORE REPORTING procedure. A PERFORM statement in a USE BEFORE REPORTING procedure must not have GENERATE, INITIATE, or TERMINATE statements in its range.

3001 `The sending operand is illegal in a SET statement`

The sending identifier must be a variable, or UP/DOWN was expected and not found in the SET statement.

3002 `Illegal item used as sending operand in the SET statement`

The sending operand of the SET statement must be a variable name, a constant, or an array, and its data type must be index or arithmetic.

3003 `The operands are not compatible together in a SET`

If the sender is an index data item, the receiver must be a index name or index data item. If sender is an index name, the receiver must be an index name, index data item or numeric. If sender is numeric, the receiver must be an index name.

3004 `The TO object of the SET statement is missing an index name or integer`

The sender in a SET statement must be an index name or an integer.

3005 `The literal must be a positive nonzero integer`

The sender in a SET statement has a value less than 0.

3006 `Operand must be a condition name`

The receiver of this SET statement must be a condition name.

3007 `Only mnemonic name is allowed for SWITCH-13 through SWITCH-24`

The identifier of this SET statement must be a mnemonic name for a switch, and the switch number must be between 13 and 24.

3008 `The receiving operand is illegal in this SET statement`

The receiving identifier of a SET statement is the wrong type.

3009 `Illegal item used as receiving operand in the SET statement`

The receiver of the SET statement is not legal with the given sender.

3010 `Literal exceeds maximum occurrences for the index name`

The sender of a SET statement is a constant that is greater than the maximum of the array associated with the index name.

3011 `There is a limit of 98 procedure names allowed as parameters on a CALL or ENTER statement.`

There is a limit of 98 procedure names as parameters on a CALL or an ENTER statement.

3012 `EXIT PROGRAM not allowed in declarative procedure in which GLOBAL is specified`

The EXIT PROGRAM statement must not appear in a declarative procedure in which the GLOBAL phrase is specified.

3025 `The PERFORM FROM literal should be within the permitted range of the index`

When the VARYING identifier is an index name, the FROM literal should be equal to an occurrence of the table associated with the index name.

3026 `GO TO is from nondeclarative section to DECLARATIVE section or from one DECLARATIVE section to another - statement accepted`

Procedure names associated with a USE statement cannot be referenced by GO TO statement in a different declarative procedure or in a nondeclarative procedure. It

is accepted.

3027 `Object of GO TO statement is outside declarative procedure – statement is ignored`

Within a declarative procedure, there must be no reference to any nondeclarative procedures.

3028 `Identifier must be subscripted`

The DEPENDING identifier on a GO TO statement is not an array item.

3029 `Identifier must be elementary item – statement is ignored`

The DEPENDING identifier on a GO TO statement is a structure.

3030 `Identifier must be integer item – statement is ignored`

The DEPENDING identifier on a GO TO statement is not an integer.

3031 `Resultant sign stripped upon storage`

The sign of the item is stripped.

3032 `Illegal identifier for GO TO statement`

The DEPENDING identifier on a GO TO statement is in error or has an illegal data type.

3033 `An illegal procedure name is used in the GO TO statement – statement is ignored`

One of the procedure names used in a GO TO statement with DEPENDING ON option is in error or illegal.

3034 `Procedure name is required after GO TO statement – statement is omitted`

At least one procedure name must follow the GO on a GO TO statement.

3035 `Numeric/numeric edited move to alphabetic illegal – MOVE statement is ignored`

On a MOVE statement, moving from a numeric field to an alphabetic field is illegal.

3036 `Alphabetic move to numeric/numeric edited illegal – MOVE statement is ignored`

On a MOVE statement, moving from an alphabetic field to a numeric field is illegal.

3037 `Alphanumeric edited move to numeric/numeric edited illegal – MOVE statement is ignored`

On a MOVE statement, moving from an alphanumeric edited field to a numeric edited field is illegal.

3038 `Non-integer move to alphanumeric or alphanumeric edited illegal – MOVE statement is ignored`

On a MOVE statement, moving from a non-integer field to an alphanumeric or

alphanumeric edited field is illegal.

3039 `Index name or index data item is illegal for MOVE statement operand - MOVE statement is ignored`

Neither the sender nor the receiver can be an index name or index data name on a MOVE statement.

3040 `Illegal operand for MOVE statement or implied MOVE statement - statement ignored`

Either the sender or the receiver is an illegal type on a MOVE statement or an implied MOVE statement. Implied MOVE statements are verbs that move data from one variable to another and follow the same rules as the MOVE. An example of an implied MOVE would be a verb that contains the syntax INTO/FROM in order to move data from one location to another.

3041 `Operand for MOVE statement is not dimensioned - MOVE statement is ignored`

Either the sender or the receiver on a MOVE statement must be dimensioned.

3042 `Operand for MOVE CORRESPONDING statement must be group item - MOVE statement is ignored`

Either the sender or the receiver on a MOVE statement is not a structure.

3043 `No match for MOVE CORRESPONDING statement in receiving item - nothing moved`

The receiver did not match the sender on a MOVE CORRESPONDING statement.

3045 `Second operand missing - statement is omitted`

There was no receiver on an ADD CORRESPONDING statement.

3046 `Reference modification is illegal in this position - statement is omitted`

Reference modification is illegal in this statement.

3047 `Operand must be group item - statement is omitted`

The item is not a structure on an ADD CORRESPONDING statement.

3048 `No match for CORRESPONDING items - statement is ignored`

The receiver did not match the sender on an ADD CORRESPONDING statement.

3049 `66 level or an index data item is illegal for CORRESPONDING - statement is ignored`

Either the sender or the receiver on either an ADD CORRESPONDING, SUBTRACT CORRESPONDING, or MOVE CORRESPONDING is a 66 level item or an index data item.

3050 `Number of parameters in Linkage Section must equal number of USING parameters`

The number of parameters on the USING clause must equal the number of items in the Linkage Section.

3051 `Program flow does not reach this statement or these statements`

The object code for this statement has been generated; however, it cannot be reached in normal program execution.

3052 `Executable statement found with EXIT - EXIT statement is ignored`

The EXIT statement must be the only sentence in a paragraph.

3053 `Program flow cannot reach this statement or these statements`

The object code for this statement has been generated. However, unless this program is executed as a main program (the preceding EXIT PROGRAM is ignored), it cannot be reached in normal program execution.

3054 `The USING data name is in error - name rejected`

The USING data name is not a valid parameter.

3055 `Parameter level number must be 77 or 01 - 01 parent used`

The parameters must not be subordinate to any other data item.

3056 `Data name must not be redefined - name is rejected`

The data name used must not be the subject of a REDEFINE clause.

3057 `Data name must be defined in Linkage Section - name is accepted`

The data name must be defined in the Linkage Section.

3058 `Numeric literal in STOP must be unsigned integer`

The STOP literal must be an unsigned integer.

3059 `Illegal operand for CALL statement - statement omitted`

The CALL data item must be a 01/77 or an elementary item.

3060 `Non-standard CALL..USING parameter. The parameter must be an 01/77 item, an elementary dataname or be reference-modified. The statement is accepted.`

The CALL..USING parameter is a non-record-level group item.

3061 `Syntax error in PERFORM statement - statement is omitted`

Illegal syntax has been encountered in a PERFORM statement. The TIMES, UNTIL or VARYING option is expected on the PERFORM statement, or an illegal AFTER was encountered in an inline PERFORM.

3062 `Either the range of this PERFORM statement is not completely within one DECLARATIVES section, or the PERFORM is within DECLARATIVES and the range of the PERFORM is not - statement ignored`

If either the first or last paragraph name specified in a PERFORM ... THRU statement is within DECLARATIVES, the other paragraph must be within the same declarative section. Also, a PERFORM statement within DECLARATIVES can reference only paragraph or section names within DECLARATIVES.

3063 `Item must be numeric - statement is omitted`

The value of the PERFORM VARYING BY item is not numeric.

3064 `Item must be positive integer - statement is omitted`

The value of the PERFORM VARYING BY item is negative.

3065 `Item must be a numeric integer - statement is omitted`

The PERFORM item is not a whole number.

3066 `Item must not be zero - statement is omitted`

The PERFORM VARYING BY VALUE item is 0.

3067 `Paragraph cannot be altered - statement omitted`

The ALTER paragraph is not a valid paragraph name to be used in an ALTER statement.

3068 `Entry point must be a procedure name`

A paragraph name is expected at this point in the program.

3069 `Duplicate entry points not permitted`

This procedure name has already been used as an entry point.

3070 `A CALL cannot be made to a separately compiled program with the same PROGRAM-ID as the main program`

A CALL cannot be made to a separately compiled program with the same PROGRAM-ID as the main program.

3071 `Data name must not be in USING clause twice - second ignored`

A parameter has been repeated on the USING clause.

3072 `Number of CALL parameters not equal number of Procedure Division USING parameters - CALL removed`

The number of CALL parameters is less than the number of Procedure Division USING parameters.

3073 `APPLY clause not permitted with GLOBAL or EXTERNAL`

The data item has a GLOBAL or EXTERNAL clause.

3074 `The value of LINE-COUNTER and LINAGE-COUNTER cannot be explicitly changed`

The value of LINE-COUNTER and LINAGE-COUNTER cannot be explicitly changed by any command in the source program.

3075 `APPLY clause not permitted in nested or initial programs`

The APPLY clause is not allowed in nested programs or in programs in which INITIAL status has been specified.

3076 `BEFORE/AFTER phrase is illegally repeated in INSPECT statement`

No more than one BEFORE phrase and one AFTER phrase can be specified for any one ALL, LEADING, CHARACTERS, FIRST or CONVERTING phrase.

3077 `Elementary item required - statement is ignored`

An elementary item was expected and a structure was found.

3078 `Numeric literal in INSPECT statement - literal is accepted`

The INSPECT item must be nonnumeric.

3079 `DISPLAY, DISPLAY-2 or DISP-I18N USAGE required here - the statement is ignored`

All literals must be described as nonnumeric literals or Japanese nonnumeric literals, and all identifiers must be described implicitly or explicitly with the USAGE IS DISPLAY, DISPLAY-2 or DISP-I18N clause.

3080 `Elementary unedited numeric item required - statement is omitted`

The INSPECT, STRING or UNSTRING data item must be numeric.

3081 `REPLACING subject and object sizes are not equal`

The INSPECT subject item size and object item sizes are different.

3083 `BEFORE/AFTER variable should be one character long when it is associated with the CHARACTERS clause - accepted`

On an INSPECT REPLACING CHARACTERS statement, the BEFORE/AFTER item is larger than 1 character.

3084 `APPLY clause data name not permitted`

The APPLY data item is illegal. It must be a file name specified in a SELECT statement as well as in a FD or SD clause, or a Working-Storage level 01 data name that is not overlaid. When APPLY EXREF is specified, the VALUE clause must not be present for the EXREF data name. The APPLY data item must not be associated with a USING POINTER clause.

3085 `The literal on the STOP statement must not have the word ALL associated with it - the word ALL is ignored`

The literal in a STOP statement must not have the word ALL associated with it.

3086 `This is an illegal operand, it must have type ALPHANUMERIC - statement is ignored`

The identifier used in this statement must be of type ALPHANUMERIC.

3087 `Matching operands for the CORRESPONDING clause are from line numbers :` *`character-string`* `and` *`character-string`*

Sending and receiving item line numbers in a MOVE/ADD/SUBTRACT CORRESPONDING statement.

3088 `Move of ALPHANUMERIC item to NUMERIC or NUMERIC EDITED can give unpredictable results`

Move of alphanumeric items to numeric or numeric edited, even though it is legal, can produce unpredictable results.

3089 `Illegal CALL parameter - accepted`

The CALL parameter must be a 01/77 level or an elementary item.

3090 `An identifier can occur only once in an APPLY clause of a program, and must be unique in the first 12 characters - identifier ignored`

An identifier can occur only once in an APPLY clause of a program and must be unique in the first 12 characters.

3091 `The CALL literal cannot be greater than 30 characters, which is the maximum size of COBOL name - truncated`

The CALL literal cannot be greater than 30 characters, which is the maximum size of a COBOL program name.

3092 `The CONTROL name '`*`character-string`*`' cannot be referenced in this DECLARATIVE section - statement ignored`

Neither the sender nor the receiver can reference a CONTROL name in any statement in USE BEFORE REPORTING DECLARATIVE section for a PAGE HEADING or a PAGE FOOTING.

3093 `Maximum of '`*`integer-value`*`' parameters allowed on a CALL USING or PROCEDURE DIVISION USING statement`

The maximum number of parameters that is allowed to be passed or received in a UCS COBOL program has been reached. The number of parameters in the CALL USING or PROCEDURE DIVISION USING statement must be reduced.

3094 `Key used in SORT/MERGE statement must not be USAGE DISP-I18N unless the keyword option I18N-IO was specified.`

A data item associated with the key clause in a SORT or MERGE statement is declared as USAGE DISP-I18N. This is not legal unless the keyword option I18N-IO was specified.

3095 `Linkage Section parameter 'data-name' or subordinate item is referenced in a COBOL statement but not specified in USING clause.`

If a LINKAGE SECTION 01 level item is not specified in the PROCEDURE DIVISION USING clause for that program, that item or any data item within that item cannot be referenced by any statement in the program. Any code generated for the statement will result in runtime error.

4001 `Syntax error in STRING - statement is omitted`

Literals must not be figurative constants that begin with the word ALL. All literals must be described as nonnumeric. All identifiers, except following POINTER, must be described as USAGE IS DISPLAY.

4002 `Only integer values can be used - statement is ignored`

An identifier must be described as an elementary numeric integer data item of sufficient size to contain a value equal to 1 plus the size of the item referred to by the INTO identifier. The symbol P cannot be used in the PICTURE character string of any identifier that is an integer.

4003 `Must be nonnumeric literal - statement is ignored`

All literals must be described as nonnumeric literals, and all identifiers, except the one following POINTER, must be described as USAGE IS DISPLAY.

4004 `Illegal operand in a STRING - statement is omitted`

Literals must not be figurative constants that begin with the word ALL. All literals must be described as nonnumeric literals. All identifiers, except those following POINTER, must be described as USAGE IS DISPLAY. Identifier following INTO must not be reference modified or must not represent an edited data item.

4005 `Receiving item must not be JUSTIFIED - statement is omitted`

The identifier following INTO must not represent an edited data item and must not be described with the JUSTIFIED clause.

4006 `Identifiers or literals must not have mixed usage of DISPLAY and DISPLAY-2 - the statement is omitted`

If one of the identifiers is described as USAGE IS DISPLAY-2, other identifiers must be described as USAGE IS DISPLAY-2, and all literals must be Japanese nonnumeric literals. If one of the identifiers is described as other than USAGE IS DISPLAY-2, other identifiers must not be described as USAGE IS DISPLAY-2 and no literal can be a Japanese nonnumeric literal.

4026 `Illegal operand for UNSTRING - statement is omitted`

The operand specified in the UNSTRING statement is illegal. Refer to the rules for the UNSTRING statement in the reference manual to see the correct usages of the various data items in an UNSTRING statement.

4027 `Alphanumeric item expected here - item is accepted`

The identifiers following UNSTRING, DELIMITED, OR ALL, and DELIMITER, must refer to data items described, implicitly or explicitly, as category alphanumeric.

4028 `Editing of this item ignored`

The identifier following INTO can be described as either the category alphabetic, alphanumeric, or numeric (except that the symbol P cannot be used in the PICTURE character string), and must be described, implicitly or explicitly, as USAGE IS DISPLAY.

4029 `DELIMITER is meaningless when no DELIMITED BY phrase specified`

The DELIMITER IN phrase and the COUNT IN phrase can be specified only if the DELIMITED BY phrase is specified.

4030 `COUNT is meaningless when no DELIMITED BY phrase specified`

The DELIMITER IN phrase and the COUNT IN phrase can be specified only if the DELIMITED BY phrase is specified.

4031 `Illegal clause used with CALL 'LOC$'`

The use of the ON EXCEPTION, NOT ON EXCEPTION, ON OVERFLOW, BY REFERENCE, and BY CONTENT clauses are not allowed in the CALL 'LOC$' statement. The CALL statement is ignored.

4032 `Incorrect number of parameters in CALL 'LOC$' statement`

The CALL 'LOC$' statement must specify exactly two parameters. Either too few or too many parameters have been specified. The CALL statement is ignored.

4033 `Invalid parameter used in CALL 'LOC$'`

An invalid parameter was used in the CALL 'LOC$' statement. The first parameter must be an 01 level item. The second parameter must be defined as PIC 1(36)

BINARY-1. Neither parameter may reference an array or be referenced modified. The CALL statement is ignored.

4052 `Missing index name, index data item, or integer - statement is omitted`

The identifier following VARYING must refer to a data item described as USAGE IS INDEX or as an integer.

4053 `Identifier after SEARCH must have an index name - statement is omitted`

The identifier following SEARCH or SEARCH ALL must not be subscripted or reference modified, but its description must contain an OCCURS clause including an INDEXED BY phrase.

4054 `Not an array - statement is omitted`

The identifier following SEARCH and SEARCH ALL must not be subscripted or reference modified, but its description must contain an OCCURS clause including an INDEXED BY phrase.

4055 `Incomplete KEY tests for SEARCH ALL statement - statement is omitted`

In SEARCH ALL when a data name, or a condition name associated with a data name, in the KEY IS phrase in the OCCURS clause referred to by the SEARCH ALL identifier is referenced, all preceding data names or their associated condition names must also be referenced.

4057 `Identifier for SEARCH ALL statement must contain a KEY clause - statement is omitted`

The identifier following SEARCH ALL must contain the KEY IS phrase in its OCCURS clause.

4058 `Data name illegal in a VARYING clause - statement is omitted`

The identifier following VARYING must refer to a data item described as USAGE IS INDEX or as an integer.

4059 `VARYING identifier must not be subscripted by index from identifier-1 - statement is omitted`

The identifier following VARYING cannot be subscripted by the first (or only) index name specified in the INDEXED BY phrase in the OCCURS clause associated with the SEARCH identifier.

4060 `Scope of SEARCH statement must terminate following NEXT SENTENCE - statement is omitted`

If the END-SEARCH phrase is specified, the NEXT SENTENCE phrase must not be specified.

4061 `Subject of WHEN clause must be a subscripted item - statement is omitted`

Each data name of the WHEN clause must be subscripted by the first index name associated with the SEARCH ALL identifier along with other subscripts as required.

4062 Must be subscripted by first index name - statement is omitted

Each data name of the WHEN clause must be subscripted by the first index name associated with the SEARCH ALL identifier along with other subscripts as required.

4063 Must be referenced in KEY phrase of OCCURS clause - statement is omitted

Each data name of the WHEN clause must be referred to in the KEY IS phrase in the OCCURS clause referred to by the SEARCH ALL identifier.

4064 Duplicate keys appear in clause - only last one is used

The same data name or condition name cannot be referenced more than once in the WHEN clause of the SEARCH ALL statement.

4065 Condition name must be defined as having only a single value - statement is omitted

With SEARCH ALL, all referenced condition names must be defined as having only a single value.

4066 First index name must not be used in object of the WHEN clause - statement is omitted

Identifiers, including identifiers specified in arithmetic expressions, in the WHEN clause of the SEARCH ALL statement must not be subscripted by the first index name associated with the SEARCH ALL identifier.

4067 KEY name must not appear in object of the WHEN clause - statement is omitted

Identifiers, including identifiers specified in arithmetic expressions, in the WHEN clause of the SEARCH ALL statement must not be referenced in the KEY IS phrase in the OCCURS clause referred to by the SEARCH ALL identifier.

4068 Too many nested SEARCH ALL statements - statement is omitted

The maximum number of nested SEARCH ALL statements is ten.

4076 A CLASS name has been used incorrectly as a condition in an EVALUATE statement - statement is omitted

A CLASS name has been used as one of the conditions in an EVALUATE statement. The CLASS name must be used with another identifier in order to form a valid condition for use in the EVALUATE statement.

4077 Number of subjects and objects in an EVALUATE statement must be equal - statement is omitted

The number of selection objects within each set of selection objects must be equal to the number of selection subjects.

4078 Operands in THROUGH phrase must have the same data class - statement is omitted

The operands connected by a THROUGH phrase must be of the same class. The two operands thus connected constitute a single selection object.

4079 The WHEN clause prior to the WHEN OTHER clause is missing an imperative statement — statement is omitted.

The imperative statement that is associated with the WHEN OTHER clause cannot be used for a previous WHEN statement. The previous WHEN clause is missing an imperative statement and must have one affiliated with it.

4100 `Parameter must be a file name`

The first parameter must be the file name for the data information being requested.

4101 `Parameter must be an 01 level data name`

The second parameter must be an 01 level data name in Working-Storage or File Section to receive the information being requested.

4102 `INTRESULT specified - this operation will use ASCII COBOL intermediate results algorithm`

The keyword option INTRESULT has been specified and the current statement will use the ASCII COBOL algorithm to calculate the intermediate results. Operations that qualify for this algorithm are * (multiply) in a COMPUTE statement and a DIVIDE statement that contains multiple receivers that have different precisions and scales. The result of doing this operation will be equivalent to the result that is received in ASCII COBOL. This result may contain less precision than the result using the standard UCS COBOL intermediate results algorithm.

4103 `Operation mode is float decimal, INTRESULT ignored for entire statement - split up statement for INTRESULT`

The COMPUTE statement contains at least one multiply (*) operator and one or more of the operators +, -, or /. The additional operators have forced the calculation mode of this statement to float decimal. The keyword option INTRESULT was specified, however it will be ignored for this statement. In order for the INTRESULT to have an affect on the multiply in this COMPUTE statement, the statement must be split up into two or more COMPUTE statements.

4104 `This statement will use ASCII COBOL intermediate results algorithm`

The keyword option INTRESULT has been specified and the current statement will use the ASCII COBOL algorithm to calculate the intermediate results. Operations that qualify for this algorithm are * (multiply) in a COMPUTE statement and a DIVIDE statement that contains multiple receivers that have different precisions and scales. The result of doing this operation will be equivalent to the result that is received in ASCII COBOL. This result may contain less precision than the result using the standard UCS COBOL intermediate results algorithm.

5001 `Index data item illegal in an INITIALIZE statement - statement is omitted`

An index data item cannot appear as an operand of an INITIALIZE statement.

5002 `Data type of sender must be compatible with category in REPLACING phrase - statement is omitted`

Each category stated in the REPLACING phrase must be a permissible category as a receiving operand in a MOVE statement where the corresponding data item referenced by the identifier of the literal following DATA are used as the sending operand. All operations are performed as if a series of MOVE statements had been written.

5003 `Illegal operand in an INITIALIZE statement - statement is omitted`

The description of the data item referenced by the identifier following INITIALIZE or any items subordinate to it cannot contain the DEPENDING phrase of the OCCURS clause. The data description also must not contain a RENAMES clause. An index item cannot appear as an operand of an INITIALIZE statement. Categories must be permissible.

5004 `Index data item illegal in an INITIALIZE statement - statement is omitted`

An index data item cannot appear as an operand of an INITIALIZE statement.

5005 `Data type of sender must be compatible with category in REPLACING phrase - statement is omitted`

Each category stated in the REPLACING phrase must be a permissible category as a receiving operand in a MOVE statement where the corresponding data item referenced by the identifier or the literal following DATA is used as the sending operand. All operations are performed as if a series of MOVE statements had been written.

5006 `Receiver must not have OCCURS DEPENDING ON phrase associated with it - statement is omitted`

The description of the data item following INITIALIZE or any items subordinate to it cannot contain the DEPENDING phrase of the OCCURS clause.

5007 `Same category cannot be repeated in a REPLACING phrase - statement is omitted`

The same REPLACING category cannot be repeated more than once in an INITIALIZE statement. The only exception to this rule is for ALPHANUMERIC or ALPHANUMERIC-EDITED when the USAGE DISP is specified on one case and USAGE DISP-2 is specified on the other case.

5009 `Condition name is not legal as receiver of an INITIALIZE statement - statement is omitted`

A condition name is not legal as a receiver of an INITIALIZE statement. If explicitly referenced, a condition name must be unique or be made unique.

5025 `Files must not be repeated within the MERGE statement`

A file name cannot occur more than once in a MERGE statement.

5026 `SORT/MERGE statement is illegal in a declaratives procedure - statement is omitted`

A SORT or MERGE statement can appear anywhere in the Procedure Division except in the declaratives portion.

5027 `This file is not described in a sort-merge description entry - statement is omitted`

The file name following SORT or MERGE statement must be described in a sort-merge file description entry in the Data Division.

5028 `This KEY is not associated with sort-merge file records - statement is omitted`

5029 `KEY cannot have variable occurrence data items - statement is omitted`

The data items identified by key data names, following KEY of SORT/MERGE, must not be group items which contain variable occurrence data items, such as an OCCURS clause.

5030 `KEY cannot have OCCURS clause or be subordinate to one - statement is omitted`

None of the data items identified by key data names, following KEY of SORT/MERGE, can be described by an entry that either contains an OCCURS clause or is subordinate to an entry that contains as OCCURS clause.

5031 `This file is not described in a file description entry - statement is omitted`

File names not following SORT or MERGE but contained in one of the statements must be described in a file description entry, not a sort-merge file description entry, in the Data Division.

5032 `Files on same SORT/MERGE cannot have the same SAME SORT-MERGE AREA clause - statement accepted`

No pair of file names in the same SORT statement can be specified in the same SAME SORT AREA or SAME SORT-MERGE AREA clause. No pair of file names in the same MERGE statement can be specified in the same SAME AREA, SAME SORT AREA, or SAME SORT-MERGE clause.

5033 `GIVING files cannot be specified in the SAME AREA clause - statement accepted`

In the SORT statement, file names associated with the GIVING phrase cannot be specified in the same SAME AREA clause.

5034 `For indexed files, major SORT/MERGE key must be associated with ASCENDING phrase - statement omitted`

If the file name in the GIVING clause of the SORT statement refers to an indexed file, the first specification of a data name must be associated with an ASCENDING phrase, and the data item referenced by that data name must occupy the same character positions in its record as the data item associated with the prime record key for that file.

5035 `Record name must be associated with a sort-merge file description - statement omitted`

The record name in the RELEASE statement must be the name of a logical record in a sort-merge file description entry and can be qualified.

5036 `'`*`character-string`*`' must not refer to the sort-merge file record area - statement omitted`

The identifier associated with the INTO clause of the RETURN statement or the FROM clause of the RELEASE statement cannot be in the record area of the sort-merge file.

5037 `COLLATING SEQUENCE must be an alphabet name - statement omitted`

The name following the COLLATING SEQUENCE clause of the SORT or MERGE statement must be an alphabet name.

5038 `Maximum number of keys cannot be more than 64 - statement omitted`

The maximum number of data names in the KEY clause of a SORT or MERGE statement is 64.

5039 `RANDOM ACCESS files are not legal for USING/GIVING files - statement omitted`

The ACCESS MODE IS RANDOM clause must not be used for file names specified in the USING or GIVING phrase of a SORT or MERGE statement.

5040 `Illegal identifier for INTO phrase in RETURN statement - statement omitted`

The storage area associated with the identifier following INTO and the record area associated with file name following RETURN must not be the same storage area. Only one record description can be subordinate to the sort-merge file description entry, and all record names associated with the file name and the data item referred to by the identifier must describe a group item or an elementary alphanumeric item.

5041 `Illegal file name for INTO phrase in RETURN statement - statement omitted`

The RETURN INTO name can only be a record name or a data name.

5042 `RETURN/RELEASE must be in the range of INPUT/OUTPUT procedures associated with a SORT/MERGE.`

A RETURN statement can be used only within the range of an OUTPUT procedure associated with a SORT or MERGE statement for the RETURN file name. A RELEASE statement can be used only within the range of an INPUT procedure associated with a SORT statement for the file name whose sort-merge file description contains the RELEASE record name.

5043 `Files on same MERGE cannot have the same SAME AREA clause - statement accepted`

No pair of file names in the same MERGE statement can be specified in the same SAME AREA, SAME SORT AREA, or SAME SORT-MERGE clause.

5044 `Only the GIVING files of the MERGE can have the same SAME RECORD AREA clause - statement accepted`

The only file names in a MERGE statement that can be specified in the same SAME RECORD AREA clause are those associated with the GIVING phrase.

5045 `DATE KEY declaration is incorrect.`

DATE KEY must be described as USAGE DISPLAY and at least 2 characters long. If DATE KEY is numeric, it must be unsigned.

5046 `FIRST DATE declaration is incorrect.`

FIRST DATE must be an integer or a literal.

5047 `FIRST DATE value is incorrect.`

FIRST DATE value must be between 00 and 99.

5049 `The record name 'character string' is not a 01 level item in WORKING, COMMON or FILE SELECTION.`

When XML generation is specified for a record name, that record must exist in the source code at a 01 level in WORKING, COMMON or FILE SECTION.

5050 `The record name 'character string' specified for XML generation does not exist in WORKING, COMMON or FILE SECTION`

When XML generation is specified for a record name, that record must exist in the source code in WORKING, COMMON or FILE SECTION.

5051 `JIS-16 (Kanji) code strides over the R-margin (between column 72 and 73) - character ignored`

A Kanji character was found in columns 72 and 73. This is illegal.

5053 `JIS-16 (Kanji) code cannot be specified here - character ignored`

A Kanji character cannot appear in anything other than a Japanese literal with a CC-prefix, a comment line, or a comment entry.

5055 `JIS-8 code appeared in CC-prefix Japanese literal - only JIS-16 (Kanji) code is allowed`

A ASCII/ISO character cannot appear in anything other than a Japanese literal with a CC-prefix.

5057 `Corresponding CN-literal string is not defined in JIMFILE`

The NCONVR conversion routine has detected an error. The error status bit 4 was on.

5058 `I/O error on JIMFILE`

The NCONVR conversion routine has detected an error. The error status bit 5 was on.

5059 `JIMFILE could not be assigned`

The NCONVR conversion routine has detected an error. The error status bit 6 was on.

5060 `Kanji string exceeds 40 characters length on CN conversion`

The NCONVR conversion routine has detected an error. The error status bit 11 was on.

5061 `Illegal character found in this CX-literal`

The NCONVR conversion routine has detected an error. The error status bit 12 was on.

5062 `Illegal character found in CE- or CJ-literal`

The NCONVR conversion routine has detected an error. The error status bit 13 was on.

5063 `Out of Kanji code range is specified in CE-, CJ- or CX-literal`

The NCONVR conversion routine has detected an error. The error status bit 14 was on.

5064 Illegal number of characters in CE-, CJ- or CX-literal

The NCONVR conversion routine has detected an error. The error status bit 15 was on.

5065 JIS reserved character is specified in CE-, CJ- or CX-literal

The NCONVR conversion routine has detected an error. The error status bit 16 was on.

5066 Undefined voiced or semivoiced sound is specified in CH- or CK-literal

The NCONVR conversion routine has detected an error. The error status bit 17 was on.

5067 Illegal character is specified in CA-, CH- or CK-literal

The NCONVR conversion routine has detected an error. The error status bit 18 was on.

5068 Length of this Japanese nonnumeric literal exceeds maximum length - truncated

The literal image exceeds the maximum length.

5069 CHARACTER TYPE clause is not allowed in this section - omitted

The CHARACTER TYPE clause was specified somewhere other than the File Section.

5070 CHARACTER TYPE clause is not allowed in level 66, 77 or 88 - omitted

The CHARACTER TYPE clause was illegally specified at a level 66, 77, or 88.

5071 CHARACTER TYPE clause is not allowed in record other than PRINTER and ANS74-PRINTER - omitted

A CHARACTER TYPE clause is specified in a record description that is not a PRINTER or ANS74-PRINTER.

5072 CHARACTER TYPE specification of this item and that of group item differ - CHARACTER TYPE specification of group item is used

A CHARACTER TYPE specification conflict has occurred between the current level and the parent level.

5073 CHARACTER TYPE clause can be specified in only USAGE DISPLAY-2 item - omitted

The CHARACTER TYPE clause is specified with a USAGE other than DISPLAY-2.

5074 VALUE literal specified is illegal with the USAGE clause for this item - VALUE clause ignored

A Japanese literal is specified with a VALUE clause that does not have a USAGE DISPLAY-2, or a numeric or nonnumeric literal is specified with a VALUE clause that does have a USAGE DISPLAY-2.

5075 CHARACTER TYPE clause is not allowed within a REDEFINES clause - CHARACTER TYPE clause ignored

The CHARACTER TYPE clause cannot be used in a data description entry that

contains a REDEFINES clause, or in an entry that is subordinate to an entry containing a REDEFINES clause.

5076 `DISPLAY-2 item on a print file has an implied CHARACTER TYPE which cannot be used in association with a REDEFINES clause`

The DISPLAY-2 item is either with a REDEFINES clause or subordinate to a REDEFINES clause, so the implied CHARACTER TYPE clause cannot be created.

5077 '*character-string*' is not USAGE DISP-2 or a group item, it cannot be compared to DISP-2 item - expression ignored

The comparison between a DISPLAY-2 item and an elementary that is not DISPLAY-2 is illegal.

5078 Illegal USAGE for *character-stringfile* - DISPLAY is assumed

A USAGE DISPLAY-2 was specified in an INTERCHANGE record description.

5081 `USAGE DISPLAY-2 cannot be specified for relative or indexed report file - DISPLAY is assumed`

A USAGE DISPLAY-2 is specified in a report group description assigned to a relative or indexed organization file. This is illegal.

5082 `The usage of the data item specified as the receiver of a MOVE statement is illegal with a DISPLAY-2 sender`

The data type of the identifier used as the receiver of the MOVE statement is illegal when the sender is of USAGE DISPLAY-2.

5083 `The usage of the data item specified as the sender of a MOVE statement is illegal with a DISPLAY-2 receiver`

The data type of the identifier used as the sender of the MOVE statement is illegal when the receiver is of USAGE DISPLAY-2.

5084 `Kanji character $ is used by compiler, user should not use Kanji $ - accepted`

Kanji character $ is used by user program. However the compiler generates internal names with Kanji character $. Therefore the Kanji $ must not be used in order to avoid a possible conflict with internal system names.

5085 '*character-string*' : a Japanese user-defined word exceeds 15 Kanji characters - it is truncated to 15

Japanese user-defined words must be less than 15 Kanji characters.

5086 `An internal file name must not specify a Japanese user-defined word if the external file name is omitted`

If an external file name that is in a SELECT clause is omitted, the internal file name must not specify a Japanese user-defined word, because OS1100 does not allow Kanji characters.

5087 `The external file name that is specified in a SELECT clause must not specify a Japanese user-defined word`

An external file name that is in a SELECT clause must not specify a Japanese user-defined word, because OS1100 does not allow Kanji characters.

5088 `The external file name defined (implicitly or explicitly) in the SELECT clause is not implemented in the Open Programming Environment`

In the Open Programming Environment (OPE), a SELECT clause for an OS2200 file must contain an external file name that is specified in "[q]*fn.[elt[/ver]]" format.

5089 `The device type in the SELECT clause is not implemented in the Open Programming Environment`

In the Open Programming Environment (OPE), a SELECT clause may not refer to device-type ANS74-PRINTER. Also, a SORT DESCRIPTION (SD) may not refer to an OPE file.

6000 '*character-string*' incorrectly repeated

RECORD, AREA and SETS or set identifiers can appear only once each in the STORE statement. AREA-NAME and AREA-KEY can appear only once each in the DMS MOVE statement.

6001 '*character-string*' is illegal in the sequence of clauses in Subschema Section – it is ignored

The specified clause is illegal in the Subschema Section.

6002 `Duplicate clause in Subschema Section – Subschema Section ignored`

The specified clause has already been used in the Subschema Section, and it is expected to appear only once.

6003 `This phrase is out of sequence, check the PRM for correct syntax`

The specified clause is out of order in the COBOL program. Check the *UCS COBOL Programming Reference Manual Volume 1* for correct syntax.

6004 `This DMS statement is illegal when INVOKE is missing or is in error`

All DMS statements are illegal when INVOKE is missing or is in error. The syntax of the statements is checked but semantic routines are skipped.

6005 `This type USAGE is illegal when INVOKE is missing or is in error – assume USAGE IS DISPLAY`

A USAGE clause of type AREA-NAME, RECORD-NAME, SET-NAME, ACQUIRE-LIST, AREA-KEY and DATABASE-KEY are illegal when INVOKE is missing or in error.

6006 `This identifier must be a subschema record name or a generalized record name`

Record name must be included in the subschema named by the INVOKE clause of the Subschema Section.

6007 `This identifier must be 77 level or 01 elementary item – statement is ignored`

Identifier must be a 77 level entry with a USAGE IS RECORD-NAME and initialized with the name of a record defined in the subschema named by the INVOKE clause of the Subschema Section.

6008 '*character-string*' cannot be in the declarative portion of the Procedure Division – clause is ignored

The error paragraph and end paragraph must be a paragraph name or section name

defined in the nondeclarative portion of the Procedure Division.

6009 `This identifier must be an area name in Subschema Section or identifier with usage of AREA-NAME - statement is ignored`

Area names used in this statement must be the name of an area defined in the subschema named by the INVOKE clause of the Subschema Section. Identifier used in this statement must have been defined with a USAGE AREA-NAME clause and initialized with the name of area defined in the subschema named by the INVOKE clause of Subschema Section.

6010 `Area names and identifiers cannot be mixed in a single CLOSE statement - statement is ignored`

Area names and identifiers cannot both be used on the same CLOSE statement.

6011 `Maximum number of area names that can be used in this statement is 47, maximum number is exceeded - statement is ignored`

More than 47 area names have been specified for this statement.

6012 `This identifier must be a set name in Subschema Section or identifier with usage of SET-NAME - statement is ignored`

The set name must be the name of a set defined in the subschema named by the INVOKE clause of the Subschema Section or it must have been defined with a USAGE SET-NAME clause and initialized with the name of a set defined in the subschema named by the INVOKE clause of Subschema Section.

6013 `Set names and identifiers cannot be mixed in a single INSERT/REMOVE statement - statement is ignored`

Set names and identifiers cannot both be used on the same INSERT/REMOVE statement.

6014 `Maximum number of set names that can be used in this statement is 36, maximum number is exceeded - statement is ignored`

No more than 36 set names can be specified on this single statement.

6015 `This identifier must have an owner record named in Subschema Section - statement is ignored`

The identifier must have an owner record defined in the subschema named by the INVOKE clause of the Subschema Section.

6016 `The owner record of this identifier must be of type record name - statement is ignored`

The owner record of the identifier used in this statement must be of type record name.

6017 `This DMS identifier usage must be DATABASE-KEY - statement is ignored`

The identifier used in this location of this DMS statement must have the usage of DATABASE-KEY.

6018 `The usage of this DMS identifier must AREA-KEY or AREA-NAME - statement is ignored`

The identifier used in this location of this DMS statement must have the usage

AREA-KEY or AREA-NAME.

6019 `Usage of this identifier must be BINARY - statement is ignored`

The usage of the DMS identifier must be BINARY. LOG statement is ignored.

6020 `FROM option is required in this statement when RECORD DELIVERY AREA is missing in Subschema Section - LOG statement is ignored`

If the RECORD DELIVERY AREA of the Subschema Section has been omitted, the FROM identifer-2 option is required.

6021 `This identifier must be defined in Common/ Working-Storage Section - statement is ignored`

This identifier must be defined as a COBOL 01 level entry in either the Working-Storage or Common-Storage Section.

6022 `Maximum number of set names that can be used in this statement is 63, maximum number is exceeded - statement is ignored`

No more than 63 set names can be specified on this single statement.

6023 `Set names and identifiers cannot be mixed in a single statement - statement is ignored`

It is illegal to mix identifiers and set names in a single STORE, FIND or FETCH statement.

6024 `This identifier must be a set name - statement is ignored`

Illegal type of identifier is used at this position in DMS ACQUIRE statement.

6028 `Incorrect identifier type for a DMS statement - statement is ignored`

The type of this identifier must be RECORD-NAME, SET-NAME, AREA-NAME or be a record name, set name, area name, or generalized name.

6029 `PICTURE string is illegal for this identifier when used in this statement`

The identifier definition is not correct. The identifier must be defined with a PICTURE clause of PIC 9(9) for FIND/FETCH and 9(10) for LOG.

6030 `OWNER option is not correct when area names are used in this statement - statement is ignored`

In format 3 of FIND/FETCH, the OWNER option cannot be used if an area name is specified.

6031 `Incorrect combination of set/area name is used in this statement - statement is ignored`

This identifier must be a set name or set name generalized name if next identifier is a set name. If next identifier is an area name, this identifier must be an area name generalized name.

6032 `This identifier must have been defined as data item within the record name used as the first identifier in this statement - statement is ignored`

In formats 6 and 7 of the FIND/FETCH statement, database identifiers must be

defined as data items within the record name.

6033 `AT END clause is only legal with the NEXT[DUPLICATE] and PRIOR option - statement is ignored`

AT END option must only be used when NEXT[DUPLICATE] or PRIOR options are used. This happens only in format 3, 4, 5 of the FIND/FETCH statement.

6034 `This identifier must be a set name in Subschema Section or identifier with usage of SET-NAME - statement is ignored`

The set name must be the name of a set defined in the subschema named by the INVOKE clause of the Subschema Section or it must be defined with a USAGE SET-NAME clause and initialized with the name of a set defined in the subschema named by the INVOKE clause of Subschema Section.

6035 `This identifier must be a subschema record name`

Record name must be included in the subschema named by the INVOKE clause of the Subschema Section.

6036 `When the usage of the previous identifier is DATABASE-KEY, this identifier is not allowed in format 1 of FIND/FETCH`

In format 1 of the FIND/FETCH, when the usage of the second identifier is DATABASE-KEY, the third identifier must not be used.

6038 `When ALL option is used in an OVERLAY clause, only one OVERLAY is permitted - Subschema Section ignored`

If the ALL option is used, then only the record delivery area can be specified as the overlaid record and only one OVERLAY clause is permitted.

6039 `This record name or record delivery area is undefined - statement is ignored`

The record delivery area name or record name used in an OVERLAY clause is not defined.

6040 `With the ALL option, the overlaid record must be the record delivery area - statement is ignored`

If the ALL option is used, then only the record delivery area can be specified as the overlaid record.

6041 `The record delivery area and overlaying records must be defined in the same section (WORKING or COMMON) - statement is ignored`

If the record delivery area is to be overlaid, both it and the overlaying records must all be allocated to either Working-Storage or Common-Storage; but not be mixed.

6042 `A record name cannot appear in more than one OVERLAY clause - statement is ignored`

A record name must not appear in more than one OVERLAY clause or be overlaid by redefinition.

6043 `Overlaying item must not be used in more than one OVERLAY clause - statement is ignored`

An overlaying item must not appear in more than one OVERLAY clause or be overlaid by redefinition.

6044 `Overlaid record must be a record name or the record delivery area - statement is ignored`

An illegal type of identifier is used for overlaid record. It must have a usage of record name or be the record delivery area.

6045 `The USING item must be a record name or a data item within the record name - statement is ignored`

The USING identifier must be a record name or a data item defined within the record name used as the first identifier in this statement.

6046 `DMCA declaration is missing or is defined in a non-defined section - IMPART statement is ignored`

The DMCA in the INVOKE clause is defined in a section, either WORKING-STORAGE or COMMON-STORAGE, which does not exist or DMCA is not defined.

6047 `When AREA-KEY and AREA-NAME are both present, two identifiers must be specified as the receiving items, the second identifier is missing in this case - MOVE is ignored`

In a DMS type MOVE, when both AREA-KEY and AREA-NAME are present, the user must specify two items as the receiving items. At this time, only one is specified.

6048 `This identifier must be an area or set name in Subschema Section or identifier with usage of SET-NAME or AREA-NAME.`

Formats 3 and 4 of the FIND/FETCH statement require that the identifier associated with the AREA or SET clause be a set name, area name or defined with a USAGE SET-NAME or AREA-NAME clause.

6049 `This identifier must be an area name in Subschema Section or identifier with usage of AREA-NAME`

If the identifier is an area name, it must be the name of an area defined in the subschema named by the INVOKE clause of the Subschema Section. If the identifier is not an area name, it must be defined with a USAGE AREA-NAME clause and initialized with the name of area defined in the subschema named by the INVOKE clause of Subschema Section.

6050 `IMPART is illegal when the LINKAGE option is used in the INVOKE clause - IMPART is ignored`

If the LINKAGE option is specified in any one of the COPYING, RECORD, or DMCA clauses of the INVOKE clause, an IMPART statement cannot appear in the compilation.

6051 `Paragraph names of ERROR and ROLLBACK clauses cannot be defined in declaratives portion of Procedure Division`

The general ERROR paragraph and ROLLBACK paragraph must be defined in the nondeclaratives portion of the Procedure Division.

6052 `Maximum value of the integer allowed in this statement is exceeded`

The integer value in the DMCA clause must be a positive integer less than or equal to 50.

6053 `Overlaid record must be a record name`

Illegal type of identifier is used for overlaid record. It must have a usage of record name.

6054 `DMS data items not copied into WORKING-STORAGE, COMMON-STORAGE, or Linkage Section as specified in INVOKE clause`

The DMCA, RDA, record names, or data names were not copied into the Data Division as specified in the INVOKE clause.

6055 '*character-string*' illegal in this INVOKE clause

The INVOKE clause cannot appear more than once in a COBOL program. The KEY and COPYING clauses are part of the INVOKE clause and must immediately follow the INVOKE clause if they are present.

6056 `Owner of the database identifier must be a DMS record`

The owner of the database identifier used in a MODIFY statement must be a DMS record.

6057 `SET or AREA name + '`*character-string*`' which is defined in the subschema is not unique, it duplicates an existing name from the COBOL program - name is accepted`

The SET name or AREA named defined in the subschema is not unique. All names within a COBOL program must be able to be uniquely referenced. An ambiguity can result if this name is used later on in the program.

6058 `INVOKE may only occur in first program of back to back or nested programs`

The CDML INVOKE clause may only appear in the first program of a series of separately compiled programs in a single source element (back to back programs). Also, the INVOKE may occur only in the outer most program of nested programs. This INVOKE clause will be ignored and will cause all CDML commands in the source code to be ignored or improperly processed.

6059 `USAGE COMP-1 and COMP-2 items in the schema have been allocated in COBOL '74 format; the COMPAT option must be specified for this compilation.`

The COBOL source input created by DDL is in COBOL '85 format. However, the corresponding COMP-1, COMP-2 items in the schema have been allocated on COBOL '74 (ASCII COBOL) boundaries. The data will be processed incorrectly unless the COMPAT option is specified.

6060 `Schema created with COBOL 85 syntax and UCOB data allocation; COMPAT option or COMP-BIN option must not be specified for this compilation.`

The COBOL source input and data allocation created by DDL is in COBOL '85 format. In order for the compiler to handle the syntax and data allocation the same as DDL, the COMPAT or COMP-BIN option must not be used. Remove the COMPAT or

COMP-BIN option and retry the compilation.

6501 `Bad status returned on IOW$`

A nonzero IOW status was returned on a subschema table read from the absolute subschema tables.

6505 `DMS record attribute entry expected - not found`

The records specified in the absolute subschema do not match the records specified in the S$PROC element.

6508 `The subschema element was not processed with the HOST LANGUAGE UCS COBOL clause - clause is ignored`

The subschema element that is processed by SDDL must contain the HOST LANGUAGE IS UCS COBOL clause. The HOST LANGUAGE clause specified another language besides UCS COBOL. The INVOKE clause will be ignored.

6509 `SELECT and FD clauses must both be invoked from Repository- clause is accepted`

When using the Repository, both SELECT and FD clauses must be invoked.

6510 `Incorrect INVOKE syntax for Repository - clause is ignored`

A syntax error has been detected is INVOKED statement. Check the PRM for correct syntax.

7000 '*character-string*' is an unknown or not implemented function name - clause is ignored

The function name used in the function call is unknown or not implemented.

7001 `Incorrect number of arguments is used for this function - clause is ignored`

The number of arguments passed to the function is not what the function expects. Check the PRM for the correct syntax.

7002 `An integer argument is expected here - clause is ignored`

The arguments passed to the function must be an integer value. Error value of -2 is returned.

7003 `The arguments of the function must have the same type - clause is ignored`

The types of the arguments passed to the function must have the same type.

7004 `Illegal argument for specified FUNCTION - clause is ignored`

An illegal argument type is used as the argument to a function.

7005 `A function-identifier is not allowed at this point in the statement`

A function-identifier was used incorrectly in the current statement.

7006 `Maximum dimension allowed for arguments in this function is 7 - clause is ignored`

A maximum of seven nested levels allowed for processing by intrinsic functions.

7007 `Reference modification not allowed on arithmetic functions - statement ignored`

Reference modification is not allowed on functions that have an arithmetic result.

7008 '*character-string*' can be used only as a subscript in a function argument - it is replaced by a 1.

Reserved word 'ALL' can be used only as a subscript within the range of function argument. It is replaced by a subscript of 1.

7009 `Function argument considered as total array - subscript is ignored`

Due to presence of reserved word 'ALL' as the argument of a function, the argument is considered as total array.

7010 `RSA SQL STANDARD WARNING FOLLOWS:`

RSA parsing has detected an SQL standard violation. This is a warning header. The RSA standard warning messages follow.

7011 `RSA ERROR FOLLOWS:`

RSA parsing has detected illegal syntax. The statement is ignored. This is a major error header. The RSA messages will follow.

7012 `RSA FATAL ERROR FOLLOWS:`

RSA parsing has detected a fatal situation. The compilation is terminated. This is a fatal error header. The RSA messages will follow.

7014 '*character-string*' is duplicate or non-existent cursor-name; statement ignored

An embedded SQL command, 'DECLARE cursor-name' has specified an already specified cursorname OR an 'OPEN cursor-name' statement has referenced a nonexistent cursorname. The statement is ignored in either case.

7015 `Host program variable in an embedded SQL statement is not declared within the SQL DECLARE SECTION; statement accepted.`

The SQL standard specifies that COBOL program variables specified within an SQL statement must be declared after the embedded SQL directive BEGIN DECLARE SECTION and before the embedded SQL directive END DECLARE SECTION; the statement is accepted.

7016 `SQLCODE is not declared in the host program or not declared properly; statement accepted.`

The SQL standard specifies that SQLCODE must be declared in a COBOL program containing embedded SQL statements and that it must be declared as S9(nn) USAGE COMPUTATIONAL, where nn is between 4 and 18 inclusive.

7017 `Host-variable AUX-INFO implicitly used in an SQL statement; statement is accepted`

Implicit host-variable AUX-INFO is used on an embedded SQL statement if it is defined properly and is declared after the embedded SQL directive BEGIN DECLARE SECTION and before the embedded SQL directive END DECLARE SECTION; this is an extension

to the SQL standard, however.

7018 `Host-variable ERROR-STATUS implicitly used in an SQL statement; statement is accepted`

Implicit host-variable ERROR-STATUS is used on an embedded SQL statement if it is defined properly and is declared after the embedded SQL directive BEGIN DECLARE SECTION and before the embedded SQL directive END DECLARE SECTION; this is an extension to the SQL standard, however.

7019 `This embedded SQL command not formed in accordance with SQL standard; statement is accepted`

The SQL standard specifies that embedded SQL commands BEGIN DECLARE SECTION and END DECLARE SECTION must be completely specified on one source input line. The SQL standard also specifies that the beginning delimiters EXEC and SQL must be completely specified on one source input line.

7020 `END DECLARE SECTION not preceded by BEGIN DECLARE SECTION; statement is ignored`

The END DECLARE SECTION which marks the end of the standard-specified range of allowable host-variables for PROCEDURE DIVISION embedded SQL statements has no corresponding preceding BEGIN DECLARE SECTION statement, so the statement has no effect.

7021 `BEGIN DECLARE SECTION follows another BEGIN DECLARE SECTION; statement is ignored`

The BEGIN DECLARE SECTION which marks the beginning of the standard-specified range of allowable host-variables for PROCEDURE DIVISION embedded SQL statements follows another BEGIN DECLARE SECTION statement, with no intervening END DECLARE SECTION statement, so the statement has no effect.

7022 `Indicator variable not signed fixed integer; indicator is ignored`

A user variable specified as an indicator variable in an embedded SQL statement must be a signed numeric item with no scale and USAGE of COMP, DISP or PACKED-DECIMAL. The indicator specification is ignored.

7023 `One or more COBOL host variables or indicator variables within an embedded SQL multi-record statement not declared properly OR an unsubscripted array reference used in another embedded SQL statement; statement is ignored`

A user array specified as a host variable or indicator variable in a multi-record SQL statement, such as FETCH NEXT N, must be a one-dimensional array with no subscripts attached, the maximum occurrence value of the array must be greater than or equal the value of N, and the resultant operand must not be edited. Conversely, an array reference without a subscript is not allowed as a host variable or indicator variable in other embedded SQL statements. In either case, the statement is ignored.

7024 `Invoked entity could not be found in the Repository.`

The Repository does not contain an entity with this name. An incorrect name is used or such an entity was never created in the Repository.

7025 `Repository processing problems. UREP returned the following status:`*`integer-value`*

UCS LSS Codegen UREP processing encountered a problem. The problem is elaborated by the status returned from UREP and included in this message.

7026 `RSA SQL WARNING MESSAGE FOLLOWS:`

RSA parsing has detected an SQL usage problem. This is a warning header. The RSA warning messages follow.

7027 `Host variable is index name, an index data item, USAGE BYTE-I18N or a condition-name; the host variable is ignored`

A host variable specified in an embedded SQL statement must not be an index name, an index data item, USAGE BYTE-I18N or a condition-name. The host variable specification is ignored.

7028 `SQLCODE is not specified as a numeric integer of at least size 5 OR SQLSTATE is not specified as PIC X(5) DISPLAY at an 01 or 77 level; the variable will not be implicitly used by embedded-SQL statements.`

The SQL standard specifies that SQLCODE must be present in an embedded-SQL program and must be declared as PIC S9(9). The SQL 92 standard specifies that SQLSTATE must be present in an embedded-SQL program and must be declared as PIC X(5) DISPLAY. SQLSTATE or SQLCODE will be considered to be a legal user-defined variable, but will not be used as an implicit parameter in any embedded-SQL processing.

7029 `USAGE SQL specified outside SQL DECLARE SECTION or not specified with level number 01 or 77; USAGE IGNORED.`

USAGE SQL may be specified only for a level 01 or 77 item and must be declared between EXEC SQL BEGIN DECLARE SECTION and EXEC SQL END DECLARE SECTION directive.

7030 `Dataname specified for an item with USAGE SQL TYPE BLOB exceeds the maximum allowable characters. USAGE SQL ignored.`

An item declared with USAGE SQL TYPE BLOB causes the compiler to created a group item with the dataname specified and elementary items with suffixes defined by the SQL3 standard. The maximum suffix length is 9 characters for a simple BLOB, so the maximum size is 21 characters. The maximum suffix length is 12 for BLOB AS FILE specification, so the maximum size for the BLOB name is 18 characters.

7031 `Length specified for an item with USAGE SQL TYPE BLOB exceeds maximum allowable size, or is incorrectly specified. Size 1 will be used.`

The maximum allowable size specifications for a USAGE SQL TYPE BLOB item are:
67,108,852
65535K
63M

7032 BLOB data type used as host variable; the RDMS level for this application does not support the BLOB data type. The statement is ignored.

The RDMS level installed for this application does not support the BLOB data type. To resolve this problem, install the RDMS level that does support the BLOB data type.

9999 Bad status returned from ELMS, ERROR-STATUS: *octal-value* SUBSTATUS: *octal-value*

The Extended Language Message System (ELMS) was called by the compiler to issue a message. ELMS returned an error status indicating it was unable to complete the request. Refer to the *ELMS Programming Guide* for a description of the status values. Note that message 9999 is not issued by ELMS. It is constructed and issued internally by the UCS COBOL message routine. For this reason, this message is not translatable.

A.4. UCS Language Support System Messages (LSS)

10004 The error limit of *integer-value*, specified on the compiler keyword option MAXERRORS, has been exceeded

The keyword option MAXERRORS/n was specified on the compile line and the number of major errors detected has exceeded that limit.

10005 ELMS error status while issuing message number *integer-value*, ERR-STATUS: *octal-value* SUBSTATUS: *octal-value*

The Extended Language Message System (ELMS) was called by the compiler to issue a message. ELMS returned an error status indicating it was unable to complete the request. Refer to the ELMS Application Writers Guide document for a description of the status values. Note that message 10005 is not issued by ELMS. It is constructed and issued internally by the common back-end message routine. For this reason, this message is not translatable.

10121 Request of '*integer-value*' bits of static storage exceeds the static initialization limit

10135 Automatic storage exceeded in procedure *character-string*

The named procedure requires more words on the activity local stack than is permitted. Reduce the amount of storage required by declaring fewer variables inside the procedure, eliminating inline expansion, or creating two smaller procedures.

10141 No RUN-UNIT clause; Link to UREP not accomplished

The INVOKE statement did not specify a RUN-UNIT-ID clause; therefore, no link to UREP was made. A RUN-UNIT-ID clause is necessary to link to UREP.

10142 *** UREP not available ***

UREP FIND request was made on the RUN-UNIT entity type and a NO-FIND status was returned. This indicates that the UCS compiler is executing on a system that does not have UREP installed.

10144 UREP returned – *character-string*

UREP has returned a major error diagnostic. The UCS compiler is not directly aware of what the error was or the diagnostic for it. The UCS message just conveys the severity of the error and the message text returned by UREP. For more specific information on the message and its possible causes, see the relevant documentation for UREP, such as the *Repository for ClearPath OS 2200 Administration and Programming Reference Manual*.

10146 UREP returned – *character-string*

UREP has returned a minor error diagnostic. The UCS compiler is not directly aware of what the error was or the diagnostic for it. The UCS message just conveys the severity of the error and the message text returned by UREP for the error. For more specific information on the message and its possible causes, see the relevant documentation for UREP, such as the *Repository for ClearPath OS 2200 Administration and Programming Reference Manual*.

10147 Name *character-string* was truncated to 14 characters within UREP-XREF report

The UREP limit on a kanji name is 14 characters. The UCS compiler has detected a Kanji name of 15 characters within UREP-XREF processing. The kanji name passed to UREP for reporting purposes is truncated to 14 characters. This could be a duplicate of another 14 character name.

10148 Statement is too complex, it must be simplified

Because of recursive processing of expressions by the compiler, the compiler has run out of internal stack space. The statement must be simplified in order for it to compile error-free. You can simplify the statement by breaking it into more than one statement or inserting parentheses into the statement's expression.

10151 Illegal storage size for data type '*character-string*'

10152 Precision for INTEGER exceeds implementation limit

10153 Precision for REAL exceeds implementation limit

10154 Exponent for REAL is too large

10155 Exponent for REAL is too small

10156 RANGE BOUND of INTEGER needs to be an INTEGER constant

10157 Upper bound less than lower bound is illegal

10158 RANGE BOUND of type REAL needs to be a REAL constant

10159 Illegal value type – an immediate constant or a constant attribute entry expected

10160 An error was encountered while attempting to retrieve an integer number from '*character-string*'

10166 A UDS thread could not be established for processing SQL related actions. UDS Thread Control returned the following status: *character-string*

UCS LSS could not establish a thread for SQL processing. User intervention may be required to correct the problem.

10167 The Extended-Mode Access Module (EAM) file could not be created. Further processing can result in other SQL processing errors

UCS LSS could not acquire the EAM file required by RDMS to process SQL statements. User intervention may be required to correct the problem. Other error messages can be issued, since UCS LSS attempts to continue processing SQL statements.

10168 The size of entity '*character-string*' has not yet been determined. A size of 0 is assumed

The size of one variable cannot be used to declare the size of another variable in the same structure or overlay group when using the PLUS BITSIZE, BYTESIZE, or WORDSIZE built-in functions. In the following example, the size of item X is used to declare item Y. However, X and Y cannot be declared in the same structure or overlay group. EX: DECLARE X : 10 WORD LOGICAL ! Y(WORDSIZE(X)) : LOGICAL; <#0106>-- Illegal declaration

10502 Given storage length for '*character-string*' smaller than calculated length - (calculated length used)

The storage for the given data items was specified by the source program, but it is too small. The calculated storage length is used instead.

10504 Overlay size for '*character-string*' not identical to base's - (execution errors could result)

The specified data item is required to have the same storage as the data item (its overlay base) that it overlays, but it does not. Execution errors can result from the data item being smaller than its base, because the program may rely on the data item setting fields in its overlay base and they may not get set.

10505 Overlay '*character-string*' too large for base '*character-string*' - (execution errors could result)

The specified data item is required to have the same storage as the data item (its overlay base) that it overlays, but it does not. Execution errors can result from the data item being larger than its base, because the storage for any data following the overlay base (and belonging to another variable or containing compiler-dependent information) can be corrupted.

The storage required for the data item may be larger than what is needed for its base for various reasons. Some possibilities include:

- The start bit of the data item may not be the same as its base.
- If the data item is a structure, padding bits may have been added between elements of the structure.

10514 Calculated alignment disagrees with specified alignment for '*character-string*' - (calculated alignment used)

The alignment specified for the given data item is not adequate. This can be due to one of the following factors:

- The data item's data type or other data attributes require different alignment.
- The parent of a structure is marked as bit-aligned, for example, but it has a member that requires word alignment.

The calculated alignment replaces the given alignment.

10516 `Specified length of '`*`integer-value`*`' bits too small for '`*`character-string`*`' - set to '`*`integer-value`*`'`

The specified length for the given data item is too small. It is reset to the required value. The length has different meanings depending on the data type for the given data item. For arithmetic types such as integer and real, the length is the precision. For strings, it is the number of characters or bits, depending on the string type.

This error is most likely to occur for languages that allow either of the following:

- The user to specify the storage length of a data item
- The parent of a structure to have a specified data type or storage length independent of the structure members

10558 `Length greater than limit of '`*`integer-value`*`' bits for '`*`character-string`*`'`

The given data item exceeds the given storage length limit (the limit is in bits). The limit is usually the maximum length allowed for the given target machine, which is usually the maximum space allowed within a bank. Programs that do not exceed this limit may still not link or execute because of other storage or addressing limits that cannot be detected by the compiler.

10565 `Block size for file '`*`character-string`*`' is smaller than its record size`

Blocks contain records, the record size at most should be equal to a block size when a block contains just one record. The block size for the specific file is less than a record size.

10570 `Storage actually needed for structure '`*`character-string`*`' was '`*`integer-value`*`' bits - '`*`integer-value`*`' bits used`

The storage specified for the given structured data item is larger than what is actually required. Since this can be what the user wants, this message is only a remark. The storage for the structure remains what the user specified; storage lengths are given in bits.

10579 `Length for '`*`character-string`*`' is greater than the static data limit of '`*`integer-value`*`' bits`

The given static-storage data item is initialized, but its storage exceeds the system-imposed limit on static initialization. The user must either

- Reposition the data item such that it has a lower starting address (this is not always possible for a given language).
- Decrease the size of the data item.

- Initialize the data item with assignment statements, FOR or DO loops, and so forth.

10606 Initialization value for '*character-string*' has most significant (leading) digits truncated

The value used to initialize the given data item is too large. The leading (most significant) digits of the value have been truncated from the value.

10609 Initialization value for '*character-string*' is negative and the target is unsigned

The given data item is unsigned, but its initialization value is signed. The value may not be what the user expects.

10611 Initialization value's exponent is out of range for '*character-string*'

The initialization value has an exponent that is outside the range of values that the data item can contain.

10621 Initialization value for '*character-string*' has leading zeros truncated

The initialization value of the given data item had leading zeros truncated. Because this causes no change in the value, only a remark is issued.

10654 Magnitude of source or related scale factors too big in initialization process for '*character-string*' at 0*octal-value*

10655 Magnitude of source and related scale factors too small in initialization process for '*character-string*' at 0*octal-value*

10656 Unexpected small divisor in initialization process for '*character-string*' at 0*octal-value*

10657 The variable '*character-string*' was referenced but had not been assigned a value

The given variable was referenced in the source program without being initialized or assigned a value before the reference. This is an information-only message for the user; the compilation of the program is unaffected.

10675 The size of overlay item '*character-string*' is not the same as the size of its base '*character-string*'

10676 The records of file '*character-string*' have different sizes

10677 Variable '*character-string*' is declared but is never referenced

The variable has been declared, but neither it nor any overlay is used. Check the program for misspelling and consider eliminating the unnecessary variable.

10678 Length for '*character-string*' exceeds the limit of '*integer-value*' bits

10679 Overlay (EQUIVALENCE) group containing '*character-string*' exceeds the maximum segmented bank length of '*integer-value*' bits

The overlay group exceeds the given storage length limit (the limit is in bits) for a segmented data bank. (See the SEGDATA option or the UCS FORTRAN SEGDATA

compiler statement directive for more information.) For UCS FORTRAN, an overlay group is created by one or more EQUIVALENCE statements. This message is issued when the configuration of the members of the overlay group causes the total storage requirement for the group to exceed the maximum length allowed for a segmented data bank.

10680 `Overlay (EQUIVALENCE) group containing '`*`character-string`*`' exceeds the maximum data length of '`*`integer-value`*`' bits`

The overlay group exceeds the given storage length limit (the limit is in bits) for any data item. This message is issued when the configuration of the members of the overlay group causes the total storage requirement for the group to exceed the maximum length allowed for any data item. More restrictive limits can cause additional errors to be raised. This message is issued when the absolute length limit is exceeded.

10682 `The size of '`*`character-string`*`' is not one word. ASSIGNED storage data must be exactly one word in size`

The ASSIGNED storage data in PLUS must be the same size as a register, which is one word.

10683 `REDEFINES subject and object do not begin/end on the same type of boundary`

You tried to redefine an elementary exact binary item by an item which is not exact binary. However, the redefined exact binary item must begin and end on the same type of boundary as the redefining group.

10707 `Record size of '`*`integer-value`*`' bits greater than maximum record of '`*`integer-value`*`' bits for file '`*`character-string`*`'`

A record for the given file is greater than the specified maximum record size. The record size used is the calculated record size. Because this is larger than the maximum record size specified for the file, there can be erroneous results when the record is used.

10708 `Minimum record size of '`*`integer-value`*`' bits smaller than specified minimum record size of '`*`integer-value`*`' bits for file '`*`character-string`*`'`

A record for the given file has a record size smaller than the minimum record size specified for the file. This message is only a remark. Records can be smaller than the specified minimum.

10721 `The variable '`*`character-string`*`' in an EQUIVALENCE statement cannot be allocated on its required (word) boundary due to conflicts with the allocation of other EQUIVALENCE variables`

The variable is required to be word-aligned, but it is equivalenced to another word-aligned variable and to a variable that is less than word-aligned. Equivalencing with respect to a variable that is not word-aligned is such that only one of the word-aligned variables can actually be word-aligned. The alignment can be changed to an appropriate alignment that is less than word-aligned and allows incorrect, but executable, code to be generated for some code generation modes (such as extended mode), if the NO-ERRCHECK option is specified.

10726 `The RECORD KEYS '`*`character-string`*`' and '`*`character-string`*`' have the same start position within a record`

The leftmost character position of a RECORD KEY has the same start position as the leftmost character position of another RECORD KEY associated with a particular record in a file.

10727 `The RECORD KEY '`*`character-string`*`' extends beyond the minimum record size specified for the file`

The RECORD KEY is not contained within the first x character positions of the record, where x equals the minimum record size specified for the file.

10728 `The ALTERNATE RECORD KEY '`*`character-string`*`' extends beyond the minimum record size specified for the file`

The ALTERNATE RECORD KEY is not contained within the first x character positions of the record, where x equals the minimum record size specified for the file.

10730 `The variable '`*`character-string`*`' overlaps a segmented bank boundary. May cause unpredictable results if accessed`

For segmented data, no element is allowed to start at one bank, cross a 262,144 boundary, and end at the next bank. Any access to this item during program execution may cause unpredictable results.

10731 `The starting address of variable '`*`character-string`*`' is beyond the segmented bank starting address limit`

For segmented data, no element can start at any address beyond 16,777,215 (2**24 -1).

10732 `The extent of the character variable '`*`character-string`*`' exceeds the segmented bank address limit`

The extent of a segmented character data item cannot exceed 16,777,215 (2**24 - 1), which is the segmented bank address limit.

10733 `The record size of the file '`*`character-string`*`' exceeds the limit. The record size is truncated and reset to '`*`integer-value`*`' bits`

The limit of a record size differs from one file format to another. If the record size specified by a user exceeds the limit, the record size is truncated and is reset to the limit.

10734 `The block size in the file '`*`character-string`*`' exceeds the limit. The block size is truncated and is reset to '`*`integer-value`*`' bits`

The limit of a block size differs from one file format to another. If the block size specified by a user exceeds the limit, the block size is truncated and is reset to the limit.

10735 `The size of overlay item '`*`character-string`*`' is larger than the size of its base '`*`character-string`*`' - the larger size is used`

The size of a data item has exceeded its base. The storage is allocated for the overlay item with the largest size.

10736 `Padding bits added in front of '`*`character-string`*`' for proper alignment within a mapped structure`

Padding bits are added to a data item within a mapped structure.

10737 The lower bound value is less than the lower bound limit '*integer-value*' for '*character-string*'

10738 The upper bound value is greater than the upper bound limit '*integer-value*' for '*character-string*'

10739 An error was encountered while attempting to retrieve an integer number from '*character-string*'

10740 The storage length must be greater than 1 bit in size for the signed integer '*character-string*'

10741 The record '*character-string*' has no record selector

A file which contains multiple data records and which requires record selectors in each record, contains a record with missing record selector.

10742 Padding bits added between elements of '*character-string*' for proper alignment within mapped storage

Padding bits are added between elements of a dimensioned data item within mapped storage.

10801 Number of variables to be initialized exceeds the number of initializing constants

There are not enough values specified to initialize the number of variables specified. Some of the variables, therefore, are not initialized. It is likely the user has miscounted the number of variables or initialization values and will, therefore, not receive the initialization expected.

10802 Variable '*character-string*' is initialized but is never referenced

The given variable is initialized but it is never referenced. Therefore, the initialization is probably unnecessary.

10803 Number of initializing constants exceeds the number of variables to be initialized

There are more initialization values than there are variables to be initialized. It is likely the user has miscounted the number of variables or initialization values and will, therefore, not receive the initialization expected.

10804 Type of constant does not match type of variable '*character-string*' - NO INITIALIZATION DONE

The data type or other attribute of the constant that is to be used to initialize the given variable is not compatible with the variable's data type or other attribute. Initialization of the variable is not performed because the constant cannot be converted to the proper type or form or the rules of the source language forbid it.

10805 Type of constant does not match type of variable '*character-string*'

The data type or other attribute of the constant that is to be used to initialize the given variable is not compatible with the variable's data type or other attribute. Initialization of the variable is performed because the constant can be converted to a type or form that allows initialization. The initialization may not, however, be what the user expects.

10806 A subscript of variable '*character-string*' exceeded its maximum bound

In handling the initialization for the given variable, one of the given subscripts exceeded the maximum specified for the associated dimension. The initialization is probably incomplete or incorrect.

10807 A subscript of variable '*character-string*' exceeded its minimum bound

In handling the initialization for the given variable, one of the given subscripts is less than the minimum specified for the associated dimension. The initialization is probably incomplete or incorrect.

10808 Rightmost part of the initialization value for '*character-string*' has been truncated

This message applies only to string items. The initialization value for the item is too large. The rightmost part of the initializing constant has been truncated.

10809 Initialization of '*character-string*' to pointer value crosses a word boundary

UCS does not currently allow initialization of pointers that cross word boundaries.

10811 Initialization value for '*character-string*' has most significant (leading) bits truncated

The value used to initialize the given data item is too large. The leading (most significant) bits of the value have been truncated from the value.

10901 The FILE STATUS data item '*character-string*' must be two characters in length. This can cause an error at execution time

The FILE STATUS variable is not two characters in length. An execution time error can occur if the FILE STATUS data item is used during the execution of the program.

10906 The records of file 'filename' have different sizes. RECORDING MODE F ignored. RECORDING MODE V assumed.

11105 Constant expression evaluation produces an 'OUT OF RANGE' error

11116 Key's position does not correspond to the position of a record key associated with the file

The leftmost character position of a data item of category alphanumeric within a record of the file does not correspond to the leftmost character position of a record key associated with file-1, where file-1 is a sequential or dynamic file used in the START statement.

11120 Key may not fit in sort file record

If the file contains variable-length records, all the data items identified by key data names must be contained within the first x character positions of the record, where x is the MINIMUM record size specified for the file. If the file contains only fixed-length records, then x is the MAXIMUM record size.

11121 Record size for USING file '*character-string*' can cause record truncation

The record size of a USING file in a SORT statement is such that, due to the other sort parameters or files, it might cause incorrect results. Its records are larger than the sort files, and truncation can occur.

11122 Record size for GIVING file '*character-string*' can cause record

`truncation`

The record size of a GIVING file in a SORT statement is such that, due to the other sort parameters or files, it might cause incorrect results. Its records are smaller than the sort files, and truncation can occur.

11126 `Significant digits truncated during constant conversion`

The conversions of a constant during the TEX phase of the code generation require that significant digits (which are usually, but not always, zeros) are truncated. The constant is larger than the precision of the variables to whose common data type or precision it had to be converted. Incorrect results can occur when the compiled program or procedure is executed.

11127 `Sign stripped during constant conversion`

A signed constant is assigned to an unsigned variable or involved in operations requiring no sign. The constant is converted to an unsigned constant. If the constant was positively signed, this is not usually a problem. However, if the constant is negatively signed, erroneous results can occur. This message is issued frequently for some COBOL programs and is a usually a normal operation for that language.

11137 `Record size for USING file '`*`character-string`*`' is less than the smallest record in the SORT file '`*`character-string`*`'`

The record size of a USING file record in a SORT statement that specifies a SORT file with variable records is less than the smallest record in the SORT file. This can cause incorrect results.

11138 `Record size for SORT file '`*`character-string`*`' is less than the smallest record in the GIVING file '`*`character-string`*`'`

The record size of a SORT file record in a SORT statement that specifies a GIVING phrase and a file with variable records is less than the smallest record in the GIVING file. This can cause incorrect results.

11139 `Key '`*`character-string`*`' associated with an ASCENDING phrase does not occupy the same character position as the prime record key for file '`*`character-string`*`'`

The GIVING file is an indexed file, and the first key specification must be associated with the ASCENDING phrase and occupy the same character position as the prime record key for the GIVING file.

11145 `Previous or subsequent call of '`*`character-string`*`' passed an assumed-noncontiguous array value, so the ASHAPE processor call option is required`

An assumed-noncontiguous array value can be passed only to an assumed-shape array. Therefore, since there is a call that passes an assumed-noncontiguous array value, the dummy array must be assumed-shape. To pass a constant or adjustable array to an assumed-shape array, the ASHAPE compiler call option is required. Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for a discussion of allowed associations between actual arguments and dummy argument arrays.

11146 An assumed-size array cannot be passed to '*character-string*' since a previous or subsequent call passed an assumed-noncontiguous array value

An assumed-noncontiguous array value can be passed only to an assumed-shape array. Therefore, since there is a call that passes an assumed-noncontiguous array value, the dummy array must be assumed-shape. Passing an assumed-size array to an assumed-shape array is illegal. Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for a discussion of allowed associations between actual arguments and dummy argument arrays.

11147 A scalar value cannot be passed to '*character-string*' since a previous or subsequent call passed an assumed-noncontiguous array value

An assumed-noncontiguous array value can be passed only to an assumed-shape array. Therefore, since there is a call that passes an assumed-noncontiguous array value, the dummy array must be assumed-shape. Passing an scalar value to an assumed-shape array is illegal. Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for a discussion of allowed associations between actual arguments and dummy argument arrays.

11148 Locatable character array exceeds the segmented size limit

This message is issued for one of the following reasons:

An ALLOCATE statement is allocating a character array that exceeds the 16 megaword (2**24 = 16,777,216) limit.

A character array expression causes the compiler to allocate a segmented character array that exceeds the 16 megaword (2**24 = 16,777,216) limit.

11149 The length of '*character-string*' is not equal to the length of '*character-string*'. Structures must be of equal length

This message is issued for STRUCTURE data items in the following cases: (*) A source and a target in an assignment statement do not have equal lengths. (*) Two data items in an exchange statement do not have equal lengths.

11150 The length of the actual parameter '*character-string*' is not equal to that of the formal parameter '*character-string*' . Structures must be of equal length

The length of an actual parameter must be equal to the formal parameter length when both actual and formal parameters are structures.

11151 The length of the returned value '*character-string*' is not equal to that of the function's result. Structures must be of equal length

The length of the variable at a RETURN statement in a function must be equal to the length of the result of the function.

11152 The parameter '*character-string*' for the ORD built-in function contains more than a character

11153 SQL processing problems. RDMS returned the following status: *character-*

string

UCS LSS encountered a problem processing SQL. The problem is described by the status returned from RDMS and included in this message.

11154 `Illegal use of variable '`*`character-string`*`' declared with size ZERO`

11155 `SQL RDMCA message follows:`

This is the header for the RDMS messages that follow. The messages are returned from RDMS in the RDMCA.

11157 `RDMS SQL STANDARD WARNING FOLLOWS:`

This is the header for the RDMS messages that follow. The messages are returned from RDMS in the RDMCA.

11158 `RDMS SQL WARNING FOLLOWS:`

This is the header for the RDMS messages that follow. The messages are returned from RDMS in the RDMCA.

11159 `Host-variable data-type not compatible with data-type of Relational Data Base item. The embedded SQL statement is ignored`

The data type of the item returned by RDMS in the record descriptor (in the RDMCA) cannot be converted to the data type specified for the output host program variable or the number of host variables specified is less than the corresponding relational items. Conversion attempts can cause run-time errors.

11160 `The address operator (&) cannot be applied to va_arg within a function that has the interface pragma applied`

The address operator cannot be applied to va_arg (for example, &va_arg(ap,int *)) within a function that has the #pragma interface directive applied.

11161 `Code following goto, break, continue, or return statement is not reachable`

The code following a goto, break, continue, or return statement is not reachable.

11162 `A constant value is used as the switch expression`

A constant value is used as the switch expression. The switch will always transfer control to the same spot.

11163 `The case value is outside the range of the switch expression`

The constant value for the case label is outside the range of the switch expression. Control can never be transferred to this case label via the switch statement.

11164 `Overlapping operands may cause unpredictable results.`

The target and source for an assignment sharing the same storage. The results of the assignment are not defined.

11165 `RDMS SQL REMARK FOLLOWS:`

This is the header for the RDMS messages that follow. The messages are returned from RDMS in the RDMCA.

11166 `Host-variable specified without required preceding colon. The embedded SQL statement is ignored.`

A host-variable specified in an embedded SQL statement must be preceded by a colon. The colon is no longer optional.

11167 `Number of host variables specified is incorrect. The embedded SQL statement is ignored.`

The number of host variables specified is not equal to the number of corresponding relational items OR the number of host-variables exceeds the maximum allowed. The entire ESQL statement is ignored.

11168 `Record size for USING file '`*`character-string`*`' can cause record truncation`

The record size of a USING file in a SORT statement is such that, due to the other sort parameters or files, it might cause incorrect results. Its records are larger than the sort files, and truncation can occur.

11169 `Record size for GIVING file 'character-string' can cause record truncation`

The record size of a GIVING file in a SORT statement is such that, due to the other sort parameters or files, it might cause incorrect results. Its records are smaller than the sort files, and truncation can occur.

11170 `Record size for USING file 'character-string' is less than the smallest record in the SORT file 'character-string'`

The record size of a USING file record in a SORT statement that specifies a SORT file with variable records is less than the smallest record in the SORT file. This can cause incorrect results.

11171 `Record size for SORT file 'character-string' is less than the smallest record in the GIVING file 'character-string'`

The record size of a SORT file record in a SORT statement that specifies a GIVING phrase and a file with variable records is less than the smallest record in the GIVING file. This can cause incorrect results.

11404 `Parameter limit exceeded`

UCS allows a maximum of 200 parameters to be passed to any subprogram. The limit has been exceeded.

11405 `Maximum static nesting level exceeded`

UCS allows a maximum of 50 static nesting levels for subprograms, since the display (an internal table) used for uplevel stack references has a size of 50.

11406 `UADA front-end information for PADS debugging is incomplete`

The UADA front-end generates information that the back-end uses to generate the Symbolic Debugging Dictionary (SDD). PADS then uses the SDD for debugging. If the front-end information is incomplete, the object module is not affected; however, the SDD may be missing data that could affect PADS debugging.

11424 `OCC : Incorrect number of parameters passed to I18NLIB call to UCOB$SETLOC. Must use 5 parameters.`

The CALL to the I18NLIB routine UCOB$SETLOC must use 5 parameters.

11425 `OCC : Incorrect number of parameters passed to I18NLIB call to UCOB$LNM2CNB. Must use 4 parameters.`

The CALL to the I18NLIB routine UCOB$LNM2CNB must use 4 parameters.

11426 `OCC : Incorrect number of parameters passed to I18NLIB call to UCOB$LNB2LNM. Must use 4 parameters.`

The CALL to the I18NLIB routine UCOB$LNB2LNM must use 4 parameters.

11427 `OCC : Incorrect number of parameters passed to I18NLIB call to UCOB$LNM2LNB. Must use 3 parameters.`

The CALL to the I18NLIB routine UCOB$LNM2LNB must use 3 parameters.

11428 `OCC : Incorrect number of parameters passed to I18NLIB call to UCOB$LNB2CNB. Must use 4 parameters.`

The CALL to the I18NLIB routine UCOB$LNB2CNB must use 4 parameters.

11429 `OCC : Key used in SORT/MERGE statement must not be USAGE DISP-I18N unless the keyword option I18N-IO was specified.`

A data item associated with the KEY clause in a SORT or MERGE statement is declared as USAGE DISP-I18N. This is not legal unless the keyword option I18N-IO was specified.

11431 `Absolute subschema element not found`

The compiler was unable to locate the subschema element in the file that was specified in the INVOKE clause.

11432 `Schema invoked not equal to schema in subschema element`

The schema name on the INVOKE clause does not match the schema name in the absolute subschema SFD table.

11433 `Subschema invoked not equal to subschema in subschema element`

The subschema name on the INVOKE clause does not match the subschema name in the absolute subschema SFD table.

11434 `File invoked not equal to file in subschema element`

The file name on the INVOKE clause does not match the file name in the absolute subschema SFD table.

11435 `Bad status returned on IOW$`

A nonzero IOW$ status was returned on a subschema table read from the absolute subschema tables.

11437 `The subschema element was not processed with the HOST LANGUAGE UCS COBOL clause`

The subschema element that is processed by SDDL must contain the HOST LANGUAGE IS UCS COBOL clause. The HOST LANGUAGE clause specified another language. The INVOKE clause will be ignored.

11439 `DMS record attribute entry expected--not found`

The records specified in the absolute subschema do not match the records specified in the S$PROC element.

11441 SET or AREA name + 'name' which is defined in the subschema is not unique, it duplicates an existing name from the COBOL program - name is accepted

The SET name or AREA name defined in the subschema is not unique. All names within a COBOL program must be able to be uniquely referenced. An ambiguity can result if this name is used later on in the program.

11442 A DMS record name has been duplicately defined

The name of a DMS record has been duplicately defined by another 01 structure. DMS record names must be unique and can not have the same name as another 01 structure defined elsewhere in the program. This duplicate definiton may cause problems at a later point in the program.

11443 Mismatch for parameter 'parameter-name' on CALL statement

The specified parameter is not consistent with the corresponding parameter from a previous call to the same subprogram. Either the size is different or the passing mode (BY VALUE vs. BY REFERENCE or BY CONTENT) is different.

11444 Mismatch for parameter 'parameter-name' on CALL statement

The specified parameter is not consistent with the corresponding parameter from a previous call to the same subprogram. Either the size is different or the passing mode (BY VALUE vs. BY REFERENCE or BY CONTENT) is different.

11445 Parameter count mismatch on CALL statement

The specified CALL statement has a different number of parameters than a previous call to the same subprogram.

11446 USAGE COMP-1 and COMP-2 items in the schema have been allocated in COBOL '74 format; the COMPAT option must be specified for this compilation.

The COBOL source input created by DDL is in COBOL '85 format. However, the corresponding COMP-1, COMP-2 items in the schema have been allocated on COBOL '74 (ASCII COBOL) boundaries. The data will be processed incorrectly unless the COMPAT option is specified.

11447 Schema created with COBOL 85 syntax and data allocation; COMPAT option or COMP-BIN option must not be specified for this compilation.

The COBOL source input and data allocation created by DDL is in COBOL '85 format. In order for the compiler to handle the syntax and data allocation the same as DDL, the COMPAT or COMP-BIN option must not be used. Remove the COMPAT or COMP-BIN option and retry the compilation.

11448 Invoke key not equal to subschema lock in the SSCD table.

The invoke key on the INVOKE clause does not match the subschema lock in the SSCD table.

11449 `An identifier must be an area name defined in the subschema or an identifier with USAGE AREA-NAME - statement is ignored.`

An area name used in this statement must be the name of an area defined in the subschema named by the INVOKE clause of the subschema section. An identifier used in this statement must have been defined with a USAGE AREA-NAME clause and initialized with the name of area defined in the subschema named by the INVOKE clause of subschema section.

11450 `An identifier must be a set name defined in the subschema or an identifier with USAGE SET-NAME - statement is ignored.`

The set name must be the name of a set defined in the subschema named by the INVOKE clause of the subschema section, or it must have been defined with a USAGE SET-NAME clause and initialized with the name of a set defined in the subschema named by the INVOKE clause of subschema section.

11451 `An identifier must be a record name defined in the subschema or an identifier with USAGE RECORD-NAME - statement is ignored.`

The identifier must be the name of a record defined in the subschema named by the INVOKE clause of the subschema section, or it must have been defined with a USAGE RECORD-NAME clause and initialized with the name of a record defined in the subschema named by the INVOKE clause.

11452 `An identifier must be a DATABASE-KEY - statement is ignored.`

The identifier used in this DML statement must have been defined using USAGE DATABASE-KEY.

11453 `An identifier must be an AREA-KEY - statement is ignored.`

The identifier used in this DML statement must have been defined using USAGE AREA-KEY.

11454 `Last identifier in FIND/FETCH format 1 must be defined as AREA-NAME`

The identifier (id-5) of the FIND/FETCH format 1 statement must be defined as USAGE AREA-NAME.

11455 `Illegal identifier in FIND/FETCH format 1`

The identifier (id-5) of the FIND/FETCH format 1 statement must be not be present when id-4 is defined as USAGE DATABASE-KEY.

11456 `Invalid Shift-JIS literal found with a Kanji NX prefix`

Invalid characters were found in a kanji literal that had an NX prefix. The characters found were not valid Shift-JIS characters.

11457 `RSA SQL STANDARD WARNING FOLLOWS:`

RSA parsing has detected an SQL standard violation. This is a warning header. The RSA standard warning messages follow.

11458 `RSA ERROR FOLLOWS:`

RSA parsing has detected illegal syntax. The statement is ignored. This is a major error header. The RSA messages will follow.

11459 RSA FATAL ERROR FOLLOWS:

RSA parsing has detected a fatal situation. The compilation is terminated. This is a fatal error header. The RSA messages will follow.

11460 RSA SQL WARNING MESSAGE FOLLOWS:

RSA parsing has detected an SQL usage problem. This is a warning header. The RSA warning messages follow.

11461 'cursor-name' is duplicate or non-existent cursor-name; statement ignored

An embedded SQL command, "DECLARE cursor-name" has specified an already specified cursor name OR an 'OPEN cursor-name' statement has referenced a nonexistent cursor name. The statement is ignored in either case.

11462 BLOB data type used as host variable; the RDMS level for this application does not support the BLOB data type. The statement is ignored.

The RDMS level installed for this application does not support the BLOB data type. To resolve the problem, install the RDMS level that does support the BLOB data type.

11504 Program size causes the optimizer to reach an internal limit, several optimizations are discontinued

The number of expressions in the program has exceeded the optimizer's internal limit for expressions. Optimizations known as local common subexpression elimination and global common subexpression elimination are discontinued.

11510 Recursive routine not declared as recursive

The optimization phase of the code generator has detected a procedure or routine that is called recursively but has not been declared as a recursive routine. Erroneous results can occur, since not all UCS languages support recursion.

11515 The number of program variables causes the compiler to exceed an internal limit. The number of variables must be reduced to permit compilation of this program

The size of a data structure needed to compile a UCS FORTRAN program has exceeded an internal limit. The number of variables in the program determines the size of this data structure. You must reduce the number of variables in the program to allow its compilation.

11516 The number of program variables causes optimization to exceed an internal limit. Optimization will not be performed

The size of a data structure needed to perform optimization has exceeded an internal limit. The number of variables in the program determines the size of this data structure. You must reduce the number of variables in the program to allow the compiler to perform optimization.

12603 Temporary storage in the AFA for node '*character-string*' at 0*octal-value* has been exhausted

The AFA (Automatic Fixed Area) for the given node (such as a procedure or block) is exhausted. That is, no more automatic storage is available for the node. This can be due to either a large number of variables or a few large variables declared in the node's automatic storage, leaving no room for the allocation of temporary variables generated by the compiler.

12670 line *integer-value* : *character-string* Attempt to reference an undefined parameter

12671 line *integer-value* : *character-string* A constant was expected but not found

12672 line *integer-value* : *character-string* The $XREG or $BREG function does not have a register loaded for the given parameter; In this case, it is necessary to use $XREGVALUE or $BREGVALUE to obtain the desired X or B register

12673 line *integer-value* : *character-string* Instruction field overflow

12676 line *integer-value* : User ABORT '*integer-value*'

12681 line *integer-value* : Call to inline subroutine *character-string* exceeds maximum subroutine depth; $ENTER ignored

12682 line *integer-value* : A $DISP string exceeds the maximum number of characters allowed and is truncated

12683 line *integer-value* : The indicated label *character-string* does not exist

12684 line *integer-value* : The displacement of the $STKRES item from the ALS X-register exceeds the range accessible by $XREGVALUE; $XREGVALUE can only access stack space up to 16000 past the first window

12685 line *integer-value* : Attempt to enter main inline *character-string* ; $ENTER ignored

12686 line *integer-value* : Attempt to enter an undefined inline; $ENTER ignored

12687 line *integer-value* : Attempt to enter inline subroutine *character-string* recursively; $ENTER ignored

12688 line *integer-value* : *character-string* Illegal U-field

12689 line *integer-value* Maximum number of interpretive steps exceeded; Inline interpretation ended

12690 Return value required but not provided by the inline

12691 line *integer-value* : *character-string* is a duplicate local label definition

12692 The maximum number of parameters has been exceeded

12693 line *integer-value* : In UCS PLUS, $STKRES items cannot be referenced directly in an instruction; It is necessary to use the addressing functions: $XREGVALUE, $BREGVALUE, $WADDR etc.

12694 line *integer-value* : The BT instruction requires a constant in the d field; Addressing functions such as $WADDR and $XREGVALUE modify fields in the instruction other than the d field

12695 line *integer-value* : In UCS PLUS, A parameter must be a constant to be referenced directly in an instruction; To reference other types of parameters it is necessary to use the addressing functions: $XREGVALUE, $BREGVALUE, $WADDR etc.

12696 line *integer-value* : A relocated item was found illegally within an expression. Relocation is only supported on the first operand of addition and subtraction expressions. All other operands are expected to be constants

12697 UCS Inline has found omnibus element, *character-string* , to be in error. Inline interpretation is halted. It is necessary to recreate a correct omnibus with UCS Inline

12698 Omnibus element, *character-string* , was created with a version of UCS Inline that contains features not yet implemented in the version of the Interpreter that was used. Errors can occur unless you use a more current version of the Interpreter

13061 Location counter '*integer-value*' exceeds maximum allowable size of '*integer-value*' words

13064 '*integer-value*' words exceed code bank limit sizes

The amount of code generated for the source program is larger than the system allows; the compilation is terminated. The current code bank limit is 262,143 words for the 2200/XPA system and 65,535 words for other systems. To recover from this situation, you must either use the SEGCODE compiler keyword option or break the program into separate smaller compilations.

13412 DEBUG/MONITOR information is too large to be contained in the Symbolic Debugging Dictionary (SDD). The information required to monitor inputs and outputs during a debugging session is not placed in the SDD generated by this compilation.

The DEBUG/MONITOR option generates information in the Symbolic Debugging Dictionary (SDD) in addition to the information generated by the DEBUG, DEBUG/REFERENCE and DEBUG/FULL options. This additional information can cause the SDD to exceed the Level-11 SDD format limit of 16,777,216 words. The compiler detects this condition and does not generate the additional information in the SDD.

This error should be issued only for very large programs. Splitting the program into smaller modules that are compiled separately can help for these cases.

13413 The generated Symbolic Debugging Dictionary (SDD) table is too large for the SDD Level-11 format. If this SDD table is used by PADS during a PADS debugging session, errors can occur

The Symbolic Debugging Dictionary (SDD) table generated has exceeded the limit for the Level-11 SDD format. The limit is 16,777,216 words. If you used the DEBUG/FULL option, try DEBUG. If you used DEBUG, try DEBUG/REFERENCE. If you used DEBUG/REFERENCE, try DEBUG/STATEMENT. If you used DEBUG/STATEMENT, try DEBUG/WALKBACK. If you specified no option, the default DEBUG/WALKBACK was used. If this error is still issued for this case, use NO-DEBUG.

This error should be issued only for very large programs, although certain C programs

with heavily used, complicated type definitions can also create an SDD table that is too big. Splitting the program into smaller modules that are compiled separately can help for these cases.

14502 `Indexed section selector subscript is out of range for dimension '`*`integer-value`*`' of array '`*`character-string`*`'`

An indexed section selector (ISS) has the form:

[ss1] : [ss2] [: ss3]

and specifies values in the array dimension in which it appears from ss1 through ss2 in increments of ss3. If the first value (ss1) or the last value of the ISS is not in the range of the specified array dimension (from lower bound through upper bound), then that ISS is out of range.

14507 `Extent of section selector '`*`integer-value`*`' for '`*`character-string`*`' does not match '`*`character-string`*`'`

Conformability checking tests the extents of each dimension of an array value in an array expression or array assignment statement against the extents of the dimensions of the first array value that appeared. If the extents are not all equal, the array values are not conformable and the expression is in error. In this case, the extent of the specified section selector for the array section does not match the extent of the corresponding dimension of the first array value in the array expression.

14508 `Vector '`*`character-string`*`' in PACK/UNPACK is too small for array '`*`character-string`*`'`

In the statement PACK(v1,aexp[,ct]), where v1 is a vector (rank 1 array) and aexp is an array expression, results are undefined if the number of elements of aexp assigned to v1 exceeds the size of v1.

If the PACK statement is not in a WHERE structure (that is, an unconditional PACK statement), and aexp is an array (rather than a scalar) expression, then the number of elements assigned to v1 is the size of aexp.

In the statement UNPACK(a,vexp[,ct]), where a is an array and vexp is is a vector (rank 1) expression, results are undefined if the number of elements in vexp (if vexp is not a scalar) is insufficient to supply values for a.

14509 `Extents of dimension '`*`integer-value`*`' of parameters passed to '`*`character-string`*`' do not match`

Conformability checking tests the extents of each dimension of an array value against the extents of another array value. If the extents are not all equal, the array values are not conformable and the statement is in error. In this case, a nonelemental built-in function that requires conformable parameters has appeared. The extents of the specified dimension of the two parameters does not match, so the parameters are not conformable.

14511 Variable length character array temp cannot be generated - not implemented

A character temporary that contains array elements whose character lengths are not known at compile-time (that is, variable lengths) cannot be generated, since this feature is not implemented in UCS. A variable-length character array temporary can result from an array expression that contains concatenations or substrings and is passed as a parameter or used as an I/O list item. It can also result from a character array assignment statement that contains a possible data dependency (that is, data that appears in the right-hand expression overlays the data item that appears in the left-hand expression). In this case, an array temporary is required for the right-hand expression.

14513 Variable length character array temps not implemented – assume no dependency

This character array assignment statement has a possible data dependency (that is, data that appears in the right-hand expression overlays the data that appears in the left-hand expression). In this case, an array temporary is usually required for the right-hand expression. However, since variable-length character array temporaries are not implemented in UCS, and the right-hand expression has a variable character length, no array temporary is generated for this statement. In other words, it is as if no data dependency had been detected. A variable-length character array temporary is a character temporary that contains array elements whose character length is unknown at compile time.

14514 Extent of dimension '*integer-value*' for '*character-string*' does not match '*character-string*'

Conformability checking tests the extent of each dimension of an array value in an array expression or array assignment statement against the extent of the dimension of the first array value that appeared. If the extents are not all equal, the array values are not conformable and the expression is in error. In this case, the extent of the specified dimension for the array does not match the extent of the corresponding dimension of the first array value in the array expression.

14515 Extent '*integer-value*' of function '*character-string*' result does not match '*character-string*'

Conformability checking tests the extent of each dimension of an array value in an array expression or array assignment statement against the extent of the dimension of the first array value that appeared. If the extents are not all equal, the array values are not conformable and the expression is in error. In this case, the extent of the specified dimension of the array-valued built-in function result does not match the extent of the corresponding dimension of the first array value in the array expression.

14517 Parameter 2 (dimension number) passed to '*character-string*' is not constant

The dimension number passed to the specified built-in nonelemental reduction

function as the second parameter must be a constant.

14518 `Built-in function not implemented`

A specified nonelemental built-in function has not been implemented in UCS.

14521 `Extent of dim. '`*`integer-value`*`' of parameter 1 passed to MATMUL does not match dim. '`*`integer-value`*`' of parameter 2`

Two arrays are passed as parameters to the MATMUL intrinsic function. One of the extents of the first parameter (a1) must match one of the extents of the second parameter (a2).

If both parameters have rank 2, then the shape of a1 must be (n,m) and the shape of a2 must be (m,k).

If a1 has rank 1 and a2 has rank 2, then the shape of a1 must be (m) and the shape of a2 must be (m,k).

If a1 has rank 2 and a2 has rank 2, then the shape of a1 must be (n,m) and the shape of a2 must be (m).

14522 `A varying length character array temp cannot be generated - assume no dependency`

This character array assignment statement has a possible data dependency (that is, data that appears in the right-hand expression overlays the data that appears in the left-hand expression). In this case, an array temporary is usually required for the right-hand expression. However, since variable-length character array temporaries are not allowed in UCS, and the right-hand expression has a varying character length (caused by a Kanji intrinsic function), no array temporary is generated for this statement. In other words, it is as if no data dependency had been detected.

15500 `Name '`*`character-string`*`' is a dummy argument -- not expanded`

The procedure name specified is a dummy procedure name. A dummy procedure cannot be expanded inline.

15501 `Procedure '`*`character-string`*`' is not in this element -- not expanded`

The procedure must be in the same compilation unit or included into the same element to be expanded inline.

15502 `Recursive call to '`*`character-string`*`' is not expanded`

A procedure is not allowed to call itself. Therefore, the procedure is not expanded inline.

15503 `Procedure '`*`character-string`*`' was previously defined to be unexpandable`

A previous call to this procedure determined that it could not be expanded inline.

15504 `Procedure '`*`character-string`*`' has alternate entry points -- not expanded`

The procedure has alternate entry points. Therefore, it cannot be expanded inline.

15505 `Procedure '`*`character-string`*`' has an ellipsis (, ...) in the parameter type list -- not expanded`

The number of parameters is unknown. Therefore, the procedure cannot be

expanded inline. The procedure has ', ...' in the parameter type list.

15506 `An argument passed to procedure '`*`character-string`*`' does not fit the compiler criterion -- not expanded`

One of the following was found for an argument of the procedure; therefore, the procedure cannot be expanded inline:

- The actual argument is an alias array.
- The actual argument is an array expression.
- The dummy argument is an assumed-shape array, and one of the following is true:
- The lower bound or upper bound of the actual argument is not constant.
- The lower bound of the actual argument is not 1.
- The rank is different between the actual and dummy argument.
- The actual argument is not an array.
- The dummy argument is an array but the actual argument is not an array or array element reference.
- The dummy argument is a character array and the actual argument is a character array element reference.
- The actual argument is an array but the dummy argument is not an array.
- The actual argument type does not match that of the dummy argument.
- The actual argument is of type character and its length does not equal that of the dummy argument.
- The number of actual arguments is not equal to the number of dummy arguments.

15507 `Procedure '`*`character-string`*`' has unexpandable statements or unexpandable usage of the array argument`

One of the following was found in the procedure; therefore, the procedure cannot be expanded inline:

- An allocate statement is used.
- An embedded Structured Query Language (SQL) statement is used.
- The procedure contains an alternate return statement with a nonconstant return index.
- A dummy array name is used in an I/O list, an expression, an intrinsic function (LBOUND, UBOUND, SIZE, or EXTENT), an IDENTIFY statement, or an array section reference; and the upper bound of the last dimension is not constant or not the same as the actual argument.
- The procedure contains a call to a dummy subroutine with one or more arguments.
- The procedure contains a call to a dummy function.

- The internal table for inline expansion overflowed.

15509 `Attribute to prohibit inline expansion specified for '`*`character-string`*`' -- not expanded`

An attribute to prohibit inline expansion was specified for the subprogram. Therefore, the call to the subprogram cannot be expanded inline.

15510 `Cannot expand call to '`*`character-string`*`' because it contains a call to a nested subprogram that was not expanded`

In order to expand a subprogram that has nested subprograms, all calls to nested subprograms must first be expanded inline. The compiler automatically performs this inline expansion when you specify the EXPAND attribute on your subprogram declaration. You can also insert EXPAND directives for each call to a nested subprogram. However, expansion of nested subprograms can still fail. For example, inline expansion fails if you declare a nested subprogram with the NO_EXPAND attribute or if the nested subprogram is recursive.

15511 `Call to '`*`character-string`*`' expanded inline`

Inline expansion was performed on the call to the specified subprogram.

20002 `ELMS error while issuing message number `*`integer-value`*`, ERROR STATUS: `*`octal-value`* `SUBSTATUS: `*`octal-value`*

The Extended Language Message System (ELMS) was called by the compiler to issue a message. ELMS returned an error status indicating it was unable to complete the request. Refer to the ELMS Application Writers Guide document for a description of the status values. Note that message 20002 is not issued by ELMS. It is constructed and issued internally by the common system interface message routine. For this reason, this message is not translatable.

20016 `The error limit of `*`integer-value`*`, specified on the compiler keyword option MAXERRORS, has been exceeded`

20082 `The '`*`character-string`*`' directive for service routine '`*`character-string`*`' occurs more than once. This occurrence is ignored`

A directive occurs more than once in a service routine entry. The first occurrence will override all following occurrences.

20156 `Element or procedure `*`character-string`* `not found for PLUS or UINLINE COPY`

The element or procedure requested by a COPY statement was not found. Source errors can occur because all of the source statements may not have been found.

20157 `Invalid file assignment during PLUS COPY statement processing`

20160 `File `*`character-string`* `not assigned for COPY statement -- proc or INCLUDE element search not performed`

A file specified on a COPY or INCLUDE statement cannot be assigned. The procedure search or INCLUDE element search is not performed. Errors can occur in the compilation because all source statements may not have been found. This error can

also occur for a file given with the INCLUDE option, in which case this file will not be searched.

20161 Procedure *character-string* not found for COPY (COBOL) or INCLUDE (FORTRAN, C) statement

The procedure is not found when processing COBOL COPY, FORTRAN INCLUDE, or C #include statements.

20163 Invalid file *character-string* for COPY or INCLUDE -- proc or INCLUDE element search not performed

A file name specified on a COPY or INCLUDE statement is invalid. The procedure search or INCLUDE element search is not performed. Errors can occur in the compilation because all source statements may not have been found. This error can also occur for a file given with the INCLUDE option, in which case this file will not be searched.

20164 PROCEDURE *character-string* not found for FORTRAN INCLUDE statement

The procedure specified in a FORTRAN INCLUDE statement is not found.

20229 Source input file required for FORTRAN converter

Failure to specify a source input file when using the FORTRAN converter is illegal.

20273 Invalid front-end and back-end level combination. Front-End compatibility-value: *character-string*. LSS compatibility-value: *character-string*.

The Language Processor and LSS levels are incompatible. This error is fatal, and the compilation is terminated. The 'compatibility-values' which are printed-out give an indication of the extent of the mismatch. Incompatible release tapes were probably installed on your system.

20278 Program too large. Reduce program size

The compiler has exhausted all of its internal working storage and is unable to continue processing.

20279 Read key required for the file *character-string*

20290 Invalid keyword option * *character-string* *

An invalid keyword option is specified either on the compiler call statement or in the keyword options file.

20295 I option is invalid with FORTRAN converter

The I option is not allowed with the FORTRAN CONVERT option.

20297 Error during the opening of a file for input

An error occurred during the opening of a file for input. This error terminates the compilation.

20298 Error during attempted read

20299 Error during attempted close of a file

20312 CSF error during omnibus assignment or use of omnibus file

20313 Error when next write location retrieved during omnibus element processing

20314 Omnibus output file is not a program file

The retrieval of the next write location by the ER PFWL$ failed because the file is not a program file.

20321 RDML does not exist on this system

The user attempted to use RDML when RDML does not exist on the compiler being used.

20324 Invalid file, element, or file and element specification

20329 Option letters *character-string* are not valid for UCS compilation. Check the appropriate UCS language programming reference manual for valid options

20332 Invalid status returned when executing ER SYMBINFO$

20335 Invalid close when closing options file

20336 No keyword options specified in keyword option file

The user-specified file does not contain keyword options. The compilation continues normally, using any keyword options on the compiler call and any specified letter options.

20337 Error during read of keyword options file (4th field)

An error occurred during the reading of a user-specified file containing keyword options. This error is fatal and causes termination of the compilation.

20338 The CONTIG and NO-CONTIG options have no effect on the compilation

The CONTIG and NO-CONTIG options do not have an effect on the compilation. Use the ASHAPE compiler call option when passing a contiguous array to an assumed-shape array.

20339 *character-string* mode is illegal for this language

20343 Optimization can cause differences in error detection by RUNCHECK

RUNCHECK error condition testing either cannot be performed or is performed out of the normal statement execution sequence when the OPTIM option is active. This is due to program restructuring and statement eliminations performed by optimization.

20348 Filename not specified when using the CONVERT option

The filename is not specified in the third field on the compiler call for the FORTRAN CONVERT option.

20351 Option letter or letters *character-string* are invalid when using the FORTRAN converter. Check the UCS FORTRAN Programming Reference Manual Volume 2 for valid converter options

20352 *character-string* is an invalid keyword option when using the FORTRAN converter

20364 The SEGDATA option is valid only in extended mode

20365 * *character-string* * is valid only with the FORTRAN compiler

A user specified a keyword option that is valid only with the FORTRAN compiler.

20366 File * *character-string* * not assigned for FORTRAN converter processing

The file UFTN$NEWPROC containing newly converted procs for the FORTRAN converter is incorrectly assigned.

20367 FORTRAN convert file is not a program file

The file UFTN$NEWPROC containing newly converted procs for the FORTRAN converter is not a program file.

20368 Invalid next write location for the FORTRAN convert file

20370 Include element *character-string* not found for C or FORTRAN include statement

The include element was not found when processing C #include or FORTRAN INCLUDE statements.

20372 C include element nesting level exceeds the ANSI standard defined limit of 8

20373 C include element nesting level exceeds maximum of 25

20374 Third field usage only allowed for FORTRAN converter and C preprocessor

It is illegal to use the third field on the compiler call except for use of the FORTRAN converter and the C preprocessor.

20375 Keywords must start in the fifth field of the compiler call

It is illegal to use any other field than the fifth field to start compiler keywords.

20382 Option *character-string* is illegal for language, mode or other options

The option specified is either not allowed for this language, not legal for this mode, or conflicts with another option previously specified.

20383 Options *character-string* and *character-string* conflict. *character-string* is now in effect

The option specified is set by either two keyword options from the options element, two letter options, or two keyword options from the compiler call line. The last option specified takes precedence.

20384 Option *character-string* reset to *character-string*

Either the option was set in the options element and reset by a letter or keyword option from the compiler call line, or it was set by a letter option and reset by a keyword option from the compiler call line.

20386 An illegal version name was specified for the option *character-string*

The version specified with the given option is illegal for this language.

20388 Too many copy- and include-generated usenames

A COPY and INCLUDE statement that causes an include from a file that is not assigned to the run generates an @USE of that file. LSS allows 1000 of these use names. The 1001st call generates this error.

20389 Option *character-string* has been previously set

This option was previously specified in the options element, by a letter option, or by a keyword option on the compiler call line.

20390 TRANSFORM and NO-OPTIM is an illegal option combination

It is illegal to use both TRANSFORM and NO-OPTIM at the same time.

20396 The numeric value specified with option *character-string* is out of range. The default value is used

The value specified with the given option is either negative or is not an integer. The default value is used.

20402 Unable to assign file SERV$DF

The user specified the option SERVTBL to modify the service routine table, based on the information in the file with the use name SERV$DF. This file could not be assigned.

20403 SERV$DF not processed; service routine tables unmodified

An error condition makes processing of SERV$DF impossible.

20404 Invalid '*character-string*' directive encountered during processing of service routine '*character-string*'. This entry is ignored

An unrecognizable directive was encountered during processing of the service routine tables. This entry is ignored.

20405 An invalid value was specified for '*character-string*' directive in service routine '*character-string*'. This entry is ignored

The value specified for the given service routine table directive was not a legal value for that directive. This entry is ignored.

20406 A service routine table entry specified in SERV$DF is missing a name

A name must be specified for every service routine table entry added by the file SERV$DF. This entry is ignored.

20408 The SERVTBL option cannot be used with the CONVERT or CONVERT/PROC options

20409 Too many explicit parameters supplied with service routine *character-string*

More than the maximum allowable number of explicit parameters are supplied with the specified service routine. The excess parameter is ignored.

20410 NO-ALIAS and NO-OPTIM is an illegal option combination

The NO-ALIAS option is a subset of the OPTIM option and cannot be turned on when optimization is turned off.

20411 PLUS copy element nesting level exceeds maximum of 10

20412 *character-string* is not installed. Contact your site administrator for installation of *character-string* using mode B installation

The system include file for the alternate environment UCS FORTRAN compiler is not installed. Contact your site administrator for mode B installation of the compiler.

20413 *character-string* is not installed. Contact your site administrator to have it installed

The system include file for the UCS FORTRAN compiler is not installed. Contact your site administrator to have it installed.

20414 Option letter U is not valid for compilation, and causes unwanted, unrecoverable processing by SYSLIB routines. Check the appropriate UCS language programming reference manual for valid options

20417 Bad filename *character-string* for inline read

An invalid filename is specified for an inline read operation.

20418 Error while setting up usename for file *character-string* for inline read

20419 File *character-string* cannot be assigned for inline read

20420 Inline element *character-string* not found for inline read

20423 UREP thread for processing SQL related actions could not be terminated. UREP Thread Control returned the following status: *character-string*

UCS LSS cannot disestablish the thread acquired for SQL processing. User intervention may be required to correct the problem.

20425 Ada library file could not be found

No library file is specified on the call line, and the ADA$LIB library file cannot be assigned.

20426 Error when freeing file ADA$LIB

20427 The source input field cannot be specified when using the GEN-INST option

The source input file cannot be specified when doing generic instantiation.

20428 The combination of the I option and the GEN-INST option is illegal

20430 The combination of the PARMCHECK option and the ALT-CALL option is illegal

20435 Option name exceeds the maximum option length

An option name specified exceeds the maximum option length.

20436 Version name exceeds the maximum version length

A version name specified exceeds the maximum version length.

20437 Options *character-string* and *character-string* are an illegal option combination

It is illegal to specify these options on the same compilation.

20438 Option *character-string* is still in effect

A specified option caused a related option to be set. If the specified option is overridden the related option is still set.

20439 `EOF was encountered following the` *`character-string`* `continuation marker`

An end-of-file condition following a continuation marker was encountered during the processing of the keyword options element.

20440 *`character-string`* `string exceeds the maximum length`

A keyword option string exceeds the specified maximum length. This can happen on the keyword options DEFINE-ID, UNDEFINE-ID, ASSERT and INCLUDE.

20441 `Error when processing the SOURCE INPUT field`

An error occurred during the processing of the source input field. The error can occur for any of the following reasons: - The syntax of the source input file or element name is in error. - The compiler cannot access the source input file or element.

20442 `Error when processing the OBJECT MODULE OUTPUT field`

An error occurred during the processing of the object module output field. error can occur for any of the following reasons: - The syntax of the object module output file or element name is in error. - The compiler cannot access the object module output file or element.

20443 `Error when processing the SOURCE OUTPUT field`

An error occurred during the processing of the source output field. The error can occur for any of the following reasons: - The syntax of the source output file or element name is in error. - The compiler cannot access the source output file or element.

20444 `Error when processing the KEYWORD OPTIONS field`

An error occurred during the processing of the keyword options field. The error can occur for any of the following reasons: - The syntax of the keyword options file or element name is in error. - The compiler cannot access the keyword options file or element.

20445 `Too many INCLUDE options specified – File` *`character-string`* `ignored.`

More than the maximum allowable number of INCLUDE options were supplied. The excess INCLUDE option is ignored and the file will not be searched.

20446 `Unknown compilation environment`

The compiler is not able to find compilation environment information.

20447 `Unable to find I/O routine address`

The compiler is not able to find the address of an I/O routine.

20448 *`character-string`* `routine error:` *`integer-value character-string`*

An I/O routine generated an error message.

20449 `I option is invalid with C preprocessor`

The I option is not allowed when using the C preprocessor.

20451 `Source input file required when using the C preprocessor`

A source input file must be specified when using the C preprocessor.

Appendix B
Execution Time Diagnostic Messages

This appendix describes the diagnostic messages produced by a Universal Compiling System (UCS) COBOL program or the UCS Runtime System during the execution process, as follows:

- UCS Runtime System messages (see B.1)

  These messages have either of the identifiers URTS or CONTINGENCY on listings.
- UCS Runtime System messages for 19002 and 35001 (see B.2)

  This subsection contains special messages that can be inserted into the text for URTS messages 19002 and 35001, depending on the file-related error detected.
- Sort/Merge messages (see B.6)

  These messages occur when the sort or merge features available to UCS COBOL are used.

Format

```
*message-class(message-source) message-number: message-text
```

where *message-source* is one of the following:

- UCOB
- URTS
- CONTINGENCY

  For this value, the following phrase may be appended to the message text:

  ```
  - in another subsystem
  ```

  This phrase indicates that the contingency actually occurred in an executing subsystem different from the user program or the URTS subsystem. Typically, this happens for one of the following reasons:

  – The contingency occurred in a basic mode alternate file common bank (AFCB), since the operating system treats basic mode AFCBs as a different subsystem from extended mode.

  – The contingency occurred in an extended mode subsystem different from the subsystem of the user program.

See Appendix A for a listing of the diagnostic messages produced during program compilation. That appendix also contains general information about message classification, format, and implementation.

B.1. UCS Runtime System Messages (URTS)

01000 `Wrong index value between MERGE and RETURN statement`

The RETURN statement's index to the MERGE statement is greater than number of MERGE statements in the program.

01001 `File(s) associated with report not open`

The flow of the program has reached a report writer verb before the file is opened.

01002 `Report not INITIATED or GENERATED before TERMINATE`

The program flow has reach a TERMINATE before an INITIATE or a GENERATE of this report has been executed.

01003 `Report not TERMINATED before INITIATE`

The program flow has reached a second INITIATE of this report before a TERMINATE has been executed.

01004 `Report must get INITIATED before GENERATE`

The program flow has reached a GENERATE of this report before an INITIATE of this report has been executed.

01005 `Subprogram being canceled is active - cancel not done`

The program to be canceled is still active. No CANCEL is done.

01006 `Subprogram being called is active - call not completed`

The program being called is still active. No CALL is done.

01007 `Number of parameters of called and callee must equal`

The number of parameters passed to the called program must not exceed the number of parameters it has requested.

01008 `Subscript value negative or greater than OCCURS`

The value of the subscript is not within the range of the number of occurrences of the given identifier. This diagnostic occurs when the keyword option RUNCHECK is used.

01009 `Reference modification starting to large or negative`

The starting point for the reference modification must be within the size of the identifier. This diagnostic occurs only when the keyword option RUNCHECK is used.

01010 `Reference modification size to large or negative`

The requested size of the reference modification is larger than the remaining size of the identifier, or is less than one. This diagnostic occurs only when the keyword option RUNCHECK is used.

01011 Incorrect index is encountered during the processing of a DMS command.

The index returned by DMS is greater than any expected by the commands.

01012 A negative number is argument to the 'FACTORIAL' function.

A negative number is passed to the function 'FACTORIAL' as an argument.

01013 File used in RELEASE/RETURN is not the same as in SORT.

The file name used in the RETURN statement, or the file whose record name is referenced in the RELEASE statement, is not the same as the SORT or MERGE statements.

01014 The value of the interest rate of the ANNUITY function must be greater than or equal to zero.

01015 The number of periods for the ANNUITY function must be an integer greater than zero.

01016 The interest rate for the PRESENT VALUE function must be greater than -1.

01017 The first argument for the NUMVAL or NUMVALC function is illegal.

01018 Parameter passed to program compiled with COMPAT/ARGUMENTS option is not word aligned

When the keyword option COMPAT/ARGUMENTS is specified, all parameters passed to the program must be word aligned. This diagnostic occurs when a parameter is not word aligned and the keyword option PARMCHECK is specified.

01019 Invalid CODE TYPE value encountered

01020 Invalid LOCALE name value. The specified LOCALE is not installed

01022 Error status returned from call to UCOB$SETLOC

01023 Error status returned from call to UCOB$LNM2CNB

01024 Error status returned from call to UCOB$LNM2LNB

01025 Error status returned from call to UCOB$LNB2LNM

01026 Error status returned from call to UCOB$LNB2CNB

01027 No data available to ACCEPT statement.

01028 CODE TYPE for file not compatible with locale in SORT/MERGE

01029 Data exception error detected

01030 Float division error

01031 CALL ON EXCEPTION error

01032 ALTER statement error

01033 Illegal recursive call detected

01034 Power overflow error

01035 Intrinsic function parameter number error

01036 Intrinsic function parameter value error

01037 Report writer statement order error

01038 OO runtime error

01039 Invalid error message number generated by the compiler

01040 Kanji conversion error, illegal character found on input.

07500 - Conform - A logical error occurred.

07501 - Conform - The object referenced by the identifier to the left of AS does not conform with the AS Modifier.

07502 - Conform - A parameter in the INVOKE statement does not conform with the formal parameter in the invoked method.

An actual parameter in the INVOKE statement does not conform with the corresponding formal parameter in the invoked method, or the formal result does not conform with the actual result, or the usage of Object Property is incorrect.

07503 - Conform - The result of the invoked method must be a handle-type data item that can receive objects of the invoking class.

The actual result of the invoked method must be a handle-type data item that can receive objects of the invoking class if the definition of the formal result of the invoked method is a SELF handle type.

07550 - GC - A logical error occurred.

07551 - GC - The area required to create the object cannot be allocated.

07600 - Msearch - A logical error occurred.

07601 - Msearch - The object used in the invocation or in the reference of Object Property is the NULL object.

07602 - Msearch - The object used in the invocation or in the reference of Object Property is invalid.

07603 - Msearch - Cannot find the method to be invoked, or the Object Property is not in the object.

07604 - Msearch - Only an invariant method can be invoked by an invariant method.

07652 - misc - The area required to create the class table cannot be allocated.

07750 Array index is out of the allowable range

A subscript was used that was not in the allowable extent of one of the dimensions of the array.

07751 Variable not defined in the assigned GO TO label list

While executing an assigned GO TO statement, an attempt was made to transfer control to a statement whose label has not appeared in the label list of an assigned GO TO statement.

07752 Array already allocated

An attempt was made to allocate storage for an allocatable array whose storage had already been allocated.

An ALLOCATE statement can be executed only for allocatable arrays whose storage is not currently allocated. The ALLOCATE statement cannot be used with:

- Arrays that have never been allocated
- Arrays specified in a DEALLOCATE statement that was executed prior to the ALLOCATE statement
- Arrays that were allocated in a subprogram but whose names did not appear in a SAVE statement before execution of a RETURN or END statement

07753 Start position argument is out of range

An attempt was made to reference a character or bit position not contained in the scalar variable, array element, or expression. The starting position must satisfy the relational expression:

1 LE *starting-pos* LE *item-length*

where *item-length* is the length of the scalar variable, array element, or expression.

Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for a description of character substrings, the BITS, SBITS, SUBSTR, or IBITS intrinsic functions, or the MVBITS intrinsic subroutine.

07755 Arrays are not conformable (extents are unequal)

Conformable arrays were expected, but were not found when executing one of the following:

- An array expression
- An array assignment statement
- A WHERE statement sequence

Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for a description of conformability in array expressions, in array assignment statements, and in a WHERE statement sequence.

07756 Indexed section selector subscript is out of range

Either the first or last subscript in an indexed section selector is not in the allowable range for the array being referenced.

Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for a description of indexed section selectors.

07757 `Illegal argument passed to assumed-shape array`

An error was detected when checking for the allowable associations between the actual and dummy arguments. One of the following conditions occurred:

- A constant or adjustable dimensioned array was passed to an assumed-shape array and the calling routine was not compiled with the ASHAPE compiler keyword option. Recompile the calling routine with the ASHAPE option.
- An assumed-size array or a scalar value was passed to an assumed-shape array.

Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for a description of assumed-noncontiguous arrays, assumed-size arrays, and the allowable associations between actual and dummy argument arrays.

07758 `Bit position argument is out of range`

An attempt was made to reference a bit position not contained in a numeric storage unit. The following relationship must be satisfied:

```
0 LE bit-position LE 35
```

This error message is produced as a result of incorrectly using a BTEST, an IBSET, or an IBCLR intrinsic function.

Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for descriptions of the BTEST, IBSET, and IBCLR intrinsic functions.

07759 `Parameters are not conformable (extents are unequal)`

Conformable arrays were expected but were not found when passing parameters to a nonelemental intrinsic function.

Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for a definition of conformability and a description of which parameters passed to a nonelemental intrinsic function must be conformable.

07760 `An assumed-noncontiguous array can be passed only to an assumed-shape array`

An error was detected when checking for the allowable associations between the actual and dummy argument arrays. An assumed-noncontiguous array is illegally passed as an actual argument to a dummy argument that is not an assumed-shape array.

Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for a description of assumed-noncontiguous arrays, assumed-size arrays, and the allowable associations between actual and dummy argument arrays.

07761 `Intrinsic function dimension argument is out of range`

The dimension argument for an array query, array reduction, or array index reduction function is out of range. The following relationship must be satisfied:

```
1 <= value of dimension argument <= rank of the array
```

This error can also result when the dimension argument specifies the last dimension of an assumed-size array because the extent of the last dimension of an assumed-size array is undefined.

07762 `Return code from DMS 1100 is out of range`

When a call is made to DMS to execute an FDML statement, DMS returns a code indicating the action the program is to take after execution of the FDML statement. The possible actions are normal return (continue normal execution with the next statement), error return, at-end return, or rollback return.

07763 `RETURN from a ROLLBACK subroutine`

If an IMPART statement of the form IMPART(ROLLBACK=subr) is used and subroutine SUBR executes a RETURN statement, the program is terminated in error.

07764 `Log message length exceeds maximum of 250 words`

In the FDML LOG statement

```
LOG (length) log-area-var
```

the length of the message, length, exceeds the limit of 250 words.

07765 `Character expression exceeds maximum length`

This message is issued for either of the following reasons:

- A character expression that involves either of the following operands in a catenation operation is restricted to 4096 characters unless the expression is the source expression of an assignment statement:
- An operand whose length specification is an asterisk in parentheses (excluding a symbolic name of a constant)
- An operand that is a substring reference, where the substring length is not constant. This applies when the item meets the description in the preceding bullet.
- A character assignment statement of the form A = E is used where E has the potential for defining character positions in A that are referenced in E, and either the COMPILER statement option NONSTD_CHAR_ASSIGNMENT=ON is specified or the COMPAT compiler keyword option is specified. In addition, when one of the following conditions exists, the compiler moves E to a temporary location before the assignment to A (in which case, the expression E is restricted to 4096 characters in length):
- E represents a concatenation operation as described in the first bullet above
- E represents a substring reference as described in the second condition of the first bullet above

07766 `Allocatable character array exceeds the segmented size limit`

This message is issued for the following reasons:

- An ALLOCATE statement is allocating a segmented character array that exceeds the 16 megaword (2**24 = 16,777,216) limit
- A character array expression causes the compiler to allocate a segmented character array that exceeds the 16 megaword (2**24 = 16,777,216) limit

07767 `Shift count argument is out of range`

The shift count argument for the ISHFT or ISHFTC intrinsic function is out of range. This message is issued when one of the following limits is not satisfied:

- ISHFT: -36 LE shift-count LE 36
- ISHFTC: ABS(shift-count) LE field-size

The following additional constraint must also be satisfied:

```
1 LE field-size LE 36
```

Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for descriptions of the ISHFT and ISHFTC intrinsic functions.

07768 `Field size argument is out of range`

The field size argument for the ISHFTC intrinsic function is out of range. The following relationship must be satisfied:

```
1 LE field-size LE 36
```

Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for a description of the ISHFTC intrinsic function.

07769 `Length argument is out of range`

An attempt was made to reference a character or bit position with a length greater than the length of the scalar variable, array element, or expression. The length must satisfy the relationship:

```
1 LE length-arg LE item-length
```

where item-length is the length of the scalar variable, array element, or expression.

Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for a description of character substrings, the BITS, SBITS, SUBSTR or IBITS intrinsic functions, or the MVBITS intrinsic subroutine.

07770 `Calculated end position is out of range`

An attempt was made to reference a character or bit position not contained in the scalar variable, array element, or expression. The end position calculated from the expression (starting-pos + length) does not satisfy the relationship:

```
1 LE ending-pos LE item-length
```

where item-length is the length of the scalar variable, array element, or expression.

Refer to the *FORTRAN Compiler Programming Reference Manual Volume 1* for a description of character substrings, the BITS, SBITS, SUBSTR or IBITS intrinsic functions, or the MVBITS intrinsic subroutine.

10000 `Error mode` *`error-type`*

Base mode error mode. The Error Type, Error Code, and Contingency Type values are passed as parameters.

10001 Extended mode contingency *decimal-value error-type*

Extended mode contingency that is not one of the contingencies listed below. The extended mode contingency message number and the basic mode Error Type, Error Code, Contingency Type, and Aux Info are passed as parameters.

10100 Illegal operation

10101 24-bit indexing violation

10102 Execute repeated violation

10103 BMTC/EDDE control character violation

10104 EDDE storage reference violation

10105 BMTC bit count violation

10106 Incompatible GRS sequence

10200 Privileged instruction violation

10201 Storage limits violation

This can be an array index out of range, an invalid or null pointer, or a reference to a record area of an unopened file. If the bad array reference is on a routine call, the error does not occur until the called routine attempts to access the argument in error. (This includes implicit routine calls, such as a UCS FORTRAN WRITE statement; in this case the error occurs in the UCS Runtime System.) It also occurs by calling a routine with fewer arguments than the routine requires. You can turn on array index checking with the compiler option RUNCHECK. Invalid pointers should only occur as a result of uninitialized variables.

10202 Read access violation

10203 Write access violation

This can be an invalid (uninitialized) pointer.

10204 Control register violation *aux-info*

10205 Illegal bank type specified in bank descriptor *aux-info*

10206 Illegal BDI specified *aux-info*

This can be an invalid (uninitialized) pointer.

10207 Entry point violation *aux-info*

10208 Enter access violation *aux-info*

10209 Interface specification violation *aux-info*

10210 Re-entry capture failure *aux-info*

10211 GOTO inhibit violation *aux-info*

10212 Architectural boundary violation *aux-info*

10213 Invalid fast return *aux-info*

10214 Limit error (BDI value too large) *aux-info*

This can be an invalid (uninitialized) pointer.

10215 Architectural upper limit error *aux-info*

10216 Base register violation (entry point violation) *aux-info*

10217 Dynamic bank based not in memory

10300 Floating-point overflow

10301 Floating-point underflow

10302 Divide-fault

10400 Test and set interrupt *aux-info*

10401 Disabled ER attempted *aux-info*

10500 Breakpoint interrupt *aux-info*

10600 IP hardware fault *aux-info*

10601 IP/storage hardware fault *aux-info*

10700 Linking System has returned an error status *aux-info*

Errors 10700 through 10795 describe errors associated with the Linking System. Errors 10701 through 10724, 10730 through 10732, and 10737 are Exec detected errors. Errors 10740 through 10795 are Linking System detected errors. For errors 10740 through 10795, see the *Linking System Programming Reference Manual*. 10740 corresponds to error code 40; 10741 to error code 41; likewise for all errors up to 10776 (which is error code 76). Error 10795 corresponds to error code 77.

10701 Error occurred during creation of subsystem control table *aux-info*

10702 No executing subsystem control table for caller (possibly an unresolved reference in a ZOOM or a link fault in a common bank or an absolute) *aux-info*

10703 Linking System unable to acquire an ALS frame *aux-info*

10704 Linking System parameter list inaccessible *aux-info*

10705 Extended mode environment needed but does not exist *aux-info*

10706 Linking System not active *aux-info*

10707 Linking System returned a bad reentry address *aux-info*

10708 Illegal start address of the object module or subsystem *aux-info*

10709 Extended mode object modules not supported for TIP *aux-info*

10710 Object modules must start at a program level bank *aux-info*

10711 A higher level subsystem attempted a transition to a lower subsystem. (The called subsystem does not have the privilege to create application level banks) *aux-info*

10712 No PCT buffer space to create subsystem control table *aux-info*

10713 No more domain numbers available to create this subsystem *aux-info*

10714 Lower level flag set incorrectly by Linking System *aux-info*

10715 A subsystem transition was attempted while already in transition *aux-info*

10716 Target subsystem file not assigned *aux-info*

10717 Execute access to target subsystem file denied *aux-info*

10718 Execute access to target subsystem denied *aux-info*

10719 Error occurred while creating the target subsystem *aux-info*

10720 One of the activity-local stack bank limits is invalid or the bank is a large bank

10721 Linking System returned an illegal value for one of the gates into the Linking System

10722 Linking System did not build any gates for the fixed gate bank

10723 The initial load of the subsystem failed; the activity receiving the contingency was not responsible for the error

10724 The initial load of a subsystem failed because the subsystem was being deactivated

10725 Initial-load of Chameleon FGSS failed because it does not meet system-low criteria.

10730 The reentry address to the fixed gate shared subsystem is illegal

10731 Attempt to access a bank that belonged to a deactivated subsystem

10732 The activity was executing in a subsystem that has been deactivated *aux-info*

10737 Linking System returned an invalid error status

10740 Linking System initial load got wrong number of parameters from Exec

10742 Linking System initial load got wrong parameter format from Exec

10743 Linking System initial load got an unknown function code from Exec

10744 Linking System initial load had an error locating start address (START$)

10745 Linking System initial load could not find or read object module

10746 Link fault occurred in an activity with no Linking System activity-id

This error can be the result of an assembly-language routine starting a new activity. You should only use the high-level multitasking support routines.

10747 Linking System had an error preparing to process an SSDEF$ element

10750 Linking System initial load had an error reading the OM header record

10751 Linking System object module load could not acquire gates for own use

10752 Linking System got wrong number of parameters from Exec on link fault *link-fault-address*

10753 Call to undefined Linking System program-callable interface

This error can be the result of an improper or corrupted link vector bank.

10754 Linking System got wrong parameter format from Exec on link fault *link-fault-address*

10755 Linking System got wrong number of parameters from Exec on R-bit interrupt

10756 Linking System got wrong parameter format from Exec on R-bit interrupt

10757 User program attempted to base a bank belonging to the Linking System

10760 Linking System initial load had an error making the subsystem buffer table

10761 Linking System initial load had an error making the machine buffer table

10762 Linking System had an address overlap when receiving R-bit interrupt

10763 Linking System had an error freeing the file containing the SSDEF$ element

10764 Linking System subsystem load could not acquire gates for own use

10765 Linking System encountered an error on a link fault *link-fault-address*

This can be because of an undefined name. The Linking System prints a message describing the error. See the *Linking System Programming Reference Manual* for a complete list of Linking System messages.

10766 Linking System could not acquire banks for own use

10767 Linking System got bad latent parameter from Exec on link fault *link-fault-address*

10770 Linking System got bad latent parameter from Exec on R-bit interrupt

10771 Linking System had an error preparing to notify PADS of a bank load *link-fault-address*

10772 Linking System could not complete restart process

10773 Linking System detected an abnormal test and set wait period

10776 Linking System was invoked by Exec for an unimplemented function

10795 Linking System error *aux-info*

This is a general error code. The Linking System prints a message describing the error. See the *Linking System Programming Reference Manual* for a complete list of Linking System messages.

10800 Extended mode ER instruction interrupt *aux-info*

An ER instruction was executed in extended mode.

10801 Extended mode SGNL instruction interrupt *aux-info*

10900 Generic stack overflow *aux-info*

10901 Generic stack underflow *aux-info*

10902 Activity-local stack overflow *aux-info*

The activity local stack (ALS) is full. The routines in your program require more automatic storage than is available. Change the limits of the ALS using a static link or modify your program to use less automatic storage.

10903 Activity-local stack underflow *aux-info*

10904 Return-control stack overflow *aux-info*

Your program has subroutine calls nested too deeply. This is typically the result of recursion. Change your program to reduce the depth of subprogram calls.

10905 Return-control stack underflow *aux-info*

11000 ER ABORT$ *aux-info*

11001 ER EABT$ *aux-info*

11002 Real-time storage check *aux-info*

11003 Data corruption storage check *aux-info*

11004 Corrupted bank swap-in *aux-info*

11005 Console II keyin *aux-info*

11006 Remote break key (@@X C) *aux-info*

11007 Error mode *error-type*

11008 Interactivity interrupt *aux-info*

11009 PCT overflow *error-type*

11010 Abnormal program termination *error-type*

11011 Checkpoint/Restart contingency *error-type*

11012 Terminal timeout *aux-info*

11013 ER EXIT$ contingency generated

11014 Bank requesting basic mode contingency registration currently has extended mode contingency

The extended mode contingency must be deregistered before a basic mode contingency registration request can be performed successfully.

11020 User entered an Exec bank and attempted to access a deleted bank

11100 SSM Intercept routine call verification error

11101 Call verification: An internal gate had parameter verification

11102 Call verification: Pointer function had bad return parameter count

11103 Call verification: Reserved fields nonzero

11104 Call verification: Wrong address tree level for parameter list

11105 Call verification: More parameters passed than interface specified

11106 Call verification: Reserved fields in parameter descriptor nonzero

11107 Call verification: Caller is not passing required access in parameter

11108 Call verification: Caller does not have access it is trying to give

11109 Call verification: Target does not have access caller is trying to give

11110 Call verification: Target does not have access for length of parameter

11111 Call verification: Wrong size in the length field of a parameter

11112 Call verification: Type of parameter wrong

11113 Call verification: Secondary param list cannot have sec. param list

11114 Call verification: Parameter failed a boundary requirement

11115 Call verification: Reserved fields on immediate data param nonzero

11116 Call verification: Access fields on immediate data param nonzero

11117 Call verification: Parameter failed address tree level requirement

11118 Call verification: Parameter size field failed the size check

11119 Call verification: Invalid parameter verify list address tree level

11120 Call verification: Parameter size failed max check, or failed size check

11121 Call verification: A virtual address in a parameter was wrong

11122 Call verification: Invalid NULL param, or NULL param not allowed

11123 Call verification: A virtual address points to an undefined AFCB or subsystem

11124 Call verification: A virtual address in the parameter points to an unreferenced AFCB or subsystem

11125 Call verification: Attempt to transfer the caller's program level parameter descriptors onto ALS failed

11150 SSM Intercept routine return verification error

11151 Return verification: More parameters returned than caller expected

11152 Return verification: Sequence numbers out of order

11153 Return verification: Pointer valued function verifying nonpointer result

11154 Return verification: Reserved fields in parameter descriptor nonzero

11155 Return verification: Caller is not passing required access in a parameter

11156 Return verification: Caller does not have access it is trying to give

11157 Return verification: Target does not have access caller is trying to give

11158 Return verification: Target does not have access for length of parameter

11159 Return verification: Wrong size in the length field of a parameter

11160 Return verification: Type of parameter wrong

11161 Return verification: Secondary param list cannot have sec. param list

11162 Return verification: Parameter failed a boundary requirement

11163 Return verification: Reserved fields on immediate data param nonzero

11164 Return verification: Access fields on immediate data param nonzero

11165 Return verification: Parameter failed address tree level requirement

11166 Return verification: Parameter size field failed the size check

11167 Return verification: Invalid parameter verify list address tree level

11168 Return verification: Parameter size failed max check, or failed size check

11169 Return verification: A virtual address in a parameter was wrong

11170 Return verification: Invalid NULL param, or NULL param not allowed

11171 Return verification: A virtual address points to an undefined AFCB or subsystem

11172 Return verification: A virtual address in the parameter points to an unreferenced AFCB or subsystem

11173 Return verification: The ALS pointer (X10) has changed

11200 Illegal contingency environment requested *aux-info*

11260 Program not allowed access to the HVTIP Bank Manager interface

The program is not an HVTIP program or the program is in a state not allowed by the HVTIP Bank Manager interface.

11261 Maximum number of nested HVTIP bank switches exceeded

11262 Invalid HVTIP library number or library is not available

11263 Invalid HVTIP bank number or bank is not available

11264 I/O error attempting to load an HVTIP program bank

11266 Basic mode program attempted to execute an extended mode ICP or an extended mode program attempted to execute a basic mode ICP

HVTIP bank switch between an extended mode bank and a basic mode bank.

11267 Program was denied execute access to the HVTIP library file

11300 Undefined contingency *error-type*

The upper limit of "CONTINGENCY" type error numbers is 12999. The 13000 errors describe conditions returned by the I18NLIB Service Routines. These error numbers are only valid for UCS C.

13002 An invalid version is specified in the packet.

13003 The supplied buffer is too short.

13004 The FROM CCS transliteration table is not available.

13005 The TO CCS transliteration table is not available.

13013 The arguments contain characters outside the domain of the collating sequence.

13014 The conversion descriptor is bad.

13015 Too many parameters have been specified.

13016 Too few parameters have been specified.

13020 An invalid character was found in the environment variable name.

13021 An invalid character was found in the environment variable value.

13022 The Exec-defined area in the PCT to store environment variables and their values is full. The specified name and value cannot be stored.

13023 The specified environment variable is not allowed in TIP.

13024 The specified environment variable name was not found.

13025 The environment variable value is longer than the maximum length allowed in TIP.

13026 The specified locale cannot be found.

13027 An invalid category was specified.

13028 The environment variable name is a system-reserved environment variable.

13030 The table used by the service routine has been corrupted.

13031 The specified CCS cannot be found.

13032 In a call to iconv_open(), the fromcode and tocode arguments are the same.

13033 No more iconv_open() calls can be made, the data bank is full.

13036 The first string passed to the service routine is too long.

13037 The second string passed to the service routine is too long.

13038 An error not defined by I18NLIB was returned by the Exec to the putenv() routine.

13039 An error not defined by I18NLIB was returned by the Exec to the getenv() routine.

13040 CCS SDF/ISO number passed to the cnb2cnm() function is not unique.

13041 An error was encountered while performing an ASTORE read.

13042 An error was encountered while performing an ASTORE write.

13043 An error was encountered while performing a create bank.

13044 A checkpoint restart was encountered after an ASTORE read.

13045 The specified CCS format is invalid.

13050 Invalid locale item passed to nl_langinfo()

13051 Invalid conversion specification used in STRFMON, STRFTIME, or STRPTIME call.

13052 Invalid argument passed to STRFMON.

13053 Fewer arguments than are indicated by the format str were passed to strfmon().

13054 STRPTIME encountered a mismatch in characters between the input & format strings.

13055 The modified conversion specification used in an STRFTIME or STRPTIME format string is not supported by I18NLIB.

13056 Invalid Date/Time value for STRFTIME or STRPTIME call.

13057 Invalid NMNB identifier.

13058 Invalid NMNB identifier.

16000 Floating-point overflow condition

16001 Single precision floating-pt overflow at col: *decimal-value file-name*

The current single precision floating point number is larger than can be represented in a 35-bit field.

16002 Single precision integer overflow at col: *decimal-value file-name*

The current single precision integer is larger than can be represented in a 35-bit field.

16003 Double precision floating-pt overflow at col: *decimal-value file-name*

The current double precision floating point number is larger than can be represented in a 71-bit field. The starting column and file name are passed as parameters.

16004 Double precision integer overflow at col: *decimal-value file-name*

The current double precision integer is larger than can be represented in a 71-bit field. The starting column and file name are passed as parameters.

16100 UCS Ada CONSTRAINT_ERROR exception : Double precision floating point overflow

This message is used for UCS Ada only. It results in a CONSTRAINT_ERROR exception being raised.

16101 UCS Ada CONSTRAINT_ERROR exception : Single precision integer overflow

This message is used for UCS Ada only. It results in a CONSTRAINT_ERROR exception being raised.

16102 UCS Ada CONSTRAINT_ERROR exception : Single precision floating point overflow

This message is used for UCS Ada only. It results in a CONSTRAINT_ERROR exception being raised.

17000 Floating-point underflow condition

17001 Single precision floating-pt underflow at col: *decimal-value file-name*

The current single precision floating point number is smaller than can be represented in a 35-bit field.

17002 Double precision floating-pt underflow at col: *decimal-value file-name*

The current double precision floating point number is larger than can be represented in a 71-bit field. The starting column and file name are passed as parameters.

18000 Zero divide condition

19000 `Error condition`

19001 `Invalid language code:` *`decimal-value`*

An internal table maintained by LSS generated code and the UCS Runtime System library has been corrupted during program execution. This error could occur as a result of faulty logic in the user program, in LSS generated code or in the UCS Runtime System library. The user program should be examined for errors. Programmers should examine their use of pointers and variants for possible errors. A UCF should be submitted if the cause of the problem cannot be determined from this examination of the user program.

19002 `"FILE SYSTEM-"` *`character-string file-name`*

A bad status code was returned by the Universal Data System (UDS) (or PCIOS in early releases, see *PCIOS Administration and Programming Reference Manual* for message description) on an I/O request. The UDS message and file name are passed as parameters.

Note: *The possible message text inserts and corresponding PCIOS error codes are listed at the end of the normal UCS Runtime System error message descriptions.*

19003 `Attempt to open file marked in error` *`file-name`*

An attempt was made to open a file that had previously been marked in error. The file name is passed as a parameter.

19004 `Attempt to access file marked in error` *`file-name`*

An attempt was made to access a file that had previously been marked in error. The file name is passed as a parameter.

19005 `INQUIRE type code is invalid` *`file-name`*

An INQUIRE I/O operation was attempted with an invalid/unimplemented INQUIRE function code.

19006 `Attempt to close a file marked in error` *`file-name`*

An attempt was made to close a file that had previously been marked in error. The file name is passed as a parameter.

19007 `longjmp was called with a buffer that was not initialized by setjmp`

A UCS C program called longjmp() with a jmpbuf buffer that had not been initialized by calling setjmp() with that buffer as a parameter, or the buffer had been destroyed after the call to setjmp().

19009 `UCS Runtime System-RDMS INTERNAL ERROR RDMS request packet error code` *`decimal-value`*

A status returned from RDMS to UCS Runtime System in the RDMS request packet is out of range, or indicates an UCS Runtime System-RDMS interface internal error. The status is passed as a parameter.

19010 `UCS Runtime System-RDMS INTERNAL ERROR Unknown RDMS command` *`decimal-value`*

A call to the UCS Runtime System interface to RDMS used an invalid or unknown RDMS command code.

19011 UCS Runtime System-RDMS [INTOVF] ERROR Invalid *UCS-or-UDS* data *type-or-size field-number*

The extended mode or UDS data item had an invalid type, or the data item size was not within the expected range. The field number of the 'FETCH INTO' is passed as a parameter. The current and remaining fields are ignored.

19012 DDF error return - *data-definition-error*

Data definition error return. If a status number is given, it should be of the form 4000nn where nn is the error status of an 'ER BANK$'. Otherwise, the explanation of the error is printed.

19013 UCS Runtime System-RDMS INTERNAL ERROR - No RDMCA returned by RDMS

Information from the RDMCA was needed by UCS Runtime System to continue processing, but RDMS did not return one.

19014 'setjmp()' mark return control stack failed, status *decimal-value*

The UCS C library routine setjmp() got an error status from the operating system when it called RCSCONTROL to mark the current point on the return stack. This is an internal error, please submit a UCF.

19015 *character-string* " - STATUS=" *octal-value file-name*

A bad status code was returned by the SAR$. SAR$ message, file name, and SAR$ error status code are passed as parameters.

19016 Internal error during edit format processing *file-name*

An internal error has been detected by the edit format processing routines. The file name is passed as a parameter.

19017 Attempt to edit past end of data item

The edit format processing for an internal file or string data item exceeds the length of the item.

19018 Attempted COLUMN edit function beyond end of line *file-name*

The COLUMN edit function specifies a position past the end of the I/O buffer. The file name is passed as a parameter.

19019 Attempt to open two files on same reel

An attempt was made to open a file on a reel or sequence of reels while another file of the same reel or sequence of reels was open.

19020 Termination on Linking System contingency *octal-value*

The Linking System signalled an ERR$ (012) type 011 contingency. The error code is passed as a parameter.

19021 Edit Format Stack overflow *file-name*

Edit Format Stack (EFS) overflow. Most likely because of recursive I/O (a function call in an I/O list that initiates I/O). Each activation of edit processing requires at least one EFS frame, plus an additional frame for each level of parenthesis at the current point in the format string. An iterated item also requires one frame during its processing. The file name is passed as a parameter.

19022 `Internal Error during 'r*c' processing at col: decimal-value file-name`

Internal error during 'r*c' processing - implicit format input. The Ending column (of the 'r*c') and the file name are passed as parameters.

19023 `Parameter array RANK differs from local array RANK`

An array passed to a procedure does not have the same rank as defined in the called procedure.

19024 `A parameter array EXTENT differs from a local array EXTENT`

One of the extents of an array passed to a procedure does not have the same value as defined in the called procedure.

19025 `Parameter array LENGTH differs from local array LENGTH`

An array passed to a procedure does not have the same length as defined in the called procedure.

19026 `An ASSUMED-SIZE array was passed to an ASSUMED-SHAPE array`

An array-size array was passed to a procedure that defined an assumed-shape array.

19027 `List directed I/O to an internal file is illegal`

A UCS FORTRAN internal file write or read with an empty format [FORMAT()] and an I/O list (of data items) is not allowed.

19028 `I/O list not satisfied by last format group file-name`

A formatted write or read had I/O list items remaining when the end of the format was reached, but the format (or the last level 1 format group) did not have any data formats (so the remainder of the list could not be input or output).

19029 `Kanji input is not allowed from the console file-name`

Kanji input is not allowed from the operator's console. The file name is passed as a parameter.

19030 `Extended mode contingency registration failed, STATUS= decimal-value`

An error status was returned from the extended mode Exec contingency registration routine. The error status is passed as a parameter.

19031 `ROLLBACK Paragraph cannot be reached - it is in an invalid routine`

A DMS ROLLBACK error has occurred and the ROLLBACK paragraph is in an invalid routine. Possible cases are: * ROLLBACK occurred in a subprogram and the IMPART is in a different subprogram. * ROLLBACK occurred in the main routine and the IMPART is in a subprogram.

19032 `Conversion from RDMS character to UCS numeric is illegal field-number`

An RDMS character item cannot be placed in an extended mode numeric item. The field number of the 'FETCH INTO' is passed as a parameter.

19033 `RDMS - Begin Thread not done`

Attempt to perform an RDMS command before a BEGIN THREAD was done.

19034 Thread control command error *thread-error*

A call to a thread control command returned an error status. The status is passed as a parameter. Current values for this status are:

1 - Command rollback
2 - (Should never be returned to the user)
3 - Bad application name
4 - Wrong BDR on entry (internal extended mode-UDS interface error)

19035 UCS Runtime System-RDMS ASCII to kanji conversion error (octal *octal-value*) *field-number*

Conversion from ASCII to kanji or kanji to ASCII when delivering data to the program variable resulted in an error status returned from the conversion routine. The error status and the field number of the 'FETCH INTO' are passed as parameters. The status is documented in error message 22032.

19036 Thread control error *thread-error*

Thread control error during an RDMS command execution. Current thread control error statuses are:

```
20000 - Bad LDM ID supplied to UDS.
20010 - LDM ID is currently down.
20020 - Tried to open two LDMS with recoverable files and one of
        them is DMR.
20030 - ICR requested expansion of table beyond initially
        allocated space.
20040 - LDM status being returned to UDS is not valid.
20050 - Short recovery failed
20060 - Tried to do a begin thread when already connected to UDS
20070 - Tried to do an explicit thread control command when in
        implicit mode.
20080 - Bad Thread Control command.
```

If your error status is not listed in the above list, please look in UDS$$SRC*UTIL$.MESSAGES for a detailed description of your error.

19037 'longjmp()' unwind return control stack failed, status *decimal-value*

The UCS C library routine longjmp() got an error status from the operating system when it called RCSCONTROL to mark the current point on the return stack. This occurs if the routine that called setjmp() has returned, or if the jump buffer was modified. This also occurs if an entry on the return stack being unwound causes a change in domain (in which case the status code is 4).

19038 Length of string is less than zero to BITS/SBITS

BITS/SBITS parameter in error.

19039 Initial bit position is less than zero to BITS/SBITS

BITS/SBITS parameter in error.

19040 Length of string is greater than string variable to BITS/SBITS

BITS/SBITS parameter in error.

19041 `Bad parameter descriptor passed to Service Library`

ERTRAN called with a bad parameter.

19042 `Illegal call to UCS C service library`

A UCS C library call was illegal (for example, missing or bad parameter)

19043 `File pointer passed to UCS C service library bad`

File pointer passed to a UCS C library routine was bad (for example, NULL, or not a file, not opened, closed).

19044 `Attempt to get too many arguments with va_arg`

A UCS C program tried to get more arguments than the calling program gave it, using the va_arg macro from <stdarg.h>. This also occurs if va_arg is used after va_end, or before va_start.

19045 `Too few arguments given to printf`

A UCS C program called printf (or related routine, such as fprintf) with fewer arguments than the format string requires. Zero is used for the missing arguments.

19046 `UCS C error code 0`

A UCS C program called strerror with an argument of zero, or perror when 'errno' was zero. This is not an error condition, except in the sense that the program perhaps should not have called strerror or perror.

19047 `Bad pointer argument for printf %s or %n format`

A printf call (or related routine, such as fprintf) attempted to use a NULL or illegal pointer with a %s or %n format.

19048 `Error message` *`decimal-value`* `" does not exist"`

A UCS C program called strerror with a value that does not correspond to any existing RTS message.

Note: *That perror calls strerror.*

19050 `Cannot do a longjmp across subsystems`

longjmp cannot unwind the return control stack because an entry on the return stack is in a different subsystem than the setjmp routine (this includes the routine that called setjmp).

19051 `Cannot do a longjmp across domains`

longjmp cannot unwind the return control stack because an entry on the return stack is in a different domain than the setjmp routine (this includes the routine that called setjmp).

19052 *`decimal-value`* `is an illegal signal number`

A UCS C program passed the printed number as a signal number to signal( ), raise( ), or kill( ). The number is not a legal signal number or cannot be caught (such as signal 9, SIGKILL).

19053 Signal *decimal-value* is not implemented

A UCS C program called signal( ) or kill( ) with a signal number that is not implemented in UCS C.

19054 'kill()' of a nonzero process-id not implemented

A UCS to C program called kill( ) with a nonzero second argument (process-id; PID. UCS C does not support kill with a nonzero PID.

19055 'raise()' could not raise signal *decimal-value* , status *octal-value*

A UCS C program called raise() (or kill), but raise() could not raise the specified signal. This error also occurs if the signal handler modifies the saved register A0 to a negative value.

19056 Unable to find name *character-string* " given on explicit resolve"

The subroutine name given on a UCS COBOL "CALL identifier" statement could not be found (and the CALL statement contained no ON EXCEPTION clause).

19057 Internal link error *decimal-value* " on explicit resolve of" *character-string*

An internal Linking System error occurred when attempting to find the name given on a UCS COBOL "CALL identifier" or "CALL ... ON EXCEPTION" statement. Please submit a UCF.

19058 *lock-type* " of a lock that is released"

A service library routine (LOCKON, LOCKOFF, or LOCKTEST) was called with a lock variable that was previously marked unusable by LOCKREL; or the variable was invalid (either because of modification or improper initialization). The service library routine that detected the error is named in the message.

19059 LOCKREL of a lock that is on

The service library routine LOCKREL was given a lock that was on; no call to LOCKOFF was made before the LOCKREL call.

19060 *event-type* " of an event that is released"

A service library routine (EVPOST, EVCLEAR, EVWAIT, or EVTEST) was called with an event variable that was previously marked unusable by EVREL; or the variable was invalid (either because of modification or improper initialization). The service library routine that detected the error is named in the message.

19061 EVREL of an event for which tasks are waiting

The service library routine EVREL was called with an event variable which had tasks waiting for it to be posted.

19062 Multitasking internal error: LS$INIT_MTSK returned error *decimal-value*

The service library encountered an error attempting to initialize the multitasking environment (on the first call to TSKSTART). This is an internal problem; please submit a UCF.

19063 Multitasking internal error: LS$VA_OM_REL returned error *decimal-value*

The service library encountered an error attempting to start a new task. This is an internal problem; please submit a UCF.

19064 `Multitasking internal error: LS$INS_TSK returned error` *`decimal-value`*

The service library encountered an error attempting to start a new task. This is an internal problem; please submit a UCF.

19065 `Multitasking internal error: LS$DEL_TSK returned error` *`decimal-value`*

The service library encountered an error attempting to terminate a task. This is an internal problem; please submit a UCF.

19066 `Incorrect number of arguments on service library call`

A service library routine does not have the required number of arguments, or an incorrect number of arguments. An argument is missing from the argument list, or there is an extra argument. In the case of UCSGENERAL$ calls, if a parameter is invalid it is considered missing.

19067 `Task terminated by an error condition in another task`

Multitasking only; another task had a fatal error. All tasks are terminated in error.

19068 `User executed ER ERR$`

An execution of ER ERR$ was captured. This can be generated by a call to FERR (UCS FORTRAN) or 'abort()' (UCS C). No other tasks, if any, are terminated.

19069 `User executed ER ABORT$`

An execution of ER ABORT$ was captured. This does not happen under normal operation of extended mode.

19070 `User executed ER EABT$`

An execution of ER EABT$ was captured. This does not happen under normal operation of extended mode.

19071 `Task terminated in error by other task's service library call`

Another task called the service library routine TSK$ERRALL to cause error termination of all tasks in the program (and thus error termination of the program). Another task terminates (in error) with message 19072.

19072 `Call to TSK$ERRALL to cause error termination of all tasks`

The service routine TSK$ERRALL was called to cause error termination of all tasks in the program (and thus error termination of the program).

19073 `Multitasking deadlock condition detected`

All tasks in a multitasking program have become deactivated by calling LOCKON, EVWAIT, TSKWAIT, or TSK$SUSPEND.

19074 `Invalid number of names to register on UCSCLIST$ call`

The ERTRAN UCSCLIST$ call contained a name count that was invalid. Legal counts must be in the range 0 - 63.

19075 `Invalid parameter to MOVWD$ - data not word aligned or invalid precision`

The precision parameter to MOVWD$ was not in the range 1:4, or the source or target area for the move was not word-aligned.

19076 `Invalid task-id` *`octal-value`* `passed to multitasking service library routine`

An invalid task-id was passed to TSKTEST, TSKWAIT, or TSK$RESUME. The only valid task-ids are those returned by TSKSTART (or TSK$ID) or the value zero. The invalid task-id is printed in octal.

19077 `Second argument given on TSKSTART call is not a subroutine`

The second argument given to the service routine TSKSTART must be an external subroutine. The most likely cause of this error is omitting an EXTERNAL statement in the routine that calls TSKSTART.

19078 `Processor call line has too many arguments`

A UCS C program was executed as a processor (@program-name arg, arg, ...) and too many arguments were supplied. Reduce the number of arguments.

19079 `Call to FACSF (ERTRAN) returned status` *`octal-value`*

A call to the ERTRAN subroutine FACSF resulted in a bad status. The status is passed as a parameter.

19080 `I/O via BUFFERIN / BUFFEROUT must start and end on a word boundary`

The second parameter to BUFFERIN or BUFFEROUT does not start on a word boundary or the third parameter does not end on a word boundary.

19081 `An activity cannot pass activity-local parameters to another activity`

Of the arguments to TSKSTART intended for the new activity, at least one is activity-local. This means that the new task error terminates or behaves unpredictably when trying to access/use this data since it is not addressable by the new activity.

19082 *`decimal-value`* `is an illegal function code of UCSBANK$`

This function code of UCSBANK$ is not used. The function code is passed as a parameter.

19083 `ER address for UCSGENERAL$:` *`octal-value`* `is invalid`

The ER instruction parameter (Parameter 7) of UCSBANK$ is:

- at an offset of less than 0200, or
- its address is not in the same bank as the DBANK, or
- its offset is not valid within the bank, or
- the parameter is not word aligned

UCSGENERAL$ cannot work under these conditions.

19084 `I/O error` *`octal-value`* `during getc write-then-read function`

An ER SYMB$ issued from the UCS C function "getc" resulted in an I/O error. The ER SYMB$ was issued in order to do both a write and read from the same line of the terminal.

19085 `Invalid test and set cell address`

This error occurs when the VA of the test and set cell passed as input in the parameter list is nonaddressable at the time of the call.

19086 `Cell passed as input is not a test and set queuing cell`

This error occurs if the cell passed as the parameter to the interface is not a legal test and set queuing cell. This means that the cell is not identified for test and set queuing because S2 of the cell does not contain 057.

19087 `Test and set queuing cell not locked by calling activity (TSQUNLCKACT$)`

This error can result only when the TSQUNLCKACT$ interface is called. It occurs if the cell passed as the parameter to the interface is not locked by the calling activity. This means that the calling activity did not perform a TS instruction on the cell before calling this interface.

19088 `Subsystem deactivation performed on subsystem owning the TSQ cell`

This error can result only when the TSQUNLCKACT$ interface is called. It occurs when the calling activity is queued on a test and set queuing cell contained in a bank owned by a FGSS and a subsystem deactivation is performed on that FGSS. The activity is removed from the queue and given this error status.

19089 `UCS Runtime System-RDMS INTERNAL ERROR RDMCA packet error code` *`decimal-value`*

RDMS returned a status of 10 in the request packet but the error code given in the RDMS Communications Area was invalid.

19090 `RTS-RDMS INTERNAL ERROR RDMS request packet error code` *`decimal-value`*

The RDMS status returned in the request packet is undefined.

19091 `Multitasking error: undefined entry point passed to TSKSTART`

Multitasking: the entry point passed to TSKSTART is undefined.

19092 `Attempt to unlock a data item or structure that has not been locked`

This error occurs when the UNLOCK command in UCS PLUS attempts to unlock a data item or data structure that has not been locked.

19093 `The output buffer passed to service routine OM$XREFINFO is invalid.`

The output buffer passed to OM$XREFINFO must be word aligned and the length must be a multiple of 4 bytes.

19094 `**NTRAN ERROR* UNIT` *`decimal-value`* `IS NOT AVAILABLE FOR NTRAN.`

A reference is made to a unit already in use by normal UCS FORTRAN I/O processing. Suggested Action: Change unit number.

19095 `**NTRAN ERROR* UNIT` *`decimal-value`* `NOT ASSIGNED.`

A reference to an unassigned unit is made with a function other than write function 1 or assign function 19. Suggested Action: If a write function 1 is used as the first reference, a dynamic assign of a mass storage temporary file is created. If a temporary file is not intended, an assignment must be made either by assign function 19 or by @ASG control image.

19096 **NTRAN ERROR* UNIT *decimal-value* HAS IMPROPER DEVICE.

Requested function is not available for the device assigned to this unit. The requested function is ignored. Suggested Action: If you want action for the requested function, a unit with another device assigned must be used.

19097 **NTRAN ERROR* UNIT *decimal-value* ILLEGAL FUNCTION CODE.

The function code requested is out of range or is not supported. Suggested Action: See the documentation for function codes that are legal.

19098 **NTRAN ERROR* UNIT *decimal-value* SYNTAX ERROR FOR FILE NAME.

This message indicates an illegal character in the file name for NTRAN$ function 19. Suggested Action: See the *ECL End Use and Programming Reference Manual* for legal file name characters.

19099 **NTRAN ERROR* UNIT *decimal-value* IS NOT IN CONFIGURED RANGE.

The unit number on a UCS FORTRAN NTRAN I/O statement is too large (greater than UT$MAXUNIT) or is negative. Suggested Action: Change unit number.

19100 **NTRAN ERROR* UNIT *decimal-value* BUFFER FIELD DOES NOT START ON A WORD BOUNDARY.

The buffer array must start on a word boundary. Suggested Action: Adjust the buffer array so it starts on a word boundary.

19101 UCS Runtime System-RDMS conversion error Invalid floating point value.

In representing the floating point value as an integer for the floating to ASCII conversion, the exponent bias is greater than 36 for single precision and greater than 72 for double precision arguments.

19102 Multitasking internal error using CREATE$ACT

This error message is issued when the CREATE$ACT interface in SRVC-TSKSTRT returns LLMS Msg-ID status other than the following: 0, 0300, 0313, 0301, 0302. Status values outside the preceding list indicate errors that should never occur in SRVC-TSKSTRT.

19103 Multitasking does not support execution in TIP memory.

CREATE$ACT is not supported for programs executing in TIP memory.

19104 Multitasking error: insufficient Exec resources to create activity.

Exec resources were not available to grant the CREATE$ACT request.

19105 RDMS to UCS conversion buffer size error

The buffer for floating point to character conversion is too small.

19106 CREATE$ACT returned fatal error status to TSKSTART: STATUS = *octal-value*

A fatal error status was returned to TSKSTART by CREATE$ACT

19107 Multitasking: Exec does not create activity-local banks for TIP ZOOMS

The entry point at which TSKSTART was asked to create a new activity had either an activity-local code VA or an activity-local latent parameter. ZOOMs are loaded by the Exec which will not create activity-local banks for TIP ZOOMs.

19108 `The Linking System could not resolve the called entry point` *`character-string`*

The user has called an entry point name that was specified in a SSDP SCSS EXTERNAL ENTRY POINTS statement. The name, however, could not be found in the called subsystem's $LOCAL search chain when the SSDP- created intercept routine called SS$INIT. Now, that undefined entry point has been called. Make sure that your $LOCAL search chain provides a patch to that entry point's definition.

19109 `The destination parameter's size is insufficient.`

See the appropriate language manual and the service library routine declaration for the call to determine the correct parameter size.

19110 `ER is not supported from UCSGENERAL$.`

The request ER is not supported from the UCSGENERAL$ routine

19111 `Incompatible data structure within URTS$TABLES`

URTS has encountered a data structure within URTS$TABLES which cannot support the called function. Relinking the application with a new level of URTS should resolve this problem.

19200 `Access before elaboration error.`

PROGRAM_ERROR exception raised from Ada tasking code. An attempt was made to access code before elaboration.

19201 `All rendezvous alternatives are closed.`

PROGRAM_ERROR exception raised from Ada tasking code. The rendezvous cannot be completed because all rendezvous alternatives are closed.

19202 `Unable to rendezvous - acceptor completed or abnormal.`

TASKING_ERROR exception raised from Ada tasking code. The rendezvous cannot be completed because the acceptor is either completed or abnormal.

19203 `One or more task activations failed.`

TASKING_ERROR exception raised from Ada tasking code. The activation of one or more tasks failed.

19204 `An error occurred during rendezvous.`

RENDEZVOUS_ERROR exception raised from Ada tasking code. An error occurred during rendezvous.

20000 `Area condition`

21000 `User defined condition`

21001 `Unmatched number of parameters detected on call to entry point:` *`entry-point-name`*

A CALL operation was performed and the number of parameters passed does not match the number of parameters expected by the called subroutine. The entry point name is displayed as part of the message text.

21002 `Reference parameter passed to a UC function with the interface pragma applied has incorrect alignment.`

Reference parameter passed to a UC function with the interface pragma applied has incorrect alignment. The reference parameter passed to a UC function does not match the alignment of the item pointed to by the UC pointer formal parameter.

21003 `The pointer passed to the function has an offset which is invalid according to the function's formal parameter declaration`

UC pointers contain a word and byte offset. A pointer to char or pointer to void has an offset value of 0, 9, 18, or 27. A pointer to short has an offset value of 0 or 18. A pointer to enum has an offset value of 0 or 18 if the enum is allocated as a short int. All other pointer types have an offset value of zero. If a pointer with an invalid offset is used, unexpected results can occur. When the RUNCHECK option is used, UC generates extra code to compare a function's formal parameter pointer offset to the actual incoming parameter's pointer offset value. The following cases are tested:

- The formal parameter is declared as pointer to short or pointer to enum (allocated as short int) and the byte offset of the actual parameter is not equal to 0 or 18.
- The formal parameter is not declared as pointer to char, pointer to void, pointer to short, or pointer to enum (allocated as short int) and the byte offset of the actual parameter is not equal to 0.

21004 `Pointer conversion has produced a pointer with an invalid offset`

UC pointers contain a word and byte offset. A pointer to char or pointer to void has an offset value of 0, 9, 18, or 27. A pointer to short has an offset value of 0 or 18. A pointer to enum has an offset value of 0 or 18 if the enum is allocated as a short int. All other pointer types have an offset value of zero. If a pointer with an invalid offset is used, unexpected results can occur. When the RUNCHECK option is used, UC generates extra code to detect invalid pointer conversions for the following cases:

- When a pointer to char or pointer to void is converted to pointer to short or pointer to enum (allocated as short int), the pointer offset must be 0 or 18.
- When a pointer to char, pointer to void, pointer to short, or pointer to enum (allocated as short int) is converted to a pointer of type associated with a word aligned item, the offset must be 0.

The extra code is generated for the cases listed above when the following operations are performed:

- Explicit pointer cast operations
- Pointer assignments that require implicit pointer cast operations
- Pointers passed as arguments to functions that require implicit pointer cast operations
- Functions that return pointers where the function return must be implicitly cast to a pointer of different type

22000 `Conversion condition`

22001 `Invalid integer representation on input at col: decimal-value file-name`

The current integer being read has an invalid character sequence for an integer beginning at the specified column of the record. The starting column and file name are passed as parameters.

22002 `Integer conversion error at col: decimal-value file-name`

An error was found in converting a character string to an integer format. The starting column and file name are passed as parameters.

22003 `Invalid floating point representation on input at col: decimal-value file-name`

The current floating point number being read has an invalid character sequence for a floating point number beginning at the specified column of the record. The starting column and file name are passed as parameters.

22004 `Packed decimal conversion error at col: decimal-value file-name`

An error was found in converting a character string to packed decimal. The starting column and file name are passed as parameters.

22005 `Radix specified is invalid at col: decimal-value file-name`

The radix specified for a logical input item must be B, H, or O. The starting column and file name are passed as parameters.

22006 `Quote required in logical data input at col: decimal-value file-name`

The logical data input item must be enclosed in quotes. The starting column and file name are passed as parameters.

22007 `Logical digit is out of range for radix at col: decimal-value file-name`

The logical data input character is out of range for the radix specified. The starting column and file name are passed as parameters.

22008 `Quote expected at col: decimal-value file-name`

The input string is expected to be enclosed in quotes. The starting column and file name are passed as parameters.

22009 `Floating point input limit exceeded at col: decimal-value file-name`

22010 `Inappropriate format for I/O list item at col: decimal-value file-name`

The format is inappropriate for its I/O list item. The list item is treated as a real variable. (UCS FORTRAN F, G, E, and D formats). The starting column (in the output or input record) and the file name are passed as parameters.

Note: *The column is not the column in the format string.*

22011 `Illegal format characters - treated as blanks - file-name`

A variable format contains illegal characters.

22012 `Invalid variable format contents - file-name`

A variable format is not valid.

22015 `UCS FORTRAN LOGICAL input invalid at col: decimal-value file-name`

The input text read for a UCS FORTRAN LOGICAL item does not start with the letter T or the letter F, or does not have T or F following the period. The I/O list item is unchanged. The starting column and file name are passed as parameters.

22016 `Missing parenthesis on COMPLEX input at col: decimal-value file-name`

An open or close parenthesis was expected on input of a complex value. The starting column and file name are passed as parameters.

22017 `Missing REAL part of COMPLEX input at col: decimal-value file-name`

The first nonblank character after the open parenthesis '(' was a comma, or there were no nonblank characters before the end-of-file. The starting column and file name are passed as parameters.

22018 `Missing IMAGINARY part of COMPLEX input at col: decimal-value file-name`

The first nonblank character after the comma ',' separating the REAL part from the imaginary part was a comma, or there were no nonblank characters before the end-of-file. The starting column and file name are passed as parameters.

22019 `Delimiter between input items missing at col: decimal-value file-name`

The required space or comma between two input items in an input list is missing. The starting column and file name are passed as parameters.

22020 `List item cannot be assigned 'r*c' value at col: decimal-value file-name`

The datum 'c' (in a repeated input value) cannot be assigned to the I/O list item. For example, 'c' can be character and the I/O list item an INTEGER variable. The ending (note!) column and the file name are passed as parameters.

Note: *If 'c' is a real number and the I/O list item is INTEGER, the error 22001 (Integer format error) is produced.*

22021 `Repeat 'r' in 'r*c' construct 0 or too large at col: decimal-value file-name`

The repetition value 'r' in the 'r*c' repetition construct is zero, or exceeds the largest value that can be represented in 35 bits. The starting column and file name are passed as parameters.

22022 `Literal editing illegal on input - ignored`

The editing of literal code is illegal for formatted input - it is ignored.

22023 `No comma between REAL and IMAGINARY at col: decimal-value file-name`

There was no comma between the REAL and IMAGINARY parts of an input Complex value, during list-directed input. The column at which the error was detected and the file name are passed as parameters.

22024 `Incorrect character in NAMELIST input at col: decimal-value file-name`

This error is caused by a number of things, including: the equal sign is missing; the field separator between constants is missing.

22025 NAMELIST does not contain variable name at col: *decimal-value* *file-name*

The variable name in the namelist input is not contained in the namelist declaration.

22026 NAMELIST HOLLERITH size is zero or too large at col: *decimal-value* *file-name*

A HOLLERITH literal in namelist input had a size of zero or too large. Too large is defined as a size that does not fit in a 35 bit integer.

22027 Subscript out of range, dimension number : *decimal-value* *file-name*

A subscript supplied does not fit in a 1-word integer or is not within the bounds given for that dimension.

22030 Wrong # of subscripts supplied at col: *decimal-value* *file-name*

The number of subscripts supplied to an array for NAMELIST INPUT was not the number that was in the declaration of the array.

22031 Input data does not correspond to type at col: *decimal-value* *file-name*

The variable type in the namelist is inappropriate for the corresponding input data.

22032 ASCII/kanji conversion error return from NCONVRR (*octal-value* " OCTAL)"

The NCONVRR conversion routine detected an error. The error status bits are passed as parameter. The status bit meanings are (left-to-right):

```
Bits 1-3 : Internal NCONVRR error
Bit-4 : No corresponding Business Terminology in JIMFILE
Bit-5 : I/O error on JIMFILE
Bit-6 : JIMFILE could not be assigned
Bits 7-9 : Internal NCONVRR error
Bit-10 : Kanji character will not convert to ASCII
Bit-11 : Kanji string converted for CN exceeds 40 kanji
         characters.  For CA, CE, CH, CJ, CK, CX, RA, and
         RH conversions; the result of the conversion
         exceeds the number of the characters specified
         in OUTCOUNT.
Bit-12 : Illegal character for CX conversion
Bit-13 : Illegal character for CE or CJ conversion
Bit-14 : Character out of range for CE, CJ, or CX conversion
Bit-15 : Illegal # of characters for CE, CJ, or CX conversion
Bit-16 : JIS reserved character for CE, CJ, or CX conversion
Bit-17 : Undefined / semi-voiced sound in CH or CK conversion
Bit-18 : Illegal character for CA, CH, or CK conversion
```

22033 Prefix character of Japanese literal is illegal at col: *decimal-value* *file-name*

Prefix character of Japanese literal is other than CA, CC, CE, CH, CJ, CK, CN, CX (lowercase characters as well).

22034 ASCII/ISO character cannot appear in Japanese literal without prefix or with prefix CC at col: *decimal-value* *file-name*

Japanese literal without prefix or with prefix CC can contain only kanji characters.

22035 Format contains both TL and Japanese formats at col: *decimal-value file-name*

The format contains a TL or Tc format (where c <= current column) as well as a N, NA (with Japanese data), NX or Japanese literal format. Movement to the left in a format containing Japanese formats is not permitted.

Note: *This error is not detected in the case where the left-movement occurs prior to the first Japanese format; also, NA with ASCII data is not considered to be a Japanese format. This error does not occur for internal files or ENCODE/DECODE.*

22036 Japanese Apostrophe editing illegal on input - ignored

Japanese Apostrophe editing is illegal for formatted input - it is ignored.

22037 ASCII/kanji conversion error while processing variable format (*octal-value* " OCTAL)"

The NCONVRR conversion routine detected an error during the process of a variable format. The error status bits are passed as a parameter. The status bit meanings are described in the documentation of error 22032.

22038 Default conversion table used because of conflicting DFP options *file-name*

The default conversion table specified in URTS$TABLES by the tag IBMCNVTBL is used because there were conflicting options set by the DFP OPTIONS keyword. The DFP options were ignored because of the conflict.

22039 Illegal integer value. It must represent a number of days succeeding December 31,1600. (Larger than 0 and smaller than 3067672)

The argument for the UCOB DATE-OF-INTEGER or DAY-OF-INTEGER intrinsic function is illegal.

22040 Integer does not represent a legal date in the Gregorian calendar. It must be in the standard form YYYYMMDD.

The argument for the UCOB INTEGER-OF-DATE intrinsic function is illegal.

22041 Integer does not represent a legal date in the Gregorian calendar. It must be in the Julian date form YYYYDDD.

The argument for the UCOB INTEGER-OF-DAY intrinsic function is illegal.

22042 Integer is not a legal argument for RANDOM. It must be 0 or larger.

The argument for the UCOB RANDOM intrinsic function is illegal.

23000 End-of-file condition

23001 EOLN invalid at end-of-file *file-name*

The EOLN function is invalid when the end-of-file condition is true. The file name is passed as a parameter.

23002 Attempt to read at end-of-file *file-name*

An attempt was made to read next record when the end-of-file condition is true. The file name is passed as a parameter.

23003 `Beginning-of-file detected on read-previous` *`file-name`*

The read-previous operation caused the beginning-of-file to be detected and no BOF transfer address was specified in the record parameter list. The file name is passed as a parameter.

23004 `Attempt to read-previous at beginning-of-file` *`file-name`*

An attempt was made to read previous record when the beginning-of-file condition is true. The file name is passed as a parameter.

23005 `End-of-file detected on read-next` *`file-name`*

The read-next operation caused the end-of-file to be detected and no EOF transfer address was specified in the record parameter list. The file name is passed as a parameter.

23006 `First read-next of an optional file not present` *`file-name`*

The first sequential read was attempted on an optional file that is not present and no EOF transfer address was specified in the record parameter list. The file name is passed as a parameter.

23007 `UCS Ada I/O exception: Attempt EOF on a file not open for input` *`file-name`*

An attempt was made to try to check if the file was at the end of file. If the file was not of mode IN_FILE or INOUT_FILE, then a MODE_ERROR is raised.

23008 `UCS Ada I/O exception: Attempt to read a record greater than the size of the file` *`file-name`*

An attempt was made to read a record that happens to be past the current last record of the direct file. An END_ERROR is raised.

23009 `UCS Ada I/O exception: Attempt to write a record past the maximum size of the file` *`file-name`*

An attempt was made to write a record to a direct file that happens to be greater than the MAX_REC_NUM specified in the FORM string or greater than the default value. A USE_ERROR is raised.

23010 `UCS Ada I/O exception: Attempt to set the column past the END_OF_FILE marker` *`file-name`*

An attempt was made to read the end of file marker when setting the column of the input file to the specified number. An END_ERROR exception is raised.

23011 `UCS Ada I/O exception: Attempt to set the line past the END_OF_FILE marker` *`file-name`*

An attempt was made to read the end of file marker when setting the line of the input file to the specified number. An END_ERROR exception is raised.

23012 `UCS Ada I/O exception: Attempt to GET from an empty string`

An attempt was made to read from an empty string from INTEGER_IO, FLOAT_IO, FIXED_IO, or ENUMERATION_IO. An END_ERROR exception is raised.

24000 `End-of-page condition`

25000 `Finish condition`

25001 User program called ABORT

UCS C Library function ABORT was called by user.

26000 Fixed-point overflow condition

27000 Key condition

27001 Record not found on random delete or rewrite *file-name*

A random delete or rewrite was attempted but the record was not found and no INVKEY transfer address was specified in the record parameter list. The file name is passed as a parameter.

27002 Record not found on random access *file-name*

A start or random read was attempted but the record was not found and no INVKEY transfer address was specified in the record parameter list. The file name is passed as a parameter.

27003 Record already exists on random write or rewrite *file-name*

A random write or rewrite was attempted but the record already exists in the file and no INVKEY Transfer Address was specified in the Record Parameter List. The file name is passed as a parameter.

27004 Start or random read of an opt file not present *file-name*

A random read was attempted of an optional file which is not present and no INVKEY transfer address was specified in the record parameter list. The file name is passed as a parameter.

27005 Write causes sequence error on file *file-name*

The write operation causes records out of sequenced on an indexed file and no INVKEY transfer address is present in the record parameter list. The file name is passed as a parameter.

27006 Random file access attempted beyond physical end-of-file *file-name*

A random write was attempted beyond the end-of-file and no INVKEY transfer address was specified in the record parameter list. The file name is passed as a parameter.

27007 Invalid TEXT file format for FSEEK/FSETPOS operation

The only text file format allowed for 'C' seek operations is a text file which was created with 'C' without translation of newline and tab characters.

27008 Invalid pointer type supplied to FSEEK

The pointer type supplied to 'C' function FSEEK was not SEEK_SET, SEEK_CUR, or SEEK_END

27009 Record number overflows key container on output *file-name*

A direct write was attempted and the relative record number overflowed the key container specified in the program. The file name is passed as a parameter.

28000 Name condition

29000 Record condition

29001 `Write length is less than min or greater than max` *`file-name`*

The actual length of the record to be written is either less than the minimum record size required or greater than the maximum record size. The file name is passed as a parameter.

29002 `Read length is less than min or greater than max` *`file-name`*

The actual length of the record read is either less than the minimum record size required or greater than the maximum record size. The file name is passed as a parameter.

29003 `Output exceeds maximum record size` *`decimal-value`* `, truncated` *`file-name`*

A write operation has attempted to create a record longer than the maximum permissible length for the file. The record is truncated to the maximum length. The file name and maximum length are passed as parameters.

29004 `Input exceeds maximum record length` *`decimal-value file-name`*

An input operation has attempted to read a record longer than the the maximum length for the record.

29005 `Attempt to read past end of re-read buffer`

A re-read operation attempted to implicitly read the next record, or a '/' format was encountered during a reread.

29006 `Output editing overflowed record " "` *`file-name`*

An output editing operation (PUT EDIT) generated a record longer than the maximum record length for the file. The file name is passed as a parameter.

29007 `Unformatted write to direct file exceeds record size` *`file-name`*

An unformatted UCS FORTRAN write creates a record longer than the maximum record length on a direct file.

29008 `Unformatted read requested more data than in the record` *`file-name`*

A UCS FORTRAN unformatted read operation needed more data that was in the input record. The I/O list for the read operation is longer than the I/O list used to write the record, or the data in the I/O list occupies more storage than the list used to write the record.

29009 `Write overflowed record area` *`file-name`*

A write operation attempted to create a record longer than the allowable maximum.

30000 `Size condition`

31000 `Storage condition`

31001 `Edited move caused Activity Local Stack overflow`

An edited move could not be performed because it requires more Activity Local Stack space than is currently available.

31002 `Pointer argument is NULL.`

31003 `Pointer + offset is out of range.`

The resultant pointer created by adding the input pointer value and the input offset is beyond the word address limit of a very large bank (VLB) which is 2**30-1 = 1,073,741,823

32000 `String range condition`

33000 `String size condition`

34000 `Subscript range condition`

35000 `Transmit condition`

35001 `"FILE SYSTEM-"` *`character-string octal-value file-name`*

A bad status code was returned by the Universal Data System (UDS) (or PCIOS in early releases, see *PCIOS Administration and Programming Reference Manual* for message description) on an I/O request. The UDS Message, I/O Status (if returned by UDS), and the file name are passed as parameters.

Note: *The possible message text inserts and corresponding PCIOS error codes are listed at the end of the normal UCS Runtime System error message descriptions.*

35002 `Incorrect access mode` *`file-name`*

The specified access mode conflicts with the file organization requested. The file name is passed as a parameter.

35003 `Attempt to write on file opened for input` *`file-name`*

An attempt was made to write on a file that is open for input. The file name is passed as a parameter.

35004 `Attempt to read file opened for output` *`file-name`*

An attempt was made to read a file that is opened for output. The file name is passed as a parameter.

35005 `Attempt to locate on file not open for output` *`file-name`*

An attempt was made to do a locate operation on a file not opened for output operations. The file name is passed as a parameter.

35006 `File organization conflicts with KEYED attr` *`file-name`*

The KEYED Attribute cannot be used with the File Organization specified. The file name is passed as a parameter.

35007 `Attempt to delete/rewrite on file not open for I/O` *`file-name`*

A file must be open for I/O (Update) operations to perform delete or rewrite functions. The file name is passed as a parameter.

35008 `Sequential delete/rewrite must be preceded by read` *`file-name`*

A delete/rewrite function in sequential access mode must be preceded by a read operation. The file name is passed as a parameter.

35009 `Key specified for a sequential file operation` *`file-name`*

A key was specified on a file operation for a file opened in sequential access mode. The file name is passed as a parameter.

35010 `Next record undefined for sequential access` *`file-name`*

A read next or backspace was attempted but the file position indicator for sequential i/o has become undefined. The file name is passed as a parameter.

35011 `Read/start attempted on a file not open for input or I/O` *`file-name`*

Read or start operations require that the file be opened in the input or I/O mode. The file name is passed as a parameter.

35012 `Write/rewrite requires the FROM option` *`file-name`*

The FROM Option is required (by PL1) to be used with the write or rewrite operations. The file name is passed as a parameter.

35013 `KEY or KEYFROM options cannot be used` *`file-name`*

The sequential I/O operation specified cannot use the KEY or KEYFROM options. The file name is passed as a parameter.

35014 `KEY option required or usage conflicts with file org` *`file-name`*

Either the KEY Option is missing and required (by PL1) or the KEY Option is specified for a sequential I/O operation. The file name is passed as a parameter.

35015 `IGNORE option not allowed for random read` *`file-name`*

Records cannot be ignored on input by random read operations. The file name is passed as a parameter.

35016 `An INTO, SET, or IGNORE option is required` *`file-name`*

An INTO, SET, or IGNORE Option is required (by PL1) for a read next operation with the KEYED attribute specified for the file. The file name is passed as a parameter.

35017 `KEY or KEYTO options cannot be specified` *`file-name`*

The sequential I/O operation specified cannot use the KEY or KEYTO option. The file name is passed as a parameter.

35018 `Attempted EOLN function on a file not open for input` *`file-name`*

The EOLN Function can only be used if the file is open for input. The file name is passed as a parameter.

35019 `Attempted write to a file opened for sequential I/O` *`file-name`*

The REWRITE operation must be used if a record is to be rewritten in sequential I/O mode. The file name is passed as a parameter.

35020 `Formatted access attempted on unformatted file` *`file-name`*

An attempt was made to WRITE/READ a formatted record to/from a file opened for unformatted I/O. The file name is passed as a parameter.

35021 `Unformatted access attempted on formatted file` *`file-name`*

An attempt was made to WRITE/READ an unformatted record to/from a file opened for formatted I/O. The file name is passed as a parameter.

35022 `Unit number` *`file-name`* `" not in configured range"`

The unit number on a UCS FORTRAN I/O statement is too large (>UT$MAXUNIT) or is negative.

35023 `Invalid I/O or output to re-read unit` *`file-name`*

An I/O operation which is not permitted on a re-read unit (unformatted read, name list read, BACKSPACE, REWIND) or an output type operation (including ENDFILE) was attempted on the given re-read unit.

35024 `Illegal character in file name variable`

An illegal character was detected in a file name variable. Allowable characters are: $, -, 0 thru 9, A thru Z, and a thru z.

35025 `Incorrect file organization` *`file-name`*

The specified file organization does not match with the way the file was created. The file name is passed as a parameter.

35026 `Attempted direct I/O to a file opened for sequential I/O` *`file-name`*

A UCS FORTRAN program executed a direct read or write statement (with a REC= clause) that specified a unit that was not opened with ACCESS='DIRECT'.

35027 `Attempted sequential I/O to a file opened for direct I/O` *`file-name`*

A UCS FORTRAN program executed a sequential read or write (without a REC= clause), a REWIND, or an ENDFILE statement that specified a unit that was opened with ACCESS='DIRECT'.

35028 `SFS returned DEADLOCK status for file` *`file-name`*

SFS DEADLOCK error return was detected. The file name is passed as a parameter.

35029 `SFS returned ROLLBACK status for file` *`file-name`*

SFS ROLLBACK error return was detected. The file name is passed as a parameter.

35030 `Formatted I/O on a direct file defined for unformatted records` *`file-name`*

A file is opened with a variable record size format. Formatted I/O with a fixed record size is attempted. It is not correct to do this in UCS FORTRAN. When a file is opened specifying variable record sizes in UCS FORTRAN, the I/O must be done with variable size records.

35031 `ENDFILE is not allowed on a program file` *`file-name`*

An ENDFILE operation is not supported on program files; PCIOS does not support stacked files within elements.

35032 `REWIND operation illegal for symbiont files` *`file-name`*

REWIND operations can only be performed on SDF and ANSI files. It cannot be done on symbiont files such as 'AREAD','APRINT', 'APUNCH','AREADA','APRNTA', or 'APNCHA'.

35034 `BACKSPACE operation illegal for symbiont files` *`file-name`*

The BACKSPACE operation can only be performed on SDF and ANSI files. It cannot be done on symbiont files such as 'AREAD','APRINT', 'APUNCH','AREADA','APRNTA', or 'APNCHA'.

35035 `UCS Ada I/O exception: Attempt to close STD_IN or STD_OUT`

An attempt was made to close either the standard input or the standard output file. A USE_ERROR exception is raised.

35036 `UCS Ada I/O exception: Attempt to RESET STD_IN or STD_OUT`

An attempt was made to reset either the standard input or the standard output file. A MODE_ERROR exception is raised.

35037 `UCS Ada I/O exception: Attempt to set the index of the file past the maximum record number` *`file-name`*

An attempt was made to SET the INDEX of the file past MAX_REC_NUM specified in the FORM parameter. A USE_ERROR exception is raised.

35038 `UCS Ada I/O exception: Attempt to delete STD_IN or STD_OUT`

An attempt was made to delete either the standard input or the standard output file. A USE_ERROR exception is raised.

35039 `UCS Ada I/O exception: Attempt to set a file of mode OUT_FILE to the current default input file` *`file-name`*

An attempt was made to set the current input file to a file of mode OUT_FILE. A MODE_ERROR exception is raised.

35040 `UCS Ada I/O exception: Attempt to set a file of mode IN_FILE to the current default output file` *`file-name`*

An attempt was made to set the current output file to a file of mode IN_FILE. A MODE_ERROR exception is raised.

35041 `UCS Ada I/O exception: Attempt to set the line length on a file not opened for output` *`file-name`*

An attempt was made to set the line length of a file of mode IN_FILE. A MODE_ERROR exception is raised.

35042 `UCS Ada I/O exception: Attempt to set the page length on a file not opened for output` *`file-name`*

An attempt was made to set the page length of a file of mode IN_FILE. A MODE_ERROR exception is raised.

35043 `UCS Ada I/O exception: Attempt to get the line length from a file not opened for output` *`file-name`*

An attempt was made to get the line length of a file of mode IN_FILE. A MODE_ERROR exception is raised.

35044 `UCS Ada I/O exception: Attempt to get the page length from a file not opened for output` *`file-name`*

An attempt was made to get the page length of a file of mode IN_FILE. A MODE_ERROR exception is raised.

35045 `UCS Ada I/O exception: Attempt to do a NEW_LINE on a file not opened for output` *`file-name`*

An attempt was made to output a new line to a file of mode IN_FILE. A MODE_ERROR exception is raised.

35046 `UCS Ada I/O exception: An invalid keyword was detected in the FORM STRING`

An invalid keyword was found during the parsing of the FORM STRING to get the specified attributes. The valid keywords are: BLOCK_SIZE, RECORD_SIZE, APPEND, PREP_FACTOR, LABELS, DISPOSITION, MAX_REC_NUM, SKELETONIZE, SHARED, and SYM_TYPE.

35047 `UCS Ada I/O exception: You cannot have two qualifiers in a row in a UCS Ada FORM STRING`

During the parsing of the FORM STRING two association operators in a row were specified between the keyword and the keyword_value. The form should be the following: keyword => keyword_value where the '=>' is the association operator.

35048 `UCS Ada I/O exception: There must be a qualifier after a keyword in a UCS Ada FORM STRING`

During the parsing of the FORM STRING an association operator was missing. Each attribute specified in the FORM STRING must be of the form: keyword => keyword_value where the '=>' is the association operator.

35049 `UCS Ada I/O exception: The attribute, MAX_REC_NUM, only applies to a DIRECT file`

This attribute specifies the largest record number allowed for a direct access file and is only valid for DIRECT files. This is defined at implementation.

35050 `UCS Ada I/O exception: The attribute, SKELETONIZE, only applies to a DIRECT file`

This attribute is only valid for DIRECT files. This is defined at implementation.

35051 `UCS Ada I/O exception: The attribute, SHARED, only applies to DIRECT files`

DIRECT files are the only files that can be shared in UCS Ada. This is defined at implementation.

35052 `UCS Ada I/O exception: The attribute, SYMB_TYPE, only applies to TEXT files`

This attribute is only valid for TEXT files. This is defined at implementation. The valid values for this attribute are: SDF, READ, ALT_READ, PRINT, ALT_PRINT, PUNCH, and ALT_PUNCH.

35053 `UCS Ada I/O exception: Some type of constraint was violated during the processing of the FORM STRING.`

During the parsing and processing of the FORM STRING, some constraint was violated, possibly one of the values of an attribute.

35054 `UCS Ada I/O exception: Attempt to create either STD_IN or STD_OUT`

An attempt was made to create either the standard input or the standard output file. A USE_ERROR exception is raised.

35055 `UCS Ada I/O exception: An attempt was made to open either STD_IN or STD_OUT`

An attempt was made to open either the standard input or the standard output file. A USE_ERROR exception is raised.

35056 `UCS Ada I/O exception: Attempt to set the line length past maximum record size` *`file-name`*

An attempt was made to set the line length of a TEXT file past either the default line length of 256 characters or past the record number specified in the FORM STRING with the RECORD_SIZE attribute.

35057 `UCS Ada I/O exception: Attempt to skip a line on an output file` *`file-name`*

An attempt was made to skip a line on a file opened for output. One can only skip a line on an input file. A MODE_ERROR exception is raised.

35058 `UCS Ada I/O exception: Attempt to check for END_OF_LINE on an output file` *`file-name`*

An attempt was made to check for END_OF_LINE on a file open for output. One can only do this check on an input file. A MODE_ERROR exception is raised.

35059 `UCS Ada I/O exception: Attempt to output a NEW_PAGE on an input file` *`file-name`*

An attempt was made to put out a new page to a file open for input. One can only perform this operation on an output file. A MODE_ERROR exception is raised.

35060 `UCS Ada I/O exception: Attempt to skip a page on an output file` *`file-name`*

An attempt was made to skip a page on a file open for output. One can only skip a page on an input file. A MODE_ERROR exception is raised.

35061 `UCS Ada I/O exception: Attempt to check for END_OF_PAGE on an output file` *`file-name`*

An attempt was made to check for the END_OF_PAGE on a file opened for output. One can only perform this check on an input file. A MODE_ERROR exception is raised.

35062 `UCS Ada I/O exception: Cannot perform a GET on an output file` *`file-name`*

An attempt was made to GET an item from a file opened for output. One can only perform this operation on an input file. A MODE_ERROR exception is raised.

35063 `UCS Ada I/O exception: Cannot perform a PUT on an input file` *`file-name`*

An attempt was made to PUT an item to a file opened for input. One can only perform this operation on an output file. A MODE_ERROR exception is raised.

35064 `UCS Ada I/O exception: Cannot perform a GET_LINE on an output file` *`file-name`*

An attempt was made to get a line from a file opened for output. One can only perform this operation on an input file. A MODE_ERROR exception is raised.

35065 `UCS Ada I/O exception: Cannot perform a PUT_LINE on an input file` *`file-name`*

An attempt was made to put a line to a file opened for input. One can only perform this operation on an output file. A MODE_ERROR exception is raised.

35066 `UCS Ada I/O exception: Attempt to GET an invalid enumeration type`

An attempt was made to GET an enumeration literal that does not appear to be an enumeration type or is not the same enumeration type that was instantiated with the ENUMERATION_IO package. A DATA_ERROR exception is raised.

35067 `UCS Ada I/O exception: Attempt to GET an invalid integer type`

An attempt was made to GET an integer item that does not appear to be an integer or is not the same type of integer instantiated with the INTEGER_IO package. A DATA_ERROR exception is raised.

35068 `UCS Ada I/O exception: Attempt to set column past the line length` *`file-name`*

An attempt was made to set the column number past the line length specified on a SET_LINE call. A LAYOUT_ERROR exception is raised.

35069 `UCS Ada I/O exception: Attempt to set the line number past the page length` *`file-name`*

An attempt was made to set the line number in the page past the page length specified on a call to PAGE_LENGTH. A LAYOUT_ERROR exception is raised.

35070 `UCS Ada I/O exception: Attempt to PUT an item larger than the line length of file` *`file-name`*

An attempt was made to PUT an item that is larger than the current line length that was specified in a call to LINE_LENGTH. A LAYOUT_ERROR exception is raised.

35071 `UCS Ada I/O exception: Attempt to PUT an enumeration literal into a string smaller than the literal`

An attempt was made to PUT a string that represents an enumeration literal into a string that is smaller than the enumeration literal string. A LAYOUT_ERROR exception is raised.

35072 `UCS Ada I/O exception: Attempt to PUT an integer literal into a string smaller than the literal.`

An attempt was made to PUT a string that represents an integer literal into a string that is smaller than the integer literal string. A LAYOUT_ERROR exception is raised.

35073 `UNIT NUMBER` *`file-name`* `is not available for normal UCS FORTRAN I/O.`

A reference is made to a unit already in use by NTRAN$ I/O processing. Normal UCS FORTRAN I/O statements (such as READ, WRITE, BACKSPACE, ENDFILE, OPEN, CLOSE, and INQUIRE) cannot refer to a unit that is being used with NTRAN$ Service Library routine. Suggested Action: Change unit number.

35074 `WRITE to symbiont file requires formatted data` *file-name*

An attempt was made to write unformatted data to a symbiont file.

35075 `Ada I/O exception: Attempt to GET an invalid real number.`

An attempt was made to GET a real number (FLOAT or FIXED) that does not appear to be a real number or is not the same type that was instantiated with. A DATA_ERROR exception is raised.

35076 `Ada I/O exception: Invalid character found on input of real number.`

An invalid character was found in the input string in an attempt to read a real number from a text file. A DATA_ERROR exception is raised.

35077 `Ada I/O exception: Real number does not fit in image string.`

When doing the equivalent of a 'IMAGE for the real number (FLOAT or FIXED), the number does not fit into the image string. Or the number does not fit into the output string specified. A LAYOUT_ERROR exception is raised.

35078 `Illegal record number specified for a direct access I/O file` *file-name*

A REC=0 clause was present in a READ/WRITE statement for a direct access I/O file.

35079 `Ada I/O exception: Attempt to RESET CURRENT_INPUT or CURRENT_OUTPUT.`

An attempt was made to reset either the current input to output or reset the current output file to input. A MODE_ERROR exception is raised.

35080 `Ada I/O exception: Attempt to OPEN the CURRENT_INPUT file for OUTPUT` *file-name*

An attempt was made to open the current input file for an output file. A MODE_ERROR exception is raised.

35081 `Ada I/O exception: Attempt to OPEN the CURRENT_OUTPUT file for INPUT` *file-name*

An attempt was made to open the current output file for an input file. A MODE_ERROR exception is raised.

36000 `Undefined file condition`

36001 `"FILE SYSTEM-"` *character-string file-name*

A bad status code was returned by the Universal Data System (UDS) on an I/O (DEFINE) request. The UDS message and file name are passed as parameters.

36002 `Attempt to open file closed with lock` *file-name*

An attempt was made to open a file that had previously been closed with lock. The file name is passed as a parameter.

36003 `Attempt to write on unopen file` *file-name*

An attempt was made to write to a file that was not previously opened. The file name is passed as a parameter.

36004 `Attempt to read unopened file` *file-name*

An attempt was made to read a file that was not previously opened. The file name is passed as a parameter.

36005 `Attempt to close unopened file` *`file-name`*

An attempt was made to close a file that has not previously been opened. The file name is passed as a parameter.

36006 `Reset (of INPUT) ignored`

The PASCAL RESET (OPEN FOR INPUT) of the previous implicitly opened input file is ignored. This is a warning. The file name is passed as a parameter.

36007 `Rewrite (of OUTPUT) ignored`

The PASCAL REWRITE (OPEN FOR OUTPUT) of the previous implicitly opened OUTPUT file is ignored. This is a WARNING. The file name is passed as a parameter.

36008 `Attempt to open input device for output` *`file-name`*

An attempt was made to open an input device (for example, card reader) for output. The file name is passed as a parameter.

36009 `Attempt to open output device for input` *`file-name`*

An attempt was made to open an output device (for example, printer) for input. The file name is passed as a parameter.

36010 `Attempt to access unopened file` *`file-name`*

An attempt was made to access a file that is not currently open. The file name is passed as a parameter.

36011 *`format`* `" conflicts with"` *`format  file-name`*

FIP formatting code conflicts with OPL formatting mode.

36012 *`open-mode`* `" conflicts with "` *`open-mode  file-name`*

FIP open mode conflicts with OPL open mode

36013 *`access-mode`* `" conflicts with "` *`access-mode  file-name`*

FIP access mode conflicts with OPL access mode

36014 *`open-disposition`* `" conflicts with "` *`open-disposition  file-name`*

A conflict exists between the open disposition expected in the FIP and the OPL. REWIND conflicts with LEAVE, etc. The file name is generated following the message.

36015 `Attempt to open a file already opened`

An open was attempted on a file already opened.

36016 *`attribute`* `" conflicts with "` *`attribute  file-name`*

A conflict exists between two file attributes. The file name is generated following the message.

36017 *`close-disposition`* `" conflicts with "` *`close-disposition  file-name`*

A conflict exists between the close disposition expected in the FIP and the OPL. REWIND conflicts with LEAVE, etc. The file name is generated following the message.

36018 `"OPEN/CLOSE option " [OPT1] " illegal" file-name`

The file attribute variable on the OPEN/CLOSE statement is not a legal option for UCS FORTRAN.

36019 `Attempt to print record file file-name`

An attempt was made to print a file opened for record I/O. The file name is passed as a parameter.

36020 `Invalid value for [OPT1] file-name`

The value for the specified attribute is invalid (negative or too large). The file name is passed as a parameter.

36021 `File RENAME attempt failed`

The UCS C function RENAME cannot be performed because the OPERATING SYSTEM has no support for this type of function.

36022 `UCS C error code 2: 'No such file'`

This error message is used when strerror is passed a value of 2 and it cannot determine the corresponding UCS Runtime System error number. This happens if the last error, before the call to strerror, was not this type of error (that is, the last error set 'errno' to a value other than 2). The UCS Runtime System error number is normally 19002, with appropriate text.

Note: *That 'perror' calls strerror to get the error message.*

36023 `UCS C error code 5: 'I/O error'`

This error message is used when strerror is passed a value of 5 and it cannot determine the corresponding UCS Runtime System error number. This happens if the last error, before the call to strerror, was not this type of error (that is, the last error set 'errno' to a value other than 5). The UCS Runtime System error number is normally 35001, although 19002 or 36001 are also possible values. Message text varies.

Note: *That 'perror' calls strerror to get the error message.*

36024 `UCS C error code 13: 'Permission denied'`

This error message is used when strerror is passed a value of 13 and it cannot determine the corresponding UCS Runtime System error number. This happens if the last error, before the call to strerror, was not this type of error (that is, the last error set 'errno' to a value other than 13). The UCS Runtime System error number is normally 19002, with appropriate text.

Note: *That 'perror' calls strerror to get the error message.*

36025 `UCS C error code 27: 'File too big'`

This error message is used when strerror is passed a value of 27 and it cannot determine the corresponding UCS Runtime System error number. This happens if the last error, before the call to strerror, was not this type of error (that is, the last error set 'errno' to a value other than 27). The UCS Runtime System error number is normally 19002, with appropriate text.

Note: *That 'perror' calls strerror to get the error message.*

36026 `Attempt to open too many files.`

An attempt has been made to open more files than allowed by UCS C. The file is not opened.

36027 `Attempt to open more files than std. system permits. File count =` *`decimal-value`*

The open request opens more files than permitted by a standard UCS C program. The file open is allowed, but the program cannot be portable beyond this point in execution. The file count is passed as a parameter.

36028 `Attempt to open UCS C file for update in COMPAT mode`

A UCS C file cannot be updated if it is a text file. This message is used when a program running in COMPAT mode attempts to open a file with the modes 'r+', 'w+', or a+'. These files are opened as binary files when COMPAT mode is not used. The fopen is allowed in COMPAT mode if the open mode is changed to 'r+b', 'w+b', or 'a+b'.

36029 `File does not exist` *`file-name`*

A UCS FORTRAN file with status = OLD on the OPEN statement does not exist. The file name is passed as a parameter.

36030 `[OPT1] " conflicts with" [OPT2]` *`file-name`*

A UCS FORTRAN OPEN statement has conflicting option clauses specified. The file name is passed as a parameter.

36031 `FILE clause name is a duplicate of another open unit` *`file-name`*

A UCS FORTRAN OPEN statement has a FILE= clause which specifies a name that is a duplicate of another open unit. The file name is passed as a parameter.

36032 `Invalid file type for element I/O` *`file-name`*

A UCS FORTRAN OPEN statement has a TYPE= clause that specifies the type of file to be opened. Only a file type of SDF can be used for element I/O.

36033 `Direct access not allowed for element I/O` *`file-name`*

A UCS FORTRAN OPEN statement has a ACCESS= clause that specifies the mode of access the file should be opened for. Direct access cannot be performed in element I/O.

36034 `UCS Ada I/O exception: Attempt to CLOSE an unopened file`

An attempt was made to close a file that was not open. A STATUS_ERROR exception is raised.

36035 `UCS Ada I/O exception: Attempt to DELETE an unopened file`

An attempt was made to delete a file that was not open. A STATUS_ERROR exception is raised.

36036 `UCS Ada I/O exception: Attempt to READ from an unopened file`

An attempt was made to read from a file that was not open. A STATUS_ERROR exception is raised.

36037 `UCS Ada I/O exception: Attempt to WRITE to an unopened file`

An attempt was made to write to a file that was not open. A STATUS_ERROR exception is raised.

36038 `UCS Ada I/O exception: Cannot get the MODE of an unopened file`

An attempt was made to get the MODE of a file that was not open. A STATUS_ERROR exception is raised.

36039 `UCS Ada I/O exception: Cannot get the NAME of an unopened file`

An attempt was made to get the NAME of a file that was not open. A STATUS_ERROR exception is raised.

36040 `UCS Ada I/O exception: Cannot get the FORM string of an unopened file`

An attempt was made to get the FORM string of a file that was not open. A STATUS_ERROR exception is raised.

36041 `UCS Ada I/O exception: Cannot SET an INDEX on an unopened file`

An attempt was made to SET the INDEX on a file that was not open. A STATUS_ERROR exception is raised.

36042 `UCS Ada I/O exception: Cannot get the INDEX of an unopened file`

An attempt was made to get the INDEX of a file that was not open. A STATUS_ERROR exception is raised.

36043 `UCS Ada I/O exception: Cannot get the SIZE of an unopened file`

An attempt was made to get the SIZE of a file that was not open. A STATUS_ERROR exception is raised.

36044 `UCS Ada I/O exception: Cannot check for EOF on an unopened file`

An attempt was made to check for the END_OF_FILE on a file that was not open A STATUS_ERROR exception is raised.

36045 `Element does not exist` *`file-name`*

An element I/O element with STATUS='OLD' on the (UCS FORTRAN) OPEN clause does not exist.

36046 `ER PFS$ error for element I/O` *`file-name`*

'OLD' on the (UCS FORTRAN) OPEN clause returned an error other than the element does not exist.

36047 `UCS Ada I/O exception: Cannot RESET an unopened file`

An attempt was made to RESET a file that is not open. A STATUS_ERROR exception is raised.

36048 `UCS Ada I/O exception: Cannot set an unopened file to the current input file`

An attempt was made to set the current input file to an unopened file. A STATUS_ERROR exception is raised.

36049 `UCS Ada I/O exception: Cannot set an unopened file to the current output file`

An attempt was made to set the current output file to an unopened file. A STATUS_ERROR exception is raised.

36050 `UCS Ada I/O exception: Cannot set the LINE_LENGTH of an unopened file`

An attempt was made to set the LINE_LENGTH of an unopened file. A STATUS_ERROR exception is raised.

36051 `UCS Ada I/O exception: Cannot set the PAGE_LENGTH of an unopened file`

An attempt was made to set the PAGE_LENGTH of an unopened file. A STATUS_ERROR exception is raised.

36052 `UCS Ada I/O exception: Cannot get the LINE_LENGTH of an unopened file`

An attempt was made to get the LINE_LENGTH of an unopened file. A STATUS_ERROR exception is raised.

36053 `UCS Ada I/O exception: Cannot get the PAGE_LENGTH of an unopened file`

An attempt was made to get the PAGE_LENGTH of an unopened file. A STATUS_ERROR exception is raised.

36054 `UCS Ada I/O exception: Cannot do a NEW_LINE on an unopened file`

An attempt was made to output a NEW_LINE to an unopened file. A STATUS_ERROR exception is raised.

36055 `UCS Ada I/O exception: Cannot do a SKIP_LINE on an unopened file`

An attempt was made to do a NEW_LINE on an unopened file. A STATUS_ERROR exception is raised.

36056 `UCS Ada I/O exception: Cannot check for END_OF_LINE on an unopened file`

An attempt was made to check an unopened file for the END_OF_LINE. A STATUS_ERROR exception is raised.

36057 `UCS Ada I/O exception: Cannot do a NEW_PAGE on an unopened file`

An attempt was made to output a NEW_PAGE to an unopened file. A STATUS_ERROR exception is raised.

36058 `UCS Ada I/O exception: Cannot do a SKIP_PAGE on an unopened file`

An attempt was made to do a SKIP_PAGE on an unopened file. A STATUS_ERROR exception is raised.

36059 `UCS Ada I/O exception: Cannot check for END_OF_PAGE on an unopened file`

An attempt was made to check an unopened file for END_OF_PAGE. A STATUS_ERROR exception is raised.

36060 `UCS Ada I/O exception: Cannot SET_COL on an unopened file`

An attempt was made to set the column number of an unopened file. A STATUS_ERROR exception is raised.

36061 `UCS Ada I/O exception: Cannot SET_LINE on an unopened file`

An attempt was made to set the line number of an unopened file. A STATUS_ERROR exception is raised.

36062 `UCS Ada I/O exception: Cannot get the COL of an unopened file`

An attempt was made to get the current column number of an unopened file. A STATUS_ERROR exception is raised.

36063 `UCS Ada I/O exception: Cannot get the LINE of an unopened file`

An attempt was made to get the current line number of an unopened file. A STATUS_ERROR exception is raised.

36064 `UCS Ada I/O exception: Cannot get the PAGE of an unopened file`

An attempt was made to get the current page number of an unopened file. A STATUS_ERROR exception is raised.

36065 `UCS Ada I/O exception: Cannot do a GET on an unopened file`

An attempt was made to GET an item from an unopened file. A STATUS_ERROR exception is raised.

36066 `UCS Ada I/O exception: Cannot do a PUT on an unopened file`

An attempt was made to PUT an item to an unopened file. A STATUS_ERROR exception is raised.

36067 `UCS Ada I/O exception: Cannot do a GET_LINE on an unopened file`

An attempt was made to get a line from an unopened file. A STATUS_ERROR exception is raised.

36068 `UCS Ada I/O exception: Cannot do a PUT_LINE on an unopened file`

An attempt was made to put a line to an unopened file. A STATUS_ERROR exception is raised.

36069 `Attempt to open file that is already open with different attributes` *`file-name`*

An attempt was made to open a UCS FORTRAN unit that was already open with one or more open clause attributes that are different from the previous open attributes (or defaults).

36070 `Direct access file requires RECL clause specification` *`file-name`*

An attempt was made to open a direct access UCS FORTRAN unit and the RECL clause was not specified. The RECL clause must be specified when opening UCS FORTRAN direct access files.

37000 `Storage management system condition`

37001 `Attempt to expand URTS$TABLES beyond its maximum upper limit – all basic mode addressable space has been used.`

An attempt was made to allocate a block beyond storage limits. Use the keyword option URTS$TABLES/*n* on the main program compile to circumvent this problem.

37002 URTS$TABLES heap area – storage block free length is invalid

Block size passed to the Storage Management System (SMS) was not the same as the SMS size. Possible cause: random data transfer.

37003 URTS$TABLES heap area – storage block to free is invalid

Validity check on this Storage Management System (SMS) block failed A possible cause is an attempt to free a block that was never allocated.

37004 URTS$TABLES heap area – invalid link in storage free chain

Validity check on a free chain block failed. The probable cause is random data transfer.

37005 URTS$TABLES heap area – attempt to free block at address zero

The address of the block to free was zero. This could be caused by a null or noninitialized pointer.

37006 Extended mode CREATE-BANK failed, STATUS= *decimal-value*

An attempt to create an extended mode data bank failed. The Exec request status is passed as a parameter.

37007 Extended mode DELETE-BANK failed, STATUS= *decimal-value* " L,BDI=" *octal-value*

An attempt to delete an extended mode data bank failed. The Exec request status and the L,BDI value are passed as parameters.

37008 Extended mode INSPECT-BANK failed, STATUS= *decimal-value* " L,BDI=" *octal-value*

An attempt to inspect an extended mode data bank failed. The Exec request status and the L,BDI value are passed as parameters.

37009 Extended mode MODIFY-BANK failed, STATUS= *decimal-value* " L,BDI=" *octal-value*

An attempt to modify an extended mode data bank failed. The Exec request status and the L,BDI value are passed as parameters.

37010 Storage allocation error – memory full

An attempt was made to allocate a block beyond storage limits.

37011 Block to free is not a valid storage block

Validity check on this Storage Management System (SMS) block failed. A possible cause is an attempt to free a block that was never allocated.

37100 FIXED-HEAP - Heap exhausted, current request cannot be granted

The UCS Ada fixed heap is full. The current request for more space cannot be fulfilled.

37101 VARIABLE-HEAP - Bad size field found while checking heap

While checking the UCS Ada variable heap, a bad size field was found.

37102 VARIABLE-HEAP - Incorrect 'top of heap' value found during check

While checking the UCS Ada variable heap, an incorrect top of heap value was found.

37103 `VARIABLE-HEAP - Incorrect global object count found during check`

While checking the UCS Ada variable heap, an incorrect global object count was found.

37104 `VARIABLE-HEAP - 'Size of heap' error found while attempting to initialize`

While attempting to initialize the UCS Ada variable heap, an error in the size of the heap was found.

37105 `VARIABLE-HEAP - Bad size field found while attempting to coalesce`

While attempting to coalesce the UCS Ada variable heap, a bad size field was found.

37106 `VARIABLE-HEAP - Stepped past end of heap while attempting to allocate`

While attempting to allocate space on the UCS Ada variable heap, the end of the heap was passed.

37107 `VARIABLE-HEAP - Heap exhausted, current request cannot be granted`

The UCS Ada variable heap is full. The current request for more space cannot be fulfilled.

37108 `VARIABLE-HEAP - Object to be deallocated not in this heap`

The object to be deallocated cannot be found in the UCS Ada variable heap.

37109 `VARIABLE-HEAP - Attempt to truncate an unallocated object`

Truncation of an unallocated object was attempted in the UCS Ada variable heap.

37110 `VARIABLE-HEAP - Post-truncation size bigger than pre-truncation size`

The object size in the UCS Ada variable heap is larger after truncation than before.

37111 `GLOBAL_HEAP - Heap exhausted, current request cannot be granted.`

The global heap bank is full. The current request for more space cannot be granted.

37112 `GLOBAL_HEAP - Error detected while trying to obtain more heap space.`

Unable to grant current heap request because it would either result in using more heap space than the binder input allows, or it would require expansion into a bank reserved for fixed stack offset ranges.

37113 `SMALL_OBJECTS - Heap list error detected.`

Found end of small objects collection before finding the block that contains the object to be freed.

38000 `Control structure management condition`

38001 `Extended Mode RCS Control (MARK/UNWIND) failed, STATUS=` *`decimal-value`*

38002 `Procedure passed as parameter invalid at call`

A procedure passed as a parameter can only be entered if the block immediately containing that procedure is currently active.

38003 A subroutine on the ISP cannot be called from within the UCS Runtime System

A subroutine name passed as a parameter to the UCS FORTRAN SORT/MERGE package is an ISP subroutine. This is neither practical nor desirable for efficiency reasons.

39000 Data manipulation condition

39001 ENUM_REP - VAL not found in enumeration array.

Unable to return the position associated with the value passed to ENUM1_VAL_TO_POS because a search through the array of the enumeration values did not locate it.

39002 ENUM_REP - The first enumeration has no predecessor.

A predecessor cannot be returned because the first enumeration does not have a predecessor.

39003 ENUM_REP - The last enumeration has no successor.

A successor cannot be returned because the last enumeration does not have a successor.

39004 ATTRIBUTES - The value of the string is larger than INTEGER'LAST.

The string passed in to find INTEGER'VALUE contains a number larger than INTEGER'LAST.

39005 ATTRIBUTES - The value of the string is smaller than INTEGER'FIRST.

The string passed in to find INTEGER'VALUE contains a number smaller than INTEGER'LAST.

39006 ATTRIBUTES - No number found, or an underscore was found in the wrong location in the string.

The string passed in to find the INTEGER'VALUE does not contain a valid number, or the number in the string had the underscore character, '_', at the beginning or at the end of the number.

39007 ATTRIBUTES - Cannot return the value of a NULL string.

The string passed in to find the INTEGER'VALUE was a null string.

39008 ATTRIBUTES - Cannot return the value of a string of blanks.

The string passed in to find the INTEGER'VALUE was full of blanks.

39009 ATTRIBUTES - An invalid number base was encountered.

The base passed in to find the INTEGER'VALUE was not in the valid base range of 2..16.

39010 ATTRIBUTES - Must have a beginning and ending # (:) in the number.

The string passed in to find the INTEGER'VALUE does not contain a pair of delimiters, either '#' or ':'.

39011 `ATTRIBUTES - Must be a number following the 'E' or 'e' for the exponent.`

The string passed in to find the INTEGER'VALUE does not contain a number after the 'E' or 'e' for the exponent.

39012 `ATTRIBUTES - Invalid characters after the number literal.`

The string passed in to find the INTEGER'VALUE contains invalid characters after the number literal.

39013 `ATTRIBUTES - A null string is an invalid enumeration literal.`

The string passed in to find the ENUMERATION'POS was a null string.

39014 `ATTRIBUTES - Enumeration literal not in the enumeration type.`

The string passed in to find the ENUMERATION'POS was not found in the given enumeration type.

40000 `Math condition`

40001 `Argument is unnormalized - CML-function`

A Common Math Library routine that requires a normalized argument has been passed a nonnormalized argument or an argument that is out of range.

40002 `Argument out of range - CML-function`

A Common Math Library routine has been passed an argument that is out of range.

40003 `Function value out of range - CML-function`

A Common Math Library routine has returned a value that is out of range for the receiving variable.

40100 `UCS Ada CONSTRAINT_ERROR exception : Argument is unnormalized`

This message is used for UCS Ada only. It results in a CONSTRAINT_ERROR exception being raised.

40101 `UCS Ada CONSTRAINT_ERROR exception : Integer raised to a negative power`

This message is used for UCS Ada only. It results in a CONSTRAINT_ERROR exception being raised.

40102 `UCS Ada CONSTRAINT_ERROR exception : Floating point zero raised to a negative power`

This message is used for UCS Ada only. It results in a CONSTRAINT_ERROR exception being raised.

40103 `UCS Ada CONSTRAINT_ERROR exception : Argument is out of range`

This message is used for UCS Ada only. It results in an CONSTRAINT_ERROR exception being raised.

40104 `UCS Ada CONSTRAINT_ERROR exception : Value returned is out of range`

This message is used for UCS Ada only. It results in a CONSTRAINT_ERROR exception being raised.

41000 `Sort/Merge condition`

41001 Wrong number of parameters to Sort/Merge call, *decimal-value* " expected "

The UCS Runtime System SORT routine called from the user program had the wrong number of parameters.

41002 UCS FORTRAN Sort feature *FSORT-function* " not implemented "

UCS FORTRAN SORT FEATURE *FSORT-function* is not implemented in this version of the UCS FORTRAN interface.

41003 UCS FORTRAN FUNCTION *FSORT-function* " Cannot be called during a sort or merge"

The function FSSEQ or FSCOPY cannot be called during a sort or merge operation.

41004 UCS FORTRAN FUNCTION *FSORT-function* " Can be called only during a sort"

FSGIVE and FSTAKE can only be called from the input and output subroutines used by FSORT during the execution of a call to FSORT.

41005 UCS FORTRAN FUNCTION FSGIVE was not called by user's input subroutine

The function FSGIVE was not called from within a user input subroutine.

41006 UCS FORTRAN FUNCTION FSTAKE was not called by user's output subroutine

The function FSTAKE was not called from within a user output subroutine.

41007 SORT/MERGE interface Internal error *octal-value*

Internal error 1 = Saved stack pointer invalid on FSGIVE EOF return. Internal error 2 = Invalid call to user data reduction/ data compare routine.

41008 Parameter # *decimal-value* " to Sort/Merge must be a user subroutine"

A parameter which should be a user data reduction or data compare routine was not a subroutine name.

42000 Collating Sequence Comparison Condition

42001 String character value is greater than 0377

The character to be translated has a value greater than the greatest value in the collating sequence (0377).

43001 Normalization error for function *CML-function*

A parameter to a service library math function contained an unnormalized floating point number.

43002 DOMAIN error for MATHLIB function *CML-function*

A parameter to a service library math function was not within the DOMAIN of the function called.

43003 RANGE error for MATHLIB function *CML-function*

A call to a service library math function produced a result which is too large.

43004 Complete loss of significance for function *CML-function*

52001 - CLS301A - The environment variable CBLDATE is set to an incorrect value.

52110 - CLS202A - In the INSPECT statement, the length of literal or identifier in the CONVERTING phrase is not the same as in the TO phrase.

52111 - CLS201A - In the INSPECT statement, the length of identifier must be one character in the REPLACING CHARACTERS BY phrase or the following phrase.

52112 - CLS201A - In the INSPECT statement, the length of literal or identifier in the REPLACING ALL, LEADING, or FIRST is not the same as in the BY phrase.

52113 - CLS201A - The length of an identifier of a TALLYING phrase in an INSPECT statement is 0 or less.

52114 - CLS201A - The length of an identifier of a REPLACING phrase in an INSPECT statement is 0 or less.

52115 - CLS202A - The length of an identifier of a CONVERTING phrase in an INSPECT statement is 0 or less.

52116 - CLS203A - The length of an identifier in a STRING statement is 0 or less.

52117 - CLS204A - The length of an identifier in an UNSTRING statement is 0 or less.

52300 - CLS554-579A - The argument specified for the intrinsic function is incorrect.

52301 - CLS578A - The argument specified for the intrinsic function is incorrect.

52302 - CLS561-569A - An error occurred during the intrinsic function processing.

52303 - CLS571A,572A - There is insufficient memory for the intrinsic processing.

57500 - Conform - A logical error occurred.

57501 - Conform - The object referenced by the identifier to the left of AS does not conform with the AS Modifier.

57502 - Conform - A parameter in the INVOKE statement does not conform with the formal parameter in the invoked method.

An actual parameter in the INVOKE statement does not conform with the corresponding formal parameter in the invoked method, or the formal result does not conform with the actual result, or the usage of Object Property is incorrect.

57503 - Conform - The result of the invoked method must be a handle-type data item that can receive objects of the invoking class.

The actual result of the invoked method must be a handle-type data item that can receive objects of the invoking class if the definition of the formal result of the invoked method is a SELF handle type.

57550 - GC - A logical error occurred.

57551 - GC - The area required to create the object cannot be allocated.

57600 - Msearch - A logical error occurred.

57601 - Msearch - The object used in the invocation or in the reference of Object Property is the NULL object.

57602 - Msearch - The object used in the invocation or in the reference of Object Property is invalid.

57603 - Msearch - Cannot find the method to be invoked, or the Object Property is not in the object.

57604 - Msearch - Only an invariant method can be invoked by an invariant method.

57652 - misc - The area required to create the class table cannot be allocated.

B.2. UCS Runtime System Messages for 19002 and 35001

The following error message text is inserted into UCS Runtime System errors 19002 and 35001, depending on the file-related error detected. The first 3 digits are an internal message code used by the UCS Runtime System and are displayed as an index into the text below. The text for most of these errors corresponds to the PCIOS C2ERR or the PCIOS MSAM errors that are noted. SFS errors are displayed by the individual UDS error codes returned or as selected messages from the text listed here if they correspond to PCIOS SDF Direct or PCIOS MSAM errors.

See the *PCIOS Programming Reference Manual* or the *SFS Programming Reference Manual* for further details about message descriptions.

The following error message text is inserted into UCS Runtime System errors 19002 and 35001, depending on the file-related error detected. The first 3 digits are an internal message code used by the UCS Runtime System and are displayed as an index into the text below. The text for most of these errors corresponds to the PCIOS C2ERR or the PCIOS MSAM errors that are noted. SFS errors are displayed by the individual UDS error codes returned, or as selected messages from the text listed here if they correspond to PCIOS SDF Direct or PCIOS MSAM errors. See the *PCIOS Administration and Programming Reference Manual* or the *SFS Administration and Support Reference Manual* for further details about message descriptions).

0 Unexpected error returned by PCIOS/SFS

An unexpected error code was returned by the PCIOS/SFS interface and is not recognized by UCS Runtime System. This most likely reflects an internal error, please submit a UCF.

1 I/O error returned - Code : *error-value*

Corresponds to PCIOS C2ERR/MSAM 1. The actual EXEC I/O status is displayed as *error-value*.

2 Invalid equipment type assigned

Corresponds to PCIOS C2ERR 2 and PCIOS MSAM 19.

5 Block size specified is too large

Corresponds to PCIOS C2ERR 5.

6 Minimum/maximum record size specified does not match actual file

Corresponds to PCIOS C2ERR 6 and PCIOS MSAM 39.

9 Invalid PCIOS type file structure encountered

Corresponds to PCIOS C2ERR 9.

10 Invalid PCIOS type data block structure encountered

Corresponds to PCIOS C2ERR 10.

11 Attempt to create file with too many index levels

Corresponds to PCIOS MSAM 11.

12 File is not an SDF Direct file

Corresponds to PCIOS C2ERR 12.

13 File attributes not consistent with file label

Corresponds to PCIOS C2ERR 13 and PCIOS MSAM 17 and 27.

15 Insufficient mass storage assigned for operation requested

Corresponds to PCIOS MSAM 15.

16 File is not a valid SDF file

Corresponds to PCIOS C2ERR 16 and 86.

29 Attempt to read after a write operation

Corresponds to PCIOS C2ERR 29.

32 Attempt to read past an end-of-file marker

Corresponds to PCIOS C2ERR 32 and PCIOS MSAM 13.

37 Block size must be a multiple of fixed record size

Corresponds to PCIOS C2ERR 37.

39 Incorrect data block count detected

Corresponds to PCIOS C2ERR 39.

43 Invalid indexed file structure detected

Corresponds to PCIOS MSAM 43.

46 Invalid data in MSAM file - trying to retrieve a page which lies beyond the physical end of the page

Corresponds to PCIOS MSAM 46.

47 Invalid data found in MSAM file - input record > max record size or < the min record size or record not on current page

Corresponds to PCIOS MSAM 47.

48 Attempt to rewrite record is illegal

Corresponds to PCIOS C2ERR 48.

49 Invalid data found in the data block - key in the record does not match the key in the index block pointing to it

Corresponds to PCIOS MSAM 49.

50 Error on ER TLBL$ request

Corresponds to PCIOS C2ERR 50.

51 Invalid data found in the index block - a deleted record in the data block is pointed to by this key

Corresponds to PCIOS MSAM 48.

52 Index structure is corrupted - a path through the index has a length that differs from the number of levels in the IKT

Corresponds to PCIOS MSAM 50.

57 Block size specified is not large enough to do I/O

Corresponds to PCIOS C2ERR 57.

58 Record length must be equal to maximum record length

Corresponds to PCIOS C2ERR 58.

59 Record length specified exceeds maximum

Corresponds to PCIOS C2ERR 59.

60 Backspace operation not allowed on blocked records

Corresponds to PCIOS C2ERR 60.

61 Record length of 0 is invalid

Corresponds to PCIOS C2ERR 61.

62 Buffer offset exceeds block size

Corresponds to PCIOS C2ERR 62.

63 Current block has no record

Corresponds to PCIOS C2ERR 63.

64 Records do not match on validate

Corresponds to PCIOS C2ERR 64.

65 Mass storage file overflowed

Corresponds to PCIOS C2ERR 65.

66 Block size is inconsistent with file label

Corresponds to PCIOS C2ERR 66 and PCIOS MSAM 9.

67 Incorrect variable format contents

Corresponds to PCIOS C2ERR 67.

68 File lacks label parameters for open extend

Corresponds to PCIOS C2ERR 68.

69 File lacks end-of-file sentinel

Corresponds to PCIOS C2ERR 69.

70 Maximum record size exceeded

Corresponds to PCIOS C2ERR 70.

71 System tape labeling (TLS) is not available

Corresponds to PCIOS C2ERR 71.

72 Loss of position detected on tape unit

Corresponds to PCIOS C2ERR 72.

73 Skeletonization inconsistent with file label

Corresponds to PCIOS C2ERR 73.

74 Extended file is smaller than original file

Corresponds to PCIOS C2ERR 74.

75 I/O words written is less than number requested

Corresponds to PCIOS C2ERR 75.

76 Illegal call while processing RERUN

Corresponds to PCIOS C2ERR 76.

77 Block size or record size specified is 0

Corresponds to PCIOS C2ERR 77.

78 Illegal function attempted in output mode

Corresponds to PCIOS C2ERR 78.

79 Illegal return from RERUN processing

Corresponds to PCIOS C2ERR 79.

80 Labels encountered on tape assigned with J option

Corresponds to PCIOS C2ERR 80.

87 File corruption encountered

Corresponds to PCIOS C2ERR 87.

88 The COMMIT record is not allowed

Corresponds to PCIOS C2ERR 88.

90 EXPIRE request not allowed

Corresponds to PCIOS C2ERR 90.

91 Error on ER TSWAP$ request

Corresponds to PCIOS C2ERR 91.

92 `Error on ER TINTL$ request`

Corresponds to PCIOS C2ERR 92.

100 `File cannot hold maximum record number specified`

The direct file being opened cannot hold the maximum record number specified.

101 `SAR$ error condition - Code :` *`error-value`*

A SAR$ error was returned while processing symbiont file I/O. The SAR$ error code is displayed as *`error-value`*.

102 `Scratch file is already assigned`

A file was opened as SCRATCH but the file was already assigned.

103 `Fieldata input file is not allowed`

An attempt was made to open or process a file with Fieldata records.

104 `Invalid IBM hardware translate specified`

105 `File specified is not catalogued or assigned`

Corresponds to PCIOS MSAM 20.

106 `File assignment via ER CSI$ failed`

UCS Runtime System attempted to assign a file and the request failed.

107 `Wrong entry point for file organization specified`

An attempt was made to intermix PCIOS SDF Direct and Sequential file processing.

108 `J option @ASG invalid for tape with standard labels`

109 `Interchange tape assigned without Q mode option`

110 `Read/write key missing or required access inhibited`

Corresponds to PCIOS MSAM 22, 23, and 24.

111 `System tape labeling (TLS) is not available`

112 `Tape file cannot be opened for update mode`

113 `Record size specified is too small`

The record size specified is not large enough to hold all of the characters which the record contains on the actual file.

114 `Standard label value differs from specified label`

UCS Runtime System has checked minor label values and such as standard/omitted header label indicators and found that the program does not match the actual file. This is processed as a warning only.

115 `Open reverse is invalid for file specified`

116 `I/O statement is not supported`

This is an internal error and should be reported via UCF.

117 `Rewrite record size conflicts with file`

Corresponds to PCIOS MSAM 8.

118 `Illegal printer advancing specified`

This is an internal error and should be reported via UCF.

119 `Attempt to write past last logical line on page`

This is an internal error and should be reported via UCF.

120 `Open extend not supported for file specified`

The file being opened is not the type of file which supports file extension operations.

121 `Attempt to open file closed with lock`

Corresponds to PCIOS MSAM 4.

122 `Internal and external file types do not match`

Corresponds to PCIOS MSAM 21.

123 `Not enough indexed buffers specified to do I/O`

Corresponds to PCIOS MSAM 35.

124 `Buffer size too small for indexed I/O`

Corresponds to PCIOS MSAM 26.

125 `Invalid key parameter specified`

Corresponds to PCIOS MSAM 36.

126 `RERUN$ file assignment error - CKPT not taken`

The RERUN$ file cannot be assigned, no RERUN checkpoint can be taken.

127 `Error in taking CKPT dump`

A CSF$ error was returned by the EXEC when performing the @CKPT request.

128 `HDR1 label not found`

Label checking is being performed on an MSAM file and the HDR1 label cannot be found.

129 `Label type does not correspond to file label`

There is a conflict between the user program label specification and the type of labels present on the actual file.

130 `Specified key is inconsistent with file key`

Corresponds to PCIOS MSAM 18.

131 `CSF$ error while closing symbiont file`

The @BRKPT CSF$ request failed when closing a symbiont.

132 `@USE via ER CSI$ failed - invalid external file name`

UCS Runtime System attempted to perform a @USE of internal file name to the external file name and the external name was invalid.

133 `[XX] Error on ER PFS$/PFI$/PFWL$ program file operation`

Error code [XX] was returned by the EXEC when performing one of the listed program file ERs to support an element I/O open or close request.

134 `Program file element not found`

The appropriate element could not be found when opening for input operations.

135 `Program file I/O error or file not assigned`

A program file I/O request failed with an I/O or the program file required was not assigned to the run.

136 `File is not a program file`

Element I/O was requested and the file specified is not a valid program file.

137 `Program file table of contents overflow`

Element I/O attempted to insert an element into a file which has reached the maximum number of elements which can be held in a program file.

138 `Another file is open for output to the same external file`

An attempt was made to open two elements for output in the same physical file.

139 `Invalid device for Element I/O`

Element I/O is only supported for non-tape PCIOS SDF Sequential type files.

140 `Elements cannot be updated or extended`

141 `Element I/O cannot be used with attributed data`

142 `No valid characters in a single transfer to console`

143 `Record size specified is too large`

The record size specified in the user program is larger than the records supported by the actual file and device.

144 `Maximum element size exceeded`

An attempt was made to write an element larger than the system allowed maximum size.

777 `Unexpected error status returned by PCIOS/SFS`

Same as message text 000 described previously.

B.3. HVTIP2/FLSS Error Messages

The following error messages are emitted from the UCS Runtime System HVTIP heapmanager and FLSS nonshared data subsystem loader.

```
*ERROR HVTIP2/FLSS 1: Heap bank not initialized or was corrupted.
```

The heap control block constant was not found during a FREE operation. Detected by CDM$FREE under the name NOT_INIT_STAT. AUX = B10.

```
*ERROR HVTIP2/FLSS 2: Unexpected error in the heap manager.
```

A8 contains an unknown error status. Detected by ERRHANDLE under the name EH_ERRM2. AUX = B10.

```
*ERROR HVTIP2/FLSS 3: Heap bank not initialized or was corrupted.
```

The heap control block constant was not found during an ALLOCATE operation Detected by CDM$ALLOC under the (same) name NOT_INIT_STAT. AUX = B10.

```
*ERROR HVTIP2/FLSS 4: Heap bank could not be expanded.
```

An allocation was attempted but not enough contiguous space was available in the free chain of the requested heap bank. Either the user specified no expansion (SET EXPANSION_SIZE = 0) or the end of the heap bank was reached. Detected by CDM$ALLOC under the name STORAGE_STAT. AUX = B10.

```
*ERROR HVTIP2/FLSS 5: Allocation request size is less than 2 words.
```

An attempt to allocate less than two words was made. An allocation must request at least 2 words because 2 words are required to hold the information in a free chain block. Detected by CDM$ALLOC under the name TOO_SML_STAT. AUX = B10.

```
*ERROR HVTIP2 6: Nested HVTIP Calls limit exceeded.  ReLINK ICP.
```

The default HVTIP nested call limit of 8 has been reached. To get this message, the user must have already reconfigured the Exec for more than 8. To get more than 8 levels from the HVTIP2 heap manager, the ICP must be relinked specifying a larger BOOKKEEPING size. For example, SET BOOKKEEPING_SIZE = n. Detected by NEWHM under the name N_ERRM6. AUX = B10.

```
*ERROR HVTIP2 7: An invalid HVTIP id was called.  The timestamps
mismatch.
```

An HVTIP ZOOM is being called again but the timestamp from the first call doesn't match the current timestamp. The ZOOM was probably updated (copied) while the transaction was in process. Detected by LOADSEGS under the name L_ERRM7. AUX = B10.

```
*ERROR HVTIP2/FLSS 8: CREATE$BANK returned the following statuses:
```

The HVTIP2 heap manager determined that the requested heap must be acquired by a CREATE$BANK Exec request. The primary and bank-specific statuses returned from the Exec request are printed in the error message. Detected by Exec function CREATE$BANK (which is called by ALLOCATESEG) under the heap manager name A_ERRM8. AUX = B10.

```
*ERROR HVTIP2 9: CREATE$BANK returned the wrong LBDI:
```

The LBDI returned by CREATE$BANK did not match the one requested by the HVTIP2 heap manager. Detected by ALLOCATESEG under the name A_ERRM9. AUX = B10.

```
*ERROR HVTIP2/FLSS 10: Illegal relocation information found in ZOOM/OM.
```

While attempting to relocate a segment, the HVTIP heap manager encountered improper relocation information in the template information of the HVTIP ZOOM. Detected by RELOCATESEGS under the name R_ERRMA. AUX = B10.

```
*ERROR HVTIP2 11: An HVTIP2 ZOOM cannot run in the old environment.
```

You are running in the preHVTIP2 environment because the Exec presented the HeapManager with the old format ALS packet. The ZOOM that was loaded by the Exec into program level BDI 4 has been determined to be HVTIP2 format because the field ACW_FORMAT in the ZOOM's Management Info structure has a value greater than 1. To solve the problem, do one of the following: 1) Use an Exec level that supports HVTIP2. 2) Replace the 'HVTIP2' keyword from your LINK source OUTPUT statement with 'HVTIP' and link again. Detected by Heap_Manager under the name O_ERRMB.

```
*WARNING HVTIP2/FLSS 12: Program contains both NAMED and INDEXED common
blocks.
```

The HVTIP2 transaction contains both NAMED common blocks and INDEXED common blocks. The condition was first noted during the loading of the library bank number indicated. This is not an error but because this mixture was not allowed prior to HVTIP2, it may indicate a problem. Note, however, that a particular common block cannot be INDEXED in one ZOOM and not INDEXED in another ZOOM, but there is not enough information at static link time or at run time to detect this erroneous situation. To solve the problem (if one exists), either use the INDEXED COMMON BANKS statement in all or none of the HVTIP ZOOMs involved in the transaction, or turn off the printing of this warning. Detected by LOADSEGS under the name L_ERRMC.

```
*HVTIP2/FLSS 13: CREATE$BANK was called by the heap manager.
```

1) For an HVTIP2 ZOOM, the user has specified an LBDI that isn't included in one of the HVTIP2 VALTAB bank count specifications (for example, DSP). 2) For a FLSS, the user has not specified a usable heap bank for an FLSS primary heap. Turning on the FAST_LOAD_MONITOR (by a SET FAST_LOAD_MONITOR = n LINK command in the ICP LINK source or FLSS LINK source) caused this information to be printed. This is not an error. It is only provided to inform you that a performance improvement should be possible if, for HVTIP, the VALTAB is appropriately modified (for example, increasing the DSP small program-level bank count value to include the indicated L,BDI), or, for nonHVTIP, specify an LBDI (in the SSDEF source of your FLSS) which can be used by your program. Detected by BANKCREATE under the name BC_ERRMD.

```
*HVTIP2/FLSS 14: URTS/Exec versions do not match.
```

Version 2 is not specified in the parameter packet (X10) that the EXEC/RTS$SSLOAD passed to the HVTIP heap manager. Probably a newer URTS version should be installed. Detected by NEWHM under the name N_ERRME.

```
*HVTIP2/FLSS 15: The ADDR of an HVTIP2 segment is in a pooled bank.
```

Only nonpooled banks are intended to be used by HVTIP2 segments (data segs and common block segs). Correcting this problem may involve a combination of the following: 1) changing your SET LOWADR command for the given segment 2) changing a VALTAB bank count 3) changing one of your OUTPUT HVTIP2 pooled bank counts Detected by NEWHM under the name N_ERRMF.

```
*HVTIP2/FLSS 16: A HVTIP FLSS is only allowed in HVTIP2 environment.
```

In an HVTIP transaction, calling an FLSS subsystem is only supported if the environment is HVTIP2, that is, the Exec passes information to the heap manager using the HVTIP2 format. You must use a version of the Exec that supports HVTIP2 in order to use the FLSS feature. Detected by RTS$SSLOAD under the name SL_ERRMG.

```
*HVTIP2/FLSS 17: No heap bank is available for FLSS nonshared data.
```

In an HVTIP2 transaction, if an FLSS is called, and there are no more pooled banks with the specified locality, the ICP LINK specifies that no CREATE$BANK Exec calls are allowed, and the FLSS SSDEF indicates that no sharing of heaps is allowed, the transaction is errored. You should increase the number of pooled banks of the specified locality (ICP LINK OUTPUT HVTIP2 statement), specify ALLOW_CREATE_BANK = TRUE (ICP LINK OUTPUT HVTIP2 statement) or specify HEAP BANKS ARE SHARED WITH OTHER SUBSYSTEMS INCLUDING HVTIP (FLSS SSDEF statement). Detected by RTS$SSLOAD under the name SL_ERRMH.

```
*HVTIP2 18: Improper ZOOM format.  Relink ICP using latest @LINK.
```

B.4. URTS AFCB-Detected Mixedmode Errors

```
*ERROR MM 0: Illegal error number.
```

The error number reported is not a known error number. Detected by ERRHANDLE.

```
*ERROR MM 1: BMSTACK not initialized or was corrupted.
```

The BMSTACK length in the P_ parameter does not agree with the length found in the BMSTACK header. Detected by RTSMMBE, RTSMMEB, RTSMMBX, RTSMMEX under the name EH_BADSTACKLEN.

```
*ERROR MM 2: BMSTACK overflow.
```

A BMSTACK BUY operation resulted in the frame pointer being too small. Detected by RTSMMBE, RTSMMEB, RTSMMEBR, RTSMMBX, RTSMMEX under the name EH_BMSOVRFLO.

```
*ERROR MM 3: BMSTACK underflow.
```

A BMSTACK SELL operation resulted in the frame pointer being greater than the BMSTACK LENGTH. Detected by RTSMMBER, RTSMMEBR under the name EH_BMSUNDFLO.

```
*ERROR MM 4: RTS$NPERESET returned a bad status.
```

A call to RTS$NPERESET returned a bad status (no find B10-) and extended mode has already been seen (start was in extended mode or extended mode has already been called). Someone clobbered B10 or the ALS frame that it should point to, or basic mode was called without the required RTS$NPESAVE function being performed. Detected by RTSMMBE, RTSMMEBR under the name EH_RESETERR.

```
*ERROR MM 5: The specified translation type is not supported.
```

The translation type is illegal for the RTS MM routine being called. Detected by RTSMMBE under the name EH_BADTRANS.

```
*ERROR MM 6: Current ALS frame is not a Parameter List frame.
```

Current frame is not a Parameter List frame. Detected by RTSMMBER, RTSMMEBR under the name EH_NOTPLERR.

```
*ERROR MM 7: Current BMS frame is not a Save Register frame.
```

RTSMMBER got control with current BMSTACK frame not being a Save Register frame. Detected by RTSMMBER under the name EH_NOTSRERR.

```
*ERROR MM 8: BMSTACK does not contain a Save Register frame.
```

Even though basic mode has already been seen, a basic mode Save Register frame cannot be found amongst the three latest BMSTACK frames. Detected by RTS$MMEB under the name EH_NO_SRFRM.

```
*ERROR MM 9: The specified BM_LANGUAGE is not supported.
```

The BM_LANGUAGE value in the BMSTACK header is illegal for this level of RTS. Detected by RTSMMBX, RTSMMEX under the name EH_ILLEG_ENV.

```
*ERROR MM 10: RTS$TPENPE returned a bad status.
```

The call to RTS$TPENPE during the first call to NPE resulted in a bad status. The status can be found in word 040 of BMSTACK. Detected by RTSMMBE under the name EH_TNERR.

```
*ERROR MM 11: Inconsistent number of FTN CHAR arguments.
```

The number of FTN CHARACTER arguments for the indicated FTN entrypoint does not match the number indicated by the FTN typelist and the PARAM TYPE info specified.

> ***Note:*** *That FTN does not provide the typelist if optimization (z-option) or argchk=off is specified, so for those cases PARAM TYPE must be used to indicate which arguments are CHARACTER.*

Detected by RTSMMBE under the name EH_BADCHARCT.

```
 *ERROR MM 12: R$INIT was not called before calling subsystem.
```

A DBank subsystem entrypoint was called, but the subsystem's dbank intercept element address is zero. It should be initialized by the R$INIT call. Detected by RTSMMSC under the name EH_BADINIT.

```
*ERROR MM 13: Invalid VA produced. Try a different DETERMINELBDI.
```

During the formation of EM parameter list VAs in the basic mode to extended mode process, an invalid EM VA was formed. The method used to convert a basic mode offset to an extended mode VA is determined by the value of the DETERMINELBDI keyword clause of the associated ENTRYPOINT command. You should select TRA or TVA if this error occurs. Detected by RTSMMBE (in GETLBDI) under the name EH_BADVA.

B.5. Mixedmodeskl Error List

```
01: PARMCOUNT for entrypoint <entrypoint> is too large.
```

More than 8 continuation lines were needed in the creation of an internal EP SGS for the given entrypoint.

```
02: Unknown command (SGS label): <label>
```

The indicated command may be misspelled.

```
03: An illegal number of <command> commands (<count>) were specified.
```

The indicated command was specified the indicated number of times. Certain commands should only be specified once.

```
04: One of the following keyword clauses should occur 1 (not <count>)
    times in
    <command> number <number>: <keyword list>.
```

You specified one of the indicated keyword clauses the indicated number of times in the indicated command. You should use one and only one of the listed keyword clauses in that command.

```
05: The keyword clause <keyword> occurred <count> times (in <command>
    number
    <number>).
```

The given keyword clause should have appeared exactly once in the given indicated command.

```
06: The keyword clause <keyword> is used <count> times (in <command>
    number
    <number>.  It should occur no more than once.
```

The given keyword clause is allowed to appear zero or one time in the given indicated command.

```
07: Keyword clause <keyword> has <count> values (in <command> number
    <number>.  It should have <count> values.
```

A keyword clause consist of a keyword, followed by the keyword's value or values, separated by commas. You specified an incorrect number of values.

```
08: The keyword clause <keyword> value <value> (in <command> number
    <number>
    is unknown.
```

The following values are known: <value list>

The user must specify one of the indicated values for the indicated keyword clause.

```
09: PARAM <number> subkeyword clause <subkeyword> is specified <count>
    times (for entrypoint <entrypoint>).
```

The given PARAM subkeyword should be specified one or fewer times per parameter. It was specified the indicated number of times in that parameter.

```
10: PARAM <number> subkeyword clause <subkeyword> value <value> for
    entrypoint <entrypoint> is unknown.
```

The indicated ENTRYPOINT command contains a known subkeyword, but the subkeyword's value may be misspelled.

```
11: Keyword <keyword> (in <command> number <number>) is unknown.
```

The indicated command contains a misspelled keyword clause.

```
12: Subkeyword <subkeyword> for PARAM <number> of entrypoint <entrypoint>
    is unknown.
```

The indicated PARAM parameter of the indicated ENTRYPOINT command contains a PARAM subkeyword that is misspelled, or contains a missing comma or value clause.

```
13: Entrypoint <entrypoint> PARMCOUNT is > 63 (<count>).
```

The indicated ENTRYPOINT command contains a PARMCOUNT that is greater than 63. Basic mode products probably cannot handle that many.

```
14: Entrypoint <entrypoint> has a PARAM number > its PARMCOUNT
    (<PARAM number> > <PARMCOUNT value>).
```

One of the PARAM numbers is greater than the indicated entrypoint's PARMCOUNT value.

```
15: Entrypoint <entrypoint> has more PARAMs (<PARAM count>) than
    PARMCOUNT
    (<PARMCOUNT value>).
```

The indicated ENTRYPOINT command contains more PARAM clauses than are indicated by its PARAMCOUNT value. You can have fewer but not more.

```
16: The BMSTACK LBDI/BDR value combination is illegal.
```

One and only one of the mentioned BMSTACK command's keyword clauses must be present. And the BDR clause cannot be present if any EMOM intercepts are to be created.

```
17: No ENTRYPOINT command was found for <entrypoint> which is in
    INTERCEPT_ELT
    command number <number>'s list.
```

The indicated entrypoint name could not be found in any of the ENTRYPOINT commands specified.

Note: *Entrypoint names are case-sensitive.*

```
18: More than one ENTRYPOINT [INTERCEPT_ELT,E,EPFLD,EP] command was
    found.
```

The indicated entrypoint name was found more than once amongst the ENTRYPOINT commands specified. Each entrypoint name must be unique.

```
19: Entrypoint <entrypoint> parameter is not STAGINGWORDSIZE.
```

A translation from extended mode to basic mode will most likely need parameters to be staged from a UCS data bank to a basic mode bank. For this case, parameter size is used to indicate the staging size of the parameter. So, if a parameter size PARAM subkeyword is specified, STAGINGWORDSIZE is the only one allowed.

```
20: Entrypoint <entrypoint> parameter shouldn't be STAGINGWORDSIZE.
```

A translation from basic mode to extended mode should not require data staging since basic mode data is visible from extended mode. The STAGINGWORDSIZE PARAM subkeyword clause is only meaningful for data staging cases. If you specify a parameter size for a basic mode to extended mode translation, you should use one of the following PARAM subkeywords: WORDSIZE, BYTESIZE, BITSIZE.

```
21: Starting_Mode is illegal for the environment specified.
```

A BMHVTIP environment must start in basic mode (BM).

A MMHVTIP environment must start in extended mode (EM).

A NONHVTIP environment must start in basic mode (BM).

```
22: Improper usage of HVTIP type for entrypoint <entrypoint>:
```

a. CALLTYPE indicates HVTIP while target type does not.

   You specified an HVTIP CALLTYPE (HVTIPCALL or HVTIPXFR) for the indicated entrypoint. However, the target type you chose is not available for calling an HVTIP subprogram. Instead, you must use one of the following: HVTARGET_LB, HVTARGET_NLB or TARGET_NAME.

b. Target type indicates HVTIP while CALLTYPE does not.

   You specified one of the following target types: HVTARGET_LB or HVTARGET_NLB. However, the CALLTYPE you chose is not available for calling an HVTIP subprogram. Either change your target type or CALLTYPE so that they match the type of call needed.

c. CALLTYPE or target type indicates HVTIP while ENVIRONMENT does not. You chose a CALLTYPE of HVTIPCALL or HVTIPXFR, or a target type of HVTARGET_LB or HVTARGET_NLB but the ENVIRONMENT type is NONHVTIP. Change one or more of them so that they match the intended ENVIRONMENT.

d. Calling an HVTIP ZOOM in a BMHVTIP or NONHVTIP environment is not supported.

   HVTIP ZOOMs can only be referenced from a MMHVTIP environment. An MMHVTIP environment starts in extended mode. You should create an extended mode HVTIP ZOOM that calls the basic mode absolute ICP. Then, calls to extended mode HVTIP ZOOMs are allowed.

e. Calling an HVTIP ABSOLUTE in a NONHVTIP environment is not supported. HVTIP absolutes can only be referenced from a BMHVTIP or MMHVTIP environment.

```
23: Entrypoint [EP,EP,1,FEPNAME]'s target value doesn't look right.
```

This warning is given when a TARGET_VA or LBDI value has a leading zero but the length is not 13 (or 7) or the next digit (presumably containing the BDI's L or E value) is not 2 (for EMOM) or 4 (for BMRB). The TARGET_VA should be represented in the referencer's execution mode.

```
24: Entrypoint <entrypoint> must be CALLTYPE,NORMAL and a BMRB in order
   to be DBANK,ACOB.
```

The ENTRYPOINT command DBANK,ACOB selection can only be specified for cases in which the target is a basic mode RB (not an EMOM) and is a nonHVTIP subprogram. Make sure that the CALLTYPE, TRANSLATION and DBANK clauses in the indicated ENTRYPOINT and INTERCEPT_ELT commands are compatible.

```
25: Element <element name>'s type cannot be <type> AND <type>.
```

More than one INTERCEPT_ELT specified the same element name and the translations specified are not compatible. You should specify a different element name for one of them if you really want OMs AND RBs.

```
26: Entrypoint <entrypoint> is not in an INTERCEPT_ELT list.
```

The indicated entrypoint was found in an ENTRYPOINT command, but it could not be found in any of the INTERCEPT_ELT commands ENTRYPOINT lists.

Note: *Entrypoint names are case-sensitive. They must match exactly.*

```
27: The element/version name <element> has an illegal length.
```

The indicated element/version name has more than one '/' or the element name or version name is greater than 12 characters.

```
28: An improper BDR value was specified for BM_INITIALBASING.
```

Only BDR1, BDR3, UTILITYI and UTILITYD are allowable values for this keyword clause. If you want to cause ACOB's C$DML to be initially based you should specify BM_INITIALBASING,UTILITYI,C$DML in your EXECUTION command.

```
29: Intercept entrypoint [EP,EP,1,FEPNAME] calls itself. See TARGET_NAME.
```

The specified entrypoint and its TARGET_NAME are the same. If the intercept routine calls itself, the probable result is an infinite loop. You should choose a different name for TARGET_NAME and its HVCALLS/$MNQ counterpart.

```
30: SCSS calltype entrypoint <entrypoint> must use target-type
    FIXED_GATE_LBDI and a BMRB intercept.
```

You chose a CALLTYPE of SCSS_DB or SCSS_NODB or FLSS_DB or FLSS_NODB. Either your target-type is wrong or your intercept type (or both).

The target-type is determined by the presence of one of the following ENTRYPOINT clauses:

TARGET_NAME, HVTARGET_LB, HVTARGET_NLB, TARGET_VA, FIXED_GATE_LBDI. You should use the FIXED_GATE_LBDI clause.

Intercept type is determined by the INTERCEPT_ELT TRANSLATION clause. For example, an ACOB2UCOB translation will produce a BMRB intercept.

B.6. Sort/Merge Messages

1	unknown information string header
2	wrong number of arguments in call
3	impossible start character for key
4	impossible key length
5	invalid key type
6	key sequence must be 'A' or 'D'
7	Impossible word position in key
8	impossible bit position in key
9	character not in sequencing table
10	impossible record size
11	no record size specified
12	impossible link size
13	no key and no user comparison routine
14	sequencing table is too short
17	key extends beyond record
18	a 'BIT' key of type A, D or P does not start on a 6-bit byte boundary
19	the length of a 'BIT' key of type A, D or P is not a multiple of 6 bits
20	erroneous return code on exit from user's comparison routine
21	erroneous record length specified by user
22	link size not given for variable length records and no keys specified
23	there is an illegal character in the information string
24	output destination not in parameter list
25	no call to define a to-be-copied parameter table has been done
26	input source(s) not in argument list
27	impossible bias value given

28 duplicate header in information string

29 size of core scratch area is out of range

30 invalid 'device' in NOCH specification

31 user's data reduction routine is not in argument list

32 scratch file neither mass storage nor tape

33 alphabetic field after equals sign illegal here

34 more than 26 scratch files specified

35 the first member of a SELECT pair is not greater than the previous pairs's second

36 the second member of a SELECT pair is less than the first

37 erroneous return code on exit from user's data reduction routine

38 default sort scratch file(s) could not be assigned

39 key extends beyond link field

40 this column contains a letter instead of a digit

41 this column contains a digit instead of a letter

42 this header must be followed by '='

43 numeric field found when alphabetic expected

44 non-blank characters between '=' and '('

45 invalid field after KEY=

46 alphabetic field found when numeric expected

47 integer field contained a decimal point

48 invalid delimiter

49 numeric field after '=' illegal here

50 end of information string terminated specification

51 too many arguments in call

52 too many pairs in a SELECT list

53 data reduction routine may not be used with variable-length records

54 zero-length record given to sort

55 first word of user-provided sort parameter table is wrong

```
56    keyed files not allowed as input sources or output destinations
57    scratch file not assigned to run
58    file currently not available for access
59    OUTPUT or UPDATE file given as input source
60    key length (in bits) not divisible by 9
61    INPUT file given as output destination
62    key does not start on a character boundary
63    file, element or version name longer than 12 characters
64    mesh size previously given for this device type
65    illegal device type in mesh specification
66    illegal delimiter in mesh specification
67    keys of incompatible key types may not overlap
68    INFO file not assigned or not on sector-addressed mass storage
69    INFO element not in file or file is not a program file
70    INFO element contains a non-ASCII image
71    I/O error when reading INFO element
72    mixture of 'no-seq-nbr' keys and 'seq-nbr' keys in user's table
73    sequencing table contains invalid character
74    sequencing table contains duplicate character
75    Too many scratch files for a sort
76    Too many input sources
77    Too many output destinations
78    Too few input sources
79    Invalid input id for merge
80    File id not that returned by previous mergetake call
81    Merge input from already-closed file
82    Two records from same merge source in startup loop
83    Two mergetake calls without intervening mergegive call
```

84 mergetake called before startup loop complete

85 error exit from sort/merge subroutine

86 error message from storage manager

87 Sort-selected scratch file not available

91 excess right parenthesis

92 illegal character terminated numeric field

93 left parenthesis terminated numeric field

94 equals sign terminated numeric field

95 illegal character terminated alphabetic field

96 left parenthesis terminated alphabetic field

97 period terminated alphabetic field

98 a second decimal point found in a numeric field

99 ***** INTERNAL ERROR - PLEASE SUBMIT A UCF *****

100 EOF already detected on a RETURN

101 EOF already detected on a RELEASE

102 DEFYEAR must be in the range of 0 to 99

Appendix C
Compiler Limits

C.1. Compiler Limits

Table C-1 lists the compiler limits of UCS COBOL.

Table C–1. Compiler Limits of UCS COBOL

Language Elements	UCS COBOL Limits
Number of nested levels	31
Number of nested programs	4095
Source program size	2**30 - 1
External names 30 characters long †	16,364
SPECIAL-NAMES Paragraph	
SWITCH-*n* ... ‡	24
alphabet-name is ...	
literal THRU / ALSO ...	256
ALSO literal ...	256
FILE-CONTROL Paragraph	
SELECT clauses ‡	4,000
Exec files per run unit ‡	4,000

* Picture replication limits are dynamic and are a function of both the run-time environment and the amount of memory needed by the executing program.

** The INSPECT statement can contain up to 200 identifiers, excluding the BEFORE, AFTER, and BY keywords.

*** The UNSTRING statement can contain up to 200 identifiers, excluding the BY, OR, INTO, DELIMITER, IN, COUNT, WITH, POINTER, TALLYING, ON, NOT, and OVERFLOW keywords.

† This restriction is imposed by the Linking System.

‡ This restriction is imposed by the Exec operating system.

▲ This restriction is imposed by the UCS Runtime System.

Table C–1. Compiler Limits of UCS COBOL

Language Elements	UCS COBOL Limits
FILE-CONTROL Paragraph	
ALTERNATE RECORD KEY	
data-name ... ▲	58
RESERVE integer (buffers) ▲	63
Block length ▲	
TAPE	99,996 characters
Interchange	99,999 characters
IBM	32,760 characters
I-O-CONTROL Paragraph	
RERUN ON *system-name*... ‡	
integer RECORDS	26,214
SAME RECORD AREA ▲	511
FOR *file-name*... ▲	511
SAME SORT-MERGE-AREA	
MULTIPLE FILE ... ▲	511
file-name... ▲	511
Data Division Entries	2**30 - 1
Record Size	
Card reader	8188 characters
Card punch	8188 characters
Printer	63 words

* Picture replication limits are dynamic and are a function of both the run-time environment and the amount of memory needed by the executing program.

** The INSPECT statement can contain up to 200 identifiers, excluding the BEFORE, AFTER, and BY keywords.

*** The UNSTRING statement can contain up to 200 identifiers, excluding the BY, OR, INTO, DELIMITER, IN, COUNT, WITH, POINTER, TALLYING, ON, NOT, and OVERFLOW keywords.

† This restriction is imposed by the Linking System.

‡ This restriction is imposed by the Exec operating system.

▲ This restriction is imposed by the UCS Runtime System.

Table C–1. Compiler Limits of UCS COBOL

Language Elements	UCS COBOL Limits
Record Size	
ANS TAPE	9,992 characters
Interchange (Recording Mode F or U)	99,999 characters
Interchange (Recording Mode D)	9,995 characters
IBM	32,760 characters
DISC	261,632 characters
Record Key Size	255 words
Working-Storage Section	
PICTURE *character-string*	30
Numeric item digit positions	18
PICTURE replication ( )	dynamic *
PIC replication (editing)	999
Maximum editing characters	2000
PIC maximum size of field	1,864,135
Report Writer	
COLUMN NUMBER	999
Procedure Division	
Procedure Division statements	2**30 - 1
INSPECT TALLY / REPL *identifier*	200 **
MERGE *file-name*	
ASC / DES KEY ... ▲	64

* Picture replication limits are dynamic and are a function of both the run-time environment and the amount of memory needed by the executing program.

** The INSPECT statement can contain up to 200 identifiers, excluding the BEFORE, AFTER, and BY keywords.

*** The UNSTRING statement can contain up to 200 identifiers, excluding the BY, OR, INTO, DELIMITER, IN, COUNT, WITH, POINTER, TALLYING, ON, NOT, and OVERFLOW keywords.

† This restriction is imposed by the Linking System.

‡ This restriction is imposed by the Exec operating system.

▲ This restriction is imposed by the UCS Runtime System.

Table C–1. Compiler Limits of UCS COBOL

Language Elements	UCS COBOL Limits
Procedure Division	
SORT / *file-name* ▲	
ASC / DES KEY ▲	64
UNSTRING *id*	
DELIMITED *id* / *lit* OR *id* / *lit*	200 ***
DATA DIVISION entries	2**30 - 1
Source program size	2**30 - 1
Maximum block length for DISC	261,632 bits
Maximum number of external names 30 characters long	4,095

* Picture replication limits are dynamic and are a function of both the run-time environment and the amount of memory needed by the executing program.

** The INSPECT statement can contain up to 200 identifiers, excluding the BEFORE, AFTER, and BY keywords.

*** The UNSTRING statement can contain up to 200 identifiers, excluding the BY, OR, INTO, DELIMITER, IN, COUNT, WITH, POINTER, TALLYING, ON, NOT, and OVERFLOW keywords.

† This restriction is imposed by the Linking System.

‡ This restriction is imposed by the Exec operating system.

▲ This restriction is imposed by the UCS Runtime System.

C.2. Additional Limits

The following limits are not statically determinable, but depend on the program environment. If these limits are encountered, you may have to make changes to your program.

C.2.1. Size of an Array or Common Block

The size of a single object, such as an array or common block, is limited by the physical memory available for the system. The maximum size of a single object is the total physical memory available minus the memory used by support software such as the operating and linking systems, as well as the application program. The size of a single object is limited to 16 megawords.

C.2.2. Program Addresses

The amount of actual main storage can vary. However, because program storage is divided into parts, known as banks, you can execute programs whose storage size exceeds the actual memory size. This capability is known as virtual storage.

Virtual address space on the 2200/XPA is about 69 billion words (262,144 banks multiplied by 262,144 words per allocated bank descriptor index (BDI), with up to 64 BDIs per bank). These 69 billion words are partitioned into eight sharing levels. This allows a program to address an area many times larger than the physical main storage.

For 2200/XPA systems, a program can execute if only a subset of the program banks resides in main storage at one time. Since these systems support hardware paging, only a small segment of each bank currently addressed during execution must fit into real memory.

A data bank (D-bank) can be of varying size depending on its use. The entire bank need not be loaded into memory when addressed by the program. Small segments of the bank are loaded and unloaded as needed.

By default, the compiler produces one I-bank for a compilation unit. The size of an I-bank produced from compiling one compilation unit can be up to 262,143 words. If you encounter this limit, split the program into smaller, separate compilation units or use the SEGCODE compiler keyword option. The SEGCODE option creates multiple I-banks with each I-bank having a limit of 262,143 words.

When merging I-banks during the linking process, the merged I-bank can also contain up to 262,143 words (see the description of the SEGCODE processor option in Section 2).

C.2.3. Expression Evaluation

Compile time expression processing utilizes a push-down expression stack. As statement constructs such as identifiers, operators, and parentheses are recognized, a frame size is allocated on the stack. A stack frame is freed when the statement construct(s) can be processed by the compiler. This depends on the hierarchy of operators, use of parentheses, and what is currently being processed. Conditions that exceed the stack limit are diagnosed with a major error. The expression must be simplified in order for it to be compiled error free.

Only one condition can exceed the stack size: nested parentheses. In the simplest form, an expression can have 50 left parentheses before any intermediate text needs to be generated. This number is reduced by whatever expression constructs are currently active on the stack. Also, the items within the nested parentheses require stack space.

Because of recursive processing of expressions by the compiler, internal compiler stack space can run out. Thus text entries can only be nested 155 levels. Ways to simplify the statement include breaking the statement into two or more statements or inserting parentheses into the statement's expressions.

C.2.4. Nested Procedure Calls

The maximum number of nested procedure calls depends on the size of the run-time activity-local stack (ALS) and the stack sizes used by the nested procedures that are called. The default maximum size for the ALS stack is 229,376 words (0700000 octal). The ALS stack is a small OS 2200 Dbank and cannot exceed 0700000 words in size.

When a procedure is entered, a frame size of 16+N words is allocated on the stack, where N is the amount of automatic storage required by the procedure. This includes all program variables and compiler-generated temporary allocations that are allocated automatic storage. When the procedure is exited, a frame is released. If the ALS size is exceeded, an error message specifying ALS stack overflow or underflow is issued. Handle this situation by performing the following steps:

1. Increase the size of the ALS stack. Normally, this is not possible, since by default the ALS bank has the maximum size limit.
2. Modify your procedures so that program variables use static storage rather than automatic, reduce the size of any large Working-Storage buffers, or reduce the number of nested calls. For more information, refer to the discussion of the MIN-ALS option.

Following are the two examples, where ALS stack overflows during program execution.

Example 1

In this example the ALS stack overflow is due to the program running out of ALS stack.

Example # 1, COBOL program gets an ALS overflow:

```
*ERROR(CONTINGENCY) 10902: Activity-local stack overflow(AUX INFO
000000000004)

                     ERROR-ADR: 001006  L-BDI: 400005

*ERROR(URTS): Runtime error termination (ER ERR$/EABT$)
 EXM   TYPE 03 CODE 02 CONT 21 REENT ADR: 001006 BDI: 000005 L,BDI:
400005
                        02 OCT 2008  16:27:41 CDT
 AUX INFO:  000000000004
SYSTEM-UNIQUE ID: 040000000000000000403302
```

Example 2

In this example, the Universal Data System (UDS) is called by the program and ALS does not have enough stack size for the UDS to function. UDS tests the amount of ALS stack remaining and raises an error ticket if the stack size not sufficient.

Example #2, UDS senses insufficient ALS space remaining:

```
Remaining stack space on ALS is not sufficient to complete this request
SYSTEM=TSYS       ISPEC=HF01  STN=000000  - RDMS ERROR 8008  HAS
OCCURRED

RDMS AUX INFO 00000001
PROBLEM RELATES TO THE FASTPATH COMMAND LISTED BELOW
*ERROR 8008: Rollback code 50430 was returned from Thread
            Control. The component was either DCS, RDM, RSM, TCS,
            IRU, or DMR. Consult the Universal Data System (UDS)
            Administration and Support Reference Manual
            (7831 0737) to assist you in locating the UDS
            Control message associated with this rollback code.
```

MIN-ALS

The program variables in the WORKING-STORAGE section are normally allocated in static storage in UCS COBOL. However, if the INITIAL clause is used on the PROGRAM-ID statement, then variables in the WORKING-STORAGE section are instead allocated on the Activity Local Stack (ALS). (Exception: Those 01 variables having the EXTERNAL clause are made into named COMMON blocks, which are always allocated in static storage.) The ALS stack is a small bank, with a maximum size of only 229,376 words (0700000 octal). If you have large buffers in the WORKING-STORAGE section that are causing ALS overflows (the ALS might run out of space due to nested calls), you can reduce the amount of ALS used in your COBOL programs by using the MIN-ALS keyword option on your UCOB compilations. This keyword option causes the compiler to allocate those local WORKING-STORAGE variables that do not have VALUE clauses on them in static storage instead of allocating them on the ALS. If the large buffers causing the overflow problem have VALUE clauses on them, remove the VALUE clauses and add code at the beginning of your procedures to initialize them.

Note: *The MIN-ALS keyword option has no effect on programs that do not have the INITIAL clause on their PROGRAM-ID statements.*

C.2.5. Number of External Names in an Object Module

The number of external references, external entry points, and common block names is limited by the maximum size of the object module's name table. The maximum size, defined by the linking system, is 262,144 words. All names associated with the object module are in this table. The minimum size for a name is two words, because each name has one word of control information and at least one word to hold the name. Four characters are stored per word. Thus, a 30-character name requires a nine-word entry-one control word and eight words for the name.

Appendix D
Character Code

This appendix lists values for ASCII character codes in Table D-1.

Table D–1. ASCII Character Codes

ASCII				Character Name	Symbol	Card Punches
Ordinal Value	Oct	Hex	Dec			
1	0	0	0	NUL		12-0-9-8-1
2	1	1	1	SOH		12-9-1
3	2	2	2	STX		12-9-2
4	3	3	3	ETX		12-9-3
5	4	4	4	EOT		9-7
6	5	5	5	ENQ		0-9-8-5
7	6	6	6	ACK		0-9-8-6
8	7	7	7	BEL		0-9-8-7
9	10	8	8	BS		11-9-6
10	11	9	9	HT		12-9-5
11	12	A	10	LF		0-9-5
12	13	B	11	VT		12-9-8-3
13	14	C	12	FF		12-9-8-4
14	15	D	13	CR		12-9-8-5
15	16	E	14	SO		12-9-8-6
16	17	F	15	SI		12-9-8-7
17	20	10	16	DLE		12-11-9-8-1
18	21	11	17	DC1		11-9-1

Table D–1. ASCII Character Codes

ASCII				Character Name	Symbol	Card Punches
Ordinal Value	Oct	Hex	Dec			
19	22	12	18	DC2		11-9-2
20	23	13	19	DC3		11-9-3
21	24	14	20	DC4		9-8-4
22	25	15	21	NAK		9-8-5
23	26	16	22	SYN		9-2
24	27	17	23	ETB		0-9-6
25	30	18	24	CAN		11-9-8
26	31	19	25	EM		11-9-8-1
27	32	1A	26	SUB		9-8-7
28	33	1B	27	ESC		0-9-7
29	34	1C	28	FS		11-9-8-4
30	35	1D	29	GS		11-9-8-5
31	36	1E	30	RS		11-9-8-6
32	37	1F	31	US		11-9-8-7
33	40	20	32	SP,Space		
34	41	21	33	Exclamation Point	!	12-8-7
35	42	22	34	Quotation Mark, Dieresis	"	8-7
36	43	23	35	Number Sign, Pound Sign	#	8-3
37	44	24	36	Dollar Sign	$	11-8-3
38	45	25	37	Percent Sign	%	0-8-4
39	46	26	38	Ampersand	&	12
40	47	27	39	Apostrophe, Acute Accent	'	8-5

Table D–1. ASCII Character Codes

ASCII						
Ordinal Value	Oct	Hex	Dec	Character Name	Symbol	Card Punches
41	50	28	40	Opening Parenthesis	(	12-8-5
42	51	29	41	Closing Parenthesis	)	11-8-5
43	52	2A	42	Asterisk	*	11-8-4
44	53	2B	43	Plus Sign	+	12-8-6
45	54	2C	44	Comma, Cedilla	,	0-8-3
46	55	2D	45	Minus Sign, Hyphen	-	11
47	56	2E	46	Period, Decimal Point	.	12-8-3
48	57	2F	47	Slash, Virgule, Solidus	/	0-1
49	60	30	48	0, Zero	0	0
50	61	31	49	1, One	1	1
51	62	32	50	2, Two	2	2
52	63	33	51	3, Three	3	3
53	64	34	52	4, Four	4	4
54	65	35	53	5, Five	5	5
55	66	36	54	6, Six	6	6
56	67	37	55	7, Seven	7	7
57	70	38	56	8, Eight	8	8
58	71	39	57	9, Nine	9	9
59	72	3A	58	Colon	:	8-2
60	73	3B	59	Semicolon	;	11-8-6
61	74	3C	60	Less Than	<	12-8-4
62	75	3D	61	Equal Sign	=	8-6

Table D–1. ASCII Character Codes

ASCII						
Ordinal Value	Oct	Hex	Dec	Character Name	Symbol	Card Punches
63	76	3E	62	Greater Than	>	0-8-6
64	77	3F	63	Question Mark	-	0-8-7
65	100	40	64	Commercial At Symbol	@	8-4
66	101	41	65	Capital A	A	12-1
67	102	42	66	Capital B	B	12-2
68	103	43	67	Capital C	C	12-3
69	104	44	68	Capital D	D	12-4
70	105	45	69	Capital E	E	12-5
71	106	46	70	Capital F	F	12-6
72	107	47	71	Capital G	G	12-7
73	110	48	72	Capital H	H	12-8
74	111	49	73	Capital I	I	12-9
75	112	4A	74	Capital J	J	11-1
76	113	4B	75	Capital K	K	11-2
77	114	4C	76	Capital L	L	11-3
78	115	4D	77	Capital M	M	11-4
79	116	4E	78	Capital N	N	11-5
80	117	4F	79	Capital O	O	11-6
81	120	50	80	Capital P	P	11-7
82	121	51	81	Capital Q	Q	11-8
83	122	52	82	Capital R	R	11-9
84	123	53	83	Capital S	S	O-2
85	124	54	84	Capital T	T	0-3
86	125	55	85	Capital U	U	0-4

Table D–1. ASCII Character Codes

ASCII				Character Name	Symbol	Card Punches
Ordinal Value	Oct	Hex	Dec			
87	126	56	86	Capital V	V	0-5
88	127	57	87	Capital W	W	0-6
89	130	58	88	Capital X	X	0-7
90	131	59	89	Capital Y	Y	0-8
91	132	5A	90	Capital Z	Z	0-9
92	133	5B	91	Opening Bracket	[	12-8-2
93	134	5C	92	Reverse Slash	\	0-8-2
94	135	5D	93	Closing Bracket	]	11-8-2
95	136	5E	94	Circumflex	^	11-8-7
96	137	5F	95	Underline	_	0-8-5
97	140	60	96	Grave Accent	‘	8-1
98	141	61	97	Small A	a	12-0-1
99	142	62	98	Small B	b	12-0-2
100	143	63	99	Small C	c	12-0-3
101	144	64	100	Small D	d	12-0-4
102	145	65	101	Small E	e	12-0-5
103	146	66	102	Small F	f	12-0-6
104	147	67	103	Small G	g	12-0-7
105	150	68	104	Small H	h	12-0-8
106	151	69	105	Small I	i	12-0-9
107	152	6A	106	Small J	j	12-11-1
108	153	6B	107	Small K	k	12-11-2
109	154	6C	108	Small L	l	12-11-3
110	155	6D	109	Small M	m	12-11-4
111	156	6E	110	Small N	n	12-11-5

Table D–1. ASCII Character Codes

ASCII				Character Name	Symbol	Card Punches
Ordinal Value	Oct	Hex	Dec			
112	157	6F	111	Small O	o	12-11-6
113	160	70	112	Small P	p	12-11-7
114	161	71	113	Small Q	q	12-11-8
115	162	72	114	Small R	r	12-11-9
116	163	73	115	Small S	s	11-0-2
117	164	74	116	Small T	t	11-0-3
118	165	75	117	Small U	u	11-0-4
119	166	76	118	Small V	v	11-0-5
120	167	77	119	Small W	w	11-0-6
121	170	78	120	Small X	x	11-0-7
122	171	79	121	Small Y	y	11-0-8
123	172	7A	122	Small Z	z	11-0-9
124	173	7B	123	Opening Brace	{	12-0
125	174	7C	124	Vertical Line	|	12-11
126	175	7D	125	Closing Brace	}	11-0
127	176	7E	126	Overline, Tilde	~	11-0-1
128	177	7F	127	DEL,Delete		12-9-7
129	200	80	128			11-0-9-8-1
130	201	81	129			0-9-1
131	202	82	130			0-9-2
132	203	83	131			0-9-3
133	204	84	132			0-9-4
134	205	85	133			11-9-5
135	206	86	134			12-9-6
136	207	87	135			11-9-7

Table D–1. ASCII Character Codes

ASCII				Character Name	Symbol	Card Punches
Ordinal Value	Oct	Hex	Dec			
137	210	88	136			0-9-8
138	211	89	137			0-9-8-1
139	212	8A	138			0-9-8-2
140	213	8B	139			0-9-8-3
141	214	8C	140			0-9-8-4
142	215	8D	141			12-9-8-1
143	216	8E	142			12-9-8-2
144	217	8F	143			11-9-8-3
145	220	90	144			12-11-0-9-8-1
146	221	91	145			9-1
147	222	92	146			11-9-8-2
148	223	93	147			9-3
149	224	94	148			9-4
150	225	95	149			9-5
151	226	96	150			9-6
152	227	97	151			12-9-8
153	230	98	152			9-8
154	231	99	153			9-8-1
155	232	9A	154			9-8-2
156	233	9B	155			9-8-3
157	234	9C	156			12-9-4
158	235	9D	157			11-9-4
159	236	9E	158			9-8-6
160	237	9F	159			11-0-9-1
161	240	A0	160			12-0-9-1

Table D–1. ASCII Character Codes

ASCII				Character Name	Symbol	Card Punches
Ordinal Value	Oct	Hex	Dec			
162	241	A1	161			12-0-9-2
163	242	A2	162			12-0-9-3
164	243	A3	163			12-0-9-4
165	244	A4	164			12-0-9-5
166	245	A5	165			12-0-9-6
167	246	A6	166			12-0-9-7
168	247	A7	167			12-0-9-8
169	250	A8	168			12-8-1
170	251	A9	169			12-11-9-1
171	252	AA	170			12-11-9-2
172	253	AB	171			12-11-9-3
173	254	AC	172			12-11-9-4
174	255	AD	173			12-11-9-5
175	256	AE	174			12-11-9-6
176	257	AF	175			12-11-9-7
177	260	B0	176			12-11-9-8
178	261	B1	177			11-8-1
179	262	B2	178			11-0-9-2
180	263	B3	179			11-0-9-3
181	264	B4	180			11-0-9-4
182	265	B5	181			11-0-9-5
183	266	B6	182			11-0-9-6
184	267	B7	183			11-0-9-7
185	270	B8	184			11-0-9-8
186	271	B9	185			0-8-1

Table D–1. ASCII Character Codes

ASCII				Character Name	Symbol	Card Punches
Ordinal Value	Oct	Hex	Dec			
187	272	BA	186			12-11-0
188	273	BB	187			12-11-0-9-1
189	274	BC	188			12-11-0-9-2
190	275	BD	189			12-11-0-9-3
191	276	BE	190			12-11-0-9-4
192	277	BF	191			12-11-0-9-5
193	300	C0	192			12-11-0-9-6
194	301	C1	193			12-11-0-9-7
195	302	C2	194			12-11-0-9-8
196	303	C3	195			12-0-8-1
197	304	C4	196			12-0-8-2
198	305	C5	197			12-0-8-3
199	306	C6	198			12-0-8-4
200	307	C7	199			12-0-8-5
201	310	C8	200			12-0-8-6
202	311	C9	201			12-0-8-7
203	312	CA	202			12-11-8-1
204	313	CB	203			12-11-8-2
205	314	CC	204			12-11-8-3
206	315	CD	205			12-11-8-4
207	316	CE	206			12-11-8-5
208	317	CF	207			12-11-8-6
209	320	D0	208			12-11-8-7
210	321	D1	209			11-0-8-1
211	322	D2	210			11-0-8-2

Table D–1. ASCII Character Codes

ASCII				Character Name	Symbol	Card Punches
Ordinal Value	Oct	Hex	Dec			
212	323	D3	211			11-0-8-3
213	324	D4	212			11-0-8-4
214	325	D5	213			11-0-8-5
215	326	D6	214			11-0-8-6
216	327	D7	215			11-0-8-7
217	330	D8	216			12-11-0-8-1
218	331	D9	217			12-11-0-1
219	332	DA	218			12-11-0-2
220	333	DB	219			12-11-0-3
221	334	DC	220			12-11-0-4
222	335	DD	221			12-11-0-5
223	336	DE	222			12-11-0-6
224	337	DF	223			12-11-0-7
225	340	E0	224			12-11-0-8
226	341	E1	225			12-11-0-9
227	342	E2	226			12-11-0-8-2
228	343	E3	227			12-11-0-8-3
229	344	E4	228			12-11-0-8-4
230	345	E5	229			12-11-0-8-5
231	346	E6	230			12-11-0-8-6
232	347	E7	231			12-11-0-8-7
233	350	E8	232			12-0-9-8-2
234	351	E9	233			12-0-9-8-3
235	352	EA	234			12-0-9-8-4
236	353	EB	235			12-0-9-8-5

Table D–1. ASCII Character Codes

ASCII				Character Name	Symbol	Card Punches
Ordinal Value	Oct	Hex	Dec			
237	354	EC	236			12-0-9-8-6
238	355	ED	237			12-0-9-8-7
239	356	EE	238			12-11-9-8-2
240	357	EF	239			12-11-9-8-3
241	360	F0	240			12-11-9-8-4
242	361	F1	241			12-11-9-8-5
243	362	F2	242			12-11-9-8-6
244	363	F3	243			12-11-9-8-7
245	364	F4	244			11-0-9-8-2
246	365	F5	245			11-0-9-8-3
247	366	F6	246			11-0-9-8-4
248	367	F7	247			11-0-9-8-5
249	370	F8	248			11-0-9-8-6
250	371	F9	249			11-0-9-8-7
251	372	FA	250			12-11-0-9-8-2
252	373	FB	251			12-11-0-9-8-3
253	374	FC	252			12-11-0-9-8-4
254	375	FD	253			12-11-0-9-8-5
255	376	FE	254			12-11-0-9-8-6
256	377	FF	255	EO,Eight Ones		12-11-0-9-8-7

Overpunch Sign Conversions

The overpunched sign condition for +0 and -0 is represented as follows:

	Sign Condition
+0	173_8
	077_8
-0	175_8
	041_8

Appendix E
Label Processing

This appendix discusses all of the file label processing that is performed on disk and tape files. An explanation is given of the COBOL source statements that can alter labeling procedures. Label formats are shown and default label values are indicated. The sequence of label processing events is outlined for each COBOL I-O verb affecting label processing.

E.1. The Label Records Clause

When the STANDARD phrase of the LABEL RECORDS clause is specified for any file, explicit label information is created and maintained on the file. The COBOL user can control the contents or the validation of selected portions of this label information through the use of the VALUE OF clause.

When the OMITTED phrase of the LABEL RECORDS clause is specified for any DISC, TAPE, or INTERCHANGE file, implicit label information is created and maintained on the file. In this case, you cannot use the VALUE OF clause to override any of the system default values used to create or validate this label information. However, if a TAPE or INTERCHANGE file is assigned with the J option, no implicit label processing occurs and the tape must be unlabeled.

E.2. Label Formats

The method by which label information is recorded on a file varies with the type of file.

E.2.1. TAPE and INTERCHANGE Files

The labels recorded on files having TAPE or INTERCHANGE as implementor-name-1 in the ASSIGN clause are maintained by the Tape Labeling System in the operating system. These labels include VOL1, HDR1, HDR2, EOF1, EOF2, EOV1, and EOV2. See the *PCIOS Administration and Programming Reference Manual* for more information.

Volume Header Label (VOL1)

The VOL1 label exists as the first label block on every reel. It is maintained by the system Executive to verify that the proper reel has been mounted. The COBOL program has no access to this label.

Table E–1. Volume Header Label (VOL1) Format

Character Positions	Length	Standard Content	Description
1-3	3	VOL	Label identifier
4	1	1	Label number
5-10	6	Reel number	Six alphanumeric characters identifying the physical reel
11	1	Blank or nonblank	Nonblank indicates restricted access, as the tape is privately owned
12-37	26	Spaces	Not used
38-51	14	Account number or spaces*	Account number of owner if this is a privately owned tape reel
52-79	28	Spaces	Not used
80	1	1	Indicates tape contains standard labels

* The account number is placed in this label only in standard ASCII labels controlled by the Tape Labeling System, and then only when the proper configuration parameters are set. In all other cases, this field is set to spaces.

First File Header Label (HDR1)

The HDR1 label exists as the first label block on every file or file section on a tape reel. It is maintained by the system Executive System to control access to the file.

Table E–2. First File Header Label (HDR1) Format

Character Positions	Length	Standard Default Values	Description	VALUE OF Name
1-3	3	HDR	Label identifier	None
4	1	1	Label number	None
5-21	17	External *qualifier*file-name* †	File identifier	FILE-ID
22-27	6	Reel number of first reel of tape set	Set identifier	None

Table E–2. First File Header Label (HDR1) Format

Character Positions	Length	Standard Default Values	Description	VALUE OF Name
28-31	4	0001	File section number (reel sequence number within file)	None
32-35	4	0001	File sequence number (within a reel)	None
36-39	4	0001	Generation number	None
40-41	2	00	Generation version number	None
42-47	6	Date when file created	Creation date in Julian form preceded by one space	None
48-53	6	Creation date plus period from ASG statement field-4 (or operating system default)	Purge date in Julian form preceded by one space	PURGE-DATE
54	1	Space	Accessibility	FILE-ACCESS
55-60	6	000000	Block count	None
61-73	13	U1100-1	Identifier of system that recorded the file	None
74-80	7	Spaces	Unused	None

† External *qualifier*file-name* consists of a character string that identifies the qualifier and file name used on the @ASG statement that created this file. If the number of character positions of the qualifier and file name (the sum of these two) exceeds 16 positions, the rightmost portion of the qualifier is truncated, but the entire file name is always maintained. For example, if the actual qualifier and file name was: MYPROJECTID*CREATEDFILE, the base level inserts MYPRO*CREATEDFILE into this field. In the default case, the same character string occurs in this field for all files within a file set. In other words, all HDR1 records for all files on a volume or multiple volumes contain the same file identifier in the default case. The file identifier in the default case is left-justified, space-filled.

Second File Header (HDR2)

The HDR2 label follows each HDR1 label. This label contains additional file format information.

Table E–3. Second File Header Label (HDR2) Format

Character Positions	Length	Standard Default Values	Description
1-3	3	HDR	Label identifier
4	1	2	Label number
5	1	D, F, S, or U for INTERCHANGE files or D for TAPE files	D - Variable number of characters in a record
			F - Fixed number of characters in a record
			S - Variable number of characters in a record and the records can span across two or more blocks.
			U - Variable number of characters in a record and only one record in each block.
6-10	5	Block length	Five numeric characters specifying the maximum number of characters per block
11-15	5	Record length	Maximum record length including any control fields
16-50	35	Reserved for operating systems	Reserved for operating systems use. Any alphanumeric characters.
51-52	2	00	Buffer Offset. Two numeric characters specifying the length in characters of any additional field inserted before a data block, for example, block length. This length is included in the block length (character positions 6-10).
53-80	28	Spaces	Must be space characters

End-of-File Label (EOF1)

Except for character positions 1-3, which contain EOF, the EOF1 label has the same format as the HDR1 label. The values are the same, except for the Block count field, which is set to the number of physical data blocks since the previous HDR label group.

Second End-of-File Label (EOF2)

Except for character positions 1-3, which contain EOF, the EOF2 label has the same format as the HDR2 label. This label is located on all tapes following the EOF1 label.

End-of-Volume Label (EOV1)

Except for character positions 1-3, which contain EOV, the EOV1 label has the same format as the EOF1 label. This label is located at the end of a file section of a tape file that continues on another reel of tape.

Second End-of-Volume Label (EOV2)

Except for character positions 1-3, which contain EOV, the EOV2 label has the same format as the EOF2 label. This label is located on all tapes following the EOV1 label.

E.2.2. INDEXED DISC Files

For files that have DISC as implementor-name-1 in the ASSIGN clause and the ORGANIZATION IS INDEXED clause in the file-control entry, the labels are maintained by COBOL. Only HDR1 and EOF1 labels are recorded on the files. These labels are similar in format to the HDR1 and EOF1 labels of the TAPE or INTERCHANGE files except that these labels consist of 80 9-bit ASCII characters. Character positions 22-27 of HDR1 contain UNISYS. Character positions 61-73 of HDR1 contain the file qualifier from the @ASG statement. The HDR1 and EOF1 labels are located at sector addresses 0 and 2, respectively. In the remaining words of sector 2, additional information about the file is maintained.

For a complete description of these words, see the *PCIOS Administration and Programming Reference Manual*.

E.2.3. RELATIVE and SEQUENTIAL DISC Files

The labels recorded on files that have DISC as implementor-name-1 in the ASSIGN clause and the ORGANIZATION IS RELATIVE or SEQUENTIAL clause in the file-control entry are maintained by the Processor Common Input/Output System (PCIOS) as System Data Format (SDF) labels (see the *PCIOS Administration and Programming Reference Manual*). UCS COBOL uses the first seventeen 9-bit ASCII characters starting at word 14 in these SDF labels to contain the FILE-ID field.

E.3. The VALUE OF Clause

The VALUE OF clause allows you to override system default values for certain fields within selected labels for unique label creation on output or validation on input. System default values generally originate on the @ASG statement for the file.

E.3.1. FILE-ACCESS

The VALUE OF FILE-ACCESS clause is allowed only for files assigned to tape. The only allowable functioning characters are ASCII 1 through 7 and space (octal values 061 through 067 and 040, respectively). The system treats any other character as a space.

The function of the values of 061-067 correspond to the R, W, and F options on the @ASG statement; that is, if the R option is on the assignment when the file is created, the file is read only until it expires.

Accessibility Character	Assign Options		
	F	W	R
" " or 040	X		
"1" or 061			
"2" or 062	X	X	
"3" or 063		X	
"4" or 064	X		X
"5" or 065			X
"6" or 066	X	X	X
"7" or 067		X	X

E.3.2. FILE-ID

The contents of the literal or the data item referenced by the data name specified in the VALUE OF FILE-ID clause is used to override the qualifier*file-name specified on the @ASG statement for the file during label creation or validation.

E.3.3. PURGE-DATE

The VALUE OF PURGE-DATE clause is allowed only for files assigned to tape. The contents of the literal or the data item referenced by the data name specified in the VALUE OF PURGE-DATE clause is used to override the purge date specified in field-4 of the @ASG statement for the file during file creation. These contents must be of the form mmddyy, where mm is the number of the month (01-12), dd is the number of the day (01-31), and yy is the last two digits of the year. For example, July 4, 1991, would be indicated as 070491.

E.4. Procedure Division I-O Statements

This subsection explains the logical sequence of labeling operations that occurs implicitly during the execution of certain I-O statements. Since the labeling of files assigned as INTERCHANGE and TAPE files is under control of the system Executive Tape Labeling System, this logical sequence of operations often does not correspond directly to the physical sequence of operations performed on the file.

E.4.1. The CLOSE Statement

CLOSE File

Output Files

During the execution of a CLOSE statement without the REEL phrase for a file opened in the output mode, the following sequence of operations occurs:

- The EOF1 label is created with the standard values.
- If any VALUE OF clauses are specified for the file, the standard values in the EOF1 label are replaced by the user-specified values.
- The EOF1 label is written to the external media.

In addition, for files assigned to tape, the following operations also occur:

- The EOF1 label is preceded on the tape by a tape mark.
- Following the EOF1 label, an EOF2 label and a double tape mark are also written on the tape.

Input Files

During the execution of a CLOSE statement without the REEL phrase for a file opened in the input mode, no label processing occurs if the file is positioned at a position other than its end. If the file is positioned at its end, the following sequence of operations occurs:

- The EOF1 label is read from the external media and standard label values are verified.
- If any VALUE OF clauses are specified for the file, the VALUE OF label entries are verified against the user-specified values.
- For tape files, the EOF2 label is also read from the external media.

I-O Files

During the execution of a CLOSE statement without the REEL phrase for a file opened in the I-O mode, label processing is identical to that for a file opened in the input mode. Subsequent to the other operations, the EOF1 label is rewritten to the external media. This includes the supplementary information contained in words 20 through 27 of sector 2 for indexed files.

Reversed Files

Only tape files can be opened in the reverse mode.

During the execution of a CLOSE statement for a file opened in the reverse mode, label processing occurs only if the beginning of file condition is encountered. If this condition has been encountered, label processing occurs similar to that stated for the CLOSE statement without the REEL phrase for a file opened in the input mode, except the HDR labels are treated as EOF labels.

CLOSE REEL

The CLOSE statement with the REEL phrase applies only to tape files.

Output Files

During the execution of a CLOSE statement with the REEL phrase for a file opened in the output mode, the following sequence of operations occurs:

- The EOV1 label is created with the standard values.
- If any VALUE OF clauses are specified for the file, the standard values in the EOV1 label are replaced by the user-specified values.
- A tape mark and the EOV1 label are written on the tape.
- The EOV2 label and a double tape mark are written on the tape.
- The current reel is swapped to the next reel in the sequence specified on the @ASG statement for the file.
- The labeling procedures continue on the new reel similar to those specified for an OPEN statement with the OUTPUT phrase. The file section field in the HDR1 label is updated to indicate the new section of the file.

Input Files

During the execution of a CLOSE statement with the REEL phrase for a file that is opened in the input mode, no end-of-volume label processing occurs for the current reel if the end of the file section has not been encountered. In this case, label processing resumes with the swap to the next reel in sequence below. If the end of the file section has been encountered, the following label processing occurs:

- The EOV1 label is read from the tape and standard label values are verified.
- If any VALUE OF clauses are specified for the file, the VALUE OF label entries are verified against the user-specified values.
- The EOV2 label is also read from the tape.
- The current reel is swapped to the next reel in the sequence specified on the @ASG statement for the file.
- The labeling procedures continue on the new reel similar to those specified for an OPEN statement with the INPUT phrase (see E.4.2).

E.4.2. The OPEN Statement

Output Files

During the execution of an OPEN statement with the OUTPUT phrase, the following sequence of operations occurs:

- The HDR1 label is created with the standard values.
- If any VALUE OF clauses are specified for the file, the standard values in the HDR1 label are replaced by the user-specified values.
- The HDR1 label is written to the external media.

In addition, for files assigned to tape, the following operations occur:

- If the file being created is the first file on a tape or tape set, the HDR1 label is preceded on the tape by a VOL1 label.
- Following the HDR1 label, a HDR2 label and a tape mark are also written on the tape.

Input or I-O Files

During the execution of an OPEN statement with the INPUT or I-O phrase, the following sequence of operations occurs:

- The HDR1 label is read from the external media and standard label values are verified.
- If any VALUE OF clauses are specified for the file, the VALUE OF label entries are verified against the user-specified values.
- For nonindexed disk files, the end-of-file address is extracted from the labels and made available to the program for subsequent end-of-file checks.
- For sequential-access files, the file is logically positioned at the first available data record.
- If the file being input is the first file on a tape or tape set, validation of the HDR1 label is preceded by the system validation of the VOL1 label.

Reversed Files

Only tape files can be opened in the reverse mode.

During the execution of an OPEN statement for a file opened in the reverse mode, label processing occurs similar to that stated for the OPEN statement for a file opened in the input mode except the EOF labels are treated as HDR labels.

E.4.3. The READ Statement

The READ statement can invoke label processing only in the case of a multiple reel tape file.

If, during the execution of a READ statement, an end-of-file condition occurs, label processing occurs as stated in the execution of a CLOSE statement with the REEL clause for a file opened in the input mode.

E.4.4. The WRITE Statement

The WRITE statement can invoke label processing only in the case of a multiple reel tape file.

If, during the execution of a WRITE statement, an end-of-tape condition occurs, label processing occurs as stated in the execution of a CLOSE statement with the REEL clause for a file opened in the output mode (see E.4.1).

Appendix F
UCS COBOL and ASCII COBOL Differences

Universal Compiling System (UCS) COBOL is the Unisys implementation of the American National Standards (ANS) X3.23-1985 COBOL standard in the Universal Compiling System (UCS). UCS software for extended mode is available in addition to the current, familiar software for basic mode. UCS COBOL is available in addition to ASCII COBOL and not as a total replacement for it.

UCS COBOL is the COBOL programmer's vehicle to take advantage of the Universal Compiling System and new COBOL developments as well as new system architecture developments.

Although UCS is a new compiling system, compatibility with earlier compilers is a major goal. This appendix is an attempt to identify areas which might possibly need attention when upgrading from ASCII COBOL to UCS COBOL. This appendix is composed of four divisions. They are

- Implementation Differences

  Those differences that occur primarily because UCS COBOL is a new implementation of COBOL.

- Discontinued Extensions

  Features of ASCII COBOL that are not carried over into UCS COBOL. These are primarily features that are currently documented in the *ASCII COBOL Supplementary Reference Manual*.

- Substantive Changes

  The changes in the 1985 COBOL standard issued in September, 1985, as well as the substantive changes made by the X3J4 technical committee as a result of the public review comments received during the public review period, which are identified as potentially affecting existing programs. For more complete information on the new standard, please refer to:

  X3.23-1985

  American National Standard

  Programming Language COBOL

  The changes in this section highlight the differences between the standard for ASCII COBOL (ANS X3.23-1974) and the standard for UCS COBOL (ANS X3.23-1985).

- Obsolete Language Elements

  Features that are present in this level of the standard (COBOL 85), but will be removed from future revisions of the standard. These features are either obsolete or improperly specified, but it is felt their abrupt removal from the standard would be a disservice to COBOL users. It is strongly recommended that these features not be used when writing new programs.

Note: *This appendix contains some explanations and comments which are taken directly or paraphrased from the above-mentioned standard.*

Each subsection contains somewhat subjective evaluations of the listed differences. These judgments are based on the effect these differences have on your programs, how to deal with a required change, and the potential benefits of these changes.

There is no requirement that you upgrade all ASCII COBOL programs to UCS COBOL. ASCII COBOL continues to be supported so that you can continue to execute your existing ASCII COBOL programs. The upgrade can be done over an extended period of time, with programs gradually switched to UCS COBOL when it makes good business sense for your organization. Therefore, as new programs are added to an existing application, you can write them in UCS COBOL. When existing programs require maintenance, it is a good time to upgrade them to UCS.

This appendix will be updated as new information about the software implementation or about customer use of existing software becomes available. Comments or suggestions regarding this appendix or the software described herein are welcome and should be submitted on the enclosed Business Reply Mail Form.

F.1. Conversion Tools

If you wish to migrate COBOL programs that are compatible with the ANSI-85 standard to UCS, you only need to compile with UCS COBOL. However, you need to give special consideration to ASCII COBOL programs that

- Are older than the ANSI-74 COBOL standard
- Are not in conformance with the ANSI-74 or ANSI-85 COBOL standard
- Have extensions not carried forward to UCS COBOL
- Use data file structures that are not standard

Since the UCS COBOL compiler only generates code for execution in extended mode, it is recommended that your ACOB programs are executed with Extended Instruction Set (EIS) enabled before migrating to UCOB. Enabling EIS instructions may help determine potential implementation differences. ACOB can generate EIS and non-EIS code through specific computer-names of the OBJECT-COMPUTER clause. Refer to the Environment Division in *OS 2200 ASCII COBOL Programming Reference Manual* for recommended computer-names and details.

You must rewrite extensions that are not supported by UCS COBOL (see F.2.33) before you compile the ASCII COBOL source programs that use them. Also, incompatibilities between the ANSI-74 and ANSI-85 standards must be resolved (see F.4).

The following tools are available to automate the upgrading of your programs to COBOL 85 and UCS COBOL:

COMPAT compiler keyword option

Using this option reduces the amount of changes your traditional program requires to execute. For example, the ASCII COBOL data allocation rules and ASCII COBOL-compatible implementation rules override UCS COBOL rules when you use the COMPAT keyword option to compile an upgraded COBOL program (see 2.2.7 for more information about the COMPAT option).

COMP-BIN compiler keyword option

Using this option causes an item declared as USAGE COMP to be treated as though it was declared USAGE BINARY. The COMPAT option automatically sets this option.

D$WORK-WS compiler keyword option

It is recommended that a COBOL program containing DMS statements where the DMCA is placed into COMMON using the INVOKE clause, be compiled with the D$WORK-WS keyword option. In ASCII COBOL, the ADMLP processor generates a relocatable holding the D$WORK data structure. In UCS COBOL this data structure is in the compiled OM element and by default is placed at the end of COMMON-STORAGE section when the INVOKE clause places the DMCA into COMMON. This can cause problems if the size of COMMON-STORAGE varies between the COBOL programs in an executable. The D$WORK-WS keyword option forces the D$WORK data structure to be placed into WORKING-STORAGE section instead, thereby bypassing any potential problems. Refer to Section 2.2.7, “Compiler Call Keyword Options for Program Control,” for a description of this keyword option.

IR-DIFF compiler keyword option

Using this option causes the compiler to issue a message whenever the result of a UCS COBOL calculation may differ. This difference occurs because the intermediate result in a UCS COBOL arithmetic expression assumes maximum precision. Thus, the higher accuracy of UCS COBOL may produce results that differ from the results of the same calculation in ASCII COBOL.

The following subsections refer to these tools.

F.2. Implementation Differences

The following identifies those differences that occur primarily because UCS COBOL is a new implementation of COBOL.

F.2.1. COMPUTATIONAL Fields Data Representation

Description

The default for the internal representation of COMP fields in UCS COBOL is ASCII digits. On option, COMPUTATIONAL fields are represented in binary.

ASCII COBOL has two ways of representing data that is declared as COMPUTATIONAL. COMPUTATIONAL data is represented as either binary or ASCII. The ASCII COBOL default is binary. (When binary is used, the results of arithmetic operations do not conform to the American National Standards requirements; when ASCII is used, the results of arithmetic operations do conform.) UCS COBOL also supports these two ways of representing COMPUTATIONAL data. Arithmetic operations performed on both the ASCII representation and binary representation conform to the American National Standards.

Changes

You can take one of two courses of action. First, COMPUTATIONAL data that is represented in binary can be changed to USAGE BINARY manually. Truncation of BINARY fields is by decimal digit rather than number of bits allocated as in ASCII COBOL. The second option is to use the compatibility option (COMPAT) to get binary representation which is also decimal truncated.

F.2.2. COMPUTATIONAL-3 Deleted

Description

COMP-3 has been removed and replaced by PACKED-DECIMAL. The extra "B" option made COMP-3 the same as DISP for compatibility with old levels of ASCII COBOL.

Changes

Change COMP-3 syntax to PACKED-DECIMAL manually. You can also use the compatibility option (COMPAT) or COMP-BIN to have COMP-3 allocated as PACKED-DECIMAL.

F.2.3. SYNCHRONIZED Field Alignment

Description

In ASCII COBOL, SYNC meant "word align this field." In UCS COBOL, SYNC has no effect on alignment. This change reduces problems resulting from the implicit insertion of slack bits. You need to insert slack explicitly in cases where slack bits are needed.

Changes

Use the compatibility option (COMPAT) to get ASCII COBOL style data alignment.

F.2.4. INDEX, COMP-1, COMP-2 Alignment

Description

These data types were automatically word aligned in ASCII COBOL. This is no longer true. Fewer problems result from the insertion of slack bytes.

Changes

Use the compatibility option (COMPAT) to get ASCII COBOL-style data alignment.

F.2.5. Data Allocation in IBM Records

Description

The extra A option forced data allocation in IBM records. This is no longer available. This option was used solely for the purpose of being compatible with ASCII COBOL level 3Rx and earlier.

Changes

Unisys does not know of any requirement for this capability in UCS COBOL.

F.2.6. IBM Data Allocation for Programs

Description

The extra C option could be used to force IBM data allocation throughout the program.

Changes

Unisys does not anticipate that this will be a problem since it only affects low usage binary fields in the IBM environment. UCS COBOL data allocation is compatible with recent IBM implementations.

F.2.7. Intermediate Operands

Description

The size of intermediate operands retained in calculations is different in UCS COBOL than in ASCII COBOL. This could lead to different program results. ASCII COBOL diagnosed the conditions in which an intermediate operand would exceed 21 digits.

Changes

UCS COBOL uses a single consistent algorithm. While some differences in precision can result, UCS COBOL is as precise or more precise than ASCII COBOL. An area that has created considerable confusion for customers is now eliminated.

F.2.8. Floating-Point Computations

Description

The extra T option could be used to force computations to use double precision floating point for intermediate results. This could possibly cause different results to be obtained if the feature was used.

Changes

UCS COBOL has gone to a single consistent algorithm. While some differences in precision can result, UCS COBOL is as precise or more precise than ASCII COBOL. This change also eliminates an area that has created considerable confusion for ASCII COBOL customers.

F.2.9. HIGH-VALUES

Description

HIGH-VALUES in ASCII COBOL had an octal value of 0177 unless the extra D option was used, in which case HIGH-VALUES meant 0377. In UCS COBOL, HIGH-VALUES is 0377.

Changes

Use the compatibility option (COMPAT or COMPAT/HV177) to get ASCII COBOL HIGH-VALUES (0177).

F.2.10. Undefined Results

Description

There are possible program execution differences in cases where a program relies on undefined values to be something predictable. An example is the assumption that an ASCII COBOL binary field (USAGE COMP) in WORKING-STORAGE with no VALUE clause has zero as its initial value. Reliance on these situations could lead to portability problems and is a bad programming practice.

Changes

For the specific case mentioned above, add a VALUE clause with the appropriate values to the data description in the program. You also could use the INITIALIZE verb to get the initial values desired. The keyword option CLEAR also can be used.

F.2.11. Calling Sequence

Description

The UCS COBOL calling sequence is common for all UCS languages and therefore cannot be the same as for ASCII COBOL. Non-UCS language subprograms and assembler subprograms are affected. The current formats for the CALL/ENTER common calling sequence are as follows:

X9	Dedicated register
X10	Dedicated register
R12-15	Dedicated registers
X3	Link vector of called routine
X11	Link register
A0	Contains the number of parameters in Q4, Q1-Q3 = 0
A1	Parameter list pointer (virtual address)

Each parameter is a three-word packet, where:

Word 0 contains the bit offset in S6.
Word 1 contains the virtual address.
Word 2 contains the length in bits.

Changes

Only call programs written in one of the UCS languages.

F.2.12. Register Conventions

Description

The UCS COBOL register conventions is common for all UCS languages and therefore cannot be the same as for ASCII COBOL.

Changes

Only call programs written in one of the UCS languages.

F.2.13. ASCII COBOL Level 3R1 Calling Sequence

Description

The extra G option was available to generate ASCII COBOL level 3R1 compatible calling sequences. This was documented only in the ASCII COBOL Supplementary Reference Manual. This change affects old assembler subprograms that have never been modified to use ASCII COBOL level 3R2 or higher calling sequences.

Changes

The subprogram must be modified.

F.2.14. Segmentation Map Symbolic

Description

The map symbolic element produced by ASCII COBOL is no longer produced if segmentation was specified in the source program. This change is interrelated with the switch to dynamic linking and the Linking System.

Changes

Modify the runstreams to conform to Linking System requirements.

F.2.15. CALL Identifier

Description

The CALL identifier implementation in UCS COBOL is different from that in ASCII COBOL. There is no need to assemble an element such as C$S333. The changes also are related to the change to dynamic linking and the Linking System.

Changes

Program creation runstreams are different and linking is simplified by the Linking System.

F.2.16. Extra Options Deleted

Description

The following options for sequence number checking and frequency table generation are not available in UCS COBOL.

0	Sequence of procedure file searching
2 & 3	Sequence number checking
7	Frequency table generation

Changes

Unisys does not see a requirement to carry the sequence number checking or the frequency table generation forward into UCS COBOL. The UCS COBOL options SEARCH-PF-SI and SEARCH-SI-PF control the sequence of procedure file searching.

F.2.17. Referencing Record Areas before OPEN

Description

Action in ASCII COBOL programs depends on whether the file was move-mode or locate-mode. UCS COBOL allocates record areas at OPEN time, so no record area is available prior to the OPEN.

Changes

In some cases, the ASCII COBOL implementation permitted the invalid use of record areas before the OPEN. This invalid use is not permitted within UCS COBOL. The UCS Runtime System issues errors during execution time when attempting to do this.

F.2.18. Index-Name Formats

Description

Problems could occur if an index was used with a table other than the table defining the index. If the referenced table had a format identical to the defining table, correct results were obtained.

Changes

Delete this illegal programming construct. It is not allowed in UCS COBOL.

F.2.19. INDEX Data Items

Description

Index data items might have different internal formats in UCS COBOL.

Changes

This only causes problems when the program uses index data items in a nonstandard manner.

F.2.20. SEARCH ALL

Description

A slightly different binary search algorithm might be used, possibly yielding different results if the keys of the searched table are duplicated, out of order, or contrary to the specifications in the standard.

Changes

There is no change. No known instance exists where this illegal construct was used.

F.2.21. Overpunched Signed Numeric Fields

Description

UCS COBOL, in some cases, is inconsistent with the way ASCII COBOL decides to overpunch or not overpunch sign-included signed numeric fields, when the SIGN IS SEPARATE clause was not specified. This should not cause major problems, since the numeric values are valid whether or not an overpunch is present. The potential for a difference arises when a user program treats signed data as an alphanumeric bit string.

For a data item described as signed (S in the PICTURE clause), which does not have the SIGN clause, the internal representation of the operational sign is implementor defined (ANS X3.23-1974). Unsigned values, positive-signed values, and negative-signed values are all legal in a data item described as signed. Each possibility must be allowed since the incoming data, possibly from foreign systems, should be treated correctly, regardless of whether or not an overpunched sign is present.

Numeric comparisons are done with respect to the algebraic value of the operands and must work correctly whether overpunched or not. Non-numeric comparisons are done with respect to the collating sequence of the associated characters. It is possible that nonnumeric comparisons of the data items which can contain overpunched numeric data will give an unequal result, even though the contained numeric values are algebraically equal. See the *ASCII COBOL Programming Reference Manual*.

This situation (presence or lack of overpunches) exists within ASCII COBOL as well as between ASCII COBOL and UCS COBOL.

Changes

Programs should not be coded to use only one of several legal internal forms of signed numeric data representation. There is no problem where the numeric data is treated as numeric.

F.2.22. Class Names

Description

Class name specifications in the SPECIAL-NAMES paragraph must be in the format "CLASS *class-name* IS ..." according to the COBOL 85 standard. The ASCII COBOL extension which allowed class name definitions required the format "CLASS-NAME IS *mnemonic-name* ... ."

Changes

Restructure the class name specification manually.

F.2.23. Subscript Checking

Description

In ASCII COBOL, extensions were available to do subscript range checking on all table references. Either the X extra option or the USE FOR TABLE ERROR syntax in the DECLARATIVES could be used. UCS COBOL has an option which does consistent run-time error checking for all UCS languages.

Changes

Use the UCS COBOL keyword option RUNCHECK to perform the subscript verification.

F.2.24. Numeric PROGRAM-ID Name

Description

The program name in the program-id paragraph in the Identification Division in ASCII COBOL could be numeric; for example, 1234. In UCS COBOL this is not allowed. The standard does not allow numeric program names. It is felt that this causes very little impact on existing programs.

Changes

Change the program name manually. An alternative is to use the COMPAT option, which allows a numeric PROGRAM-ID.

F.2.25. Paragraph Followed by a GO TO Statement

Description

If any unaltered paragraph name is immediately followed by a GO TO statement, the address of the paragraph name is equated to the address of the target of the GO TO statement. For example:

```
P1.
    .
    .
    .
P2.
GO TO P1
```

In this example, both P1 and P2 are assigned to the same code location.

Changes

In a PADS session, all PADS commands should refer to the target of the GO TO statement.

F.2.26. FILE STATUS with APPLY EXDEF and APPLY EXREF Clause

Description

The UCS COBOL use of the FILE STATUS variable in conjunction with the APPLY EXDEF and APPLY EXREF clauses is different from the ASCII COBOL use. In ASCII COBOL, when a program contains a file with a FILE STATUS item, and both the file and the FILE STATUS item are the object of an APPLY EXDEF clause, you do not need to specify the FILE STATUS item in the SELECT statement of the subprogram that contains the FILE STATUS item in an APPLY EXREF clause. This results in the FILE STATUS variable being updated in the subprogram even though the subprogram does not explicitly specify the FILE STATUS item in its SELECT statement. The subprogram FILE STATUS item, used in this way, is not updated in UCS COBOL.

Solution

You must specify the FILE STATUS item in the SELECT statement of both programs, even if it uses the APPLY EXDEF and APPLY EXREF clauses. UCS COBOL does not update the value of the FILE STATUS item unless it is present in the SELECT statement of the program that requires the value of FILE STATUS to be updated.

F.2.27. Duplicate Data Names from Schema

Description

User-defined names such as record names, area names, and set names that are initially defined in the schema elements must not duplicate any other name in the UCS COBOL program. Every user-defined name that is used within the UCS COBOL program must be uniquely definable. Data names can be duplicated if they can be uniquely identified in the program through the use of qualification.

ADMLP did not check for uniqueness of reference in names that were defined in the schema and duplicated in the ASCII COBOL program.

Changes

The compiler must be able to uniquely define all user-defined names. Ensure that no duplicate data names are defined in the schema and UCS COBOL program.

F.2.28. Report Format for Nonprint Files

Description

If a file designated for a report is assigned to a device other than the printer, UCS COBOL appends to the beginning of each record a 1-word field that contains carriage control information (see the *UCS COBOL Programming Reference Manual Volume 1*).

The meaning of this 1-word field, called the carriage control field (the *cc* field), is different in UCS COBOL and ASCII COBOL regarding page advancement. In UCS COBOL, if the *cc* field is negative, the printer advances to the specified line on the next page. For example, -5 in the *cc* field means advance to line 5 on the next page. In ASCII COBOL, page advancement is done with a value of -0 in the *cc* field, followed by another record that contains a positive value in the *cc* field that signifies the number of lines to advance.

Changes

A user-written program is required to extract individual records from a report file written to a nonprint file. This extraction program for ASCII COBOL must be modified to process page advancing for a nonprint report file created by UCS COBOL. However, once this extraction program has been modified for UCS COBOL, it can still process report writer records written to nonprint files for ASCII COBOL programs.

F.2.29. Communications Module Not Implemented

Description

The communications module that was implemented in ASCII COBOL is not implemented in UCS COBOL.

Changes

All DISABLE, ENABLE, RECEIVE, and SEND statements are invalid.

F.2.30. Data Allocation

Description

ASCII COBOL allocates all static variables in one bank. These variables are allocated even if they are never referenced in the program. UCS COBOL only allocates variables that are referenced in the program. In addition, the variables allocated by UCS COBOL are placed in multiple banks.

Changes

Use the keyword option COMPAT/FULL to ensure that all variables are allocated and placed in one bank.

F.2.31. Record Key/Alternate Key Alignment

Description

ASCII COBOL allowed data items specified in the RECORD KEY clause and ALTERNATE RECORD KEY clause to begin with the same leftmost character position in the record. This violates the COBOL 74 and COBOL 85 Standards and should not have been allowed. ASCII COBOL has been changed to flag this situation with a warning message.

Changes

The user must change the source program such that any RECORD KEY and ALTERNATE RECORD KEY data items specified do not start on the same leftmost character position in the record.

F.2.32. IBM Labeled Tapes

Description

In ASCII COBOL, IBM labeled tapes are handled by the ASCII COBOL runtime library when the tapes are assigned as unlabeled or with the Exec when the tapes are assigned as labeled. UCS COBOL does not handle the IBM labels itself and relies on PCIOS and the EXEC's use of the TLEBCDIC feature.

Changes

In UCS COBOL, if the IBM tape is labeled, the system must be set up for EBCDIC tape labeling and labeled tapes must not be assigned with the J option. The absence of the J option in the file's assignment will cause the Exec (level 40R3 and higher) to process the file's labels. For the Exec to process the labels, Exec configuration tag TLEBCDIC must be set to 1 or 2. TLEBCDIC set to 1 is preferred (required if IBM labeled tapes are being used with ASCII COBOL assigned as unlabelled and UCS COBOL). For more information, see the *Exec System Software Installation and Configuration Guide*.

The following table illustrates the tape labels that will be produced by ACOB and UCOB based upon the tape assignment as well as the setting of the Label Records clause.

User Specifications	Tape Produced from ACOB	Tape Produced from UCOB
@asg,t with 'Label Records Omitted'	Labeled	Labeled
@asg,t with 'Label Records Standard'	Labeled	Labeled
@asg,tj with 'Label Records Omitted'	Unlabeled	Unlabeled
@asg,tj with 'Label Records Standa`rd'	Labeled	URTS error 19002/108

F.2.33. Blocking Size

Description

In ASCII COBOL, the default block size for a file (no BLOCK CONTAINS clause specified) is 224. For UCS COBOL, the default block size is 1792.

Changes

For additional information, see Section 4.4 UCS Runtime I/O. This should not cause any problem when using the file with a default block size within the UCS environment. If the file needs to be used outside the UCS environment, the default block size can be altered by performing one of the following:

1. Change default block and prep factor in URTS$TABLES and re-link the programs.
2. Use DFP to alter the block size.
3. Use Sort to copy the resulting file and change the blocking factor.

F.3. Discontinued Extensions

This subsection identifies features of ASCII COBOL that are not carried over into UCS COBOL. These are primarily features that are currently documented in the *ASCII COBOL Supplementary Reference Manual*.

F.3.1. IOS Files

Description

Non-PCIOS files in ASCII COBOL (MASS-STORAGE, UNISERVO and SEQUENTIAL-FILE) are discontinued. All of these formats were defined in the *ASCII COBOL Supplementary Reference Manual*.

Changes

Upgrade non-PCIOS files to their PCIOS equivalents. To do this, write an ASCII COBOL program that reads in the old file type and outputs a supported file type.

F.3.2. Compatible Files

Description

Support for CFH, LION, FORM01, FORM02, and FORM03 files is discontinued. All were defined only in the *ASCII COBOL Supplementary Reference Manual*.

Changes

Upgrade non-PCIOS files to their PCIOS equivalents. To do this, write an ASCII COBOL program that reads in the old file type and outputs a supported file type.

F.3.3. 770-PRINTER Deleted

Description

770-PRINTER as the implementor name in a SELECT statement is no longer necessary.

Changes

Change the syntax manually.

F.3.4. ALPHABET SECTION Deleted

Description

The ALPHABET SECTION feature is used for extending the source character set of ASCII COBOL, used especially for katakana characters.

Changes

Remove ALPHABET SECTION and CONTROL DIVISION from your programs. (UCS COBOL allows an extended character set.)

F.3.5. Asynchronous Processing Module Deleted

Description

The Asynchronous Processing module was rarely used and has been deleted.

Changes

Remove the syntax from the programs. This change affects all HOLD, LOCK, and PROCESS statements.

F.3.6. APPLY INLINE Deleted

Description

The function of APPLY INLINE is not required. Remove the APPLY INLINE syntax and CONTROL DIVISION from your programs.

Changes

Remove the APPLY INLINE syntax and CONTROL DIVISION from the programs.

F.3.7. POINT LOCATION Clause Deleted

Description

The POINT LOCATION clause was used to specify the binary point location in PIC 1 items and is defined in the *ASCII COBOL Supplementary Reference Manual*.

Changes

Remove the POINT LOCATION clause syntax manually.

F.3.8. REMARKS Deleted

Description

The REMARKS paragraph, defined in the *ASCII COBOL Supplementary Reference Manual*, has been deleted.

Changes

Change programs to use the asterisk to comment out all lines in the REMARKS paragraph manually.

F.3.9. MONITOR Deleted

Description

Newer facilities are provided for program debugging. This should not affect existing production programs.

Changes

PADS is the primary debugging tool for future development. The keyword options MONITOR and MONITOR/LABEL provide a feature similar to the MONITOR verb within UCS COBOL. See 2.2.7 for more information on how to use MONITOR and MONITOR/LABEL.

F.3.10. Special Registers

Description

TALLY and MCFLAG were implicitly defined for use with EXAMINE and MONITOR, but could be used in any context. TALLY is not implicitly defined in UCS COBOL. MCFLAG is implicitly defined only if you specify the keyword options MONITOR or MONITOR/LABEL. See 2.2.7 for more information about MCFLAG.

Changes

Add definitions of TALLY and MCFLAG to the WORKING-STORAGE section manually.

F.3.11. ACCEPT and DISPLAY

Description

```
ACCEPT ..       FROM CARD-READER.
DISPLAY ..      UPON PRINTER.
```

Mnemonic names, defined in the SPECIAL-NAMES paragraph, must be used in conjunction with ACCEPT and DISPLAY. The extension which allows PRINTER and CARD-READER in ASCII COBOL is documented in the *ASCII COBOL Supplementary Reference Manual.*

Changes

Add mnemonic names manually to the SPECIAL-NAMES paragraph for PRINTER, CARD-READER, and CONSOLE and change ACCEPT/DISPLAY to refer to the mnemonic names.

F.3.12. EXAMINE, EXHIBIT, TRANSFORM Deleted

Description

The EXAMINE, EXHIBIT, and TRANSFORM verbs, defined in the *ASCII COBOL Supplementary Reference Manual*, have been deleted.

Changes

Change the deleted verbs to equivalent, legal syntax manually.

F.3.13. TRAILING Option of INSPECT Deleted

Description

This extension to the INSPECT statement, defined in the *ASCII COBOL Supplementary Reference Manual*, has been deleted.

Changes

Delete the TRAILING option from programs manually.

F.3.14. NOTE Statement Deleted

Description

The NOTE statement is no longer legal.

Changes

Use the asterisk (*) for all types of comments.

F.3.15. ON Integer Deleted

Description

The ON statement, defined in the *ASCII COBOL Supplementary Reference Manual*, has been deleted.

Changes

The ON statement was used infrequently. Programs using the ON statement must be changed manually.

F.3.16. COBOL 68 Format COPY Statements

Description

The extra "E" option, provided in ASCII COBOL to allow processing of unconverted COBOL 68 format COPY statements and procedures, is not available in UCS COBOL.

Changes

Convert procedures to the current UCS COBOL format.

F.3.17. Old ENTER Statements Deleted

Description

ENTER statements, defined in the *ASCII COBOL Supplementary Reference Manual*, have been deleted. UCS COBOL continues to support the ENTER MASM statement.

Changes

Unisys supports the extended mode MASM usage (assembler directive $OBJ) only in software written by Unisys or in interfaces written by the customer that explicitly require assembler-produced object modules according to the documentation written by Unisys.

F.3.18. Mnemonic Name in CODE Clause

Description

The mnemonic name in the CODE clause allowed a mnemonic name (defined in SPECIAL-NAMES) to be used in place of a literal in a report description (RD).

Changes

Change the mnemonic to a two-character literal in the CODE clause of the report description manually.

F.3.19. COBOL 68 CODE Clause Deleted

Description

The extra O option allowed the code in a report file record to be put at the end of a record instead of at the beginning. This is a COBOL 68 carryover that has been deleted.

F.3.20. EQUALS, EXCEEDS, UNEQUAL

Description

These relational operators, which are defined in the *ASCII COBOL Supplementary Reference Manual*, are not allowed in UCS COBOL.

Changes

Modify programs to use EQUAL, GREATER, or NOT EQUAL as appropriate.

F.3.21. Fieldata

Description

The Fieldata character set and sixth-word aligned data is not supported in UCS COBOL.

Changes

Fieldata to ASCII migration is the subject of separate documents. Please check with Field Marketing branches for more detail.

F.4. Substantive Changes from the American National Standard (ANS)

This subsection contains the analysis by Unisys of

- The changes in the 1985 COBOL standard issued in September, 1985
- The substantive changes made by the X3J4 technical committee as a result of the public review comments received during the public review period

Existing ASCII COBOL programs which use the ANS 1974 COBOL standard may be affected by the following changes.

F.4.1. Length of ALL Literal

Description

Delete the word ALL manually.

Expected Impact on ASCII COBOL Programs

No expected impact.

ALL is not allowed in statements such as

```
DISPLAY ALL 'ABC' UPON PRINTER.
```

Changes

Delete the word “ALL”.

F.4.2. Alphabet-Name Clause

Description

The word "ALPHABET" must be specified in the ALPHABET-NAME clause in SPECIAL-NAMES.

This change is necessary to resolve an ambiguity, since system-names and user-defined words can intersect, which was not possible in COBOL 74.

Expected Impact on ASCII COBOL Programs

Illegal usage is flagged at compilation time.

Changes

Add the word "ALPHABET" before any ALPHABET-NAME clauses in the SPECIAL-NAMES SECTION.

F.4.3. Collating Sequence

Description

The collating sequence used to access an indexed file is the native character set.

Expected Impact on ASCII COBOL Programs

No impact is anticipated because this is how ASCII COBOL currently works.

Changes

None.

F.4.4. CURRENCY SIGN Clause

Description

The literal specified by the CURRENCY SIGN clause must not be a figurative constant.

HIGH-VALUES, LOW-VALUES, and so forth, should not be allowed as currency signs.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. Illegal usage is flagged at compilation time.

Changes

Insert a literal with the appropriate value.

F.4.5. RELATIVE KEY Phrase

Description

A RELATIVE KEY must not contain the PICTURE symbol P.

A RELATIVE KEY with a P in the PICTURE clause could not be used to access all the records in a file.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. Illegal usage is flagged at compilation time.

Changes

Remove the "P" from the PICTURE clause.

F.4.6. MULTIPLE FILE TAPE Clause

Description

1. All files on a multiple file tape must have the same labeling conventions. Problems occur if different files on one multiple file tape had differing labeling conventions.
2. SORT-MERGE files must not be specified. Scratch files on a multiple file tape have no reasonable application and could only cause problems.

Expected Impact on ASCII COBOL Programs

No impact is anticipated.

Changes

Modify the program, either manually or with a converter, as follows:

1. Use STANDARD labels in all files specified on a MULTIPLE FILE TAPE clause.
2. Remove SD names from the MULTIPLE FILE TAPE clause.

F.4.7. LINAGE Clause

Description

Files with the LINAGE clause specified must not be opened in the EXTEND mode.

The size of the first logical page of a file opened in the EXTEND mode was undefined. The page number was set to one. There is not a good reason to allow this combination.

Expected Impact on ASCII COBOL Programs

No impact is anticipated.

F.4.8. FOOTING Phrase

Description

No END-OF-PAGE condition occurs if FOOTING is not specified in the FD.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. ASCII COBOL currently works as specified in COBOL 85.

F.4.9. OCCURS Clause

Description

When a variable length receiving item contains the DEPENDING ON field, the maximum length of the field is used for the move.

Contents of the receiving field in cases where COBOL 85 moves extra characters compared to COBOL 74 were, and are, undefined.

Expected Impact on ASCII COBOL Programs

Very little impact is anticipated.

ASCII COBOL uses maximum length if the receiving field contains the DEPENDING ON field.

F.4.10. PICTURE Symbol P

Description

When a data item described by a PICTURE clause containing the symbol P is referenced, the digit positions specified by the P are considered to contain zeros in the following operations:

- Any operation requiring a numeric sending operand
- A MOVE statement where the sending operand is numeric and its PICTURE character string contains the symbol P
- A MOVE statement where the sending operand is numeric edited and its PICTURE character string contains the symbol P and the receiving operand is numeric or numeric edited
- A comparison operation where both operands are numeric.

This is a clearer, more intuitively correct approach. The cases where differences might occur are obscure and should not be very common.

Expected Impact on ASCII COBOL Programs

Very little impact is anticipated.

F.4.11. PROCEDURE DIVISION Header

Description

A data item specified on the USING phrase of the PROCEDURE DIVISION header must not be the subject of a REDEFINES clause.

This restriction eliminates some problems which could occur if the same data area is passed as two separate parameters.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. ASCII COBOL, while allowing the REDEFINES subject in the USING phrase, does not allow both the subject and object of the REDEFINES to be specified on the same USING.

F.4.12. Exponentiation

Description

The following special cases of exponentiation are defined in the 1985 COBOL standard:

- If a value less than or equal to 0 is raised to a power of 0, the size error condition exists.
- If the evaluation of the exponentiation yields both a positive and a negative real number, the positive number is returned.
- If no real number exists as the result of the evaluation, the size error condition exists.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. ASCII COBOL functions as specified in the COBOL 85 standard.

F.4.13. Order of Execution for a Conditional Expression

Description

Conditional expressions are evaluated from left to right within a hierarchical level, and the evaluation terminates as soon as a truth value is determined.

Only debugging declaratives are affected in existing programs. Situations similar to the one shown below might not have worked in COBOL 74. These situations work with the COBOL 85 standard.

```
01  SS  PIC 999.
01  LITTLE-TABLE.
    02  ABC PIC 9999 OCCURS 10.
.
.
    MOVE 100 TO SS.
    IF SS < 11 AND ABC (SS) = 0
```

Expected Impact on ASCII COBOL Programs

No impact is anticipated.

F.4.14. Class Condition

Description

The ALPHABETIC class test is true for both uppercase and lowercase letters and space.

The ALPHABETIC test makes more sense if it is true for lowercase and uppercase letters. Many compilers already work this way.

Expected Impact on ASCII COBOL Programs

No impact is anticipated, since ASCII COBOL currently accepts lowercase letters as alphabetic.

F.4.15. CANCEL Statement

Description

The CANCEL statement closes all files open in the CANCEL statement subprogram.

Expected Impact on ASCII COBOL Programs

No impact is anticipated.

Change

Change programs that call files that were open in a program that was canceled. Defining this previously undefined condition ensures that a program obtains a predictable result.

F.4.16. CLOSE Statement

Description

The NO REWIND phrase cannot be specified in a CLOSE statement having the REEL/UNIT phrase.

The rules for the NO REWIND and REEL/UNIT phrases conflict in COBOL 74; both could not be processed.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. Illegal usage is flagged at compilation time.

Changes

Remove the NO REWIND phrase from CLOSE statements that have a REEL/UNIT phrase.

F.4.17. COPY Statement in Comments

Description

If a COPY statement occurs within a comment-entry or in a place where a comment-entry can appear, it is treated as part of the comment-entry.

This defines a previously undefined area and enhances program portability.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. ASCII COBOL currently treats comment-entry COPY statements as part of the comment and does not copy any text from the procedure file.

F.4.18. COPY Statement on a Debugging Line

Description

If DEBUGGING MODE is not specified in the SOURCE-COMPUTER statement, a debugging line is considered to have all the characteristics of a comment line after all COPY statements have been processed.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. ASCII COBOL currently does the REPLACING even if REPLACING is on a debug line and DEBUGGING MODE is not turned on. If DEBUGGING MODE is not on and the COPY statement begins on a debug line, ASCII COBOL treats the COPY statement itself as a comment line. UCS COBOL treats the copied text as a comment line. This should not cause a problem.

F.4.19. DISPLAY Statement

Description

After displaying the last operand in a DISPLAY statement, the hardware device is positioned at the first character position of the next line.

This rule is required so that the NO ADVANCING option can be fully specified.

Expected Impact on ASCII COBOL Programs

A program which intermixes DISPLAY statements with WRITE statements to the same output file could result in overprinting.

F.4.20. DIVIDE Statement

Description

Subscripts for *identifier-4* in the remainder phrase of a DIVIDE statement are evaluated after the result of the DIVIDE statement is stored in *identifier-3* of the GIVING phrase.

The point at which the subscript for the remainder was determined was not defined in the COBOL 74 standard. Inconsistencies were likely.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. ASCII COBOL functions as specified in the COBOL 85 standard.

F.4.21. EXIT PROGRAM Statement Used Implicitly

Description

An implicit EXIT PROGRAM statement is executed if there is no next executable statement in a subprogram. This situation is undefined in COBOL 74.

Expected Impact on ASCII COBOL Programs

No impact is anticipated since ASCII COBOL currently works this way.

F.4.22. EXIT PROGRAM Statement Used with the PERFORM Statement

Description

When an EXIT PROGRAM statement is executed, all active PERFORM statements are assumed to have reached termination.

An ambiguity is resolved. Program portability is enhanced.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. An application that relies on this undefined situation in COBOL 74 could cause a problem.

F.4.23. INSPECT Statement

Description

The order of execution for evaluating subscripts in the INSPECT statement is specified.

The time of subscript evaluation should be defined in the standards, but was not in COBOL 74.

Expected Impact on ASCII COBOL Programs

No impact is anticipated.

F.4.24. MERGE Statement

No two files in a MERGE statement can be specified in a SAME AREA or SAME SORT-MERGE AREA clause. Also, only the GIVING files in a MERGE can be specified in a SAME RECORD AREA clause.

Description

If this rule (not in COBOL 74) had been violated in a COBOL 74 program, the MERGE would probably not have worked correctly.

Expected Impact on ASCII COBOL Programs

No impact is anticipated.

Changes

Change any illegal usages that are flagged at compilation time.

F.4.25. PERFORM Statement — Initialization of Variables

Description

The order of initialization of multiple varying identifiers in a PERFORM statement is defined.

This situation was undefined in COBOL 74 and improves program portability. The chance of an incompatibility in an existing program is small.

Expected Impact on ASCII COBOL Programs

Very little impact is anticipated. To be affected, one VARYING identifier must depend on the other.

F.4.26. PERFORM Statement — Setting and Augmenting Variables

Description

In a PERFORM ... VARYING ... AFTER statement, *identifier-2* is augmented before *identifier-5* is set.

The COBOL 74 rules for this condition do not make sense, so the change is needed. Very few programs are likely to be affected since there is no logic to the COBOL 74 definition.

Expected Impact on ASCII COBOL Programs

Some problems could occur due to different increments of PERFORM statement variables.

Change

Modify PERFORM statement variables to reflect the COBOL 85 definition.

F.4.27. PERFORM Statement — Evaluation of Subscripts

Description

The order of execution for evaluating subscripts in the PERFORM VARYING statement is specified in COBOL 85.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. ASCII COBOL functions as specified in the COBOL 85 standard.

F.4.28. READ Statement

Description

A READ ... INTO statement can be specified only if

- Only one record description is subordinate to the file description
- All associated records as well as the INTO record are group items or elementary alphanumeric items

Conversion of multiple elementary records was not defined in COBOL 74. Various implementations can produce different results.

Expected Impact on ASCII COBOL Programs

Very little impact is anticipated. Illegal usage is flagged at compilation time.

F.4.29. RETURN Statement

Description

A RETURN ... INTO statement can be specified only if all associated records as well as the "INTO" record are group items or elementary alphanumeric items.

Conversion of multiple elementary records was not defined in COBOL 74. Various implementations can produce different results.

Expected Impact on ASCII COBOL Programs

Very little impact is anticipated. Illegal usage is flagged at compilation time.

F.4.30. STOP RUN Statement/Program Termination — File Closure

Description

The termination of a program with an explicit STOP RUN statement, an implicit STOP RUN statement, or an abnormal program termination causes all open files to be closed.

Leaving files in an open mode at program termination can lead to invalid files. This leads to errors, since it is possible to leave a file open. This change does not affect correct operational programs.

Expected Impact on OS 2200 ASCII COBOL Programs

No impact is anticipated. In the past it has been quite common for files to be left in an open state, either by programmer error or program-user error. Sometimes such conditions were not noticed immediately and were often quite hard to explain.

Changes

None. The definition of the STOP RUN statement in the COBOL 85 standard requires the statement to close all open files. If a STOP RUN statement is not executed, all open files will be closed via URTS runtime termination routines.

F.4.31. STOP RUN Statement — Message Queue Management

Description

The STOP RUN statement causes any messages partially received by the run unit from the message control system to be eliminated from the queue.

Expected Impact on ASCII COBOL Programs

No impact is anticipated.

F.4.32. STRING Statement

Description

The order of execution for evaluating subscripts in the STRING statement is specified in COBOL 85. The order of subscript evaluation in COBOL 74 was not defined.

Expected Impact on ASCII COBOL Programs

No impact is anticipated.

F.4.33. UNSTRING Statement

Description

The order of execution for evaluating subscripts in the UNSTRING statement is specified in COBOL 85. The order of subscript evaluation in COBOL 74 was not defined.

Expected Impact on ASCII COBOL Programs

No impact is anticipated.

F.4.34. WRITE Statement

Description

The AFTER PAGE and END-OF-PAGE phrases must not be specified in the same WRITE statement. It is not logical to have both of these phrases in one statement.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. Illegal usage is flagged at compilation time.

Changes

Change any WRITE statements that specify both the AFTER PAGE and END-OF-PAGE phrases.

F.4.35. File Position Indicator — Record Pointer Replacement

Description

The current record pointer concept of COBOL 74 is replaced by a file position indicator in COBOL 85.

Certain READ/WRITE/DELETE combinations could give undefined or surprising results. The current record pointer could point to a nonexistent record in some circumstances. The file position indicator is more specifically defined with a finite set of values.

Expected Impact on ASCII COBOL Programs

Very little impact is anticipated.

Changes

The two cases that are affected by this change are described in F.4.36 and F.4.37.

F.4.36. File Position Indicator — Effect on READ/WRITE Statements

Description

The following sequence causes the low-key record to be read according to COBOL 85:

```
OPEN I-O filename.
WRITE record-with-a-low-key.
READ filename NEXT ...
```

In COBOL 74, the previous first record would have been read.

Since the WRITE statement does not affect the file position indicator or current record pointer, it follows logically that the first READ NEXT statement for a file should give the first record in the file.

Expected Impact on ASCII COBOL Programs

Some problems could occur.

F.4.37. File Position Indicator — Effect of Rewrite Statement

Description

If an alternate key is the key of reference, and the alternate key is changed by a REWRITE statement to a value between the previous value and the previous next key value, a READ NEXT statement would make the record that had just been written available.

This is logical since the record that will be read is not the “same” record because its alternate key value has changed.

Expected Impact on ASCII COBOL Programs

No impact is anticipated since ASCII COBOL uses the current READ statement to read the rewritten record.

F.4.38. Reserved Words

The following reserved words have been added to the standard:

```
ALPHABET
ALPHABETIC-LOWER
ALPHABETIC-UPPER
ALPHANUMERIC
ALPHANUMERIC-EDITED
ANY
BINARY
CLASS
COMMON
CONTENT
CONTINUE
CONVERTING
DAY-OF-WEEK
END-ADD
END-CALL
END-COMPUTE
END-DELETE
END-DIVIDE
END-EVALUATE
END-IF
END-MULTIPLY
END-PERFORM
END-READ
END-RECEIVE
END-RETURN
END-REWRITE
END-SEARCH
END-START
END-STRING
END-SUBTRACT
END-UNSTRING
END-WRITE
EVALUATE
EXTERNAL
FALSE
FUNCTION
GLOBAL
INITIALIZE
NUMERIC-EDITED
ORDER
OTHER
PACKED-DECIMAL
PADDING
```

```
PURGE
REFERENCE
REPLACE
STANDARD-2
TEST
THEN
TRUE
```

The reserved words ALPHABET and ANY were included as Unisys extensions to the COBOL 74 reserved word list.

The following new reserved words have been added as Unisys extensions to the reserved word list for the COBOL 85 standard:

```
BINARY-1
DEPART
DISP-2
DISPLAY-2
DUPLICATE
EMPTY
EXEC
FETCH
FIND
FREE
FUNCTION
GET
GOBACK
IMPART
INSERT
INVOKE
KEEP
LOG
MODE-1
MODE-2
MODE-3
MODE-4
OWNER
REMOVE
STORE
SUSPEND
```

Description

New reserved words have been added.

Benefits are derived from the features provided, even though there can be an inconvenience due to now illegal user-defined words. The REPLACE statement should drastically reduce the inconvenience of program modification.

Expected Impact on ASCII COBOL Programs

Some problems could occur. Illegal usage is flagged at compilation time.

Changes

The required source changes can be easily made manually.

F.4.39. I-O Statuses

Description

New I-O statuses have been added. The new statuses define previously undefined actions that could have been implemented in a variety of ways.

Expected Impact on ASCII COBOL Programs

Very little impact is anticipated. New statuses do not conflict with existing statuses but there is a possible upward compatibility problem. With the new statuses that have been defined, it is possible for you to detect I-O errors from the Declarative section of the UCS COBOL program. In UCS COBOL, when a Declarative section is defined, the UCS Runtime System does not issue an error message as was done in ASCII COBOL.

F.5. Obsolete Language Elements

The following features are present in this level of the standard (COBOL 85), but will be removed from future revisions of the standard. These features are either obsolete or improperly specified, but it is felt their abrupt removal from the standard would be a disservice to COBOL users. It is strongly recommended that these features not be used when writing new programs.

F.5.1. Character Substitution

Description

Character substitution of double characters for a single UCS COBOL character has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

These specifications are a carry over from computers that could not provide the full UCS COBOL character set.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. Usage is flagged at compilation time.

F.5.2. ALL Literal and Numeric or Numeric Edited Items

Description

The figurative constant ALL literal that is associated with a numeric or numeric edited item, and that has a length greater than one, has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

Expected Impact on ASCII COBOL Programs

Some problems could occur.

A diagnostic flagging this as an obsolete feature is provided.

Changes

Moving an ALL literal to numeric or numeric edited items when the length of the literal is greater than one character is allowed in UCS COBOL.

F.5.3. Identification Division Paragraphs

Description

The AUTHOR, INSTALLATION, DATE-WRITTEN, DATE-COMPILED, and SECURITY paragraphs in the Identification Division have been placed in the obsolete category. This means that these features will be removed from future revisions of the standard and should not be used when writing new programs.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. A diagnostic flagging this as an obsolete feature is provided.

Changes

UCS COBOL allows these paragraphs, but ignores them. Remove or comment out these paragraphs.

F.5.4. MEMORY SIZE Clause

Description

The MEMORY SIZE clause has been placed in the obsolete category. This means it will be removed from future revisions of the standard and should not be used when writing new programs.

This clause was totally implementor defined and there was no commonality among implementations which have assigned a use to the phrase. Any meaning is better left for the operating system to define.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. It is not needed for programs which contain addresses over 65,535 since all UCS COBOL code generation allows reference of data at locations greater than 65,535.

A diagnostic flagging this as an obsolete feature is provided.

Changes

UCS COBOL allows the MEMORY-SIZE clause, but ignores it. Remove or comment out the MEMORY-SIZE clause.

F.5.5. RERUN Clause

Description

The RERUN clause has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

RERUN clause implementations were not necessarily transportable. This defines an operating system function that should not be a part of the language definition.

Expected Impact on ASCII COBOL Programs

A diagnostic flagging this as an obsolete feature is provided.

Changes

The RERUN clause is allowed in UCS COBOL.

F.5.6. MULTIPLE FILE TAPE Clause

Description

The MULTIPLE FILE TAPE clause in the I-O-CONTROL paragraph of the Environment Division has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

Expected Impact on ASCII COBOL Programs

No impact is anticipated. A diagnostic flagging this as an obsolete feature is provided.

Changes

The MULTIPLE FILE TAPE clause is allowed in UCS COBOL.

F.5.7. LABEL RECORDS Clause

Description

The LABEL RECORDS clause in the file description entry has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

Expected Impact on ASCII COBOL Programs

A diagnostic flagging this as an obsolete feature is provided.

Changes

The LABEL RECORDS clause is allowed in UCS COBOL.

F.5.8. VALUE OF Clause

Description

The VALUE OF clause in the file description entry has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

Expected Impact on ASCII COBOL Programs

A diagnostic flagging this as an obsolete feature is provided.

Changes

The VALUE OF clause is allowed in UCS COBOL.

F.5.9. DATA RECORDS Clause

Description

The DATA RECORDS clause in the file description entry has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

Expected Impact on ASCII COBOL Programs

A diagnostic flagging this as an obsolete feature is provided.

Changes

The DATA RECORDS clause is allowed in UCS COBOL.

F.5.10. ALTER Statement

Description

The ALTER statement has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

The use of the ALTER statement in a program results in a program that can be difficult to understand and maintain.

The ALTER statement provides no unique functionality since the GO TO DEPENDING statement serves the same purpose.

Expected Impact on ASCII COBOL Programs

Some problems could occur. A diagnostic flagging this as an obsolete feature is provided.

Changes

The ALTER statement is allowed in UCS COBOL.

F.5.11. The Optional Use of a Procedure Name in a GO TO Statement

Description

The optional use of *procedure-name-1* in the GO TO statement has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

This feature requires the use of the ALTER statement, which is an obsolete language feature. If *procedure-name-1* is not specified in format 1 of the GO TO statement, then an ALTER statement referring to that GO TO statement must be executed prior to the execution of the GO TO statement.

Expected Impact on ASCII COBOL Programs

A diagnostic flagging this as an obsolete feature is provided.

Changes

This feature is allowed in UCS COBOL.

F.5.12. REVERSED Phrase of the OPEN Statement

Description

The REVERSED phrase of the OPEN statement has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

Hardware is not widely available. Implementation was optional and the feature is infrequently used.

Expected Impact on ASCII COBOL Programs

A diagnostic flagging this as an obsolete feature is provided.

Changes

The REVERSED phrase of the OPEN statement is allowed in UCS COBOL.

F.5.13. STOP Literal Statement

Description

The literal variation of the STOP statement has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

This removes functions defined by the implementor and enhances program portability.

Expected Impact on ASCII COBOL Programs

A diagnostic flagging this as an obsolete feature is provided.

Changes

The STOP literal statement is allowed in UCS COBOL.

F.5.14. Segmentation Module

Description

The Segmentation module has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

This function is best provided by the Linking System component of the Universal Compiling System.

Expected Impact on ASCII COBOL Programs

Some problems could occur. A diagnostic flagging this as an obsolete feature is provided.

Changes

The Segmentation module is allowed in UCS COBOL.

F.5.15. Debug Module

Description

The Debug module has been placed in the obsolete category. This means that this feature will be removed from future revisions of the standard and should not be used when writing new programs.

This function is best provided by an interactive debug facility, such as the Programmers Advanced Debugging System (PADS), which does not require UCS COBOL source statements.

Expected Impact on ASCII COBOL Programs

UCS COBOL allows debug module syntax, but ignores it. A diagnostic flagging this as an obsolete feature is provided.

Changes

Use PADS to debug UCS COBOL source programs.

Appendix G
Compatibility, Restrictions, and Operational Considerations

The following subsections explain the software with which UCS COBOL level 12R1 is compatible. The restrictions and considerations that affect the operation of UCS COBOL are also explained.

G.1. Compatibility

We recommend that you always use the D$WORK-WS keyword option when compiling UCOB programs that use DMS. This forces the DMS D$WORK data structure to be allocated in WORKING-STORAGE, even if the program places the DMCA into COMMON using the INVOKE clause. If the option is **not** used and the INVOKE places the DMCA into COMMON, D$WORK will be placed at the end of COMMON-STORAGE after all the user variables. If the layout of COMMON-STORAGE section is not identical in all programs units in an executable then the user variables can overlay the D$WORK data structure, and can result in execution failure.

If your programs INVOKE the DMCA into COMMON and you want the D$WORK data structure to be allocated at the end of COMMON-STORAGE (the default), then do **not** use the D$WORK-WS keyword option. However, this is **not** recommended. Refer to Section 2.2.7, "Compiler Call Keyword Options for Program Control," for a description of this keyword option.

G.1.1. Language Support System (LSS) Level

You must install the Language Support System (LSS) level 14R1 to generate code for level 12R1 of the UCS COBOL compiler. Previous levels of LSS are not compatible with the current compiler. The following subsection describes other issues that can affect the compatibility of your existing UCS COBOL programs.

G.1.2. Compatibility with ASCII COBOL

UCS COBOL is compatible with the ANSI COBOL Standard X3.23-1985. In general, UCS COBOL supports standard features of X3.23-1974 as well as standard feature extensions documented in the current *ASCII COBOL Programming Reference Manual*.

The *ASCII COBOL Supplementary Reference Manual* documents the ASCII COBOL features that UCS COBOL does not implement. The differences in implementation of the two systems and the ASCII COBOL features that were not carried forward into UCS COBOL affect the compatibility between ASCII COBOL and UCS COBOL. UCS COBOL contains several compiler keyword options that reduce the incompatibilities due to implementation and standard differences.

G.1.3. Compatibility with Previous Versions of UCOB

GOBACK is now a new COBOL verb. Any use of the word GOBACK, either as a data name or a procedure name, will no longer be allowed in a UCS COBOL program. Any use of a data name or procedure name of GOBACK will need to be changed.

ADDRESS, NULL, and NULLS are now unreserved words. An incompatibility was introduced if a user had a data name declared as ADDRESS and used that data name as a sender or receiver in a SET statement. A similar incompatibility was also introduced for data names NULL or NULLS used as sending item in a SET statement or in a conditional statement (i.e. an IF statement). In those situations, NULL and NULLS are now treated as a two-word value of binary zeroes. Any use of the data names ADDRESS, NULL or NULLS within SET or relational statements will need to be changed.

G.2. Operational Considerations

Operational considerations are permanent and are provided to promote the effective use of a software product.

The following operational considerations apply to UCS COBOL:

- You must use the SDDL processor with the appropriate options to produce an object module S$WORK. If you use the SDDL to create a relocatable S$WORK, you cannot use the S$WORK in the same run unit with UCS object modules.
- The UCS COBOL compiler now performs all the functions of the ASCII COBOL Data Manipulation Language Preprocessor (ADMLP). Therefore, do not use the source output from the ADMLP processor as input to the UCS COBOL compiler.

  If you try to use source output from the ADMLP processor as input to UCS COBOL and execute the resulting object module, an incorrect entry point into DMS 2200 occurs and the execution fails.

- When you compile a DMS program with UCOB, it is recommended that you always compile it with the D$WORK-WS keyword option. This keyword option ensures that the DMS D$WORK data structure is always allocated in WORKING-STORAGE section. Refer to the discussion at the beginning of this appendix and the explanation for the keyword option in section 2.2.7, "Compiler Call Keyword Options for Program Control."
- Checkpoint Restart (CKRS) will not work for a program that contains calls to I18NLIB (either implicit or explicit) because I18NLIB is a subsystem that cannot be restarted with the current design of CKRS. To avoid the problem, remove any explicit calls to I18NLIB routines from the source program and use the keyword option NO-I18N-IO to eliminate any implicit calls to the I18NLIB.

UCS compiler implementations are different from basic mode compilers in several ways:

- UCS compiler output is in the object module format; it is not in relocatable binary (RB) format.
- Data allocation uses a byte-aligned algorithm.
- User calling sequences follow different rules for parameter passing and for code and data referencing.
- The library supporting UCS is different from the libraries used by the basic mode compilers.
- The product LSS generates the code for the UCS compilers and is separately installed. The version of LSS installed on your system must be compatible with the language front ends (for example, UC, UCOB, and UFTN).

Index

A

A2/A3
  bits and contents, 10-54
A4
  bits and contents, 10-54
A5
  bits and contents, 10-54
ABORT$
  executive request function, 10-2
ABSW$
  I/O executive request, 10-16
ACCESS MODE clause
  SEQUENTIAL access, with IBM tapes, 14-3, 14-4
accessing
  network database, 6-1
  relational database, 7-1
  Shared File System (SFS 2200), 5-10, 5-12
accessing, a relational database, 7-42
ACOB$RDMR, calling the RDMS interpreter interface, 7-33
ACQ$
  I/O executive requests, 10-16
ACSF$
  executive request function, 10-2
address
  program, limits on, C-5
address limits, program, C-5
address-name-2, 10-82
ALLOC
  keyword option, 2-10, 2-18, 2-53
Allocation Listing
  description of, 2-61
  example of NARROW option, 2-64
  fields of, 2-61
ALS stack, C-6
  overflow, C-7
American National Standard (ANS)
  1985 COBOL standard, UCS COBOL implementation, F-1
American Standard Code for Information Interchange (ASCII)
  COBOL, differences of UCS COBOL from, F-1
AMICUS environment
  using the MONITOR keyword option, 2-41
ANSI tape files, assignment device dependencies, 5-3
APIs, COBOL for CIFS functionality, 17-2
APPLICATION
  keyword option, 2-10, 2-23, 2-25
application group, *See* APPLICATION keyword option; Unisys Repository Manager
APPLICATION keyword option
  using to specify UREP repository, 8-2, 8-3
  using with RDMS programs, 7-3, 7-40
AREA/AREAS clause
  Information Interchange Tape Processing module, 13-2
arguments
  passing procedure name arguments to non-COBOL programs, 5-14
arrays, size limits of, C-5
ASCII COBOL, *See* UCS COBOL/ASCII COBOL differences
A-SHARED-FILE
  use of, 5-12
ASSIGN clause
  for label processing, E-1, E-5
  Information Interchange Tape Processing module, 13-3
  INTERCHANGE keyword in, 13-2
  with IBM tapes, 14-4
assigning
  ANSI tape files, 5-3
  UCS COBOL files
    alternate symbiont files, 5-4
    symbiont files, 5-3
AT END condition, with IBM tapes, 14-24
at-end condition, with IBM tapes, 14-2, 14-24, 14-26
AUXPROLOG
  keyword option, 2-10, 2-23, 2-26

B

bank descriptor index (BDI)
  I/O operations, 10-12
    with FIO, 10-12
    with FIOW, 10-13
BDI, *See* bank descriptor index
BDR$
  I/O executive request, 10-16
BDRB$
  I/O executive request, 10-16
BDW$
  I/O executive request, 10-16
BLOCK CONTAINS clause, with IBM tapes, 14-7
blocks, common, size limits of, C-5
BLOCKSIZE
  changing the default value of, 4-1
  SGS defaults for, 4-5
block-transfer loading and sticking options, 11-8
BRD$
  executive request function, 10-3
BSD$
  executive request function, 10-3
BSF$
  I/O executive request, 10-16
BSRD$
  executive request function, 10-3
buffers
  I/O processing, 10-20
bulk I/O, example, 10-76
BULKCLOSE$ subprogram, 10-68
BULKOPEN$ subprogram, 10-63
BULKREAD$ subprogram, 10-69
BULKWRITE$ subprogram, 10-72

C

CACHE
  keyword option, 2-10, 2-23
CACHE, keyword option, 2-27
called object modules, linked by Linking System, 1-6
calling programs in other languages, *See* interlanguage calls
CCONET
  primitive, 11-5
CDML, *See* COBOL Data Manipulation Language
CHARACTER
  RDM$ data type, 7-24
CHARACTERS
  RECORD CONTAINS clause, 14-9
CIFS
  copy procedures, 17-1
  example program, 17-2
CIFS COBOL program interface, 17-1
CLEAR
  keyword option, 2-10, 2-23, 2-28
CLOSE parameter, use of, 5-9
CLOSE statement
  label processing during, E-7
  with IBM tapes, 14-20
CNVTIM$
  executive request function, 10-2
COBOL data item definitions, generating with UREP, *See* INVOKE statement, generating
COBOL Data Manipulation Language (CDML)
  statements, 6-1
  using with UCS COBOL, 6-1
COBOL I/O executive request
  FWANY, 10-11, 10-13
COBOL I/O executive requests
  FIO, 10-11, 10-12
  FIOW, 10-11, 10-12
  FSYMB, 10-11, 10-13
  FUNLCK, 10-11, 10-13
  FWST, 10-11, 10-13
COBOL, ASCII, *See* UCS COBOL/ASCII COBOL differences
COBOL, CIFS program interface, 17-1
CODE
  keyword option, 2-10, 2-15
code generator, *See* UCS Language Support System
code location counters, 2-43
CODE-LVE
  keyword option, 2-10, 2-23, 2-28
common banks
  shared subsystems in UCS, 1-8
common blocks
  linkage of, 1-6
communications interface modules, UCS Runtime System management of, 1-7
comparisons
  UCS Runtime System management of, 1-7
COMPAT
  keyword option, 2-23, 2-29, 2-30
  keyword options, 2-11
COMPAT keyword option

affecting size and alignment of data objects, 15-3
upgrading old programs with, F-3
compatibility
UCS COBOL issues
calling sequence parameters, G-1
differences from ANSI-85 COBOL, G-1
UCS COBOL with ASCII COBOL, F-1
COMP-BIN
keyword option, 2-11, 2-23
compiler call letter options
D, 2-9
E, 2-9
I, 2-9
L, 2-9
N, 2-9
O, 2-9
S, 2-9
X, 2-9
compiler call options
combinations, common, 2-5
description of, 2-5
format and rules for, 2-1
priority of, 2-5
compiler listing, composition of, 2-53
compiling, *See also* Universal Compiling System
additional rules for, 2-4
calling the UCS COBOL compiler, 2-1
command interface in UCS, single, 1-4
detecting errors with run-condition word, 2-81
diagnostic messages, classes of (table), A-2
diagnostic messages, described (examples), A-1
DMS programs, 6-2
letter options for, 2-9
limits of UCS COBOL compiler (table), C-1
programs that access DMS, 6-2
output, 6-2
programs that access RDMS, 7-39
separate files as one with DFP, 5-10
UCS, overview of, 1-6
COMPOOL message-buffering primitives, 11-1, 11-3, 11-5
concatenating files, using DFP for, 5-10
COND$
executive request function, 10-2
console messages, interpreting, 5-17
contingencies
UCS Runtime System management of, 1-7
contingency handling, 12-1
CONTINGENCY, execution-time message source, B-1
control statement listing, 10-32
control statements
@ADD, 10-4
@ASG, 10-4
@BRKPT, 10-4
@CAT, 10-4
@CKPT, 10-4
@FREE, 10-5
@LOG, 10-5
@MODE, 10-5
@QUAL, 10-5
@RSTRT, 10-5
@START, 10-5
@SYM, 10-5
@USE, 10-5
conversion
ASCII COBOL programs to UCS COBOL, F-2
SGS defaults for, 4-6
table, changing the default value of:, 4-1
tools for UCS COBOL
COMPAT keyword option, F-3
conversions
UCS Runtime System management of, 1-7
coordinated universal time, 10-97
COPY procs for PLUS, C, COBOL
as a part of I18NLIB, 16-1
creating
fixed-gate subsystems, 3-5
UCS COBOL programs, 5-2
UCS COBOL programs (and assigning files), 5-2
Cross-Reference Listing
description of, 2-65
example of the NARROW option, 2-67
fields of, 2-65

D

D
compiler call letter option, 2-9
record format, 5-7
data
access in UCS, 1-4
external items, linkage of, 1-6
Data Definition Language (DDL) processor
interface with UCS COBOL, 6-1
sample program using, 6-4
Data Division

Information Interchange Tape Processing module, 13-4
with IBM tapes, 14-6
data management communications area (DMCA) element
produced by UCS COBOL, 6-1
Data Management Routine (DMR)
executing a DMS program, 6-2
updating and accessing a DMS database with, 6-2
Data Management System (DMS 2200)
accessing from UCS COBOL, 6-1
interface management by UCS Runtime System, 1-7
linking, 6-2
overview of, 6-3
sample program using, 6-4
schema processing, 6-1
subschema processing, 6-2
UCS interface, 11-1
using INVOKE command, 6-2
DATA RECORDS clause
with IBM tapes, 14-7
DATE$
executive request functions, 10-2
DB
record format, 5-8
DDL, *See* Data Definition Language processor
deadlock processing, with shared files, 5-12
DEBUG
keyword option, 2-11, 2-23, 2-31
DEBUG keyword option, 3-10
DEBUG/FULL keyword option, 3-10
DEBUG/REFERENCE keyword option, 3-10
DEBUG/WALKBACK keyword option, 3-10, 3-11
debugging
available functions for, 3-8
changing source code to invoke, 3-7, 3-13
compiler keyword options for, 3-10
invoking PADS, 3-12
overview of, 1-7
UCS Runtime System management of, 1-7
with PADS, 3-7
DECIMAL
RDM$ data type, 7-24
declaration section, for Embedded SQL variables, 7-5
Define File Block (DFB), creating with DFP, 5-4
Define File Processor (DFP), *See also* Shared File System
blocking factors
for MSAM files, 5-5
for non-MSAM files, 5-5
CLOSE Parameter, 5-9
concatenating data files, 5-10
creating a Define File Block (DFB), 5-4
EBCDIC Translate, 5-9
labelling records, 5-8
OPEN Parameter, 5-9
overriding I/O defaults with, 4-1
PREP formats of, 5-8
record formats of, 5-7
SHARE parameter, 5-10
use of, 5-4
DENSITY clause, system constraints for, 5-17
DFB, *See* Define File Block
diagnostic messages
compilation time, classes of (table), A-2
compilation time, described (examples), A-1
description of listing, 2-56
listing of
for UCS Runtime System, text for 19002 and 35001, B-57
rules for listing, 2-56
UCS Runtime System management of, 1-7
URTS run-time
text for 19002 and 35001, B-57
DISC files, label processing
INDEXED files, E-5
RELATIVE files, E-5
SEQUENTIAL files, E-5
Display Processing System (DPS 2200)
coding transactions with, 11-3
UCS interface, 11-1
DMCA, *See* data management communications area element
DMR, *See* Data Management Routine
DMS 2200, *See* Data Management System
documentation
library, organization of, 1-8
DOUBLE-PRECISION
RDM$ data type, 7-24
DPS 2200, *See* Display Processing System
DS
record format, 5-8
DWTIME$
executive request function, 10-2
dynamic linking, *See also* linking
description of, 3-1
use of, 3-3
dynamic memory allocation
UCOBCALLOC$, 10-81

UCOBFREE$, 10-82
UCOBMALLOC$, 10-81
UCOBREALLOC$, 10-82

E

E
compiler call letter option, 2-9
EABT$
executive request function, 10-2
EACQ$
I/O executive request, 10-16
EBCDIC, translating to ASCII, 5-9
element
delete (UCSPFD$), 10-25
insert (UCSPFI$), 10-25
search (UCSPFS$), 10-25
Embedded Structured Query Language, *See* Structured Query Language; Relational Data Management System
end-of-file labels, formats, E-5
end-of-volume labels, formats, E-5
ENTER MASM statement
using with interpreted SQL, 7-1, 7-8, 7-33
using with RDML commands, 7-33
entry point name, 2-38
environment
developing programs in UCS, 1-4
run-time, 1-4
Environment Division
Information Interchange Tape Processing module, 13-2
with IBM tapes, 14-2, 14-5
EOF1 labels
during CLOSE statement, E-7
format, E-5
EOF2 labels
during CLOSE statement, E-7
format, E-5
EOV1 labels
during CLOSE statement, E-8
format, E-5
EOV2 labels
during CLOSE statement, E-8
format, E-5
ER, *See* Executive request
ER IOW$, 10-62
ERR$
executive request function, 10-2
ERRCHECK
keyword option, 2-15
ERREXIT
keyword option, 2-11, 2-16
keyword options, 2-15
error messages
compilation time diagnostic, classes of (table), A-2
compilation time diagnostic, described (examples), A-1
errors
compiling, severity of (table), A-2
detecting with run-condition word, 2-81
printing listing of diagnostic messages, 2-56
run-time messages, 1-7
evaluation of expressions, limits on, C-6
Exec environment variables
defined as part of I18NLIB, 16-1
executing
assigning files, 5-2
changing runtime attributes with DFP, 5-4
overview of, 1-6
UCS COBOL programs, 3-6
URTS diagnostic messages
text for 19002 and 35001, B-57
Executive request
service library functions
UCSASCFD$, 10-56
UCSEMTOBM$, 10-58
UCSFDASC$, 10-56
subroutines, 10-4
ADATE, 10-8
ERACSF, 10-4
EREABT$, 10-10
ERERR$, 10-10
FABORT, 10-10
FACSF, 10-4
FACSF2, 10-4
FCOND, 10-6
FEABT, 10-10
FERR, 10-10
FEXIT, 10-10
FSETC, 10-6
IACSF, 10-4
IACSF2, 10-4
UCSASCFD$, 10-56
UCSCLIST$, 10-32
UCSCNVTIM$, 10-51
UCSDWTIME$, 10-50
UCSEMTOBM$, 10-58
UCSFDASC$, 10-56
UCSFITEM$, 10-33
UCSGENERAL$, 10-43

UCSINFO$, 10-34
UCSINITREG$, 10-53
UCSMAKEPFPK$, 10-24
UCSOPT$, 10-28
UCSPCT$, 10-28
UCSPFD$, 10-25
UCSPFI$, 10-25
UCSPFS$, 10-25
UCSPFUWL$, 10-26
UCSPFWL$, 10-26
UCSSYMBPK$, 10-13
UCSTDATE$, 10-8
UCSTIME$, 10-49
UCSTINTL$, 10-30
UCSTLBL$, 10-29
UCSTSWAP$, 10-31
executive request function
PFWL$, 10-3
executive request functions
ABORT$, 10-2
ACSF$, 10-2
BRD$, 10-3
BSD$, 10-3
BSRD$, 10-3
CLIST$, 10-2
CNVTIM$, 10-2
COND$, 10-2
DATE$, 10-2
DWTIME$, 10-2
EABT$, 10-2
ERR$, 10-2
EXIT$, 10-2
FITEM$, 10-2
INFO$, 10-2
INTREG$, 10-2
IO$, 10-2
IOW$, 10-2
OPT$, 10-2
PCT$, 10-2
PFD$, 10-2
PFI$, 10-2
PFS$, 10-2
PFUWL$, 10-3
PSA$, 10-3
RDL$, 10-3
RRSD, 10-3
SETC$, 10-3
SRD$, 10-3
SW$, 10-3
SYMB$, 10-3
TDATE$, 10-3
TIME$, 10-3
TINTL$, 10-3
TLBL$, 10-3
TSA$, 10-3
TSF$, 10-3
TSWAP$, 10-3
UNLCK$, 10-3
WAIT$, 10-3
WANY$, 10-3
Executive system
using run-condition word to detect compilation errors, 2-81
Executive system (Exec)
execution of object modules, 1-6
UCS programming environment, 1-4
EXIT PROGRAM statement, 2-39
EXIT$
executive request function, 10-2
expressions, evaluation of, limits on, C-6
EXTENDED
keyword option, 2-11, 2-23, 2-33
extended mode
UCS software for, F-1
extended mode architecture, 1-4
external
data items, linkage of, 1-6
routines, linkage of, 1-6
EXTERNAL clause, C-8
external names in object module, limits on number, C-8

F

FB
record format, 5-8
FBS
record formats, 5-8
FCS, *See* File Control System
FCSS, *See* File Control Super Structure
FD level indicator
with IBM tapes, 14-7
Federal Information Processing Standards (FIPS)
using compiler flagging to monitor compliance, A-4
file control entry
Information Interchange Tape Processing module, 13-2
with IBM tapes, 14-3
File Control Super Structure (FCSS)
transaction processing primitives, 11-5
UCS interface, 11-1
File Control System (FCS)

assigning files for, 5-2
file descriptions, generating with UREP, *See* INVOKE statement, generating
file position indicator
with IBM tapes
defined, 14-2
set by OPEN statement, 14-24
set for READ statement, 14-25, 14-26
File Section
Information Interchange Tape Processing module, 13-4
with IBM tapes, 14-6
FILE STATUS clause
Information Interchange Tape Processing module, 13-3
to detect rollback or deadlock, 5-12
FILE-CONTROL paragraph
Information Interchange Tape Processing module, 13-2
with IBM tapes, 14-2
files
assigning, 5-2
blocking factors for MSAM files, 5-5
blocking factors for non-MSAM files, 5-5
multithread access, 5-12
status
DFP CLOSE parameter, 5-9
DFP OPEN parameter, 5-9
DFP SHARE parameter, 5-10
FIO
COBOL I/O executive request, 10-11, 10-12
FIOW
COBOL I/O executive request, 10-11, 10-12
FITEM$
executive request function, 10-2
fixed-gate subsystems, creating, 3-5
FLOAT
RDM$ data type, 7-24
FROM phrase (WRITE statement), with IBM tapes, 14-27
FS
record format, 5-8
FSAFE$
I/O executive requests, 10-16
FSF$
I/O executive request, 10-17
FSYMB
COBOL I/O executive request, 10-11, 10-13
FUNLCK
COBOL I/O executive request, 10-11, 10-13
FURPUR, Program-Callable (PCFP)
accessing, 9-1
FWANY
COBOL I/O executive request, 10-11, 10-13
FWST
COBOL I/O executive request, 10-11, 10-13

G

generating source code from UREP with INVOKE statement, 8-3, *See also* INVOKE statement, generating; Unisys Repository Manager
Greenwich Mean Time (GMT), 10-97
GW$
I/O executive request, 10-17

H

HDR1 file header labels
during CLOSE statement, E-8
during OPEN statement, E-9
format (table), E-2
HDR2 file header labels
during OPEN statement, E-9
format (table), E-4
header
labels, formats, E-1
HEAPSIZE
changing the default value of, 4-1
SGS defaults for, 4-5
High Volume Transaction Processing (HVTIP)
specifying RDMS work area size, 7-38
High-Volume Transaction Processing (HVTIP)
using UCS COBOL with, 11-1
HVTIP, *See* High-Volume Transaction Processing

I

I
compiler call letter option, 2-9
I/O executive request
MR$, 10-17
I/O executive requests
ABSW$, 10-16
ACQ$, 10-16
BDR$, 10-16
BDRB$, 10-16
BDW$, 10-16
BSF$, 10-16
EACQ$, 10-16

FSAFE$, 10-16
FSF$, 10-17
GW$, 10-17
MB$, 10-17
MF$, 10-17
MS$, 10-17
R$, 10-17
RB$, 10-17
REL$, 10-17
REW$, 10-17
REWI$, 10-17
RR$, 10-17
SCR$, 10-17
SCRB$, 10-17
SM$, 10-17
UNL$, 10-17
W$, 10-17
WEF$, 10-17
I/O packets
use of UCSMAKEIOPK$, 10-15
I/O, subprograms for performing system-level, 10-62
I18N, *See* internationalization
I18N IO
keyword option, 2-11, 2-24
I18NLIB, *See* Internationalization Library
I-bank, limits on, C-5
IBM Tape Processing module, 14-1
INFO$
executive request function, 10-2
Information Interchange Tape Processing module, 13-1
INITIAL clause, C-8
initializing tape files, 10-30
input, *See* input/output; input files
input files
closing, E-7, E-8
opening, E-9
input/output
defaults for, 4-1
overriding defaults, 4-1
reasons for changing defaults, 4-2
using alternate tables, 4-6
input/output processing
operations, ER restrictions for, 10-20
Input-Output Section
Information Interchange Tape Processing module, 13-2
with IBM tapes, 14-2
inserting source code, *See* generating source code
installation options, MCBDEF-UCOB file, 11-4
INTEGER
RDM$ data type, 7-24
INTERCHANGE
file record description restrictions, 13-4
files, label processing, E-1
keyword in ASSIGN clause, 13-2
record formats of, 5-7
interface to zip utilities, 10-86
interfaces for UCS COBOL, 5-1
interlanguage calls
calling programs in other UCS languages, 15-1
calling UCS C
data types (table), 15-7
program example, 15-9
restrictions, 15-8
calling UCS FORTRAN
data types (table), 15-3
program example, 15-5
restrictions, 15-4
data types
COBOL vs C (table), 15-7
COBOL vs FORTRAN (table), 15-3
relations between, 15-2
passing modes and restrictions, 15-1
UCS Runtime System management of, 1-7
Internationalization Library (I18NLIB)
description of, 16-1
interpreter interface, *See* Structured Query Language; Relational Data Management System
interrupts, UCS Runtime System management of, 1-7
INTREG$
executive request function, 10-2
intrinsic functions
UCS Runtime System management of, 1-7
INVOKE statement
generating
data descriptions from UREP, overview, 8-7
file description from UREP, 8-5
relational data definitions from UREP, 8-8
screen definitions from UREP, 8-10
SELECT statement from UREP, 8-3
I-O, *See* input/output; input files; output files; I-O files; I-O packets; I-O status; I-O-CONTROL paragraph
I-O files
closing, E-7
opening, E-9
I-O packets, building symbiont, with subroutine UCSSYMBPK$, 10-13

IO$
executive request function, 10-2
I-O-CONTROL paragraph, with IBM tapes, 14-4
IOW$
executive request function, 10-2
IR DIFF
keyword option, 2-11
IR-DIFF
keyword option, 2-50

K

keyword
REMARK, 2-50
SPACE, 2-18, 2-19
keyword option element for the compiler call, 2-2
keyword options
ALLOC, 2-10, 2-18, 2-53
APPLICATION, 2-10, 2-23, 2-25
AUXPROLOG, 2-10, 2-23, 2-26
CACHE, 2-10, 2-23
CLEAR, 2-10, 2-23, 2-28
CODE, 2-10, 2-15
CODE-LVE, 2-10, 2-23, 2-28
COMPAT, 2-11, 2-23, 2-29, 2-30
COMP-BIN, 2-11, 2-23
DEBUG, 2-11, 2-23, 2-31
ERRCHECK, 2-15
ERREXIT, 2-11, 2-15, 2-16
EXTENDED, 2-11, 2-23, 2-33
for compilation control
CODE, 2-15
NO-CODE, 2-15
for source control
MULTI-PF, 2-13
NO-MULTI-PF, 2-13
SEARCH-PF-SI, 2-14
SEARCH-SI-PF, 2-14
I18N IO, 2-11, 2-24
IR DIFF, 2-11
IR-DIFF, 2-50
LEVEL, 2-11, 2-50, 2-51, 2-53
LEVEL/RPW, 2-11, 2-50
LEVEL/SEG, 2-50
LEVEL/SEG2, 2-50
LEVEL/SEG2L, 2-11
LEVEL/SEGL, 2-11
LEVEL/SQL, 2-11, 2-50, 2-52
LINKINFO, 2-11, 2-18, 2-19, 2-54
LISTCOPY, 2-11, 2-15, 2-16
LITERAL, 2-11, 2-18, 2-20
MAINPROGRAM, 2-11
MAIN-PROGRAM, 2-24
MAIN-PROGRAM, 2-34
MAXERRORS, 2-11, 2-15, 2-17
MAXMEM, 2-35
MAXSIZE, 2-11
MAX-SIZE, 2-24
MAX-SIZE, 2-35
MIN-DBANK, *2-37*, C-8
MONITOR, 2-11, 2-24, *2-38*
MULTI PF, 2-11, 2-13
NARROW, 2-11, 2-18, 2-20
OBJECT, 2-11, 2-18, 2-20, 2-54
OBJECT-ONLY, 2-54
OBJPKT, 2-11, 2-18, 2-21
OBJ-PKT, 2-54
OPTIM, 2-24, 2-42
OPTIONS, 2-11, 2-18, 2-21, 2-53
PARMCHECK, 2-11, 2-24, 2-43
program control
CACHE, 2-27
REL18, 2-43
TIPMALLOC, 2-46
QUOTE, 2-11, 2-18, 2-19
REF, 2-53
REMARK, 2-12, 2-53
REORDER, 2-12, 2-24, 2-44
RUNCHECK, 2-12, 2-24, 2-45
SEARCH, 2-12, 2-13
SEGCODE, 2-12, 2-24, 2-45
SHIFT, 2-12, 2-24, 2-46
SHORT, 2-53
SIGNON, 2-12, 2-18, 2-21, 2-53, 2-54
SOURCE, 2-12, 2-18, 2-22, 2-53
SPACE, 2-12
SUBPROGRAM, 2-11, 2-24
TABLES, 2-48
TIPMALLOC, 2-12, 2-24
TRACECOPY, 2-12, 2-18, 2-22, 2-53
TRACEMSG, 2-12, 2-18
TRANSFORM, 2-12, 2-24, 2-47
TRIM-CRLF, 2-17
UREP XREF, 2-12
UREP-XREF, 2-25, 2-47
WARNING, 2-12, 2-50, 2-53
WIDE, 2-11, 2-18, 2-20
XREF, 2-12, 2-18, 2-23, 2-53

L

L
  compiler call letter option, 2-9
label formats for files, E-1
label handling, use of, 5-8
label processing, for disk and tape files, E-1
LABEL RECORDS clause
  using for label processing, E-1
  with IBM tapes, 14-8
language differences in UCS COBOL, *See* UCS COBOL/ASCII COBOL differences
Language Support System, *See* UCS Language Support System
letter options
  compiler call (table), 2-9
  in @UCOB statement, 2-1
LEVEL
  keyword option, 2-11, 2-50, 2-51, 2-53
LEVEL/RPW
  keyword option, 2-11, 2-50
LEVEL/SEG
  keyword option, 2-50
LEVEL/SEG2
  keyword option, 2-50
LEVEL/SEG2L
  keyword option, 2-11
LEVEL/SEGL
  keyword option, 2-11
LEVEL/SQL
  keyword option, 2-11, 2-50, 2-52
LEVEL/SQL keyword option
  using with RDMS programs, 7-6
library, preparing for UCS COBOL, 5-1
limits
  arrays, size, C-5
  common blocks, size, C-5
  expression evaluation, C-6
  external names in an object module, number of, C-8
  nested procedure calls, C-6
  program address, C-5
  UCS COBOL compiler internal (table), C-1
LINAGE clause, use with DENSITY clause, 5-17
LINK$PF.SSDEF$, creating, 3-3
linker, *See* Linking System
Linker Information Listing
  description of, 2-6, 2-71
  fields of, 2-71
LINKINFO
  keyword option, 2-11, 2-18, 2-19, 2-54
linking, *See also* Linking System
  description of, 3-1
  dynamic, timing of, 1-6
  programs that access RDMS, 7-40
  static, timing of, 1-6
  types
    use of dynamic, 3-3
    use of static, 3-4
  UCS methods, overview of, 1-6
  using with DMS programs, 6-2
  using with RDMS programs, 7-42
Linking System, *See also* linking
  description of, 3-1
  function of, 1-6
  types of linking
    dynamic, 3-1
    static, 3-1
  UCS programming environment, 1-4
  using with DMS programs, 6-2
LISTCOPY
  keyword option, 2-11, 2-15, 2-16
listing of a program
  components of
    Allocation Listing, 2-61
    Cross-Reference Listing, 2-65
    diagnostic messages, 2-56
    Linker Information Listing, 2-6, 2-71
    Object Code Listing, 2-68
    Options Listing, 2-56
    sign-off line, 2-76
    sign-on line, 2-54
    Source Listing, 2-57
    table, 2-53
    Tracecopy Listing, 2-60
  description of, 2-53
LITERAL
  keyword option, 2-11, 2-18, 2-20
local definition files
  as a part of I18NLIB, 16-1
LSS, *See* UCS Language Support System

M

Magnetic Tape Labels standard, 13-1, 13-4
main storage
  addressing of programs, limits on, C-5
  limited by shared subsystems, 1-8
  UCS Runtime System management of, 1-7
MAIN-PROGRAM
  keyword option, 2-11, 2-24, 2-34

Malloc function, 7-40
MAXERRORS
  keyword option, 2-11, 2-15, 2-17
MAXMEM
  keyword option, 2-35
MAXSIZE
  keyword option, 2-11
MAX-SIZE
  keyword option, 2-24
MAX-SIZE
  keyword option, 2-35
MB$
  I/O executive request, 10-17
MCB, *See* Message Control Bank
merging
  changing the size of SORT-MERGE main storage, 5-16
Message Control Bank (MCB)
  coding transactions with, 11-4
  message buffering for DPS 2200 transactions, 11-3
  UCS interface, 11-1
messages
  compilation time diagnostic, classes of (table), A-2
  compilation time diagnostic, described (examples), A-1
  for Embedded SQL, 7-6
  interpreting console messages, 5-17
MF$
  I/O executive request, 10-17
MIN-ALS option, C-8
MIN-DBANK
  keyword option, *2-37*, C-8
MONITOR
  keyword option, 2-11, 2-24, *2-38*
    usage in the Open Programming Environment, 2-41
MR$
  I/O executive request, 10-17
MS$
  I/O executive request, 10-17
MSAM files, blocking factors of, 5-5
MULTI PF
  keyword option, 2-11, 2-13
MULTI-PF keyword option, 2-13
multithread access, 5-12

N

N
  compiler call letter option, 2-9
NARROW
  keyword option, 2-11, 2-18, 2-20
NCHARACTER
  RDM$ data type, 7-24
nesting procedure calls, limits, C-6
network database, *See* Data Management System
NO-CODE keyword option, 2-15
NO-DEBUG keyword option, 3-11
NO-MULTI-PF keyword option, 2-13
non-MSAM files, blocking factors of, 5-5
NO-OPTIM keyword option
  using with PADS, 3-11
NUMERIC
  RDM$ data type, 7-24

O

O
  compiler call letter option, 2-9
OBJECT
  keyword option, 2-11, 2-18, 2-20, 2-54
Object Code Listing
  description of, 2-68
  examples of, 2-70
  fields of, 2-68
OBJECT keyword option
  producing Object Code Listing with, 2-68
object modules
  in UCS environment, 1-6
OBJECT-ONLY
  keyword option, 2-54
OBJECT-ONLY keyword option
  producing Object Code Listing with, 2-68
object-program, for the UCS COBOL compiler call, 2-2
OBJPKT
  keyword option, 2-11, 2-18, 2-21
OBJ-PKT
  keyword option, 2-54
OBJ-PKT keyword option
  using with Object Code Listing, 2-71
OPEN parameter, use of, 5-9
OPEN statement
  label processing during, E-9
  with IBM tapes, 14-23
OPT$
  executive request function, 10-2
OPTIM
  keyword option, 2-11, 2-24, 2-42

OPTIONAL clause, use of, 5-11
OPTIONAL phrase
  in CLOSE statement, with IBM tapes, 14-23
  in READ statement, with IBM tapes, 14-26
options, *See* letter options; keyword options; compiler call options
OPTIONS
  keyword option, 2-11, 2-18, 2-21, 2-53
Options Listing, use in a program listing, 2-56
output, *See* input/output; output files
output files
  closing, E-7, E-8, E-9
overhead, reducing for RDMS programs, 7-38
overpunch sign conversions (table), D-11

P

packet routines
  UCSMAKEIOPK$ procedure, 10-15
  using ERs, 10-12, 10-13
packets, symbiont I-O
  building with subroutine UCSSYMBPK$, 10-13
PADS, *See* Programmers Advanced Debugging System
PARMCHECK
  keyword option, 2-11, 2-24, 2-43
passing modes for interlanguage calls
  pass by reference, 15-1
  pass by reference value, 15-1
  pass by value, 15-1
PBR, pass by reference, 15-1
PBRV, pass by reference value, 15-1
PBV, pass by value, 15-1
PCFP, *See* Program-Callable FURPUR
PCIOS, *See* Processor Common Input/Output System
PCT$
  executive request function, 10-2
performance, CACHE keyword option for, 2-27
PFD$
  executive request function, 10-2
PFI$
  executive request function, 10-2
PFS$
  executive request function, 10-2
PFUWL$
  executive request function, 10-3
porting files, *See* Information Interchange Tape Processing module
PREPFACTOR
  changing the default of, 4-1
  SGS defaults for, 4-4
primitives, coding transactions with, 11-5
printer output, setting lines with DENSITY clause, 5-17
procedure calls, nesting limits, C-6
Procedure Definition Processor (PDP)
  preparing UCS COBOL libraries with, 5-1
Procedure Division
  I-O statements for label processing, E-7
  with IBM tapes, 14-20
procedures
  passing names to non-COBOL programs, 5-14
  putting into UCS COBOL libraries, 5-1
processor call
  for DDL processor, 6-1
  for PADS, 3-12
  for SDDL, 6-2
Processor Common Input/Output System (PCIOS)
  accessing, 5-12
  interface management by UCS Runtime System, 1-7
  UCS interface, 11-1
Processor Common Input-Output System (PCIOS)
  UCS interface, 11-1
program
  address limits, C-5
  debugging with PADS, 3-7
  development, within UCS, 1-4
  dynamically linking, 3-3
  executing, 3-6
  performance using CACHE, 2-27
  statically linking, 3-4
program control, addresses, limits on, C-5
program environment
  run-time, 1-4
Program-Callable FURPUR (PCFP)
  accessing, 9-1
Programmers Advanced Debugging System (PADS)
  compiler keyword options for, 3-10
  debugging programs with, 3-7
  description of, 1-7
  functions, 3-8
    conditional command execution, 3-9
    data change trapping, 3-9

data display, 3-8
dump, 3-9
procedures, 3-9
stepping, 3-8
tracing, 3-9
trapping, 3-8
variables, 3-9
interface management by UCS Runtime System, 1-7
invoking, 3-12
UCS programming environment, 1-4
use of NO-OPTIM keyword option, 3-11
PSA$
executive request function, 10-3

Q

QUOTE
keyword option, 2-11, 2-18, 2-19

R

R
compiler call letter option, 2-9
record format, 5-7
R$
I/O executive request, 10-17
R$CORE feature, using to dynamically change SORT-MERGE main storage, 5-16
R1
bits and contents, 10-54
R11/R12
bits and contents, 10-54
R13/R14
bits and contents, 10-54
R15
bits and contents, 10-54
R2
bits and contents, 10-54
R3
bits and contents, 10-54
RB$
I/O executive request, 10-17
RDA, *See* record delivery area (RDA) element
RDL$
executive request function, 10-3
RDM$ data types
CHARACTER, 7-24
DECIMAL, 7-24
DOUBLE-PRECISION, 7-24
FLOAT, 7-24
INTEGER, 7-24
NCHARACTER, 7-24
NUMERIC, 7-24
REAL, 7-24
SMALLINT, 7-24
RDMCA variables, use as Embedded SQL error variables, 7-6
RDMS 2200, *See* Relational Data Management System
READ statement
label processing during, E-10
with IBM tapes, 14-25
REAL
RDM$ data type, 7-24
record blocking formats for Information Interchange Tape Processing module, 13-4
RECORD CONTAINS clause
Sequential I<#106>O module, 14-8
record delivery area (RDA) element
produced by UCS COBOL, 6-1
record description
Information Interchange Tape Processing module, 13-4
with IBM tapes, 14-13
record formats
D, 5-7
DB, 5-8
DS, 5-8
FB, 5-8
FBS, 5-8
FS, 5-8
R, 5-7
U, 5-7
VBS, 5-8
RECORDING MODE clause
Information Interchange Tape Processing module, 13-4
with IBM tapes, 14-11
REEL phrase, in CLOSE statement, E-7
reentrant transaction processing program, 11-8
REF
keyword option, 2-53
referenced object modules, linked by Linking System, 1-6
Registers
A2/A3, 10-54
A4, 10-54
A5, 10-54

R1, 10-54
R11/R12, 10-54
R13/R14, 10-54
R15, 10-54
R2, 10-54
REL$
I/O executive request, 10-17
REL18 keyword option, 2-43
relational data definitions, generating with UREP, *See* INVOKE statement, generating
Relational Data Management System (RDMS 2200), *See also* Structured Query Language
accessing, 7-1
compiling, 7-39
description of, 7-1
dynamically linking, 7-42
Embedded SQL
declaring variables, 7-5
dynamically linking, 7-42
example of, 7-29
LEVEL/SQL keyword option, 7-6
marking commands, 7-28
specifying work area size, 7-38
use of, 7-1
interface management by UCS Runtime System, 1-7
interpreter interface, 7-40
calling, 7-8, 7-33
determining required work area size, 7-38
error variables for, 7-34
example of, 7-34
specifying work area size, 7-38
use of, 7-1, 7-33
linking, 7-40
passing values
from RDMS programs, 7-27
to RDMS programs, 7-25
UCS interface, 11-1
work area size for, 7-38
REMARK
keyword option, 2-12, 2-50, 2-53
REORDER
keyword option, 2-12, 2-24, 2-44
report writer transaction processing restrictions, 11-8
RERUN clause, with IBM tapes, 14-4
RESERVE clause
Information Interchange Tape Processing module, 13-3
with IBM tapes, 14-4
restrictions, I/O processing, 10-20
retrieving coordinated universal time, 10-97
return-status, 10-81, 10-82
reversed (tape) files
closing, E-8
opening, E-9
REW$
I/O executive requests, 10-17
REWI$
I/O executive request, 10-17
rollback processing, with shared files, 5-12
routines, external, linkage of, 1-6
RR$
I/O executive request, 10-17
RRSD
executive request function, 10-3
RSA$INIT, specifying the RDMS work area with, 7-38
RUNCHECK
keyword option, 2-12, 2-24, 2-45
run-condition word, detecting compilation errors with, 2-81
Runtime System, *See* UCS Runtime System

S

S
compiler call letter option, 2-9
S$PROC object module, using in DMS subschema, 6-2
S$WORK object module, using in DMS subschema, 6-2
SAME clause
with IBM tapes, 14-4
schema
processing with DDL, 6-1
UCS COBOL interface, 6-1
SCR$
I/O executive request, 10-17
SCRB$
I/O executive request, 10-17
screen definitions, generating with UREP, *See* INVOKE statement, generating
SDDL, *See* Subschema Data Definition Languag
SDF, *See* System Data Format
SEARCH
keyword option, 2-12, 2-13
SEARCH-PF-SI keyword option, 2-14
SEARCH-SI-PF keyword option, 2-14

security in UCS, 1-4
SEGCODE
  keyword option, 2-12, 2-24, 2-45
SELECT clause
  Information Interchange Tape Processing module, 13-2
  with IBM tapes, 14-4
SELECT statement
  generating from definition in UREP, 8-3
  relationship to assigning files, 5-3, 5-4
self-initializing transaction processing program, 11-8
Service Library in UCS programming environment, 1-4
service routines
  as a part of I18NLIB, 16-1
SETC$
  executive request function, 10-3
SFS 2200, *See* Shared File System
SHARE parameter, using, 5-10
shared banks, 1-8
shared data, 1-4
Shared File System (SFS 2200)
  accessing, 5-12
  rollback and deadlock processing, 5-12
  UCS interface, 11-1
  using DFP to specify shared files, 5-10
SHIFT
  keyword option, 2-12, 2-24, 2-46
SHORT
  keyword option, 2-53
sign-off line
  description of, 2-76
SIGNON
  keyword option, 2-12, 2-18, 2-21, 2-53, 2-54
sign-on line, use in program listing, 2-54
SILAS, *See* SILAS
SILAS, message buffering for transaction processing, 11-3
SM$
  I/O executive request, 10-17
SMALLINT
  RDM$ data type, 7-24
Sort/Merge
  interface management by UCS Runtime System, 1-7
Sort-Merge
  main storage size, changing, 5-16
SOURCE
  keyword option, 2-12, 2-18, 2-22, 2-53
source control using compiler call keyword options, 2-13, 2-14
Source Listing
  description of, 2-57
  example of WIDE option, 2-58
  fields of, 2-57
source-program for the UCS COBOL compiler call, 2-2
SPACE
  keyword option, 2-12
SQL, *See* Structured Query Language, *See* Structured Query Language
SQLCODE
  use as embedded SQL error variable, 7-6
SQLSTATE
  use as embedded SQL error variable, 7-6
SRD$
  executive request function, 10-3
SSG, *See* Symbolic Stream Generator
standard calling sequence (SCS)
  in UCS, 1-4
standards for programming languages, 1-4
static linking
  description of, 3-1
  use of, 3-4
STOP RUN, 2-39
storage
  addressing of programs, limits on, C-5
  virtual
    limits on, C-5
Structured Query Language (SQL), *See also* Relational Data Management System
  accessing, 7-1
  compiling, 7-39
  description of, 7-1
  dynamically linking, 7-42
  Embedded SQL
    APPLICATION keyword option, 7-3
    commands, use of, 7-28
    declaring variables, 7-5
    dynamically linking, 7-42
    example of, 7-29
    LEVEL/SQL keyword option, 7-6
    marking commands, 7-28
    specifying work area size, 7-38
    use of, 7-1
  interpreter interface
    calling, 7-8, 7-33
    determining required work area size, 7-38
    error variables for, 7-34
    example of, 7-34
    specifying work area size, 7-38
    use of, 7-33

passing values
from RDMS programs, 7-27
to RDMS programs, 7-25
UCS interface, 11-1
using with RDMS, 7-1
SUBPROGRAM
keyword option, 2-11, 2-24
subprograms, for performing system level I/O, 10-62
subschema
processing with SDDL, 6-2
UCS COBOL interface, 6-2
using S$PROC object module, 6-2
using S$WORK object module, 6-2
Subschema Data Definition Language (SDDL)
interface with UCS COBOL, 6-2
Subsystem Definition Processor (SSDP)
use in linking, 3-3
subsystems
creating, 1-8
shared, description of, 1-8
UCS programming environment, 1-4
SW$
executive request function, 10-3
SYMB$
executive request function, 10-3
symbiont files
assigning, 5-3
assigning alternate files, 5-4
symbiont I-O packets, building with subroutine UCSSYMBPK$, 10-13
Symbolic Stream Generator (SSG)
using to build an alternate URTS$TABLES, 4-2
using UPDATETABLE to alter URTS$TABLES, 4-3
SYNCHRONIZED (SYNC) clause
with IBM tapes, 14-15
syntax
conversion tools for UCS COBOL
COMPAT keyword option, F-3
syntax differences in UCS COBOL, *See* UCS COBOL/ASCII COBOL differences
system
interface modules for, UCS Runtime System management of, 1-7
System Data Format (SDF)
record formats of, 5-7
system-level I/O subprograms
BULKCLOSE$, 10-68
BULKOPEN$, 10-63
BULKREAD$, 10-69
BULKWRITE$, 10-72

T

tape processing, *See* Information Interchange Tape Processing module; tapes
tapes
initializing files on, 10-30
label processing, E-1
reading and writing label blocks, 10-29
swapping, 10-31
using IBM, with IBM Tape Processing module, 14-1
TDATE$
executive request function, 10-3
The entry point name, 2-38
TIME$
executive request function, 10-3
time, coordinated universal, 10-97
TIMEB2DWTIM$, 10-99
TINTL$
executive request function, 10-3
TIP, *See* Transaction Processing (TIP)
TIPMALLOC
keyword option, 2-12
TIPMALLOC, keyword option, 2-46
TLBL$
executive request summary, 10-3
TRACECOPY
keyword option, 2-12, 2-18, 2-22, 2-53
Tracecopy Listing
description of, 2-60
TRACEMSG
keyword option, 2-12, 2-18
transaction processing
description of, 11-1
UCS interfaces, 11-1
using DPS 2200, 11-3
using MCB, 11-4
using primitives, 11-5
transaction processing (TIP)
interface management by UCS Runtime System, 1-7
Transaction Processing (TIP)
specifying RDMS work area size, 7-38
using UCS COBOL with, 11-1
transaction processing environment
restrictions for source code, 11-8
TRANSFORM
keyword option, 2-12, 2-24, 2-47
TRIM-CRLF
keyword option, 2-17
TSA$

executive request function, 10-3
TSF$
executive request function, 10-3
TSWAP$
executive request function, 10-3

U

U
record format, 5-7
UCOBCALLOC$ subprogram, 10-81
UCOBFREE$ subprogram, 10-82
UCOBMALLOC$ subprogram, 10-81
UCOBREALLOC$ subprogram, 10-82
UCS, *See* Universal Compiling System
UCS C
calling from UCS COBOL, 15-7
new features and enhancements, 1-10
UCS COBOL
accessing
DMS database, 6-1
other product interfaces, 5-1
relational database, 7-1
assigning files
alternate symbiont, 5-4
symbiont, 5-3
calling the compiler, 2-1
compatibility with other software, G-1
compliance with FIPS, monitoring through compiler, A-4
debugging programs of, 3-7
executing programs, 3-6
implementation of ANS 1985 COBOL standard, F-1
limits of compiler (table), C-1
linking programs of, 3-1
modules, levels, and subsets (table), A-5
operational considerations
compiler implementation differences, G-3
File Information Packet, G-3
for use with SDDL, G-2
replacing ADMLP functions, G-2
passing procedure name arguments to non-COBOL programs, 5-14
preparing a library for, 5-1
transaction processing, description of, 11-1
UCS COBOL/ASCII COBOL differences
changes from ANS
ALL literal, length, F-20
ALPHABET-NAME clause, F-21
CANCEL statement, F-25
class condition, F-25
CLOSE statement, F-25
collating sequence, F-21
conditional expression, order of execution, F-24
COPY statement in comments, F-26
COPY statement on debugging line, F-26
CURRENCY SIGN clause, F-21
DISPLAY statement, F-26
DIVIDE statement, F-26
EXIT PROGRAM statement terminates all PERFORM statements, F-27
EXIT PROGRAM, implicit, F-27
exponentiation, F-24
file position indicator, effect of REWRITE, F-31
file position indicator, effect on READ and WRITE, F-31
file position indicator, replaces record pointer, F-31
FOOTING phrase, F-23
INSPECT statement, F-27
I-O statuses, F-34
LINAGE clause, F-22
MERGE statement, F-27
MULTIPLE FILE TAPE clause, F-22
OCCURS clause, F-23
order of execution, conditional expression, F-24
PERFORM statement, evaluating subscripts, F-28
PERFORM statement, initializing variables, F-28
PERFORM statement, setting and augmenting variables, F-28
PICTURE symbol 'P', F-23
PROCEDURE DIVISION header, F-24
READ statement, F-29
RELATIVE KEY phrase, F-22
reserved words added, F-32
RETURN statement, F-29
STOP RUN statement closes files, F-29
STOP RUN statement, message queue management, F-30
STRING statement, F-30
UNSTRING statement, F-30
WRITE statement, F-30
discontinued extensions
770-PRINTER deleted, F-16
ALPHABET SECTION deleted, F-16

APPLY INLINE deleted, F-16
Asynchronous Processing module deleted, F-16
CARD-READER with ACCEPT and DISPLAY, F-18
CODE clause mnemonics not supported, F-19
CODE clause, COBOL 68 format, not supported, F-19
compatible files, F-15
CONSOLE with ACCEPT and DISPLAY, F-18
COPY statements, COBOL 68 format, not supported, F-19
ENTER statements deleted, F-19
EQUALS operator not supported, F-20
EXAMINE deleted, F-18
EXCEEDS operator not supported, F-20
EXHIBIT deleted, F-18
FIELDATA not supported, F-20
INSPECT option TRAILING deleted, F-18
IOS files, F-15
MONITOR deleted, F-17
NOTE statement deleted, F-18
ON statement deleted, F-19
POINT LOCATION clause deleted, F-16
PRINTER with ACCEPT and DISPLAY, F-18
REMARKS deleted, F-17
special registers MCFLAG, TALLY, F-17
TRAILING option of INSPECT deleted, F-18
TRANSFORM deleted, F-18
UNEQUAL operator not supported, F-20
implementation differences
ASCII COBOL level 3R1 calling sequence, F-7
CALL identifier, F-8
calling sequence, F-7
calling sequence, ASCII COBOL level 3R1, F-7
class names, F-10
communication module, F-13
COMP-1 alignment, F-5
COMP-2 alignment, F-5
computational, F-4
COMPUTATIONAL fields data representation, F-4
COMPUTATIONAL-3 deleted, F-4
data allocation, F-13
duplicate data names from schema, F-12
FILE STATUS with APPLY EXDEF, F-12
FILE STATUS with APPLY EXREF, F-12
floating-point computations, F-6
frequency table generation deleted, F-8
HIGH-VALUES, F-6
IBM data allocation for programs, F-5
IBM records, data allocation, F-5
INDEX alignment, F-5
INDEX data items, F-9
index-name formats, F-9
intermediate operands, F-5
numeric PROGRAM-ID name, F-11
overpunched signed numeric fields, F-10
paragraph followed by GO TO, F-11
record areas, referencing before OPEN, F-9
register conventions, F-7
report format for nonprint files, F-12
SEARCH ALL, F-9
segmentation map symbolic, F-8
sequence number checking deleted, F-8
subscript checking, F-11
SYNCHRONIZED field alignment, F-4
undefined results, F-6
obsolete language elements
ALL literal, with numeric or numeric edited items, F-34
ALTER statement, F-37
character substitution, F-34
DATA RECORDS clause, F-37
Debug module, F-39
Identification Division paragraphs, F-35
LABEL RECORDS clause, F-36
MEMORY SIZE clause, F-35
MULTIPLE FILE TAPE clause, F-36
OPEN statement, REVERSED phrase, F-38
procedure name, optional use in GO TO, F-38
RERUN clause, F-36
REVERSED phrase, OPEN statement, F-38
Segmentation module, F-39
STOP literal statement, F-38
VALUE OF clause, F-37
UCS FORTRAN
calling from UCS COBOL, 15-3
code generator, 1-4
component of UCS, 1-4
documentation library, organization of, 1-8
UCSMAKEIOPK$ procedure, 10-15

UCS Language Support System (LSS)
component of UCS, 1-4
results of, 1-6
UCS Runtime System
component of UCS, 1-4
contingency handling, 12-16
execution environment provided, 1-6
run-time messages
text for 19002 and 35001, B-57
subroutine calls for Executive requests, 10-1
UCSMAKEIOPK$, packet routine
buffers for, 10-19, 10-20
construct packets with, 10-15
description of, 10-15
error flag, 10-20
FIO usage, 10-12
FIOW usage, 10-13
words used for, 10-19
UCSSYMBPK$, packet routine
FSYMB usage, 10-13
Unisys Repository Manager (UREP)
functions with UCS COBOL, overview, 8-1
generating
source code with INVOKE, 8-3
specifying an application name, 8-2
storing application attributes in, 8-3
UCS COBOL features to access, 8-1
updating with the UREP-XREF keyword option, 8-30
using with UCS COBOL, 8-1
Universal Compiling System (UCS)
changing source code to invoke, 3-13
changing source code to invoke PADS, 3-7
compiling a program, overview of, 1-6
components of, 1-4
debugging a program in, 1-7
description of, 1-4
developing programs in, 1-4
executing programs, 1-6
linking object modules, 1-6
program development with, 1-4
transaction processing, 11-1
UCS Runtime System, environment of, 1-6
Universal Data System (UDS)
using with DMS, 6-1
using with RDMS, 7-1
using with SFS, 5-12
using with UREP, overview, 8-1
UNL$
I/O executive request, 10-17
UNLCK$
executive request function, 10-3
UPDATETABLE
using to change I/O defaults, 4-3
updating
DMS database, 6-3
UREP with the UREP-XREF keyword option, 8-30
upgrading ASCII COBOL programs to UCS COBOL, F-2
UREP entities and relationships
as a part of I18NLIB, 16-1
UREP XREF
keyword option, 2-12
UREP-XREF
keyword option, 2-25, 2-47
UREP-XREF keyword option
updating UREP cross-reference information with, 8-30
URTS$TABLES
changing the default values of, 4-1
keyword option, 2-48
regenerating, 4-3
using alternates, 4-6
URTS$TABLES element, 4-1
URTS, execution-time message source, B-1
USAGE BINARY
as affected by COMP BIN, 2-23
USAGE clause
with IBM tapes, 14-17
USAGE COMP
as affected by COMP BIN, 2-23
USE procedure, for shared files, 5-12
UTC$TIME, 10-98

V

VALTAB keyword, 11-3
VALUE clause
with IBM tapes, 14-18
VALUE OF clause
FILE-ACCESS, E-6
FILE-ID, E-6
PURGE-DATE, E-6
using to control label processing, E-1, E-2, E-6
with IBM tapes, 14-12
variables
declaring error variables for Embedded SQL, 7-6
declaring for RDMS programs, 7-5
PADS, 3-9

VBS
  record format, 5-8
virtual
  storage, limits on, C-5
VOL1 volume header labels
  during OPEN statement, E-9
  format (table), E-2
volume labels, formats, E-1

W

W$
  I/O executive request, 10-17
WAIT$
  executive request function, 10-3
WANY$
  executive request features, 10-3
WARNING
  keyword option, 2-12, 2-50, 2-53
WEF$
  I/O executive request, 10-17
WIDE
  keyword option, 2-11, 2-18, 2-20
work area size, *See also* Relational Data Management System; Structured Query Language
  determining required size, RDMS, 7-38
write limits for updating and retrieving, 10-26
WRITE statement
  label processing during, E-10
  with IBM tapes, 14-26

X

X
  compiler call letter option, 2-9
XREF
  keyword option, 2-12, 2-18, 2-23, 2-53

Z

Zip utilities
  ZIP$CLOSEENTRY, 10-92
  ZIP$NAMEENTRIES, 10-87
  ZIP$NBRENTRIES, 10-86
  ZIP$PRINTEXCEPTION, 10-92
  ZIP$READENTRIES, 10-89
  ZIP$SETCOMPRESSION, 10-93
  ZIP$WRITEENTRY, 10-90
ZIP$CLOSEENTRY, 10-92
ZIP$NAMEENTRIES, 10-87
ZIP$NBRENTRIES, 10-86
ZIP$PRINTEXCEPTION, 10-92
ZIP$READENTRIES, 10-89
ZIP$SETCOMPRESSION, 10-93
ZIP$WRITEENTRY, 10-90

Special Characters

@

@ADD
  control statement, 10-4
@ASG
  control statement, 10-4
@ASG statement
  affecting label processing, E-3, E-5, E-6, E-8
@BRKPT
  control statement, 10-4
@CAT
  control statement, 10-4
@CKPT
  control statement, 10-4
@DFP statement
  use of, 5-4
@FREE
  control statement, 10-5
@LOG
  control statement, 10-5
@MODE
  control statement, 10-5
@PADS ON statement, 3-12
@PDP statement, preparing UCS COBOL libraries with, 5-1
@QUAL
  control statement, 10-5
@RSTRT
  control statement, 10-5
@SSDP statement, 3-3
@START
  control statement, 10-5
@SYM
  control statement, 10-5
@SYM statement, using with alternate symbiont files, 5-4

@UCOB statement
  calling the UCS COBOL compiler, 2-1
  examples of, 2-3
  format of, 2-1
  parameters for, 2-1
@UFTN, statement
  calling the UCS FORTRAN compiler, 1-6
@USE
  control statements, 10-5
@XQT statement
  executing programs using, 3-6
  retrieving @XQT statement options using UCSOPT$, 10-28

© 2018 Unisys Corporation.
All rights reserved.

7831 0455–021